React Handbook

Components · state · effects · hooks · forms & actions · routing & data · performance · TypeScript · testing · Server Components · interview prep

16 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — Why React, and Setting Up**
   - 1.1 What React is
   - 1.2 Declarative UI
   - 1.3 Rendering, the virtual DOM and reconciliation (brief)
   - 1.4 SPA vs MPA vs SSR (and SSG)
   - 1.5 Setting up a React project
   - 1.6 Project structure
   - 1.7 Your first component
   - 1.8 JSX
   - Chapter 1 TL;DR
2. **Chapter 2 — Components and Props**
   - 2.1 Function components
   - 2.2 Props
   - 2.3 The `children` prop
   - 2.4 Composition
   - 2.5 Pure components
   - 2.6 Keeping components small
   - 2.7 Conditional rendering
   - 2.8 Rendering lists and keys
   - 2.9 Putting it together — a product catalog
   - Chapter 2 TL;DR
3. **Chapter 3 — State and Events**
   - 3.1 Responding to events
   - 3.2 State: a component's memory — `useState`
   - 3.3 State as a snapshot
   - 3.4 Batching
   - 3.5 Updater functions — queueing updates
   - 3.6 Updating objects and arrays immutably
   - 3.7 Lifting state up
   - 3.8 Derived state — choosing the minimal state
   - 3.9 Controlled inputs (basics)
   - 3.10 State is tied to a position in the tree
   - 3.11 Putting it together — a to-do app
   - Chapter 3 TL;DR
4. **Chapter 4 — Forms and Actions**
   - 4.1 How a form submission works
   - 4.2 Controlled vs uncontrolled inputs
   - 4.3 Validation
   - 4.4 Form Actions — `<form action={fn}>`
   - 4.5 `useActionState` — the result of the last submission
   - 4.6 `useFormStatus` — status of the parent form
   - 4.7 `useOptimistic` — instant feedback, automatic rollback
   - 4.8 Server Functions and progressive enhancement (overview)
   - 4.9 Form libraries — React Hook Form and Zod (overview)
   - 4.10 Putting it together — an RSVP form
   - Chapter 4 TL;DR
5. **Chapter 5 — Effects and Refs**
   - 5.1 Where side effects belong
   - 5.2 `useEffect` basics
   - 5.3 Cleanup — undo what setup did
   - 5.4 StrictMode's double run (development only)
   - 5.5 Dependencies in depth
   - 5.6 You might not need an Effect
   - 5.7 Fetching data in Effects and race conditions
   - 5.8 `useRef` for values
   - 5.9 Refs to DOM nodes
   - 5.10 Passing refs to your own components
   - 5.11 `useLayoutEffect` — before the browser paints
   - 5.12 Putting it together — a city search
   - Chapter 5 TL;DR
6. **Chapter 6 — State Management**
   - 6.1 Kinds of state — and where each belongs
   - 6.2 `useReducer` — consolidate update logic
   - 6.3 Context — pass data deeply without props
   - 6.4 Context performance caveats
   - 6.5 Reducer + context — a store with only React
   - 6.6 When to reach for a library
   - 6.7 Redux Toolkit
   - 6.8 Zustand
   - 6.9 TanStack Query — server state
   - 6.10 Choosing — comparison and decision flow
   - 6.11 Putting it together — a bookshop
   - Chapter 6 TL;DR
7. **Chapter 7 — Custom Hooks and the Rules of Hooks**
   - 7.1 What a Hook really is
   - 7.2 The Rules of Hooks
   - 7.3 Custom Hooks — reusing stateful logic
   - 7.4 `useLocalStorage` — state that survives a reload
   - 7.5 `useDebounce` — wait until the user stops
   - 7.6 `useFetch` — loading, error, success, and no races
   - 7.7 Hooks that take callbacks — `useEffectEvent`
   - 7.8 `useId` — IDs for accessibility
   - 7.9 `useSyncExternalStore` — subscribing to outside data
   - 7.10 `useTransition` — non-urgent updates
   - 7.11 `useDeferredValue` — let part of the UI lag behind
   - 7.12 Putting it together — a user search
   - Chapter 7 TL;DR
8. **Chapter 8 — Styling React Apps**
   - 8.1 The styling landscape
   - 8.2 Plain CSS files
   - 8.3 Inline styles — the `style` prop
   - 8.4 CSS Modules
   - 8.5 Composing class names from props
   - 8.6 Tailwind CSS
   - 8.7 CSS-in-JS
   - 8.8 Theming and dark mode
   - 8.9 Component libraries
   - 8.10 Choosing an approach

- Chapter 8 TL;DR

9. **Chapter 9 — Routing and Data Fetching**
   - 9.1 Client-side routing
   - 9.2 React Router v8 and its three modes
   - 9.3 Routes, links and URL parameters (declarative mode)
   - 9.4 Nested routes and layouts
   - 9.5 Loaders — data before render (data mode)
   - 9.6 Actions, `<Form>` and fetchers
   - 9.7 Middleware, lazy routes and framework mode
   - 9.8 TanStack Query basics
   - 9.9 Query keys, pagination and dependent queries
   - 9.10 Mutations, invalidation and optimistic updates
   - 9.11 Suspense and `use()`
   - 9.12 Loading and error states that users like
   - 9.13 Putting it together — loaders + TanStack Query
   - Chapter 9 TL;DR
10. **Chapter 10 — Performance**
   - 10.1 The render cycle: trigger → render → commit
   - 10.2 When components re-render
   - 10.3 Measure first: DevTools Profiler and `<Profiler>`
   - 10.4 `memo` — skipping a child's re-render
   - 10.5 `useCallback` and `useMemo` for stable props
   - 10.6 `useMemo` for expensive calculations
   - 10.7 React Compiler
   - 10.8 Code splitting with `lazy` and `<Suspense>`
   - 10.9 List virtualization
   - 10.10 Common performance bugs
   - 10.11 A performance checklist
   - Chapter 10 TL;DR
11. **Chapter 11 — TypeScript with React**
   - 11.1 Setup and how type-checking fits in
   - 11.2 Typing props
   - 11.3 Typing children and extending native elements
   - 11.4 Typing events
   - 11.5 Typing state and reducers
   - 11.6 Typing refs
   - 11.7 Typing custom Hooks and Context
   - 11.8 Generic components
   - 11.9 Discriminated unions for props
   - 11.10 Reading type errors, and common mistakes
   - Chapter 11 TL;DR
12. **Chapter 12 — Testing React**
   - 12.1 What to test, and the Testing Library philosophy
   - 12.2 Your first component test
   - 12.3 Queries and their priority
   - 12.4 `user-event` — interacting like a user
   - 12.5 Async tests
   - 12.6 Mocking `fetch` (and when to use MSW)
   - 12.7 Testing custom Hooks
   - 12.8 Providers, routers and custom render
   - 12.9 Setting up Vitest or Jest in a real project
   - 12.10 Good tests, bad tests, and the bigger picture
   - Chapter 12 TL;DR
13. **Chapter 13 — Server Rendering and Frameworks**
   - 13.1 Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR
   - 13.2 The server rendering APIs
   - 13.3 Streaming SSR with `<Suspense>`
   - 13.4 Hydration end to end
   - 13.5 Hydration mismatches
   - 13.6 Static generation and partial prerendering
   - 13.7 React Server Components
   - 13.8 Directives: `'use client'` and `'use server'`
   - 13.9 Next.js App Router overview
   - 13.10 React Router framework mode
   - 13.11 Choosing a framework
   - Chapter 13 TL;DR
14. **Chapter 14 — Patterns and Architecture**
   - 14.1 Composition first
   - 14.2 Container / presentational
   - 14.3 Extracting custom Hooks
   - 14.4 Compound components
   - 14.5 Render props
   - 14.6 Higher-order components (HOCs)
   - 14.7 Choosing a reuse pattern
   - 14.8 Portals
   - 14.9 Error boundaries
   - 14.10 Accessibility in React
   - 14.11 Folder structure
   - 14.12 Design systems
   - 14.13 Client architecture patterns: MVC, MVP, MVVM, VIPER — and where React fits
   - Chapter 14 TL;DR
15. **Chapter 15 — React Interview and Machine Coding**
   - 15.1 How React interviews are run
   - 15.2 Interview questions: rendering, reconciliation and keys
   - 15.3 Interview questions: state, events and forms
   - 15.4 Interview questions: effects and refs
   - 15.5 Interview questions: hooks, context and state management
   - 15.6 Interview questions: performance and the React Compiler
   - 15.7 Interview questions: Suspense, `use()` and transitions
   - 15.8 Interview questions: server rendering, hydration and Server Components
   - 15.9 Interview questions: error boundaries, portals, patterns, testing and accessibility
   - 15.10 Build 1 — Todo list with filters
   - 15.11 Build 2 — Autocomplete (debounce, abort, keyboard)
   - 15.12 Build 3 — Infinite scroll
   - 15.13 Build 4 — Modal dialog (portal, focus trap, Escape)
   - 15.14 Build 5 — Accessible tabs
   - 15.15 Build 6 — Star rating
   - 15.16 Build 7 — Pagination
   - 15.17 Build 8 — Multi-step form wizard with validation
   - 15.18 Other common prompts — which techniques they need
   - Chapter 15 TL;DR
16. **Chapter 16 — Cheat Sheets and Glossary**

- 16.1 JSX rules
- 16.2 Components and props
- 16.3 Every Hook in one table
- 16.4 State update rules
- 16.5 The render and commit lifecycle
- 16.6 Effects: decision guide
- 16.7 Refs
- 16.8 Forms and Actions (React 19)
- 16.9 Context and choosing state management
- 16.10 Routing (React Router v8) and TanStack Query
- 16.11 Performance: checklist, memo and the React Compiler
- 16.12 Styling
- 16.13 TypeScript + React types
- 16.14 Testing: query priority and user-event
- 16.15 Rendering strategies and directives
- 16.16 Patterns
- 16.17 Accessibility checklist
- 16.18 Common errors and fixes
- 16.19 Interview question index
- 16.20 Where to go next
- 16.21 Glossary (A–Z)
- Chapter 16 TL;DR

Chapter 1 — Why React, and Setting Up

This chapter answers three questions before you write any serious React: *what problem does React solve*, *how does it update the screen*, and *how do I start a project*. By the end you will have read and run a first app, and you will know the handful of JSX rules that every later chapter assumes.

What you'll learn

#	Topic	You will be able to…
1.1	What React is	explain React in one minute, and say what it is *not*
1.2	Declarative UI	contrast "describe the result" with "list the DOM steps"
1.3	Rendering, the virtual DOM and reconciliation	describe render → commit and why React touches only changed nodes
1.4	SPA vs MPA vs SSR	pick a rendering strategy and name its costs
1.5	Setting up a project	create an app with a framework or with Vite, and know which to choose
1.6	Project structure	find your way around `index.html`, `main.jsx`, `App.jsx`, `package.json`
1.7	Your first component	write, export, import and nest components
1.8	JSX rules	one root, closed tags, `className`, `{expressions}`, fragments, `style` objects

All code lives in `code/`. It was run with **React 19.3** (current as of September 2026) on Node 24: `.jsx` files are compiled by esbuild and rendered into **jsdom** (a browser-like DOM that runs inside Node), and `@testing-library/react` is used to click buttons and read what a user would see. Run everything with `bash tools_run_js.sh react` from the repository root.

1.1 What React is

What it is

React is a JavaScript **library** for building **user interfaces** (UIs) out of **components**. A *component* is a JavaScript function that returns a description of a piece of screen — a button, a profile card, a whole page. You combine small components into bigger ones, the way you build a wall out of bricks.

Analogy: think of a restaurant. You (the developer) write the *menu* — for each situation, what the plate should look like. React is the kitchen: it works out how to get from the plate on the table now to the plate described on the menu, and does only the necessary work.

Key vocabulary used for the rest of the book:

Term	Meaning
DOM (Document Object Model)	The browser's live tree of objects for the page (`<div>`, `<button>` …). Changing it changes what you see.
Component	A function whose name starts with a capital letter and that returns JSX.
JSX	An HTML-like syntax inside JavaScript (`<h1>Hi</h1>`) that compiles to plain function calls (§1.8).
Props	Inputs passed to a component, like function arguments (Chapter 2).
State	Data a component remembers between renders, such as "how many likes" (Chapter 3).
Render	React calling your component to find out what the UI should look like.
Hook	A function starting with `use` (e.g. `useState`) that lets a component use React features.

Library, not framework

React itself handles only one job: turning components + data into UI and keeping it up to date. Routing (which page for which URL), data fetching, and build tooling come from other packages or from a **framework** built on React

(Next.js, React Router, Expo — §1.5).

Package	What it does
`react`	Components, JSX runtime, Hooks — platform independent
`react-dom`	Renders React to a browser DOM (`react-dom/client`) or to HTML strings/streams on a server (`react-dom/server`)
`react-native`	Renders the same kind of components to native iOS/Android views (separate project)

A short history

Year	Event
2011–2012	Jordan Walke builds React at Facebook; it runs the Facebook News Feed and then Instagram.
May 2013	Open-sourced at JSConf US. JSX was controversial ("HTML in my JavaScript?").
2015	React Native brings the model to mobile apps.
Feb 2019	React 16.8 introduces **Hooks**; function components become the standard way to write React.
Mar 2022	React 18: concurrent rendering and automatic batching of state updates.
Dec 5, 2024	**React 19**: Actions, `useActionState`, `useOptimistic`, `use`, `ref` as a normal prop, Server Components stable.
Feb 14, 2025	Create React App (CRA) deprecated for new apps.
Oct 2025	React 19.2; React Compiler 1.0 (automatic memoization); Meta announces React will move to an independent React Foundation hosted by the Linux Foundation.
Sep 9, 2026	React 19.3 (`<ViewTransition>`, refs on `<Fragment>`) — the version used in this book.

Why it matters

- **Reusable pieces.** A `<Button>` written once is used in 200 places; fixing a bug in that one file repairs every place it is used.
- **Predictable updates.** You describe the UI for the current data; React keeps the screen in sync (§1.2).
- **Huge ecosystem.** Routers, form libraries, component kits, testing tools, and jobs. React is used by Meta (Facebook, Instagram, WhatsApp Web, Threads), Netflix, Airbnb, Discord, Shopify, and many others.
- **One mental model, many targets.** Browser (React DOM), server HTML (SSR), mobile (React Native), even terminals and PDFs via community renderers.

When not to use it

Situation	Better fit
A mostly static site (blog, docs, landing page) with little interactivity	Plain HTML/CSS, or a static-site generator; add a sprinkle of JS
A tiny widget on a server-rendered page	Vanilla JS or a very small library
Hard limits on JavaScript size (very slow devices/networks)	Server-rendered HTML with minimal JS

React's runtime (react + react-dom) adds roughly 60–70 KB of gzipped JavaScript (the Figure 1.1 bundle, React included, is 69 KB gzipped) — small for an app, large for a single dropdown.

Common mistakes

- Thinking React *is* the whole app. You still need routing, data fetching, and a build tool — which is why react.dev recommends starting with a framework (§1.5).
- Learning old tutorials: class components, `componentDidMount`, and CRA are legacy. This book uses function components and Hooks only.

TL;DR

React = a UI library: components (functions returning JSX) + React DOM to put them in the browser. Everything else (routing, data, build) comes from the ecosystem or a framework.

1.2 Declarative UI

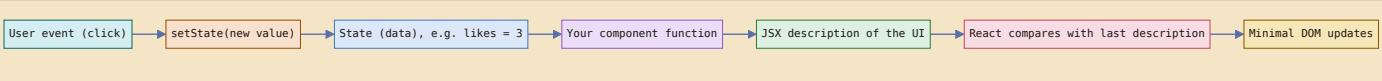
What it is

There are two ways to tell a computer how the screen should change.

- **Imperative**: you give step-by-step commands. "Find the label. Set its text to 3 likes. Find the button. If likes ≥ 3 change its text to Popular!"
- **Declarative**: you describe the *result* for any given data. "The label says *N like(s)*; the button says *Popular!* when N ≥ 3." A system (React) figures out the steps.

Analogy: a taxi. Imperative = "left, then right, then straight 200 m". Declarative = "take me to the station". You say *where*, the driver works out *how*.

React's core idea is often written as **UI = f(state)**: the screen is a function of the current data. When the data changes, React calls your function again and updates the page to match.



The problem it solves

In an imperative UI every event handler must remember *every* piece of screen that depends on the changed data. With three labels, a badge and a disabled button all depending on `likes`, forgetting one line produces an inconsistent screen. Bugs grow with the number of (state × places-that-show-it) combinations. In the declarative version there is only one place — the JSX — that says what each piece looks like.

How it works — the same button both ways

`code/01_imperative_vs_declarative.jsx`

```
// ---- Imperative: YOU write every step that changes the page ----
let likes = 0;
const btn = document.createElement('button');
const label = document.createElement('span');
label.textContent = 'No likes yet';
btn.textContent = 'Like';
btn.addEventListener('click', () => {
  likes++;                                          // 1. change the data
  label.textContent = `${likes} like(s)`;           // 2. remember to update the label
  btn.textContent = likes >= 3 ? 'Popular!' : 'Like'; // 3. ...and the button
});

// ---- Declarative: you describe WHAT the UI looks like for a given state ----
function LikeButton() {
  const [likes, setLikes] = useState(0);
  return (
    <div>
      <button onClick={() => setLikes(likes + 1)}>{likes >= 3 ? 'Popular!' : 'Like'}</button>{' '}
      <span>{likes === 0 ? 'No likes yet' : `${likes} like(s)`}</span>
    </div>
  );
}
// ... both are clicked three times, then printed
```

Output:

```
imperative: <button>Popular!</button> <span>3 like(s)</span>
declarative: <div><button>Popular!</button> <span>3 like(s)</span></div>
```

Both give the same HTML. The difference is *where the knowledge lives*: in the imperative version the click handler knows about the label and the button; in React the click handler only changes data (`setLikes`), and the JSX alone decides what is shown. `useState` is explained fully in Chapter 3.

Trade-offs

	Imperative DOM code	Declarative React
Small widget	Shortest code, no dependency	Overhead of a library and build step
Growing UI	Handlers get tangled; easy to forget an update	Each piece described once; scales well
Performance	You can hand-optimise every step	React does diffing work; fast enough for almost all UIs, with tools for hot spots
Debugging	Trace which handler changed what	Inspect state and props in React DevTools

In the interview

30-second answer: "React is declarative: I describe what the UI should look like for the current state, and React works out the DOM operations. When state changes, React re-runs my component, compares the new output with the previous one, and applies only the differences. That removes a whole class of bugs where some part of the screen isn't updated."

- *Is React fully declarative?* — The UI description is. You still write imperative code in event handlers (e.g. "on click, set state") and escape hatches such as refs and Effects (Chapter 5).
- *What does UI = f(state) mean?* — Given the same state and props, a component should return the same JSX; the screen is derived from data rather than edited by hand.

TL;DR

Imperative = list the steps; declarative = describe the result. React components describe the result for the current state and React applies the steps.

1.3 Rendering, the virtual DOM and reconciliation (brief)

What it is

- A **React element** is a small, plain JavaScript object that describes one piece of UI: `{ type: 'h1', props: { children: 'Hi' } }`. JSX creates these objects (§1.8).
- The tree of elements your components return is often called the **virtual DOM**: a lightweight *description* of the page that lives in JavaScript memory, not the real DOM. (react.dev rarely uses the term; it talks about the *render tree* instead, but you will hear "virtual DOM" in interviews.)
- **Reconciliation** is React's process of comparing the new element tree with the previous one to work out the smallest set of real DOM changes.

Analogy: an architect's plan. When the client asks for a bigger window, the builder compares the new plan to the old one and replaces only the window — not the whole house.

How it works — three steps on every update

1. **Trigger** — something asks for a render: the initial `root.render(<App />)`, or a state update (`setLikes`).
2. **Render** — React *calls your components* to get a new element tree and compares (diffs) it with the previous tree to work out what changed. This is pure calculation; nothing on screen changes yet.
3. **Commit** — React applies only those differences to the real DOM. The browser then paints.



The diff uses two simple rules instead of an expensive general tree comparison (a general algorithm is roughly $O(n^3)$; React's heuristics make it about $O(n)$):

Rule	Consequence
Different type at the same position (`<div>` → `<section>`, or `<Profile>` → `<Settings>`)	React throws away the old subtree (DOM *and* component state) and builds a new one.
Same type at the same position	React keeps the DOM node and component state, and updates only changed attributes/text.
Lists	Items are matched by their `key` prop, so React can tell "moved" from "changed" (Chapter 2 §2.8).

Since React 16 the engine that performs this work is called **Fiber**; it can split rendering into small units, pause and resume (the basis of React 18+ *concurrent rendering*). You do not need its details to write React.

Code — watching what React actually changes

`code/04_reconciliation.jsx` renders a clock, records every DOM mutation with a `MutationObserver` (a browser API that reports DOM changes), then renders again with a new `time` prop.

```
function Clock({ time }) {
  return (
    <div className="clock">
      <h1>Hello, world!</h1>
      <p>It is {time}.</p>
    </div>
  );
}
await act(() => root.render(<Clock time="10:00:00" />));
// ... start observing, then:
await act(() => root.render(<Clock time="10:00:01" />));
```

Output:

```
DOM changes made by React:
  characterData on <p>: "10:00:00" -> "10:00:01"
same <h1> node kept? true
same <p> node kept?  true
HTML now: <div class="clock"><h1>Hello, world!</h1><p>It is 10:00:01.</p></div>
```

Although the whole `Clock` function ran again, exactly **one** text node changed in the real DOM. (`act()` from `react` is a test helper that makes React finish all pending work before the next line runs; you only use it in tests and scripts like these.)

Common mistakes

- "React re-renders = React rewrites the DOM." No: *re-render* means your function is called again; the DOM changes only where output differs.
- "The virtual DOM makes React faster than hand-written DOM code." Not necessarily — perfectly hand-tuned DOM code does less work. The benefit is getting *good enough* performance automatically while writing declarative code.
- Defining a component *inside* another component: each render creates a new function = a new *type*, so rule 1 applies and state is reset every time (§1.7).

In the interview

30-second answer: "When state changes, React re-renders: it calls the components to build a new tree of elements, diffs it against the previous tree, and commits only the differences to the DOM. The diff is O(n) thanks to two heuristics — elements of different types produce different trees, and keys identify list items across renders."

- *What is the difference between render and commit?* — Render = calling components to compute the new tree (pure, may be repeated or discarded); commit = mutating the DOM and running layout effects/refs.
- *Why can changing a wrapper from `<div>` to `<section>` lose input text?* — Different type at the same position → the subtree is destroyed and recreated, including its state.

Self-check

1. After `setState`, which of your functions run? → The component that owns the state and, by default, all its children (they re-render); only changed DOM nodes are updated.
2. What does a React element look like? → A plain object with `type` and `props` (plus internals such as `key`).
3. What happens to state when a component's type at a position changes? → It is destroyed; the new component starts fresh.

TL;DR

Trigger → render (call components, build element tree) → commit (apply minimal DOM diff). Same type at same position = keep and update; different type = replace; lists matched by key.

1.4 SPA vs MPA vs SSR (and SSG)

What it is

These are strategies for **where and when HTML is produced**.

Term	Full name	How the page is produced
MPA	Multi-Page Application	Every navigation asks the server for a new HTML page (classic PHP/Rails/Django sites).
SPA	Single-Page Application	The server sends one almost-empty HTML file plus a JavaScript bundle; JavaScript builds every screen in the browser and handles navigation without full reloads. Also called **CSR** — client-side rendering.
SSR	Server-Side Rendering	The server runs your React components per request and sends finished HTML; the browser then **hydrates** it (React attaches event handlers to existing HTML).
SSG	Static Site Generation	Like SSR, but done once at **build time**; the HTML files are served from a CDN (Content Delivery Network — servers around the world that cache files close to users).

How it works — see the difference in HTML

`code/05_csr_vs_ssr.jsx` renders one component both ways.

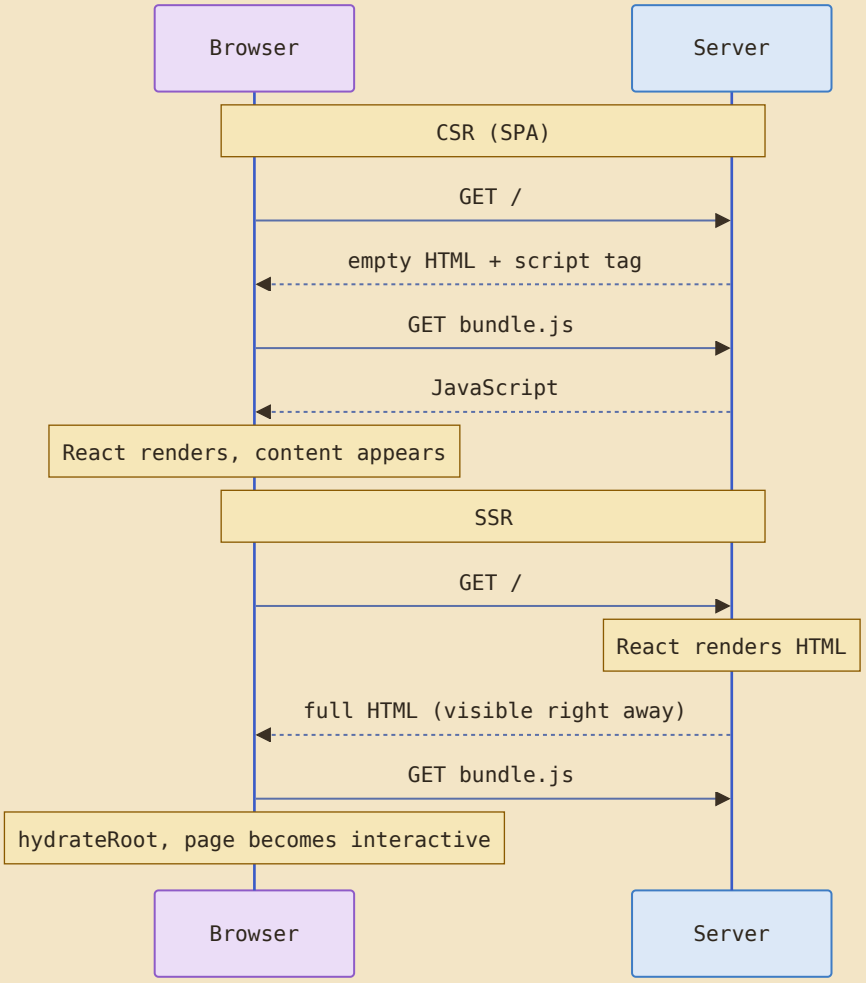
```
function App() {
  const [n, setN] = useState(0);
  return <main><h1>Shop</h1><button onClick={() => setN(n + 1)}>Cart: {n}</button></main>;
}
const html = renderToString(<App />);                  // SSR: component -> HTML string
document.getElementById('root').innerHTML = html;      // what the browser receives
button.click();                                        // before hydration
await act(() => { hydrateRoot(document.getElementById('root'), <App />); });
await act(() => button.click());                       // after hydration
```

Output:

```
CSR first HTML : <div id="root"></div>  (+ <script src="bundle.js">)
SSR first HTML : <div id="root"><main><h1>Shop</h1><button>Cart: <!-- -->0</button></main></div>
click before hydration -> Cart: 0 (nothing happens yet)
click after hydration  -> Cart: 1 | same DOM node reused: true
```

What the output teaches:

1. With **CSR**, the first HTML has no content — users and search-engine crawlers see nothing until JavaScript downloads and runs.
2. With **SSR**, the content is in the first HTML (the `<!-- -->` comment separates two adjacent text pieces so hydration can match them).
3. Server HTML is *not interactive* until **hydration** finishes; a click before that does nothing.
4. Hydration **reuses** the server's DOM nodes rather than recreating them.



Trade-offs

	MPA	SPA / CSR	SSR	SSG
First content shown	Fast	Slow (wait for JS)	Fast	Fastest (from CDN)
Navigation between pages	Full reload	Instant, app-like	App-like after hydration	App-like after hydration
SEO (search-engine optimisation) / link previews	Good	Weaker (crawlers must run JS)	Good	Good
Server needed	Yes	No — static hosting	Yes (Node or edge runtime)	No — static hosting
Fresh per-request data	Yes	Via API calls	Yes	Only at build (or revalidation)
Typical examples	Wikipedia, classic admin tools	Figma-style tools, dashboards behind login	E-commerce, news	Docs, blogs, marketing pages

Modern React frameworks mix these **per route**: a marketing page can be static, a product page server-rendered, a dashboard client-rendered. React 19's **Server Components** go one step further (components that run only on the server and send no JS) — covered in Chapter 13.

When not to use SSR

- An internal tool behind a login where SEO does not matter and users keep the tab open all day — a plain SPA is simpler and cheaper to host.
- When you cannot run a server (pure static hosting) — choose CSR or SSG.

Common mistakes

- Using browser-only APIs (`window`, `localStorage`) during render in an SSR app: the server has no `window`, so it crashes or the HTML differs from the client (a **hydration mismatch**).
- Assuming SSR makes the page interactive sooner. It makes content *visible* sooner; interactivity still waits for JS + hydration.

In the interview

> **30-second answer:** "An SPA ships an empty shell and renders everything in the browser — simple hosting, app-like navigation, but slow first paint and weaker SEO. SSR renders HTML on the server per request, so content appears immediately, then hydration attaches React to it. SSG does the same at build time for pages that rarely change. Frameworks like Next.js let you choose per route."

- *What is hydration?* — React running in the browser over server-generated HTML, reusing the existing DOM and attaching event handlers and state.
- *What causes a hydration mismatch?* — Rendering different output on server and client, e.g. `Date.now()`, `Math.random()`, or `window` checks during render.

TL;DR

MPA = server page per navigation; SPA/CSR = JS builds everything in the browser; SSR = server HTML + hydration; SSG = HTML at build time. Pick per route.

1.5 Setting up a React project

What you need first

Tool	Why	Check
Node.js (current LTS; Vite needs 20.19+ or 22.12+)	Runs the dev server and build tools on your computer — your React code still runs in the browser	`node -v`
npm (comes with Node), or pnpm / yarn / bun	Installs packages listed in `package.json`	`npm -v`
An editor — VS Code is the common choice	Syntax highlighting, JSX support, extensions (ESLint, Prettier)	—
React Developer Tools browser extension (Chrome, Firefox, Edge)	Inspect the component tree, props, and state; profile renders	Components tab in DevTools

Option A — start with a framework (react.dev's recommendation for new apps)

A **framework** bundles React with routing, data loading, server rendering and a build pipeline, so you do not assemble these yourself. The React docs currently recommend:

Framework	Command	Good for
Next.js (App Router)	`npx create-next-app@latest`	Full-stack web apps, SSR/SSG, Server Components; maintained by Vercel
React Router v8 (framework mode)	`npx create-react-router@latest`	Apps that grow from an SPA to full stack; built on Vite, maintained by Shopify
Expo	`npx create-expo-app@latest`	iOS, Android and web from one codebase (React Native)

react.dev also lists newer options such as TanStack Start and RedwoodSDK. `npx` downloads and runs a package's command once without installing it globally; `@latest` picks the newest version.

Option B — build from scratch with Vite

Use this when you want a plain client-side app (learning, dashboards, embedding in an existing site) and are happy to add routing and data fetching yourself. React docs list **Vite**, **Parcel** and **Rsbuild** as build tools; Vite is the most common.

```
# (not run here — needs network) create, install, start
npm create vite@latest my-react-app -- --template react   # or react-ts for TypeScript
cd my-react-app
npm install          # download react, react-dom, vite, plugins into node_modules/
npm run dev          # start the dev server -> http://localhost:5173
```

Command	What happens
`npm create vite@latest …`	Runs the `create-vite` scaffolder, which copies a template folder. The extra `--` passes `--template react` through npm to the scaffolder. Templates include `react`, `react-ts`, and `react-compiler` / `react-compiler-ts` (pre-configured React Compiler).
`npm install`	Reads `package.json`, installs dependencies into `node_modules/`, writes `package-lock.json` (exact versions).
`npm run dev`	Starts Vite's **dev server**: serves files as native ES modules, compiles JSX on the fly, and uses **HMR** (Hot Module Replacement — edits appear in the browser in milliseconds without losing component state).
`npm run build`	Produces an optimised production bundle in `dist/` (minified, tree-shaken, hashed file names for caching).
`npm run preview`	Serves `dist/` locally to test the production build.

Equivalent scaffolding commands: `pnpm create vite`, `yarn create vite`, `bun create vite`.

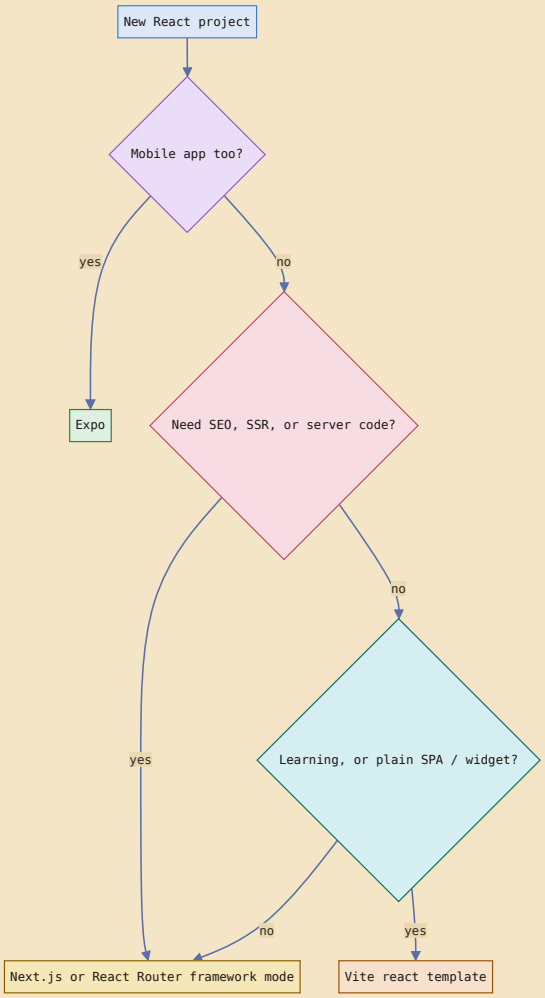
What about Create React App?

Create React App (CRA) was the standard starter from 2016. The React team deprecated it for new apps on **14 February 2025** (it still works in maintenance mode) because it had no active maintainers, was slow (webpack-based) and gave no path to routing, data fetching or SSR. Old tutorials using `npx create-react-app` are outdated — use a framework or Vite.

Adding React to an existing page

You do not need a new project to use React. On an existing server-rendered site you can mount React into one element: add a build step that bundles your component, put `<div id="cart-widget"></div>` in the HTML, and call `createRoot(document.getElementById('cart-widget')).render(<Cart />)`. You can have several roots on one page.

Choosing



How this book runs React without a browser

You cannot open a browser from a script, so each runnable example does what Vite does — compile JSX with **esbuild** (a fast compiler of the same kind Vite uses; Vite 8 uses Oxc, earlier versions esbuild) — and then renders into **jsdom**, a pure-JavaScript implementation of the DOM:

```
import '../../../../tools_dom.mjs';          // 1. create window/document globals (jsdom)
import { render, screen } from '@testing-library/react';
// 2. render components, click/type like a user, console.log what is on screen
```

For pictures of real pages, the same code is bundled for the browser and screenshotted in headless Chrome. That is exactly how React apps are tested in real projects (Vitest/Jest + jsdom + Testing Library — Chapter 12).

Common mistakes

- Running `npm run dev` outside the project folder (no `package.json` → "Missing script: dev").
- Opening `index.html` by double-clicking. Vite's `index.html` loads `/src/main.jsx`, which a browser cannot run directly; use the dev server or build first.
- Committing `node_modules/` to git; commit `package.json` and the lockfile instead.
- Mixing package managers (`npm` and `yarn` lockfiles in one repo).

In the interview

30-second answer: "For a new production app, react.dev recommends a framework — Next.js, React Router, or Expo for native — because you get routing, data loading and SSR out of the box. For a pure client-side app or learning, I'd use Vite with the react or react-ts template. CRA is deprecated since February 2025."

- *Why Vite over CRA/webpack?* — The dev server serves native ES modules and compiles only the files the browser requests, so startup and HMR stay fast as the app grows; the production build is still fully bundled and optimised. CRA bundled everything with webpack before showing anything.
- *What is HMR?* — Hot Module Replacement: the dev server swaps the changed module in the running page, keeping state, instead of reloading.

TL;DR

Framework (Next.js / React Router / Expo) for real apps; `npm create vite@latest app -- --template react` for SPAs and learning; CRA is deprecated. `npm run dev` / `build` / `preview`.

1.6 Project structure

What it is

A new Vite React project looks like this (file list from the current `template-react`; the linter config file name varies by template version):

```
my-react-app/
├── index.html          ← the single HTML page; contains <div id="root"> and loads src/main.jsx
├── package.json        ← name, scripts (dev/build/preview/lint), dependencies
├── package-lock.json   ← exact installed versions (after npm install)
├── vite.config.js      ← Vite settings; registers @vitejs/plugin-react (JSX, Fast Refresh)
├── public/             ← files copied as-is (favicon, robots.txt), served from /
├── src/
│   ├── main.jsx        ← entry point: finds #root and renders <App />
│   ├── App.jsx         ← the top-level component you edit first
│   ├── App.css, index.css
│   └── assets/         ← images imported from code (get hashed file names on build)
└── node_modules/       ← installed packages (never edit, never commit)
```

How it works — from `index.html` to pixels

1. The browser loads `index.html`, which has an empty `<div id="root"></div>` and `<script type="module" src="/src/main.jsx">`.
2. `main.jsx` calls **`createRoot(domNode)`** — this creates a React **root**, React's handle on that DOM element.
3. `root.render(<App />)` renders your component tree into it. From now on React owns everything inside `#root`.
4. **`<StrictMode>`** wraps the app in development only: it calls each component twice per render and runs every Effect's setup → cleanup → setup once extra on mount, to expose impure code and missing cleanups early (Chapter 2 §2.5, Chapter 5). It has no effect in production builds.

`code/vite-app/src/main.jsx` (identical in shape to the template):

```jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

`code/vite-app/src/App.jsx`:

```jsx
import { useState } from 'react';

function Welcome({ name }) {
  return <h1>Welcome to React, {name}!</h1>;
}

export default function App() {
  const [count, setCount] = useState(0);
  return (
    <div className="card">
      <Welcome name="Sam" />
      <p>This page was built from components.</p>
      <button onClick={() => setCount((c) => c + 1)}>count is {count}</button>
    </div>
  );
}
```

The runnable twin

Because the Vite dev server is not run here (no network), `code/06_first_app.jsx` mounts the **same** `App` into jsdom exactly as `main.jsx` does, then clicks twice:

```jsx
import '../../../../tools_dom.mjs';
import { StrictMode, act } from 'react';
import { createRoot } from 'react-dom/client';
import App from './vite-app/src/App.jsx';

const root = createRoot(document.getElementById('root'));
await act(() => root.render(<StrictMode><App /></StrictMode>));
console.log('after first render :', document.getElementById('root').textContent);
const button = document.querySelector('button');
await act(() => button.click());
await act(() => button.click());
console.log('after two clicks   :', button.textContent);
```

Output:

```
after first render : Welcome to React, Sam!This page was built from components.count is 0
after two clicks   : count is 2
```

And the same `main.jsx` bundled with esbuild and opened in Chrome (`code/first-app.html`):

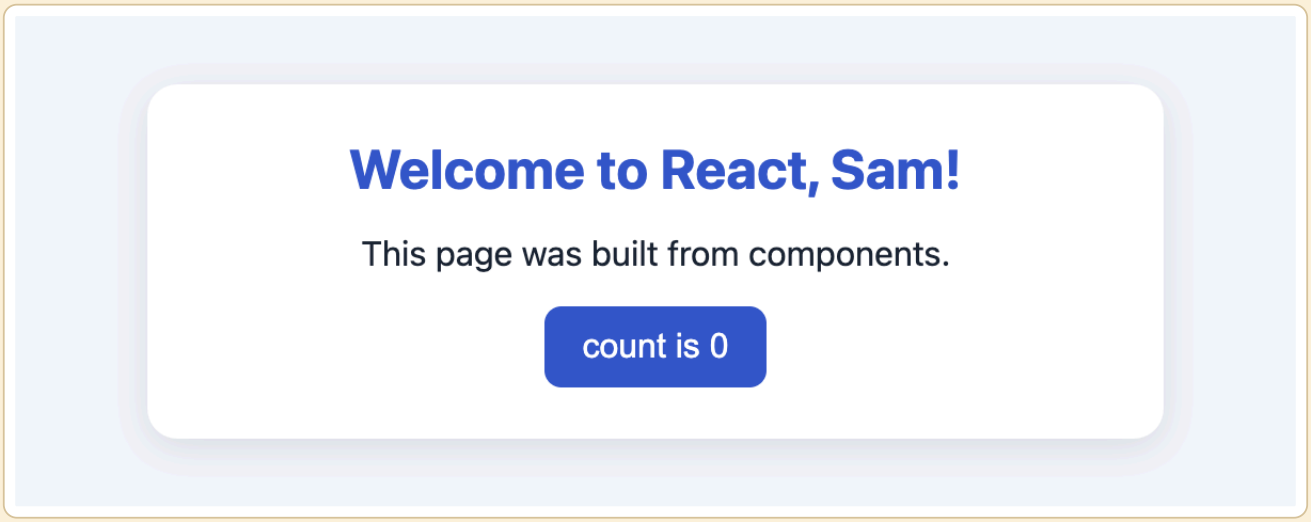


Figure 1.1 — `App` from `vite-app/src` running in a real browser.

`package.json` in brief

```
{
  "type": "module",
  "scripts": { "dev": "vite", "build": "vite build", "preview": "vite preview" },
  "dependencies": { "react": "^19.x", "react-dom": "^19.x" },
  "devDependencies": { "vite": "…", "@vitejs/plugin-react": "…" }
}
```

`dependencies` ship in your app; `devDependencies` are only needed to build it. `^19.x` means "any 19.* version" (semantic versioning — see the JavaScript book's modules chapter, `../../../javascript/chapters/10-modules-and-tooling/README.md`).

How to grow the structure

Start flat; add folders when a pattern appears. A common medium-size layout: `src/components/` (reusable UI), `src/features/<feature>/` (components + hooks + tests for one feature), `src/hooks/`, `src/lib/` (non-React helpers). Keep one component per file when it is exported. Architecture is covered in Chapter 14.

Common mistakes

- Calling `createRoot` twice on the same element (React warns); create the root once, call `render` as needed.
- Changing DOM inside `#root` by hand (`innerHTML = …`) — React will overwrite it or get confused.
- Removing `<StrictMode>` to "fix" doubled console logs; the doubles reveal real bugs.

Self-check

1. Which file does the browser load first? → `index.html`, which loads `src/main.jsx`.
2. What does `createRoot` need? → A DOM element (e.g. `#root`) that React will manage.
3. Does `StrictMode` slow down production? → No; its checks run only in development.

TL;DR

`index.html` (#root) → `main.jsx` (`createRoot(...).render(<StrictMode><App/></StrictMode>)`) → `App.jsx` (your components). `public/` is copied as-is; `node_modules/` is never committed.

1.7 Your first component

What it is

A **component** is a JavaScript function that:

1. has a name starting with a **capital letter** (`Profile`, not `profile`),
2. returns **JSX** (or `null` to render nothing).

```
function Profile() {
  return <img src="https://i.imgur.com/MK3eW3As.jpg" alt="Katherine Johnson" />;
}

export default function Gallery() {
  return (
    <section>
      <h1>Amazing scientists</h1>
      <Profile />
      <Profile />
      <Profile />
    </section>
  );
}
```

`Gallery` is a **parent** that renders three **child** `Profile` components. (A runnable version with its output is `01_function_components.jsx` in Chapter 2 §2.1.) The tree of components is the **component tree**; React DevTools shows it.

Why the capital letter?

JSX uses it to decide the `type`: lowercase `<section>` → the string `'section'` (an HTML tag); capitalised `<Profile />` → the variable `Profile` (your function). A lowercase component would be treated as an unknown HTML tag (Chapter 2 §2.1 shows the output).

Exporting and importing

Style	Export	Import
Default (one per file)	`export default function Button() {}`	`import Button from './Button.jsx'` (any name)
Named (many per file)	`export function Button() {}`	`import { Button } from './Button.jsx'` (exact name)

Teams often pick one style for consistency; named exports make renames and auto-imports more reliable.

Rules

- **Define components at the top level of a module, never inside another component.** A component defined inside a parent is a brand-new function every render, so React sees a *different type* each time (§1.3) and destroys its state and DOM — slow and buggy.
- **A component should be pure while rendering**: same inputs → same JSX, no changing outside variables (Chapter 2 §2.5).
- Keep components small: when a piece of JSX has its own meaning (a card, a row, a header) extract it.

Common mistakes

Mistake	Symptom	Fix
`function profile()` (lowercase)	Renders `<profile>` tag, nothing shown	Capitalise
Missing `return` or a line break after `return`	Component renders nothing (`return` alone returns `undefined`)	Wrap multi-line JSX in `( … )` starting on the `return` line
Nested component definitions	Inputs lose focus on every keystroke	Move the inner function to module level
Calling a component as a function `{Profile()}`	Hooks inside break rules, no own identity in the tree	Use `<Profile />`

TL;DR

Component = capitalised function returning JSX, defined at top level, exported default or named, used as `<Name />`.

1.8 JSX

What it is

JSX (JavaScript XML) is a syntax extension that lets you write markup inside JavaScript. It is *not* HTML and not a string: the compiler (esbuild, Babel, SWC, Oxc) turns each tag into a function call that returns a React element object. You can use React without JSX, but almost nobody does.

`code/02_jsx_is_javascript.jsx`:

```jsx
import { createElement, isValidElement } from 'react';

const fromJsx = <h1 className="title">Hello, {'Ada'}!</h1>;
const byHand = createElement('h1', { className: 'title' }, 'Hello, ', 'Ada', '!');

const show = (el) => JSON.stringify({ type: el.type, props: el.props });
console.log('JSX         ->', show(fromJsx));
console.log('createElement->', show(byHand));
console.log('is a React element?', isValidElement(fromJsx));
console.log('typeof element:', typeof fromJsx, '| frozen (dev):', Object.isFrozen(fromJsx));
```

Output:

```
JSX         -> {"type":"h1","props":{"className":"title","children":["Hello, ","Ada","!"]}}
createElement-> {"type":"h1","props":{"className":"title","children":["Hello, ","Ada","!"]}}
is a React element? true
typeof element: object | frozen (dev): true
```

So JSX is just a nicer way to write `createElement(type, props, ...children)`. Since React 17 the compiler uses the **automatic JSX runtime** (`import { jsx } from 'react/jsx-runtime'` inserted for you), which is why modern files do not need `import React from 'react'`. Elements are **immutable** (frozen in development): to change the UI you render a new element, you never edit an old one.

The rules

`code/03_jsx_rules.jsx`:

```jsx
const user = { name: 'Grace Hopper', avatar: 'https://i.imgur.com/yXOvdOSs.jpg', online: true };
const size = 64;

function ProfileHeader() {
  // Rule 1: return ONE root element. A Fragment (<>...</>) groups without adding a DOM node.
  return (
    <>
      {/* Rule 2: attributes are camelCase; class -> className, for -> htmlFor */}
      <label htmlFor="bio" className="label">Bio</label>
      {/* Rule 3: every tag must be closed, even <img /> and <input /> */}
      <img src={user.avatar} alt={user.name} width={size} height={size} />
      <input id="bio" defaultValue="Wrote the first compiler" />
      {/* Rule 4: curly braces hold any JavaScript EXPRESSION */}
      <h2>{user.name.toUpperCase()} ({user.online ? 'online' : 'offline'})</h2>
      {/* Rule 5: style takes an object; property names are camelCase */}
      <p style={{ backgroundColor: 'black', color: 'pink', fontSize: 14 }}>{2 + 3} papers</p>
    </>
  );
}
```

Output (the HTML React produced):

```
<label for="bio" class="label">Bio</label>
<img alt="Grace Hopper" width="64" height="64" src="https://i.imgur.com/yXOvdOSs.jpg">
<input id="bio" value="Wrote the first compiler">
<h2>GRACE HOPPER (online)</h2>
<p style="background-color: black; color: pink; font-size: 14px;">5 papers</p>
```

#	Rule	Why	Example
1	**One root element**	A function can return only one value; JSX becomes one object	Wrap in `<div>` or a **Fragment** `<>…</>` (no extra DOM node, as the output shows)
2	**Close every tag**	JSX is stricter than HTML	` `, `<img />`, `<li>…</li>`
3	**camelCase attributes**	Attributes become JS object keys; `class` and `for` are reserved words	`className`, `htmlFor`, `onClick`, `tabIndex`, `strokeWidth`. Exceptions: `aria-*` and `data-*` keep dashes
4	**`{}` for JavaScript expressions**	Anything that produces a value: variables, calls, ternaries, `&&`, `.map()`	`{user.name}`, `{count * 2}`, `{formatDate(d)}`
5	**`style` is an object**	`{{ … }}` = braces for JS + an object literal	`style={{ fontSize: 14 }}` — numbers get `px` automatically for most properties

Details that trip beginners:

- **Quotes vs braces for attributes:** `alt="text"` passes a string literal; `alt={name}` passes a JS value. Do not write `src="{avatar}"` — that is the literal text `{avatar}`.
- **Expressions, not statements:** `if`, `for`, and `const` cannot go inside `{}`. Use a ternary, `&&`, `.map()`, or compute a variable *above* the `return` (Chapter 2 §2.7).
- **Comments** inside JSX: `{/* like this */}`.

- **What renders nothing:** `null`, `undefined`, `true`, `false` render nothing; but `0` and `NaN` *do* render (the `&&` trap, Chapter 2 §2.7).
- **Objects cannot be rendered:** `{user}` throws "Objects are not valid as a React child" — render `{user.name}` instead. Arrays of elements are fine.
- **Keyed fragments:** the short `<>` cannot take props; when you need a `key` in a list use `<Fragment key={id}>` (import `Fragment` from `react`).
- **Safety:** text in `{}` is escaped automatically, so `{"<script>…"}` shows as text rather than running — protection against XSS (cross-site scripting). Inserting raw HTML requires the deliberately scary `dangerouslySetInnerHTML={{ __html: … }}`; only use it with sanitised content.
- `defaultValue` (seen above) sets an input's *initial* value without React controlling it; `value` makes it a **controlled input** (Chapter 3 §3.9).

Converting HTML to JSX by hand is error-prone; online "HTML to JSX" converters and editor extensions do it automatically.

Trade-offs

JSX	Alternatives (template languages in Vue/Angular/Svelte)
Full JavaScript inside markup — no new loop/if syntax to learn	Special directives (`v-for`, `@if`) are more restricted but easier for designers to read
Type-checked with TypeScript like any code	Needs dedicated tooling
Needs a compile step	Also needs a compile step (in practice)

In the interview

30-second answer: "JSX is syntax sugar: the compiler turns `<h1 className='x'>Hi</h1>` into a `jsx('h1', {className:'x', children:'Hi'})` call that returns a plain, immutable element object. The rules come from that: one root because a function returns one value, `className` because `class` is reserved, braces for expressions only, and style as an object."

- *Why do you not need `import React` any more?* — The automatic JSX runtime (React 17+) imports `jsx` from `react/jsx-runtime` for you.
- *Is JSX safe from XSS?* — Values in braces are escaped before insertion; the risks are `dangerouslySetInnerHTML`, user-supplied URLs in `href` (React 19 replaces a `javascript:` URL with one that throws "React has blocked a javascript: URL as a security precaution"; still validate URLs yourself) and third-party HTML.
- *Fragment vs div?* — A fragment groups children without an extra DOM node, keeping CSS layouts (flex/grid/tables) and HTML validity intact.

Self-check

1. What does `<Card title="x" />` compile to? → A call like `jsx(Card, { title: 'x' })` returning `{ type: Card, props: { title: 'x' } }`.
2. Why is `<div class="box">` wrong in JSX? → `class` must be `className`; React still sets it but logs "Invalid DOM property `class`. Did you mean `className`?".
3. How do you return two sibling elements? → Wrap them in a Fragment `<>…</>`.
4. Which renders nothing: `false`, `0`, `null`, `''`? → `false`, `null` and `''` (empty text); `0` renders "0".

TL;DR

JSX = `createElement` calls → immutable element objects. One root (use `<>`), close all tags, camelCase attributes (`className`, `htmlFor`), `{expressions}`, `style={{…}}`, text is auto-escaped.

Chapter 1 TL;DR

Idea	Remember
React	Library for UIs built from components; `react-dom` renders to the browser
Declarative	Describe UI for the current state; React applies the DOM changes
Update cycle	Trigger → render (call components) → commit (minimal DOM diff)
Diff rules	Same type = keep + update; different type = replace; lists by key
Rendering strategies	CSR/SPA, SSR (+ hydration), SSG, MPA — frameworks mix them per route
Setup	Framework (Next.js / React Router / Expo) or Vite (`--template react`); CRA deprecated Feb 2025
Entry point	`createRoot(document.getElementById('root')).render(<StrictMode><App/></StrictMode>)`
Components	Capitalised functions returning JSX, defined at module top level
JSX	One root, closed tags, `className`, `{expression}`, `style={{}}`, fragments

Next: Chapter 2 — Components and Props.

Chapter 2 — Components and Props

Components are React's unit of reuse, and **props** are how data flows into them. This chapter shows how to write components, feed them data, nest them with `children`, keep them pure and small, and decide *what* to render with conditions and lists. It ends with a small product catalog that uses every idea.

What you'll learn

#	Topic	You will be able to…
2.1	Function components	write components and avoid the lowercase-name trap
2.2	Props	pass, read, destructure, default and forward props; know they are read-only
2.3	`children`	build wrapper components such as `Card` and `Layout`
2.4	Composition	assemble UIs from small parts, use slots, and avoid prop drilling
2.5	Pure components	explain the purity rule and what `StrictMode` double rendering reveals
2.6	Keeping components small	know when and how to split a component
2.7	Conditional rendering	use `if`, `? :`, `&&`, `null` — and avoid the `0 &&` bug
2.8	Lists and keys	render arrays with `map` / `filter` and pick correct keys
2.9	Putting it together	read a complete small UI built from these parts

Code: `code/` (React 19.3, run in jsdom with Testing Library — see Chapter 1 §1.5).

2.1 Function components

What it is

A **function component** is a plain JavaScript function that takes one argument (the **props** object) and returns JSX. Its name **must start with a capital letter**. React calls it whenever it needs to know what that part of the screen should look like.

Analogy: a component is a *recipe card*. The recipe (function) is written once; each `<Profile />` in your JSX is one dish cooked from it. Props (next section) are the ingredients you hand over for that particular dish.

> Older React code uses **class components** (`class X extends React.Component` with `render()` and lifecycle methods). They still work but are legacy; everything new — Hooks, Server Components, the React Compiler — targets function components. You only need to *read* classes when maintaining old code (and for error boundaries, Chapter 14).

How it works

`code/01_function_components.jsx`:

```jsx
function Profile() {
  return <img src="https://i.imgur.com/MK3eW3As.jpg" alt="Katherine Johnson" />;
}

function Gallery() {
  return (
    <section>
      <h1>Amazing scientists</h1>
      <Profile />
      <Profile />
    </section>
  );
}

// Lowercase name: JSX treats <profile /> as an unknown HTML tag, NOT as the function below.
function profile() {
  return <img alt="never shown" />;
}
// ... render(<Gallery />) and render(<div><profile /></div>)
```

Output:

```
Gallery      : <section><h1>Amazing scientists</h1><img alt="Katherine Johnson" src="https://i.imgur.com/MK3eW3As.jpg">
<img alt="Katherine Johnson" src="https://i.imgur.com/MK3eW3As.jpg"></section>
<profile />  : <div><profile></profile></div>
React warned : The tag <profile> is unrecognized in this browser. If you meant to render a React component, start its
name with an uppercase letter.
```

Step by step:

1. `render(<Gallery />)` creates an element `{ type: Gallery }`.
2. React sees that `type` is a function, so it **calls** `Gallery(props)`.
3. `Gallery` returns elements including two `{ type: Profile }`; React calls `Profile` for each.
4. When only strings (`'section'`, `'img'`) remain, React creates the DOM nodes.
5. `<profile />` became the *string* `'profile'`, so React created a meaningless `<profile>` tag and warned.

Notice the components themselves (`Gallery`, `Profile`) do **not** appear in the HTML — only what they return does. The component tree exists in React (visible in React DevTools); the DOM tree contains only host elements.

Rules recap (from Chapter 1 §1.7)

- Capitalised name; returns JSX, a string/number, an array, or `null`.
- Declared at the top level of a module — never inside another component.
- Use it as `<Profile />`, not `Profile()` (calling it directly bypasses React: no own place in the tree, and Hooks inside would belong to the caller).

TL;DR

A component is a capitalised function `(props) => JSX`. React calls it; only the host elements it returns reach the DOM.

2.2 Props

What it is

Props (short for *properties*) are the inputs a parent passes to a child, written like HTML attributes: `<Avatar person={ada} size={50} />`. React collects them into **one object** and passes it as the component's first argument. Any JavaScript value can be a prop: strings, numbers, booleans, objects, arrays, functions, and even JSX.

Analogy: props are like the settings on a coffee machine order — size, milk, sugar. The machine (component) reads them to make the drink; it does not rewrite your order.

Why it matters

Without props every `Profile` would show the same person. Props make one component reusable with different data, and they are the main channel of the **one-way data flow**: data moves *down* the tree from parent to child. (Data moves *up* by passing a function as a prop — Chapter 3.)

How it works

1. **Pass** props in JSX: strings in quotes, anything else in braces. A bare attribute (`admin`) means `true`.
2. **Read** them: the function receives `props`; usually you **destructure** in the parameter list: `function Avatar({ person, size })`.
3. **Default** a prop with destructuring defaults: `{ size = 100 }`. The default applies only when the prop is missing or `undefined` (not for `null` or `0`).
4. **Forward** props you do not use yourself with the rest/spread syntax: `function Button({ variant, ...rest }) { return <button {...rest} /> }`.

`code/02_props.jsx`:

```
function getImageUrl(person, size = 's') {
  return `https://i.imgur.com/${person.imageId}${size}.jpg`;
}

// Destructure props in the parameter list; `size = 100` is a default used when size is undefined.
function Avatar({ person, size = 100 }) {
  return <img className="avatar" src={getImageUrl(person)} alt={person.name} width={size} height={size} />;
}

function Profile() {
  return (
    <div>
      <Avatar person={{ name: 'Lin Lanying', imageId: '1bX5QH6' }} size={50} />
      <Avatar person={{ name: 'Aklilu Lemma', imageId: 'OKS67lh' }} />
    </div>
  );
}

// Forwarding: pass every prop through with spread syntax.
function Button({ variant = 'primary', ...rest }) {
  return <button className={`btn btn-${variant}`} {...rest} />;
}
render(<Button type="submit" disabled aria-label="Save">Save</Button>);

// Props are read-only: in development React freezes the props object.
function Mutator(props) {
  try { props.title = 'changed'; } catch (e) { console.log('mutating props ->', e.constructor.name + ':', e.message); }
  return <h2>{props.title}</h2>;
}

// What a component actually receives: ONE object.
function Spy(props) { console.log('props object:', JSON.stringify(props)); return null; }
render(<Spy name="Ada" age={36} admin tags={['math', 'code']} />);
```

Output:

```
<div>
<img class="avatar" alt="Lin Lanying" width="50" height="50" src="https://i.imgur.com/1bX5QH6s.jpg">
<img class="avatar" alt="Aklilu Lemma" width="100" height="100" src="https://i.imgur.com/OKS67lhs.jpg">
</div>

forwarded   : <button class="btn btn-primary" type="submit" disabled="" aria-label="Save">Save</button>
mutating props -> TypeError: Cannot assign to read only property 'title' of object '#<Object>'
props object: {"name":"Ada","age":36,"admin":true,"tags":["math","code"]}
```

What to notice:

- `person={{ … }}` — the outer braces mean "JavaScript here", the inner braces are an object literal (same as `style={{…}}`).
- The second `Avatar` had no `size`, so the default `100` was used.

- `children` ("Save") was forwarded by `{...rest}` too, because `children` is just another prop.
- `admin` with no value became `true`.
- Assigning to a prop threw: in development React **freezes** the props object. In production it is not frozen, but mutating props is still a bug — the parent owns that data.

Props are a snapshot, and change only from above

Each render receives a fresh props object. When a parent re-renders with different values, the child is called again with the new props — you never "update" your own props. If a component needs to change something over time in response to the user, that is **state** (Chapter 3).

	Props	State
Owned by	The parent	The component itself
Changed by	The parent re-rendering with new values	The component calling its setter (`setX`)
Purpose	Configure a component from outside	Remember something that changes with interaction
Analogy	Arguments to a function	A function's private memory between calls

Special props

Prop	Behaviour
`key`	Used by React to identify list items (§2.8). **Not** passed to your component: `props.key` is `undefined`.
`children`	Whatever you nest between the tags (§2.3).
`ref`	In React 19 a normal prop for function components (`forwardRef` is no longer needed and will be deprecated) — see Chapter 5.

React 19 removed `propTypes` checks and `defaultProps` for function components; use destructuring defaults, and TypeScript for type checking (Chapter 11).

Trade-offs — spreading props

`{...props}` forwarding	Listing props explicitly
Short; wrappers pass through any native attribute (`aria-*`, `onClick`)	Clear which props the component supports
Can accidentally pass unknown props to the DOM (React warns) or override your own ones (order matters: later wins)	More typing when wrapping native elements

Rule of thumb: spread in thin wrappers around a native element (`Button`, `Input`); list explicitly in feature components.

Common mistakes

- `size="50"` passes a **string**; use `size={50}` for a number.
- Forgetting braces in destructuring: `function Avatar(person, size)` — the second parameter is not a prop (it is `undefined`).
- Copying a prop into state "to be able to change it" — the copy ignores later updates from the parent (Chapter 3 §3.8).
- Using `defaultProps` on a function component in React 19 — it is ignored.

In the interview

30-second answer: "Props are the read-only inputs a parent passes to a child, collected into one object. They make components reusable and implement one-way data flow: data goes down via props, and events go up via callback props. A component never mutates its props; if it needs changing data it uses state."

- *Why are props read-only?* — The parent owns the data. If a child mutated it, the parent and siblings would show stale or inconsistent values and React could not know a re-render is needed.
- *How do you give a prop a default?* — Destructuring default `({ size = 100 })`; `defaultProps` is gone for function components in React 19.

- *Is `key` a prop?* — It is written like one but React consumes it; the component cannot read it.

Self-check

1. How do you pass the number 3? → `count={3}` (quotes would pass `"3"`).
2. What does `<Toggle on />` pass? → `{ on: true }`.
3. When does `({ size = 100 })` use 100? → When `size` is missing or `undefined`.
4. Can a child change its props? → No; it asks the parent (callback prop) or uses its own state.

TL;DR

Props = one read-only object from the parent. Destructure with defaults, spread to forward, never mutate. Changing data belongs in state.

2.3 The `children` prop

What it is

Whatever you put *between* a component's opening and closing tags arrives as `props.children`. This lets you write **wrapper** components — cards, panels, modals, layouts — that do not need to know what they contain.

Analogy: a picture frame. The frame (component) provides border and glass; the picture (children) can be anything.

How it works

`code/03_children_composition.jsx` (first part):

```jsx
function Card({ title, children }) {
  return (
    <div className="card">
      {title && <h3>{title}</h3>}
      <div className="card-body">{children}</div>
    </div>
  );
}

// children can be anything renderable: text, elements, arrays, or nothing.
function Show({ children }) { return <span>[{children}]</span>; }
render(<Show>hi</Show>); render(<Show>{['a', 'b']}</Show>); render(<Show />);
```

Output (last three lines of the program):

```
text   : <span>[hi]</span>
array  : <span>[ab]</span>
none   : <span>[]</span>
```

`children` may be a string, one element, an array of elements, or `undefined`. Render it with `{children}` and let React deal with the shape. (There is a `Children` utility in `react` for counting or mapping over children, but react.dev files it under *legacy APIs* because it is fragile; prefer passing data as props.)

TL;DR

Nested JSX arrives as `props.children`; render it with `{children}` to build generic wrappers.

2.4 Composition

What it is

Composition means building complex UIs by *combining* components — passing components into other components — instead of creating special versions through inheritance. React has no component inheritance; composition covers all its use cases.

How it works — slots

When a layout has several "holes", pass JSX through **named props** (often called *slots*) as well as `children`:

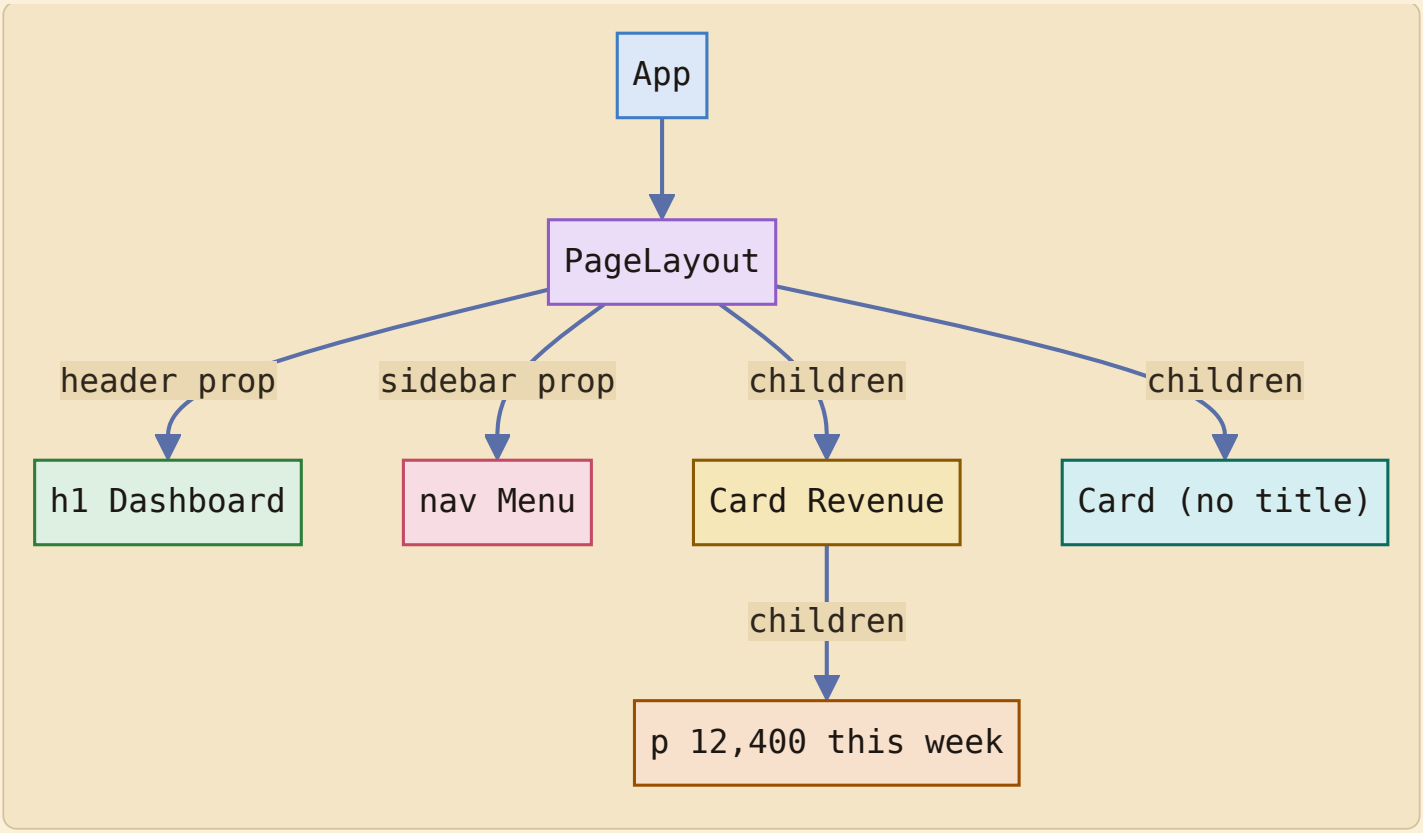
```
function PageLayout({ header, sidebar, children }) {
  return (
    <div className="page">
      <header>{header}</header>
      <aside>{sidebar}</aside>
      <main>{children}</main>
    </div>
  );
}

render(
  <PageLayout header={<h1>Dashboard</h1>} sidebar={<nav>Menu</nav>}>
    <Card title="Revenue">
      <p>$12,400 this week</p>
    </Card>
    <Card>
      <p>No title here</p>
    </Card>
  </PageLayout>,
);
```

Output (HTML, one tag per line):

```
<div class="page">
<header>
<h1>Dashboard</h1>
</header>
<aside>
<nav>Menu</nav>
</aside>
<main>
<div class="card">
<h3>Revenue</h3>
<div class="card-body">
<p>$12,400 this week</p>
</div>
</div>
<div class="card">
<div class="card-body">
<p>No title here</p>
</div>
</div>
</main>
</div>
```

The second card has no `<h3>` because `title` was `undefined` and `{title && …}` rendered nothing (§2.7).



Why it matters — composition beats prop drilling

Prop drilling is passing a prop through several components that do not use it, only to reach a deep child. Example: `App → Page → Sidebar → UserMenu → Avatar`, where only `Avatar` needs `user`. Two fixes:

1. **Composition (try first):** let `App` create `<Avatar user={user} />` itself and pass it down as `children` or a slot. The middle components just place `{children}` and never see `user`.
2. **Context** (Chapter 6) when many distant components need the same data (theme, current user, locale).

Composition vs inheritance

Need	OOP inheritance idea	React composition
A special kind of dialog	`class WelcomeDialog extends Dialog`	`<Dialog title="Welcome">…</Dialog>` — configure via props
Shared behaviour	Base class methods	A custom Hook (Chapter 7) or a wrapper component
Varying inner content	Override a method	`children` or a slot prop

Real-world examples

- UI libraries (MUI, Chakra, Radix/shadcn) expose `Card`, `Dialog`, `Tabs` that accept `children` and slot props such as `title`, `actions`, `icon`.
- App shells: a `Layout` with header/sidebar/content slots wrapping every page; routers render the current page into an outlet slot (Chapter 9).

In the interview

30-second answer: "Composition means building UIs by nesting components and passing JSX as `children` or slot props. React recommends it over inheritance. It also solves most prop drilling: the top component creates the deep child itself and passes it through as children, so intermediate components never touch the data."

- *When would you use context instead?* — When many components at different depths need the same value (theme, auth, locale) and passing it through children would be awkward.
- *Can a prop be a component?* — Yes: either an element (`icon={<Star />}`) or a component type (`as={Link}`), rendered as `<Icon />` after capitalising the variable.

TL;DR

Combine small components; pass JSX via `children` and slot props. Composition replaces inheritance and is the first cure for prop drilling.

2.5 Pure components

What it is

In mathematics a **pure function** (1) always returns the same result for the same inputs and (2) changes nothing outside itself (no **side effects**). React assumes every component is pure **during rendering**: given the same props, state and context, it returns the same JSX and does not modify pre-existing variables or objects, the DOM, or anything else.

Analogy: a calculator's `2 + 3` always shows 5 and does not change the number on your neighbour's calculator.

Why it matters

React may call your component **more than once, in any order, or throw a render away** (for concurrent rendering, Suspense – React's way of showing a fallback while part of the UI waits, Chapter 9 – StrictMode checks, server rendering). If rendering has side effects, those extra calls produce wrong results. Purity also enables caching: if inputs are unchanged, React (and the React Compiler) can skip the call.

How it works – StrictMode exposes impurity

`<StrictMode>` (development only) renders each component **twice** and keeps the second result. A pure component produces identical output both times; an impure one shows the bug.

`code/04_purity.jsx`:

```jsx
let guest = 0;
function ImpureCup() {
  guest = guest + 1;              // BAD: changes a variable outside the component during render
  return <h2>Tea cup for guest #{guest}</h2>;
}

function PureCup({ guest }) {     // GOOD: output depends only on props
  return <h2>Tea cup for guest #{guest}</h2>;
}

// Local mutation is fine: the array is created during this render.
function TeaGathering() {
  const cups = [];
  for (let i = 1; i <= 3; i++) cups.push(<PureCup key={i} guest={i} />);
  return <>{cups}</>;
}
```

Output:

```
impure, no StrictMode: Tea cup for guest #1Tea cup for guest #2Tea cup for guest #3
impure, StrictMode   : Tea cup for guest #2Tea cup for guest #4Tea cup for guest #6
pure,   StrictMode   : Tea cup for guest #1Tea cup for guest #2Tea cup for guest #3
local mutation OK    : Tea cup for guest #1Tea cup for guest #2Tea cup for guest #3
```

Without StrictMode the impure version *looks* right – which is exactly why the bug would stay hidden until React happened to re-render. StrictMode's double call makes it obvious (2, 4, 6).

Where side effects belong

Allowed during render	Not allowed during render	Put it here instead
Reading props, state, context	Changing variables/objects that existed before the render	Local variables, or state
Creating and mutating **local** variables/arrays	Mutating props or state objects	Create new objects (Chapter 3 §3.6)
Calculating derived values	DOM manipulation, `fetch`, timers, subscriptions, logging analytics	**Event handlers** (first choice) or **Effects** `useEffect` (Chapter 5)
	`Math.random()`, `Date.now()` for output (non-deterministic)	Compute once in state initialiser, handler, or Effect

Event handlers do not need to be pure: they run in response to a user action, not during rendering.

Common mistakes

- "Removing StrictMode fixes my doubled logs." It hides the symptom; fix the impurity.
- Sorting a prop array in place: `items.sort()` mutates the parent's array. Use `items.toSorted()` or `[...items].sort()`.
- Pushing to a module-level array during render (a render counter, a cache) — works once, breaks under re-render.

In the interview

30-second answer: "A React component must be pure while rendering: same props, state and context in, same JSX out, and no mutation of anything that existed before the render. React relies on this because it may render a component several times or discard a render. Side effects go in event handlers, or in Effects when they must sync with an external system. StrictMode double-renders in development to catch violations."

- *Is `console.log` in render impure?* — Technically a side effect but harmless; React DevTools shows the second StrictMode log dimmed so you can tell them apart. Avoid anything that changes app data.
- *Can a component mutate a local array?* — Yes — "local mutation" of values created during the same render is fine.

Self-check

1. Why did StrictMode print 2, 4, 6? → Each cup rendered twice; the shared counter incremented on both calls.
2. Is `items.sort()` on a prop OK in render? → No; it mutates the parent's array. Copy first.
3. Where should `fetch` go? → Event handler, or an Effect / data library (Chapters 5, 9).

TL;DR

Render = pure calculation. Same inputs → same JSX; mutate only what you created in this render; side effects go to event handlers or Effects. StrictMode renders twice to catch mistakes.

2.6 Keeping components small

What it is

A guideline, not a rule: each component should have **one clear job**. When a component grows several responsibilities, extract pieces into their own components.

When to extract

Signal	Example	Extract
A block of JSX has its own name in conversation	"the price", "the stock label"	`Price`, `StockLabel`
The same markup appears twice	Two identical badge spans	`Badge`
A section has its own logic (conditions, formatting)	Stock thresholds	`StockLabel` with its `if`s
The file is hard to scroll / review (a few hundred lines)	A 400-line `Dashboard`	Sub-components per panel
A part needs its own state	An expandable row	`Row` with local state

How to extract (steps)

1. Cut the JSX block into a new capitalised function at module top level.
2. Every variable the block used from the parent becomes a **prop**.
3. Replace the block with `<NewComponent prop={value} />`.
4. If more than one file uses it, move it to its own file and export it.

Trade-offs

Too big	Too small
Hard to read, test, and reuse; many reasons to change	Many files; you jump around to understand one screen; props passed through many thin layers

A useful split: **presentational** components (take props, render UI — `Price`, `Badge`) versus **feature** components (know the data and wire things together — `Catalog`). Chapter 14 covers container/presentational patterns further.

TL;DR

One job per component. Extract when JSX has a name, repeats, carries its own logic, or needs its own state — its free variables become props.

2.7 Conditional rendering

What it is

Showing different JSX depending on props or state. Because JSX is JavaScript, you use ordinary JavaScript: `if`, the ternary `? :`, logical `&&`, and variables.

How it works — four techniques

`code/05_conditional_rendering.jsx`:

```jsx
// 1. if / early return (statements are allowed ABOVE the JSX)
function Item({ name, isPacked }) {
  if (isPacked) return <li className="item">{name} ✅</li>;
  return <li className="item">{name}</li>;
}
// 2. ternary inside JSX
function Item2({ name, isPacked }) {
  return <li className="item">{isPacked ? <del>{name}</del> : name}</li>;
}
// 3. && : render something or nothing
function Item3({ name, isPacked }) {
  return <li className="item">{name} {isPacked && '✅'}</li>;
}
// 4. store JSX in a variable
function Item4({ name, isPacked }) {
  let content = name;
  if (isPacked) content = <strong>{name} (packed)</strong>;
  return <li className="item">{content}</li>;
}
// returning null renders nothing at all
function Hidden({ show }) { return show ? <p>visible</p> : null; }

// The trap: a number on the left of && is rendered when it is 0.
function Inbox({ messages }) {
  return <p>{messages.length && `You have ${messages.length} messages`}</p>;
}
function InboxFixed({ messages }) {
  return <p>{messages.length > 0 && `You have ${messages.length} messages`}</p>;
}
```

Output:

```
if/return : <ul><li class="item">Space suit ✅</li><li class="item">Photo</li></ul>
ternary   : <ul><li class="item"><del>Helmet</del></li></ul>
&&        : <ul><li class="item">Helmet ✅</li><li class="item">Map </li></ul>
variable  : <ul><li class="item"><strong>Helmet (packed)</strong></li></ul>
null      : ""
0 && bug  : <p>0</p>
fixed     : <p></p>
```

Which to use

Technique	Best for	Watch out
`if` + early `return`	Whole component differs (loading / error / empty states)	Duplicate wrapper markup
`cond ? a : b`	Choosing between two small pieces inline	Nested ternaries become unreadable → extract a component
`cond && a`	Show-or-nothing	Left side `0` or `NaN` is rendered — make it a real boolean (`> 0`, `!!x`, `Boolean(x)`)
Variable (`let content`)	Several conditions building one piece	Slightly more lines
`return null`	Component decides to hide itself	The *parent* often should decide instead

Why `0` shows: `a && b` returns `a` when `a` is falsy. `false`, `null`, `undefined` render nothing, but `0` is a number, and numbers render. (`''` renders an empty text node — invisible.)

A note on identity: `isPacked ? <del>{name}</del> : name` produces different *types* at the same position, so React replaces the DOM there (Chapter 1 §1.3). When both branches are the same component (`cond ? <Counter a /> : <Counter b />`), React keeps the same instance and its state — Chapter 3 §3.10 shows this and how `key` changes it.

In the interview

- *Why does `{count && <Badge />}` show "0"?* — `&&` returns its falsy left operand; 0 is renderable. Use `count > 0 &&`.
- *How do you render nothing?* — Return `null` (or `false`/`undefined` inside JSX).

TL;DR

`if`/early return for big branches, `? :` for either-or, `&&` for show-or-nothing (with a boolean on the left!), `null` for nothing.

2.8 Rendering lists and keys

What it is

To render a collection you turn an **array of data** into an **array of elements**, usually with `array.map()` (and `filter()` to select). Each element in the array needs a `key`: a string or number that identifies that item among its siblings.

Analogy: keys are like name tags at a conference. If people change seats, you still know who is who by the tag, not by the seat number.

How it works

`code/06_lists_and_keys.jsx`:

```
const people = [
  { id: 0, name: 'Creola Katherine Johnson', profession: 'mathematician' },
  { id: 1, name: 'Mario José Molina-Pasquel Henríquez', profession: 'chemist' },
  { id: 2, name: 'Mohammad Abdus Salam', profession: 'physicist' },
  { id: 3, name: 'Percy Lavon Julian', profession: 'chemist' },
];

function ChemistList() {
  const chemists = people.filter((p) => p.profession === 'chemist');
  return <ul>{chemists.map((p) => <li key={p.id}>{p.name}</li>)}</ul>;
}

// Missing key -> React logs a warning in development.
render(<ul>{people.map((p) => <li>{p.name}</li>)}</ul>);
```

Output:

```
<ul>
<li>Mario José Molina-Pasquel Henríquez</li>
<li>Percy Lavon Julian</li>
</ul>

warning: Each child in a list should have a unique "key" prop. See https://react.dev/link/warning-keys for more
information.
```

Arrow function reminder: `p => <li>…</li>` returns implicitly; with a body `p => { return <li>…</li>; }` you must write `return`, or the list is empty.

Why keys matter

On re-render React must match old list items to new ones (reconciliation, Chapter 1 §1.3). The key is how it matches. With a stable ID React knows "the Bread row moved down one place"; with the **array index** as key, it thinks "row 0 changed its text from Bread to Eggs", and any **state** in the row (DOM input text, component state, focus, animations) stays at the old position.

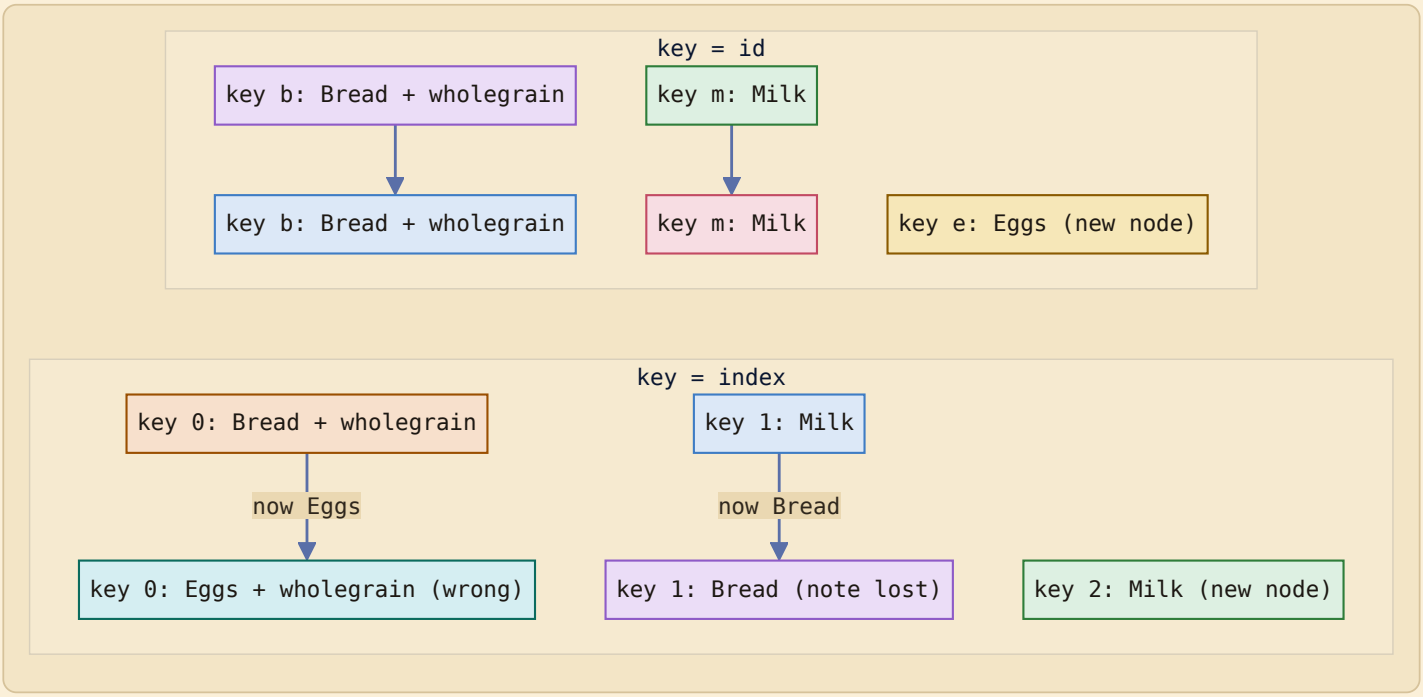
The program renders a list of rows, each with an uncontrolled `<input>`, types *wholegrain* into Bread's input, then inserts Eggs at the top:

```
function NoteList({ items, useIndexKey }) {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={useIndexKey ? index : item.id}>
          {item.label}: <input aria-label={`note for ${item.label}`} defaultValue="" />
        </li>
      ))}
    </ul>
  );
}
// type 'wholegrain' into Bread's input, then rerender with Eggs prepended
```

Output:

```
key=index after prepend -> Eggs"wholegrain"  Bread""  Milk""
key=id    after prepend -> Eggs""  Bread"wholegrain"  Milk""
```

With index keys the note jumped to **Eggs** — a real data-corruption bug users would notice. With ID keys it stayed with Bread.



Rules for keys

1. **Unique among siblings** (not globally). Two different lists can reuse the same keys.
2. **Stable**: the same item gets the same key on every render. Use IDs from your data (database IDs, UUIDs created *when the item is created*).
3. **Never generate during render**: `key={Math.random()}` or `key={crypto.randomUUID()}` in JSX creates new keys each render → every item is destroyed and recreated (slow, loses input focus and state).
4. **Index is acceptable only** when the list is static: never reordered, filtered, or inserted into (except at the end), and items have no state.
5. Put the key on the **outermost element returned from** `map` — on `<ProductCard key=… />`, not on the `<li>` inside `ProductCard`.
6. The key is not passed to the component; pass the ID separately if the child needs it (`<Row key={id} id={id} />`).
7. When one item renders several siblings, wrap them in `<Fragment key={id}>…</Fragment>` (the short `<>` cannot take a key).

Common mistakes

Mistake	Symptom
Missing key	Console warning; index used implicitly → same bugs as index keys
Index key on a sortable/filterable list	Inputs, checkboxes, or expanded state attach to the wrong row
`Math.random()` key	Inputs lose focus on each keystroke, poor performance
Duplicate keys	Warning "Encountered two children with the same key"; items duplicated or dropped
Key inside the child component instead of on the element in `map`	Warning persists

Real-world examples

- A chat app keys messages by message ID so new messages at the bottom don't re-create older ones.
- A Kanban board keys cards by card ID so dragging a card between positions keeps its state.
- Keys also work outside lists to **reset** a component: `<Profile key={userId} />` gives each user a fresh instance (Chapter 3 §3.10).

In the interview

30-second answer: "`map` data to elements and give each a key that is stable and unique among siblings, usually the item's ID. React uses keys to match items between renders, so it can move DOM nodes and keep state with the right item. Index keys break when the list is reordered, filtered or prepended — state stays at the position instead of following the item. Random keys are worse: everything remounts every render."

- *When is index as key fine?* — Static lists that never change order and have no per-item state (e.g. a fixed footer menu).
- *Do keys need to be globally unique?* — No, only among siblings of the same parent.
- *Can I read `props.key`?* — No; pass the value under another prop name.

Self-check

1. Where does the key go in `items.map(i => <Row item={i} />)`? → On `<Row key={i.id} … />`.
2. What goes wrong with `key={index}` after deleting the first item? → Each remaining item gets the next item's state/DOM (everything shifts).
3. Why not `key={crypto.randomUUID()}`? → New key each render → remount everything.

TL;DR

`data.map(item => <X key={item.id} … />)`. Keys: unique among siblings, stable, from data, never random, index only for static lists.

2.9 Putting it together — a product catalog

`code/catalog.jsx` combines every idea in this chapter: small presentational components (`Price`, `Badge`, `StockLabel`), a card that composes them, a `Section` wrapper using `children`, conditional rendering, and a filtered list with keys.

```jsx
export function Price({ amount, currency = 'USD' }) {
  const text = new Intl.NumberFormat('en-US', { style: 'currency', currency }).format(amount);
  return <span className="price">{text}</span>;
}

export function Badge({ tone = 'grey', children }) {
  return <span className={`badge badge-${tone}`}>{children}</span>;
}

export function StockLabel({ stock }) {
  if (stock === 0) return <Badge tone="red">Out of stock</Badge>;
  if (stock < 5) return <Badge tone="amber">Only {stock} left</Badge>;
  return null;                                  // plenty in stock: show nothing
}

export function ProductCard({ product }) {
  const { name, price, stock, rating, tags } = product;
  return (
    <article className="product" aria-label={name}>
      <h3>{name}</h3>
      <div className="badges">
        {tags.includes('new') && <Badge tone="blue">New</Badge>}
        {tags.includes('sale') && <Badge tone="green">Sale</Badge>}
        <StockLabel stock={stock} />
      </div>
      <p><Price amount={price} /> · ★ {rating.toFixed(1)}</p>
      <button disabled={stock === 0}>{stock === 0 ? 'Sold out' : 'Add to cart'}</button>
    </article>
  );
}

export function Section({ title, children }) {
  return (
    <section className="section">
      <h2>{title}</h2>
      {children}
    </section>
  );
}

export function Catalog({ products, minRating = 0 }) {
  const visible = products.filter((p) => p.rating >= minRating);
  if (visible.length === 0) return <p>No products match.</p>;
  return (
    <Section title={`Products (${visible.length})`}>
      <div className="grid">
        {visible.map((p) => <ProductCard key={p.id} product={p} />)}
      </div>
    </Section>
  );
}
```

`code/07_product_catalog.jsx` renders it and reads the screen the way a user (or a test) would — by roles such as `article`, `heading` and `button`:

```jsx
const { rerender } = render(<Catalog products={PRODUCTS} />);
// ... print each card's heading, price, badges and button
rerender(<Catalog products={PRODUCTS} minRating={4.5} />);
rerender(<Catalog products={PRODUCTS} minRating={5} />);
```

Output:

```
Products (4)
  Mechanical keyboard  |  $89.00 | badges: New               | button: Add to cart
  USB-C hub            |  $35.00 | badges: Out of stock      | button: Sold out (disabled)
  27" monitor          | $249.00 | badges: Sale, Only 3 left | button: Add to cart
  Laptop stand         |  $42.00 | badges: -                 | button: Add to cart

minRating=4.5 -> Products (2) [ 'Mechanical keyboard', '27" monitor' ]
minRating=5   -> No products match.
```

The same components bundled for the browser (`code/catalog.html`, styles in `catalog.css`):

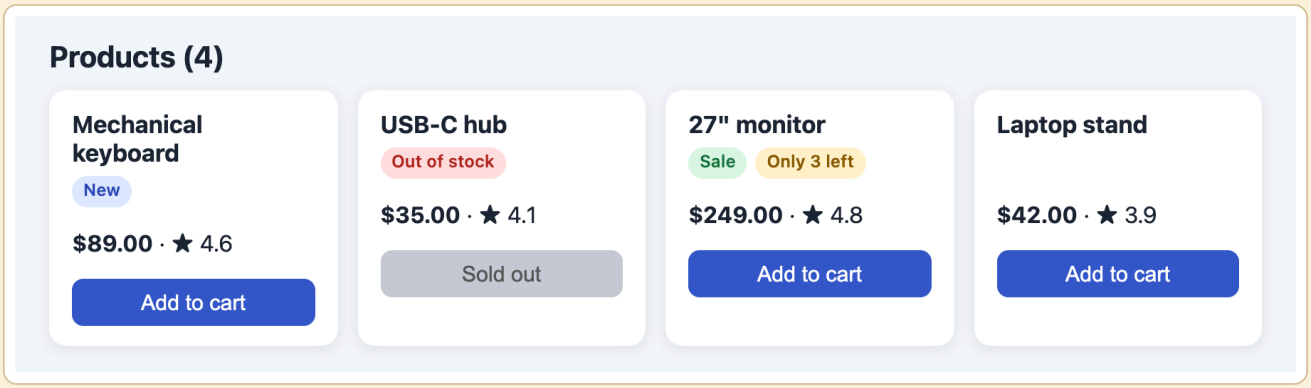


Figure 2.1 — `Catalog` rendered in Chrome. Every badge and button state comes from props and conditional rendering.

Things to notice: `rerender` with a new `minRating` prop re-ran `Catalog`; nothing in the components "updates" itself — they just describe the UI for the props they receive. The buttons do nothing yet: making them respond to clicks needs **state and events**, the subject of Chapter 3.

Chapter 2 TL;DR

Idea	Remember
Component	Capitalised function `(props) => JSX`, top-level, used as `<X />`
Props	One read-only object; destructure with defaults; strings in quotes, other values in braces
Forwarding	`({ a, ...rest }) => <button {...rest} />`; later props override earlier
`children`	Nested JSX; enables wrappers; slots = JSX in named props
Composition	Combine components instead of inheriting; first fix for prop drilling
Purity	Same inputs → same JSX, no outside mutation during render; StrictMode renders twice in dev
Small components	One job each; free variables become props
Conditions	`if`/return, `? :`, `&&` (boolean on the left — the `0` trap), `null`
Lists	`map` + `key` from stable IDs; index only for static lists; never random

Next: Chapter 3 — State and Events.

Chapter 3 — State and Events

Chapter 2's catalog could *show* data but not *react* to the user. This chapter adds interactivity: **event handlers** respond to clicks and typing, and **state** lets a component remember values between renders. Most React bugs beginners meet — "my counter only goes up by one", "the screen doesn't update", "the note moved to the wrong row" — come from the ideas here, so each is shown with a program you can run.

What you'll learn

#	Topic	You will be able to…
3.1	Responding to events	attach handlers, pass arguments, use `onX` props, stop propagation, prevent defaults
3.2	`useState`	give a component memory and trigger re-renders
3.3	State as a snapshot	predict what a handler sees after calling a setter
3.4	Batching	explain why several `setX` calls produce one re-render
3.5	Updater functions	queue updates based on the latest value (`n => n + 1`)
3.6	Objects and arrays in state	update them immutably (spread, `map`, `filter`, `toSorted`) and with Immer
3.7	Lifting state up	share state between siblings through their parent
3.8	Derived state	keep the minimal state and compute the rest
3.9	Controlled inputs	bind inputs to state and transform what the user types
3.10	State and tree position	know when state is kept or reset, and use `key` to reset it
3.11	Putting it together	a to-do app using every idea

Code: `code/` — React 19.3 in jsdom; user actions are simulated with `fireEvent` (dispatches one DOM event) and `@testing-library/user-event` (simulates a real user: focus, key presses, clicks).

3.1 Responding to events

What it is

An **event** is something that happens in the browser — a click, a key press, a form submission. An **event handler** is a function you give React to run when that event happens. In JSX you pass it as a prop named `on` + event name in camelCase: `onClick`, `onChange`, `onSubmit`, `onKeyDown`, `onPointerMove`, `onFocus`, …

Analogy: a doorbell. You install the bell (pass the handler); it rings (runs) only when someone presses the button — not while you are installing it.

How it works

1. Define a function (inside the component, so it can read props and state). Convention: name it `handle` + event, e.g. `handleClick`.
2. Pass the **function itself** — `onClick={handleClick}` or an inline arrow `onClick={() => …}`.
3. When the event fires, React calls it with an **event object** (`e`): `e.target` (the element), `e.type`, `e.key` for keyboard events, `e.preventDefault()`, `e.stopPropagation()`.

`code/02_events.jsx`:

```jsx
// 1. Pass the function; do not call it.
function Right() { return <button onClick={() => console.log('  Right: clicked')}>right</button>; }
function Wrong() {
  const handleClick = () => console.log('  Wrong: ran during RENDER, not on click');
  return <button onClick={handleClick()}>wrong</button>; // calls it now; onClick receives undefined
}

// 2. Passing arguments + custom event props (onX / handleX naming).
function Toolbar({ onPlay, onUpload }) {
  return (
    <div>
      <button onClick={() => onPlay('Kiki')}>Play movie</button>
      <button onClick={onUpload}>Upload image</button>
    </div>
  );
}
function App() {
  function handlePlay(title) { console.log(`  Playing ${title}`); }
  return <Toolbar onPlay={handlePlay} onUpload={(e) => console.log('  Uploading... event type =', e.type)} />;
}

// 3. Propagation: events bubble from child to parent; stopPropagation halts it.
function Panel({ stop }) {
  return (
    <div onClick={() => console.log('    parent <div> onClick')}>
      <button onClick={(e) => { if (stop) e.stopPropagation(); console.log('    child <button> onClick'); }}>
        {stop ? 'stop' : 'bubble'}
      </button>
    </div>
  );
}

// 4. preventDefault: stop the browser's default action (a form submit reloads the page).
function Signup() {
  return (
    <form onSubmit={(e) => { e.preventDefault(); console.log('  submitted without reload; defaultPrevented =',
e.defaultPrevented); }}>
      <input aria-label="email" />
      <button>Send</button>
    </form>
  );
}
```

Output:

```
rendering <Wrong/>:
  Wrong: ran during RENDER, not on click
clicking wrong:
clicking right:
  Right: clicked
  Playing Kiki
  Uploading... event type = click
click "bubble":
    child <button> onClick
    parent <div> onClick
click "stop":
    child <button> onClick
  submitted without reload; defaultPrevented = true
```

What the output proves:

- **`onClick={handleClick()}` is a bug**: the parentheses call the function *during rendering*, and the button receives its return value (`undefined`), so clicking does nothing. If the function sets state, you get an infinite loop ("Too many re-renders"). Pass `handleClick`, or wrap in an arrow when you need arguments: `onClick={() => onPlay('Kiki')}`.
- **Custom components can define their own event props** (`onPlay`, `onUpload`). They are just function props; the name `on…` is a convention. The child decides *when* to call them; the parent decides *what* happens. This is how data flows **up** the tree.

- **Bubbling (propagation)**: the click fired on the button, then on its parent `<div>`. `e.stopPropagation()` stopped it at the button. (All React events bubble except `onScroll`. To catch an event on the way *down* before children see it, use the capture version, e.g. `onClickCapture`.)
- `e.preventDefault()` stops the browser's built-in behaviour — here, the full page reload a form submit would cause. It does *not* stop bubbling; that's `stopPropagation`.

React's event object

React passes a **SyntheticEvent**: a wrapper around the browser's native event with the same interface (`target`, `currentTarget`, `preventDefault` …) that behaves consistently across browsers; `e.nativeEvent` gives the original. React attaches one listener per event type at the root container and dispatches to your handlers (event delegation) — you never call `addEventListener` for normal UI events.

Common mistakes

Mistake	Fix
`onClick={handle()}`	`onClick={handle}` or `onClick={() => handle(id)}`
`onclick` (lowercase)	`onClick` — React warns about the unknown prop
Putting handlers on a `<div>` for clickable things	Use `<button>` — keyboard and screen-reader support for free
Using `stopPropagation` to fix a design problem	Usually a sign the parent should get an explicit callback instead
Forgetting `preventDefault` on `onSubmit`	The page reloads and state is lost

In the interview

30-second answer: "Handlers are functions passed as `on…` props — pass the function, don't call it. React gives them a SyntheticEvent that mirrors the native one. Events bubble, so `stopPropagation` stops parents from seeing them, and `preventDefault` stops browser defaults like form submission. Components expose their own `onSomething` props so parents decide what happens."

- *Can handlers have side effects?* — Yes: handlers are *the* place for side effects (set state, fetch, log). Only rendering must be pure.
- *Difference between `e.target` and `e.currentTarget`?* — `target` is the element where the event started; `currentTarget` is the element whose handler is running.

TL;DR

`onEvent={fn}` — pass, don't call. Arrow to pass arguments. Custom `onX` props send events up. `stopPropagation` = stop bubbling; `preventDefault` = stop browser default.

3.2 State: a component's memory — `useState`

What it is

State is data a component remembers between renders and that changes over time because of interaction: the current input text, which tab is open, the items in a cart. You add state with the **`useState` Hook**:

```
const [count, setCount] = useState(0);
//     value  setter          initial value (used on the first render only)
```

`useState` returns a pair (array destructuring gives them names): the **current value** and a **setter function**. Calling the setter (1) stores the new value and (2) asks React to **re-render** the component.

Analogy: a whiteboard in a meeting room. A normal local variable is a note written on your hand — gone when you leave (the function returns). State is written on the room's whiteboard: React keeps it for you, and when you rewrite it, everyone in the room (the UI) sees the new value.

The problem it solves

`code/01_local_variable_vs_state.jsx`:

```
function BrokenCounter() {
  let count = 0;                          // re-created as 0 on every render
  renders++;
  return <button onClick={() => { count = count + 1; console.log('  inside handler count =', count); }}>Broken: {count}
</button>;
}

function Counter() {
  const [count, setCount] = useState(0);  // React keeps this value between renders
  return <button onClick={() => setCount(count + 1)}>Working: {count}</button>;
}
// click each twice; then render two <Counter />s and click only the first
```

Output:

```
  inside handler count = 1
  inside handler count = 2
screen shows: Broken: 0 | renders: 1
screen shows: Working: 2
two instances: Working: 1 | Working: 0
```

The broken version fails for two reasons: (1) changing a local variable does **not trigger a render** (renders stayed at 1), and (2) even if React re-rendered, `let count = 0` would reset it. `useState` fixes both: it **persists** the value and the setter **triggers** a render. The last line shows that **state is local to each instance**: two `<Counter />`s have independent counts.

How it works

1. First render: `useState(0)` returns `[0, setCount]`; React stores `0` in its internal record for *this component instance at this position in the tree*.
2. Click → `setCount(1)` → React schedules a re-render.
3. Second render: `useState(0)` ignores the `0` and returns `[1, setCount]`.
4. React diffs and commits the new text.

React matches `useState` calls to stored values **by call order**. That is why Hooks have rules: call them only at the **top level** of a component (or custom Hook) — never inside `if`, loops, or nested functions (Chapter 7 explains why in detail).

Details worth knowing

- **Several state variables** are fine: `const [name, setName] = useState(''); const [age, setAge] = useState(0);` Group values that always change together into one object.
- **Lazy initial state**: `useState(createInitialTodos)` (pass the function, not its result) runs an expensive initialiser only on the first render. `useState(createInitialTodos())` would call it every render and throw the result away.
- **Setter identity is stable**: `setCount` is the same function across renders — safe to pass down.
- **Same value → no re-render**: if the new value is identical to the current one (compared with `Object.is`), React skips re-rendering the component and its children (see §3.6).
- Naming: `[thing, setThing]` by convention.

When not to use state

Value	Where it belongs
Never changes	A constant outside the component
Comes from the parent	Props (don't copy into state — §3.8)
Can be computed from props/state	A variable in render (§3.8)
Must persist but must **not** re-render when it changes (timer ID, DOM node)	`useRef` (Chapter 5)
Shared by many distant components	Lift up (§3.7), then context / a store (Chapter 6)
Server data (lists from an API)	A data library such as TanStack Query (Chapters 6, 9)

In the interview

30-second answer: "`useState` gives a component memory. It returns the current value and a setter; calling the setter stores the new value and schedules a re-render. Local variables don't work because they reset each render and changing them doesn't trigger one. State is private to each component instance and identified by the Hook's call order, which is why Hooks must be called unconditionally at the top level."

- *What does `useState(fn)` do vs `useState(fn())`?* — Passing `fn` lets React call it once for the initial value; `fn()` runs every render.
- *Is state shared between two instances?* — No; each instance has its own.

Self-check

1. Why does `let count` not work? → No re-render is triggered and it resets every render.
2. When is the argument of `useState(x)` used? → Only on the first render.
3. Can you call `useState` inside an `if`? → No — Hook order must be the same every render.

TL;DR

`const [v, setV] = useState(initial)`. State persists between renders, is private per instance, and the setter triggers a re-render. Top-level calls only.

3.3 State as a snapshot

What it is

When React renders your component, it hands you the state values **for that render**, like a photograph. Everything created during that render — the JSX *and the event handlers* — sees those fixed values. Calling a setter does not change the variable you already have; it requests a *new* render with a new snapshot.

How it works

`code/03_state_as_snapshot.jsx`:

```
function Counter() {
  const [number, setNumber] = useState(0);
  return (
    <>
      <h1>{number}</h1>
      <button onClick={() => {
        setNumber(number + 1);   // setNumber(0 + 1)
        setNumber(number + 1);   // setNumber(0 + 1)
        setNumber(number + 1);   // setNumber(0 + 1)
        console.log('  right after the three calls, number =', number);
      }}>+3</button>
    </>
  );
}

// Even a delayed callback sees the snapshot from the render it was created in.
function Delayed() {
  const [n, setN] = useState(0);
  return (
    <p>
      n is <output>{n}</output>{' '}
      <button onClick={() => { setN(n + 5); setTimeout(() => console.log('  timeout fired, n =', n), 10); }}>+5
later</button>
    </p>
  );
}
```

Output:

```
  right after the three calls, number = 0
heading after one click: 1
  timeout fired, n = 0
screen shows n = 5 (the timeout logged the old snapshot)
```

Step by step for the "+3" button:

1. The render where `number` is 0 created the click handler. Inside it, `number` is the constant `0`.
2. `setNumber(number + 1)` three times = `setNumber(1)` three times.
3. `console.log` still sees `0` — the setter does not mutate the variable.
4. React re-renders once with `number = 1`.

The timeout shows that the snapshot is permanent: even 10 ms later, after the screen shows 5, the closure still sees `n = 0` from the render that created it. This follows directly from JavaScript **closures** (a function remembers the variables in scope where it was created — see the JavaScript book, `../../../javascript/chapters/04-functions-scope-and-closures/README.md`).

Why React works this way

Snapshots make handlers predictable: if a user clicks "Send to Alice", then quickly switches the recipient to Bob, the message still goes to Alice — the handler captured the recipient at click time. No surprises from values changing mid-handler.

Common mistakes

- Reading state right after setting it and expecting the new value → use the value you computed: `const next = number + 1; setNumber(next); send(next);`.
- Expecting three `setX(x + 1)` calls to add 3 → use updater functions (§3.5).
- **Stale closures**: a timer or subscription created once keeps reading an old snapshot. Use an updater function, or refs/Effects (Chapter 5).

TL;DR

State inside a render is a constant snapshot. Setters request a new render; they never change the variable you are holding.

3.4 Batching

What it is

Batching means React collects all state updates made during one event and performs **a single re-render** after the handler finishes, instead of one render per `setX` call.

Analogy: a waiter takes the whole table's order before going to the kitchen, rather than walking to the kitchen after each person speaks.

Why it matters

- **Performance**: five setters in one click = one render, not five.
- **Consistency**: the screen never shows a half-updated state (e.g. a new name with an old avatar).

Since **React 18**, batching is **automatic everywhere**: in React event handlers *and* inside `setTimeout`, promises, `fetch` callbacks and native event listeners. (Before React 18 only React event handlers were batched.) React also never batches *across* separate user events: two clicks are two renders.

How it works — counting renders

`code/04_batching_and_updaters.jsx` counts renders for several buttons (the full component is in §3.5):

Output:

```
replace x3       -> number =  1 | re-renders caused: 1
updater x3       -> number =  4 | re-renders caused: 1
+5 then n=>n+1   -> number = 10 | re-renders caused: 1
+5, n=>n+1, 42   -> number = 42 | re-renders caused: 1
in timeout       -> number = 43 | re-renders caused: 1
```

Every row caused exactly **one** re-render, even "replace x3" (four setter calls on two state variables) and "in timeout" (two setters inside `setTimeout`).

Opting out: in rare cases you must have the DOM updated synchronously before the next line (e.g. to scroll to a just-added item); `flushSync(() => setX(v))` from `react-dom` forces it. Use sparingly — it hurts performance.

TL;DR

All setter calls in one event → one re-render (everywhere since React 18). `flushSync` opts out.

3.5 Updater functions — queueing updates

What it is

Instead of a value, you can pass a setter a **function** that receives the **pending** state and returns the next one: `setNumber(n => n + 1)`. React puts it in a **queue** and, on the next render, applies the queue in order.

How it works

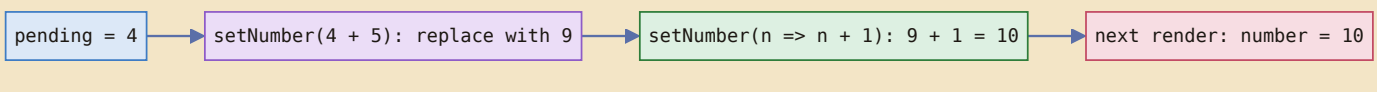
`code/04_batching_and_updaters.jsx`:

```jsx
function Counter() {
  const [number, setNumber] = useState(0);
  const [clicks, setClicks] = useState(0);
  renders++;
  return (
    <>
      <h1>{number}</h1>
      <button onClick={() => { setNumber(number + 1); setNumber(number + 1); setNumber(number + 1); setClicks(clicks + 1); }}>replace x3</button>
      <button onClick={() => { setNumber((n) => n + 1); setNumber((n) => n + 1); setNumber((n) => n + 1); }}>updater x3</button>
      <button onClick={() => { setNumber(number + 5); setNumber((n) => n + 1); }}>+5 then n=&gt;n+1</button>
      <button onClick={() => { setNumber(number + 5); setNumber((n) => n + 1); setNumber(42); }}>+5, n=&gt;n+1, 42</button>
      <button onClick={() => setTimeout(() => { setNumber((n) => n + 1); setClicks((c) => c + 1); }, 0)}>in timeout</button>
    </>
  );
}
```

(Output in §3.4: 1, 4, 10, 42, 43.)

Work through the queue exactly as React does. Each value in the queue means "replace with this"; each function means "compute from the previous result":

Button (starting number)	Queue	Processing	Result
replace x3 (0)	`1`, `1`, `1`	replace 1, replace 1, replace 1	**1**
updater x3 (1)	`n=>n+1` ×3	1→2→3→4	**4**
+5 then n=>n+1 (4)	`9`, `n=>n+1`	replace 9, then 9+1	**10**
+5, n=>n+1, 42 (10)	`15`, `n=>n+1`, `42`	15 → 16 → replace 42	**42**
in timeout (42)	`n=>n+1`	42+1	**43**



When to use which

Use a plain value `setX(v)`	Use an updater `setX(prev => …)`
The new value doesn't depend on the old one (`setName('Ada')`, `setOpen(false)`)	The new value is computed from the old one and several updates may queue (`+1` twice)
Readability: most setters	Inside timers/subscriptions created earlier (avoids stale snapshots)
	Handlers passed to memoised children (no dependency on current state)

Conventions: name the parameter after the state's first letter (`n`, `c`) or `prev`. Updaters must be **pure** — StrictMode calls them twice in development to check.

Common mistakes

- `setCount(count++)` — mutates the snapshot variable and passes the *old* value. Use `count + 1`.
- Side effects inside an updater (`setX(x => { fetch(…); return x + 1; })`) — it may run twice.

In the interview

> **30-second answer:** "Setters queue updates. A plain value replaces the state; an updater function receives the pending value, so three `n => n + 1` calls add 3 while three `setN(n + 1)` calls add 1 because they all read the same snapshot. React processes the queue during the next render, and all of it is batched into one render."

- *What is `setN(n + 5); setN(m => m + 1)` from 0?* — 6.
- *What about `setN(m => m + 1); setN(n + 5)` from 0?* — 5 (the last replace wins).

TL;DR

`setX(v)` replaces; `setX(prev => next)` computes from the queued value. Queue processed in order at the next render.

3.6 Updating objects and arrays immutably

What it is

State can hold objects and arrays, but you must treat them as **immutable** (read-only): to change one, create a **new** object/array with the change and pass that to the setter. **Mutation** means changing an existing object in place (`obj.x = 1`, `arr.push(…)`).

Analogy: editing a shared document. Instead of scribbling on the original (others may be reading it, and nobody can tell it changed), you make a new version with the edit and announce "here's v2".

Why it matters

1. **React only notices new references.** A setter compares old and new with `Object.is`. The same object — even if you changed its insides — looks "unchanged", so React may skip the render.
2. **Snapshots and history.** Old renders keep pointing at old objects; undo/redo and time-travel debugging work because previous versions are untouched.
3. **Optimisations** (`memo`, the React Compiler) rely on "new reference = changed".

Objects

`code/05_updating_objects.jsx`:

```jsx
const [person, setPerson] = useState({
  name: 'Niki de Saint Phalle',
  artwork: { title: 'Blue Nana', city: 'Hamburg' },
});
// ...
<button onClick={() => { person.name = 'MUTATED'; setPerson(person); }}>mutate</button>
<button onClick={() => setPerson({ ...person, name: 'Niki' })}>copy name</button>
<button onClick={() => setPerson({ ...person, artwork: { ...person.artwork, city: 'Berlin' } })}>copy nested
city</button>
```

Output:

```
initial           -> "Niki de Saint Phalle — Blue Nana (Hamburg)" | renders so far: 1
mutate + set same -> "Niki de Saint Phalle — Blue Nana (Hamburg)" | renders so far: 1
copy name         -> "Niki — Blue Nana (Hamburg)" | renders so far: 2
copy nested city  -> "Niki — Blue Nana (Berlin)" | renders so far: 3
```

- **mutate**: the object *was* changed, but `setPerson(person)` passed the same reference → no re-render → the screen is stale. (Worse: the change appears later at a random moment when something else re-renders.)
- **copy name**: `{ ...person, name }` — the **spread syntax** copies all top-level properties into a new object, then overrides `name`.
- **nested**: spread is *shallow* (one level). To change `artwork.city`, copy each level on the path: a new `person` containing a new `artwork`.

Arrays — the recipe table

`code/06_updating_arrays.jsx`:

```jsx
const add = () => setItems([...items, { id: nextId++, title: 'Aurora', seen: false }]);          // add to end
const addFirst = () => setItems([{ id: nextId++, title: 'Fjords', seen: false }, ...items]);     // add to start
const insertAt1 = () => setItems([...items.slice(0, 1), { id: nextId++, title: 'Glacier', seen: false },
...items.slice(1)]);
const remove = (id) => setItems(items.filter((it) => it.id !== id));                              // remove
const toggle = (id) => setItems(items.map((it) => (it.id === id ? { ...it, seen: !it.seen } : it))); // update one
const sortAZ = () => setItems(items.toSorted((a, b) => a.title.localeCompare(b.title)));          // sort a copy
const reverse = () => setItems([...items].reverse());                                             // copy, then
reverse
const immerToggle = (id) => setItems(produce((draft) => {                                         // Immer: "mutate" a
draft
  const it = draft.find((d) => d.id === id); it.seen = !it.seen;
}));
```

Output:

```
initial                -> [ ] Big Bellies, [ ] Lunar Landscape, [x] Terracotta Army
add                    -> [ ] Big Bellies, [ ] Lunar Landscape, [x] Terracotta Army, [ ] Aurora
add first              -> [ ] Fjords, [ ] Big Bellies, [ ] Lunar Landscape, [x] Terracotta Army, [ ] Aurora
insert at 1            -> [ ] Fjords, [ ] Glacier, [ ] Big Bellies, [ ] Lunar Landscape, [x] Terracotta Army, [ ]
Aurora
remove Lunar Landscape -> [ ] Fjords, [ ] Glacier, [ ] Big Bellies, [x] Terracotta Army, [ ] Aurora
toggle Big Bellies     -> [ ] Fjords, [ ] Glacier, [x] Big Bellies, [x] Terracotta Army, [ ] Aurora
immer toggle Aurora    -> [ ] Fjords, [ ] Glacier, [x] Big Bellies, [x] Terracotta Army, [x] Aurora
sort                   -> [x] Aurora, [x] Big Bellies, [ ] Fjords, [ ] Glacier, [x] Terracotta Army
reverse                -> [x] Terracotta Army, [ ] Glacier, [ ] Fjords, [x] Big Bellies, [x] Aurora
original array untouched: [ ] Big Bellies, [ ] Lunar Landscape, [x] Terracotta Army
```

Goal	Avoid (mutates)	Use (returns a new array)
Add	`push`, `unshift`	`[...arr, x]`, `[x, ...arr]`
Insert at i	`splice(i, 0, x)`	`[...arr.slice(0, i), x, ...arr.slice(i)]` or `arr.toSpliced(i, 0, x)`
Remove	`splice`, `pop`, `shift`	`arr.filter(item => item.id !== id)`
Replace / update one item	`arr[i] = x`, `item.done = true`	`arr.map(item => item.id === id ? { ...item, done: true } : item)` or `arr.with(i, x)`
Sort / reverse	`sort()`, `reverse()`	`toSorted()`, `toReversed()` (ES2023), or copy first: `[...arr].sort()`

The last line of the output confirms the original `initial` array was never modified.

Important: `[...arr]` copies the **array**, not the objects inside it. `const copy = [...items]; copy[0].seen = true` still mutates the shared object — that's why the toggle recipe also spreads the item (`{ ...it, seen: !it.seen }`).

Immer — write "mutations" safely

Deep copying by hand gets noisy. **Immer** gives you a **draft** — a proxy object that records your changes — and produces a new immutable state with structural sharing (unchanged parts are reused). Above, `produce(draft => { … })` is passed to the setter as an updater. In components, the `use-immer` package wraps this as `useImmer`:

```
// (use-immer — not run; same idea as produce above)
import { useImmer } from 'use-immer';
const [person, updatePerson] = useImmer({ name: 'Niki', artwork: { city: 'Hamburg' } });
updatePerson(draft => { draft.artwork.city = 'Berlin'; });
```

Redux Toolkit uses Immer internally (Chapter 6). Mutating the draft is fine; mutating real state is not.

Trade-offs

Manual spread	Immer
No dependency; explicit	Small extra dependency (a few KB)
Verbose for nested data	Reads like normal mutation, even deep
	Proxies: debugging drafts is slightly less direct

Also consider **flattening** deeply nested state (normalise by ID, like a database table) — updates get simpler with or without Immer.

Common mistakes

- `arr.push(x); setArr(arr)` → no re-render.
- `items.sort()` in render or before setting → mutates state.
- Copying only the outer level of nested data, then mutating inside.

In the interview

30-second answer: "State must be treated as immutable: create new objects and arrays instead of mutating. React decides whether something changed by reference (`Object.is`), so mutating and setting the same object may not re-render and breaks snapshots and memoisation. Use spread for objects, `map` / `filter` / `toSorted` / `with` for arrays, copy every level for nested updates, or use Immer to write mutation-style code on a draft."

- *Is `[...arr]` a deep copy?* — No, shallow: the elements are shared.
- *How does Immer stay efficient?* — Structural sharing: only changed paths get new objects.

Self-check

1. Why did "mutate" not update the screen? → Same reference passed; `Object.is` says unchanged → render skipped.
2. Update `user.address.zip` immutably? → `setUser({ ...user, address: { ...user.address, zip } })`.
3. Remove item with id 3? → `setItems(items.filter(i => i.id !== 3))`.

TL;DR

Never mutate state. New object: `{ ...obj, key }`; new array: spread, `filter`, `map`, `toSorted`, `with`, `toSpliced`. Copy each nested level, or use Immer.

3.7 Lifting state up

What it is

When two components must **share or coordinate** state, move the state to their **closest common parent** and pass it down as props, together with callbacks to change it. This is called **lifting state up**. The children become **controlled** (driven by props) instead of **uncontrolled** (driven by their own state).

Analogy: two siblings arguing over the TV remote. Give the remote to the parent; each child *asks* the parent to change the channel.

How it works — an accordion

`code/07_lifting_state_up.jsx`. Requirement: only one panel open at a time.

```
// Before: each Panel owns isActive -> both can be open at once.
function PanelLocal({ title, children }) {
  const [isActive, setIsActive] = useState(false);
  // ...
}

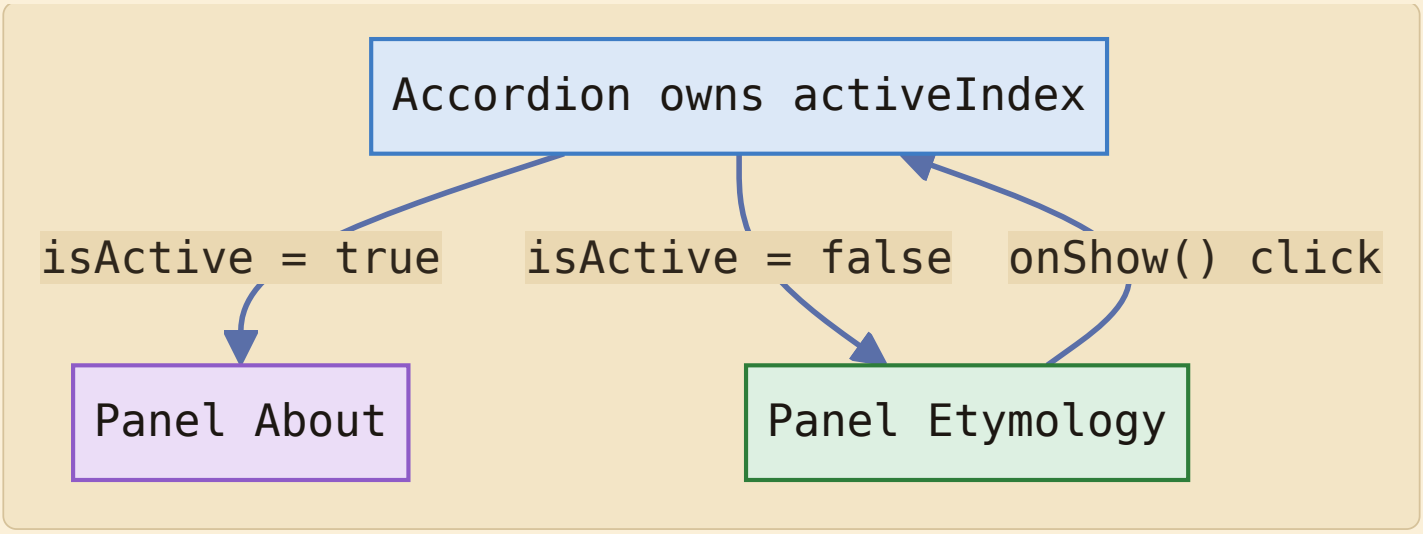
// After: Panel is controlled by props; the parent owns activeIndex.
function Panel({ title, isActive, onShow, children }) {
  return (
    <section>
      <h3>{title}</h3>
      {isActive ? <p>{children}</p> : <button onClick={onShow}>Show {title}</button>}
    </section>
  );
}
function Accordion() {
  const [activeIndex, setActiveIndex] = useState(0);
  return (
    <>
      <Panel title="About" isActive={activeIndex === 0} onShow={() => setActiveIndex(0)}>Almaty is Kazakhstan's largest
city.</Panel>
      <Panel title="Etymology" isActive={activeIndex === 1} onShow={() => setActiveIndex(1)}>The name comes from the
word for apple.</Panel>
    </>
  );
}
```

Output:

```
local state : open panels after clicking both = 2
lifted state: initially open = Almaty is Kazakhstan's largest city.
lifted state: after "Show Etymology" open panels = 1 -> The name comes from the word for apple.
```

The three steps:

1. **Remove** state from the children (`isActive` becomes a prop).
2. **Pass** hard-coded data from the common parent (`isActive={activeIndex === 0}`).
3. **Add** state to the parent (`activeIndex`) and pass a callback (`onShow`) so children can request changes.



Single source of truth

For every piece of state there should be **one owner** component. Others receive it via props. If you find the same data stored in two places and "kept in sync", lift it to one owner instead.

Trade-offs

Lift up	Keep local
Siblings stay in sync; parent can reset/control	Simpler; state changes re-render less of the tree
Parent re-renders on every change (usually fine)	Parent cannot coordinate or read it
Too high → prop drilling (then context/composition, Chapter 6 / Chapter 2 §2.4)	

Rule: keep state **as low as possible, as high as necessary**.

In the interview

30-second answer: "When siblings need the same state, move it to their closest common parent, pass the value down as props and a callback to change it. The children become controlled components. It gives one source of truth; if lifting goes too high and causes prop drilling, use composition or context."

TL;DR

Shared state → closest common parent. Pass value + `onChange`-style callback down. One owner per piece of state.

3.8 Derived state — choosing the minimal state

What it is

Derived state is any value you can **calculate** from props or other state. It should be a normal variable computed during render, **not** stored with `useState`. Keep state **minimal** and let everything else be derived.

Analogy: you store your date of birth, not your age. Age is derived — storing it means updating it every year and risking it being wrong.

How it works

`code/08_derived_state.jsx` shows three classic mistakes and fixes:

```
// 1. Redundant state: fullName can be computed from first + last.
const fullName = `${first} ${last}`;              // derived: no useState, no sync code

// 2. Duplicated state: store the selected ID, not a copy of the selected object.
const [selectedItem, setSelectedItem] = useState(items[0]);     // BAD: a copy
const [selectedId, setSelectedId] = useState(0);                // GOOD: a reference by ID
const selected = storeObject ? selectedItem : items.find((i) => i.id === selectedId);

// 3. Mirroring a prop in state ignores later prop changes.
function MessageMirror({ color }) { const [c] = useState(color); return <p style={{ color: c }}>mirror: {c}</p>; }
function MessageDirect({ color }) { return <p style={{ color }}>direct: {color}</p>; }
```

Output:

```
derived fullName   : Ticket for: Ada King
store object copy  : You picked: pretzels
store selected id  : You picked: PRETZELS (renamed)
parent changed to red -> mirror: blue | direct: red
```

- Storing a **copy** of the selected object meant renaming the item left the selection showing the stale name. Storing the **ID** and deriving the object always shows current data.
- **Mirroring props** (`useState(color)`) captures only the first value; when the parent changed `color` to red, the mirror stayed blue. Use the prop directly. (If you really want only the initial value, name the prop `initialColor` to make that explicit.)

Principles for structuring state

Principle	Example
Group related state that always changes together	`{ x, y }` position in one object
Avoid contradictions	Instead of `isSending` + `isSent` booleans (both true = impossible), one `status: 'typing' | 'sending' | 'sent'`
Avoid redundant state	Compute `fullName`, `total`, `filteredList`, `remaining` in render
Avoid duplication	Store IDs, look objects up
Avoid deep nesting	Normalise: `{ byId: {…}, ids: […] }`

"But isn't recalculating slow?" For typical lists (hundreds of items) filtering during render is microseconds. If a calculation is genuinely expensive, cache it with `useMemo` (Chapter 10) — still derived, never manually synced.

In the interview

30-second answer: "I keep the minimal state and derive everything else during render — full names, totals, filtered lists. Storing derived values creates two sources of truth that can drift. I store IDs instead of copied objects, avoid mirroring props into state, and replace contradictory booleans with a single status value. Expensive derivations can be memoised with `useMemo`."

- *Should I use `useEffect` to update derived state when inputs change?* — No; that causes an extra render and a moment of stale UI. Compute it in render (Chapter 5 "You might not need an Effect").

TL;DR

If it can be computed, compute it. Store IDs not copies, don't mirror props, prefer one status over several booleans.

3.9 Controlled inputs (basics)

What it is

A **controlled input** is a form field whose displayed value comes from React state: you pass `value={state}` and update the state in `onChange`. React state is the **single source of truth**. An **uncontrolled input** keeps its own value in the DOM (you may set a starting `defaultValue` and read it later) — covered with forms and actions in Chapter 4.

Element	Controlled prop	Read new value from
`<input type="text">`, `<textarea>`	`value`	`e.target.value`
`<select>`	`value` (on the `<select>`, not `selected` on options)	`e.target.value`
`<input type="checkbox">` / `radio`	`checked`	`e.target.checked`

Note: React's `onChange` fires on **every keystroke** (like the DOM `input` event), not only on blur as the native `change` event does.

How it works

`code/09_controlled_inputs.jsx`:

```
function Profile() {
  const [name, setName] = useState('');
  const [bio, setBio] = useState('');
  const [plan, setPlan] = useState('free');
  const [agree, setAgree] = useState(false);
  return (
    <form onSubmit={(e) => { e.preventDefault(); console.log('  submit ->', JSON.stringify({ name, bio, plan, agree
})); }}>
      <label>Username <input value={name} onChange={(e) => setName(e.target.value.toLowerCase().replace(/\s/g, ''))} />
</label>
      <label>Bio <textarea value={bio} onChange={(e) => setBio(e.target.value.slice(0, 20))} /></label>
      <label>Plan
        <select value={plan} onChange={(e) => setPlan(e.target.value)}>
          <option value="free">Free</option><option value="pro">Pro</option>
        </select>
      </label>
      <label><input type="checkbox" checked={agree} onChange={(e) => setAgree(e.target.checked)} /> I agree</label>
      <p>{name ? `@${name}` : 'pick a username'} · {bio.length}/20</p>
      <button disabled={!agree || !name}>Create</button>
    </form>
  );
}
// user types "Ada Love Lace" and a 28-character bio, picks Pro, then ticks the box and submits
```

Output:

```
username field shows: adalovelace
bio field shows     : First programmer, 18
status line         : @adalovelace · 20/20
Create disabled?    : true
  submit -> {"name":"adalovelace","bio":"First programmer, 18","plan":"pro","agree":true}

value without onChange -> field shows: fixed
warning: You provided a `value` prop to a form field without an `onChange` handler. This will render a read-only
field....
```

What control buys you:

1. **Transform input as it's typed** — lowercase, no spaces: "Ada Love Lace" became `adalovelace`.
2. **Enforce limits** — the bio stopped at 20 characters.
3. **Live derived UI** — the `@handle` preview, the counter, and the disabled button all react instantly.
4. **Submit** reads state directly — no DOM querying.

The last lines show the classic mistake: `value` without `onChange` makes a **read-only** field (typing did nothing) and React warns.

Common mistakes

Mistake	Symptom / fix
`value` without `onChange`	Read-only field + warning. Add `onChange`, or use `defaultValue` for uncontrolled, or `readOnly` if intended
`value={undefined}` first, then a string	Warning "changing an uncontrolled input to be controlled". Initialise with `''`, never `undefined` / `null`
Reading `e.target.value` for a checkbox	Use `e.target.checked`
Setting `selected` on `<option>`	Put `value` on `<select>`
One `useState` per field in a big form	Use one object + `[e.target.name]: e.target.value`, a reducer (Chapter 6), or a form library (Chapter 4)

Trade-offs

Controlled	Uncontrolled
Instant validation, formatting, dependent fields	Less code; the DOM holds the value
Re-renders on each keystroke (fine for most forms)	Read values on submit via `FormData` or refs
Easy to reset/prefill from state	React 19 form **Actions** make this style first-class (Chapter 4)

TL;DR

Controlled input = `value={state}` + `onChange={e => setState(e.target.value)}` (`checked`/`e.target.checked` for checkboxes). Initialise with `''`. Gives formatting, limits and live UI.

3.10 State is tied to a position in the tree

What it is

React stores state **not inside your component function** but in React's tree, attached to a **position** (the path of element types from the root). This explains when state is kept and when it resets:

Situation	State
Same component type at the same position on the next render	**Kept** (even if props differ)
Different type at that position, or the component is removed	**Destroyed**; a new instance starts from initial state
Same type and position but a **different** `key`	**Destroyed and recreated**

How it works

`code/10_state_and_position.jsx`:

```
function Counter({ person }) {
  const [score, setScore] = useState(0);
  return <p>{person}'s score: {score} <button onClick={() => setScore(score + 1)}>+1 {person}</button></p>;
}

function Scoreboard({ mode }) {
  const [isTaylor, setIsTaylor] = useState(true);
  let counter;
  if (mode === 'same-position') counter = isTaylor ? <Counter person="Taylor" /> : <Counter person="Sarah" />;
  if (mode === 'with-key') counter = isTaylor ? <Counter key="Taylor" person="Taylor" /> : <Counter key="Sarah"
person="Sarah" />;
  return <div>{counter}<button onClick={() => setIsTaylor(!isTaylor)}>Next player</button></div>;
}
// Taylor scores 2, switch to Sarah, switch back
```

Output:

```
same-position: Taylor 2 -> switch -> Sarah's score: 2  -> switch back -> Taylor's score: 2
with-key     : Taylor 2 -> switch -> Sarah's score: 0  -> switch back -> Taylor's score: 0
```

- **Without keys** both branches are `<Counter>` at the same position, so React treats them as *the same component with a new* `person` *prop* — Sarah "inherits" Taylor's score. Usually a bug.
- **With keys** they are different identities: switching destroys one counter and creates the other fresh. (To *remember* each player's score while hidden, lift the scores up into the parent, §3.7.)

Real-world use of `key` to reset

- A chat app: `<Chat key={contact.id} contact={contact} />` — switching contact clears the draft message instead of sending your text to the wrong person.
- An edit form: `<EditProfile key={userId} />` resets all fields when a different user is selected — no Effect needed.

Common mistakes

- Defining a component inside another component (new type every render → state reset every render; Chapter 1 §1.7).
- Expecting different props to reset state — they don't; use a `key`.

TL;DR

State lives at a tree position. Same type + same position = kept; different type or different key = reset. Use `key` to reset on purpose.

3.11 Putting it together — a to-do app

`code/todo.jsx` uses every idea from this chapter: a controlled input with `preventDefault` on submit, custom event props (`onAdd`, `onToggle`, `onDelete`), state lifted into `TodoApp`, immutable updates with updater functions, and derived `visible`/`remaining` values.

```jsx
function AddTodo({ onAdd }) {
  const [text, setText] = useState('');                    // controlled input
  function handleSubmit(e) {
    e.preventDefault();                                    // no page reload
    const title = text.trim();
    if (!title) return;
    onAdd(title);                                          // tell the parent
    setText('');
  }
  return (
    <form className="add" onSubmit={handleSubmit}>
      <input aria-label="New task" placeholder="What needs doing?" value={text} onChange={(e) =>
setText(e.target.value)} />
      <button disabled={!text.trim()}>Add</button>
    </form>
  );
}

function TodoItem({ todo, onToggle, onDelete }) {
  return (
    <li className={todo.done ? 'done' : ''}>
      <label>
        <input type="checkbox" checked={todo.done} onChange={() => onToggle(todo.id)} /> {todo.title}
      </label>
      <button aria-label={`Delete ${todo.title}`} onClick={() => onDelete(todo.id)}>×</button>
    </li>
  );
}

const FILTERS = { all: () => true, active: (t) => !t.done, done: (t) => t.done };
// FilterBar renders one button per filter with aria-pressed ...

export default function TodoApp({ initialTodos = [] }) {
  const [todos, setTodos] = useState(initialTodos);          // lifted state: shared by all children
  const [filter, setFilter] = useState('all');

  const visible = todos.filter(FILTERS[filter]);             // derived, not stored
  const remaining = todos.filter((t) => !t.done).length;     // derived, not stored

  const add = (title) => setTodos((ts) => [...ts, { id: nextId++, title, done: false }]);
  const toggle = (id) => setTodos((ts) => ts.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  const remove = (id) => setTodos((ts) => ts.filter((t) => t.id !== id));
  const clearDone = () => setTodos((ts) => ts.filter((t) => !t.done));

  return (
    <div className="todo">
      <h1>Today</h1>
      <AddTodo onAdd={add} />
      <FilterBar filter={filter} onChange={setFilter} />
      {visible.length === 0 ? <p className="empty">Nothing here.</p> : (
        <ul>{visible.map((t) => <TodoItem key={t.id} todo={t} onToggle={toggle} onDelete={remove} />)}</ul>
      )}
      <footer>
        <span>{remaining} {remaining === 1 ? 'task' : 'tasks'} left</span>
        {todos.length > remaining && <button onClick={clearDone}>Clear completed</button>}
      </footer>
    </div>
  );
}
```

`code/11_todo_app.jsx` drives it like a real user with `user-event` — typing, pressing Enter, clicking checkboxes and filter buttons:

```
await user.type(screen.getByLabelText('New task'), 'Build the todo app{Enter}');
await user.type(screen.getByLabelText('New task'), 'Explain batching');
await user.click(screen.getByRole('button', { name: 'Add' }));
await user.click(screen.getByLabelText('Build the todo app'));
await user.click(screen.getByRole('button', { name: 'active' }));
// ... done filter, clear completed, delete
```

Output (label | visible items | footer):

```
start                      | [x] Read chapter 3                                                    | 0 tasks left
added two                  | [x] Read chapter 3, [ ] Build the todo app, [ ] Explain batching | 2 tasks left
input cleared after add?   : true
ticked "Build the todo app" | [x] Read chapter 3, [x] Build the todo app, [ ] Explain batching | 1 task left
filter = active            | [ ] Explain batching                                                  | 1 task left
filter = done              | [x] Read chapter 3, [x] Build the todo app                           | 1 task left
clear completed            | Nothing here.                                                         | 1 task left
filter = all, delete last  | Nothing here.                                                         | 0 tasks left
```

In the browser (`code/todo.html`, styles in `todo.css`):

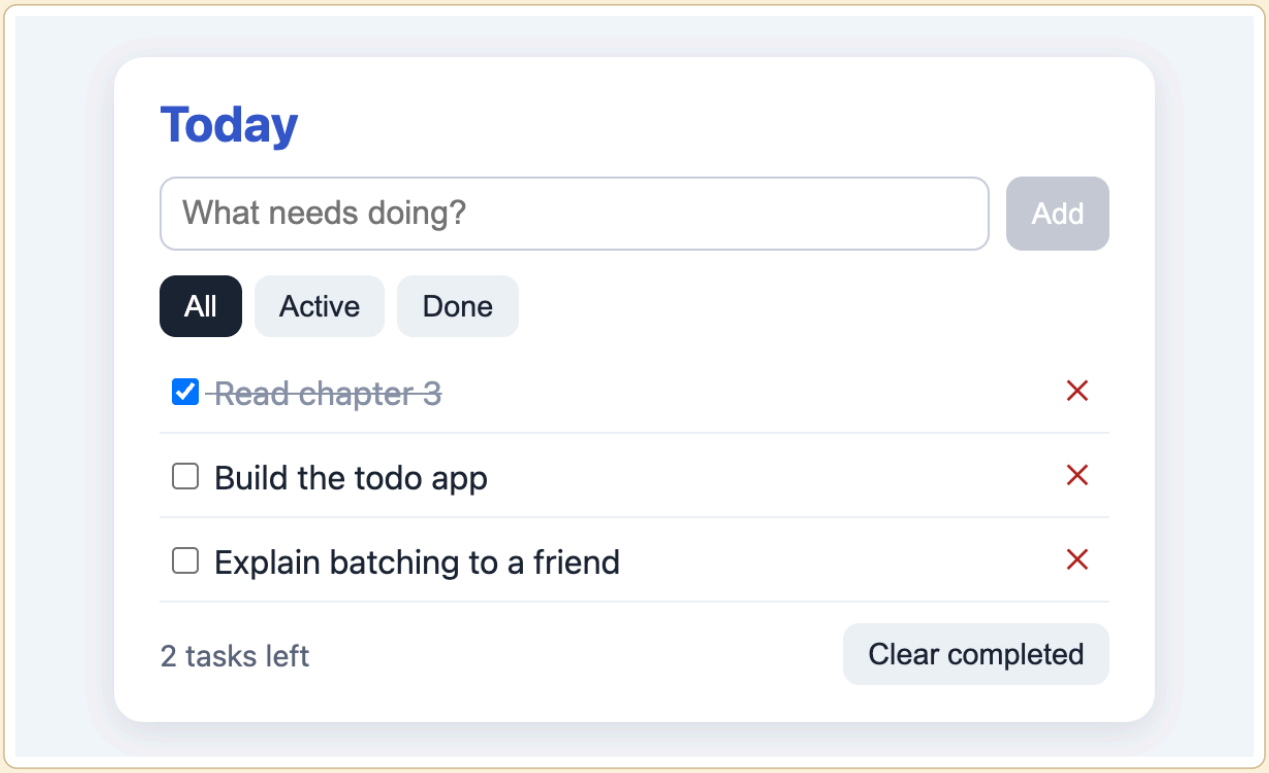


Figure 3.1 — `TodoApp` in Chrome. The struck-through item, the "2 tasks left" count and the "Clear completed" button are all derived from the one `todos` array.

Design notes:

- **One owner**: only `TodoApp` holds `todos`; children get data + callbacks.
- **Minimal state**: `todos` and `filter`; `visible`, `remaining`, and whether "Clear completed" shows are derived.
- **Updater functions** in `add`/`toggle`/`remove` make each safe even if called several times in one event.
- **Accessible queries**: the test finds elements by label and role, which is also how screen-reader users find them (Chapter 12).

When this grows (editing, undo, persistence), the update logic moves into a **reducer** (`useReducer`, Chapter 6) and saving to `localStorage` needs an **Effect** (Chapter 5).

Chapter 3 TL;DR

Idea	Remember
Events	`onClick={handle}` — pass, don't call; `e.stopPropagation()`, `e.preventDefault()`; custom `onX` props send events up
`useState`	`[value, setValue]`; persists between renders, per instance; top-level calls only
Snapshot	Inside a render state is constant; setters request a new render
Batching	All updates in one event → one render (everywhere since React 18)
Updaters	`setX(prev => …)` to build on queued updates
Immutability	New objects/arrays: spread, `map`, `filter`, `toSorted`, `with`; Immer for deep updates
Lifting state	Shared state goes to the closest common parent; one owner
Derived state	Compute, don't store; store IDs, don't mirror props
Controlled inputs	`value` + `onChange` (`checked` for checkboxes); start with `''`
Position & key	Same type + position keeps state; a new `key` resets it

Next: Chapter 4 — Forms and Actions.

Chapter 4 — Forms and Actions

Almost every real app has forms: sign-up, search, checkout, settings, comments. Chapter 3 §3.9 showed the basic **controlled input** (`value` + `onChange`). This chapter covers the whole topic: the two ways to own a field's value, validation that is helpful and accessible, and React 19's **Actions** — a built-in way to submit forms that tracks "pending", keeps the last result, resets the form, and shows optimistic updates, with far less code than the classic `onSubmit` + several `useState` calls.

What you'll learn

#	Topic	You will be able to…
4.1	How a form submission works	name fields, read them with `FormData`, know what the browser does by default
4.2	Controlled vs uncontrolled inputs	choose who owns a field's value, use `defaultValue`, refs and `FormData`
4.3	Validation	combine built-in HTML constraints with your own rules; show errors at the right time, accessibly
4.4	Form Actions	pass a function to `<form action>`; use `formAction`; understand the automatic reset
4.5	`useActionState`	keep the result of the last submission (errors / success) and a pending flag
4.6	`useFormStatus`	build a submit button that knows its form is submitting
4.7	`useOptimistic`	show the expected result instantly and roll back on failure
4.8	Server Functions and progressive enhancement	know what `'use server'` adds (overview)
4.9	Form libraries	when React Hook Form + Zod pay off, and what the code looks like
4.10	Putting it together	an RSVP form using every idea

Code: `code/` — React 19.3 in jsdom, driven by `@testing-library/user-event`. Two small notes for anyone reusing the tests: each program sets `globalThis.FormData = window.FormData` (Node has its own `FormData` that cannot read a `<form>` element), and after React resets a form, `user.clear()` is called before typing again (user-event keeps its own copy of the field's text and doesn't notice the reset).

4.1 How a form submission works

What it is

A **form** (`<form>`) groups input fields. When the user presses a submit button or hits Enter in a text field, the browser fires a `submit` **event** and then, by default, *navigates*: it sends the field values to the URL in the form's `action` attribute and loads a new page. (The HTML side — every input type, labels, fieldsets — is in the HTML & CSS book, `../../../html-css/chapters/05-tables-and-forms/README.md`.)

Only fields with a `name` attribute are sent. The data is a list of **name/value pairs**, which JavaScript exposes as a `FormData` object:

`FormData` method	Returns
`fd.get('email')`	The first value for that name (a string, or a `File` for file inputs), or `null`
`fd.getAll('topic')`	Every value for that name — for several checkboxes sharing one name
`Object.fromEntries(fd)`	A plain object `{ name: value }` (keeps only the last value of repeated names)
`new FormData(formElement)`	Builds the pairs from a form's current field values

Rules worth memorising: an **unchecked** checkbox is **not included at all** (not `false`); a checked checkbox without a `value` sends `"on"`; every value is a **string** (convert numbers yourself); disabled fields are skipped.

In a single-page app

A **single-page app** (SPA — Chapter 1 §1.4) must not reload the page, or all React state is lost. There are two ways to stop that:

Classic (any React version)	React 19 Actions
`<form onSubmit={e => { e.preventDefault(); … }}>`	`<form action={fn}>`
You call `preventDefault()` and read values yourself	React prevents the navigation and calls `fn(formData)`
You track "submitting" and errors with `useState`	Pending state, result and reset are built in (§4.4–§4.7)

Both are fine and both appear in real code bases; this chapter shows when each fits.

TL;DR

Fields need `name` to be submitted. `FormData` reads them; unchecked boxes are missing and all values are strings. In React either `preventDefault()` in `onSubmit` or pass a function to `action`.

4.2 Controlled vs uncontrolled inputs

What it is

Every input has a current value. The question is **who owns it**:

- **Controlled** — React state owns it. You pass `value={state}` (or `checked` for checkboxes/radios) and update state in `onChange`. The DOM always shows exactly what state says. (Chapter 3 §3.9.)
- **Uncontrolled** — the DOM owns it, as in plain HTML. You may give a **starting** value with `defaultValue` (or `defaultChecked`), and you read the value **when you need it**: from `FormData` on submit, or through a **ref** (a handle to the DOM element — Chapter 5 §5.9).

Analogy: a controlled input is a puppet — it moves only when React pulls the strings. An uncontrolled input is a person you ask "what did you write?" when the time comes.

How it works

`code/01_controlled_vs_uncontrolled.jsx`:

```jsx
// 1. Uncontrolled: the DOM keeps the value. defaultValue/defaultChecked set only the START value.
function UncontrolledSignup() {
  renders++;
  function handleSubmit(e) {
    e.preventDefault();
    const data = new FormData(e.currentTarget);          // read every named field at once
    console.log('  submit ->', JSON.stringify(Object.fromEntries(data)));
  }
  return (
    <form onSubmit={handleSubmit}>
      <input name="city" aria-label="City" defaultValue="Oslo" />
      <select name="size" aria-label="Size" defaultValue="m">…</select>
      <label><input type="checkbox" name="gift" defaultChecked /> Gift wrap</label>
      <button>Order</button>
    </form>
  );
}

// 2. Uncontrolled + ref: read one field on demand without re-rendering.
function SearchBox() {
  const inputRef = useRef(null);
  return (
    <>
      <input ref={inputRef} aria-label="Search" />
      <button onClick={() => console.log('  ref reads:', inputRef.current.value)}>Go</button>
    </>
  );
}

// 3. Controlled: React state owns the value, so every keystroke re-renders.
function ControlledCity() {
  const [city, setCity] = useState('Oslo');
  renders++;
  return <input aria-label="Controlled city" value={city} onChange={(e) => setCity(e.target.value.toUpperCase())} />;
}

// 4. The classic warning: value starts undefined (uncontrolled) and later becomes a string (controlled).
function Flip() {
  const [v, setV] = useState();                          // undefined!
  return <input aria-label="Flip" value={v} onChange={(e) => setV(e.target.value)} />;
}
// user: replaces "Oslo" with "Bergen", picks L, unticks Gift wrap, clicks Order; types "hooks" + Go;
// types "ville" into the controlled field; types "a" into Flip
```

Output:

```
  submit -> {"city":"Bergen","size":"l"}
uncontrolled renders while typing 6 letters: 1
  ref reads: hooks
controlled shows: OSLOVILLE | renders: 6
warning: A component is changing an uncontrolled input to be controlled. This is likely caused by t...
```

What the output shows:

1. **Uncontrolled forms render once.** Typing "Bergen" caused no re-renders — the browser handled it. `FormData` collected everything at submit time.
2. **The unticked checkbox vanished** from the submitted data (`gift` is missing), exactly as §4.1 said.
3. **A ref** reads a single field on demand.
4. **Controlled inputs re-render on every keystroke** (1 initial + 5 letters = 6) — the price of being able to transform the text (upper-casing here) as it is typed.
5. **Switching modes** — `value` going from `undefined` to a string — triggers React's warning. Initialise controlled state with `''`.

`defaultValue` is only a starting value

`defaultValue` is read when the element is created; afterwards the DOM value belongs to the user. Two consequences:

- To **reset** an uncontrolled field to new defaults, remount it with a new `key` (Chapter 3 §3.10) or call `form.reset()` (which restores the *default* values).
- For `<select>`, React **ignores a changed `defaultValue` after mount** (it only applies it on creation). §4.10 hits this and uses `key` to remount the select.

Which elements must be uncontrolled?

`<input type="file">` is **always uncontrolled** — for security, JavaScript cannot set the file a user picks. Read `input.files` or `formData.get('avatar')` (a `File` object).

Trade-offs

	Controlled	Uncontrolled
Who owns the value	React state	The DOM
Re-renders	Every keystroke	None while typing
Instant formatting / limits (`toUpperCase`, max length)	Easy	Awkward
Live dependent UI (character counter, enable button)	Easy	Needs extra events
Code for a big form	One state + handler per field (or one object)	Just `name` attributes
Works with React 19 form Actions and auto-reset	Values come from state, not reset	Natural fit
Prefill / reset from data	Set state	`defaultValue` + `key` or `form.reset()`

When to use which

- **Uncontrolled + `FormData`** (usually with a form Action): most "fill in and submit" forms — sign-up, contact, settings.
- **Controlled**: when the UI must react to every keystroke — live search, input masks (credit-card spacing), a counter, fields that depend on each other, a wizard that keeps values across steps.
- Mixing is fine: one controlled search box in an otherwise uncontrolled form.

Common mistakes

Mistake	Fix
`value` without `onChange` (read-only field)	Add `onChange`, or use `defaultValue`
`useState()` / `useState(null)` for a text field	`useState('')`
Changing `defaultValue` and expecting the field to update	Remount with `key`, or make it controlled
Forgetting `name` on uncontrolled fields	`FormData` silently skips them
Treating `formData.get('age')` as a number	`Number(formData.get('age'))`

In the interview

30-second answer: "A controlled input's value lives in React state — `value` plus `onChange` — so React can validate or format on every keystroke, at the cost of a render per keystroke. An uncontrolled input keeps its value in the DOM; you set a starting `defaultValue` and read it on submit with `FormData` or a ref. File inputs are always uncontrolled. In React 19 I default to uncontrolled fields with a form Action and use controlled fields where the UI must react while typing."

- *What causes 'changing an uncontrolled input to be controlled'?* — `value` was `undefined`/`null` on one render and a string on a later one.
- *How do you reset an uncontrolled form?* — `form.reset()` (a successful form Action does it automatically) or change the form's `key`.

Self-check

1. Where does an unchecked checkbox appear in `FormData`? → Nowhere; the name is absent.
2. Why did the uncontrolled form render only once? → The browser updates the field; no React state changed.
3. How do you read a `<input type="file">`? → It's uncontrolled: `input.files` or `formData.get(name)`.

TL;DR

Controlled = React owns the value (`value` + `onChange`), re-renders per keystroke, good for live behaviour. Uncontrolled = DOM owns it (`defaultValue`, read with `FormData`/ref), less code, pairs with Actions. File inputs are always uncontrolled.

4.3 Validation

What it is

Validation checks that input is acceptable before you act on it: required fields filled, email looks like an email, password long enough, two fields match. Good validation tells the user **what** is wrong, **where**, and **how to fix it** — at a moment that helps rather than nags.

There are three layers, and real apps use all of them:

Layer	Where it runs	Purpose
Built-in HTML constraints (`required`, `type="email"`, `minLength`, `pattern`, `min`/`max`)	Browser	Free, instant, works without JavaScript
Client-side rules in React	Browser	Custom messages, cross-field rules ("passwords match"), consistent styling
Server-side validation	Server	The only one you can **trust** — anyone can bypass the browser

Client validation is for **user experience**; server validation is for **correctness and security**. Never skip the server.

How built-in validation works

The browser checks each field's constraints; if any fails, it blocks the submit event and shows its own bubble. Each field exposes a `validity` object (`valueMissing`, `typeMismatch`, `tooShort`, `patternMismatch`, `rangeOverflow`, …, and the summary `valid`) and `validationMessage`. Adding `noValidate` to the `<form>` turns the browser's blocking and bubbles off so your own UI takes over — while `validity` still works.

How custom validation works

1. Write **one pure** `validate(values)` **function** that returns an `errors` object (`{ email: 'Enter a valid email' }`). Pure means it's easy to test and to reuse on the server.
2. Compute `errors` **during render** from the current values — it's derived state (Chapter 3 §3.8), so never store it.
3. Decide **when to show** each error. Showing "Email is invalid" after the first keystroke is hostile. A widely used rule: show a field's error once the user has **left** it (**blur**, tracked in a `touched` object) or after the first **submit** attempt; then update it live as they fix it.
4. On submit with errors, **move focus to the first invalid field** so keyboard and screen-reader users land where the problem is.
5. Make errors **accessible**: `aria-invalid="true"` on the field, the message linked with `aria-describedby`, and `role="alert"` so it's announced (details in the HTML & CSS book, `../../../html-css/chapters/03-semantic-html-and-accessibility/README.md`).

`code/02_validation.jsx`:

```jsx
// 1. Built-in constraint validation: the browser blocks submit while a field is invalid.
function Native() {
  return (
    <form onSubmit={(e) => { e.preventDefault(); console.log('  native form submitted'); }}>
      <input aria-label="Nick" name="nick" required minLength={3} pattern="[a-z]+" />
      <button>Save</button>
    </form>
  );
}

// 2. Custom validation in React: one pure validate() function, errors shown after blur or submit.
function validate({ email, password, confirm }) {
  const errors = {};
  if (!email) errors.email = 'Email is required';
  else if (!/^\S+@\S+\.\S+$/.test(email)) errors.email = 'Enter a valid email';
  if (password.length < 8) errors.password = 'Use at least 8 characters';
  if (confirm !== password) errors.confirm = 'Passwords do not match';   // cross-field rule
  return errors;
}

function Field({ name, label, type = 'text', value, error, show, onChange, onBlur }) {
  const errorId = `${name}-error`;
  return (
    <div>
      <label htmlFor={name}>{label}</label>
      <input id={name} name={name} type={type} value={value} onChange={onChange} onBlur={onBlur}
        aria-invalid={show && !!error} aria-describedby={show && error ? errorId : undefined} />
      {show && error && <p id={errorId} role="alert">{error}</p>}
    </div>
  );
}

function SignupForm() {
  const [values, setValues] = useState({ email: '', password: '', confirm: '' });
  const [touched, setTouched] = useState({});          // which fields the user has left
  const [submitted, setSubmitted] = useState(false);
  const errors = validate(values);                      // derived — never stored
  const show = (name) => submitted || touched[name];

  const handleChange = (e) => setValues({ ...values, [e.target.name]: e.target.value });
  const handleBlur = (e) => setTouched({ ...touched, [e.target.name]: true });
  function handleSubmit(e) {
    e.preventDefault();
    setSubmitted(true);
    const first = Object.keys(errors)[0];
    if (first) { document.getElementById(first).focus(); return; }   // move focus to the first problem
    console.log('  sending', values.email);
  }
  // ... <form noValidate onSubmit={handleSubmit}> with three <Field>s and a button
}
```

Output:

```
native: "Al" blocked | valid = false | patternMismatch = true
  native form submitted
while typing (not yet touched): []
after leaving Email           : [ 'Enter a valid email' ]
after submit                  : [ 'Enter a valid email', 'Use at least 8 characters' ]
focused field                 : email | aria-invalid = true
after fixing                  : []
  sending ada@example.com
```

Reading it:

- **Native**: "Al" broke `pattern="[a-z]+"` (capital A), so the browser refused to submit; after typing "alan" it submitted.
- **Custom**: nothing was shown while the user was still typing the email; leaving the field revealed its error; submitting revealed *all* errors and moved focus to the first; fixing the values cleared them live and the next

submit went through.

- One `handleChange` serves every field thanks to the **computed property name** `[e.target.name]`.

Trade-offs — when to show errors

Strategy	Feels	Use for
On every change from the start	Nagging ("invalid" after one letter)	Only for positive feedback like password-strength meters
On blur (touched)	Balanced	Most fields
On submit only	Calm but late	Short forms (login)
On blur, then live while fixing	Best of both	Default recommendation
Async (server) checks such as "username taken"	Needs debouncing and pending UI	Uniqueness checks — or just return the error from the submit Action (§4.5)

Common mistakes

- Trusting client validation — attackers call your API directly. Validate on the server too.
- Storing `errors` in state and "syncing" it in an Effect — compute it (Chapter 5 §5.6).
- Showing errors only by colour — also use text, `aria-invalid` and `aria-describedby`.
- Disabling the submit button until the form is valid — users can't discover *why* it's disabled. Prefer allowing submit and showing errors.
- Over-strict regexes for emails and names; keep client checks simple and let the server (or a confirmation email) decide.

In the interview

30-second answer: "I layer validation: HTML constraints for free basics, a pure `validate` function in React for custom and cross-field rules, and always server-side validation because the client can be bypassed. Errors are derived from values, shown after blur or first submit, linked with `aria-describedby`, flagged with `aria-invalid`, and on a failed submit I focus the first invalid field. For big forms I'd use a schema library such as Zod so the same rules run on both sides."

- *What does `noValidate` do?* — Turns off the browser's blocking and bubbles; constraint APIs (`validity`) still work.
- *Why a pure `validate` function?* — Testable, reusable on the server, and errors become derived state.

Self-check

1. Which validation layer is mandatory? → Server-side.
2. Where do `errors` live? → Computed during render from values — not in state.
3. Two ARIA attributes for an invalid field? → `aria-invalid="true"` and `aria-describedby` pointing at the message.

TL;DR

HTML constraints + a pure `validate()` + server checks. Derive errors; show them on blur or submit; announce and link them accessibly; focus the first invalid field.

4.4 Form Actions — `<form action={fn}>`

What it is

In React 19 the `action` prop of `<form>` accepts a **function**. When the form is submitted, React prevents the page navigation, builds a `FormData` from the form, and calls your function with it. In React's vocabulary a function that runs inside a **Transition** (a state update React treats as non-urgent and tracks as "pending" — Chapter 7) is called an **Action**. By convention such functions are named `…Action` or describe what they do (`subscribe`, `saveNote`).

The same idea exists on buttons: **`formAction={fn}`** on a `<button>` (or `<input type="submit">`) overrides the form's action for submissions made with that button — e.g. "Publish" vs "Save draft".

Why it matters

Compared with `onSubmit`:

- No `e.preventDefault()`, no reading fields by hand — you get `FormData`.
- The function may be **async**; React tracks it as pending, which powers `useFormStatus` (§4.6), `useActionState`'s `isPending` (§4.5) and `useOptimistic` (§4.7).
- After the Action **finishes successfully**, React **resets the form**: uncontrolled fields go back to their `defaultValue`s (a fresh, empty form — exactly what you want after "Send").
- With Server Functions (§4.8) the same form can even work **before JavaScript loads**.

How it works

1. User submits (button click or Enter).
2. React calls `preventDefault()`, creates `new FormData(form, submitter)` and starts a Transition.
3. Your function runs with `formData`; while its promise is pending, the form is "pending".
4. When it resolves, React commits any state updates and **resets** the form's uncontrolled fields.
5. If it **throws** (or rejects), the error goes to the nearest **error boundary** (Chapter 14) — so catch expected errors yourself and turn them into state (§4.5).

`code/03_form_action.jsx`:

```
function Newsletter() {
  const [log, setLog] = useState([]);
  async function subscribe(formData) {            // no event, no preventDefault
    const email = formData.get('email');
    const topics = formData.getAll('topic');       // several checkboxes with the same name
    await wait(50);                                // pretend network call
    setLog((l) => [...l, `${email} -> ${topics.join('+') || 'none'}`]);
  }
  function saveDraft(formData) {                   // a second button can run a different action
    setLog((l) => [...l, `draft saved for ${formData.get('email')}`]);
  }
  return (
    <form action={subscribe}>
      <input name="email" type="email" aria-label="Email" />
      <label><input type="checkbox" name="topic" value="react" /> React</label>
      <label><input type="checkbox" name="topic" value="css" /> CSS</label>
      <button type="submit">Subscribe</button>
      <button formAction={saveDraft}>Save draft</button>
      <ul>{log.map((line) => <li key={line}>{line}</li>)}</ul>
    </form>
  );
}
// user: types ada@example.com, ticks React and CSS, clicks Subscribe; later types bo@example.com, clicks Save draft
```

Output:

```
right after click, email field = "ada@example.com"
after the action finished:
  log          = [ 'ada@example.com -> react+css' ]
  email field  = "" (auto-reset)
  React ticked = false
  log          = [ 'ada@example.com -> react+css', 'draft saved for bo@example.com' ]
```

- While the Action was running the field still showed the email; **after** it finished the form was reset (empty field, boxes unticked). The `log` state was *not* reset — the reset only affects the DOM values of uncontrolled fields.
- `getAll('topic')` collected both ticked checkboxes.
- "Save draft" ran `saveDraft` instead of `subscribe` because of `formAction`.

Details worth knowing

Detail	Behaviour
HTTP method	With a function action the `method` prop is irrelevant — nothing is sent by the browser; React calls your function
Controlled fields	Not reset (their value comes from state) — clear the state yourself
Keep values after an error	Return them from the Action and use them as `defaultValue` (§4.5), or use controlled fields
Reset by hand	`requestFormReset(formElement)` from `react-dom`, called inside a Transition/Action
Mixing with `onSubmit`	If an `onSubmit` handler calls `preventDefault()`, the function action does not run — use one or the other
Where it runs	In the browser. Only a Server Function (`'use server'`, §4.8) runs on the server

Trade-offs

`onSubmit` + `preventDefault`	`action={fn}`
Works in every React version	React 19+
Full access to the event	Receives `FormData` only
Pending/error/reset are manual	Pending state, reset, optimistic UI, progressive enhancement built in
Natural with controlled fields	Natural with uncontrolled fields

Common mistakes

- Expecting the form *not* to reset and losing the user's input after a failed validation → return the values (§4.5) or throw nothing and keep them in state.
- Letting expected errors (validation, "email taken") throw — that shows the error boundary. Return them instead.
- Forgetting `name` attributes — `FormData` is empty.

In the interview

> **30-second answer:** "In React 19 `<form action>` accepts a function. React prevents the navigation, passes `FormData`, and runs it inside a Transition, so it can be async and its pending state is visible to `useFormStatus` and `useActionState`. After a successful Action, uncontrolled fields are reset. `formAction` on a button overrides the action per button. Uncaught errors go to an error boundary."

- *What's an "Action" in React 19?* — A function run inside a Transition, e.g. passed to `action`/`formAction` or called in `startTransition`.
- *Does the form reset if the Action throws?* — The reset happens after a successful completion; errors go to the error boundary.

TL;DR

`<form action={fn}>` → `fn(formData)` inside a Transition; async allowed; auto-reset of uncontrolled fields after success; `formAction` per button; return expected errors, don't throw them.

4.5 `useActionState` — the result of the last submission

What it is

A form usually needs to show **what happened last time**: validation errors, "Saved!", or a server error. `useActionState` wraps your Action and stores its **return value** as state:

```
const [state, formAction, isPending] = useActionState(actionFn, initialState, permalink?);
//      last result  pass to action=   true while running
// actionFn(previousState, payload) -> new state (sync or async)
```

Piece	Meaning
`actionFn(prevState, formData)`	Your function. First argument is the **previous state** (then the form data). Whatever it returns becomes the new `state`
`initialState`	The state before the first submission
`state`	The latest return value
`formAction`	A wrapped function to pass to `<form action>` / `formAction` (or call yourself, see below)
`isPending`	`true` while the Action runs
`permalink` (optional)	For Server Functions: the URL the form should post to if JavaScript hasn't loaded

Analogy: a receipt printer on the form. Each submission prints a receipt (the returned state); the form shows the latest receipt, and the printer can see the previous one.

It was called `useFormState` (in `react-dom`) during the React 19 canary period; the stable API is `useActionState` from `react`.

How it works

`code/04_use_action_state.jsx`:

```
// Pretend server call: validates, may fail, returns a result object.
async function registerUser(prevState, formData) {
  const username = String(formData.get('username') ?? '').trim();
  const age = Number(formData.get('age'));
  await wait(30);
  const errors = {};
  if (username.length < 3) errors.username = 'At least 3 characters';
  else if (taken.has(username)) errors.username = `"${username}" is taken`;
  if (!(age >= 13)) errors.age = 'You must be 13 or older';
  if (Object.keys(errors).length) return { errors, values: { username, age: formData.get('age') }, attempts:
prevState.attempts + 1 };
  taken.add(username);
  return { message: `Welcome, ${username}!`, errors: {}, values: { username: '', age: '' }, attempts:
prevState.attempts + 1 };
}

const initialState = { errors: {}, values: { username: '', age: '' }, attempts: 0 };

function Register() {
  const [state, formAction, isPending] = useActionState(registerUser, initialState);
  return (
    <form action={formAction}>
      {/* defaultValue from the last result: after the automatic reset the fields show what the user typed */}
      <input name="username" aria-label="Username" defaultValue={state.values.username} />
      {state.errors.username && <p role="alert">{state.errors.username}</p>}
      <input name="age" aria-label="Age" inputMode="numeric" defaultValue={state.values.age} />
      {state.errors.age && <p role="alert">{state.errors.age}</p>}
      <button disabled={isPending}>{isPending ? 'Registering…' : 'Register'}</button>
      {state.message && <p role="status">{state.message}</p>}
      <small>attempts: {state.attempts}</small>
    </form>
  );
}

// Calling the dispatch function yourself: wrap it in startTransition. Calls are queued in order.
function Stock() {
  const [count, dispatch, isPending] = useActionState(async (prev, delta) => { await wait(20); return prev + delta; },
0);
  return (
    <p>
      <button onClick={() => startTransition(() => dispatch(+1))}>+1</button>
      <output>{count}</output>{isPending && ' (saving…)'}
    </p>
  );
}
```

Output:

```
pending   : button = "Registering…" | disabled = true
attempt 1 : [ '"ada" is taken', 'You must be 13 or older' ] | fields kept: ada 9
attempt 2 : [] | Welcome, grace! | fields: "" "" | attempts: 2

three quick clicks -> shows +10 (saving…)
after the queue ran  -> shows +13
```

Step by step:

1. **Pending**: while `registerUser` awaited, `isPending` was `true` — the button said "Registering…" and was disabled (no double submits).
2. **Attempt 1** returned errors *and the submitted values*. The form was reset after the Action, but because the inputs' `defaultValue` now came from `state.values`, the reset put the user's text back. This is the standard pattern for "don't wipe the form on a validation error" with uncontrolled fields.
3. **Attempt 2** succeeded and returned empty values → a clean form and a success message. `prevState.attempts + 1` shows the previous state being passed in.
4. **Manual dispatch**: three clicks queued three calls; React runs them **one after another**, each receiving the previous result (0→1→2→3). Calling `dispatch` outside a Transition logs an error in development — wrap it in `startTransition` (or pass it to an `action` prop, which does that for you).

Rules and caveats

- The action receives `(prevState, payload)` — a very common bug is writing `async (formData) => …` and getting the previous state as "formData".
- **Queued and sequential**: multiple dispatches don't run in parallel. Good for correctness, but a slow request delays the next one.
- **No built-in reset** of the state: return the initial state from the action, or remount the component with a new `key`.
- **Throwing** cancels queued actions and shows the nearest error boundary — return errors instead.
- If you set *other* state after an `await` inside the action, currently wrap that update in `startTransition` (§4.7 shows this).
- With Server Functions, `initialState` and the payload must be **serializable** (plain data).

`useActionState` vs doing it yourself

Manual (`onSubmit`)	`useActionState`
`useState` for errors, message, isSubmitting	One Hook: `[state, action, isPending]`
`try/finally` to clear `isSubmitting`	Pending is automatic
Double submit possible unless guarded	Calls are queued; button can use `isPending`
Works with any event	Designed for Actions (forms, buttons, `startTransition`)

It is also a **reducer for async work**: `(prevState, action) => newState`, like `useReducer` (Chapter 6 §6.2) but allowed to be async and have side effects.

In the interview

30-second answer: "`useActionState(fn, initialState)` returns the latest result of an Action, a wrapped action to pass to a form, and `isPending`. The function gets the previous state and the form data and returns the next state, so it's the natural place to return validation errors or a success message. Calls are queued sequentially; when dispatching manually you wrap it in `startTransition`. With uncontrolled fields I return the submitted values and use them as `defaultValue`s so a failed submit doesn't wipe the form."

- *What's the first argument of the action?* — The previous state; the form data is second.
- *How do you reset the state?* — Return `initialState` from the action or remount with a `key`.

Self-check

1. Where do validation errors from the server go? → Returned from the action → `state.errors`.

2. Why did the fields keep "ada" and "9" after the reset? → `defaultValue={state.values…}` — reset restores defaults, which now hold the submitted values.
3. What happens if you call `dispatch()` in a plain `onClick`? → Dev error; not treated as a Transition, `isPending` doesn't update.

TL;DR

`const [state, action, isPending] = useActionState(async (prev, formData) => next, initial)`. Return errors/results, don't throw. Sequential queue. Manual calls go in `startTransition`.

4.6 `useFormStatus` — status of the parent form

What it is

`useFormStatus()` (imported from `react-dom`) lets any component **rendered inside a `<form>`** read that form's current submission — without the form passing props down. It returns:

Field	Meaning
`pending`	`true` while the parent form's Action is running
`data`	The `FormData` being submitted (or `null`)
`method`	The form's `method` attribute: `'get'` (default) or `'post'`
`action`	The function passed to the parent's `action` (or `null`)

Analogy: a light above the office door. Anyone inside the room (a child of the form) sees whether a meeting is in progress; people outside don't.

How it works

`code/05_use_form_status.jsx`:

```jsx
// Reusable: works in any form, because it asks its parent <form> for the status.
function SubmitButton({ children }) {
  const { pending, data, method } = useFormStatus();
  return (
    <button disabled={pending}>
      {pending ? `Saving "${data.get('title')}" (${method})…` : children}
    </button>
  );
}

// Wrong place: the Hook is in the SAME component that renders the <form>, so it sees no parent form.
function WrongPlace() {
  const { pending } = useFormStatus();
  return (
    <form action={() => wait(30)}>
      <input name="title" aria-label="Title 2" defaultValue="x" />
      <button>{pending ? 'pending' : 'never pending'}</button>
    </form>
  );
}

async function saveNote(formData) { await wait(30); console.log('  saved:', formData.get('title')); }

function NoteForm() {
  return (
    <form action={saveNote}>
      <input name="title" aria-label="Title" />
      <SubmitButton>Save note</SubmitButton>
    </form>
  );
}
```

Output:

```
during  : "Saving \"Groceries\" (get)…" disabled = true | "never pending"
  saved: Groceries
after   : "Save note" disabled = false
```

- `SubmitButton` knew the form was pending and even showed the title being submitted from `data`.
- `method` reported `'get'` — it simply mirrors the form's `method` attribute (default `get`); with a function action no HTTP request is made anyway.
- `WrongPlace` never saw `pending`: the Hook must be in a **child component of the form**, not in the component that renders it.

`useFormStatus` vs `useActionState`'s `isPending`

`useFormStatus`	`isPending` from `useActionState`
Read by any **descendant** of the form	Available in the component that owns the action
Great for reusable design-system buttons/spinners	Great when the owner needs it (disable the whole form)
Also exposes the submitted `data`	Also gives the action's result `state`

Common mistakes

- Calling it in the same component as the `<form>` (always `pending: false`).
- Importing it from `react` — it's in **`react-dom`**.
- Expecting it to track sibling or nested forms — only the nearest **parent** `<form>`.

TL;DR

`const { pending, data } = useFormStatus()` (from `react-dom`) inside a child of the form. Perfect for a reusable Submit button.

4.7 `useOptimistic` — instant feedback, automatic rollback

What it is

An **optimistic update** shows the result you *expect* before the server confirms it: a sent chat message appears immediately, a like count goes up at once. If the server later fails, you undo it. Users perceive the app as fast because most requests succeed.

```
const [optimisticState, addOptimistic] = useOptimistic(value, reducer?);
// value: the real (confirmed) state, e.g. from useState or props
// reducer(currentState, optimisticValue) -> the state to show while an Action runs (pure)
```

- When **no Action is running**, `optimisticState` equals `value`.
- Calling `addOptimistic(x)` **inside an Action** (a form action or `startTransition`) immediately shows `reducer(value, x)` (or `x` itself if there is no reducer).
- When the Action **ends**, the optimistic value is thrown away and `optimisticState` is `value` again — which by then includes the real result if you updated it, or the old data if it failed. **Rollback is automatic.**

Analogy: a restaurant writes your order on the board as "pending" the moment you order; once the kitchen confirms, it becomes a normal line; if the kitchen is out of that dish, the line is simply erased.

How it works

`code/06_use_optimistic.jsx`:

```
function Chat() {
  const [messages, setMessages] = useState([{ id: 'm1', text: 'Hi!' }]);
  const [error, setError] = useState(null);
  // optimisticMessages === messages when no Action is running
  const [optimisticMessages, addOptimistic] = useOptimistic(
    messages,
    (current, text) => [...current, { id: 'temp-' + text, text, sending: true }],   // pure reducer
  );

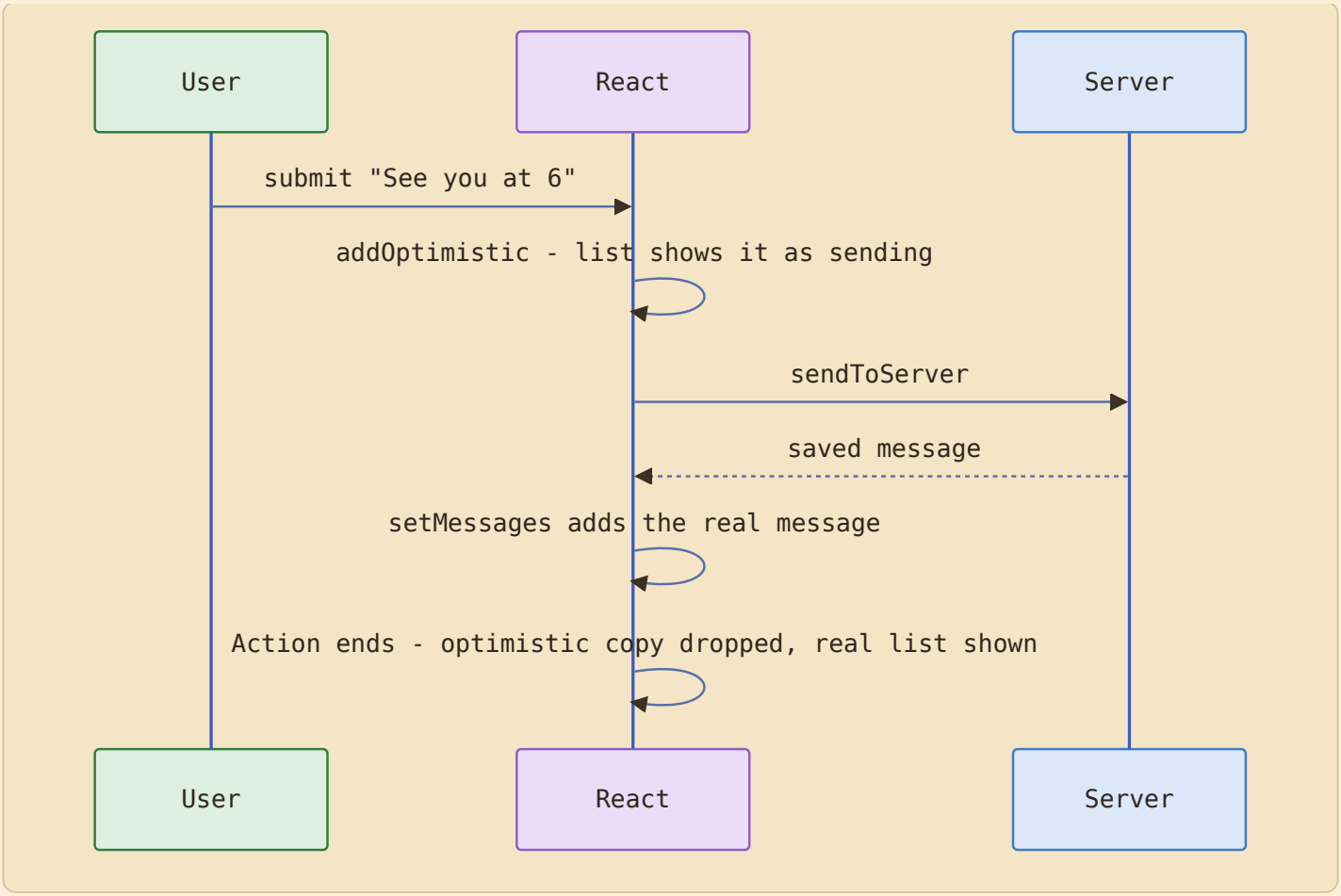
  async function sendAction(formData) {               // runs inside a Transition (form action)
    const text = formData.get('text');
    setError(null);
    addOptimistic(text);                              // 1. appears instantly, marked "sending"
    try {
      const saved = await sendToServer(text);         // 2. wait for the server
      startTransition(() => setMessages((ms) => [...ms, saved]));  // 3. real state (after await -> wrap)
    } catch (e) {
      setError(e.message);                            // optimistic item disappears by itself
    }
  }

  return (
    <>
      <ul>
        {optimisticMessages.map((m) => <li key={m.id}>{m.text}{m.sending && ' (sending…)'}</li>)}
      </ul>
      {error && <p role="alert">{error}</p>}
      <form action={sendAction}>
        <input name="text" aria-label="Message" />
        <button>Send</button>
      </form>
    </>
  );
}
```

Output:

```
instantly      : [ 'Hi!', 'See you at 6 (sending…)' ]
server said OK : [ 'Hi!', 'See you at 6' ]
instantly      : [ 'Hi!', 'See you at 6', 'buy spam now (sending…)' ]
server said NO : [ 'Hi!', 'See you at 6' ]
```

(the second run also shows the alert "Message rejected")



Notes:

- The optimistic item had a temporary id and a `sending` flag, so the UI could grey it out.
- On failure, no "undo" code was written — the Action ended without `messages` changing, so the optimistic line vanished; the `catch` just showed a message.
- The `setMessages` after `await` is wrapped in `startTransition` (a current React requirement for state updates after `await` inside an Action) so it lands together with the end of the Action.

Outside forms

In a click handler, start the Action yourself:

```
// (not run separately — same rules as above)
function LikeButton({ likes, onSave }) {
  const [optimisticLikes, setOptimisticLikes] = useOptimistic(likes);
  return (
    <button onClick={() => startTransition(async () => {
      setOptimisticLikes((n) => n + 1);   // must be inside startTransition / an Action
      await onSave();                     // parent updates `likes` when the server confirms
    })}>♥ {optimisticLikes}</button>
  );
}
```

Calling the setter outside an Action produces a development warning — there is no pending Action to keep the optimistic value alive.

When not to use it

- When failure is common or costly to "take back" (payments, bookings, irreversible actions) — show a real pending state instead.
- When the server's result differs a lot from what you can predict (generated IDs are fine; complex server-side calculations are not).

Common mistakes

- Treating the optimistic state as the source of truth — it's temporary; always update the real state (`setMessages`, refetch, or parent props).
- Side effects in the reducer — it must be pure (it can re-run if `value` changes while the Action is pending).
- Forgetting to show that an item is pending — users should be able to tell "sent" from "sending".

In the interview

> **30-second answer:** "`useOptimistic(value, reducer)` gives a state that equals the real value except while an Action is running, when it shows what I optimistically added. I call the setter inside the Action, then await the server and update the real state. When the Action ends React drops the optimistic value, so on success the real data takes over and on failure it rolls back automatically."

- *Where must `addOptimistic` be called?* — Inside an Action: a form/button action or `startTransition`.
- *How does rollback work?* — The optimistic value only lives for the Action's duration; if the real state wasn't updated, the old value shows again.

TL;DR

`useOptimistic(realValue, reducer)` → show the expected result during an Action; update the real state on success; failure rolls back by itself. Only call the setter inside an Action.

4.8 Server Functions and progressive enhancement (overview)

What it is

In a framework that supports **React Server Components** (for example Next.js App Router — Chapter 13), you can mark an async function with the directive `'use server'`. It becomes a **Server Function**: its code stays on the server, and the client gets a reference that, when called, sends the arguments over the network and runs the function there. Passed to a form's `action`, it's often called a **Server Action**.

```
// (framework-only — not run; needs a React Server Components bundler such as Next.js)
// app/actions.js
'use server';
export async function addToCart(prevState, formData) {
  const id = formData.get('productId');
  await db.cart.add(id);                      // runs on the server: database, secrets are safe here
  return { added: id };
}

// app/AddButton.jsx
'use client';
import { useActionState } from 'react';
import { addToCart } from './actions';
export function AddButton({ productId }) {
  const [state, action, isPending] = useActionState(addToCart, null, `/product/${productId}`);
  return (
    <form action={action}>
      <input type="hidden" name="productId" value={productId} />
      <button disabled={isPending}>Add to cart</button>
      {state?.added && <p>Added!</p>}
    </form>
  );
}
```

Why it matters — progressive enhancement

Progressive enhancement means the basic feature works with plain HTML, and JavaScript makes it nicer. Because a Server Function has a URL, a server-rendered form can be submitted **before the JavaScript bundle has loaded** (slow phone, flaky network): the browser does a normal POST, the server runs the function, and the page comes back with the result. The optional `permalink` argument of `useActionState` tells React which page to return in that case. Once hydrated, the same form works as the Actions in this chapter.

Rules to remember

- Arguments and return values must be **serializable** (strings, numbers, plain objects, arrays, `FormData`, …) — no functions or class instances.
- Treat every Server Function as a **public API endpoint**: authenticate, authorise and validate inside it.
- `'use server'` marks server-callable functions; it is **not** how you mark Server Components (those need no directive). Details in Chapter 13.

> **TL;DR**
>
> `'use server'` functions run on the server but are called like local functions; used as form actions they enable forms that work before JS loads. Validate and authorise inside them.

4.9 Form libraries — React Hook Form and Zod (overview)

> **The problem they solve**

Hand-written forms (§4.3) are fine for 3–5 fields. Big forms — dozens of fields, nested objects, arrays of rows ("add another address"), async checks, multi-step wizards — accumulate a lot of `values` / `touched` / `errors` plumbing. Two popular libraries split that work:

Library	Job
React Hook Form (RHF)	Manages field registration, values, touched/dirty state, errors, submit handling. Uses **uncontrolled inputs** + refs, so typing doesn't re-render the whole form
Zod	A **schema** library: describe the shape and rules of data once, then `parse` / `safeParse` any value. Also infers TypeScript types from the schema
`@hookform/resolvers`	Glue: lets RHF validate with a Zod (or Yup, Valibot…) schema

Sharing one Zod schema between the browser form and the server handler means the rules can't drift apart.

What the code looks like

```
// (React Hook Form 7 + Zod 4 + @hookform/resolvers — not run; these packages aren't installed for this book)
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

const schema = z.object({
  email: z.email({ error: 'Enter a valid email' }),          // Zod 4: top-level z.email()
  password: z.string().min(8, { error: 'At least 8 characters' }),
  confirm: z.string(),
}).refine((d) => d.password === d.confirm, { error: 'Passwords do not match', path: ['confirm'] });

export function SignupForm({ onSignup }) {
  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm({ resolver: zodResolver(schema), mode: 'onTouched' });   // validate after blur, then live
  return (
    <form onSubmit={handleSubmit(onSignup)} noValidate>              {/* onSignup gets typed, valid data */}
      <input type="email" {...register('email')} aria-invalid={!!errors.email} />
      {errors.email && <p role="alert">{errors.email.message}</p>}
      <input type="password" {...register('password')} />
      {errors.password && <p role="alert">{errors.password.message}</p>}
      <input type="password" {...register('confirm')} />
      {errors.confirm && <p role="alert">{errors.confirm.message}</p>}
      <button disabled={isSubmitting}>Create account</button>
    </form>
  );
}

// On the server, the same schema:
// const result = schema.safeParse(body);      // { success: true, data } | { success: false, error }
// if (!result.success) return { errors: z.flattenError(result.error).fieldErrors };
```

- `register('email')` returns `{ name, onChange, onBlur, ref }`, spread onto the input.

- `handleSubmit(fn)` prevents the default, validates, and calls `fn(data)` only when valid (focusing the first invalid field by default).
- In Zod 4, string formats such as email are top-level (`z.email()`), and custom messages use the `error` option (`message` still works but is deprecated). `safeParse` never throws.

When to use what

Situation	Recommendation
Small form, submit → server	Plain form Action + `useActionState` (§4.5)
Needs live UI per keystroke	A few controlled fields
Large/complex client-side form, field arrays, wizards	React Hook Form (+ Zod)
Same rules on client and server, TypeScript types from rules	Zod (or Valibot) schema shared by both

Alternatives you'll meet: Formik (older, controlled-input based, less active), TanStack Form, Conform (built around Actions/progressive enhancement), Yup/Valibot schemas.

> **TL;DR**
>
> RHF manages big forms efficiently with uncontrolled inputs; Zod describes validation rules once for client and server; `zodResolver` connects them. Don't add them for a 3-field form.

4.10 Putting it together — an RSVP form

`code/rsvp.jsx` combines the chapter: uncontrolled fields read with `FormData`, a pure `validateRsvp`, `useActionState` returning errors + submitted values, a reusable `SubmitButton` with `useFormStatus`, and an optimistic guest list with `useOptimistic`.

```
function SubmitButton() {
  const { pending } = useFormStatus();                 // reads the parent <form>
  return <button className="primary" disabled={pending}>{pending ? 'Sending…' : 'Send RSVP'}</button>;
}

export default function Rsvp({ saveRsvp, initialGuests = [], initialResult = emptyResult }) {
  const [guests, setGuests] = useState(initialGuests);
  const [optimisticGuests, addOptimistic] = useOptimistic(guests, (list, g) => [...list, { ...g, id: 'pending',
pending: true }]);

  async function rsvpAction(prev, formData) {
    const values = Object.fromEntries(formData);        // { name, email, guests, diet }
    const rsvp = { name: values.name.trim(), email: values.email.trim(), guests: Number(values.guests), diet:
values.diet };
    const errors = validateRsvp(rsvp);
    if (Object.keys(errors).length) return { errors, values, message: '' };   // keep what they typed
    addOptimistic(rsvp);                                  // show in the list right away
    try {
      const saved = await saveRsvp(rsvp);
      startTransition(() => setGuests((gs) => [...gs, saved]));
      return { ...emptyResult, message: `Thanks ${saved.name}, see you there!` };
    } catch (e) {
      return { errors: { form: e.message }, values, message: '' };
    }
  }
  const [result, formAction] = useActionState(rsvpAction, initialResult);
  const { errors, values } = result;
  const total = optimisticGuests.reduce((n, g) => n + g.guests, 0);   // derived

  return (
    <div className="card">
      <h1>Team dinner — RSVP</h1>
      <form action={formAction} noValidate>
        <label htmlFor="name">Name</label>
        <input id="name" name="name" defaultValue={values.name} aria-invalid={!!errors.name} aria-describedby="name-
err" />
        <FieldError id="name-err" message={errors.name} />
        {/* ... email the same way ... */}
        {/* key: React ignores a CHANGED defaultValue on <select>, so remount it with the new default */}
        <select key={values.guests} id="guests" name="guests" defaultValue={values.guests}>…</select>
        {/* ... radio buttons with defaultChecked={values.diet === d} ... */}
        <FieldError id="form-err" message={errors.form} />
        {result.message && <p className="ok" role="status">{result.message}</p>}
        <SubmitButton />
      </form>
      <h2>Coming ({total})</h2>
      <ul className="guests">{/* optimisticGuests.map(...) with "(saving…)" for pending */}</ul>
    </div>
  );
}
```

`code/07_rsvp_app.jsx` drives it: an invalid submit (with 2 guests and "vegan" chosen), a valid one, then a duplicate that the fake server rejects:

Output:

```
1 invalid      : [ 'Please enter your name', 'Enter an email like ada@example.com' ] | kept: A 2 true
2 sending      : [ 'Host · 1 seat', 'Ada · 2 seats · vegan (saving…)' ] | button: Sending…
2 saved        : [ 'Host · 1 seat', 'Ada · 2 seats · vegan' ] | Thanks Ada, see you there! | heading: Coming (3)
3 server error : [ 'ada@example.com has already replied' ] | list: [ 'Host · 1 seat', 'Ada · 2 seats · vegan' ]
```

1. Invalid input → two errors; the name, guest count and diet choice survived the automatic reset (returned `values` → `defaultValue`s; the select remounted through its `key`).
2. Valid input → the guest appeared instantly as "(saving…)", the button read "Sending…", the headcount already counted the optimistic seats.
3. The server's answer replaced the optimistic row; the success message appeared and the form was cleared.

4. A duplicate email → the server threw; the action **caught** it and returned it as a form-level error, and the optimistic row rolled back by itself.

In the browser (`code/rsvp.html`, styles in `rsvp.css`; the entry `rsvp-main.jsx` starts the form in the state after one invalid submit):

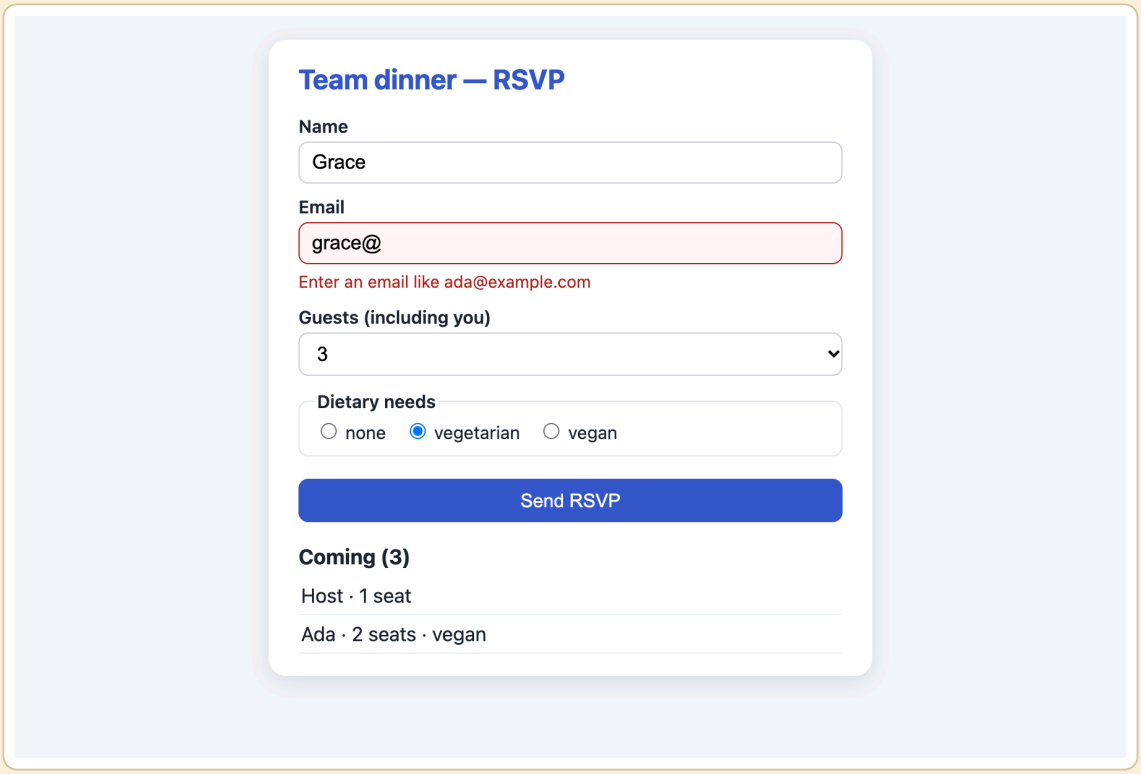


Figure 4.1 — The RSVP form after a failed submit: the typed values are kept, the invalid field is marked with `aria-invalid` (red border) and its message is linked by `aria-describedby`.

Design notes:

- **Minimal state**: only `guests` is `useState`; the action result lives in `useActionState`, the pending flag in the form, the headcount is derived.
- **Validation in one pure function** that a real server would call too.
- **Expected failures are returned, not thrown**, so no error boundary is needed for "already replied".

Chapter 4 TL;DR

Idea	Remember
`FormData`	Needs `name`; unchecked boxes missing; values are strings; `getAll` for repeated names
Controlled	`value` + `onChange`; re-render per keystroke; live formatting/limits
Uncontrolled	`defaultValue` / `defaultChecked`; read via `FormData` or a ref; file inputs always uncontrolled
Validation	HTML constraints + pure `validate()` + **server**; show on blur/submit; `aria-invalid` + `aria-describedby`; focus first error
Form Action	`<form action={fn}>` → `fn(formData)` in a Transition; auto-reset after success; `formAction` per button
`useActionState`	`[state, action, isPending] = useActionState((prev, formData) => next, initial)`; return errors; queued calls; manual calls in `startTransition`
`useFormStatus`	From `react-dom`, in a **child** of the form: `{ pending, data, method, action }`
`useOptimistic`	Show expected result during an Action; update real state; auto rollback
Server Functions	`'use server'`; progressive enhancement; always validate/authorise inside
Libraries	React Hook Form + Zod for big forms; plain Actions for small ones

Next: Chapter 5 — Effects and Refs.

Chapter 5 — Effects and Refs

So far every component has been a pure function of props and state (Chapter 2 §2.5), and every change has come from an event handler (Chapter 3). Real apps also need to talk to things **outside React**: a chat server connection, a timer, a browser API such as `document.title`, a third-party map widget, a DOM node that must be focused or measured. React's two **escape hatches** for this are **Effects** (`useEffect`, "run this code after rendering to synchronise with something outside") and **refs** (`useRef`, "remember a value, or hold a DOM node, without re-rendering").

They are also the most over-used Hooks. Half of this chapter is about **when not to use an Effect**, because unnecessary Effects make apps slower and buggier.

What you'll learn

#	Topic	You will be able to…
5.1	Where side effects belong	choose between render, event handler and Effect
5.2	`useEffect` basics	write an Effect and predict when it runs
5.3	Cleanup	undo subscriptions, connections, timers and listeners
5.4	StrictMode's double run	understand setup → cleanup → setup in development and why it's your friend
5.5	Dependencies in depth	satisfy the linter by changing code; use updaters and `useEffectEvent`
5.6	You might not need an Effect	replace Effects with rendering, `key`, handlers and better state
5.7	Fetching data in Effects	avoid race conditions with an ignore flag or `AbortController`; know the limits
5.8	`useRef` for values	keep mutable values that don't affect rendering
5.9	Refs to DOM nodes	focus, scroll, measure; ref callbacks and lists of refs
5.10	Passing refs to components	`ref` as a prop (React 19), `forwardRef` (legacy), `useImperativeHandle`
5.11	`useLayoutEffect`	measure before the browser paints; know its cost
5.12	Putting it together	a city search with five Effects and three refs

Code: `code/` — React 19.3 in jsdom. jsdom has no layout engine, so the examples that measure or scroll fake the few layout numbers they need and say so.

5.1 Where side effects belong

What it is

A **side effect** is anything a piece of code does besides computing a return value: changing the DOM by hand, sending a request, starting a timer, writing to `localStorage`, logging analytics. React gives you three places to put code:

Place	Runs when	Must be	Use it for
Rendering (the component body)	Every time React needs the UI	**Pure** — same inputs, same JSX, no side effects	Computing JSX and derived values
Event handlers	When a specific user action happens	Anything goes	Side effects **caused by a particular interaction**: submit a form, buy a product, start a timer on click
Effects (`useEffect`)	**After** a render is committed to the screen	Should be undoable (cleanup)	Side effects **caused by rendering itself** — keeping the component **synchronised** with an external system while it's on screen

The question to ask is **"why does this code run?"**

- *Because the user clicked "Send"* → event handler.
- *Because this component is visible and must stay connected / subscribed / in sync* → Effect.
- *Because I need a value computed from props or state* → neither: compute it during render.

Analogy: a shop assistant (your component). Answering a customer's question is rendering. Ringing up a sale because a customer asked to pay is an event handler. Keeping the shop lights on **while the shop is open** — and switching them off at closing time — is an Effect with cleanup.

TL;DR

Pure calculations → render. Caused by a user action → handler. Needed because the component is on screen, to sync with something outside React → Effect.

5.2 `useEffect` basics

What it is

```
useEffect(setup, dependencies?)
// setup: function run after the render is committed; may return a cleanup function
// dependencies: array of every reactive value setup reads (props, state, values derived from them)
```

An **Effect** runs **after** React has updated the DOM (and normally after the browser has painted), so it never blocks the user from seeing the new screen. A **reactive value** is anything declared in the component body that can change between renders: props, state, and variables computed from them.

The dependency array controls re-runs:

You write	Setup runs
`useEffect(fn)` — no array	After **every** render
`useEffect(fn, [])` — empty array	Once after the first render (**mount**); cleanup on **unmount**
`useEffect(fn, [a, b])`	After mount, and after any render where `a` or `b` changed (compared with `Object.is`)

How it works

`code/01_effect_basics.jsx`:

```
function Demo() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState('Ada');
  console.log(`render      count=${count} name=${name} | DOM says: ${document.querySelector('h1')?.textContent ??
'(nothing yet)'}`);

  useEffect(() => {
    console.log(`  effect A (no array)   -> after EVERY render; DOM says:
${document.querySelector('h1').textContent}`);
  });
  useEffect(() => {
    console.log('  effect B ([])         -> once, after the first render (mount)');
    return () => console.log('  cleanup B            -> on unmount');
  }, []);
  useEffect(() => {
    console.log(`  effect C ([name])     -> name is now ${name}`);
    return () => console.log(`  cleanup C            -> cleans up the "${name}" effect`);
  }, [name]);

  return (
    <>
      <h1>{name}: {count}</h1>
      <button onClick={() => setCount((c) => c + 1)}>count</button>
      <button onClick={() => setName('Grace')}>name</button>
    </>
  );
}
```

Output:

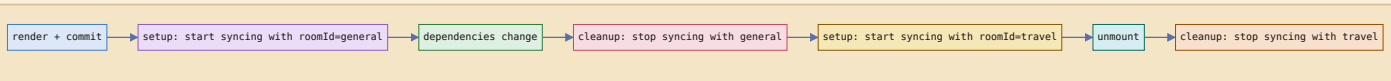
```
--- mount
render      count=0 name=Ada | DOM says: (nothing yet)
  effect A (no array)   -> after EVERY render; DOM says: Ada: 0
  effect B ([])         -> once, after the first render (mount)
  effect C ([name])     -> name is now Ada
--- click count (name unchanged)
render      count=1 name=Ada | DOM says: Ada: 0
  effect A (no array)   -> after EVERY render; DOM says: Ada: 1
--- click name
render      count=1 name=Grace | DOM says: Ada: 1
  cleanup C             -> cleans up the "Ada" effect
  effect A (no array)   -> after EVERY render; DOM says: Grace: 1
  effect C ([name])     -> name is now Grace
--- click name again (same value: nothing committed, so no effects)
render      count=1 name=Grace | DOM says: Grace: 1
--- unmount
  cleanup B             -> on unmount
  cleanup C             -> cleans up the "Grace" effect
```

What to notice:

1. **Effects run after the DOM is updated.** During render the `<h1>` still shows the old text; inside the Effect it shows the new text.
2. **Dependencies are compared with the previous render.** Clicking "count" didn't re-run C (name unchanged); clicking "name" did.
3. **Cleanup runs before the next setup** — and it sees the *old* values (`"Ada"`), because each render's Effect closes over that render's snapshot (Chapter 3 §3.3).
4. **Same state value → nothing is committed.** React may call the component once more to check (the extra "render" line) but doesn't update the DOM or run Effects.
5. **Unmount** runs every remaining cleanup.

An Effect's life, from its own point of view

Don't think in "mount/update/unmount" (class-component lifecycles). Think: **"start synchronising with X; stop synchronising with X."** React may start and stop as many times as needed — every time the dependencies change, and once extra in development (§5.4). If your setup/cleanup pair is symmetrical, how often it runs doesn't matter.



Rules

- Call `useEffect` at the **top level** of the component (Hook rules — Chapter 7).
- The setup function itself must not be `async` (it would return a promise, not a cleanup). Define an async function inside and call it (§5.7).
- Effects **don't run on the server** (server rendering — Chapter 13).

In the interview

30-second answer: "`useEffect` synchronises a component with an external system. The setup runs after the render is committed; the optional cleanup runs before the next setup and on unmount. The dependency array lists every reactive value the Effect reads: no array runs after every render, an empty array only on mount, and otherwise when a dependency changes by `Object.is`. I think of it as start-sync/stop-sync rather than lifecycle events."

- *Does `useEffect` run before or after paint?* — Normally after the browser paints; `useLayoutEffect` runs before (§5.11).
- *Why can't the setup be `async`?* — It must return nothing or a cleanup function, not a promise.

TL;DR

`useEffect(() => { start; return () => stop; }, [deps])`. Runs after commit; re-runs when deps change; cleanup before the next run and on unmount.

5.3 Cleanup — undo what setup did

What it is

The **cleanup function** is what your setup returns. It must **undo** the setup: disconnect what you connected, unsubscribe what you subscribed, clear the timer you started, remove the listener you added, cancel the request you sent. Without it, each re-run *adds* another connection/timer/listener — a **leak**.

How it works

`code/02_cleanup_and_strict_mode.jsx`:

```
function ChatRoom({ roomId }) {
  useEffect(() => {
    const connection = createConnection(roomId);   // setup
    connection.connect();
    return () => connection.disconnect();           // cleanup
  }, [roomId]);
  return <h2>Welcome to {roomId}</h2>;
}

function LeakyChatRoom({ roomId }) {
  useEffect(() => {
    createConnection(roomId).connect();             // no cleanup!
  }, [roomId]);
  return <h2>Leaky {roomId}</h2>;
}
// rendered inside <StrictMode>; the user switches room, then closes the chat

// Timers and listeners follow the same shape.
function Clock() {
  const [ticks, setTicks] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setTicks((t) => t + 1), 10);
    const onKey = (e) => console.log('  key pressed:', e.key);
    window.addEventListener('keydown', onKey);
    return () => { clearInterval(id); window.removeEventListener('keydown', onKey); };
  }, []);
  return <p>ticks: {ticks}</p>;
}
```

Output:

```
=== WITH cleanup, inside <StrictMode> (development)
mount:
  connect    general (open: 1)
  disconnect general (open: 0)
  connect    general (open: 1)
switch room:
  disconnect general (open: 0)
  connect    travel (open: 1)
close:
  disconnect travel (open: 0)
connections still open: 0

=== WITHOUT cleanup, inside <StrictMode> (development)
mount:
  connect    general (open: 1)
  connect    general (open: 2)
switch room:
  connect    travel (open: 3)
close:
connections still open: 3

  key pressed: Escape
clock shows ticks: 5 (one interval, one listener — the StrictMode duplicate was cleaned up)
after unmount the listener is gone (no "key pressed: Enter" above)
```

With cleanup, there is always **exactly one** open connection, to the right room. Without it, connections pile up — to rooms the user has already left — and keep running after the component is gone.

Common setup/cleanup pairs

Setup	Cleanup
`connection.connect()`	`connection.disconnect()`
`const id = setInterval(…)` / `setTimeout(…)`	`clearInterval(id)` / `clearTimeout(id)`
`window.addEventListener('resize', fn)`	`window.removeEventListener('resize', fn)` — the **same** function reference
`const unsub = store.subscribe(fn)`	`unsub()`
`const controller = new AbortController(); fetch(url, { signal })`	`controller.abort()`
`const obs = new IntersectionObserver(…); obs.observe(el)`	`obs.disconnect()`
`const map = new MapWidget(el)`	`map.destroy()`
`dialog.showModal()`	`dialog.close()`
Start an animation	Reset it to the initial state

Common mistakes

- Removing a listener with a *new* arrow function (`removeEventListener('x', () => fn())`) — nothing is removed. Keep the function in a variable.
- Forgetting that a cleanup sees the old render's values — that's usually what you want (disconnect the *old* room).
- Writing cleanup for something that can't be undone (a one-off POST request) — that code probably belongs in an event handler (§5.6).

TL;DR

Return a function that exactly undoes the setup. Then re-runs, room switches and unmounts never leak.

5.4 StrictMode's double run (development only)

What it is

When your app is wrapped in `<StrictMode>` (Vite and most templates do this — Chapter 1 §1.6), React in **development** mounts every component, then **immediately simulates an unmount and a re-mount**: setup → cleanup → setup. In the output above, that's the `connect / disconnect / connect` on mount. Production builds run the setup once.

React 19 applies the same check to **ref callbacks** (§5.9) in development, and React 19.3 also does it for components that were server-rendered and then hydrated.

Why React does this

Users *do* unmount and remount components: they navigate away and back, a tab is hidden with `<Activity>` (React 19.2's component for hiding a subtree while keeping its state) and shown again, a list is filtered. If setup and cleanup are not symmetrical, those real situations produce bugs that are hard to reproduce. StrictMode forces the situation on every mount in development so you see the bug immediately — the leaky version shows `open: 2` right at mount.

How to handle it

The fix is **never** "stop React running it twice". The fix is a correct cleanup:

Symptom in development	Wrong fix	Right fix
Two connections / subscriptions	A ref flag "already connected"	Return `disconnect` / `unsubscribe`
Two intervals ticking twice as fast	Disable StrictMode	`return () => clearInterval(id)`
Two fetches in the Network tab	Ref flag	Harmless if you ignore/abort the first (§5.7); or use a data library with caching
Analytics event logged twice	Ref flag	Acceptable in dev (production logs once), or send analytics from the event that caused it
Code that must run once per **app load** (initialising a SDK)	Effect with a ref	Top-level module code, or a module-level `let didInit = false` guard

In the interview

30-second answer: "In development, StrictMode runs every Effect's setup, cleanup and setup again on mount to check the cleanup mirrors the setup. It doesn't happen in production. If the double run causes a visible problem, the Effect is missing a cleanup — the fix is to add one, not to suppress the second run."

TL;DR

Dev-only setup → cleanup → setup exposes missing cleanups. Fix the cleanup; don't fight the double run.

5.5 Dependencies in depth

The rule

Every **reactive value** the Effect reads must be in its dependency array. The **`react-hooks/exhaustive-deps`** lint rule (in `eslint-plugin-react-hooks`, included by Vite's React template) checks this for you. Values that never change — module-level constants, `useRef` objects, state setters, the `dispatch` from `useReducer`/`useActionState` — can be left out.

You don't choose the dependencies; the code does. If an Effect re-runs too often, change the code so it reads fewer reactive values. Never silence the linter with `// eslint-disable-next-line` — it leads to Effects reading stale values.

Techniques

`code/03_dependencies.jsx` shows three:

```
// 1. An object created during render is a NEW object every render -> Effect re-runs on every keystroke.
function ObjectDep({ roomId }) {
  const [draft, setDraft] = useState('');
  const options = { serverUrl: 'https://chat.example', roomId };     // new {} each render
  useEffect(() => connect(options), [options]);
  // ... controlled <input> for draft
}
// Fix: create the object INSIDE the Effect and depend on the primitive.
function PrimitiveDep({ roomId }) {
  const [draft, setDraft] = useState('');
  useEffect(() => {
    const options = { serverUrl: 'https://chat.example', roomId };
    return connect(options);
  }, [roomId]);
  // ...
}

// 2. Reading state only to compute the next state: use an updater, and the dependency disappears.
useEffect(() => {
  const id = setInterval(() => (useUpdater ? setCount((c) => c + 1) : setCount(count + 1)), 10);
  return () => clearInterval(id);
}, useUpdater ? [] : [count]);

// 3. useEffectEvent (React 19.2+): read the LATEST props/state inside an Effect without making them dependencies.
function ChatRoom({ roomId, theme }) {
  const onConnected = useEffectEvent(() => log.push(`connected to ${roomId}, toast in ${theme} theme`));
  useEffect(() => {
    log.push(`connect ${roomId}`);
    const t = setTimeout(() => onConnected(), 5);                // "the server said connected"
    return () => { clearTimeout(t); log.push(`disconnect ${roomId}`); };
  }, [roomId]);                                                   // theme is NOT a dependency
  return null;
}
// theme changes light -> dark while connecting; later roomId changes to travel
```

Output (the counts in the middle two lines depend on timer timing):

```
object dependency   : connects after typing 5 letters = 6
primitive dependency: connects after typing 5 letters = 1
count dep: count=5 interval restarts=6
updater: count=5 interval restarts=1
useEffectEvent: [
  'connect general',
  'connected to general, toast in dark theme',
  'disconnect general',
  'connect travel',
  'connected to travel, toast in dark theme'
]
```

1. **Objects/functions created during render** are new on every render, so `Object.is` always says "changed": 6 connections for 5 keystrokes. Move them **inside** the Effect (or outside the component if they don't use props/state) and depend on primitives (`roomId`).
2. **Updater functions** (Chapter 3 §3.5) let an interval increment `count` without reading it, so the interval is created once instead of being torn down and restarted on every tick.
3. `useEffectEvent` splits logic in two: the *reactive* part (connect to `roomId` — should re-run when it changes) and the *non-reactive* part (show a toast in the *current* `theme` — should see the latest value but must **not** reconnect when the theme changes). Changing the theme did not reconnect, yet the toast used "dark". Changing the room did reconnect.

`useEffectEvent` rules (stable since React 19.2)

- Call the returned function **only from inside Effects** (or other Effect Events), never during render or from event handlers.
- **Don't** list it in dependencies (the linter knows it's non-reactive).
- Declare it in the same component/Hook as the Effect that uses it; don't pass it to other components.

- Use it for genuinely event-like logic ("connected", "a message arrived", "log a visit with the current cart size"), not as a trick to hide real dependencies.

Other techniques

Problem	Technique
Function prop from parent re-creates every render	Call it from an Effect Event, or have the parent memoise it with `useCallback` (Chapter 10)
Effect reads a whole object prop but needs one field	Destructure the primitive outside the Effect: `const { id } = user;` → `[id]`
Two unrelated things in one Effect	Split into two Effects, each with its own dependencies
Mutable value such as `location.pathname` or a ref's `.current`	Not a valid dependency (changes don't re-render). Subscribe with `useSyncExternalStore` (Chapter 7)

Self-check

1. Why did `ObjectDep` reconnect on every keystroke? → A new `options` object each render; `Object.is` compares references.
2. How can an interval Effect avoid depending on `count`? → `setCount(c => c + 1)`.
3. What is `useEffectEvent` for? → Reading latest values in an Effect without making them re-run triggers.

TL;DR

List everything the Effect reads; reduce the list by changing code — move objects inside, use updaters, split Effects, and use `useEffectEvent` for non-reactive logic. Never lie to the linter.

5.6 You might not need an Effect

The problem it solves

Beginners reach for `useEffect` whenever "something should happen when X changes". Most of those cases involve **no external system** and are better written without an Effect. Unneeded Effects cost an **extra render**, can show **stale UI for a frame**, and create **chains** that are hard to follow.

How to recognise it

If the Effect only (a) calls `setState` based on props/state, or (b) does something because of a *user action* that just happened — it's probably not needed.

`code/04_you_might_not_need_an_effect.jsx` runs five before/after pairs:

```jsx
// 1. Derived data: computing in an Effect = one extra render + one stale frame.
function FullNameEffect({ first, last }) {
  const [fullName, setFullName] = useState('');
  useEffect(() => setFullName(`${first} ${last}`), [first, last]);   // 🔴
  return <p id="n">{fullName}</p>;
}
function FullNameRender({ first, last }) {
  const fullName = `${first} ${last}`;                               // ✅ just compute it
  return <p id="n">{fullName}</p>;
}

// 2. Resetting state when a prop changes: Effect shows the OLD comment for one frame; a key doesn't.
function CommentBoxEffect({ userId }) {
  const [comment, setComment] = useState('');
  useEffect(() => setComment(''), [userId]);                         // 🔴
  // ...
}
function Profile({ userId }) {
  return <CommentBox key={userId} userId={userId} />;                // ✅ new key = fresh state
}

// 3. Event-specific logic belongs in the event handler, not in an Effect watching state.
function ProductEffect({ initialInCart }) {
  const [inCart, setInCart] = useState(initialInCart);
  useEffect(() => { if (inCart) toasts.push('effect: "Added to cart!"'); }, [inCart]);      // 🔴 also fires on page load
  return <button onClick={() => setInCart(true)}>Buy</button>;
}
function ProductHandler({ initialInCart }) {
  const [inCart, setInCart] = useState(initialInCart);
  function handleBuy() { setInCart(true); toasts.push('handler: "Added to cart!"'); }      // ✅ only when the user buys
  return <button onClick={handleBuy}>Buy</button>;
}

// 4. Notifying the parent: from an Effect = a second render pass; from the handler = one pass.
function Toggle({ onChange, viaEffect }) {
  const [on, setOn] = useState(false);
  useEffect(() => { if (viaEffect) onChange(on); }, [on]);           // 🔴 (and runs on mount too)
  function handleClick() { const next = !on; setOn(next); if (!viaEffect) onChange(next); }   // ✅ same event
  return <button onClick={handleClick}>{on ? 'On' : 'Off'}</button>;
}

// 5. Adjusting SOME state when a prop changes: best is to derive; else set state during render (no Effect).
function ListAdjust({ items }) {
  const [selection, setSelection] = useState(items[0]);
  const [prevItems, setPrevItems] = useState(items);
  if (items !== prevItems) {            // prop changed since last render -> fix up state right now
    setPrevItems(items);
    setSelection(null);                 // React re-runs this component immediately, before touching the DOM
  }
  return <p id="s">{selection ?? 'none'}</p>;
}
function ListDerived({ items }) {
  const [selectedId, setSelectedId] = useState('a');
  const selection = items.find((i) => i === selectedId) ?? null;     // ✅ derive: kept if still present
  return <p id="s">{selection ?? 'none'}</p>;
}
```

Output ("screen after each commit" is recorded right after every DOM update):

```
1 derived via Effect : renders=4 | screen after each commit: ["","Ada Lovelace","Ada Lovelace","Grace Hopper"]
1 derived via render : renders=2 | screen after each commit: ["Ada Lovelace","Grace Hopper"]
2 reset via Effect: screen after switching user: ["hi Ada","(empty)"]
2 reset via key   : screen after switching user: ["(empty)"]
3 ProductEffect : toasts on page load = ["effect: \"Added to cart!\""]
3 ProductHandler: toasts on page load = []
4 notify via Effect : component renders for one click = 3 | parent shows on
4 notify via handler: component renders for one click = 2 | parent shows on
5 ListAdjust : renders=3 | selection after items changed = none
5 ListDerived: renders=2 | selection after items changed = a
```

Reading the evidence:

1. The Effect version rendered **twice as often** and committed an **empty name** first, then a **stale "Ada Lovelace"** while the props already said Grace Hopper.
2. The Effect reset committed the previous user's draft ("hi Ada") to the screen before clearing it — on a chat app that's a privacy bug. The `key` version never shows it.
3. The Effect "Added to cart!" fired when a page simply **loaded** with an item already in the cart. The handler version fires only when the user buys.
4. Notifying the parent from an Effect costs an extra render pass (and fires on mount). Calling `onChange` in the same handler batches both updates into one render.
5. Setting state during render (with a "previous value" state) avoids the Effect, but deriving from an ID is simpler still — and keeps the selection when the item survives.

The full catalogue

You want to…	Instead of an Effect…
Compute something from props/state	Compute during render (Chapter 3 §3.8)
Cache an expensive calculation	`useMemo` (Chapter 10)
Reset **all** state when a prop changes	Give the component a `key`
Adjust **some** state when a prop changes	Derive it; if you must, set state during render with a stored previous value
Share logic between two event handlers	Extract a function and call it from both
Send a request because the user did something (POST on submit)	Do it in the event handler / form Action (Chapter 4)
Run a chain of state updates (A changes → set B → set C…)	Calculate all next state in the one handler
Initialise the app once (read a token, init a SDK)	Top-level module code or a `didInit` module flag
Tell the parent about a change	Call the parent's callback in the same handler, or lift the state up (Chapter 3 §3.7)
Pass fetched data up to the parent	Fetch in the parent and pass it down
Subscribe to an external store	`useSyncExternalStore` (Chapter 7)
Fetch data	An Effect *with cleanup* is acceptable (§5.7), but prefer a framework loader or TanStack Query (Chapter 6 §6.9, Chapter 9)

What *does* need an Effect

Connecting to a server or socket, subscribing to browser events (`resize`, `online`, keyboard shortcuts), timers that run while a component is visible, synchronising with a non-React widget (a map, a video player, `dialog.showModal()`), updating `document.title`, logging an impression **because the component was shown**, and fetching (with cleanup).

> **In the interview**

> **30-second answer:** "Effects are for synchronising with external systems. If there's no external system — just transforming props into state or reacting to a user event — I don't use one. Derived values are computed during render, expensive ones memoised; full resets use a `key`; event-caused logic goes in the handler; parents are

notified in the same handler. Unneeded Effects add renders, show stale frames, and chain into hard-to-follow code."

- *How do you reset a form when `userId` changes?* — `<Form key={userId} />`.
- *Where does 'POST analytics when the form is submitted' go?* — In the submit handler. 'Log that the page was viewed' goes in an Effect.

Self-check

1. `useEffect(() => setTotal(a + b), [a, b])` — better? → `const total = a + b;` in render.
2. Why is the Effect reset of a comment box a privacy risk? → The old user's text is committed to the screen for a frame.
3. Name three legitimate Effects. → Socket connection, window listener, `document.title` sync (also timers, widgets, fetching with cleanup).

TL;DR

No external system → no Effect. Compute in render, reset with `key`, act in handlers, notify parents in the same event, subscribe with `useSyncExternalStore`.

5.7 Fetching data in Effects and race conditions

What it is

Fetching in an Effect means: when the component shows with some input (a person, a search query), request the matching data and put it in state. It is a legitimate Effect — the component is synchronising with a server — but it has a classic trap: a **race condition**, where two requests are in flight and the **slower, older** one finishes last and overwrites the correct result.

How it works — three versions

`code/05_fetch_race_conditions.jsx` (the fake API answers Alice in 60 ms and Bob in 10 ms; the user picks Alice, then quickly Bob):

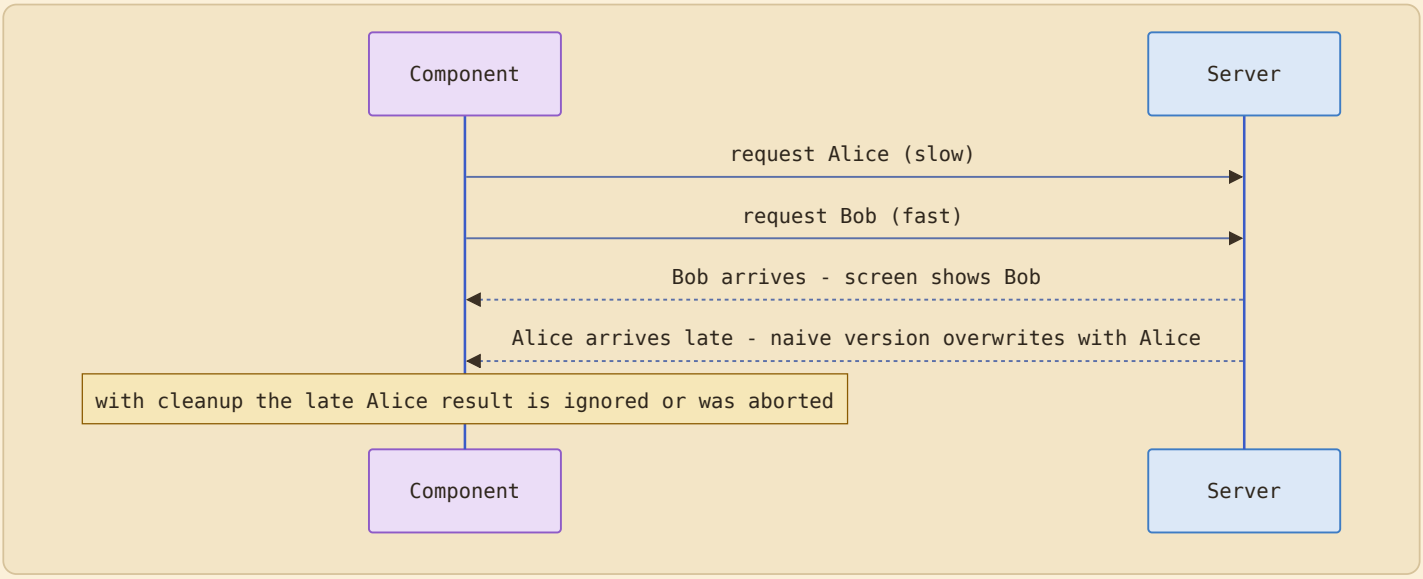
```
function BioNaive({ person }) {                          // 🔴 no cleanup
  const [bio, setBio] = useState(null);
  useEffect(() => { fetchBio(person).then(setBio); }, [person]);
  return <p>{bio ?? 'Loading…'}</p>;
}

function BioIgnore({ person }) {                         // ✅ ignore stale responses
  const [bio, setBio] = useState(null);
  useEffect(() => {
    let ignore = false;
    setBio(null);
    fetchBio(person).then((result) => { if (!ignore) setBio(result); });
    return () => { ignore = true; };                     // runs when person changes or on unmount
  }, [person]);
  return <p>{bio ?? 'Loading…'}</p>;
}

function BioAbort({ person }) {                          // ✅ cancel the request itself + loading/error states
  const [state, setState] = useState({ status: 'loading' });
  useEffect(() => {
    const controller = new AbortController();
    setState({ status: 'loading' });
    fetchBio(person, { signal: controller.signal })
      .then((bio) => setState({ status: 'success', bio }))
      .catch((err) => { if (err.name !== 'AbortError') setState({ status: 'error', message: err.message }); });
    return () => controller.abort();
  }, [person]);
  if (state.status === 'loading') return <p>Loading…</p>;
  if (state.status === 'error') return <p role="alert">Could not load: {state.message}</p>;
  return <p>{state.bio}</p>;
}
```

Output:

```
BioNaive  selected Bob, screen shows: "This is Alice's bio."
BioIgnore selected Bob, screen shows: "This is Bob's bio."
BioAbort  selected Bob, screen shows: "This is Bob's bio." [ 'aborted Alice' ]
BioAbort  error state: Could not load: 404 Not Found
```



- **Naive**: the user selected Bob but sees Alice — the late response won.
- **Ignore flag**: each Effect run has its own `ignore` variable; the cleanup of the Alice run sets *its* flag, so Alice's result is dropped.
- **`AbortController`**: actually cancels the old request (saves bandwidth; `fetch` rejects with an `AbortError`, which you must not show as an error). Also models the three states you always need: **loading, error, success**.

With real `fetch`, check `response.ok` yourself — `fetch` only rejects on network failure, not on HTTP 404/500:

```
// (pattern — not run)
const res = await fetch(`/api/bio/${person}`, { signal });
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const data = await res.json();
```

Why fetching in Effects isn't the end of the story

Downside	Consequence
Starts only after render + paint	Slower than fetching during navigation or on the server
Parent fetches, then child renders and fetches	**Network waterfalls**
No cache	Going back and forth re-fetches everything
Boilerplate	Loading/error/abort/retry in every component
No deduplication	Two components needing the same data make two requests

That's why real apps use a **framework's data loading** (React Router loaders, Next.js Server Components — Chapters 9, 13) or a **client cache** such as **TanStack Query** (Chapter 6 §6.9), which handle caching, deduplication, retries, and races for you. React 19's `use()` with Suspense is another option for promises created outside render (Chapter 9).

Common mistakes

- `useEffect(async () => …)` — setup must not return a promise.
- No cleanup → race conditions and "state update on unmounted component" style bugs.
- Showing "Error: AbortError" after a cancellation — filter it out.
- Putting the fetched data *and* a copy of it filtered into state — keep the data, derive the filtered list.

In the interview

> **30-second answer:** "Fetching in an Effect is fine if you handle races: either an `ignore` flag set in cleanup, or an `AbortController` aborted in cleanup, so a stale response can't overwrite a newer one. You also need loading and error states and to check `response.ok`. In production I'd rather use a data library like TanStack Query or framework loaders, which add caching, deduplication, retries and avoid waterfalls."

TL;DR

Fetch-in-Effect needs cleanup (ignore flag or `AbortController`) plus loading/error states. For real apps, prefer TanStack Query or framework loaders.

5.8 `useRef` for values

What it is

```
const ref = useRef(initialValue);   // -> { current: initialValue }, the same object on every render
```

A **ref** is a plain object with a single `current` property that React keeps for the component's lifetime. You can read and change `ref.current` freely, and **changing it does not trigger a re-render**.

Analogy: state is the scoreboard everyone watches — change it and the stadium updates. A ref is a note in the coach's pocket — useful to the coach, invisible to the crowd.

	useState	useRef
Returns	`[value, setValue]`	`{ current: value }`
Changing it	Via setter → **re-render**	Assign `ref.current = x` → **no re-render**
Value during a render	A fixed snapshot	Mutable; could change at any time
Read it during render?	Yes	**No** (except lazy initialisation)
Use for	Anything shown on screen	IDs of timers, DOM nodes, latest-value holders, instances of non-React objects

How it works

`code/06_use_ref_values.jsx`:

```jsx
// 1. Ref vs state: changing a ref never re-renders; the screen shows a stale value until something else renders.
function Compare() {
  const [stateClicks, setStateClicks] = useState(0);
  const refClicks = useRef(0);
  return (
    <>
      <button onClick={() => { refClicks.current++; console.log('  ref is now', refClicks.current); }}>ref +1</button>
      <button onClick={() => setStateClicks(stateClicks + 1)}>state +1</button>
      <p>state: {stateClicks} | ref (as of last render): {refClicks.current}</p>   {/* reading in render: a bug, shown on purpose */}
    </>
  );
}

// 2. Stopwatch: the interval ID is needed by a later event but never shown -> ref.
function Stopwatch() {
  const [startTime, setStartTime] = useState(null);
  const [now, setNow] = useState(null);
  const intervalRef = useRef(null);
  function handleStart() {
    setStartTime(Date.now()); setNow(Date.now());
    clearInterval(intervalRef.current);
    intervalRef.current = setInterval(() => setNow(Date.now()), 10);
  }
  function handleStop() { clearInterval(intervalRef.current); }
  const secondsPassed = startTime != null && now != null ? (now - startTime) / 1000 : 0;
  // ... <output>{secondsPassed.toFixed(3)} s</output> + Start / Stop buttons
}

// 3. Lazy-initialise an expensive object once (the only OK "write during render" pattern).
function Player() {
  const engine = useRef(null);
  if (engine.current === null) engine.current = new VideoPlayerEngine();   // runs on the first render only
  // ...
}
```

Output:

```
  ref is now 1
  ref is now 2
after 2 ref clicks  : state: 0 | ref (as of last render): 0
after 1 state click : state: 1 | ref (as of last render): 2
stopwatch ran        : true | still frozen 50 ms after Stop: true
engines created over 4 renders: 1
```

1. The ref really was 2, but the screen said 0 until an unrelated state change re-rendered. That is why **anything displayed must be state**.
2. The stopwatch keeps the interval ID in a ref: "Stop" (a later event) needs it, the UI doesn't. Storing it in state would cause a pointless re-render.
3. `useRef(new VideoPlayerEngine())` would construct an engine on **every** render and throw it away; the `if (ref.current === null)` pattern creates it once. (For state, `useState(() => …)` does the same job — Chapter 3 §3.2.)

Rules

- **Don't read or write `ref.current` during rendering** (except the lazy-init pattern). Rendering must be pure; a value React can't track makes output unpredictable — and the React Compiler (Chapter 10) assumes you follow this rule.
- Read and write refs in **event handlers and Effects**.
- A ref is **per component instance**, like state. A module-level variable would be shared by all instances.
- The "`usePrevious`" hook you may see online (writing the previous value into a ref in an Effect and reading it during render) breaks the rule above; store the previous value in state as in §5.6 case 5 instead.

In the interview

30-second answer: "`useRef` returns a stable `{ current }` object that persists across renders; mutating it doesn't re-render. I use it for values the UI doesn't display — timer IDs, DOM nodes, instances of non-React

libraries, the latest value for a callback — and I only touch it in handlers and Effects, not during render. Anything shown on screen belongs in state."

Self-check

1. Why did the screen show `ref: 0` after two clicks? → Changing a ref doesn't re-render.
2. Where should a `setInterval` ID live if only "Stop" needs it? → A ref.
3. Is `const r = useRef(expensive())` efficient? → No, `expensive()` runs every render; lazy-init with `if (r.current === null)`.

TL;DR

`useRef` = a persistent box whose changes don't re-render. Use it for non-visual values; use state for anything on screen; don't touch refs during render.

5.9 Refs to DOM nodes

What it is

Pass a ref object to a built-in element — `<input ref={inputRef} />` — and React puts the **DOM node** in `inputRef.current` after it creates the element (and sets it back to `null` when the node is removed). Then you can call browser APIs React doesn't wrap: `focus()`, `scrollIntoView()`, `getBoundingClientRect()`, `play()`/`pause()` on media, `showModal()` on `<dialog>`.

How it works

`code/07_dom_refs.jsx`, part 1 and 2:

```
// 1. The basic pattern: useRef(null) + ref={...}; React fills .current after the DOM node exists.
function FocusForm() {
  const inputRef = useRef(null);
  console.log('  during render, inputRef.current =', inputRef.current?.tagName ?? null);
  return (
    <>
      <input ref={inputRef} aria-label="name" />
      <button onClick={() => inputRef.current.focus()}>Focus the input</button>
    </>
  );
}

// 2. A list of refs: one ref holding a Map, filled by ref CALLBACKS. React 19: a callback may return a cleanup.
function CatGallery({ cats }) {
  const itemsRef = useRef(new Map());
  return (
    <>
      <nav>{cats.map((c) => <button key={c} onClick={() => itemsRef.current.get(c).scrollIntoView({ behavior: 'smooth'
})}>{c}</button>)}</nav>
      <ul>
        {cats.map((c) => (
          <li key={c}>
            <img alt={c} ref={(node) => {
              itemsRef.current.set(c, node);            // called with the node when attached
              return () => itemsRef.current.delete(c);  // cleanup when detached (React 19+)
            }} />
          </li>
        ))}
      </ul>
    </>
  );
}
```

Output:

```
1 focus:
  during render, inputRef.current = null
  focused element: name
2 list of refs:
  scrollIntoView -> Jellylorum
  after removing Jellylorum, the Map holds: Tom, Maru
```

- During the first render the ref is still `null` — the DOM node doesn't exist yet. React sets refs during the **commit**, before layout Effects run. Use refs in handlers and Effects.
- You can't call `useRef` inside a loop (Hook rules), so for a list use **one ref holding a `Map`** and a **ref callback** per item.

Ref callbacks

A **ref callback** is a function passed as `ref`. React calls it with the DOM node when the element is attached. In **React 19** it can **return a cleanup function**, called when the node is detached — the example removes the cat from the Map. (Before React 19, and still if you return nothing, React calls the callback again with `null` on detach.) Two details:

- An inline callback is a new function each render, so React detaches (runs cleanup) and re-attaches it after every render. That's harmless for simple bookkeeping; wrap it in `useCallback` if the work is expensive.
- Arrow functions with an implicit return, like `ref={(n) => (instance = n)}`, now look like "returning a cleanup"; TypeScript rejects them. Use a block body `{ instance = n; }`.

When to use DOM refs — and when not

Good uses	Avoid
Focus management (modals, forms, "press / to search")	Changing text, classes, children by hand — let React render them
Scrolling an item into view	Removing nodes React created (React will crash or be confused)
Measuring size/position (§5.11)	Reading input values you could keep in state/`FormData` (unless uncontrolled, Chapter 4 §4.2)
Integrating non-React libraries (charts, maps, editors)	
Media: `play()`, `pause()`; `<dialog>`: `showModal()`	

Tip: to scroll to an item you *just* added with `setState`, the DOM doesn't have it yet inside the handler. Wrap the update in `flushSync(() => setItems(…))` from `react-dom` to commit synchronously, then scroll (Chapter 3 §3.4).

TL;DR

`ref={myRef}` → `myRef.current` is the DOM node after commit (`null` during the first render). Use ref callbacks + a `Map` for lists; React 19 callbacks may return a cleanup. Don't edit DOM that React manages.

5.10 Passing refs to your own components

What it is

By default a component's DOM nodes are private. When a parent legitimately needs a child's node — a design-system `<TextField>` that the page wants to focus — the child must **pass the ref through**.

- **React 19**: `ref` is an **ordinary prop** for function components. Destructure it and put it on the element.
- **React 18 and earlier**: you had to wrap the component in **`forwardRef`**, which passes `ref` as a second argument. It still works in React 19, but React has announced it will be deprecated; new code should use the prop. (Class components still don't receive `ref` as a prop.)
- **`useImperativeHandle(ref, () => api, deps)`** lets the child expose a **custom, limited object** instead of the raw DOM node — e.g. only `focus()` and `clear()`.

How it works

`code/07_dom_refs.jsx`, parts 3–5:

```jsx
// 3. React 19: `ref` is an ordinary prop for function components — no forwardRef needed.
function TextField({ label, ref }) {
  return <label>{label} <input ref={ref} /></label>;
}
// Legacy (React 18 and earlier) — still works in 19, but forwardRef is planned for deprecation.
const OldTextField = forwardRef(function OldTextField({ label }, ref) {
  return <label>{label} <input ref={ref} /></label>;
});

// 4. useImperativeHandle: expose a small, deliberate API instead of the raw DOM node.
function VideoLikeInput({ ref }) {
  const real = useRef(null);
  useImperativeHandle(ref, () => ({
    focus: () => real.current.focus(),
    clear: () => { real.current.value = ''; },
  }), []);
  return <input ref={real} aria-label="limited" defaultValue="draft" />;
}

// 5. React 19.3: a ref on a <Fragment> gives a FragmentInstance that acts on its children without a wrapper <div>.
function Toolbar() {
  const group = useRef(null);
  return (
    <>
      <Fragment ref={group}>
        <button>Bold</button><button>Italic</button>
      </Fragment>
      <button onClick={() => group.current.focus()}>focus toolbar</button>
    </>
  );
}
```

Output:

```
3 ref as a prop:
  new-style ref -> INPUT | old-style ref -> INPUT
  focused: New
4 useImperativeHandle:
  exposed keys: [ 'focus', 'clear' ]
  value now "" | focused: limited
5 Fragment ref:
  focused: Bold
```

- Both styles delivered the `<input>` to the parent; the React 19 version is just a prop.
- With `useImperativeHandle` the parent could only `focus` and `clear` — it can't restyle or remove the node. This is good encapsulation for component libraries.
- **Fragment refs** (stable in React 19.3) give a `FragmentInstance` with methods such as `focus()` (focuses the first focusable child), `addEventListener`, `observeUsing(observer)` and `getClientRects()` — useful when you'd otherwise add a wrapper `<div>` only to hold a ref.

Migration

Old	New (React 19)
`const X = forwardRef((props, ref) => …)`	`function X({ ref, ...props }) { … }`
`X.displayName` needed for DevTools with anonymous `forwardRef`	Normal named function
`<Ctx.Provider value>` (related 19 change)	`<Ctx value>` (Chapter 6 §6.3)

React provides a codemod for the `forwardRef` migration.

In the interview

> **30-second answer:** "To give a parent access to a child's DOM node, in React 19 the child just accepts `ref` as a prop and passes it to the element; `forwardRef` is the older way and is on the path to deprecation. If I don't want to expose the whole node, `useImperativeHandle` exposes a small API like `focus`. Refs are an escape hatch — I use them for focus, scroll, measurement and third-party widgets, not for rendering."

TL;DR

React 19: `function Field({ ref }) { return <input ref={ref} /> }`. `forwardRef` = legacy. `useImperativeHandle` exposes a limited API. React 19.3 adds refs on `<Fragment>`.

5.11 `useLayoutEffect` — before the browser paints

What it is

`useLayoutEffect` has the same signature as `useEffect`, but it runs **synchronously after React updates the DOM and before the browser paints**. The browser waits for it. That makes it the right tool for one job: **measure layout and adjust before the user sees anything** — e.g. position a tooltip, or scroll a chat to the bottom without a visible jump.

`useInsertionEffect` runs even earlier (before layout effects) and exists for CSS-in-JS libraries to insert `<style>` tags; app code rarely needs it.

How it works

`code/08_layout_effect.jsx` — first the order of work, then a tooltip that must flip below its target when there's no room above:

```
function Order() {
  const [n, setN] = useState(0);
  console.log(`  render (n=${n})`);
  useInsertionEffect(() => { console.log('  useInsertionEffect  (for CSS-in-JS libraries: inject <style> first)'); });
  useLayoutEffect(() => { console.log('  useLayoutEffect     (DOM updated, not painted yet: measure here)'); return () => console.log('  layout cleanup'); });
  useEffect(() => { console.log('  useEffect           (after paint: subscriptions, fetching, logging)'); return () => console.log('  effect cleanup'); });
  return <button onClick={() => setN(n + 1)}>n={n}</button>;
}

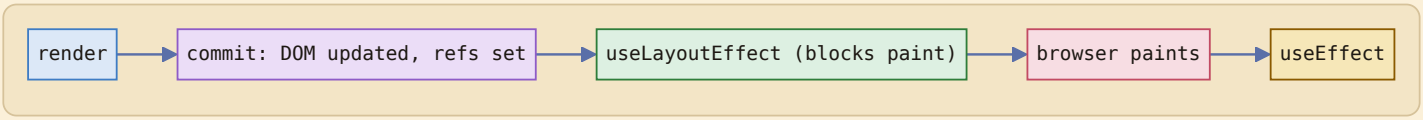
function Tooltip({ targetTop, useLayout }) {
  const ref = useRef(null);
  const [height, setHeight] = useState(0);                      // unknown until measured
  const measure = () => setHeight(ref.current.getBoundingClientRect().height);
  (useLayout ? useLayoutEffect : useEffect)(measure, []);
  const placeAbove = targetTop - height >= 0;                    // enough room above the target?
  const top = placeAbove ? targetTop - height : targetTop + 30;
  return <div ref={ref} data-h="80" role="tooltip" style={{ position: 'absolute', top }}>{placeAbove ? 'above' : 'below'} the button</div>;
}
// Rendered with createRoot + flushSync like a browser task; "first paint" = the DOM at the end of that task.
```

Output:

```
mount:
  render (n=0)
  useInsertionEffect  (for CSS-in-JS libraries: inject <style> first)
  useLayoutEffect     (DOM updated, not painted yet: measure here)
  useEffect           (after paint: subscriptions, fetching, logging)
click:
  render (n=1)
  useInsertionEffect  (for CSS-in-JS libraries: inject <style> first)
  layout cleanup
  useLayoutEffect     (DOM updated, not painted yet: measure here)
  effect cleanup
  useEffect           (after paint: subscriptions, fetching, logging)
  layout cleanup
  effect cleanup
useEffect      : first paint "above the button" top=50px | settled "below the button" top=80px
useLayoutEffect: first paint "below the button" top=80px | settled "below the button" top=80px
```

(The last two "cleanup" lines are the unmount between demos. The tooltip's 80 px height is faked because jsdom has no layout.)

- The tooltip can't know its height until it's in the DOM, so the first render guesses (height 0 → "above").
- With `useEffect`, that wrong first position reaches the screen; the fix arrives a frame later — a visible **flicker**.
- With `useLayoutEffect`, React measures and re-renders **before** the browser paints, so the user only ever sees the correct position.



Trade-offs

useEffect	useLayoutEffect
Doesn't block painting — **default choice**	Blocks painting — slow code here makes the app feel janky
May show one frame of the "before" state	No flicker for measure-then-adjust
Fine for 99% of Effects	Only for reading layout and synchronously fixing it
—	Does nothing during server rendering (the HTML is sent before any layout exists)

TL;DR

`useLayoutEffect` = runs after DOM update, before paint; use it to measure and reposition without flicker. Everything else: `useEffect`.

5.12 Putting it together — a city search

`code/citysearch.jsx` is a small search page that uses each tool for its proper job:

Need	Tool	Why
Debounce the query (wait until typing pauses)	Effect with `setTimeout` + cleanup (in a tiny `useDebounced` Hook — custom Hooks are Chapter 7)	Timer = external system; each keystroke's cleanup cancels the previous timer
Focus the box when the page opens	DOM ref + Effect	Focus is a browser API
Press `/` anywhere to focus search	Window listener in an Effect + cleanup	Global subscription while mounted
Search the (fake) API	Effect with `AbortController`	Server sync, race-safe
Keep the tab title in sync	Effect writing `document.title`	Browser API
Tooltip that flips if there's no room	Refs + `useLayoutEffect`	Measure before paint
Result count	Plain variable	Derived — no Effect

Key parts:

```
// Effect 1 (timer): a debounced copy of a value — only updates after `delay` ms without changes.
function useDebounced(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);                  // a new keystroke cancels the pending update
  }, [value, delay]);
  return debounced;
}

export default function CitySearch({ searchCities, delay = 250, initialQuery = '', tipFor = null }) {
  const [query, setQuery] = useState(initialQuery);
  const [result, setResult] = useState({ status: 'idle', cities: [] });
  const inputRef = useRef(null);
  const debouncedQuery = useDebounced(query.trim(), delay);

  // Effect 2 (DOM): focus the box when the page opens.
  useEffect(() => { inputRef.current.focus(); }, []);

  // Effect 3 (global listener): press "/" anywhere to jump to the search box.
  useEffect(() => {
    function onKeyDown(e) {
      if (e.key === '/' && document.activeElement !== inputRef.current) { e.preventDefault(); inputRef.current.focus();
}
    }
    window.addEventListener('keydown', onKeyDown);
    return () => window.removeEventListener('keydown', onKeyDown);
  }, []);

  // Effect 4 (network): fetch for the debounced query; abort the previous request on change.
  useEffect(() => {
    if (!debouncedQuery) { setResult({ status: 'idle', cities: [] }); return; }
    const controller = new AbortController();
    setResult((r) => ({ ...r, status: 'loading' }));  // keep old results visible while loading
    searchCities(debouncedQuery, { signal: controller.signal })
      .then((cities) => setResult({ status: 'done', cities }))
      .catch((e) => { if (e.name !== 'AbortError') setResult({ status: 'error', cities: [], message: e.message }); });
    return () => controller.abort();
  }, [debouncedQuery, searchCities]);

  // Effect 5 (browser API): keep the tab title in sync.
  useEffect(() => {
    document.title = debouncedQuery ? `“${debouncedQuery}” – City search` : 'City search';
  }, [debouncedQuery]);

  const count = result.cities.length;                   // derived
  // ... input, status line, <CityRow> list
}

// In CityRow: measure the tooltip before paint and flip it below the row if it would overflow the top.
// (It lives in the row, not in the tooltip: a parent's layout effect runs after the refs of its children are
attached.)
useLayoutEffect(() => {
  if (!hover) { setPos(null); return; }
  const row = rowRef.current.getBoundingClientRect();
  const tipHeight = tipRef.current.getBoundingClientRect().height;
  const below = row.top - tipHeight - 6 < 0;          // no room above -> show below
  setPos({ top: (below ? row.bottom + 6 : row.top - tipHeight - 6) + window.scrollY, left: row.left + 180, below });
}, [hover]);
```

`code/09_city_search.jsx` drives it (debounce 50 ms; the fake API is slow for short queries so a race can happen):

Output:

```
on open, focused: true | Start typing to search.
typed "osa"  -> Searching… | requests so far: [ 'osa' ]
             -> 1 city for “osa” [ 'Osaka' ] | title: “osa” – City search
"to" then "tor" -> 1 city for “tor” [ 'Toronto' ] | requests: [ 'to', 'aborted "to"', 'tor' ]
"error"      -> Error: Service unavailable
pressed "/"  -> focused: true
```

- Typing "o", "s", "a" produced **one** request, for "osa" — the debounce cleanups cancelled the earlier timers.
- When the user typed "r" while the "to" request was still in flight, the Effect's cleanup **aborted** it; only "tor" results were shown.
- Errors became a visible state; aborts didn't.
- The `/` shortcut worked because the listener was registered while the component was mounted.

In the browser (`code/citysearch.html`, styles in `citysearch.css`; the entry opens with the query "o" and Tokyo's tooltip shown):

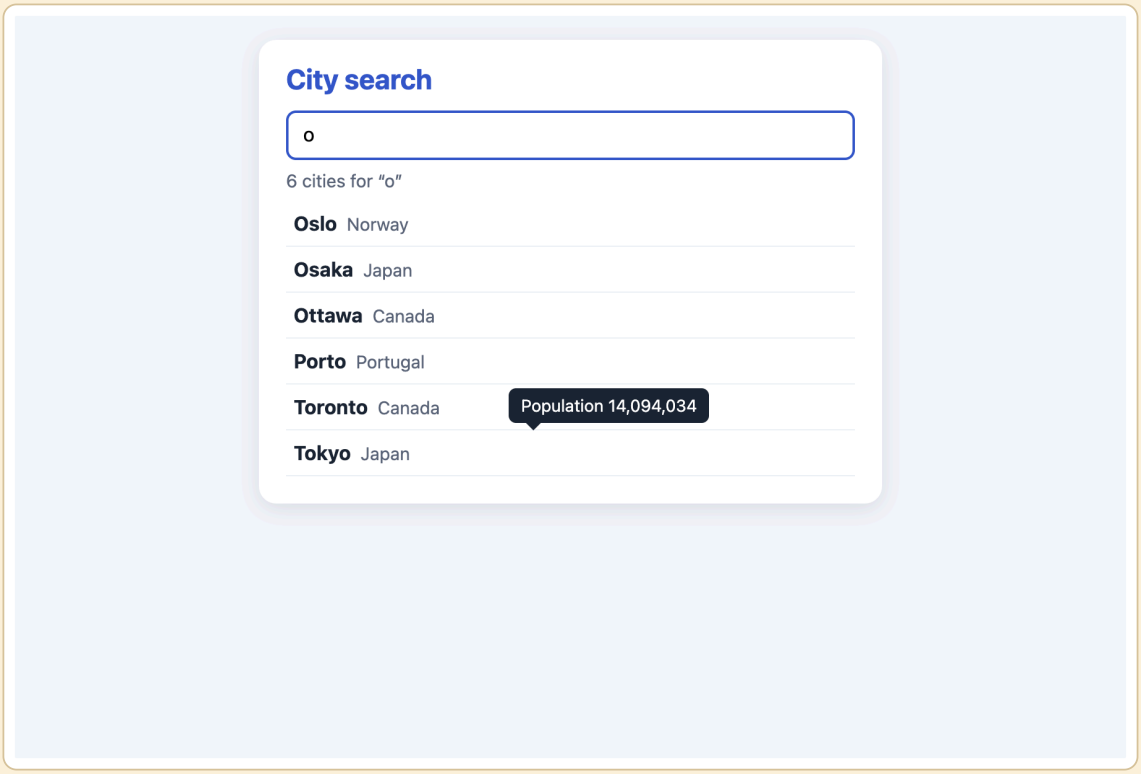


Figure 5.1 — The city search. The focus came from a ref + Effect, the list from an aborted-when-stale fetch Effect, and the tooltip's position from `useLayoutEffect` measuring before paint.

Design notes:

- **Every Effect names its external system** (timer, DOM focus, window events, network, document title). Nothing merely copies props into state.
- **Every Effect cleans up**, so StrictMode's double run and fast typing are both harmless.
- In a production app, Effect 4 would be one line with TanStack Query (`useQuery({ queryKey: ['cities', q], queryFn })` — Chapter 6 §6.9), which also caches results.

Chapter 5 TL;DR

Idea	Remember
Where code goes	Pure calc → render; user action → handler; sync with outside while on screen → Effect
`useEffect`	Runs after commit; `[]` = mount only; `[a]` = when `a` changes; no array = every render
Cleanup	Return a function that undoes setup: disconnect, clear, remove, abort
StrictMode	Dev-only setup → cleanup → setup; fix cleanups, don't suppress
Dependencies	Everything reactive the Effect reads; reduce by moving objects inside, updaters, splitting, `useEffectEvent`
Not needed	Derived data, resets (`key`), event logic, notifying parents, chains, app init
Fetching	Ignore flag or `AbortController`; loading/error; prefer TanStack Query / loaders
`useRef`	`{ current }`, persists, no re-render; not for displayed data; not read in render
DOM refs	`ref={r}` → node after commit; ref callbacks (+ cleanup in 19) for lists
Refs to components	React 19 `ref` prop; `forwardRef` legacy; `useImperativeHandle` for limited APIs; Fragment refs in 19.3
`useLayoutEffect`	Before paint; measure and adjust without flicker; otherwise use `useEffect`

Next: Chapter 6 — State Management.

Chapter 6 — State Management

Chapter 3 gave each component its own memory with `useState` and showed how to **lift state up** to share it. That works until an app grows: update logic gets scattered across many handlers, and data needed by distant components has to be passed through every layer in between. This chapter covers React's two built-in tools for that — `useReducer` (organise *how* state changes) and **Context** (deliver values *far down* the tree) — then the three libraries you'll meet in most codebases: **Redux Toolkit** and **Zustand** for client state, and **TanStack Query** for server state. Every library example runs with the real packages.

What you'll learn

#	Topic	You will be able to…
6.1	Kinds of state	classify state and pick the right home for each kind
6.2	`useReducer`	move update logic into a pure reducer; dispatch actions
6.3	Context	provide and read values anywhere below, override them, set defaults
6.4	Context performance	know when consumers re-render and how to limit it
6.5	Reducer + context	build a small app-wide store with only React
6.6	When to use a library	recognise the signals that built-ins aren't enough
6.7	Redux Toolkit	slices, store, `useSelector` / `useDispatch`, thunks, memoised selectors
6.8	Zustand	a store in a few lines; selectors, `useShallow`, middleware
6.9	TanStack Query	server state: queries, cache keys, staleness, mutations, invalidation
6.10	Choosing	a comparison table and a decision flow
6.11	Putting it together	a bookshop using each kind of state in its natural home

Code: `code/` — React 19.3, Redux Toolkit 2.12 + React Redux 9.3, Zustand 5.0, TanStack Query 5.104, run in jsdom.

6.1 Kinds of state — and where each belongs

What it is

"State management" sounds like one problem, but apps have several **kinds** of state with different needs. Most state-management pain comes from putting one kind in the wrong home — especially treating **server data** as if it were ordinary client state.

Kind	Examples	Needs	Natural home
Local UI state	Is this dropdown open? Current tab? Text in this input?	Nothing special	`useState` in the component (Chapter 3)
Complex local state	A form wizard, an editor with many actions	Clear update rules	`useReducer` (§6.2)
Shared / global client state	Current user, theme, cart, feature flags, a multi-step draft	Reachable from far away; efficient updates	Lift up → Context (§6.3) → Zustand / Redux (§6.7–6.8)
Server state (a cached copy of data owned by a server)	Product list, user profile, search results	Fetching, caching, de-duplication, refetching, staleness, retries	TanStack Query (§6.9), RTK Query, or framework loaders (Chapter 9)
URL state	Filters, page number, selected item ID	Shareable, survives reload, back button	The URL: route params and search params (Chapter 9)
Form state	Field values, errors, pending	Validation, submission	Form Actions / `useActionState`, or a form library (Chapter 4)
Derived state	Totals, filtered lists, "is valid"	Must never be out of sync	**Not state** — compute during render (Chapter 3 §3.8)

Analogy: a house. Things you use in one room stay in that room (local state). Things the whole family needs go in the hallway (context/global store). Food you buy from the shop goes in the fridge, which has expiry dates and needs restocking (server state in a cache). The house address is written on the door (URL state).

Why server state is different

Server data is **owned by someone else**: other users can change it at any moment, so your copy is always potentially **stale**. It needs loading and error states, caching, de-duplication of identical requests, background refetching, and invalidation after you change it. Writing all that by hand with `useEffect` + `useState` (Chapter 5 §5.7) or storing it in Redux is a major source of complexity. A dedicated server-state library usually removes most of the "global state" an app thought it had.

TL;DR

Classify first: local, complex local, shared client, server, URL, form, derived. Keep state as local as possible; put server data in a query cache; never store what you can derive.

6.2 `useReducer` — consolidate update logic

What it is

With `useState`, each event handler decides the next state itself. As a component grows (add, edit, delete, reorder, clear…), that logic spreads across many handlers. `useReducer` moves it into **one function outside the component**:

```
const [state, dispatch] = useReducer(reducer, initialArg, init?);
// reducer(state, action) -> nextState     (pure: no side effects, no mutation)
// dispatch(action)                        (action = a plain object describing what happened, e.g. { type: 'added',
text })
// init?: optional function; initial state = init(initialArg), computed once
```

Vocabulary: an **action** is a plain object describing **what happened** (by convention with a `type` string, e.g. `'added'`, plus any data). **Dispatch** means "send this action to the reducer". The **reducer** takes the current state and an action and returns the **next** state.

Analogy: a bank teller. You don't open the vault and change the balance yourself; you hand over a slip ("deposit 50") and the teller, following fixed rules, updates the ledger. The slips form a clear history of what happened.

How it works

The reducer (`code/tasks-reducer.js`) — note that it never mutates, and throws on unknown actions:

```
export function tasksReducer(tasks, action) {
  switch (action.type) {
    case 'added':
      return [...tasks, { id: action.id, text: action.text, done: false }];
    case 'toggled':
      return tasks.map((t) => (t.id === action.id ? { ...t, done: !t.done } : t));
    case 'renamed':
      return tasks.map((t) => (t.id === action.id ? { ...t, text: action.text } : t));
    case 'deleted':
      return tasks.filter((t) => t.id !== action.id);
    case 'cleared_done':
      return tasks.filter((t) => !t.done);
    default:
      throw new Error(`Unknown action: ${action.type}`);   // catches typos early
  }
}
```

`code/01_use_reducer.jsx`:

```
// 1. Reducers are plain functions: test them without React.
const s0 = [{ id: 1, text: 'Visit Kafka Museum', done: true }];
const s1 = tasksReducer(s0, { type: 'added', id: 2, text: 'Watch a puppet show' });
const s2 = tasksReducer(s1, { type: 'toggled', id: 2 });
// ...
// 2. In a component: dispatch describes WHAT happened; the reducer decides HOW state changes.
function createInitialTasks(username) {                // lazy initialiser: runs once
  console.log(`  (init for ${username})`);
  return [{ id: 1, text: `${username}'s first task`, done: false }];
}
function TaskApp({ username }) {
  const [tasks, dispatch] = useReducer(tasksReducer, username, createInitialTasks);
  return (
    <>
      <button onClick={() => dispatch({ type: 'added', id: nextId++, text: 'Buy stamps' })}>add</button>
      <button onClick={() => dispatch({ type: 'cleared_done' })}>clear done</button>
      <ul>
        {tasks.map((t) => (
          <li key={t.id}>
            <label><input type="checkbox" checked={t.done} onChange={() => dispatch({ type: 'toggled', id: t.id })} />
{t.text}</label>
            <button onClick={() => dispatch({ type: 'deleted', id: t.id })}>delete {t.text}</button>
          </li>
        ))}
      </ul>
    </>
  );
}
```

Output:

```
pure reducer: ["[x] Visit Kafka Museum","[x] Watch a puppet show"] | s0 untouched: true
typo caught  : Unknown action: addded
  (init for Ada)
after 2 adds       : [ "[ ] Ada's first task", '[ ] Buy stamps', '[ ] Buy stamps' ]
toggle + clear done: [ '[ ] Buy stamps', '[ ] Buy stamps' ]
delete one         : [ '[ ] Buy stamps' ]
```

- The reducer was **tested without rendering anything**, and the original array stayed untouched (immutability — Chapter 3 §3.6).
- A misspelt action type failed loudly instead of silently doing nothing.
- The **lazy initialiser** ran once, even though the component rendered several times.
- Event handlers became one-liners that say *what happened*; all the *how* is in one place.

Rules for reducers

1. **Pure**: same state + action → same result. No requests, timers, random IDs, or `Date.now()` inside — create those in the handler and put them in the action (`id: nextId++` above). StrictMode calls reducers twice in development to

catch impurity.

2. **Immutable**: return new objects/arrays. (Immer's `useImmerReducer` lets you write mutating code safely — the same trick Redux Toolkit uses, §6.7.)
3. **Actions describe events, not setters**: prefer `{ type: 'task_completed', id }` over `{ type: 'set_done', value: true }`. One user action = one dispatch, even if several fields change.
4. `dispatch` has a **stable identity**, so it's safe to pass down or omit from Effect dependencies.
5. Like setters, `dispatch` only affects the **next** render; reading `state` right after dispatching gives the old snapshot (Chapter 3 §3.3).

`useState` vs `useReducer`

	`useState`	`useReducer`
Code size	Small for simple values	More upfront (reducer + actions)
Many update types on one piece of state	Logic scattered in handlers	Centralised, readable
Testing	Needs a render	Test the reducer as a plain function
Debugging	Where did this change?	Log every action + state in one place
Next state depends on several fields	Several setters, risk of inconsistency	One transition computes everything

Rule of thumb: start with `useState`; switch to `useReducer` when you see several handlers modifying the same state in related ways, or bugs from inconsistent updates. You can mix both in one component.

In the interview

30-second answer: "`useReducer` takes a reducer — a pure function from state and action to the next state — and returns the state plus a stable `dispatch`. Handlers dispatch actions describing what happened, and all update logic lives in the reducer, which I can unit-test without React. I switch from `useState` when many handlers update the same state or the transitions get complex, and I combine it with context for app-wide state."

- *Can a reducer fetch data?* — No; it must be pure. Fetch in an event handler or Effect and dispatch the result.
- *What's the third argument?* — An initialiser: initial state is `init(initialArg)`, computed once.

Self-check

1. Why does the reducer throw on unknown types? → To catch typos immediately instead of silently ignoring actions.
2. Where does a random ID for a new item come from? → Generated in the handler and passed in the action.
3. What does `useReducer` return? → `[state, dispatch]`.

TL;DR

`const [state, dispatch] = useReducer(reducer, initial)`. Reducer = pure `(state, action) => next`. Dispatch events, not setters. Test reducers without React.

6.3 Context — pass data deeply without props

The problem it solves

Prop drilling is passing a prop through many components that don't use it, only so a deep child can. With a theme, the current user or a locale needed in dozens of places, it becomes noise and couples every layer to data it doesn't care about. **Context** lets a parent make a value available to **every component below it**, however deep.

Analogy: a building's Wi-Fi. Instead of running a cable (prop) from the router to each room through every room in between, the router broadcasts, and any device inside the building connects directly. A device in another building (outside the provider) gets nothing — or a default.

How it works

Three steps: **create**, **provide**, **use**.

`code/02_context_basics.jsx`:

```jsx
// 1. Create: the argument is the DEFAULT, used only when there is no provider above.
const ThemeContext = createContext('light');
const LevelContext = createContext(1);

// 2. Read: useContext(Ctx) — or use(Ctx), which (unlike Hooks) may be called conditionally.
function Button({ children }) {
  const theme = useContext(ThemeContext);
  return <button className={`btn-${theme}`}>{children}</button>;
}
function Heading({ children }) {
  if (!children) return null;                  // early return...
  const level = use(LevelContext);             // ...then read context: allowed with use(), not with useContext
  const Tag = `h${Math.min(level, 6)}`;
  return <Tag>{children}</Tag>;
}
// 3. A component can read the nearest value and provide a new one for its subtree.
function Section({ children }) {
  const level = useContext(LevelContext);
  return <section><LevelContext value={level + 1}>{children}</LevelContext></section>;   // React 19: <Ctx value>
}

function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext value={theme}>
      <Heading>Title</Heading>
      <Section>
        <Heading>Chapter</Heading>
        <Section>
          <Heading>Sub-section</Heading>
          <Heading />
          <Button>Save</Button>
          <ThemeContext.Provider value="high-contrast">   {/* older syntax, still works */}
            <Button>Nested override</Button>
          </ThemeContext.Provider>
        </Section>
      </Section>
      <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>toggle theme</button>
    </ThemeContext>
  );
}
```

Output:

```
headings : H1:Title  H2:Chapter  H3:Sub-section
buttons  : Save -> btn-light | Nested override -> btn-high-contrast
toggled  : Save -> btn-dark | Nested override -> btn-high-contrast
no provider -> btn-light (the default from createContext)
```

What happened:

1. Each `Section` read the level from the **nearest provider above** and provided `level + 1` to its children — so headings got the right tag without any `level` prop.
2. A component reads the **closest** provider; the inner `high-contrast` provider overrode the outer theme only for its subtree.
3. When the provided value changed (toggle), consumers **re-rendered automatically** with the new value.
4. With **no provider**, `useContext` returned the **default** passed to `createContext`.

Syntax notes (React 19)

Topic	Detail
Providing	`<MyContext value={…}>` is the React 19 way; `<MyContext.Provider value={…}>` still works and is planned for deprecation
Reading	`useContext(MyContext)` (a Hook: top level only) or `use(MyContext)` (can be inside `if`/after early returns)
Default value	Used only when no provider is above — not when the provider's value is `undefined`
Scope	Context passes through any intermediate components, including ones from libraries
Class components	`static contextType` (legacy)

Common uses

Theme, locale/i18n, current user/auth, feature flags, a router's location, a form library's form state, design-system settings (density, direction), and dependency injection for tests (provide a fake API client).

Before you use context

- **Try passing props first.** Explicit props through two or three levels are fine and easier to follow.
- **Try composition** — pass JSX as `children` so intermediate components don't need to know about the data at all (Chapter 2 §2.4): `<Layout sidebar={<UserMenu user={user} />} />`.
- Context is a **delivery mechanism**, not a state manager: it moves a value down; the value itself still lives in `useState`/`useReducer`/a store.

In the interview

30-second answer: "Context lets a component provide a value to its entire subtree without prop drilling: `createContext(default)`, provide with `<Ctx value>` (or `.Provider` in older code), read with `useContext` or `use`. Consumers get the nearest provider's value and re-render when it changes. I use it for rarely-changing, widely-needed values like theme, locale or auth, and I try props or composition first."

- *What if there's no provider?* — The default from `createContext`.
- *Difference between `useContext` and `use(Context)`?* — `use` can be called conditionally (after early returns, in loops); `useContext` follows Hook rules.

TL;DR

`createContext(default)` → `<Ctx value={v}>` → `useContext(Ctx)`/`use(Ctx)`. Nearest provider wins; consumers re-render on change. Props and composition first.

6.4 Context performance caveats

What it is

When a provider's `value` changes — compared with `Object.is` — **every component that reads that context re-renders**, even if it's wrapped in `memo` (a wrapper that skips re-rendering when props are unchanged — Chapter 10) and even if it only uses one field of the value. That's usually fine for rarely-changing values (theme, locale). It becomes a problem when a context holds frequently-changing data or a new object is created on every render.

How it works — measuring renders

`code/03_context_performance.jsx`:

```jsx
const AuthContext = createContext(null);
const UserBadge = memo(function UserBadge() {           // reads the user
  count('UserBadge'); const { user } = useContext(AuthContext); return <span>{user}</span>;
});
const LogoutButton = memo(function LogoutButton() {     // only needs the function
  count('LogoutButton'); const { logout } = useContext(AuthContext); return <button onClick={logout}>logout</button>;
});
const Sidebar = memo(function Sidebar() { count('Sidebar(no context)'); return <nav>links</nav>; });

function App({ memoValue }) {
  const [user, setUser] = useState('ada');
  const [clock, setClock] = useState(0);                // unrelated state in the same component
  const logout = useCallback(() => setUser(null), []);
  const literal = { user, logout };                      // 🔴 new object every render
  const memoised = useMemo(() => ({ user, logout }), [user, logout]);   // ✅ same object until user changes
  return (
    <AuthContext value={memoValue ? memoised : literal}>
      <UserBadge /><LogoutButton /><Sidebar />
      <button onClick={() => setClock(clock + 1)}>tick</button>
    </AuthContext>
  );
}

// Split state and actions into two contexts: action-only consumers stop re-rendering.
function AuthProvider({ children }) {
  const [user, setUser] = useState('ada');
  const actions = useMemo(() => ({ logout: () => setUser(null), login: (u) => setUser(u) }), []);  // never changes
  return <UserContext value={user}><AuthActionsContext value={actions}>{children}</AuthActionsContext></UserContext>;
}
// render(<AuthProvider><Badge2 /><Logout2 /><Sidebar /></AuthProvider>)
```

Output (renders per component for each step):

```
mount (literal value)                       {"UserBadge":1,"LogoutButton":1,"Sidebar(no context)":1}
  unrelated state change ->                 {"UserBadge":1,"LogoutButton":1}
  user changed ->                           {"UserBadge":1,"LogoutButton":1}
mount (useMemo value)                       {"UserBadge":1,"LogoutButton":1,"Sidebar(no context)":1}
  unrelated state change ->                 {}
  user changed ->                           {"UserBadge":1,"LogoutButton":1}
split contexts: mount                       {"Badge2":1,"Logout2":1,"Sidebar(no context)":1}
  user changed ->                           {"Badge2":1}
```

Three lessons:

1. **Object literal as `value`** → a new object on every render of the provider → **all consumers re-render** on *unrelated* changes (the clock tick), defeating `memo`. Wrapping the value in `useMemo` (and functions in `useCallback`) fixed it. (The React Compiler does this memoisation automatically — Chapter 10.)
2. **Consumers re-render when anything in the value changes.** `LogoutButton` only needs `logout`, but re-rendered when `user` changed. **Splitting** into a state context and an actions context fixed it: the action-only consumer no longer re-renders.
3. `memo` components that **don't read** the context (`Sidebar`) are unaffected. And a provider component that receives its subtree as `children` (like `AuthProvider`) doesn't re-render that subtree when its own state changes — only the consumers.

Techniques summary

Problem	Fix
New object/function as `value` each render	`useMemo` for the value, `useCallback` for functions (or the React Compiler)
Some consumers only need actions/dispatch	Separate "state" and "dispatch/actions" contexts (§6.5)
One big context with unrelated values	Split per concern: `ThemeContext`, `AuthContext`, `CartContext`
High-frequency values (mouse position, text being typed, timers)	Don't put them in widely-used context; keep local or use a store with **selectors** (Zustand/Redux re-render only components whose selected slice changed)
Provider re-renders its whole subtree	Put state in a dedicated provider component that takes `children`

When not to use context

As a general-purpose global store for frequently changing data. Context has no **selectors** — a consumer can't say "re-render me only when `cart.count` changes". That's exactly what external stores add.

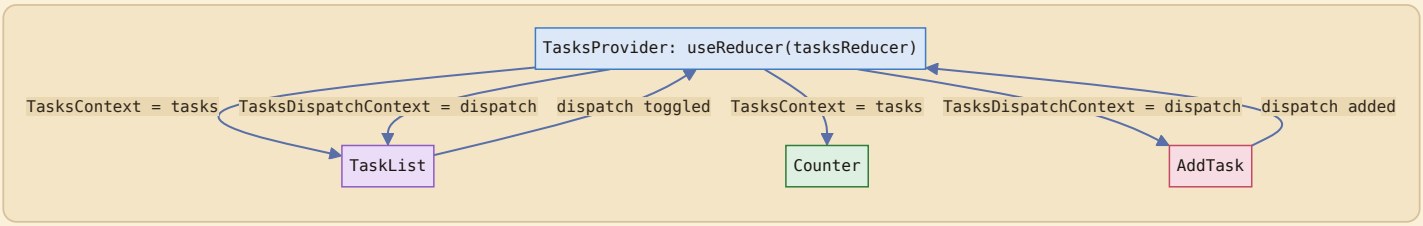
TL;DR

All consumers re-render when the value's identity changes. Memoise the value, split state from actions, split by concern, and use a selector-based store for fast-changing shared data.

6.5 Reducer + context — a store with only React

What it is

Combine the two tools: a **reducer** owns the state and its update rules; **two contexts** deliver the state and the `dispatch` function to any component. Wrap it all in a provider component and two custom Hooks, and you have a tidy, dependency-free store — a good fit for a feature-sized slice of state.



How it works

`code/04_reducer_plus_context.jsx`:

```jsx
// ---- TasksContext.jsx (in a real app: its own file exporting the provider + two Hooks)
const TasksContext = createContext(null);
const TasksDispatchContext = createContext(null);

export function TasksProvider({ initialTasks, children }) {
  const [tasks, dispatch] = useReducer(tasksReducer, initialTasks);
  return (
    <TasksContext value={tasks}>
      <TasksDispatchContext value={dispatch}>{children}</TasksDispatchContext>
    </TasksContext>
  );
}
export function useTasks() {
  const tasks = useContext(TasksContext);
  if (tasks === null) throw new Error('useTasks must be used inside <TasksProvider>');
  return tasks;
}
export function useTasksDispatch() {
  const dispatch = useContext(TasksDispatchContext);
  if (dispatch === null) throw new Error('useTasksDispatch must be used inside <TasksProvider>');
  return dispatch;                                    // stable identity: components using only this rarely re-render
}

// ---- Components deep in the tree: no tasks/onAdd/onToggle props passed through the middle.
function AddTask() {
  const dispatch = useTasksDispatch();
  const [text, setText] = useState('');
  return (
    <form onSubmit={(e) => { e.preventDefault(); dispatch({ type: 'added', id: nextId++, text }); setText(''); }}>
      <input aria-label="New task" value={text} onChange={(e) => setText(e.target.value)} /><button>Add</button>
    </form>
  );
}
function TaskList() { const tasks = useTasks(); const dispatch = useTasksDispatch(); /* ... checkboxes dispatch
'toggled' */ }
function Counter() { const tasks = useTasks(); return <p>{tasks.filter((t) => !t.done).length} left</p>; }
function Layout() { return <main><AddTask /><section><TaskList /></section><footer><Counter /></footer></main>; }  //
passes nothing down
```

Output:

```
tasks  : [ '[x] Philosopher’s Path', '[ ] Visit the temple' ]
footer : 1 left
outside provider -> useTasks must be used inside <TasksProvider>
```

- `Layout` passes **no props** — `AddTask`, `TaskList` and `Counter` connect directly.
- The **custom Hooks** hide the context objects (components never import them) and turn "forgot the provider" into a clear error instead of `null` bugs. Using `null` as the default makes that check possible.
- Because `dispatch` never changes, `AddTask` doesn't re-render when tasks change (§6.4 lesson 2).

Limits

It re-renders every `useTasks()` consumer on every change, has no selectors, no devtools/time travel, no middleware, and async flows (fetching) are up to you. Fine for a feature or a small app; for large shared client state with frequent updates, a library is more efficient and conventional.

> **TL;DR**

`useReducer` in a provider + separate state and dispatch contexts + `useX()` Hooks that throw outside the provider = a small, dependency-free store.

6.6 When to reach for a library

Built-ins (local state, lifting, `useReducer`, context) take most apps a long way. Signals that a library would pay off:

Signal	Likely answer
Lots of `useEffect` + `useState` for fetching; loading/error flags everywhere; duplicate requests; stale lists after edits	**TanStack Query** (or RTK Query / SWR / framework loaders) — this alone removes most "global state"
Shared client state that changes often and is read by many components (editor, canvas, dashboard filters, cart)	A store with selectors: **Zustand** (small) or **Redux Toolkit** (structured)
Large team, many features, need strict conventions, action logs, time-travel debugging, middleware	**Redux Toolkit**
State updates from outside React (WebSocket, timers, non-React code)	A store usable outside React (Zustand's `getState/setState`, Redux `store.dispatch`)
Complex flows with explicit states and transitions (checkout, wizard, media player)	A reducer, or a state-machine library such as XState
Fine-grained derived values (spreadsheet-like)	An atom library such as Jotai

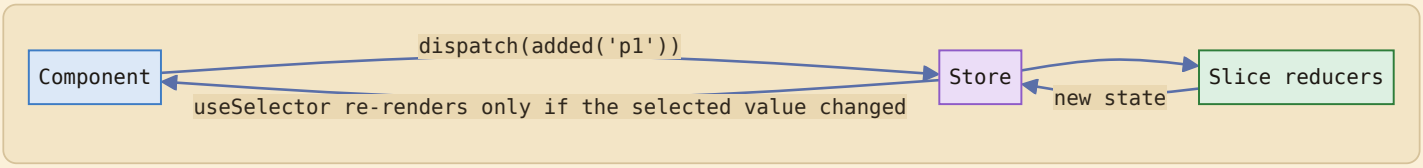
And signals that you *don't* need one: a handful of shared values that rarely change (theme, user) — context is enough; or "global state" that is really server data — use a query library.

6.7 Redux Toolkit

What it is

Redux is a pattern and library for keeping app-wide state in **one store**, changed only by **dispatching actions** to **reducers** (the same idea as `useReducer`, but global and with a big ecosystem). **Redux Toolkit (RTK)** is the official, recommended way to write Redux today: it removes the old boilerplate (hand-written action types, switch statements, spread-heavy immutable updates, store setup).

Term	Meaning
Store	Holds the whole state tree; `getState()`, `dispatch(action)`, `subscribe(listener)`
Slice	One feature's state + reducers + generated action creators (`createSlice`)
Action creator	Function returning an action object: `added('p1')` → `{ type: 'cart/added', payload: 'p1' }`
Selector	Function that reads (and maybe derives) data from state: `state => state.cart.lines`
Thunk	A function you dispatch for async logic; RTK's `createAsyncThunk` dispatches `pending`/`fulfilled`/`rejected` for you
Middleware	Code between `dispatch` and the reducer (logging, thunks, analytics)



How it works

`code/05_redux_toolkit.jsx`:

```
// ---- features/products/productsSlice.js : async data with createAsyncThunk
export const fetchProducts = createAsyncThunk('products/fetch', () => fakeApi.fetchProducts());
const productsSlice = createSlice({
  name: 'products',
  initialState: { items: [], status: 'idle', error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchProducts.pending, (state) => { state.status = 'loading'; })
      .addCase(fetchProducts.fulfilled, (state, action) => { state.status = 'succeeded'; state.items = action.payload;
})
      .addCase(fetchProducts.rejected, (state, action) => { state.status = 'failed'; state.error =
action.error.message; });
  },
});

// ---- features/cart/cartSlice.js : "mutating" code is safe here — Immer produces a new immutable state
const cartSlice = createSlice({
  name: 'cart',
  initialState: { lines: {} },                        // { [productId]: quantity }
  reducers: {
    added(state, action) { const id = action.payload; state.lines[id] = (state.lines[id] ?? 0) + 1; },
    removed(state, action) { delete state.lines[action.payload]; },
    cleared() { return { lines: {} }; },              // or return a brand-new state
  },
});
export const { added, removed, cleared } = cartSlice.actions;   // action creators generated for you

// ---- selectors: read and derive; createSelector memoises derived data
const selectLines = (state) => state.cart.lines;
const selectProducts = (state) => state.products.items;
export const selectCartTotal = createSelector([selectLines, selectProducts], (lines, products) =>
  products.reduce((sum, p) => sum + (lines[p.id] ?? 0) * p.price, 0));

// ---- app/store.js
export const makeStore = () => configureStore({ reducer: { products: productsSlice.reducer, cart: cartSlice.reducer }
});

// ---- components
function ProductList() {
  const { items, status } = useSelector((s) => s.products);
  const dispatch = useDispatch();
  if (status === 'loading') return <p>Loading…</p>;
  return <ul>{items.map((p) => <li key={p.id}>{p.name} ${p.price} <button onClick={() => dispatch(added(p.id))}>add
{p.name}</button></li>)}</ul>;
}
function CartTotal() {
  const total = useSelector(selectCartTotal);          // re-renders only when THIS value changes
  return <p>Total: ${total}</p>;
}
// render(<Provider store={store}><ProductList /><CartTotal /></Provider>); store.dispatch(fetchProducts()); click
"add" ×3
```

Output:

```
initial state : {"products":{"items":[],"status":"idle","error":null},"cart":{"lines":{}}}
after fetch   : Notebook, Pen | succeeded
cart lines    : {"p1":2,"p2":1} | Total: $10
renders       : {"CartTotal":3} (ProductList does not re-render: the products slice did not change)
memoised selector recomputed on repeat calls: 0 times
after cleared : Total: $0 | action object: {"type":"cart/added","payload":"p9"}
```

What each part did:

1. `createSlice` generated the reducer *and* action creators (`added('p9')` → `{ type: 'cart/added', payload: 'p9' }`). Its reducers **look** mutating (`state.lines[id] = …`) but RTK runs them through **Immer**, which produces a new immutable state. (Either mutate the draft *or* return a new value — not both.)

2. **`createAsyncThunk`** handled the async fetch and dispatched `pending` → `fulfilled`; the slice reacted with `extraReducers` to track `status`.
3. **`configureStore`** combined the slices and, by default, added the thunk middleware, Redux DevTools support, and development checks that warn if you mutate state outside a reducer or put non-serializable values (class instances, functions) in state.
4. **`useSelector`** subscribes a component to **just the value its selector returns**. Adding to the cart re-rendered `CartTotal` three times but `ProductList` zero times — the products slice was unchanged.
5. **`createSelector`** (from Reselect, re-exported by RTK) memoised the derived total: repeat calls with the same inputs didn't recompute.

Rules and good practices

- One store per app; **slices per feature** (`features/cart/cartSlice.js`). Put state in Redux only if other parts of the app care about it — form input text usually doesn't belong there.
- Keep state **serializable** (plain objects/arrays/primitives) — needed for DevTools, persistence and SSR.
- **Selectors must not return new objects/arrays on every call** (e.g. `useSelector(s => s.items.filter(...))`): the component would re-render on every dispatch (React Redux warns in development). Use `createSelector` or select primitives.
- Model actions as **events** (`cart/added`), not setters (`cart/setLines`).
- For server data, prefer **RTK Query** (included in RTK: `createApi`, auto-generated hooks such as `useGetProductsQuery`, caching and invalidation by tags) over hand-written thunks — same ideas as TanStack Query (§6.9).
- TypeScript: infer `RootState` and `AppDispatch` from the store and use pre-typed hooks (Chapter 11).

Trade-offs

Pros	Cons
Predictable one-way data flow; every change is an action you can log	More concepts and files than Zustand
Excellent DevTools: action log, state diff, time travel	Overkill for small apps
Conventions scale to large teams	Needs a `<Provider>`
Middleware ecosystem; RTK Query built in	Global by default — easy to over-centralise

In the interview

> **30-second answer:** "Redux keeps app state in one store changed only by dispatching actions to pure reducers. Redux Toolkit is the modern way: `createSlice` generates reducers and action creators and uses Immer so I can write mutating-style updates; `configureStore` sets up thunks, DevTools and safety checks; `createAsyncThunk` or RTK Query handle async data. Components read with `useSelector`, which re-renders only when the selected value changes, and write with `useDispatch`. I'd pick it for large apps with lots of shared client state and a need for strict conventions and tooling."

- *Why can `state.count++` appear in an RTK reducer?* — Immer gives a draft and produces a new immutable state from the changes.
- *Why might a component re-render on every dispatch?* — Its selector returns a new object/array each time; memoise with `createSelector`.
- *Redux vs Context?* — Context only delivers a value; Redux is a store with selectors, middleware and tooling, so consumers re-render selectively.

TL;DR

RTK: `createSlice` (reducers + actions, Immer), `configureStore`, `<Provider>`, `useSelector` (fine-grained), `useDispatch`, `createAsyncThunk`/RTK Query, `createSelector`. Structured and tool-rich; best for large apps.

6.8 Zustand

What it is

Zustand ("state" in German) is a small store library. You create a store with one function that returns the state *and* the actions; `create` gives back a **Hook**. Components call the Hook with a **selector** and re-render only when that selection changes. No provider, no action types, no reducers required.

How it works

`code/06_zustand.jsx`:

```
// ---- stores/useBearStore.js : state and the actions that change it live together
const useBearStore = create((set, get) => ({
  bears: 0,
  fish: 10,
  addBear: () => set((state) => ({ bears: state.bears + 1 })),     // set merges (shallowly) into the state
  eatFish: () => set((state) => ({ fish: Math.max(0, state.fish - get().bears) })),
  reset: () => set({ bears: 0, fish: 10 }),
}));

function BearCounter() { const bears = useBearStore((s) => s.bears); return <p>{bears} bears</p>; }
function FishCounter() { const fish = useBearStore((s) => s.fish); return <p>{fish} fish</p>; }
function Controls() {
  // several values at once: useShallow compares the picked object's fields, not its identity
  const { addBear, eatFish } = useBearStore(useShallow((s) => ({ addBear: s.addBear, eatFish: s.eatFish })));
  return <><button onClick={addBear}>add bear</button><button onClick={eatFish}>eat</button></>;
}
function Everything() { const s = useBearStore(); return <small>{s.bears}/{s.fish}</small>; }   // no selector

// Outside React: read, write, subscribe (tests, WebSocket handlers, other modules).
const unsub = useBearStore.subscribe((state, prev) => console.log(`  subscriber: bears ${prev.bears} ->
${state.bears}`));
useBearStore.getState().addBear();
useBearStore.setState({ fish: 99 });

// ---- Middleware: persist to localStorage.
const useSettings = create(persist(
  (set) => ({ theme: 'light', toggle: () => set((s) => ({ theme: s.theme === 'light' ? 'dark' : 'light' })) }),
  { name: 'settings', storage: createJSONStorage(() => localStorage) },
));

// ---- Common mistake: a selector that builds a NEW object each call, without useShallow.
function Broken() { const { bears, fish } = useBearStore((s) => ({ bears: s.bears, fish: s.fish })); return <p>
{bears}/{fish}</p>; }
```

Output:

```
after 2 bears: 2 bears | 10 fish | renders: {"BearCounter":2,"Everything(no selector)":2}
after eating : 8 fish | renders: {"FishCounter":1,"Everything(no selector)":1}
  subscriber: bears 2 -> 3
  subscriber: bears 3 -> 3
getState()   : {"bears":3,"fish":99} | screen: 99 fish
persisted    : {"state":{"theme":"dark"},"version":0}
new-object selector -> Maximum update depth exceeded. This can …  (fix: useShallow)
```

- **Selective re-renders**: adding bears re-rendered only `BearCounter`; eating fish only `FishCounter`. `Controls` (selecting stable action functions) never re-rendered. `Everything`, with **no selector**, subscribed to the whole store and re-rendered on every change — avoid that in real code.
- `set` **merges** the object you return into the state (one level deep, like the old class `setState`); pass `true` as a second argument to replace instead. `get()` reads current state inside actions.
- **Outside React**: `getState`, `setState` and `subscribe` work anywhere (the subscriber fired for both changes, including the `fish` one where bears stayed 3).
- **Middleware**: `persist` saved the settings store to `localStorage` (and rehydrates it on load); `devtools` connects to Redux DevTools; `immer` allows mutating-style `set`.

- **The classic Zustand 5 bug**: a selector returning a **new object** every call makes React think the store changed on every check → infinite loop. Select primitives separately, or wrap object/array selectors in `useShallow`.

Trade-offs

Pros	Cons
Tiny API; almost no boilerplate	Fewer conventions — large teams must agree on structure
No provider; usable outside React	Stores are module singletons: for SSR per-request state or tests you may need a store-per-provider pattern (`createStore` + context)
Selector-based re-renders	Less tooling than Redux (DevTools via middleware)
Middleware for persist/devtools/immer	Easy to grow one giant store — split into slices/stores

In the interview

> **30-second answer:** "Zustand creates a store with `create((set, get) => ({ state…, actions… }))` and returns a Hook. Components select what they need — `useStore(s => s.count)` — and re-render only when that value changes; for multiple values I wrap the selector in `useShallow`. It needs no provider, works outside React via `getState/setState`, and has middleware like `persist` and `devtools`. I choose it for shared client state when Redux's structure would be overkill."

TL;DR

`const useStore = create((set, get) => ({ …state, …actions }))`; read with selectors; `useShallow` for objects; `getState/setState/subscribe` outside React; `persist`/`devtools` middleware.

6.9 TanStack Query — server state

What it is

TanStack Query (formerly React Query) is a **cache for server data**. You describe *what* data you need with a **query key** and *how* to fetch it with a **query function**; the library handles loading and error states, caching, de-duplication, background refetching, retries, and marking data stale when you change it.

Concept	Meaning	Default
`queryKey`	Array that identifies the data, e.g. `['todos']`, `['user', id]`. Include every variable the fetch depends on	—
`queryFn`	Async function returning the data (or throwing)	—
`staleTime`	How long fetched data counts as **fresh** (no automatic refetch)	`0` (stale immediately)
`gcTime`	How long **unused** cached data is kept before garbage collection	5 minutes
Retries	Failed queries retried with exponential backoff	3 retries in the browser
Refetch triggers	While stale: component mount, window refocus, network reconnect	On
`status`	`'pending'` (no data yet), `'error'`, `'success'`	—
`fetchStatus`	`'fetching'`, `'paused'` (offline), `'idle'` — can be fetching while `success` (background refresh)	—

Analogy: a well-run library desk. Ask for a book by its catalogue number (query key); if a recent copy is on the desk you get it at once (cache), and if it's old the librarian fetches a new edition in the background (stale-while-revalidate). If two people ask at once, one trip to the stacks serves both (de-duplication).

How it works

`code/07_tanstack_query.jsx`:

```
function useTodos() {                                   // wrap each query in a small custom Hook
  return useQuery({ queryKey: ['todos'], queryFn: api.getTodos, staleTime: 30_000 });
}
function TodoCount() { const { data } = useTodos(); return <p>count: {data?.length ?? '…'}</p>; }
function TodoList() {
  const { data, isPending, isError, error, isFetching } = useTodos();
  if (isPending) return <p>Loading…</p>;                // no data yet
  if (isError) return <p role="alert">{error.message}</p>;
  return <ul aria-busy={isFetching}>{data.map((t) => <li key={t.id}>{t.title}</li>)}</ul>;
}
function AddTodo() {
  const queryClient = useQueryClient();
  const mutation = useMutation({
    mutationFn: api.addTodo,
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['todos'] }),   // mark stale + refetch
  });
  return <button disabled={mutation.isPending} onClick={() => mutation.mutate('Review chapter 6')}>{mutation.isPending
? 'Adding…' : 'Add'}</button>;
}
function UserCard({ id }) {                              // the key includes the variable -> one cache entry per id
  const { data, status, error } = useQuery({ queryKey: ['user', id], queryFn: () => api.getUser(id) });
  return <p>user {id}: {status === 'success' ? data.name : status === 'error' ? `error (${error.message})` : 'loading'}
</p>;
}

// Test/script settings: no retries (default is 3) and no 5-minute garbage-collection timers keeping Node alive.
const queryClient = new QueryClient({ defaultOptions: { queries: { retry: false, gcTime: Infinity }, mutations: {
gcTime: Infinity } } });
const App = ({ children }) => <QueryClientProvider client={queryClient}>{children}</QueryClientProvider>;
```

Output:

```
first render  : Loading… | count: …
loaded        : [ 'Write chapter 6' ] | count: 1 | requests: [ 'GET /todos' ]
remount       : shows 1 item(s) instantly from cache | requests: []
mutating      : button = Adding…
after mutation: [ 'Write chapter 6', 'Review chapter 6' ] | requests: [ 'POST /todos', 'GET /todos' ]
user switch   : user 1: User 1 (from cache at once; refetched in the background because staleTime is 0)
missing user  : user 404: error (User not found) | requests: [ 'GET /users/1', 'GET /users/2', 'GET /users/1', 'GET
/users/404' ]
cache keys    : ["todos"] ["user",1] ["user",2] ["user",404]
```

Reading it:

1. **De-duplication**: `TodoList` and `TodoCount` both asked for `['todos']` — **one** request.
2. **Caching**: after unmounting and remounting within `staleTime` (30 s), the list appeared **instantly with zero requests**.
3. **Mutations + invalidation**: `useMutation` exposed `isPending` for the button; `invalidateQueries(['todos'])` marked the list stale and refetched it — no manual "push the new item into state".
4. **Keys with variables**: each user ID got its own cache entry. Switching back to user 1 showed the cached name at once and, because the default `staleTime` is 0, refetched in the background (**stale-while-revalidate**).
5. **Errors** are a state (`status === 'error'`, `error.message`), not a crash. In the browser the failed request would first be retried 3 times; here retries are off.

What you no longer write

Compare with Chapter 5 §5.7: no `useEffect`, no `ignore` flag, no loading/error `useState`s, no cache, no refetch-after-save code. Race conditions are handled because results are stored per key.

More features (brief)

- **Optimistic updates** with `onMutate` (update the cache, return a rollback) or by rendering `mutation.variables` while pending.
- `enabled: false` for dependent queries (fetch B only when A's result exists).
- `placeholderData: keepPreviousData` for pagination without flashing a spinner.
- `useInfiniteQuery` for "load more"/infinite scroll; `useSuspenseQuery` for Suspense (Chapter 9).

- **Prefetching** (`queryClient.prefetchQuery`) on hover or in route loaders; **DevTools** panel.

Common mistakes

- Leaving a variable out of the key (`['user']` for every id) — different users overwrite each other's cache.
- Copying query `data` into `useState`/Redux — you lose updates; read from the query (derive filtered views in render).
- Creating the `QueryClient` inside a component body without `useState` — a new cache every render. Create it once (module level or `useState(() => new QueryClient())`).
- Expecting `isPending` during background refetches — use `isFetching` for that.

In the interview

30-second answer: "Server state is a cache of data someone else owns, so it needs fetching, caching, de-duplication, refetching and invalidation — TanStack Query does that. `useQuery({ queryKey, queryFn })` returns status, data and error; the key identifies the cache entry and must include every variable. `staleTime` controls freshness, `gcTime` how long unused data stays. Writes go through `useMutation`, and on success I invalidate the affected keys so they refetch. With it, most 'global state' disappears from Redux or context."

- *`isPending` vs `isFetching`?* — `isPending`: no data yet. `isFetching`: any request in flight, including background refetches.
- *`staleTime` vs `gcTime`?* — Fresh vs stale (refetch triggers) vs how long unused data remains in memory.

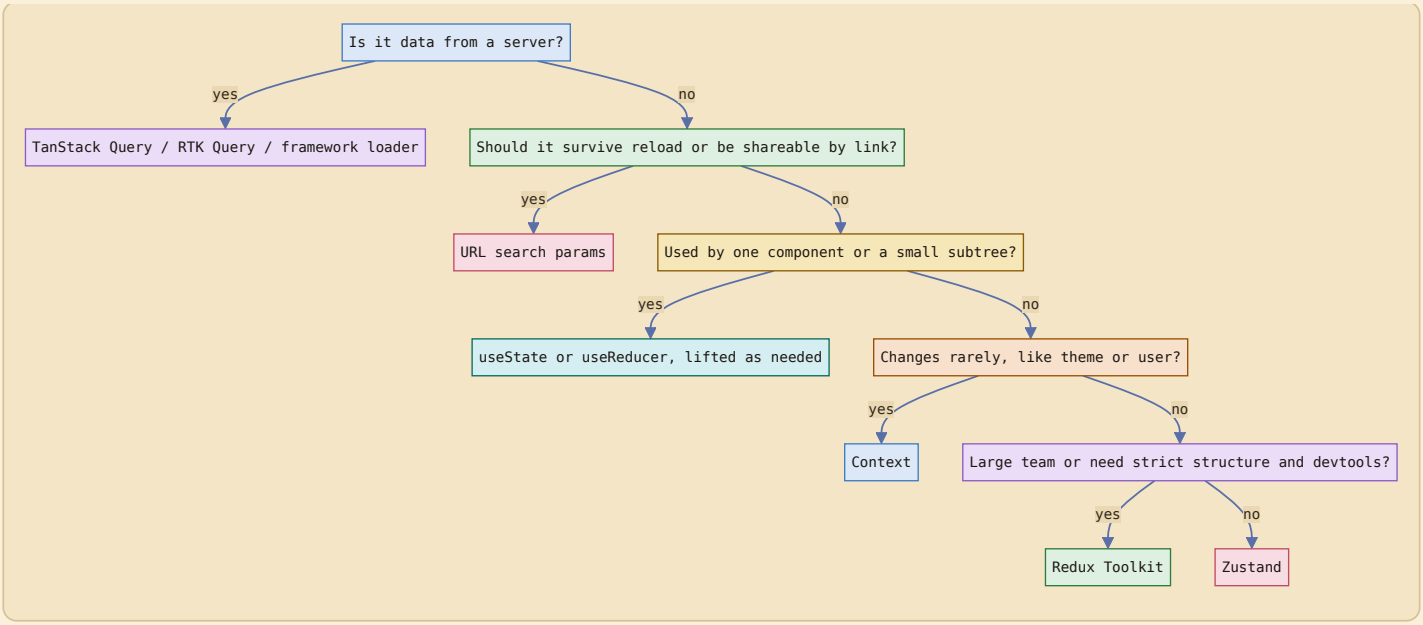
TL;DR

TanStack Query = server-state cache. `useQuery({ queryKey: [...vars], queryFn })`, `useMutation` + `invalidateQueries`. Defaults: `staleTime` 0, `gcTime` 5 min, 3 retries, refetch on focus/reconnect/mount.

6.10 Choosing — comparison and decision flow

	Context (+ reducer)	Redux Toolkit	Zustand	TanStack Query
For	Rarely-changing shared values; small feature stores	Large shared client state, strict conventions	Shared client state with little ceremony	**Server** state
Provider needed	Yes	Yes	No	Yes (`QueryClientProvider`)
Selective re-renders	No (all consumers)	Yes (`useSelector`)	Yes (selectors)	Per query/key
Async	Your code	Thunks, RTK Query	Plain async actions	Built in (it's the whole point)
DevTools	React DevTools only	Redux DevTools (time travel)	Via `devtools` middleware	TanStack Query DevTools
Outside React	No	`store.dispatch/getState`	`getState/setState`	`queryClient` methods
Extra bundle size	None	Moderate (RTK + React Redux)	Very small (about 1 kB gzipped)	Moderate

Other names you'll meet: **Jotai** (atoms — small independent pieces of state composed together), **MobX** (observable objects with automatic tracking), **Valtio** (proxy-based mutable state), **XState** (state machines), **SWR** (server-state library by Vercel, similar to TanStack Query), **Recoil** (Meta's atom library — its repository was archived in January 2025; don't start new projects with it).



In the interview

30-second answer: "I start by classifying the state. Server data goes into a query cache like TanStack Query; shareable UI state like filters goes in the URL; local state stays in components with `useState` / `useReducer`; rarely-changing app-wide values use context. Only frequently-changing shared client state needs a store — Zustand for simplicity, Redux Toolkit when the team benefits from its structure and tooling."

6.11 Putting it together — a bookshop

`code/shop.jsx` puts each kind of state in its natural home:

State	Home	Why
Book catalogue	TanStack Query (`['books']`)	Server data: cached, shared by `Catalog` and `Cart` with one request
Cart lines	Zustand store	Shared client state, updated often, needed in several places, cleared from outside React after checkout
Currency	Context	App-wide, changes rarely
Genre filter	`useState` in `Catalog`	Local UI state
Visible books, cart items, totals	Computed during render	Derived

Key parts:

```
// ---- Zustand: cart store (usable from any component, no provider)
export const useCart = create((set) => ({
  lines: {},                                                  // { [bookId]: qty }
  add: (id) => set((s) => ({ lines: { ...s.lines, [id]: (s.lines[id] ?? 0) + 1 } })),
  remove: (id) => set((s) => { const { [id]: _, ...rest } = s.lines; return { lines: rest }; }),
  clear: () => set({ lines: {} }),
}));

// ---- Context: currency formatter chosen once at the top
const CurrencyContext = createContext({ code: 'USD', locale: 'en-US' });
function useMoney() {
  const { code, locale } = useContext(CurrencyContext);
  return (n) => new Intl.NumberFormat(locale, { style: 'currency', currency: code }).format(n);
}

// ---- TanStack Query: the catalog
function useBooks(fetchBooks) {
  return useQuery({ queryKey: ['books'], queryFn: fetchBooks, staleTime: 60_000 });
}

function Catalog({ fetchBooks }) {
  const [genre, setGenre] = useState('all');                  // local UI state
  const { data: books, isPending, isError, error } = useBooks(fetchBooks);
  const add = useCart((s) => s.add);                           // select just the action -> never re-renders for cart
changes
  const money = useMoney();
  if (isPending) return <p className="muted">Loading catalog…</p>;
  if (isError) return <p role="alert">{error.message}</p>;
  const visible = genre === 'all' ? books : books.filter((b) => b.genre === genre);   // derived
  // ... genre buttons + list with "Add" buttons
}

function Cart({ fetchBooks }) {
  const { lines, remove, clear } = useCart(useShallow((s) => ({ lines: s.lines, remove: s.remove, clear: s.clear })));
  const { data: books = [] } = useBooks(fetchBooks);            // same key -> same cache, no extra request
  const money = useMoney();
  const items = books.filter((b) => lines[b.id]).map((b) => ({ ...b, qty: lines[b.id] }));
  const total = items.reduce((sum, i) => sum + i.qty * i.price, 0);   // derived
  // ... list with remove buttons, total, "Empty cart"
}

export default function Shop({ fetchBooks, queryClient = new QueryClient(), currency = { code: 'USD', locale: 'en-US' }
}) {
  return (
    <QueryClientProvider client={queryClient}>
      <CurrencyContext value={currency}>
        <div className="shop">
          <header><h1>Corner Bookshop</h1></header>
          <Catalog fetchBooks={fetchBooks} />
          <Cart fetchBooks={fetchBooks} />
        </div>
      </CurrencyContext>
    </QueryClientProvider>
  );
}
```

(The default `new QueryClient()` in the parameter list is fine here because `Shop` is the app root and renders once; in a component that re-renders, create the client once with `useState(() => new QueryClient())`.)

`code/08_bookshop.jsx` drives it with a euro/German-locale currency provided through context:

Output:

```
start      : Loading catalog…
catalog    : 4 books | catalog requests: 1 (Catalog and Cart share one query)
fiction    : [ 'The Hobbit', 'Dune' ]
cart       : Cart (3) -> 1 × The Pragmatic Programmer; 2 × Dune | Total 72,00 €
store      : {"b4":2,"b1":1}
removed    : 1 × The Pragmatic Programmer | Total 42,00 €
cleared    : Your cart is empty.
```

- One catalogue request served both panels; the filter was purely local; money formatting followed the context (`72,00 €`); the cart store was cleared from outside React, as a checkout callback would.

In the browser (`code/shop.html`, styles in `shop.css`; the entry seeds the cart store with two titles before rendering, using the default USD currency):

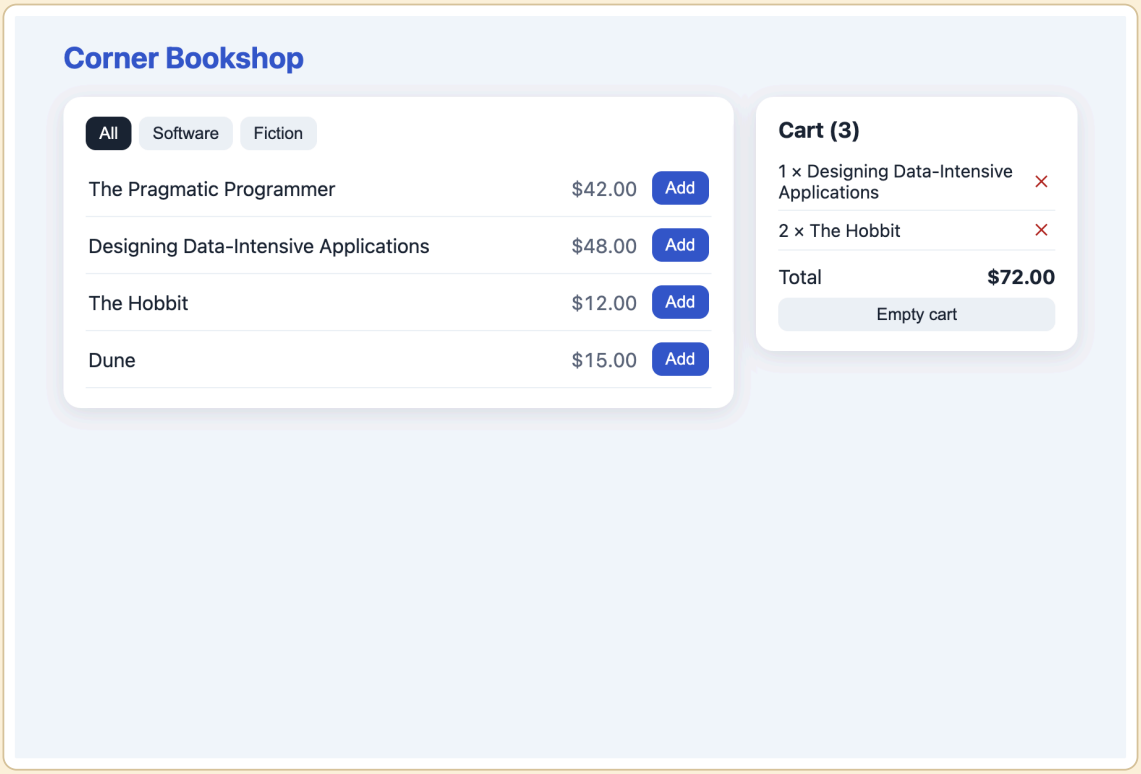


Figure 6.1 — The bookshop. The catalogue comes from the TanStack Query cache, the cart from a Zustand store seeded with `useCart.setState` before rendering, and the prices are formatted using the currency context.

Design notes:

- **No Redux, no Effects**: server data lives in the query cache; the store holds only what the client owns.
- **Selectors keep renders narrow**: `Catalog` selects only `add`, so it doesn't re-render when the cart changes.
- **Everything countable is derived** (visible books, item count, total).

Chapter 6 TL;DR

Idea	Remember
Kinds of state	Local, complex local, shared client, server, URL, form, derived — each has a home
`useReducer`	Pure `(state, action) => next`; dispatch events; test without React
Context	`createContext(default)`, `<Ctx value>`, `useContext` / `use`; nearest provider wins
Context performance	All consumers re-render on value change; memoise value, split state/actions, split by concern
Reducer + context	Provider + two contexts + `useX()` Hooks that throw outside the provider
Redux Toolkit	`createSlice` (Immer), `configureStore`, `useSelector` / `useDispatch`, thunks, `createSelector`, RTK Query
Zustand	`create((set, get) => …)`; selectors; `useShallow`; no provider; `persist` / `devtools`
TanStack Query	`useQuery({ queryKey, queryFn })`, `useMutation` + `invalidateQueries`; `staleTime` vs `gcTime`
Choosing	Server → query cache; shareable → URL; local → `useState`; rare global → context; busy global → Zustand/RTK

Next: Chapter 7 — Custom Hooks and the Rules of Hooks.

Chapter 7 — Custom Hooks and the Rules of Hooks

You have used a handful of built-in Hooks — `useState` (Chapter 3), `useActionState` (Chapter 4), `useEffect` and `useRef` (Chapter 5), `useReducer` and `useContext` (Chapter 6). This chapter does two things. First it explains the **Rules of Hooks** properly: what they are, *why* React needs them, and what breaks when you ignore them. Then it shows how to write your **own** Hooks — reusable pieces of stateful logic such as `useLocalStorage`, `useDebounce` and `useFetch` — and finishes with four built-in Hooks that are easiest to understand once you think in "Hooks": `useId`, `useSyncExternalStore`, `useTransition` and `useDeferredValue` (plus `useEffectEvent`, which makes callback-taking custom Hooks correct).

What you'll learn

#	Topic	You will be able to…
7.1	What a Hook really is	explain how React matches Hook calls to stored state
7.2	The Rules of Hooks	list both rules, the forbidden places, and the lint rules that enforce them
7.3	Custom Hooks	extract logic into `useSomething`, name it, design its inputs and outputs
7.4	`useLocalStorage`	persist state across reloads and tabs
7.5	`useDebounce`	wait until the user stops typing
7.6	`useFetch`	load data with loading/error states and no race conditions
7.7	Hooks that take callbacks — `useEffectEvent`	write `useInterval` without stale closures or restarts
7.8	`useId`	generate IDs for labels and ARIA attributes that survive SSR
7.9	`useSyncExternalStore`	subscribe to browser APIs and your own stores safely
7.10	`useTransition`	keep the UI responsive during expensive or async updates
7.11	`useDeferredValue`	let a slow part of the screen lag behind fast input
7.12	Putting it together	a user search built from four custom Hooks

Code: `code/` — React 19.3 in jsdom (a browser-like DOM for Node), driven with `@testing-library/react`. Examples that fetch data start a tiny `node:http` server on `127.0.0.1` inside the same script, so no internet is needed.

7.1 What a Hook really is

What it is

A **Hook** is a special function, whose name starts with `use`, that lets a component "hook into" a React feature: remembering state (`useState`), running side effects (`useEffect`), reading context (`useContext`), and so on. Hooks only work **while React is rendering a component**, because that is the only time React knows *which* component the call belongs to.

How it works — state is matched by call order

A component is just a function; its local variables disappear after it returns. So where does `useState` keep the value? React stores it *outside* your function, in an internal record for that component instance (called a **fiber**). That record holds a **list of Hook slots**. On every render React walks the list in step with your calls:

1. Before calling your component, React points a cursor at slot 0.
2. The first Hook call reads slot 0 and moves the cursor to slot 1; the second reads slot 1; and so on.
3. The **only** thing that tells React "this `useState` is the `name` one" is its **position** in the sequence. Hooks have no names or keys.

The toy model below reproduces that mechanism in 15 lines of plain JavaScript — no React at all.

`code/01_hooks_by_call_order.js`:

```
let slots = [];      // this component instance's remembered values
let index = 0;       // which slot the next useState call reads

function useState(initial) {
  const i = index++;                      // position = identity
  if (!(i in slots)) slots[i] = initial;   // first render: store the initial value
  const setState = (v) => { slots[i] = v; };
  return [slots[i], setState];
}
function render(Component, props) { index = 0; return Component(props); }

// A "component" that breaks the rule: it calls a Hook only when a condition is true.
function Profile({ showNickname }) {
  const [name] = useState('Ada');
  if (showNickname) {
    const [nick] = useState('Countess');   // slot 1 ONLY on some renders
    void nick;
  }
  const [age] = useState(36);              // slot 1 or 2 depending on the condition!
  return `name=${name} age=${age}`;
}

console.log('render 1 (showNickname=true): ', render(Profile, { showNickname: true }));
console.log('slots now:', JSON.stringify(slots));
console.log('render 2 (showNickname=false):', render(Profile, { showNickname: false }));
```

Output:

```
render 1 (showNickname=true):  name=Ada age=36
slots now: ["Ada","Countess",36]
render 2 (showNickname=false): name=Ada age=Countess
-> age read slot 1, which holds the nickname. State got mixed up.
```

On the second render the conditional call was skipped, so `useState(36)` became the *second* call and read slot 1 — the nickname. That is the whole reason the Rules of Hooks exist.

Analogy: a cloakroom that hands out coats by **queue position**, not by ticket. It works perfectly as long as the same people always queue in the same order. If one person skips the queue, everyone behind them gets the wrong coat.

Why React chose call order

The React team considered giving each Hook a name or key. Position won because it keeps Hooks cheap (no string keys to compare), makes custom Hooks composable (two custom Hooks can each call `useState` without name clashes) and lets the same Hook be used several times in one component (`useState` twice). The price is one simple rule: *call Hooks in the same order every render*.

TL;DR

React stores Hook state in a per-component list and matches calls by **position**. Change the number or order of calls between renders and values land in the wrong slots.

7.2 The Rules of Hooks

What it is

Two rules, from react.dev:

1. **Only call Hooks at the top level** of a function component or a custom Hook — never inside conditions, loops, or nested functions.
2. **Only call Hooks from React functions** — function components and custom Hooks, not ordinary JavaScript functions or class components.

The allowed and forbidden places

✅ Allowed	🔴 Not allowed
Top level of a function component body	Inside `if`, `&&`, `? :`, `switch` branches
Top level of a custom Hook body	Inside `for` / `while` loops
	After an early `return` (the Hook is skipped on some renders)
	Inside event handlers (`onClick={() => useState()}`)
	Inside the callbacks you pass to `useMemo`, `useReducer`, `useEffect`
	Inside `try` / `catch` / `finally` blocks
	In class components, or in plain helper functions

One exception: the `use` API (§9.11 of Chapter 9) *may* be called inside `if` and loops, because it does not keep a slot. It still may not be called inside `try/catch`.

What React does when you break rule 1

`code/02_rules_of_hooks_errors.jsx`:

```
// BAD: a Hook after an early return
function Greeting({ user }) {
  const [lang] = useState('en');
  if (!user) return <p>Please sign in ({lang})</p>;
  const [greeting] = useState('Hello');     // skipped when user is missing
  return <p>{greeting}, {user}</p>;
}

// BAD: a Hook inside a condition
function Tracker({ tracking }) {
  const [clicks] = useState(0);
  if (tracking) useEffect(() => {});
  return <p>clicks: {clicks}</p>;
}
// ... tryRender(label, firstUi, nextUi) renders, re-renders, prints the error ...

// GOOD: call every Hook unconditionally at the top, put the condition inside.
function GreetingFixed({ user }) {
  const [greeting] = useState('Hello');
  if (!user) return <p>Please sign in</p>;
  return <p>{greeting}, {user}</p>;
}
function TrackerFixed({ tracking }) {
  const [clicks] = useState(0);
  useEffect(() => { if (!tracking) return; /* start tracking */ }, [tracking]);
  return <p>clicks: {clicks}</p>;
}
```

Output:

```
  [dev warning] React has detected a change in the order of Hooks called by Greeting. This will lead to bugs and errors if not fixed. For more information, read the Rules of Hooks: https://react.dev/link/rules-of-hooks
Greeting: signed-out then signed-in -> Rendered more hooks than during the previous render.
Greeting: signed-in then signed-out -> Rendered fewer hooks than expected. This may be caused by an accidental early return statement.
  [dev warning] React has detected a change in the order of Hooks called by Tracker. This will lead to bugs and errors if not fixed. For more information, read the Rules of Hooks: https://react.dev/link/rules-of-hooks
Tracker : off then on               -> Rendered more hooks than during the previous render.
GreetingFixed     -> Hello, Ada
TrackerFixed      -> clicks: 0
```

React detects the changed count and **throws** ("Rendered more/fewer hooks…"), which crashes that part of the tree. If the count stays the same but the *kind* of Hook changes, development builds print the "change in the order of Hooks" warning and you get the mixed-up state from §7.1. The fix is always the same shape: **call the Hook unconditionally, move the condition inside it** (into the Effect body, into the returned JSX, or into the arguments).

Why rule 2 exists

A Hook needs a "currently rendering component" to attach its slot to. A plain function called from an event handler, a timer or module scope has none — React throws *"Invalid hook call. Hooks can only be called inside of the body of a function component."* (The same message appears if your app accidentally bundles **two copies of React**, e.g. a linked library with its own `react` — a classic monorepo bug.)

The linter: `eslint-plugin-react-hooks`

You should never have to find these bugs at runtime. The official ESLint plugin checks them as you type:

Rule	What it catches
`react-hooks/rules-of-hooks`	Hooks in conditions, loops, after early returns, in non-`use` functions
`react-hooks/exhaustive-deps`	Missing or unnecessary values in `useEffect` / `useMemo` / `useCallback` dependency arrays (Chapter 5)
Compiler-based rules (purity, refs, set-state-in-effect, …)	Code the React Compiler cannot safely optimise — included in the plugin's recommended preset in recent versions

Setup with ESLint 9's flat config (not run — Vite's React template already includes it):

```
// eslint.config.js
import reactHooks from 'eslint-plugin-react-hooks';
import { defineConfig } from 'eslint/config';

export default defineConfig([
  reactHooks.configs.flat.recommended,
]);
```

The linter recognises Hooks **by name**: any function called `useSomething` (with a capital letter after `use`) is checked as a Hook, and any function whose name starts with a capital letter is treated as a component. That is why naming matters (§7.3).

Common mistakes

Mistake	Fix
`if (!data) return <Spinner/>;` above a `useMemo`	Move all Hooks above the first `return`
Calling a Hook in a `.map()` to make one per item	Extract an `<Item>` component and call the Hook inside it
Calling `useContext` in an event handler "to get the latest value"	Read it at top level; the handler closes over it
Silencing `exhaustive-deps` with `// eslint-disable-next-line`	Almost always hides a stale-value bug; restructure (Chapter 5, §7.7)
Naming a Hook `getUser()`	Call it `useUser()` so the linter checks it and its callers

In the interview

30-second answer: "Two rules: call Hooks only at the top level of components or custom Hooks, and only from React functions. React stores Hook state in a list per component and matches calls by order, so the order must be identical every render — a Hook inside an `if` or after an early return shifts every later slot. React throws 'Rendered more/fewer hooks' when the count changes, and `eslint-plugin-react-hooks` catches it statically."

- *Why can `use` be conditional but `useState` can't?* — `use` doesn't store anything in a Hook slot; it reads a promise or context each time.
- *How do you conditionally run an effect?* — Always call `useEffect`; put the `if` inside its body and include the condition in the dependencies.
- *What else causes "Invalid hook call"?* — Calling a Hook outside a component, or two copies of React in the bundle.

Self-check

1. What identifies a `useState` call to React? → Its position among the component's Hook calls.

2. Is `const [a] = cond ? useState(1) : [0]` allowed? → No — the Hook is conditional.
3. Where do you put the condition instead? → Inside the Hook's callback or arguments, or in the JSX.
4. Which lint rule checks dependency arrays? → `react-hooks/exhaustive-deps`.

TL;DR

Top level only, React functions only. The rule protects call order; the linter enforces it.

7.3 Custom Hooks — reusing stateful logic

What it is

A **custom Hook** is a JavaScript function whose name starts with `use` and that **calls other Hooks**. It lets you pull a piece of stateful logic out of a component and reuse it in many components — the same way you extract a helper function, but for logic that needs state or effects.

Analogy: a recipe card. Two cooks follow the same card, but each has their **own pot**. The recipe (logic) is shared; the soup (state) is not.

The problem it solves

Before Hooks, sharing stateful logic needed wrapper patterns — **higher-order components** and **render props** (Chapter 14) — that nested the tree deeply ("wrapper hell"). A custom Hook is a plain function call: no extra components, no extra nesting, and you can call several.

How it works

1. Notice duplicated Hook logic in two components (a toggle, a subscription, a fetch).
2. Move it into `function useSomething(args) { … }`.
3. Return what callers need — a value, a tuple `[value, setter]`, or an object `{ data, error, status }`.
4. Every call site gets its **own** state: calling `useToggle()` in two components runs `useState` twice, in two different fibers.
5. The Hook re-runs on every render of its caller, so it always sees the latest arguments.

`code/03_custom_hook_basics.jsx`:

```
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = () => setOn(v => !v);
  return [on, toggle];                    // return whatever shape is convenient
}

function Spoiler({ title, children }) {
  const [open, toggleOpen] = useToggle();
  return (
    <section>
      <button onClick={toggleOpen}>{open ? 'Hide' : 'Show'} {title}</button>
      {open && <p>{children}</p>}
    </section>
  );
}

function DarkSwitch() {
  const [dark, toggleDark] = useToggle(true);
  return <button onClick={toggleDark}>theme: {dark ? 'dark' : 'light'}</button>;
}
// render two <Spoiler>s and a <DarkSwitch>; click "Show ending", then "theme"

// Not every helper is a Hook: this one calls no Hooks, so it should NOT start with "use".
function formatCount(n) { return n === 1 ? '1 item' : `${n} items`; }
```

Output:

```
start          : Show ending | Show twist | theme: dark
open "ending"  : Hide ending | The butler did it. | Show twist | theme: dark
toggle theme   : Hide ending | The butler did it. | Show twist | theme: light
plain helper   : 3 items
```

Opening the first spoiler did not open the second, and the theme switch has its own value: **custom Hooks share logic, never state**. If you *want* shared state, lift it up, put it in context, or use an external store (Chapter 6, §7.9).

Designing a good custom Hook

Guideline	Why
Name it `use` + CapitalLetter (`useOnlineStatus`)	The linter and React Compiler treat it as a Hook; readers know it may hold state
If it calls **no** Hooks, don't prefix it with `use`	It's a normal function; the prefix would wrongly restrict where it can be called
Name it after its **purpose**, not its mechanism	`useChatRoom(roomId)` beats `useEffectWithWebSocket`
Mirror built-in shapes where it helps	`useLocalStorage` returning `[value, setValue]` feels like `useState`
Return objects when there are 3+ values	`{ data, error, status }` is self-documenting and order-free
Keep it pure during render	Same rule as components: side effects go in Effects or handlers
Avoid "lifecycle" Hooks like `useMount(fn)`	They hide dependencies from the linter; write the Effect you mean
Add `useDebugValue(label)` for library Hooks	Shows a label next to the Hook in React DevTools (used in `code/hooks.js`)

When not to write one

- The logic is used once and is short — inline is clearer.
- It is **server data** (fetch + cache + refetch): a data library does this better than any hand-written Hook (TanStack Query, Chapter 9).
- You only want to share a *value*, not logic — use props or context.

Testing custom Hooks

Test them through a tiny component (as every example here does) or with `renderHook` from `@testing-library/react` — see Chapter 12.

In the interview

30-second answer: "A custom Hook is a function starting with `use` that calls other Hooks. It's how you reuse stateful logic — like a subscription or a debounced value — without wrapper components. Each call has independent state, it re-runs on every render so it sees fresh arguments, and it must follow the Rules of Hooks. The `use` prefix is what lets the linter check it."

- *Do two components using the same custom Hook share state?* — No; each call has its own state. Share state via lifting, context, or a store.
- *Custom Hooks vs HOCs/render props?* — Hooks compose without extra tree nesting and make data flow explicit; HOCs and render props are still valid but rarely needed now.

TL;DR

`useX` = a function that calls Hooks. It shares logic, not state. Name by purpose; don't prefix non-Hooks with `use`.

7.4 `useLocalStorage` — state that survives a reload

What it is

A Hook with the same API as `useState`, but whose value is saved in the browser's **`localStorage`** (a small per-site key–value store that keeps strings across reloads and browser restarts) and read back on the next visit.

How it works

1. **Lazy initial state**: pass a function to `useState` so storage is read once, on mount (reading storage on every render would be wasteful).
2. `JSON.parse` / `JSON.stringify` because storage only holds strings.
3. `try/catch` around every storage call: storage can throw (Safari private mode, quota full, disabled cookies) and stored JSON can be corrupted.
4. An Effect writes the value whenever it changes — writing is a side effect, so it does not belong in render.
5. A second Effect listens for the **`storage` event**, which the browser fires in *other* tabs of the same site when one tab changes a key — so all tabs stay in sync.

`code/04_use_local_storage.jsx`:

```jsx
export function useLocalStorage(key, initialValue) {
  // 1. Lazy initial state: read storage once, on the first render only.
  const [value, setValue] = useState(() => {
    try {
      const raw = window.localStorage.getItem(key);
      return raw !== null ? JSON.parse(raw) : initialValue;
    } catch {
      return initialValue;                  // corrupted JSON or storage blocked
    }
  });

  // 2. Write whenever the value (or key) changes.
  useEffect(() => {
    try { window.localStorage.setItem(key, JSON.stringify(value)); } catch { /* quota / private mode */ }
  }, [key, value]);

  // 3. Other tabs fire a "storage" event when they change the same key.
  useEffect(() => {
    function onStorage(e) {
      if (e.key === key && e.newValue !== null) setValue(JSON.parse(e.newValue));
    }
    window.addEventListener('storage', onStorage);
    return () => window.removeEventListener('storage', onStorage);
  }, [key]);

  return [value, setValue];                 // same shape as useState, so updaters work too
}

function Settings() {
  const [fontSize, setFontSize] = useLocalStorage('fontSize', 16);
  return (
    <div>
      <p>font size: {fontSize}px</p>
      <button onClick={() => setFontSize(s => s + 2)}>bigger</button>
    </div>
  );
}
```

Output:

```
first visit     : font size: 16px
after 2 clicks  : font size: 20px | stored = 20
after reload    : font size: 20px
other tab -> 24 : font size: 24px
corrupted JSON  : font size: 16px (fell back to the default)
```

Trade-offs and limits

Concern	Note
Server rendering	`localStorage` doesn't exist on the server. Reading it during render makes the server HTML (default value) differ from the client (stored value) → hydration mismatch. With SSR, render the default first and load the stored value in an Effect, or use `useSyncExternalStore` with a `getServerSnapshot` (§7.9).
Two components, same key, same tab	Each has its own `useState`; they don't see each other's changes (the `storage` event only fires in *other* tabs). If that matters, turn storage into an external store read with `useSyncExternalStore`.
Size	About 5 MB per origin in most browsers; strings only; synchronous (blocks the main thread). Large data → IndexedDB.
Security	Readable by any script on the page — never store tokens or personal data here (see the JS book's storage chapter).

TL;DR

Lazy-read once, write in an Effect, `try/catch` everything, listen to `storage` for other tabs. Beware SSR mismatches.

7.5 `useDebounce` — wait until the user stops

What it is

Debouncing delays an action until input has been quiet for a set time. A search box that fires a request on every keystroke sends 5 requests for "react"; debounced by 300 ms it sends 1, after the user pauses.

Analogy: a lift door that waits until nobody has walked in for a few seconds before closing.

How it works

1. Keep a second state variable, `debounced`, that starts equal to `value`.
2. Every time `value` changes, an Effect schedules `setDebounced(value)` after `delayMs`.
3. The Effect's **cleanup** cancels the previous timer. Because the Effect re-runs on every change, each keystroke cancels the last pending update — only the final value survives the pause.

`code/05_use_debounce.jsx`:

```jsx
export function useDebounce(value, delayMs = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);  // schedule
    return () => clearTimeout(id);           // a newer value cancels the pending one
  }, [value, delayMs]);
  return debounced;
}

function Search() {
  const [text, setText] = useState('');
  const query = useDebounce(text, 100);
  useEffect(() => { if (query) console.log(`  -> would search for "${query}"`); }, [query]);
  return <input aria-label="search" value={text} onChange={e => setText(e.target.value)} />;
}
// types r, re, rea, reac, react (one key every 30 ms), pauses, then types "react hooks"
```

Output:

```
typed "r"
typed "re"
typed "rea"
typed "reac"
typed "react"
(pause 150 ms)
  -> would search for "react"
typed "react hooks"
(pause 150 ms)
  -> would search for "react hooks"
```

Five keystrokes, one search. Note the input itself is **not** debounced — `text` updates immediately so typing feels instant; only the *derived* `query` waits.

Debounce vs throttle vs `useDeferredValue`

Technique	Behaviour	Use for
Debounce	Fire once after N ms of silence	Network requests while typing, autosave
Throttle	Fire at most once every N ms	Scroll/resize handlers, analytics
`useDeferredValue` (§7.11)	Re-render a slow part as soon as React has time; no fixed delay, interruptible	Expensive *rendering* (filtering a big list) — not network calls

A variant, `useDebouncedCallback(fn, ms)`, debounces a *function* instead of a value (useful for autosave on change). Libraries such as `use-debounce` provide both.

Common mistakes

- Forgetting the cleanup → every keystroke eventually fires.
- Debouncing the controlled input's own value → the caret lags and characters appear late.
- Creating the debounced function inline each render (`debounce(fn, 300)` in the body) → a new timer per render, so nothing is debounced. Keep it in a ref or use the value-based Hook above.

TL;DR

Timer in an Effect, cleared by the cleanup. Debounce the derived value, never the input.

7.6 `useFetch` — loading, error, success, and no races

What it is

A Hook that fetches a URL and returns `{ status, data, error }`. It packages the Effect-based fetching pattern from Chapter 5 so components don't repeat it.

The problem it solves — race conditions

If the URL changes while a request is in flight (the user clicks user 1, then quickly user 2), two responses are racing. Without cleanup, **whichever arrives last wins** — even if it is for the old URL.

`code/06_use_fetch.jsx` — the local server answers user 1 slowly (150 ms) and user 2 quickly (20 ms):

```jsx
// The naive version: no cleanup, so whichever response arrives LAST wins.
function useFetchNaive(url) {
  const [data, setData] = useState(null);
  useEffect(() => { fetch(url).then(r => r.json()).then(setData); }, [url]);
  return data;
}

export function useFetch(url) {
  const [state, setState] = useState({ status: 'idle', data: null, error: null });
  useEffect(() => {
    if (!url) return;
    const controller = new AbortController();          // lets cleanup cancel the request
    setState(s => ({ ...s, status: 'loading', error: null }));
    fetch(url, { signal: controller.signal })
      .then(res => { if (!res.ok) throw new Error(`HTTP ${res.status}`); return res.json(); })
      .then(data => setState({ status: 'success', data, error: null }))
      .catch(error => {
        if (error.name === 'AbortError') return;         // we cancelled it: not a real error
        setState({ status: 'error', data: null, error });
      });
    return () => controller.abort();                     // url changed or unmounted
  }, [url]);
  return state;
}

function Profile({ id }) {
  const { status, data, error } = useFetch(`${BASE}/api/users/${id}`);
  if (status === 'loading' && !data) return <p>Loading…</p>;
  if (status === 'error') return <p role="alert">Could not load user {id}: {error.message}</p>;
  if (!data) return null;
  return <p>User {id}: {data.name}{status === 'loading' ? ' (refreshing…)' : ''}</p>;
}
// render id=1, immediately switch to id=2, wait; then id=99 (does not exist)
```

Output:

```
naive         : Asked for user 2, showing: Ada <- WRONG user (race condition)
initial       : Loading…
switched to 2 : User 2: Grace
200 ms later  : User 2: Grace (the slow Ada response was aborted, not shown)
missing user  : Could not load user 99: HTTP 404
```

How it works

1. Each Effect run creates an `AbortController`; its `signal` is passed to `fetch`.
2. When `url` changes (or the component unmounts), React runs the cleanup → `controller.abort()` → the old `fetch` rejects with an `AbortError`, which we ignore.
3. `fetch` only rejects on *network* failure, so we check `res.ok` ourselves and turn 4xx/5xx into errors.
4. The state is one object with a `status` field, so impossible combinations (loading *and* error) can't happen.
5. Keeping old `data` while `status === 'loading'` lets the UI show "refreshing…" instead of flashing a spinner.

(An alternative to aborting is an `ignore` flag set in cleanup, as in Chapter 5; it prevents the stale `setState` but lets the request finish. Aborting also saves bandwidth.)

When not to use it

`useFetch` is a good exercise and fine for a one-off widget, but real apps quickly need **caching** (don't refetch what you just loaded), **deduplication** (two components, one request), **refetch on focus**, **retries**, **pagination** and **mutations with invalidation**. That list is exactly what TanStack Query or a router loader provides — see Chapter 9. Effects also start fetching only *after* render, which causes request **waterfalls** in nested components.

In the interview

30-second answer: "I'd keep `{status, data, error}` in one state object, start the fetch in an Effect keyed on the URL, pass an AbortController signal, and abort in the cleanup so a stale response can't overwrite a newer one.

I'd check `res.ok` because fetch doesn't reject on 404. For production I'd use TanStack Query or router loaders to get caching, dedupe and retries."

TL;DR

Fetch in an Effect, abort in cleanup, check `res.ok`, one status field. For real apps, reach for a data library.

7.7 Hooks that take callbacks — `useEffectEvent`

What it is

Some custom Hooks accept a function: `useInterval(callback, ms)`, `useEventListener('keydown', handler)`, `useWebSocket(url, onMessage)`. The Effect inside needs to call the **latest** callback (which sees the latest props and state), but you don't want the timer or connection to **restart** every time the parent re-renders and passes a new function.

`useEffectEvent` (stable since React 19.2) solves this. It wraps a function into an **Effect Event**: a stable function that, when called from inside an Effect, always runs the newest version of your callback. It is *not* reactive, so it doesn't go in the dependency array.

The problem it solves

Two bad options existed before:

- Put `callback` in the dependencies → the interval is torn down and recreated on every render.
- Leave it out (`[]`) → the Effect keeps the **first** callback forever (a **stale closure**) and the linter complains.

`code/11_use_effect_event.jsx`:

```jsx
export function useInterval(callback, delayMs) {
  const onTick = useEffectEvent(callback);        // always calls the newest callback
  useEffect(() => {
    if (delayMs === null) return;                  // null = paused
    timersStarted++;
    const id = setInterval(() => onTick(), delayMs);
    return () => clearInterval(id);
  }, [delayMs]);                                   // onTick is deliberately NOT a dependency
}

// The tempting-but-wrong version: an empty dependency array freezes the FIRST callback.
function useIntervalStale(callback, delayMs) {
  useEffect(() => {
    const id = setInterval(() => callback(), delayMs);
    return () => clearInterval(id);
  }, []);                                          // the linter would warn: missing 'callback', 'delayMs'
}

// Stopwatch: useIntervalImpl(() => stepsSeen.add(step), running ? 20 : null)
// wait 70 ms, click "step" twice (step becomes 3), wait 70 ms
```

Output:

```
useInterval (useEffectEvent): ticks used step values 1, 3
useIntervalStale ([] deps)  : ticks used step values 1
intervals started: 2 (mount + resume; changing step never restarted it)
```

The correct Hook saw `step = 3` after the clicks without restarting the timer; the stale one kept seeing `1`. Changing `delayMs` (pause → `null`, resume → `20`) *does* restart it, because the delay is a real dependency.

Rules for Effect Events

- Call them **only from inside Effects** (or functions called by Effects) — not during render and not from event handlers (a normal handler already sees the latest values).
- Don't pass them to other components or Hooks.
- Never list them in dependency arrays; the linter knows this.

- Use them for the *non-reactive* part of an Effect ("when this fires, read the latest X"), not as a way to hide real dependencies.

Before React 19.2 the same effect was achieved with the "latest ref" pattern: `const saved = useRef(callback); useEffect(() => { saved.current = callback; });` and then calling `saved.current()` in the interval. You will still see it in older code and libraries.

TL;DR

Callback-taking Hooks: wrap the callback in `useEffectEvent`, call it from the Effect, keep only truly reactive values in the dependencies.

7.8 `useId` — IDs for accessibility

What it is

`useId()` returns a **unique string ID** for the component instance calling it, stable across re-renders and identical between the server HTML and the hydrated client. It exists for HTML attributes that link elements by ID: `<label htmlFor>`, `aria-describedby`, `aria-labelledby`, `aria-controls`.

The problem it solves

A reusable `<PasswordField>` needs `id` on its input so its `<label>` can point at it. Hard-coding `id="password"` breaks as soon as the field appears twice on a page (duplicate IDs; the second label points at the first input). A global counter (`id = nextId++`) breaks with server rendering, because server and client may count in a different order, causing hydration mismatches.

`code/07_use_id.jsx`:

```
function PasswordField({ label }) {
  const id = useId();                       // one ID per component instance
  return (
    <div>
      <label htmlFor={`${id}-input`}>{label}</label>
      <input id={`${id}-input`} type="password" aria-describedby={`${id}-hint`} />
      <p id={`${id}-hint`}>At least 12 characters.</p>
    </div>
  );
}
function SignupForm() {
  return (<form><PasswordField label="Password" /><PasswordField label="Repeat password" /></form>);
}
// ... render, then renderToString + hydrateRoot, then an identifierPrefix
```

Output:

```
input id=_r_0_-input  aria-describedby=_r_0_-hint
input id=_r_1_-input  aria-describedby=_r_1_-hint
label lookup finds the 2nd input: true
its description: _r_1_-hint
server ids   : _R_1_-input, _R_1_-hint, _R_2_-input, _R_2_-hint
after hydrate: _R_1_-input, _R_1_-hint, _R_2_-input, _R_2_-hint
with identifierPrefix: _widget-R_0_-input
```

How it works

- The ID is derived from the component's **position in the tree** (its "parent path"), not from a counter, which is why server and client agree as long as they render the same tree.
- Call it **once** per component and derive related IDs with suffixes (`${id}-input`, `${id}-hint`).
- `identifierPrefix` (an option of `createRoot`, `hydrateRoot` and the server renderers) keeps IDs from two React roots on one page from colliding.
- The exact format is an implementation detail and has changed between React versions; never parse it or hard-code it in CSS or tests. (Query by role/label in tests instead — as the output shows, that works *because* the IDs are wired correctly.)

When not to use it

- **Not for list keys.** Keys must come from your data (`todo.id`); `useId` in a list item would produce a new key per item position and defeat the purpose.
- Not for database IDs or anything that must be globally unique beyond the page — use `crypto.randomUUID()` when *creating* data.

TL;DR

`useId()` → unique, SSR-safe ID per component for `htmlFor`/`aria-*`. One call, suffixes for related IDs. Never for keys.

7.9 `useSyncExternalStore` — subscribing to outside data

What it is

An **external store** is any data that lives outside React state and can change on its own: browser APIs (`navigator.onLine`, `matchMedia`, `window.innerWidth`, `localStorage`), third-party state libraries, a WebSocket cache. `useSyncExternalStore` lets a component **read** such a store and **re-render** when it changes, without the subtle bugs of doing it with `useState` + `useEffect`.

```
const snapshot = useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?);
```

Argument	Contract
`subscribe(callback)`	Start listening; call `callback` whenever the store changes; **return an unsubscribe function**. Should be the same function each render (define it outside the component) or React re-subscribes every render.
`getSnapshot()`	Return the current value. Must return the **same value (by `Object.is`)** if the store hasn't changed.
`getServerSnapshot()`	Optional: the value to use during server rendering and hydration. Required if the component renders on the server.

Why not `useState` + `useEffect`?

The Effect version subscribes *after* the first paint (so the first render may show a wrong value) and, with concurrent rendering, different components can read the store at slightly different moments during one render and show inconsistent values — called **tearing**. `useSyncExternalStore` reads the snapshot in a way React can check for consistency, and forces a synchronous re-render if the store changes mid-render.

`code/08_use_sync_external_store.jsx`:

```jsx
// ---- 1. A browser API: navigator.onLine + 'online'/'offline' events ----
function subscribeOnline(callback) {          // React passes a callback; call it on every change
  window.addEventListener('online', callback);
  window.addEventListener('offline', callback);
  return () => {                              // and return an unsubscribe function
    window.removeEventListener('online', callback);
    window.removeEventListener('offline', callback);
  };
}
function useOnlineStatus() {
  return useSyncExternalStore(
    subscribeOnline,                          // defined OUTSIDE the component: same function every render
    () => navigator.onLine,                   // getSnapshot (client)
    () => true                                // getServerSnapshot (used during SSR and hydration)
  );
}
function StatusBar() { return <p>{useOnlineStatus() ? 'Online' : 'Offline - changes will sync later'}</p>; }

// ---- 2. Your own tiny store (the pattern Redux/Zustand use internally) ----
function createStore(initial) {
  let state = initial;
  const listeners = new Set();
  return {
    getState: () => state,
    setState: (update) => { state = { ...state, ...update }; listeners.forEach(l => l()); },
    subscribe: (l) => { listeners.add(l); return () => listeners.delete(l); },
  };
}
const cart = createStore({ items: 0 });
function useCartCount() { return useSyncExternalStore(cart.subscribe, () => cart.getState().items); }

// ---- 3. The classic bug: getSnapshot returns a NEW object every time ----
function BadCart() {
  const snap = useSyncExternalStore(cart.subscribe, () => ({ ...cart.getState() })); // new object each call
  return <p>{snap.items}</p>;
}
```

Output:

```
start        : Online
wifi drops   : Offline - changes will sync later
wifi returns : Online
store update : header says 3 items | badge: 3
dev warning  : The result of getSnapshot should be cached to avoid an infinite loop
bad snapshot : Maximum update depth exceeded. This can happen when a component repeatedly calls setState inside
componentWillUpdate or componentDidUpdate. React limits the number of nested updates to prevent infinite loops.
```

How it works

1. On render React calls `getSnapshot()` and uses the result.
2. After commit it calls `subscribe(callback)`.
3. When the store changes, your code calls `callback`; React calls `getSnapshot()` again and re-renders only if the value differs (`Object.is`).
4. On unmount (or if `subscribe` changes) React calls the unsubscribe function.

Part 3 shows why step 3 needs a *stable* snapshot: a fresh object is "different" every time, so React re-renders forever. Fix: store immutable state and return the same object until it changes (as `cart.getState()` does), or select a primitive (`.items`).

Real-world examples

- **Zustand** and **React Redux** v8+ are built on `useSyncExternalStore` (Chapter 6).
- `useMediaQuery('(prefers-color-scheme: dark)')` subscribing to `matchMedia(...).addEventListener('change', …)` — used for theming in Chapter 8.
- Router libraries subscribe to the history/location store this way.

When not to use it

If you can keep the data in React state (`useState`, `useReducer`), do. Mutations of an external store can't be marked as non-blocking **transitions**; if a store update happens during a transition, React falls back to a blocking render — so prefer React state for UI that benefits from `useTransition`.

In the interview

> **30-second answer:** "`useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot)` is the official way to read mutable data from outside React. React subscribes after commit and re-reads the snapshot on each change, re-rendering when it differs. It prevents tearing in concurrent rendering. The snapshot must be cached — return the same reference while nothing changed — and `subscribe` should be stable. Zustand and React Redux use it internally."

- *What is tearing?* — Different parts of the UI showing different versions of the same store during one render.
- *What happens if `getSnapshot` returns a new array each time?* — An infinite re-render loop; React warns "should be cached".

TL;DR

External data → `useSyncExternalStore`. Stable `subscribe`, cached `getSnapshot`, `getServerSnapshot` for SSR.

7.10 `useTransition` — non-urgent updates

What it is

React 18 introduced **concurrent rendering**: React can prepare a new UI in the background, pause that work to handle something more urgent, and throw it away if it's out of date. You opt in by marking some updates as **transitions** (non-urgent).

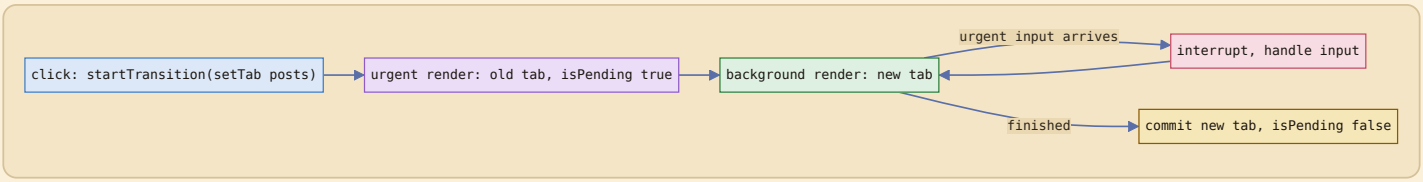
- **Urgent updates**: direct responses to input — typing, clicking, pressing. Users expect them instantly.
- **Transition updates**: moving the UI from one view to another — switching tabs, showing search results. A short delay is fine; freezing is not.

```
const [isPending, startTransition] = useTransition();
startTransition(() => setTab('posts'));          // this update is a transition
```

`isPending` is `true` while the transition is in progress, so you can dim the old UI or show a small spinner instead of a blank screen.

How it works

1. `startTransition(fn)` runs `fn` immediately; state updates made inside it are tagged as transition updates.
2. React first re-renders **urgently** with `isPending = true` and the *old* state — the current screen stays visible and interactive.
3. It then renders the new state in the background. If an urgent update (a keystroke, another click) arrives meanwhile, React interrupts, handles the urgent one, and restarts the transition with the newest state.
4. When the background render finishes, React commits it and `isPending` becomes `false`.



`code/09_use_transition.jsx`:

```
function SlowPosts() {                              // pretend this tab is expensive to render
  const start = performance.now(); while (performance.now() - start < 30) { /* busy 30 ms */ }
  return <p>500 posts</p>;
}
const TABS = { about: () => <p>About me</p>, posts: SlowPosts };

function TabApp() {
  const [tab, setTab] = useState('about');
  const [isPending, startTransition] = useTransition();
  log.push(`render tab=${tab} isPending=${isPending}`);
  const Tab = TABS[tab];
  return (
    <div style={{ opacity: isPending ? 0.6 : 1 }}>
      <button onClick={() => startTransition(() => setTab('posts'))}>Posts</button>
      <Tab />
    </div>
  );
}

// React 19: the function passed to startTransition may be ASYNC (an "Action").
function RenameForm() {
  const [name, setName] = useState('ada');
  const [isPending, startTransition] = useTransition();
  seen.push(`${isPending ? 'Saving…' : 'Idle   '} name=${name}`);
  function handleSave() {
    startTransition(async () => {
      const saved = await saveToServer('grace');
      startTransition(() => setName(saved));      // updates AFTER an await need their own startTransition
    });
  }
  return <button onClick={handleSave}>Save</button>;
}
```

Output:

```
--- click "Posts" (update inside startTransition) ---
render tab=about isPending=true
render tab=posts isPending=false
--- async action ---
Idle    name=ada
Saving… name=ada
Saving… name=ada
Idle    name=GRACE
```

The first render after the click still shows the **about** tab (dimmed, `isPending=true`); the expensive posts tab replaces it only when ready. In the async example `isPending` stays true from the click until the whole async function — including the network wait — has finished and the new name is applied. (React may render more than once while pending; what matters is the pending flag covering the whole operation.)

React 19: async transitions are "Actions"

Since React 19 you can pass an **async function** to `startTransition`. React calls such a function an **Action**: it tracks pending state for you, and it's the same mechanism behind form `action` props, `useActionState` and `useOptimistic` (Chapter 4). One current limitation: state updates that happen **after an** `await` are not automatically part of the transition — wrap them in another `startTransition`, as in the example.

Transitions and Suspense

If a transition causes a component to suspend (e.g. a route loads data, Chapter 9), React **keeps showing the old screen** instead of replacing already-visible content with a `<Suspense>` fallback. That's why routers wrap navigations in transitions: no flashing spinners between pages.

Trade-offs and caveats

Point	Detail
Can't control text inputs	Updates to an input's `value` must be urgent; a transition would make the caret lag. Keep two states (urgent text + transitioned results) or use `useDeferredValue`.
The callback runs synchronously	`startTransition` doesn't delay anything like `setTimeout`; it only lowers the priority of the updates.
Not for making slow code fast	It keeps the UI *responsive* while slow rendering happens; to reduce the work itself, see Chapter 10.
No `isPending` needed?	Import the standalone `startTransition` from `react` (usable outside components, e.g. in a store).

In the interview

> **30-second answer:** "`useTransition` marks state updates as non-urgent. React renders them in the background and can interrupt them for urgent input, while `isPending` lets me show the old UI dimmed instead of freezing or flashing a fallback. In React 19 the callback can be async — an Action — and `isPending` covers the whole async operation. I can't use it for the value of a controlled input."

- *Difference between `useTransition` and `useDeferredValue`?* — `useTransition` wraps the *update* (you own the setter); `useDeferredValue` wraps a *value* (useful when you only receive it, e.g. as a prop).
- *Is it a debounce?* — No: there's no timer; it runs as soon as possible and yields to urgent work.

TL;DR

`startTransition(() => setX(...))` = "update when you can, keep the old UI meanwhile". `isPending` for feedback. Async callbacks = Actions (React 19).

7.11 `useDeferredValue` — let part of the UI lag behind

What it is

`useDeferredValue(value)` returns a **deferred copy** of `value`. On an urgent update it first returns the **old** value (so the fast parts of the UI update immediately), then schedules a background re-render with the new value — the same interruptible, low-priority render a transition uses.

```
const deferredQuery = useDeferredValue(query);                // React 18+
const deferredQuery = useDeferredValue(query, initialValue);  // React 19: value for the very first render
```

How it works

1. The user types; `text` updates urgently and the component renders with `text = "re"` but `deferred = ""`.
2. A child wrapped in `memo` that receives `deferred` sees the **same prop** as before and is skipped — the input stays snappy.
3. React then re-renders in the background with `deferred = "re"`; now the slow child renders.
4. If another key arrives first, React abandons that background render and starts over with the newest value.

`code/10_use_deferred_value.jsx`:

```jsx
// memo is essential: it lets React skip SlowList when only the urgent `text` changed.
const SlowList = memo(function SlowList({ query }) {
  log.push(`  SlowList renders with query="${query}"`);
  const start = performance.now(); while (performance.now() - start < 20) { /* expensive */ }
  return <ul>{WORDS.filter(w => w.startsWith(query)).map(w => <li key={w}>{w}</li>)}</ul>;
});

function Search() {
  const [text, setText] = useState('');
  const deferred = useDeferredValue(text);           // lags behind `text` during typing
  const isStale = text !== deferred;
  log.push(`render text="${text}" deferred="${deferred}"${isStale ? '  (list is stale)' : ''}`);
  return (
    <>
      <input aria-label="q" value={text} onChange={e => setText(e.target.value)} />
      <div style={{ opacity: isStale ? 0.5 : 1 }}><SlowList query={deferred} /></div>
    </>
  );
}

// React 19: an initial value lets the FIRST render be cheap too.
function Preview({ text }) {
  const shown = useDeferredValue(text, '');          // first render uses '' then catches up
  log.push(`Preview render shown="${shown}"`);
  return <p>{shown}</p>;
}
```

Output:

```
render text="re" deferred=""  (list is stale)
render text="re" deferred="re"
  SlowList renders with query="re"
list now: react, redux, reducer, ref, render
Preview render shown=""
Preview render shown="hello"
```

The first render after the keystroke did **not** run `SlowList` at all (memo saw the same `query`), so the input updated instantly; the list caught up in the second, background render. `text !== deferred` is a free "stale" flag for dimming.

Common mistakes

Mistake	Fix
No `memo` on the slow child	Without it the child re-renders on the urgent pass too, and nothing is gained
Passing a new object each render (`useDeferredValue({ q })`)	Defer primitives or memoised values; a new object is "new" every time
Using it to reduce network requests	It has no delay; it doesn't limit requests — debounce instead (§7.5)
Expecting it to help with slow *effects*	It only defers rendering

With Suspense

If the deferred value makes a child suspend (it loads data for the new query), React keeps showing the **previous results** instead of a fallback — a smooth "stale-while-loading" search without extra code.

TL;DR

`useDeferredValue(v)` → urgent render with the old value, background render with the new one. Pair with `memo`. Not a debounce.

7.12 Putting it together — a user search

Four Hooks from this chapter compose into a small feature: `useId` wires the label, `useDebounce` limits requests, `useFetch` loads results without races, and `useLocalStorage` remembers recent searches. The final versions of the Hooks live in `code/hooks.js` (with a `useDebugValue` label on `useFetch`); the component is `code/UserSearch.jsx`:

```
export function UserSearch({ apiBase = '', initialQuery = '' }) {
  const id = useId();
  const [text, setText] = useState(initialQuery);
  const query = useDebounce(text.trim(), 250);
  const { status, data, error } = useFetch(query ? `${apiBase}/api/users?q=${encodeURIComponent(query)}` : null);
  const [recent, setRecent] = useLocalStorage('recent-searches', []);

  function remember(term) { setRecent(r => [term, ...r.filter(t => t !== term)].slice(0, 3)); }

  return (
    <div className="search">
      <label htmlFor={id}>Find a person</label>
      <input id={id} value={text} placeholder="Type a name…" onChange={e => setText(e.target.value)}
             onKeyDown={e => { if (e.key === 'Enter' && text.trim()) remember(text.trim()); }} />
      {recent.length > 0 && <p className="recent">Recent: {recent.join(', ')}</p>}
      {status === 'loading' && <p className="muted">Searching…</p>}
      {status === 'error' && <p role="alert">Search failed: {error.message}</p>}
      {status === 'success' && (data.length === 0
        ? <p className="muted">No one called “{query}”.</p>
        : <ul>{data.map(u => <li key={u.id}><strong>{u.name}</strong> <span>{u.role}</span></li>)}</ul>)}
    </div>
  );
}
```

`code/12_user_search.jsx` runs it against a local server that records every request:

Output:

```
typed a, ad, ada quickly -> requests sent: ["ada"]
results: Ada Lovelace Analyst
after "zed": No one called “zed”.
Recent: zed, ada | stored: ["zed","ada"]
```

In the browser (`code/search.html`; `fetch` is mocked in `search-main.jsx` so the page works offline):

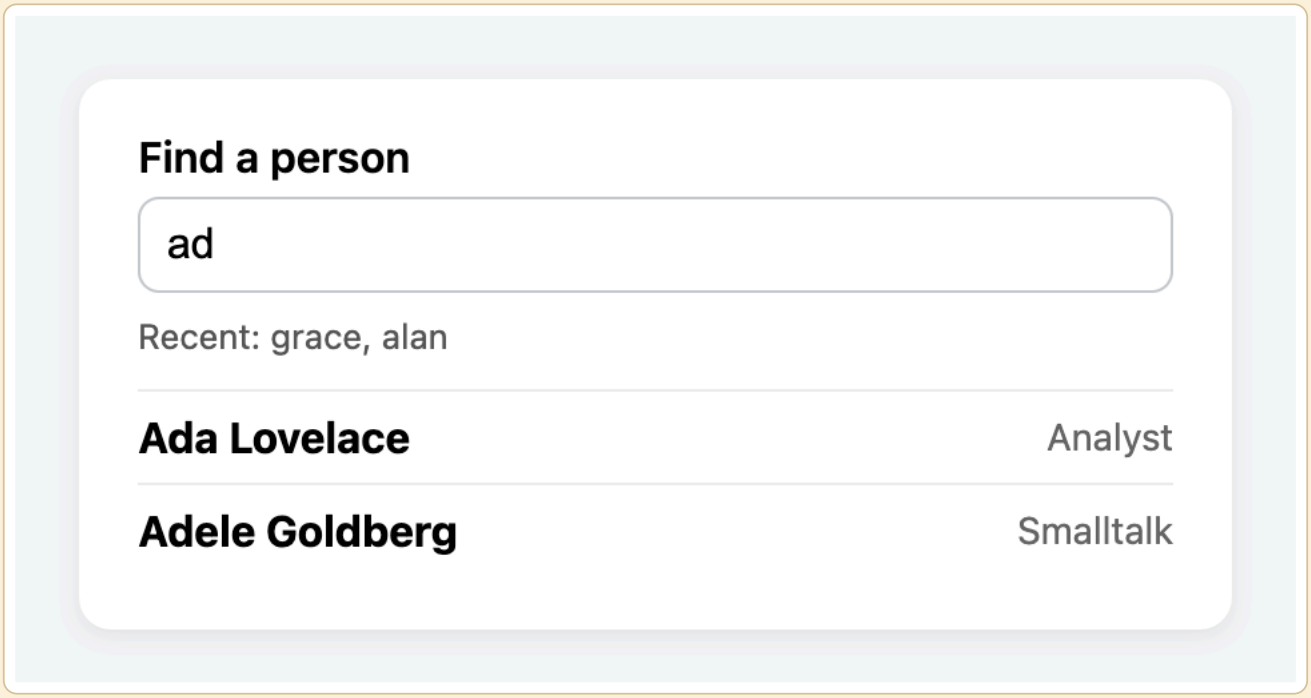


Figure 7.1 — `UserSearch` in Chrome. The component itself contains no timers, `fetch` calls or storage code — each concern is one Hook call.

Design notes:

- The component reads like a description of the feature; each Hook can be tested and reused on its own.
- Swapping `useFetch` for TanStack Query's `useQuery` later (Chapter 9) changes one line and adds caching.
- Three typed characters produced one request — debounce + abort together keep the server quiet and the results correct.

Chapter 7 TL;DR

Idea	Remember
How Hooks work	State lives in a per-component list, matched by **call order**
Rules of Hooks	Top level only (no `if`/loops/early-return/`try`/handlers); only in components and custom Hooks; enforced by `eslint-plugin-react-hooks`
Custom Hooks	`useX` functions that call Hooks; share **logic**, not state; name by purpose
`useLocalStorage`	Lazy read, write in Effect, `try/catch`, `storage` event; beware SSR mismatch
`useDebounce`	Timer in Effect + cleanup; debounce the derived value, not the input
`useFetch`	AbortController in cleanup, check `res.ok`, one status field — then prefer TanStack Query
`useEffectEvent`	Latest callback inside an Effect without re-subscribing (React 19.2)
`useId`	SSR-safe unique IDs for `htmlFor`/`aria-*`; never for keys
`useSyncExternalStore`	Read outside data; stable `subscribe`, cached `getSnapshot`, `getServerSnapshot`
`useTransition`	Mark updates non-urgent; `isPending`; async callbacks = Actions (19)
`useDeferredValue`	Deferred copy of a value; needs `memo`; `initialValue` in 19; not a debounce

Next: Chapter 8 — Styling.

Chapter 8 — Styling React Apps

React has no built-in styling system. It renders HTML elements, and you style those with CSS — the question is only *how you write, scope and ship* that CSS. This chapter compares the approaches you will meet in real codebases, shows each one running, and helps you choose. It assumes you know CSS itself (selectors, the cascade, specificity, custom properties); if not, read the HTML & CSS book first — CSS fundamentals and modern CSS & architecture.

What you'll learn

#	Topic	You will be able to…
8.1	The styling landscape	name the five families of approaches and the problem they all try to solve
8.2	Plain CSS files	use `className`, import CSS, and avoid global name clashes with conventions
8.3	Inline styles	use the `style` prop correctly — and know its limits
8.4	CSS Modules	scope class names to a file automatically
8.5	Composing class names	build `className` from props with a `clsx`-style helper
8.6	Tailwind CSS	style with utility classes, configure a theme, avoid its traps
8.7	CSS-in-JS	understand styled-components/Emotion vs zero-runtime tools, and why the trend moved
8.8	Theming and dark mode	build light/dark/system themes with CSS variables, without a flash
8.9	Component libraries	choose between styled kits, headless libraries and copy-in components
8.10	Choosing an approach	pick a strategy for a new project

Code: `code/`. Node programs print what React puts in the DOM; the visual results are screenshots of real pages in headless Chrome (bundled with esbuild; Tailwind compiled with the Tailwind v4.3 CLI).

8.1 The styling landscape

The problem every approach solves

CSS was designed for documents: every rule is **global**, and conflicts are settled by the **cascade** (origin, specificity, order). In a large app built from hundreds of components, that causes three recurring pains:

1. **Name collisions** — two teams both write `.title`; whichever file loads last wins.
2. **Dead CSS** — nobody dares delete a rule because they can't tell which components still use it.
3. **Dynamic styles** — colours or sizes that depend on props and state.

Every approach below is a different answer to these three problems.

Family	How you write styles	Scoping	Runtime cost	Typical tools
Global CSS files	`.css` files, `className="card"`	Naming conventions (BEM)	None	Plain CSS, Sass
Inline styles	`style={{ color: 'red' }}`	Per element	Tiny	Built into React
CSS Modules	`.module.css`, `className={styles.card}`	Automatic, per file	None	Vite, Next.js, webpack
Utility-first CSS	`className="p-4 rounded-xl bg-white"`	Nothing to scope — classes are single-purpose	None	**Tailwind CSS**, UnoCSS
CSS-in-JS	Styles in JS (`styled.div\`…\``)	Automatic, generated names	Runtime libs: yes; zero-runtime: none	styled-components, Emotion; vanilla-extract, StyleX, Panda

The same card, styled five ways (`code/gallery.html`, components in `code/cards.jsx`):

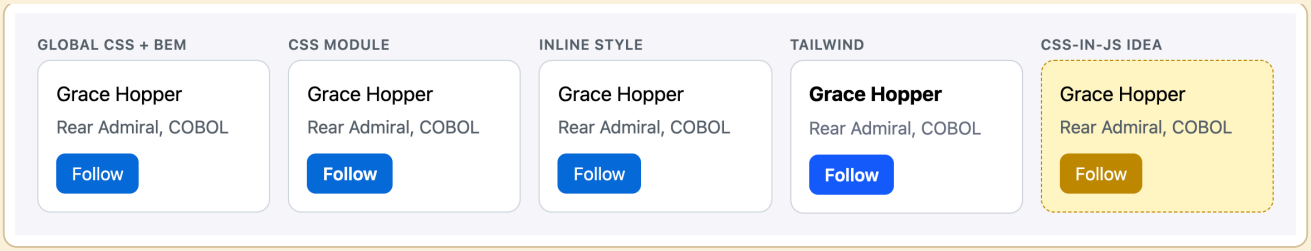


Figure 8.1 — One component, five styling approaches. The output is ordinary HTML + CSS in every case; they differ in how the CSS is authored, scoped and delivered. (The Tailwind heading uses an explicit `font-bold` — Tailwind's reset, "Preflight", removes default heading styles on the whole page, which is why the other headings look lighter than browser defaults.)

8.2 Plain CSS files

What it is

The oldest approach: write rules in a `.css` file and apply them with the `className` prop (JSX uses `className` because `class` is a reserved word in JavaScript — see Chapter 1). With a bundler such as Vite you can `import './Card.css'` from a component file; the bundler adds the file to the page's stylesheet.

```jsx
// cards.jsx
import './global.css';
export function GlobalCssCard({ name, role }) {
  const [following, setFollowing] = useState(false);
  return (
    <div className="profile-card">
      <h3 className="profile-card__name">{name}</h3>
      <p className="profile-card__role">{role}</p>
      <button className={cx('profile-card__button', following && 'profile-card__button--following')}
              onClick={() => setFollowing(f => !f)}>{following ? 'Following' : 'Follow'}</button>
    </div>
  );
}
```

```css
/* global.css — BEM naming keeps selectors from colliding by convention only. */
.profile-card { border: 1px solid #d0d7de; border-radius: 12px; padding: 16px; background: #fff; }
.profile-card__name { margin: 0 0 4px; font-size: 18px; }
.profile-card__button:hover { background: #0550ae; }
.profile-card__button--following { background: #eaeef2; color: #24292f; }
```

How it works

1. `import './global.css'` is **not** JavaScript — it's a bundler feature. The bundler collects every imported CSS file into the page (a `<link>` in production, injected `<style>` in dev).
2. **Importing CSS in a component does not scope it.** Once loaded, the rules apply to the *whole page*, even to components that never imported the file.
3. To avoid collisions, teams use naming conventions such as **BEM** (Block__Element--Modifier): `profile-card` (block), `profile-card__name` (element), `profile-card__button--following` (modifier).

Trade-offs

✅ Strengths	❌ Weaknesses
Zero tooling, zero runtime, full CSS power (`:hover`, media/container queries, animations, nesting, `@layer`)	Global by default — scoping is only as good as your discipline
Easy for designers and CSS experts	Hard to know if a rule is still used; dead CSS accumulates
Browser caches the stylesheet separately from JS	Dynamic values need CSS variables or inline styles

Modern CSS removed much of the old pain: **native nesting**, **cascade layers** (`@layer` — decide which group of rules wins regardless of specificity) and **custom properties** all work in current browsers (HTML & CSS book, Chapter 13). Sass is still common but no longer necessary for variables or nesting.

React 19: stylesheets as components

React 19 can manage stylesheets for you: render `<link rel="stylesheet" href="/card.css" precedence="default" />` anywhere in the tree and React moves it into `<head>`, loads it once even if many components render it, orders it by `precedence`, and — inside a Suspense boundary — waits for it to load before revealing the content. This is mostly used by frameworks and libraries; in a Vite app you normally just `import` CSS.

> **TL;DR**
>
> `className` + CSS file. Imports are global — use a naming convention or move to CSS Modules.

8.3 Inline styles — the `style` prop

What it is

Every HTML element accepts a `style` **prop**: a JavaScript object whose keys are CSS properties in **camelCase** (`backgroundColor`, not `background-color`). React writes it to the element's `style` attribute. The double braces in `style={{ … }}` are a JSX expression (outer) containing an object literal (inner).

`code/01_inline_styles.jsx`:

```
function ProgressBar({ percent, color = 'seagreen' }) {
  return (
    <div style={{ background: '#eee', borderRadius: 4, height: 8 }}>            {/* numbers -> px */}
      <div style={{ width: `${percent}%`, height: '100%', backgroundColor: color, borderRadius: 4 }} />
    </div>
  );
}

function Examples() {
  return (
    <>
      <ProgressBar percent={40} />
      <p style={{ lineHeight: 1.5, zIndex: 2, opacity: 0.8, flexGrow: 1 }}>unitless properties stay unitless</p>
      <p style={{ '--accent': 'tomato', color: 'var(--accent)' }}>CSS custom property set inline</p>
      <p style={{ WebkitLineClamp: 2, msTransform: 'none' }}>vendor prefixes</p>
      <p style={{ ':hover': { color: 'red' }, '@media (max-width: 600px)': { display: 'none' } }}>no pseudo-classes or
media queries</p>
    </>
  );
}
```

Output (the elements' real `style` attributes; the first line is React's development warning):

```
Unsupported style property @media (max-width: 600px). Did you mean @media (maxWidth: 600px)?
<div style="background: rgb(238, 238, 238); border-radius: 4px; height: 8px;">
<div style="width: 40%; height: 100%; background-color: seagreen; border-radius: 4px;">
<p style="line-height: 1.5; z-index: 2; opacity: 0.8; flex-grow: 1;">
<p style="--accent: tomato; color: var(--accent);">
<p style="-webkit-line-clamp: 2;">
```

How it works — the rules the output shows

1. **Numbers get `px`** for length properties (`borderRadius: 4` → `4px`), but **unitless properties** (`lineHeight`, `zIndex`, `opacity`, `flexGrow`, …) stay plain numbers. For other units pass a string: `'50%'`, `'2rem'`.
2. **Custom properties** (`'--accent'`) are passed through unchanged — the cleanest way to feed a dynamic value into a stylesheet.
3. **Vendor prefixes** start with a capital (`WebkitLineClamp`), except `ms` (`msTransform`). (jsdom, like modern browsers, dropped the obsolete `-ms-transform`.)

4. **Pseudo-classes, pseudo-elements, media queries, keyframes are impossible**: a `style` attribute can't hold selectors or at-rules. The last `<p>` got no style at all, and React warned.

When to use it (and when not)

Use inline styles for	Don't use them for
Values computed at runtime: progress width, drag position, a user-chosen colour	Whole-component styling (no hover/focus/responsive; hard to override)
Passing a value to CSS: `style={{ '--progress': ${p}% }}` with the rest in a stylesheet	Anything a designer will want to theme
Quick prototypes	Large lists where every row repeats the same object (use a class)

Inline styles have the **highest specificity** of normal declarations, so a stylesheet can override them only with `!important` — another reason to keep them for truly dynamic values.

> **TL;DR**

`style={{ camelCase: value }}`; numbers → px except unitless properties; no `:hover` or media queries. Best for dynamic values, ideally via CSS variables.

8.4 CSS Modules

> **What it is**

A **CSS Module** is a CSS file (named `*.module.css`) whose class names are **local to the file that imports it**. The build tool renames every class to something unique and gives your JavaScript an object mapping the original names to the generated ones. You write normal CSS; collisions become impossible.

Supported out of the box by Vite, Next.js, Parcel, webpack's `css-loader` and esbuild — no library to install.

`code/Card.module.css`:

```
.card { border: 1px solid #d0d7de; border-radius: 12px; padding: 16px; background: #fff; }
.title { margin: 0 0 4px; font-size: 18px; }
.role { margin: 0 0 12px; color: #57606a; }
.button { font: inherit; padding: 6px 14px; border-radius: 8px; border: 0; background: #0969da; color: #fff; cursor:
pointer; }
.button:hover { background: #0550ae; }
.primary { composes: button; font-weight: 600; }       /* reuse another class */
:global(.sr-only) { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); } /* escape
hatch */
```

Using it (`code/cards.jsx`):

```
import styles from './Card.module.css';
export function ModuleCard({ name, role }) {
  return (
    <div className={styles.card}>
      <h3 className={styles.title}>{name}</h3>
      <p className={styles.role}>{role}</p>
      <button className={styles.primary}>Follow</button>
    </div>
  );
}
```

What the bundler produces — `code/02_css_modules.mjs` runs esbuild on that file:

```
const result = await esbuild.build({
  stdin: { contents: `import styles from './Card.module.css'; console.log(JSON.stringify(styles, null, 1));`,
resolveDir: '.', loader: 'js' },
  bundle: true, write: false, outdir: 'out', format: 'esm', logLevel: 'error',
});
// ... print the JS object and the emitted CSS
```

Output:

```
--- the JS object you get from `import styles from "./Card.module.css"` ---
{ card: "Card_card", title: "Card_title", role: "Card_role", button: "Card_button", primary: "Card_button Card_primary"
}
--- the CSS the bundler emits (renamed, so it cannot clash) ---
.Card_card {
  border: 1px solid #d0d7de;
  border-radius: 12px;
  padding: 16px;
  background: #fff;
}
.Card_title {
  margin: 0 0 4px;
  font-size: 18px;
}
... (role, button and button:hover renamed the same way)
.Card_primary {
  font-weight: 600;
}
.sr-only {
  position: absolute;
  ...
}
```

How it works

1. The tool parses the file and renames each class selector: `.card` → `.Card_card` (esbuild's scheme; Vite and webpack add a short hash, e.g. `_card_1x9ab_1`, so names are unique even across files with the same name).
2. `import styles from …` returns the mapping; `styles.card` is the generated string.
3. `composes: button` makes `styles.primary` equal *both* class names — `"Card_button Card_primary"` — reuse without duplicating rules.
4. `:global(.sr-only)` opts a selector out of renaming, for the rare global class you need.
5. Element selectors (`h3 { … }`) and IDs are *not* scoped — only class names — so keep module files to classes.

Trade-offs

✅	❌
Real CSS: hover, media/container queries, animations, nesting	Styles live in a second file (some like this, some don't)
Zero runtime, works with Server Components	Dynamic values still need CSS variables or inline styles
Unused-class detection with TypeScript plugins / linters	Class names with dashes need bracket access: `styles['is-active']` (or configure camelCase conversion)
Easy migration from plain CSS	Theming across modules relies on CSS variables (§8.8)

Common mistakes

- Naming the file `Card.css` (no `.module`) → nothing is scoped and `styles` is empty.
- Writing `className="card"` instead of `className={styles.card}` → no style applied (the real name is renamed).
- Referencing a class that doesn't exist → `styles.tittle` is `undefined` and React renders `class` without it — silently. TypeScript typings for modules catch this.

TL;DR

`*.module.css` + `styles.x`. Local class names, real CSS, no runtime. A safe default for teams that like writing CSS.

8.5 Composing class names from props

What it is

Real components choose classes based on props and state: a Button has a `variant`, a `size`, a `loading` state and maybe an extra `className` from its parent. Building that string by hand with `+` and ternaries is error-prone; the tiny **`clsx`** package (and its older sibling `classnames`) does it for you. `code/cx.js` is a 10-line version so you can see there's no magic:

```
// cx('a', cond && 'b', { c: true, d: false }, ['e', null]) -> 'a b c e'
export function cx(...args) {
  const out = [];
  for (const a of args) {
    if (!a) continue;                                  // false, null, undefined, 0, ''
    if (typeof a === 'string' || typeof a === 'number') out.push(a);
    else if (Array.isArray(a)) { const inner = cx(...a); if (inner) out.push(inner); }
    else if (typeof a === 'object') for (const [k, v] of Object.entries(a)) if (v) out.push(k);
  }
  return out.join(' ');
}
```

`code/03_class_names.jsx`:

```
function Button({ variant = 'primary', size = 'md', loading = false, className, ...rest }) {
  return (
    <button
      className={cx('btn', `btn--${variant}`, `btn--${size}`, { 'btn--loading': loading }, className)}
      disabled={loading || rest.disabled}
      {...rest}
    />
  );
}
// <Button>Save</Button>  <Button variant="danger" size="sm">Delete</Button>
// <Button loading>Uploading</Button>  <Button variant="ghost" className="ml-auto">Cancel</Button>
```

Output:

```
"btn active lg"
Save      class="btn btn--primary btn--md"
Delete    class="btn btn--danger btn--sm"
Uploading class="btn btn--primary btn--md btn--loading" disabled
Cancel    class="btn btn--ghost btn--md ml-auto"
concatenated: "btn false undefined"
```

The last line is the bug `clsx` prevents: naive concatenation leaks `"false"` and `"undefined"` into the DOM.

Patterns worth knowing

- **Always accept and merge a `className` prop** in reusable components, so parents can add spacing or layout (`ml-auto`) without forking the component.
- **Variant maps** beat nested ternaries: `const VARIANTS = { primary: '…', danger: '…' }` then `VARIANTS[variant]`. Libraries like **`cva`** (class-variance-authority) formalise this for Tailwind.
- **With CSS Modules**: `cx(styles.btn, styles[variant], loading && styles.loading)`.
- **With Tailwind**, conflicting utilities (`px-2` from the component, `px-4` from the parent) are *not* resolved by order in the string; use **`tailwind-merge`** (`twMerge(clsx(...))`) so the last one wins (§8.6).

In a real project: `npm install clsx` and `import { clsx } from 'clsx'` — the API above matches it for these cases.

TL;DR

Build class strings with `clsx`/`cx`; accept a `className` prop; use variant maps instead of ternary chains.

8.6 Tailwind CSS

What it is

Tailwind CSS is a **utility-first** CSS framework: instead of inventing class names (`.profile-card`) and writing rules for them, you compose many tiny single-purpose classes directly in your markup — `p-4` (padding), `rounded-xl`, `bg-white`, `text-gray-500`, `hover:bg-blue-700`. A build step scans your source files and generates CSS for **only the classes you actually use**. The current major version is **v4** (released January 2025; v4.3 at the time of writing), which is configured in CSS rather than a JavaScript config file.

The Tailwind card from Figure 8.1:

```
export function TailwindCard({ name, role }) {
  return (
    <div className="rounded-xl border border-gray-300 bg-white p-4">
      <h3 className="mb-1 text-lg font-bold">{name}</h3>
      <p className="mb-3 text-gray-500">{role}</p>
      <button className="rounded-lg bg-blue-600 px-3.5 py-1.5 font-semibold text-white hover:bg-blue-
700">Follow</button>
    </div>
  );
}
```

How it works

1. You add one line to your main CSS file: `@import "tailwindcss";` (it brings in the default **theme** — colours, spacing, fonts as CSS variables — **Preflight**, a CSS reset, and the **utilities** layer).
2. The build tool (the Vite plugin or the CLI) scans your project's source files for anything that looks like a class name. `@source "./path"` adds places it would not find by itself.
3. For each class it recognises, it generates one rule. Variants are prefixes: `hover:`, `focus-visible:`, `md:` (screen ≥ 48rem = 768px), `dark:`, `disabled:`, `group-hover:` …
4. Values come from the theme (`px-2` = 2 × `--spacing`), or you can use an **arbitrary value** in brackets: `w-[37px]`, `grid-cols-[1fr_2fr]`.

`code/05_tailwind_build.mjs` runs the real Tailwind CLI on a one-line component:

```
<span className="px-2 font-semibold hover:bg-blue-700 md:text-lg dark:text-white w-[37px]">new</span>
```

Output (the generated utilities — nothing else from Tailwind's thousands of possible classes):

```
@layer utilities {
  .w-\[37px\] {
    width: 37px;
  }
  .px-2 {
    padding-inline: calc(var(--spacing) * 2);
  }
  .font-semibold {
    --tw-font-weight: var(--font-weight-semibold);
    font-weight: var(--font-weight-semibold);
  }
  @media (hover: hover) {
    .hover\:bg-blue-700:hover {
      background-color: var(--color-blue-700);
    }
  }
  @media (width >= 48rem) {
    .md\:text-lg {
      font-size: var(--text-lg);
      line-height: var(--tw-leading, var(--text-lg--line-height));
    }
  }
  @media (prefers-color-scheme: dark) {
    .dark\:text-white {
      color: var(--color-white);
    }
  }
}
```

Notice: variants become ordinary CSS (media queries, `:hover`), `hover:` only applies on devices that can hover (v4 behaviour), and every value points at a theme variable — change `--color-blue-700` once and every use follows.

Setting it up (Vite — not run here)

```
npm install tailwindcss @tailwindcss/vite
```

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';
export default defineConfig({ plugins: [react(), tailwindcss()] });
```

```
/* src/index.css */
@import "tailwindcss";

@theme {                                  /* your design tokens -> new utilities */
  --color-brand: oklch(0.55 0.2 260);     /* enables bg-brand, text-brand, border-brand… */
  --font-display: "Inter", sans-serif;    /* enables font-display */
}
@custom-variant dark (&:where(.dark, .dark *));   /* optional: dark: follows a .dark class instead of the OS */
```

(For this book the CLI does the same job: `tailwindcss -i tailwind.input.css -o tailwind.css` produced `code/tailwind.css` for the screenshot.)

Reusing styles: components, not `@apply`

Long class lists are repeated in one place only if you make a **component** (`<Button>`, `<Card>`) — that's the intended way to reuse Tailwind styles in React. `@apply` (copying utilities into a custom class) exists but brings back the naming and dead-CSS problems Tailwind was avoiding; use it sparingly (e.g. styling Markdown output you don't control).

Common mistakes

Mistake	Why it fails	Fix
`className={`bg-${color}-500`}`	The scanner reads source *text*; it never sees `bg-red-500`, so no CSS is generated	Map to complete class names: `{ red: 'bg-red-500', green: 'bg-green-500' }[color]`
`className="px-2 px-4"` expecting px-4 to win	Winner is decided by order *in the generated CSS*, not in the attribute	Don't pass conflicting classes; merge with `tailwind-merge`
Forgetting Preflight resets headings, lists, buttons	`<h1>` looks like body text	Style explicitly (`text-3xl font-bold`) — intended
Huge unreadable class strings	Hard to review	Extract components; order with the official Prettier plugin

Trade-offs

✅	❌
No naming, no switching files, no dead CSS (unused classes are never generated)	Markup gets long and noisy
Consistent design tokens (spacing and colour scales) out of the box	A new vocabulary to learn; team must agree to use it
Tiny production CSS that stops growing with the app	Dynamic values need arbitrary values or CSS variables
Zero runtime; works with Server Components	Styles are coupled to markup — harder to restyle a whole app by editing one stylesheet (use theme variables)

Real-world examples

Tailwind is the default in many starter kits (Next.js `create-next-app` offers it; shadcn/ui is built on it) and is used widely in product teams that value speed of iteration.

In the interview

> **30-second answer:** "Tailwind is utility-first: I compose small classes like `p-4 rounded-xl hover:bg-blue-700` in JSX, and the build scans my files and generates only the CSS I used, so there's no naming, no dead CSS and no runtime. Reuse happens through React components. The traps are dynamically-built class names, which the scanner can't see, and conflicting utilities, which need `tailwind-merge`. v4 configures the theme in CSS with `@theme`."

TL;DR

Utilities in JSX, CSS generated from what you use. Write complete class names, reuse via components, configure with `@theme`.

8.7 CSS-in-JS

What it is

CSS-in-JS means writing styles in JavaScript files, usually colocated with the component, often with access to props:

```
// styled-components / Emotion style (not run — not installed)
import styled from 'styled-components';
const Button = styled.button`
  padding: 6px 14px;
  border-radius: 8px;
  background: ${props => (props.$danger ? 'crimson' : '#0969da')};
  &:hover { filter: brightness(0.9); }
`;
<Button $danger>Delete</Button>
```

There are two very different kinds:

Kind	How it works	Examples
Runtime	In the browser, the library turns the template into CSS, hashes it into a class name, and injects a `<style>` tag while rendering	styled-components, Emotion
Zero-runtime (build-time)	A compiler extracts the styles into static `.css` files during the build; only class names remain in JS	vanilla-extract, Linaria, Panda CSS, StyleX (Meta)

How runtime CSS-in-JS works — a tiny version

`code/cards.jsx` contains a 5-line imitation, using a React 19 feature: a `<style>` element with `href` and `precedence` is **hoisted into `<head>` and de-duplicated** by React, however many components render it.

```
function css(strings) { /* hash the text into a class name, replace & with .className */ }
const cardStyle = css`& { border: 1px dashed #bf8700; border-radius: 12px; padding: 16px; background: #fff8c5; }
  & h3 { margin: 0 0 4px; font-size: 18px; } & p { margin: 0 0 12px; color: #57606a; }
  & button { font: inherit; padding: 6px 14px; border-radius: 8px; border: 0; background: #bf8700; color: #fff; }`;

export function CssInJsCard({ name, role }) {
  return (
    <div className={cardStyle.id}>
      <style href={cardStyle.id} precedence="medium">{cardStyle.text}</style>
      <h3>{name}</h3><p>{role}</p><button>Follow</button>
    </div>
  );
}
```

That is the yellow card in Figure 8.1. Real libraries add prop interpolation, theming, vendor prefixing and server-side extraction — and they must insert styles *before* layout effects read the DOM, which is why React has a dedicated Hook for library authors, `useInsertionEffect`.

Why the trend moved away from runtime CSS-in-JS

1. **Runtime cost**: serialising and hashing styles during render adds work to every render, and injecting new rules forces the browser to recalculate styles.
2. **Server Components** (Chapter 13): runtime libraries rely on React context and client-side rendering, so styled components must live in `'use client'` files.
3. **Streaming SSR** needs extra setup to collect styles per request.

In **March 2025 styled-components entered maintenance mode**; its maintainer advised against adopting it for new projects. Existing apps keep working, and Emotion remains widely used (e.g. inside MUI). New projects mostly choose CSS Modules, Tailwind, or a **zero-runtime** library that keeps the colocated, typed authoring style without the runtime.

Trade-offs

✅	❌
Styles next to the component; dynamic styles from props	Runtime libs: slower renders, bigger bundles, RSC friction
Automatic scoping; dead styles die with the component	Another abstraction to learn; debugging generated class names
Zero-runtime libs: type-safe tokens, static CSS output	Zero-runtime libs need build-tool plugins

TL;DR

Runtime CSS-in-JS (styled-components, Emotion) is convenient but costs render time and clashes with Server Components; zero-runtime tools (vanilla-extract, StyleX, Panda) or CSS Modules/Tailwind are the usual picks for new work.

8.8 Theming and dark mode

What it is

Theming means switching a whole set of design values — colours, radii, fonts — at once. The modern, framework-independent way is **CSS custom properties** (CSS variables) as **design tokens**: components only ever reference `var(--`

`bg)`, `var(--accent)`; a theme is just a different set of values for those variables.

How it works

1. Define tokens on `:root`; override them under an attribute selector such as `[data-theme="dark"]`.
2. A small React **provider** stores the user's choice (`light` / `dark` / `system`), resolves `system` using the OS preference (the `prefers-color-scheme` media query), and writes the result to `<html data-theme="…">`.
3. CSS does the rest — no component re-renders are needed to recolour the page.
4. Save the choice in `localStorage`, and apply it with a tiny inline script **before** the page paints to avoid a **flash of the wrong theme**.
5. Set the CSS `color-scheme` property so browser-drawn parts (form controls, scrollbars) match.

`code/theme.css` (tokens):

```
:root {
  --bg: #ffffff; --surface: #f6f8fa; --text: #1f2328; --muted: #59636e;
  --border: #d0d7de; --accent: #0969da; --accent-text: #ffffff;
}
[data-theme="dark"] {
  --bg: #0d1117; --surface: #161b22; --text: #e6edf3; --muted: #9198a1;
  --border: #3d444d; --accent: #4493f8; --accent-text: #0d1117;
}
body  { background: var(--bg); color: var(--text); }
.panel { background: var(--surface); border: 1px solid var(--border); border-radius: 12px; padding: 16px; }
.cta  { background: var(--accent); color: var(--accent-text); }
```

`code/theme.jsx` (the provider — the OS setting is an external store, so it uses `useSyncExternalStore` from Chapter 7):

```
const ThemeContext = createContext(null);
const QUERY = '(prefers-color-scheme: dark)';

function subscribeSystem(cb) { const m = window.matchMedia(QUERY); m.addEventListener('change', cb); return () => m.removeEventListener('change', cb); }
function useSystemDark() { return useSyncExternalStore(subscribeSystem, () => window.matchMedia(QUERY).matches, () => false); }

export function ThemeProvider({ children }) {
  const [choice, setChoice] = useState(() => localStorage.getItem('theme') ?? 'system'); // 'light' | 'dark' | 'system'
  const systemDark = useSystemDark();
  const resolved = choice === 'system' ? (systemDark ? 'dark' : 'light') : choice;

  useEffect(() => {                       // sync to the <html> element: CSS does the rest
    document.documentElement.dataset.theme = resolved;
    document.documentElement.style.colorScheme = resolved;   // native controls + scrollbars follow
    localStorage.setItem('theme', choice);
  }, [choice, resolved]);

  return <ThemeContext.Provider value={{ choice, resolved, setChoice }}>{children}</ThemeContext.Provider>;
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme must be used inside <ThemeProvider>');
  return ctx;
}
// ThemeSwitcher renders three role="radio" buttons: light / dark / system
```

`code/04_theme_provider.jsx` drives it with a fake OS setting:

Output:

```
first visit (OS light)     choice=system resolved=light | <html data-theme="light"> | saved=system
OS switches to dark        choice=system resolved=dark | <html data-theme="dark"> | saved=system
user picks "light"         choice=light resolved=light | <html data-theme="light"> | saved=light
OS changes again (ignored) choice=light resolved=light | <html data-theme="light"> | saved=light
reload                     choice=light resolved=light | <html data-theme="light"> | saved=light
outside provider -> useTheme must be used inside <ThemeProvider>
```

The same page in Chrome with the OS in light mode and in dark mode (`code/theme.html`, "system" selected):

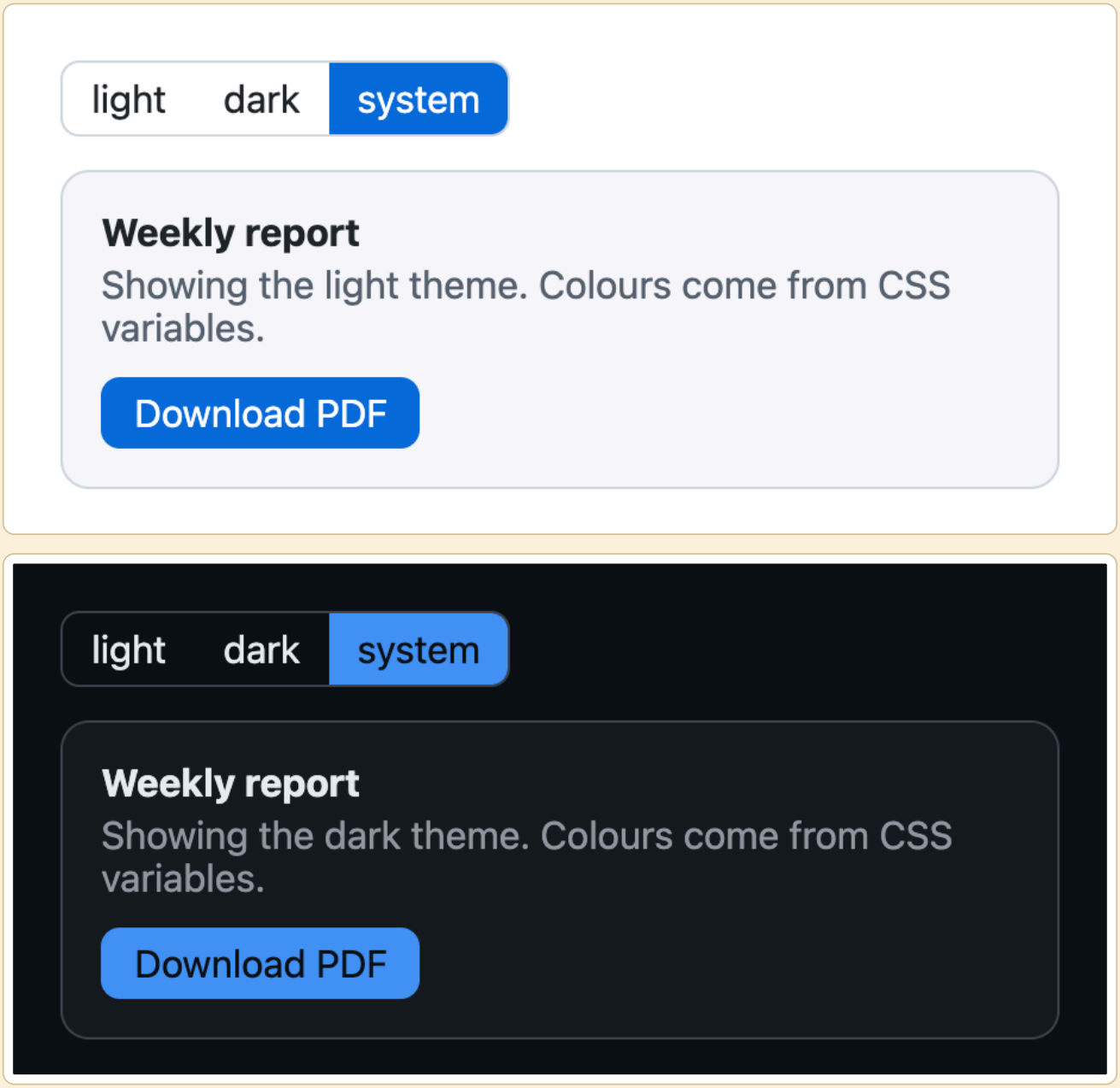


Figure 8.2 — One set of components, two token sets. Only the `data-theme` attribute on `<html>` differs.

Avoiding the flash

React runs after the HTML and CSS have been parsed, so if the theme is only applied in an Effect, a dark-mode user briefly sees the light page. `code/theme.html` fixes this with a blocking script in `<head>`:

```
<script>
  try {
    const saved = localStorage.getItem('theme') || 'system';
    const dark = saved === 'dark' || (saved === 'system' && matchMedia('(prefers-color-scheme: dark)').matches);
    document.documentElement.dataset.theme = dark ? 'dark' : 'light';
  } catch {}
</script>
```

Frameworks with SSR need the same trick (packages like `next-themes` inject it for you); a cookie lets the server render the right theme directly.

Alternatives and extras

- **CSS only**: if you don't need a manual toggle, `@media (prefers-color-scheme: dark) { :root { … } }` is enough — no JavaScript.
- **`light-dark()`**: with `color-scheme: light dark` set, `color: light-dark(#111, #eee)` picks a value per scheme in one declaration (Baseline since 2024).
- **Tailwind**: `dark:` variants follow the OS by default; `@custom-variant dark (&:where(.dark, .dark *))` or a `[data-theme=dark]` selector makes them follow your toggle.
- **Component libraries** expose a `ThemeProvider` that fills the same role (§8.9).
- **Why not pass colours through React context?** You can, but every theme change then re-renders every consumer; CSS variables recolour without rendering.

Common mistakes

- Hard-coded colours in components (`color: '#333'`) that don't respond to the theme — always use tokens.
- Reading `localStorage` during server rendering (it doesn't exist there) — read it on the client or use a cookie.
- Only inverting colours: dark themes also need adjusted shadows, borders and less saturated accents, and still must meet contrast ratios (WCAG 2.2: 4.5:1 for normal text).

TL;DR

Tokens as CSS variables, overridden by `[data-theme]`; a provider resolves light/dark/system and writes the attribute; a head script prevents the flash; `color-scheme` for native controls.

8.9 Component libraries

What it is

A **component library** gives you ready-made, accessible UI components — buttons, dialogs, menus, date pickers, tables — so you don't rebuild (and re-debug) them. Accessibility is the main reason to use one: a correct dropdown menu needs focus management, keyboard navigation and ARIA roles that take weeks to get right (see Chapter 14 and the HTML & CSS book's accessibility chapter).

Three kinds

Kind	What you get	Examples	Best when
Styled kits	Complete components with a design system and theming API	MUI (Material UI), Mantine, Chakra UI, Ant Design, React Bootstrap	Internal tools, admin panels, fast delivery, you accept their look
Headless (unstyled) primitives	Behaviour + accessibility, no styles — you bring CSS/Tailwind	Radix UI Primitives, React Aria (Adobe), Headless UI (Tailwind Labs), Base UI	A custom brand design that must still be accessible
Copy-in components	Source code copied into *your* repo (via a CLI), built on headless primitives + Tailwind	shadcn/ui	You want to own and edit the code, not depend on a package's API

How to choose

Question	Leans towards
Is there a designer with a custom design system?	Headless or copy-in
Do we need 50 components next week?	Styled kit
Are we using Tailwind?	shadcn/ui, Headless UI, Radix + Tailwind
Server Components / Next.js App Router?	Check the library's RSC guidance — many interactive components need `'use client'`
Bundle-size sensitive (public site)?	Headless + your CSS; import only what you use

Common mistakes

- Fighting a styled kit's design with overrides everywhere — pick headless instead.
- Mixing two styled kits (two resets, two themes, two button styles).
- Wrapping a library's component so thinly that you still leak its API everywhere — create your own `<Button>` that wraps it once, so you can swap later.

TL;DR

Styled kit for speed, headless for custom design with accessibility, copy-in (shadcn/ui) to own the code. Never hand-roll complex widgets without a reason.

8.10 Choosing an approach

Situation	Sensible default
New app, team likes utility classes and fast iteration	**Tailwind** + `clsx` / `tailwind-merge`, headless or shadcn/ui components
New app, team prefers writing CSS	**CSS Modules** + CSS variables for tokens
Content site with little interactivity	Plain CSS (with `@layer` and nesting) or CSS Modules
Existing styled-components/Emotion codebase	Keep it; consider zero-runtime or CSS Modules for new areas, especially if moving to Server Components
Design system shared across apps	Tokens as CSS variables + headless primitives; publish compiled CSS
Truly dynamic values (drag, progress, user colours)	Inline `style`, ideally setting a CSS variable

Whatever you choose, the rules that matter most are the same: **one approach per codebase**, **tokens in CSS variables**, **components as the unit of reuse**, and **accessibility first** — styles must never remove focus outlines without replacing them.

In the interview

> **30-second answer:** "React doesn't prescribe styling. Global CSS is simple but needs naming conventions; CSS Modules scope classes per file with zero runtime; Tailwind uses utility classes and generates only what I use; runtime CSS-in-JS like styled-components is convenient but adds render cost and doesn't fit Server Components, so zero-runtime tools are preferred now. Inline styles are for truly dynamic values. For theming I use CSS variables switched by a data attribute."

- *How do you handle dark mode?* — Tokens as CSS variables, `[data-theme=dark]` overrides, a provider for the toggle, a head script to avoid the flash, `color-scheme` for native controls.
- *CSS Modules vs Tailwind?* — Both scoped and zero-runtime; Modules keep CSS in CSS files with semantic names, Tailwind keeps styling in markup with a token scale. Team preference decides.
- *Why is `className` not `class`?* — JSX compiles to JavaScript, where `class` is a reserved word, and it matches the DOM property `element.className`.

Self-check

1. Does `import './Card.css'` scope the rules to `Card`? → No; imported CSS is global.
2. What does `style={{ zIndex: 2, width: 100 }}` produce? → `z-index: 2; width: 100px`.
3. Why won't `` `text-${size}` `` work in Tailwind? → The scanner only sees complete class names in source text.
4. What does `composes` do in a CSS Module? → Adds another class's generated name to this one.
5. How do you switch themes without re-rendering components? → Change CSS variable values via a `data-theme` attribute.

Chapter 8 TL;DR

Approach	Remember
Plain CSS	`className`; imports are global; BEM or layers for order
Inline `style`	camelCase object; numbers → px (except unitless); no `:hover`/media; for dynamic values
CSS Modules	`*.module.css`, `styles.x`; local names, `composes`, `:global`; zero runtime
Class composition	`clsx`/`cx`; accept `className`; variant maps
Tailwind v4	Utilities in JSX; only used classes generated; `@theme` tokens; no dynamic class names; `tailwind-merge` for conflicts
CSS-in-JS	Runtime (styled-components, Emotion) vs zero-runtime (vanilla-extract, StyleX, Panda); runtime is out of favour for new RSC apps
Theming	CSS variables + `data-theme`; provider for choice; head script against flash; `color-scheme`; `light-dark()`
Libraries	Styled kits vs headless vs copy-in (shadcn/ui); accessibility is the reason

Next: Chapter 9 — Routing and Data Fetching.

Chapter 9 — Routing and Data Fetching

Real apps have many screens and live on data from a server. This chapter covers the two libraries most React apps use for that: **React Router** (which screen to show for which URL, and — in its data mode — loading the data for that screen) and **TanStack Query** (caching and synchronising server data). It then connects both to React's own async tools, **Suspense** and `use()`, and ends with patterns for loading and error states that users actually like.

You already know the manual way — fetching in an Effect with cleanup (Chapter 5, and the `useFetch` Hook in Chapter 7) — and Chapter 6 explained why server data is a different kind of state. Here you'll see what the libraries add and how to use them well.

What you'll learn

#	Topic	You will be able to…
9.1	Client-side routing	explain how an SPA changes pages without reloading
9.2	React Router v8 and its modes	install it and pick declarative, data or framework mode
9.3	Routes, links, params	map URLs to components; read `:params` and `?search` params; navigate in code; 404s
9.4	Nested routes and layouts	share layouts with `<Outlet>`, use index and pathless routes, relative links
9.5	Loaders	load data *before* a route renders; pending UI; route error boundaries
9.6	Actions and forms	handle mutations with `<Form>`, validation, redirects, `useFetcher`
9.7	Middleware, lazy routes, framework mode	guard routes, split code, know what framework mode adds
9.8	TanStack Query basics	`useQuery`, states, caching, `staleTime` / `gcTime`, dedupe, retries
9.9	Query keys, pagination, dependent queries	design keys, paginate without flicker, chain queries
9.10	Mutations and invalidation	`useMutation`, invalidate, optimistic updates with rollback
9.11	Suspense and `use()`	read promises in render, fallbacks, error boundaries, transitions
9.12	Loading and error states	choose spinners vs skeletons vs stale content; handle every failure
9.13	Putting it together	router loaders + Query cache in one small shop

Versions used: **React 19.3**, **React Router 8.4**, **TanStack Query 5.104**. Code in `code/` runs in Node with jsdom; data comes from a tiny in-memory API (`code/api-server.js`) started on `127.0.0.1` inside each script, and the `[server]` lines in the outputs are that server's log.

9.1 Client-side routing

What it is

Routing means choosing what to show based on the **URL**. In a traditional multi-page site the *server* does it: every link click requests a new HTML page and the browser reloads. In a **single-page application (SPA)** the page loads once, and a **client-side router** in JavaScript does it:

1. It intercepts clicks on its links (`<Link>`), calls `event.preventDefault()`, and updates the address bar with the **History API** (`history.pushState`) — no request, no reload.
2. It listens for the Back/Forward buttons (the `popstate` event).
3. It matches the new URL against your **route definitions** and renders the matching components.

Analogy: a museum with one entrance. A server-routed site sends you outside and back in through the door for every room; a client-side router just walks you down the corridor — the building (your JS, your state) stays loaded.

Why the URL matters

The URL *is* state — the most shareable state in your app. Anything a user might want to bookmark, share, refresh or go "back" to (which page, which product, which tab, filters, sort order, page number) belongs in the URL, not only in `useState`. A good router makes that easy.

Trade-offs

Client-side routing	Server routing (MPA)
Instant page changes, state survives navigation	Every page is a fresh load
Needs JS; first load downloads the app	Works without JS
The server must return `index.html` for *every* app path (so a refresh on `/products/3` doesn't 404)	Each URL maps to a real page
SEO and first paint need SSR/pre-rendering for public content (Chapter 13)	Good by default

---

9.2 React Router v8 and its three modes

What it is

React Router is the most widely used router for React. Version **8** (June 2026) is a deliberately small upgrade from v7: it requires React 19.2.7+ and Node 22.22+, is ESM-only, removes the old `react-router-dom` package, and turns v7's "future flags" (notably **middleware**) on by default.

```
npm install react-router          # one package; there is no react-router-dom in v8
```

```
import { Link, useParams, createBrowserRouter } from 'react-router';   // everything...
import { RouterProvider } from 'react-router/dom';                      // ...except the DOM-specific pieces
```

It can be used in three **modes** — pick one per app:

Mode	You write	Router provides	Use when
Declarative	`<BrowserRouter>` + `<Routes>`/`<Route>` JSX	URL matching, links, params, navigation	Simple SPAs, adding routing to an existing app, fetching data your own way (e.g. TanStack Query)
Data	Route **objects** passed to `createBrowserRouter`, rendered with `<RouterProvider>`	+ **loaders**, **actions**, pending states, route error boundaries, fetchers, middleware	SPAs that want data loading tied to navigation
Framework	Route **modules** (files) + the React Router Vite plugin	+ server rendering, code splitting per route, type generation, pre-rendering	Full-stack apps (this is the successor of the Remix framework)

Each mode includes everything in the modes above it. The sections below use **declarative** mode first (§9.3–9.4), then **data** mode (§9.5–9.7). Tests and these examples use the **memory** variants — `<MemoryRouter>` and `createMemoryRouter` — which keep the URL history in memory instead of the address bar; in a browser app you use `<BrowserRouter>` / `createBrowserRouter` with identical route definitions.

(Alternatives worth knowing: **TanStack Router**, a fully type-safe router with built-in search-param validation, and the file-based routers of **Next.js** and other frameworks.)

9.3 Routes, links and URL parameters (declarative mode)

How it works

1. Wrap the app in a router: `<BrowserRouter>` (tests: `<MemoryRouter>`).
2. Inside, `<Routes>` holds `<Route path="…" element={…}>` entries. On each URL change React Router picks the **best** matching route (it ranks routes by specificity — static segments beat dynamic `:params`, which beat `*` — so the order you write them in doesn't matter).
3. **Dynamic segments** start with `:` — `/products/:productId` matches `/products/3` and `useParams()` returns `{ productId: '3' }` (always strings).
4. `path="*"` matches anything left over: your **404** page.

5. `<Link to>` navigates; `<NavLink>` is a Link that knows whether it's **active** (matches the current URL) and passes `isActive` to `className` / `style` (`end` makes it match exactly).
6. **Search params** (`?sort=price`) are read and written with `useSearchParams()`, which works like `useState` but stores the value in the URL.
7. `useNavigate()` returns a function for navigating in code (after a form submit, on a timer): `navigate('/products')`, `navigate(-1)` (back), `navigate('/login', { replace: true })` (replace the history entry). Declaratively, render `<Navigate to="/login" />`.

`code/01_declarative_routing.jsx`:

```
import { MemoryRouter, Routes, Route, Link, NavLink, useParams, useSearchParams, useNavigate, useLocation } from
'react-router';

function Nav() {
  const cls = ({ isActive }) => (isActive ? 'active' : undefined);   // NavLink knows if it matches the URL
  return (
    <nav>
      <NavLink to="/" end className={cls}>Home</NavLink> <NavLink to="/products" className={cls}>Products</NavLink>
    </nav>
  );
}
function ProductList() {
  const [searchParams, setSearchParams] = useSearchParams();       // ?sort=price
  const sort = searchParams.get('sort') ?? 'name';
  const items = PRODUCTS.toSorted((a, b) => (sort === 'price' ? a.price - b.price : a.name.localeCompare(b.name)));
  return (
    <>
      <h1>Products (by {sort})</h1>
      <button onClick={() => setSearchParams({ sort: 'price' })}>Sort by price</button>
      <ul>{items.map(p => <li key={p.id}><Link to={`/products/${p.id}`}>{p.name}</Link></li>)}</ul>
    </>
  );
}
function ProductPage() {
  const { productId } = useParams();                               // from the :productId segment
  const navigate = useNavigate();
  const product = PRODUCTS.find(p => p.id === productId);
  if (!product) return <h1>No product {productId}</h1>;
  return (
    <>
      <h1>{product.name} - ${product.price}</h1>
      <button onClick={() => navigate('/products')}>Back to list</button>
      <button onClick={() => navigate(-1)}>Browser back</button>
    </>
  );
}
function NotFound() { return <h1>404 - nothing at {useLocation().pathname}</h1>; }

function App() {
  return (
    <>
      <Nav />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<ProductList />} />
        <Route path="/products/:productId" element={<ProductPage />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </>
  );
}
// render(<MemoryRouter initialEntries={['/']}><App /></MemoryRouter>), then click around
```

Output:

```
start                   url=/                   active=Home     h1="Welcome"
click Products          url=/products           active=Products h1="Products (by name)"
sort by price           url=/products?sort=price active=Products h1="Products (by price)"
   list order: Mouse, Keyboard, Monitor
click Monitor           url=/products/3         active=Products h1="Monitor - $199"
navigate(-1)            url=/products?sort=price active=Products h1="Products (by price)"
click Mouse             url=/products/2         active=Products h1="Mouse - $19"
navigate("/products")   url=/products           active=Products h1="Products (by name)"
deep link /nope -> 404 - nothing at /nope
```

Things to notice:

- "Products" stays **active** on `/products/3` because NavLink matches descendants; "Home" uses `end` so it's active only on `/` exactly.
- The sort order lives in the URL: **Back** returned to `?sort=price`, not to the default sort. `useState` would have lost it.
- A deep link to an unknown path rendered the `*` route.

Common mistakes

Mistake	Fix
`<a href="/products">` inside the app	Full page reload, state lost — use `<Link to>` (keep `<a>` for external sites)
Treating `useParams()` values as numbers	They're strings: `Number(productId)` or compare as strings
Filters/tabs/page in `useState`	Put shareable UI state in search params
Calling `navigate()` during render	Navigate in handlers/Effects, or render `<Navigate>`
Server returns 404 on refresh of `/products/3`	Configure a fallback to `index.html` (Vite preview, Netlify `_redirects`, nginx `try_files`)

TL;DR

`<Routes>`/`<Route path>`; `:param` + `useParams`; `useSearchParams` for `?query`; `<Link>`/`<NavLink>`; `useNavigate` in code; `*` for 404.

9.4 Nested routes and layouts

What it is

Most apps have **layouts**: a header on every page, a sidebar on all account pages. React Router models this by **nesting** routes. A parent route renders the shared UI and an `<Outlet />` where the matching child route appears. The URL segments nest the same way: `users` + `:userId` = `/users/42`.

Three kinds of routes help:

Route kind	Written as	Matches	Use
Index route	`<Route index element=…>`	The parent's own URL, exactly	Default child: "Pick a user" at `/users`
Layout (pathless) route	`<Route element={<Layout/>}>` with no `path`	Nothing by itself; wraps its children	Share UI without adding a URL segment
Dynamic vs static children	`new` and `:userId` side by side	`/users/new` → static wins	Ranking, not order, decides

`code/02_nested_routes.jsx`:

```
function AppLayout() {                              // renders on EVERY page
  return (<div><header>MyShop</header><main><Outlet /></main><footer>(c) 2026</footer></div>);
}
function AccountLayout() {                          // pathless layout: groups routes, adds no URL segment
  const user = { name: 'Ada' };
  return (<section><aside>Account menu</aside><Outlet context={user} /></section>);
}
function Settings() { const user = useOutletContext(); return <h1>Settings for {user.name}</h1>; }
function UsersLayout() {
  return (
    <div>
      <nav><Link to="new">New user</Link> <Link to="42">User 42</Link> <Link to="..">Up</Link></nav>  {/* relative
links */}
      <Outlet />
    </div>
  );
}

function App() {
  return (
    <Routes>
      <Route element={<AppLayout />}>                          {/* layout route: no path */}
        <Route index element={<h1>Home</h1>} />                {/* matches "/" exactly */}
        <Route path="users" element={<UsersLayout />}>
          <Route index element={<UsersIndex />} />             {/* /users */}
          <Route path="new" element={<NewUser />} />           {/* /users/new - static beats dynamic */}
          <Route path=":userId" element={<User />} />          {/* /users/42 */}
        </Route>
        <Route element={<AccountLayout />}>
          <Route path="settings" element={<Settings />} />
        </Route>
      </Route>
    </Routes>
  );
}
```

Output (which layout pieces are on screen, and the heading):

```
/           -> header  h1="Home"
/users      -> header+nav  h1="Pick a user"
/users/new  -> header+nav  h1="Create a user"
/users/42   -> header+nav  h1="User #42"
/settings   -> header+aside  h1="Settings for Ada"
from /users/42 click "new" -> header+nav  h1="Create a user"
click ".."              -> header  h1="Home"
```

How it works

1. React Router finds the **chain** of matching routes (e.g. `AppLayout → UsersLayout → User`).
2. It renders the outermost element; each `<Outlet />` renders the next element in the chain.
3. **Relative links** (no leading `/`) resolve against the **route** that renders them: `to="new"` inside `UsersLayout` means `/users/new`, even when the current URL is `/users/42`. `..` goes to the parent *route* — here the pathless layout at `/`.
4. `<Outlet context={value}>` + `useOutletContext()` passes data from a layout to whichever child is rendered.

Why it matters

Layouts don't re-mount when you move between their children — the header's state, the sidebar's scroll position and any open menus survive. It also mirrors how designers think: "all account pages share this sidebar".

TL;DR

Parents render `<Outlet/>`; `index` for the default child; pathless routes for shared layouts; relative links resolve against the route tree.

9.5 Loaders — data before render (data mode)

What it is

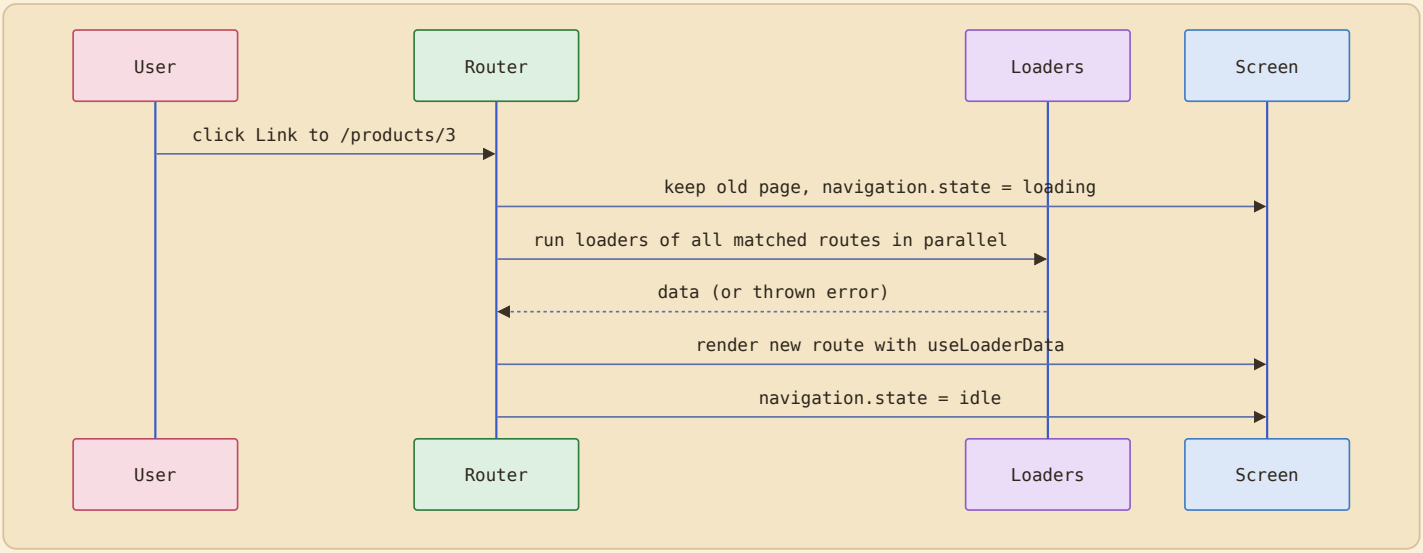
In **data mode** each route can have a `loader`: an async function that runs **when navigation to that route starts, before the route renders**. Its return value is read in the component with `useLoaderData()`.

The problem it solves

Fetching in Effects (Chapter 5) starts only *after* the component renders, and nested components each wait for their parent to render before starting their own request — a **waterfall** (request 1 → render → request 2 → render…). Loaders let the router start **all** the requests for **all** matched routes **in parallel** as soon as the user clicks — and the component renders once, with data.

How it works

1. Define routes as objects: `{ path, Component, loader, ErrorBoundary, children }`.
2. `createBrowserRouter(routes)` (tests: `createMemoryRouter(routes, { initialEntries })`) and render `<RouterProvider router={router} />`.
3. On navigation the router calls the loaders of the matched routes in parallel with `{ params, request, context }` (`request.signal` aborts if the user navigates away again).
4. While they run, the **current** screen stays visible and `useNavigation().state` is `'loading'` — use it for a progress bar.
5. When they resolve, the new route renders; `useLoaderData()` returns the data.
6. If a loader **throws**, the nearest route `ErrorBoundary` renders instead. Throw `data(message, { status })` for expected HTTP-like errors and check them with `isRouteErrorResponse(error)`.
7. On the very first load there is no "current screen", so the router shows the root route's `HydrateFallback`.



`code/03_loaders_and_errors.jsx`:

```
async function getJson(path, request) {
  const res = await fetch(api.base + path, { signal: request.signal });   // aborted if the user navigates away
  if (res.status === 404) throw data((await res.json()).message, { status: 404 });
  if (!res.ok) throw new Error(`API error ${res.status}`);
  return res.json();
}

function Root() {
  const navigation = useNavigation();               // 'idle' | 'loading' | 'submitting'
  return (
    <div>
      <nav><Link to="/">All</Link> <Link to="/products/3">Monitor</Link> <Link to="/products/9">Missing</Link></nav>
      {navigation.state === 'loading' && <p role="status">Loading…</p>}
      <Outlet />
    </div>
  );
}
function ProductList() {
  const { products, categories } = useLoaderData();
  return <h1>{products.length} products in {categories.length} categories</h1>;
}
function Product() {
  const product = useLoaderData();
  return <h1>{product.name} costs ${product.price}</h1>;
}
function ProductError() {
  const error = useRouteError();                    // whatever the loader threw
  if (isRouteErrorResponse(error)) return <h1>{error.status}: {error.data}</h1>;
  return <h1>Unexpected: {error.message}</h1>;
}

const router = createMemoryRouter([
  {
    path: '/', Component: Root, HydrateFallback: () => <p>Starting…</p>,
    children: [
      {
        index: true, Component: ProductList,
        loader: async ({ request }) => {             // two requests in PARALLEL, not a waterfall
          const [products, categories] = await Promise.all([getJson('/api/products', request),
getJson('/api/categories', request)]);
          return { products, categories };
        },
      },
      {
        path: 'products/:id', Component: Product, ErrorBoundary: ProductError,
        loader: ({ params, request }) => getJson(`/api/products/${params.id}`, request),
      },
    ],
  },
], { initialEntries: ['/'] });
render(<RouterProvider router={router} />);
```

Output:

```
first paint: Starting…
  [server] start GET /api/products
  [server] start GET /api/categories
  [server] end   GET /api/products -> 200
  [server] end   GET /api/categories -> 200
loaded     : 3 products in 2 categories
  [server] start GET /api/products/3
during nav : Loading… | still showing: 3 products in 2 categories
  [server] end   GET /api/products/3 -> 200
after nav  : Monitor costs $199
  [server] start GET /api/products/9
  [server] end   GET /api/products/9 -> 404
missing    : 404: No product 9 | nav still rendered: true
```

The server log proves the two list requests ran **in parallel**. During navigation the **old** page stayed on screen with a "Loading…" status (no blank flash). The 404 was caught by the product route's own `ErrorBoundary`, so the root layout and its nav **kept working** — errors are contained to the route that failed. (Without any `ErrorBoundary`, React Router shows a default error screen.)

Details worth knowing

- **Revalidation**: after an action (§9.6) or `useRevalidator().revalidate()`, the router re-runs the loaders on the page so data stays fresh; `shouldRevalidate` on a route can opt out.
- **Streaming non-critical data**: a loader can return an object containing an *un-awaited* promise (`return { product: await p, reviews: reviewsPromise }`); render it with `<Await resolve={reviews}>` inside `<Suspense>`, or with `use()` (§9.11). The page shows as soon as the critical data is ready.
- **Loaders are not components**: no Hooks inside them. They're plain async functions, easy to unit-test.
- **Loaders don't cache**. Navigate back to a page and its loader runs again. Pair them with a cache (TanStack Query, §9.13) when that matters.

TL;DR

`loader` runs before render, in parallel for all matched routes; `useLoaderData()` reads it; `useNavigation()` for pending UI; throw `data(msg, {status})` and handle it in a route `ErrorBoundary`.

9.6 Actions, `<Form>` and fetchers

What it is

An **action** is a route's function for **writes** (create, update, delete). React Router's `<Form method="post">` works like an HTML form, but instead of reloading the page it serialises the fields into a `Request`, calls the route's `action`, and then **automatically re-runs the loaders** on the page so the UI shows the new data.

How it works

1. `<Form method="post">` submits to the action of the route it's rendered in (or `action="/path"`).
2. The action reads `await request.formData()`, validates, and writes to the server.
3. It returns either:
   - **validation errors** — `return data({ errors }, { status: 400 })` — read in the component with **`useActionData()`**; or
   - **`redirect('/somewhere')`** — the classic *Post/Redirect/Get* pattern, so Refresh doesn't resubmit.
4. `useNavigation().state === 'submitting'` while the action runs — disable the button.
5. After the action, the router **revalidates**: loaders run again and every `useLoaderData()` gets fresh data.
6. **`useFetcher()`** does the same *without navigating* — for "like" buttons, inline edits, quick-add fields, several independent submissions at once. It has its own `fetcher.state` and `fetcher.data`.

`code/04_actions_and_forms.jsx`:

```
async function newTodoAction({ request }) {
  const form = await request.formData();              // the submitted <Form> fields
  const title = String(form.get('title') ?? '').trim();
  if (title.length < 3) return data({ errors: { title: 'At least 3 characters' } }, { status: 400 });
  await postTodo(title);
  return redirect('/todos');                          // Post/Redirect/Get
}

function TodoList() {
  const todos = useLoaderData();
  const fetcher = useFetcher();                       // submit WITHOUT navigating
  return (
    <>
      <h1>{todos.length} todos: {todos.map(t => t.title).join(', ')}</h1>
      <fetcher.Form method="post" action="/todos/new">
        <input name="title" aria-label="quick add" defaultValue="Buy milk" />
        <button>{fetcher.state === 'idle' ? 'Quick add' : 'Adding…'}</button>
      </fetcher.Form>
      <Link to="new">New todo page</Link>
    </>
  );
}
function NewTodo() {
  const actionData = useActionData();                 // what the action returned (validation errors)
  const navigation = useNavigation();
  return (
    <Form method="post">
      <h1>New todo</h1>
      <input name="title" aria-label="title" />
      {actionData?.errors?.title && <p role="alert">{actionData.errors.title}</p>}
      <button disabled={navigation.state === 'submitting'}>{navigation.state === 'submitting' ? 'Saving…' : 'Save'}
</button>
    </Form>
  );
}

const router = createMemoryRouter([
  { path: '/todos', Component: Outlet, HydrateFallback: () => null, children: [
    { index: true, Component: TodoList, loader: todosLoader },
    { path: 'new', Component: NewTodo, action: newTodoAction },
  ] },
], { initialEntries: ['/todos'] });
```

Output:

```
  [server] start GET /api/todos
  [server] end   GET /api/todos -> 200
list      : 1 todos: Read chapter 9
too short : At least 3 characters | url still /todos/new
  [server] start POST /api/todos
submitting: button says "Saving…"
  [server] end   POST /api/todos -> 201
  [server] start GET /api/todos
  [server] end   GET /api/todos -> 200
redirected: /todos | 2 todos: Read chapter 9, Write the router demo
  [server] start POST /api/todos
  [server] end   POST /api/todos -> 201
  [server] start GET /api/todos
  [server] end   GET /api/todos -> 200
fetcher   : /todos | 3 todos: Read chapter 9, Write the router demo, Buy milk
```

Notice: the too-short title never reached the server; after each successful POST the router re-fetched `GET /api/todos` **by itself** — you never wrote "refresh the list" code.

Relation to React 19 form actions

React 19's own `<form action={fn}>`, `useActionState` and `useFormStatus` (Chapter 4) solve the same problem at the component level. React Router's `<Form>` adds routing concerns: URL-addressed actions, redirects and automatic

revalidation of route data. In a data-mode app, prefer the router's `<Form>`/fetchers for route data.

> **TL;DR**
>
> `<Form method="post">` → route `action` → return errors (`useActionData`) or `redirect`; loaders revalidate automatically; `useFetcher` for submissions that shouldn't navigate.

9.7 Middleware, lazy routes and framework mode

Middleware (v8: on by default)

Middleware functions run **before** a route's loaders/actions, from parent to child, and can do work before and after the rest of the chain by awaiting `next()`. Typical uses: authentication guards, logging/timing, putting shared data (the current user) into a typed **context** that loaders read.

`code/05_middleware.jsx`:

```
const userContext = createContext(null);                  // typed slot in the router context (from react-router)

async function logMiddleware({ request }, next) {
  const path = new URL(request.url).pathname;
  console.log(`  -> ${path}`);
  await next();                                           // run the rest: child middleware + loaders
  console.log(`  <- ${path} done`);
}
async function authMiddleware({ request, context }) {
  if (!session) {
    const next = new URL(request.url).pathname;
    console.log('  auth: no session, redirecting');
    throw redirect(`/login?next=${encodeURIComponent(next)}`);
  }
  context.set(userContext, session);                      // loaders below can read it
}

const routes = [
  { path: '/', Component: Outlet, middleware: [logMiddleware], HydrateFallback: () => null, children: [
    { path: 'login', Component: Login },
    { path: 'dashboard', Component: Dashboard, middleware: [authMiddleware],
      loader: ({ context }) => ({ user: context.get(userContext) }) },
  ] },
];
```

Output:

```
signed out, open /dashboard:
  -> /dashboard
  auth: no session, redirecting
  <- /dashboard done
  -> /login
  <- /login done
  shows "Please log in" at /login?next=%2Fdashboard
signed in, open /dashboard:
  -> /dashboard
  <- /dashboard done
  shows "Dashboard for Ada"
```

One guard on the parent protects every child route, and loaders no longer each re-check the session. (Client-side guards are about **UX**, not security — the server must still reject unauthorised API calls.) In declarative mode there is no middleware; the equivalent is a wrapper component that renders `<Navigate to="/login" replace />` when there's no user.

Lazy routes (code splitting)

Large apps shouldn't download every page up front. A route's `lazy` property loads its module only when the route is first visited:

```
// not run
{ path: 'reports', lazy: () => import('./routes/reports.jsx') }   // the module exports Component, loader, …
```

React's own `lazy()` + `<Suspense>` does the same for any component (Chapter 10).

Framework mode in one page

Framework mode is React Router as a full-stack framework (the successor of Remix). Routes are declared in `app/routes.ts` and each points at a **route module** — a file that exports its pieces:

```
// app/routes.ts (not run)
import { type RouteConfig, index, route } from '@react-router/dev/routes';
export default [
  index('routes/home.tsx'),
  route('products/:id', 'routes/product.tsx'),
] satisfies RouteConfig;
```

```
// app/routes/product.tsx (not run)
import type { Route } from './+types/product';          // generated types for THIS route
export async function loader({ params }: Route.LoaderArgs) {   // runs on the server
  return { product: await db.product.find(params.id) };
}
export default function Product({ loaderData }: Route.ComponentProps) {
  return <h1>{loaderData.product.name}</h1>;             // loaderData is fully typed
}
export function ErrorBoundary({ error }: Route.ErrorBoundaryProps) { /* … */ }
```

What you get: server rendering (or `ssr: false` for an SPA, or pre-rendering to static HTML), `loader` on the server and `clientLoader` in the browser, automatic code splitting per route, and generated types so `params` and `loaderData` are type-checked. Chapter 13 compares it with Next.js.

TL;DR

Middleware = before/after hooks around loaders (auth, logging, context). `lazy` splits route code. Framework mode = route modules + SSR + typegen.

9.8 TanStack Query basics

What it is

TanStack Query (formerly React Query) is a **server-state** library: it fetches, **caches**, de-duplicates, refreshes and invalidates data that lives on a server. You tell it *what* the data is (a **query key**) and *how* to get it (a **query function**); it handles *when*.

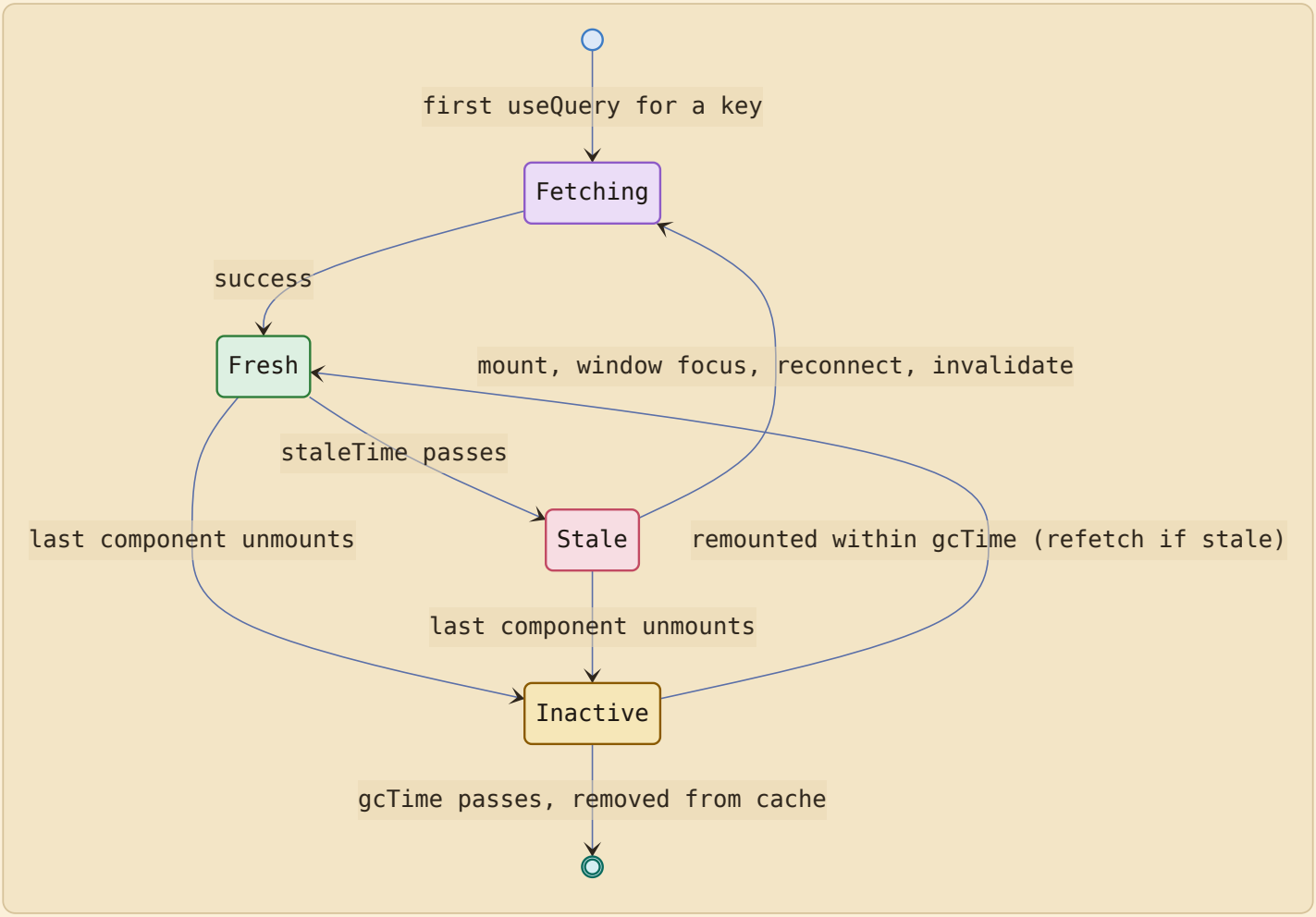
Why a special library? Server data is not like UI state (Chapter 6): someone else can change it, it can be out of date, several components need the same data, and loading/error states are everywhere. Writing that with Effects means re-implementing a cache.

How it works

1. Create one `QueryClient` (the cache) and provide it: `<QueryClientProvider client={queryClient}>`.
2. In a component: `useQuery({ queryKey: ['product', id], queryFn: () => fetchProduct(id) })`.
3. The first component to ask for a key triggers the fetch; others asking for the same key at the same time **share** it.
4. The result is stored under the key. Next time any component asks, it gets the cached data **immediately**, and — if the data is **stale** — a background refetch updates it.
5. The query function must **throw** on failure (`fetch` doesn't throw on 404 — check `res.ok`); failed queries are **retried** (3 times by default in the browser, with exponential backoff).

Two timers control the cache:

Option	Default	Meaning
`staleTime`	`0`	How long data counts as **fresh**. Fresh data is served from cache with **no** refetch. With `0`, every new mount/focus triggers a background refetch (still showing cached data instantly).
`gcTime`	5 minutes	How long **unused** data (no component using the key) stays in memory before being garbage-collected.



Stale queries are refetched automatically when a component **mounts**, when the window regains **focus**, and when the network **reconnects** (each can be turned off: `refetchOnWindowFocus: false`, …).

The states you'll use:

Field	Meaning
`status` / `isPending`, `isError`, `isSuccess`	Do we **have** data? `pending` = not yet
`fetchStatus` / `isFetching`	Is a request **in flight** right now (including background refetches)?
`isLoading`	`isPending && isFetching` — the first load
`data`, `error`, `failureCount`, `refetch()`	What you'd expect

`code/06_query_basics.jsx`:

```
function ProductName({ id, staleTime = 0 }) {
  const { data, status, fetchStatus, isPending, isFetching, error } = useQuery({
    queryKey: ['product', id],                        // cache key: identifies this data
    queryFn: () => fetchJson(`/api/products/${id}`),  // how to get it (must return a promise)
    staleTime,
  });
  timeline.push(`status=${status.padEnd(7)} fetchStatus=${fetchStatus.padEnd(8)} isPending=${isPending}
isFetching=${isFetching} data=${data?.name}`);
  if (isPending) return <p>Loading…</p>;
  if (error) return <p role="alert">Error: {error.message}</p>;
  return <p>{data.name}</p>;
}
const client = new QueryClient();
// 1) render once  2) render two at once  3) unmount + remount (staleTime 0, then 60 s)
// 4) a flaky endpoint with retry: 3   5) a 404 with retry: false

function Flaky() {
  const q = useQuery({ queryKey: ['flaky'], queryFn: () => fetchJson('/api/flaky'), retry: 3, retryDelay: 10 });
  return <p>{q.isSuccess ? 'flaky ok' : `attempts failed so far: ${q.failureCount}`}</p>;
}
```

Output:

```
1) first load:
   status=pending fetchStatus=fetching isPending=true isFetching=true data=undefined
   status=success fetchStatus=idle     isPending=false isFetching=false data=Keyboard
2) two <ProductName id={2}/> mounted together -> requests for /api/products/2: 1
3) remount id=1 (staleTime 0): first render -> status=success fetchStatus=fetching isPending=false isFetching=true
data=Keyboard
   background refetch happened: requests for /api/products/1 = 2
   remount with staleTime 60s -> requests still 2 (fresh data, no refetch)
4) retry: 3 -> succeeded after 3 requests
5) missing product -> Error: HTTP 404
```

Reading the output:

1. The first load goes `pending/fetching` → `success/idle`.
2. **De-duplication**: two components, one request.
3. **Cache**: on remount the very first render already had `data=Keyboard` (`status=success`) while `isFetching=true` refreshed it in the background — no spinner. With `staleTime: 60s` there was no refetch at all.
4. **Retries** hid two transient 503s from the user.
5. Errors arrive as `status: 'error'` with the thrown `Error`.

Choosing `staleTime`

`0` is safe but chatty. For data that changes rarely (product catalogue, user profile) a `staleTime` of 30 s–5 min removes most refetches. Set a default on the client — `new QueryClient({ defaultOptions: { queries: { staleTime: 60_000 } } })` — and override per query.

Common mistakes

Mistake	Fix
Copying query data into `useState`	Use `data` directly; derive during render
Creating `new QueryClient()` inside a component	Create it once (module scope, or `useState(() => new QueryClient())`)
Query function that doesn't throw on HTTP errors	Check `res.ok` and throw
Using `isLoading` to decide whether `data` exists	Use `isPending` (or check `data`)
Turning off all refetching "because it's noisy"	Raise `staleTime` instead

In the interview

30-second answer: "TanStack Query manages server state. `useQuery` takes a key and a fetch function; it caches by key, de-dupes concurrent requests, retries failures, and refetches stale data in the background on mount, focus

> and reconnect. `staleTime` says how long data is fresh — default 0 — and `gcTime` how long unused data stays cached — default 5 minutes. `isPending` means no data yet; `isFetching` means a request is in flight."

- *Why not Redux for server data?* — You'd hand-write caching, dedupe, refetching and invalidation; RTK Query or TanStack Query exist because that's hard to get right.
- *What happens on window focus?* — Stale queries with mounted observers refetch.

TL;DR

`useQuery({ queryKey, queryFn })`. Cache by key, dedupe, retry, background refetch of stale data. Tune `staleTime`; `isPending` vs `isFetching`.

9.9 Query keys, pagination and dependent queries

Query keys

A **query key** is an array that uniquely identifies the data: `['products']`, `['products', 7]`, `['products', { page: 2, sort: 'price' }]`. Rules:

- **Every variable the query function uses belongs in the key.** Change a variable → new key → new cache entry → fetch. Forgetting one means the query keeps showing the old data.
- Keys are compared by value (objects are hashed with sorted keys), so `{ page: 1, sort: 'x' }` and `{ sort: 'x', page: 1 }` are the same.
- Keys are **hierarchical**: invalidating `['products']` also matches `['products', 7]` and `['products', 7, 'reviews']` — design them general → specific.

`queryOptions()` bundles a key and function into one reusable definition you can pass to `useQuery`, `useSuspenseQuery`, `prefetchQuery`, `ensureQueryData` and `getQueryData` (with TypeScript, the data type flows through).

`code/08_query_keys_pagination.jsx`:

```
const productsPage = (page) => queryOptions({
  queryKey: ['products', { page }],              // variables go IN the key -> each page cached separately
  queryFn: () => getJson(`/api/products?page=${page}`),
});
const reviewsOf = (id) => queryOptions({ queryKey: ['products', id, 'reviews'], queryFn: () =>
getJson(`/api/products/${id}/reviews`) });

function Catalog() {
  const [page, setPage] = useState(1);
  const [selected, setSelected] = useState(null);
  const list = useQuery({ ...productsPage(page), placeholderData: keepPreviousData });  // keep old page while next
loads
  const reviews = useQuery({ ...reviewsOf(selected), enabled: selected !== null });     // waits until something is
picked
  if (list.isPending) return <p>Loading…</p>;
  return (
    <div style={{ opacity: list.isPlaceholderData ? 0.5 : 1 }}>
      <ul>{list.data.items.map(p => <li key={p.id}><button onClick={() => setSelected(p.id)}>{p.name}</button></li>)}
</ul>
      <button onClick={() => setPage(p => p - 1)} disabled={page === 1}>Prev</button>
      <button onClick={() => setPage(p => p + 1)} disabled={!list.data.hasMore || list.isPlaceholderData}>Next</button>
      {reviews.data && <p>Reviews: {reviews.data.join(' / ') || 'none yet'}</p>}
    </div>
  );
}
```

Output (each line is one render):

```
load page 1:
   page=1 showing=[] placeholder=false reviews=idle/pending
   page=1 showing=[Keyboard,Mouse] placeholder=false reviews=idle/pending
click Next (old page stays visible, dimmed, until page 2 arrives):
   page=2 showing=[Keyboard,Mouse] placeholder=true reviews=idle/pending
   page=2 showing=[Monitor] placeholder=false reviews=idle/pending
click Prev (page 1 is cached -> instant):
   page=1 showing=[Keyboard,Mouse] placeholder=false reviews=idle/pending
pick Mouse (the dependent reviews query is now enabled):
   page=1 showing=[Keyboard,Mouse] placeholder=false reviews=fetching/pending
   page=1 showing=[Keyboard,Mouse] placeholder=false reviews=idle/success
cache keys: ["products",{"page":1}]  ["products",null,"reviews"]  ["products",{"page":2}]  ["products",2,"reviews"]
```

- **Pagination without flicker**: normally a new key means `isPending` and a spinner. `placeholderData: keepPreviousData` keeps showing page 1 (flagged `isPlaceholderData`) until page 2 arrives. Going back to page 1 was instant — it was cached.
- **Dependent queries**: `enabled: false` keeps a query idle (`fetchStatus: 'idle'`, `status: 'pending'`) until its input exists. The cache listing shows the disabled `null` entry exists but never fetched.
- For "Load more" / infinite scroll, use **`useInfiniteQuery`**, which stores pages in one cache entry and gives you `fetchNextPage` and `hasNextPage`.

TL;DR

All inputs in the key; general → specific; `queryOptions` for reuse; `keepPreviousData` for pagination; `enabled` for dependent queries; `useInfiniteQuery` for infinite lists.

9.10 Mutations, invalidation and optimistic updates

What it is

A **mutation** changes data on the server (POST/PUT/PATCH/DELETE). **`useMutation({ mutationFn })`** gives you `mutate(variables)` plus `isPending`, `isError`, `error`, `data`. Unlike queries, mutations don't run automatically and aren't cached by key.

After a write, the cached queries that show that data are out of date. You fix that by **invalidating** them: `queryClient.invalidateQueries({ queryKey: ['todos'] })` marks matching queries stale and refetches the ones on screen.

How it works — two strategies

Strategy	Steps	Feels	Use when
Invalidate (pessimistic)	`mutate` → wait for server → invalidate → refetch → UI updates	Correct, slight delay	Default; complex server-side effects
Optimistic update	`onMutate`: cancel refetches, snapshot cache, write the expected result → server call → `onError`: restore snapshot → `onSettled`: invalidate	Instant	Likes, toggles, reordering, adding to lists — when the server almost always agrees

`code/07_mutations.jsx`:

```jsx
function Todos({ optimistic }) {
  const queryClient = useQueryClient();
  const [title, setTitle] = useState('');
  const todos = useQuery({ queryKey: ['todos'], queryFn: getTodos });

  const mutation = useMutation({
    mutationFn: addTodo,
    // Optimistic path: show the new item immediately, remember the old list for rollback.
    onMutate: optimistic ? async (newTitle) => {
      await queryClient.cancelQueries({ queryKey: ['todos'] });          // don't let a refetch overwrite us
      const previous = queryClient.getQueryData(['todos']);
      queryClient.setQueryData(['todos'], old => [...old, { id: 'temp', title: newTitle + ' (saving)' }]);
      return { previous };                                               // becomes the 3rd argument of onError
    } : undefined,
    onError: (err, newTitle, ctx) => { if (ctx?.previous) queryClient.setQueryData(['todos'], ctx.previous); },
    onSettled: () => queryClient.invalidateQueries({ queryKey: ['todos'] }),  // refetch the truth either way
  });

  if (todos.isPending) return <p>Loading…</p>;
  return (
    <form onSubmit={e => { e.preventDefault(); mutation.mutate(title); setTitle(''); }}>
      <ul>{todos.data.map(t => <li key={t.id}>{t.title}</li>)}</ul>
      <input aria-label="title" value={title} onChange={e => setTitle(e.target.value)} />
      <button disabled={mutation.isPending}>{mutation.isPending ? 'Adding…' : 'Add'}</button>
      {mutation.isError && <p role="alert">Could not add: {mutation.error.message}</p>}
    </form>
  );
}
```

Output:

```
  [server] start GET /api/todos
  [server] end   GET /api/todos -> 200
A) plain mutation + invalidate
   right after submit : Read chapter 9 | button: Adding…
  [server] start POST /api/todos
  [server] end   POST /api/todos -> 201
  [server] start GET /api/todos
  [server] end   GET /api/todos -> 200
   after refetch      : Read chapter 9 | Learn mutations
B) optimistic update
  [server] start GET /api/todos
  [server] start POST /api/todos
   right after submit : Read chapter 9 | Learn mutations | Ship it (saving)
  [server] end   GET /api/todos -> 200
  [server] end   POST /api/todos -> 201
  [server] start GET /api/todos
  [server] end   GET /api/todos -> 200
   after server + refetch: Read chapter 9 | Learn mutations | Ship it
C) optimistic update that fails
  [server] start POST /api/todos
   right after submit : Read chapter 9 | Learn mutations | Ship it | This will fail (saving)
  [server] end   POST /api/todos -> 500
  [server] start GET /api/todos
  [server] end   GET /api/todos -> 200
   after the 500      : Read chapter 9 | Learn mutations | Ship it | Could not add: Server refused
```

- **A**: the list updated only after the POST **and** the refetch.
- **B**: the new item appeared **immediately** (marked "saving"). The remount had started a background `GET`; `cancelQueries` made the client ignore its result so it couldn't overwrite the optimistic list. `onSettled` then replaced the temporary item with the server's version.
- **C**: the server refused; `onError` put the snapshot back, the item vanished, and the error message explains why. Always pair optimistic updates with a visible error.

Other tools

- `queryClient.setQueryData(key, data)` — write the server's response straight into the cache (skip a refetch) when the mutation returns the updated object.

- `mutateAsync` — returns a promise, for `await` in handlers (wrap in `try/catch`).
- **`useOptimistic`** (React 19, Chapter 4) gives optimistic UI for a single component's form action; TanStack's cache approach is better when many components show the same data.

TL;DR

`useMutation` for writes; `invalidateQueries` afterwards. Optimistic: cancel → snapshot → set → rollback on error → invalidate on settle.

9.11 Suspense and `use()`

What it is

`<Suspense fallback={…}>` is React's built-in way to show a placeholder while something inside it isn't ready — lazily loaded code or data. A component **suspends** when it asks for a value that isn't available yet; React shows the nearest Suspense boundary's fallback and retries the component when the value arrives.

`use(resource)` (React 19) is the API to read such a value during render:

- `use(promise)` returns the resolved value, **suspends** while pending, and **throws** the rejection to the nearest **error boundary**.
- `use(context)` reads context like `useContext`.
- Unlike Hooks, `use` **may be called inside `if` blocks and loops** (§7.2). It may not be called inside `try/catch`.

The one rule: don't create the promise during render

`use(fetch(url))` inside a component creates a **new** promise on every render, so React can never finish (it warns that a component "was suspended by an uncached promise"). The promise must be **created once and reused**: passed in as a prop from a Server Component or a loader, stored in a cache, or produced by a library (TanStack Query, the router).

`code/09_suspense_and_use.jsx`:

```
// A promise must be CREATED ONCE and reused across renders, so cache it (here: by id).
const cache = new Map();
function fetchProduct(id) {
  if (!cache.has(id)) cache.set(id, fetch(`${api.base}/api/products/${id}`).then(async r => {
    if (!r.ok) throw new Error(`HTTP ${r.status}`);
    return r.json();
  }));
  return cache.get(id);
}

const CurrencyContext = createContext('$');
function Product({ id, showPrice }) {
  const product = use(fetchProduct(id));                // suspends until resolved; throws to the boundary if rejected
  if (!showPrice) return <h2>{product.name}</h2>;
  const currency = use(CurrencyContext);                // unlike useContext, use() may sit inside an if
  return <h2>{product.name} {currency}{product.price}</h2>;
}

function Page() {
  const [id, setId] = useState(1);
  const [isPending, startTransition] = useTransition();
  return (
    <>
      <button onClick={() => setId(2)}>plain setId(2)</button>
      <button onClick={() => startTransition(() => setId(3))}>transition setId(3)</button>
      <button onClick={() => setId(9)}>missing</button>
      {isPending && <p>(switching…)</p>}
      <ErrorBoundary fallback={({ error, reset }) => <p role="alert">Failed: {error.message} <button onClick={() => {
reset(); setId(1); }}>Retry</button></p>}>
        <Suspense fallback={<p>Loading product…</p>}>
          <Product id={id} showPrice />
        </Suspense>
      </ErrorBoundary>
    </>
  );
}

// The same idea with TanStack Query: useSuspenseQuery never returns "pending" - data is always defined.
function SuspenseName({ id }) {
  const { data } = useSuspenseQuery({ queryKey: ['product', id], queryFn: () => fetchProduct(id) });
  return <p>useSuspenseQuery: {data.name}</p>;
}
```

The error boundary (`code/ErrorBoundary.jsx`) is a class component — React still has no Hook equivalent of `getDerivedStateFromError`; most apps use the small `react-error-boundary` package instead:

```
export class ErrorBoundary extends Component {
  state = { error: null };
  static getDerivedStateFromError(error) { return { error }; }       // render the fallback next
  componentDidCatch(error, info) { this.props.onError?.(error, info); } // log it (Sentry, console…)
  reset = () => this.setState({ error: null });
  render() {
    if (this.state.error) return this.props.fallback({ error: this.state.error, reset: this.reset });
    return this.props.children;
  }
}
```

Output (the visible text after each step):

```
first render      : Loading product…
resolved          : Keyboard $49
plain update      : Loading product… <- content replaced by the fallback
transition update : (switching…) Mouse $19 <- old content kept, isPending shown
then              : Monitor $199
rejected promise  : Failed: HTTP 404 Retry
after Retry       : Keyboard $49
useSuspenseQuery: Mouse
```

How it works

1. `Product` calls `use(promise)`; the promise is pending, so React stops rendering `Product` and shows the **closest** `<Suspense>` fallback.
2. When the promise settles, React re-renders `Product`; `use` now returns the value (or throws the error to the **closest error boundary**).
3. **Plain update** to an id whose data isn't loaded → the already-visible product is **hidden** and replaced by the fallback (a jarring flash).
4. **Transition update** (`startTransition`, Chapter 7) → React **keeps the old content** on screen and exposes `isPending` instead. Routers wrap navigations in transitions for exactly this reason.
5. `reset()` on the boundary clears the error and tries again (here with a different id; a real cache should also evict failed promises so a retry refetches).

Placing boundaries

- Put `<Suspense>` where a loading placeholder makes sense **visually** — around a card, a panel, a route — not around every component.
- Siblings inside one boundary are revealed **together** (no popcorn effect); separate boundaries let fast parts show first.
- Nest boundaries to stream progressively: page shell → main content → comments.
- Pair each important Suspense boundary with an **error boundary** outside it.

TL;DR

`use(promise)` + `<Suspense>` + error boundary. Cache the promise; never create it in render. Use transitions to keep old content instead of flashing fallbacks. `useSuspenseQuery` = Suspense with a cache.

9.12 Loading and error states that users like

Loading states — pick by situation

Situation	Show	Tool
First load of a page/section	**Skeleton** in the final layout (grey blocks) — avoids layout shift	Suspense fallback, `isPending`
Navigating between pages	**Old page + progress indicator** (top bar, dimming)	`useNavigation`, transitions
Refreshing data already shown	Keep data, small "refreshing" hint, or nothing	`isFetching` with data
Paginating / filtering	Previous results dimmed until new ones arrive	`keepPreviousData`, `useDeferredValue`
User action (save, like)	Disable button + inline "Saving…", or optimistic UI	`isPending` of mutation/action, `useOptimistic`
Very fast responses	**Nothing** — a spinner for 50 ms is a flicker	Delay spinners ~200–300 ms, or rely on transitions

Error states — every failure needs a home

Error kind	Where to handle	UI
Expected "not found" / 403	Route `ErrorBoundary` via `throw data(msg, { status })`, or query `error`	A friendly page inside the layout, with navigation still working
Transient network error	Retries (Query does 3 by default), then error UI	Message + **Retry** button (`refetch()` / boundary `reset`)
Validation error on submit	Action return value / mutation error	Inline, next to the field (`role="alert"`)
Bug (exception during render)	Error boundary at route level + one at the app root	"Something went wrong" + report to monitoring in `componentDidCatch`
Failure after optimistic update	`onError` rollback	Revert + toast explaining what didn't save

What error boundaries **don't** catch: errors in event handlers, in async code outside render (a `setTimeout`), and in the boundary itself. Handle those with `try/catch` and state, or route them into render (e.g. `mutation.error`).

Common mistakes

- A single spinner for the whole app on every fetch — users lose context.
- Showing an error page for a failed *background* refetch while good cached data exists — show the data and a subtle warning.
- Swallowing errors (`catch {}`) so the UI stays in "Loading…" forever.
- Not handling the empty state ("No results") separately from loading and error.

TL;DR

Skeletons for first loads, old content + indicator for navigations, nothing for fast refreshes. Route/query errors in boundaries with Retry; validation errors inline; roll back optimistic updates visibly.

9.13 Putting it together — loaders + TanStack Query

Loaders and TanStack Query solve different halves of the problem and combine well:

- The **router** knows *when* data is needed — as soon as a link is clicked — so its loaders **start** fetches early and in parallel.
- **TanStack Query** knows *whether* data is already cached and fresh, and keeps it updated afterwards.

The pattern: in the loader call `queryClient.ensureQueryData(options)` (return cached data or fetch it) for critical data, `prefetchQuery` (start, don't wait) for non-critical data, and read both in the component with the **same** `queryOptions` via `useSuspenseQuery`/`useQuery`.

`code/shop.jsx` (key parts):

```
const fresh = 30_000;                                          // treat data as fresh for 30 s
const productsQuery = queryOptions({ queryKey: ['products'], queryFn: () => getJson('/api/products'), staleTime: fresh
});
const productQuery = (id) => queryOptions({ queryKey: ['products', id], queryFn: () => getJson(`/api/products/${id}`),
staleTime: fresh });
const reviewsQuery = (id) => queryOptions({ queryKey: ['products', id, 'reviews'], queryFn: () =>
getJson(`/api/products/${id}/reviews`), staleTime: fresh });

function Product() {
  const { id } = useParams();
  const { data: product } = useSuspenseQuery(productQuery(id));   // already in the cache thanks to the loader
  const reviews = useQuery(reviewsQuery(id));                      // not awaited by the loader: streams in after
  // ... render product, reviews (or "Loading reviews…")
}

return [{
  path: '/', Component: Layout, ErrorBoundary: RouteError, HydrateFallback: () => <p>Loading shop…</p>,
  loader: () => queryClient.ensureQueryData(productsQuery),        // fetch if missing, reuse if cached
  children: [
    { index: true, Component: Home },
    { path: 'products/:id', Component: Product, ErrorBoundary: RouteError,
      loader: async ({ params }) => {
        queryClient.prefetchQuery(reviewsQuery(params.id));          // start, but don't wait (non-critical)
        await queryClient.ensureQueryData(productQuery(params.id));  // wait (critical)
        return null;
      } },
  ],
}];
```

`code/10_router_with_query.jsx` deep-links to `/products/1`, clicks around, then navigates to a missing product:

Output:

```
  [server] start GET /api/products
  [server] start GET /api/products/1/reviews
  [server] start GET /api/products/1
  [server] end   GET /api/products -> 200
  [server] end   GET /api/products/1/reviews -> 200
  [server] end   GET /api/products/1 -> 200
deep link /products/1 -> Keyboard | reviews: 2
  [server] start GET /api/products/3/reviews
  [server] start GET /api/products/3
  [server] end   GET /api/products/3/reviews -> 200
  [server] end   GET /api/products/3 -> 200
click Monitor         -> Monitor
back to Keyboard      -> Keyboard (served from the cache: no requests)
  [server] start GET /api/products/7/reviews
  [server] start GET /api/products/7
  [server] end   GET /api/products/7/reviews -> 200
  [server] end   GET /api/products/7 -> 404
/products/7           -> 404 - This product does not exist
```

- The deep link started **all three** requests at once (parent and child loaders run in parallel) — no waterfall.
- Returning to Keyboard made **zero** requests: loaders ran, but `ensureQueryData` found fresh cached data. Plain loaders would have refetched.
- The 404 was thrown as `data(…, { status: 404 })` from the query function and rendered by the route's `ErrorBoundary`.

In the browser (`code/shop.html`; `fetch` is mocked in `shop-main.jsx`):

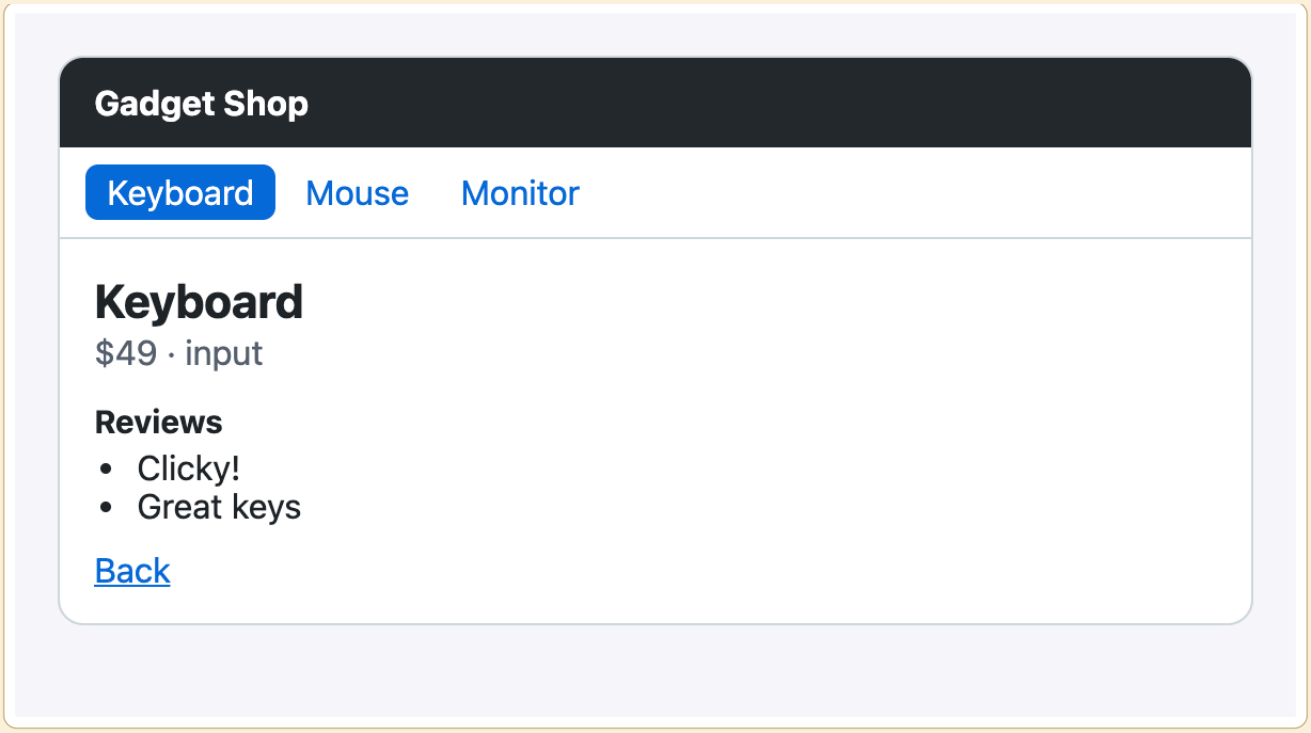


Figure 9.1 — The shop at `/products/1`. `NavLink` highlighted the active product; the product came from the loader-filled cache, the reviews from a prefetch.

In the interview

30-second answer: "I'd use React Router's data mode so loaders start fetching as soon as navigation begins, in parallel, and TanStack Query as the cache: the loader calls `ensureQueryData` with shared `queryOptions`, the component reads the same options with `useSuspenseQuery`. Critical data is awaited, non-critical data is prefetched and shown when ready. Mutations go through actions or `useMutation` and invalidate the affected keys. Errors are handled by route error boundaries, and navigations are transitions so the old page stays visible."

- *Why both a router loader and a query?* — The loader removes the render-then-fetch waterfall; the query cache removes repeated requests and keeps data fresh.
- *Where would you put the `QueryClient`?* — Created once outside components, provided at the root, and passed to route loaders.

Chapter 9 TL;DR

Idea	Remember
Client-side routing	History API + route matching; URL is shareable state
React Router v8	`npm i react-router`; `RouterProvider` from `react-router/dom`; declarative / data / framework modes
Routes	`path`, `:params` (`useParams`), `?search` (`useSearchParams`), `*` 404, `<Link>`/`<NavLink>`, `useNavigate`
Nesting	`<Outlet/>`, `index`, pathless layouts, relative links, `useOutletContext`
Loaders	Run before render, in parallel; `useLoaderData`; `useNavigation` for pending; `throw data(msg,{status})` + `ErrorBoundary`
Actions	`<Form method="post">` → `action` → errors (`useActionData`) or `redirect`; automatic revalidation; `useFetcher`
Middleware	Before/after loaders; auth guards; `createContext` + `context.get/set`
TanStack Query	Key + fn; cache, dedupe, retries, background refetch; `staleTime` (0) / `gcTime` (5 min); `isPending` vs `isFetching`
Keys & pagination	Every input in the key; `queryOptions`; `keepPreviousData`; `enabled`; `useInfiniteQuery`
Mutations	`useMutation` + `invalidateQueries`; optimistic: cancel → snapshot → set → rollback → invalidate
Suspense & `use()`	Cached promise only; fallback while pending; error boundary for rejections; transitions keep old UI
Loading/error UX	Skeletons first, old content during navigation, inline validation, Retry buttons, visible rollbacks

Next: Chapter 10 — Performance.

Chapter 10 — Performance

Most React apps are fast enough without any special effort. When one is not, the cause is almost always one of a small number of things: components rendering far more often than they need to, one expensive calculation repeated on every render, too many DOM nodes, or too much JavaScript downloaded up front. This chapter teaches you to **see** each problem first (with render counters, the `<Profiler>` API and React DevTools) and then fix it with the right tool: `memo`, `useMemo`, `useCallback`, React Compiler, code splitting with `lazy` + `<Suspense>`, and list virtualization.

Every demo in this chapter prints **numbers** — how many times a component rendered, how many DOM nodes exist, how many milliseconds React spent — so you can watch a fix work instead of taking it on trust.

What you'll learn

#	Topic	You will be able to…
10.1	The render cycle: trigger → render → commit	explain what "render" means in React and why a re-render is not a DOM update
10.2	When components re-render	predict which components re-render after a state, prop or context change
10.3	Measure first: DevTools Profiler and `<Profiler>`	find slow components with tools, not guesses
10.4	`memo` — skipping a child's re-render	wrap a component and know exactly when it helps
10.5	`useCallback` and `useMemo` for stable props	stop new functions and objects from defeating `memo`
10.6	`useMemo` for expensive calculations	cache a slow computation between renders
10.7	React Compiler	let the build step add memoization automatically
10.8	Code splitting with `lazy` and `<Suspense>`	load a component's code only when it is needed
10.9	List virtualization	render 10,000 rows with ~16 DOM nodes
10.10	Common performance bugs	recognise and fix the four mistakes that cause most slow React apps
10.11	A performance checklist	a step-by-step routine for a slow screen

Code: `code/` — React 19.3 in jsdom (a browser-like DOM for Node), driven with `@testing-library/react`. Timings from jsdom are only indicative (there is no real layout or paint), so the demos focus on **counts**, which are exact and repeatable. Background you may want first: state and batching (Chapter 3), Effects (Chapter 5), Context (Chapter 6), `useTransition`/`useDeferredValue` (Chapter 7).

10.1 The render cycle: trigger → render → commit

What it is

Every screen update in React goes through three steps:

1. **Trigger** — something asks for an update: the first `root.render(...)` call, or a state setter (`setCount(...)`) somewhere in the tree.
2. **Render** — React **calls your component functions** to find out what the UI should look like now. The result is a tree of React elements (plain JavaScript objects). Nothing on the screen changes yet.
3. **Commit** — React compares the new element tree with the previous one (this comparison is called **reconciliation**, introduced in Chapter 1) and applies **only the differences** to the real DOM. Then it runs layout effects (`useLayoutEffect`), the browser paints, and after that React runs ordinary Effects (`useEffect`).

Analogy: a restaurant. The *trigger* is a customer's order. The *render* is the chef writing down the plate on a ticket. The *commit* is the waiter changing only what differs on the table — if the new ticket matches what is already served, the waiter does nothing.



Why it matters

Two consequences drive the rest of this chapter:

- **"Re-render" does not mean "DOM update".** A re-render is just a function call. If the output is the same, the commit changes nothing in the DOM. Re-renders are cheap *unless* a component does heavy work while rendering or renders a huge subtree.
- **Render must be pure** (Chapter 2). Because React may call your component many times (and twice in development under StrictMode), render must not have side effects. That purity is what lets React skip, repeat or interrupt renders safely.

Seeing it

`code/01_render_cycle.jsx` logs from the render body, from a layout effect and from an effect, and counts real DOM changes with a `MutationObserver` (a browser API that reports every change to the DOM).

```
function Clock({ label }) {
  log(`  render  <Clock label="${label}">`);               // runs during the RENDER phase
  useLayoutEffect(() => log('  commit  React applied changes, then ran layout effects'));
  useEffect(() => log('  after   passive effects run (after paint)'));
  return <p>{label}</p>;
}

function App() {
  const [label, setLabel] = useState('hello');
  const [clicks, setClicks] = useState(0);
  return (
    <>
      <Clock label={label} />
      <button onClick={() => setLabel('world')}>change label</button>
      <button onClick={() => setClicks(c => c + 1)}>count {clicks}</button>
      <button onClick={() => setLabel(label)}>set same label</button>
    </>
  );
}
// ... a MutationObserver counts DOM mutations after each click ...
```

Output:

```
TRIGGER 1: initial render (createRoot().render)
  render  <Clock label="hello">
  commit  React applied changes, then ran layout effects
  after   passive effects run (after paint)
  DOM mutations: many (whole tree inserted)

TRIGGER 2: state change that changes Clock output
  render  <Clock label="world">
  commit  React applied changes, then ran layout effects
  after   passive effects run (after paint)
  DOM mutations: 1

TRIGGER 3: state change elsewhere - Clock re-renders, but its output is identical
  render  <Clock label="world">
  commit  React applied changes, then ran layout effects
  after   passive effects run (after paint)
  DOM mutations: 1  (only the button text changed; <p> untouched)

TRIGGER 4: setState with the SAME value (Object.is equal)
  DOM mutations: 0  (React bailed out: nothing rendered)
```

What to notice:

- Trigger 3: clicking **count** re-rendered `Clock` (its function ran) even though its props did not change — but the DOM received only one change, the button's text. React's diff protected the DOM.
- Trigger 4: setting state to a value that is `Object.is`-equal to the current one lets React **bail out** — it skips rendering that component and its children. (React may occasionally still call the component once before bailing out; that is harmless and its result is thrown away.)
- The effects ran on every commit here only because they have no dependency array (Chapter 5).

Common mistakes

- Treating every re-render as a bug. A re-render that takes 0.1 ms is not worth optimizing. Measure (§10.3) before changing code.
- Putting side effects (logging to a server, mutating globals) in the render body — they will run more often than you expect.

TL;DR

Trigger → render (call components, build elements) → commit (apply only the differences to the DOM, then effects). Re-rendering is calling a function; it touches the DOM only when the output changed.

10.2 When components re-render

What it is

A component re-renders in exactly these situations:

Cause	Example	Notes
Its own state changed	`setOpen(true)` inside it	Unless the new value is `Object.is`-equal to the old one
Its parent re-rendered	parent's state changed	The default: *all* children re-render, whether or not their props changed
A context it reads changed	`useContext(ThemeCtx)` and the provider's `value` changed	Happens even if the parent did not re-render and even through `memo`
An external store it subscribes to changed	Redux/Zustand selector result changed	Via `useSyncExternalStore` (Chapter 7)

What does **not** cause a re-render: a prop object being mutated in place, a `ref.current` change, or a plain variable changing. (That is why you never mutate state — React would not notice.)

A common myth is "a component re-renders when its props change". The more accurate rule: **a component re-renders when its parent re-renders**; props are not even checked unless you wrap the child in `memo` (§10.4).

Seeing it

`code/02_when_rerenders.jsx` gives each component a render counter.

```jsx
// Case A: a parent's state change re-renders ALL of its children (props or not)
function Child({ name }) { count(name); return <li>{name}</li>; }
function Static() { count('StaticNoProps'); return <li>static</li>; }
function Parent() {
  count('Parent');
  const [n, setN] = useState(0);
  return (<div>
      <button onClick={() => setN(n + 1)}>parent clicks {n}</button>
      <ul><Child name="ChildWithProps" /><Static /></ul>
    </div>);
}

// Case B: the same children passed in as JSX from ABOVE do NOT re-render
function Wrapper({ children }) {
  count('Wrapper');
  const [n, setN] = useState(0);
  return <div><button onClick={() => setN(n + 1)}>wrapper clicks {n}</button>{children}</div>;
}

// Case C: a context change re-renders every consumer, wherever it is
const ThemeCtx = createContext('light');
function Consumer() { count('ContextConsumer'); return <i>{useContext(ThemeCtx)}</i>; }
function NotConsumer() { count('NotAConsumer'); return <i>plain</i>; }
function ThemeProvider({ children }) {         // state lives here; children come from above
  const [theme, setTheme] = useState('light');
  return (<ThemeCtx value={theme}>
      <button onClick={() => setTheme(t => (t === 'light' ? 'dark' : 'light'))}>toggle theme</button>
      {children}
    </ThemeCtx>);
}
function Middle() { count('Middle'); return <><Consumer /><NotConsumer /></>; }
const ThemeApp = () => <ThemeProvider><Middle /></ThemeProvider>;

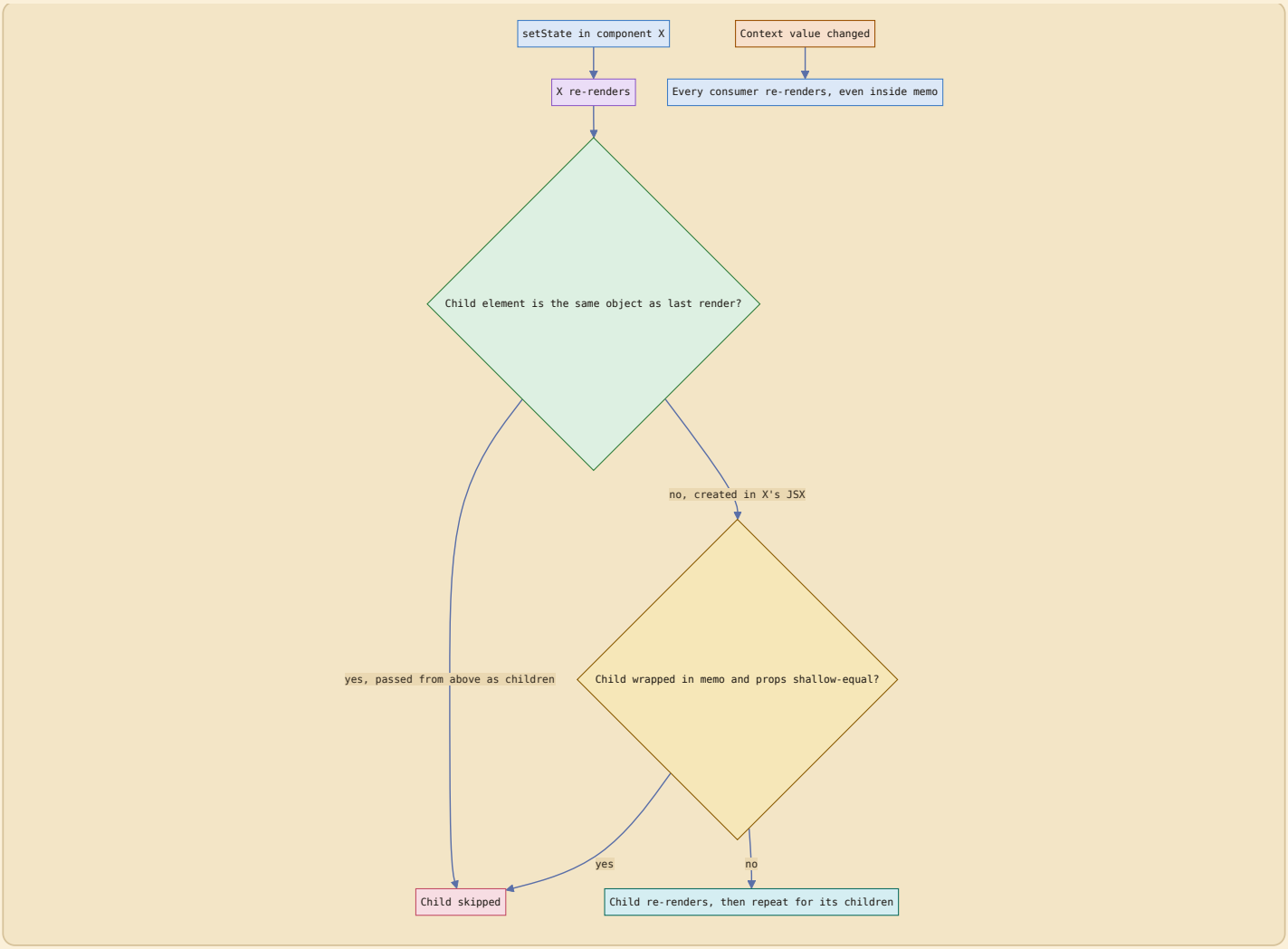
render(<><Parent /><Wrapper><Child name="ChildAsChildren" /></Wrapper><ThemeApp /></>);
// ... click each button and print the counters ...
```

Output:

```
mount
{"Parent":1,"ChildWithProps":1,"StaticNoProps":1,"Wrapper":1,"ChildAsChildren":1,"Middle":1,"ContextConsumer":1,"NotAConsumer":1}
Parent setState   {"Parent":1,"ChildWithProps":1,"StaticNoProps":1}
Wrapper setState  {"Wrapper":1}
theme change      {"ContextConsumer":1}
```

How it works

1. **Case A.** `Parent`'s state changed, so React called `Parent` again. Its JSX created **new** element objects for `<Child>` and `<Static>`, so React rendered both — even `Static`, which has no props at all.
2. **Case B.** `Wrapper`'s state changed, but `<Child name="ChildAsChildren">` was created by *Wrapper's parent*, which did not re-render. React received the **same element object** as last time in `children` and skipped it. This "pass JSX as children" trick is the cheapest optimization in React — no `memo` needed.
3. **Case C.** The provider changed `value`. React found every component that reads that context and re-rendered **only** those: `Middle` and `NotConsumer` were skipped (they came in as `children`), `Consumer` re-rendered.



Real-world examples

- A layout component that owns a "sidebar open" state and receives the page as `children` does not re-render the page when the sidebar toggles.
- Libraries such as Redux and Zustand let a component subscribe to a *slice* of state with a selector, so it re-renders only when that slice changes — this is why they scale better than one big context (Chapter 6).

In the interview

> **30-second answer:** "A component re-renders when its own state changes, when its parent re-renders, when a context it reads changes, or when a subscribed external store changes. Props being 'unchanged' does not stop a re-render by default — only `memo` checks props. Setting state to an identical value bails out. Elements passed as `children` from above keep their identity, so they are skipped when the wrapper re-renders."

- *Does changing a ref cause a re-render?* — No. Refs are deliberately invisible to rendering.
- *Does `memo` stop context updates?* — No. A memoized component that reads a changed context still re-renders.

Self-check

1. `Parent` re-renders; `Child` has no props. Does `Child` re-render? → Yes, unless it is wrapped in `memo` or passed in from above as `children`.
2. Why did `ChildAsChildren` not re-render in Case B? → It is the same element object, created by a component that did not re-render.
3. Which components re-render when a context value changes? → Those that read that context (plus whatever they render).

TL;DR

Re-render triggers: own state, parent re-render, context, external store. Props are not compared unless you use `memo`. "Children from above" and state kept low are free optimizations.

10.3 Measure first: DevTools Profiler and `<Profiler>`

Why it matters

Optimizing without measuring usually fixes the wrong thing and adds complexity. React offers three measuring tools:

Tool	Where	What it shows
React DevTools → Profiler tab	browser extension (Chrome, Firefox, Edge)	Records commits; a *flamegraph* (the tree, bar width = render time) and a *ranked* chart (slowest first); "why did this render?"
React Performance Tracks (React 19.2+)	Chrome DevTools → Performance panel	React's scheduler work (blocking vs transition updates) and a component flamegraph on the same timeline as network and JavaScript
`<Profiler>` component	in your code	A callback with timings for every commit of a subtree — for tests, logging, or monitoring

Using React DevTools (step by step)

1. Install **React Developer Tools** for your browser and open your app **in development mode** (DevTools shows production apps too, but profiling needs a development or profiling build).
2. In React DevTools settings (the gear icon) → **General**, turn on **"Highlight updates when components render"**. Now every re-render flashes an outline — you *see* what re-renders as you type or click.
3. In the settings → **Profiler** section, enable **"Record why each component rendered while profiling"**.
4. Press **Record**, perform the slow interaction once, press **Stop**.
5. Pick the tallest commit bar at the top (the slowest commit). Read the **flamegraph**: grey bars did not render in this commit; coloured bars did, and warmer colours took longer. Click a bar to see *why* it rendered ("Props changed: onAdd", "Hook 2 changed", "The parent component rendered").
6. Switch to **Ranked** to list the slowest components first. Fix the top one, record again, compare.

Tip: to measure what users actually experience, profile a **production build** too (add the profiling build, see below) and throttle the CPU (Chrome Performance panel → CPU 4× slowdown) so your fast laptop behaves like an average phone.

The `<Profiler>` API

Wrap any subtree in `<Profiler id="..." onRender={fn}>`. After each commit that involves that subtree, React calls:

```
onRender(id, phase, actualDuration, baseDuration, startTime, commitTime)
```

Parameter	Meaning
`id`	the `id` prop, to tell several profilers apart
`phase`	`"mount"` (first render), `"update"` (re-render), or `"nested-update"` (an update scheduled during a layout effect)
`actualDuration`	ms spent rendering this subtree **in this commit** — falls when memoization works
`baseDuration`	ms an estimated *full* re-render of the subtree would take (the sum of each component's latest render time) — the worst case
`startTime`, `commitTime`	timestamps; all profilers in one commit share `commitTime`

`code/09_profiler_api.jsx`:

```
const samples = [];
function onRender(id, phase, actualDuration, baseDuration, startTime, commitTime) {
  samples.push({ id, phase, actual: actualDuration, base: baseDuration, commitTime });
}
const busy = ms => { const t = performance.now(); while (performance.now() - t < ms) {} };

const Sidebar = memo(function Sidebar() { busy(3); return <nav>links</nav>; });   // memoized, 3 ms
function Feed({ posts }) { busy(5); return <section>{posts} posts</section>; }    // 5 ms every render

function App() {
  const [posts, setPosts] = useState(10);
  return (
    <Profiler id="app" onRender={onRender}>
      <button onClick={() => setPosts(p => p + 1)}>load more</button>
      <Profiler id="sidebar" onRender={onRender}><Sidebar /></Profiler>
      <Profiler id="feed" onRender={onRender}><Feed posts={posts} /></Profiler>
    </Profiler>
  );
}
render(<App />);
fireEvent.click(screen.getByText('load more'));
fireEvent.click(screen.getByText('load more'));
// ... print the samples, rounded to whole ms ...
```

Output (exact milliseconds vary from run to run):

```
id       phase    actual  base  (ms)
sidebar  mount         3     3
feed     mount         5     5
app      mount        10     9
sidebar  update        0     3
feed     update        5     5
app      update        5     8
sidebar  update        0     3
feed     update        5     5
app      update        5     8

9 onRender calls in 3 commits (profilers in one commit share commitTime)
```

How to read it: on updates, `sidebar` has `actual 0` but `base 3` — memoization saved its full 3 ms. `feed` has `actual = base = 5` — it re-renders completely every time, so it is the thing to look at next. `app`'s `actual` (5) is well below its `base` (8): the tree as a whole is being partly skipped.

Profiling in production

Profiling adds overhead, so the normal production build of React **disables** it (`onRender` is not called there). To profile production code, use React's profiling build by aliasing `react-dom/client` to `react-dom/profiling` in your bundler — some frameworks have a flag for this. Never ship the profiling build to all users without a reason.

Real-world examples

- Teams send `<Profiler>` samples from a small share of sessions (profiling build) to their metrics system to catch regressions.
- Test suites assert render counts for critical components, as this chapter's demos do, so an accidental "everything re-renders" change fails CI.
- Web Vitals such as **INP** (Interaction to Next Paint: how long the page takes to respond visually to a click or key press) are what users feel; React profiling explains *why* an interaction is slow.

Common mistakes

- Profiling only a development build on a fast laptop — development React is much slower than production, and your users' phones are slower than your laptop.
- Wrapping everything in `<Profiler>` permanently. Each one has a cost; add them where you are investigating.

TL;DR

Highlight updates to *see* re-renders, record with the DevTools Profiler to find the slowest commit and the reason, and use `<Profiler onRender>` when you need numbers in code. Compare `actualDuration` with `baseDuration` to see what memoization saves.

10.4 `memo` — skipping a child's re-render

What it is

`memo(Component)` returns a new component that React **skips re-rendering** when its parent re-renders and every prop is the same as last time. "The same" means a **shallow** comparison: each prop compared with `Object.is` (like `===`, except `NaN` equals `NaN` and `+0` differs from `-0`).

```
import { memo } from 'react';
const Row = memo(function Row({ product, onAdd }) { /* ... */ });
// optional second argument: arePropsEqual(prevProps, nextProps) => true to skip
```

Analogy: a receptionist who, if a request is identical to the last one, hands back the previous answer instead of interrupting a busy colleague.

How it works

1. The parent re-renders and produces `<Row product={p} onAdd={fn} />`.
2. React compares the new props with the previous ones, one key at a time, with `Object.is`.
3. All equal → React reuses the previous result and does not call `Row` (nor its children).
4. Any prop different → `Row` renders normally.

A memoized component **still** re-renders when its own state changes or a context it reads changes — `memo` only filters re-renders caused by the parent.

The catch: primitives compare by value, objects by identity

`Object.is(3, 3)` is `true`, but `Object.is({}, {})` and `Object.is(() => {}, () => {})` are `false`. Every render of the parent creates **new** objects, arrays and functions for inline props (`style={{ padding: 0 }}`, `onAdd={id => ...}`), so a memoized child receiving them re-renders every time and `memo` does nothing except add a comparison. §10.5 fixes that.

Measuring it: before and after

`code/03_memo_before_after.jsx` builds a page with a search box and 300 product rows, and types five characters into the box. Typing has nothing to do with the list, so ideally the rows should not render at all. A `<Profiler>` wraps the list and each `Row` increments a counter.

```
function Row({ product, onAdd }) {
  rowRenders++;
  let x = 0; for (let i = 0; i < 2000; i++) x += i;           // pretend work
  return <li>{product.name} ${product.price} <button onClick={() => onAdd(product.id)}>add</button></li>;
}
const MemoRow = memo(Row);
function List({ items, onAdd, RowComponent, style }) {
  return <ul style={style}>{items.map(p => <RowComponent key={p.id} product={p} onAdd={onAdd} />)}</ul>;
}
const MemoList = memo(List);

// V1 - no memoization at all
function PageV1() {
  const [query, setQuery] = useState('');
  const [cart, setCart] = useState([]);
  const onAdd = id => setCart(c => [...c, id]);
  return (<>
      <input aria-label="search" value={query} onChange={e => setQuery(e.target.value)} />
      <Profiler id="list" onRender={onRender}>
        <List items={PRODUCTS} onAdd={onAdd} RowComponent={Row} style={{ padding: 0 }} />
      </Profiler>
      <p>cart: {cart.length}</p>
    </>);
}
// V2 - same, but <MemoList ... RowComponent={MemoRow} ...>: new onAdd and new style each render
// V3 - memo + stable props:
//   const onAdd = useCallback(id => setCart(c => [...c, id]), []);
//   const style = useMemo(() => ({ padding: 0 }), []);
```

Output:

```
V1 no memo                        typing 5 chars -> Row renders: 1500, list profiler commits: 5 | click add -> Row
renders: 300
V2 memo, unstable props           typing 5 chars -> Row renders: 1500, list profiler commits: 5 | click add -> Row
renders: 300
V3 memo + useCallback/useMemo     typing 5 chars -> Row renders:    0, list profiler commits: 5 | click add -> Row
renders: 0

actualDuration while typing (ms, varies per run): V1 32.6, V2 26.5, V3 0.0
```

Reading the numbers:

- **V1:** 5 keystrokes × 300 rows = 1,500 row renders for nothing.
- **V2:** adding `memo` changed **nothing** — `onAdd` and `style` were new every render, so the shallow comparison failed every time. This is the most common way `memo` is misused.
- **V3:** with stable props, typing renders **zero** rows and the list's `actualDuration` drops to 0. Clicking **add** changes `cart`, which the list does not receive, so again zero rows render.
- The profiler still reports 5 commits in V3: `onRender` fires whenever the surrounding tree commits, even when the subtree was skipped (with `actualDuration` near 0).

When `memo` is worth it

- The component renders often with the **same props**, and
- its render is noticeably expensive (large subtree, heavy calculation), and
- you can keep its props stable (primitives, or memoized objects/functions).

When not to use it

- Cheap components (a button, a label), or props that change on nearly every render anyway.
- When moving state down or passing JSX as `children` (§10.10) removes the re-render entirely.
- In a project using **React Compiler** (§10.7), which memoizes for you.

Custom comparison

`memo(Chart, (prev, next) => prev.data.version === next.data.version)` lets you define equality yourself. Be careful: if you return `true` while a callback prop changed, the child keeps calling a **stale** function (one that captured old state). Compare every prop, or prefer making props stable.

Common mistakes

- Wrapping a component in `memo` and passing inline objects/functions/JSX — see V2.
- Passing `children` to a memoized component: `<MemoCard><p>hi</p></MemoCard>` creates a new `children` element every render, so `MemoCard` always re-renders.
- Deep-equality comparisons in `arePropsEqual` on big data — they can cost more than the render.

In the interview

> **30-second answer:** "`memo` makes a component skip re-rendering when its parent re-renders with shallow-equal props, compared with `Object.is`. It does not stop re-renders from the component's own state or from context. It only helps if props are stable, so it usually goes with `useCallback`/`useMemo`. I'd use it for expensive components that re-render often with the same props, after profiling — and with React Compiler I mostly don't need it."

- *Why can `memo` make things slower?* — It adds a props comparison on every parent render; if props always differ, you pay for the comparison and the render.
- *Is `memo` a guarantee?* — No; React treats it as a performance hint, not a semantic guarantee, so code must still be correct if the component re-renders.

TL;DR

`memo` = skip the re-render if props are shallow-equal. Objects and functions created during render are never equal, so stabilise them — or use React Compiler.

10.5 `useCallback` and `useMemo` for stable props

What it is

Both Hooks **cache a value between renders** and hand back the *same* reference until a dependency changes:

```
const onAdd = useCallback(id => setCart(c => [...c, id]), []);   // caches the FUNCTION
const style = useMemo(() => ({ padding: 0 }), []);                 // caches the RESULT of calling the function
```

`useCallback(fn, deps)` is exactly `useMemo(() => fn, deps)`. React compares each dependency with `Object.is` against the previous render; if all are equal it returns the cached value, otherwise it stores and returns the new one.

How it works

1. First render: React stores `[value, deps]` in the component's Hook slot (Chapter 7).
2. Next render: React compares new deps with stored deps.
3. Equal → return the stored value (same reference, so a memoized child sees an unchanged prop).
4. Different → use the new value and store it.

Keeping dependencies small

- Use an **updater function** (`setCart(c => [...c, id])`) instead of reading `cart` directly; then the callback needs no `cart` dependency and stays stable (Chapter 3).
- Move objects that do not depend on props or state **out of the component** (`const STYLE = { padding: 0 }` at module level) — no Hook needed.
- Pass **primitives** (`userId`) instead of objects (`user`) when the child only needs one field.

When they help — and when they don't

Situation	Worth it?
Passing a function/object to a `memo` child	Yes — otherwise `memo` is useless (V2 above)
The value is a dependency of `useEffect` and must not re-trigger it every render	Yes (or restructure the Effect, Chapter 5)
A custom Hook returns functions that callers may put in dependency arrays	Yes — callers expect stable functions
Passing a callback to a plain DOM element (`<button onClick>`)	No — DOM elements are not memoized
Passing to a child that is not wrapped in `memo`	No — the child re-renders anyway
Everything, "just in case"	No — adds noise and memory; hides real problems

Common mistakes

- **Missing dependencies.** `useCallback(() => save(draft), [])` keeps the first `draft` forever (a *stale closure*). Let `eslint-plugin-react-hooks` check the list.
- **Unstable dependencies.** `useMemo(() => filter(items, opts), [items, opts])` where `opts` is a new object every render — the cache never hits.
- Thinking `useMemo` guarantees the cache survives. React may discard cached values (for example, during development file edits), so the code must still be correct without the cache.

TL;DR

`useCallback` caches a function, `useMemo` caches a computed value; both keep the same reference until deps change. They matter when the reference is compared — by `memo`, by an Effect's dependencies, or by a caller.

10.6 `useMemo` for expensive calculations

What it is

The second job of `useMemo`: avoid **re-doing a slow calculation** on renders where its inputs did not change.

```
const visible = useMemo(() => filterTodos(todos, tab), [todos, tab]);
```

Seeing it

`code/04_usememo_expensive.jsx` filters 5,000 todos with a function that takes ~2 ms, then causes 4 unrelated re-renders (theme toggles) and 1 relevant one (tab change).

```
function Plain() {
  const [tab, setTab] = useState('all');
  const [dark, setDark] = useState(false);              // unrelated state
  const visible = filterTodos(TODOS, tab);              // recomputed on EVERY render
  return <View {...{ visible, dark, setDark, setTab }} />;
}
function Memoized() {
  const [tab, setTab] = useState('all');
  const [dark, setDark] = useState(false);
  const visible = useMemo(() => filterTodos(TODOS, tab), [tab]);  // cached until `tab` changes
  return <View {...{ visible, dark, setDark, setTab }} />;
}
```

Output:

```
without useMemo: 6 renders, filterTodos ran 6x, shows "1667 visible"
with useMemo   : 6 renders, filterTodos ran 2x, shows "1667 visible"
```

The memoized version ran the calculation only on mount and when `tab` changed.

How do I know a calculation is "expensive"?

Time it with `console.time('filter')` / `console.timeEnd('filter')`, ideally with CPU throttling. React's docs suggest memoizing when a calculation takes around **1 ms or more**. Sorting or filtering thousands of items, building chart data and parsing large text qualify; `a + b` or mapping 20 items does not.

Alternatives to consider first

- **Do it outside render**: compute once when data arrives (in the fetch handler, the reducer, or on the server).
- **Keep less state**: derived values (Chapter 3) are fine to recompute when cheap.
- **Don't block typing**: if the slow part is rendering results while the user types, `useDeferredValue` or `useTransition` (Chapter 7) keep input responsive; combine them with `memo` on the slow list.

Common mistakes

- Side effects inside the `useMemo` function (fetching, setting state). The function must be pure; in development StrictMode React calls it twice to help you notice.
- Expecting `useMemo` to make the **first** render faster — it only helps later renders.

TL;DR

`useMemo(() => slow(a, b), [a, b])` skips the slow work until `a` or `b` changes. Measure first; ~1 ms or more is the rough threshold.

10.7 React Compiler

What it is

React Compiler is a build-time tool (a Babel plugin, `babel-plugin-react-compiler`) that reads your components and Hooks and **automatically adds memoization** — the equivalent of well-placed `useMemo`, `useCallback` and `memo` — without you writing any of them. Version **1.0** was released as stable on **7 October 2025**. It works best with React 19 and also supports React 17 and 18 (those need the extra `react-compiler-runtime` package and a `target` option).

Analogy: a spell-checker for performance that also fixes the spelling for you, but only in sentences it fully understands.

How it works

1. At build time the compiler analyses each component and Hook: which values depend on which props and state.
2. It rewrites the function to keep a small **cache array** per component instance (via `useMemoCache`, imported from `react/compiler-runtime`).
3. Each computed value and each JSX element is recomputed only when its specific inputs changed; otherwise the cached one is reused. Returning the *same* JSX element object lets React skip that child entirely (the same effect as "children from above" in §10.2) — which is why `memo` is typically unnecessary.
4. If a component breaks the Rules of React (mutating props, reading refs during render, side effects in render), the compiler **skips** that component rather than risk changing its behaviour. Your code still runs, just unoptimised.

Seeing it — what the compiler produces (by hand)

We don't run the Babel plugin in this book's toolchain, so `code/05_compiler_output.jsx` contains a **hand-simplified** version of the kind of code the compiler generates, using the real `c` function from `react/compiler-runtime`. Real output differs in naming and detail.

```jsx
// ---- what you write ----
function ProductPage({ products, theme }) {
  const visible = products.filter(p => p.inStock);
  const onBuy = id => console.log('buy', id);
  return <div className={theme}><ProductList items={visible} onBuy={onBuy} /></div>;
}

// ---- roughly what the compiler turns it into ----
import { c as _c } from 'react/compiler-runtime';
function ProductPageCompiled({ products, theme }) {
  const $ = _c(6);                                          // 6 cache slots for this component instance
  let visible;
  if ($[0] !== products) {                                  // input changed -> recompute
    visible = products.filter(p => p.inStock);
    $[0] = products; $[1] = visible;
  } else visible = $[1];                                    // unchanged -> reuse the same array
  let onBuy;
  if ($[2] === Symbol.for('react.memo_cache_sentinel')) {   // first render only: the function uses no props
    onBuy = id => console.log('buy', id);
    $[2] = onBuy;
  } else onBuy = $[2];
  let list;
  if ($[3] !== visible || $[4] !== onBuy) {                 // the JSX element itself is cached
    list = <ProductList items={visible} onBuy={onBuy} />;
    $[3] = visible; $[4] = onBuy; $[5] = list;
  } else list = $[5];
  return <div className={theme}>{list}</div>;
}
```

`ProductList` is **not** wrapped in `memo`. Changing `theme` three times:

Output:

```
hand-written code  : 4 renders of the page -> filter ran 4x, ProductList rendered 4x
compiler-style code: 4 renders of the page -> filter ran 1x, ProductList rendered 1x
```

You can see the real output for any component in the online **React Compiler Playground** (linked from react.dev).

Setting it up (not run here)

Install the plugin with an exact version, and make sure it runs **first** among Babel plugins:

```
npm install --save-dev --save-exact babel-plugin-react-compiler@latest
```

```jsx
// babel.config.js
module.exports = {
  plugins: ['babel-plugin-react-compiler'],   // must be first
};
```

```jsx
// vite.config.js — @vitejs/plugin-react v6 and later (also: npm install -D @rolldown/plugin-babel)
import { defineConfig } from 'vite';
import react, { reactCompilerPreset } from '@vitejs/plugin-react';
import babel from '@rolldown/plugin-babel';
export default defineConfig({
  plugins: [react(), babel({ presets: [reactCompilerPreset()] })],
});

// older @vitejs/plugin-react versions:
//   plugins: [react({ babel: { plugins: ['babel-plugin-react-compiler'] } })]
```

- **Next.js** and **Vite** project templates offer it at creation time (`create-next-app`, `create-vite`); **Expo SDK 54+** enables it by default.
- The compiler's lint rules now ship in **`eslint-plugin-react-hooks`** (its `recommended` config), replacing the older separate `eslint-plugin-react-compiler`. They flag code the compiler cannot optimise, such as setting state during render or reading `ref.current` during render.

- **Verify**: in React DevTools, compiled components show a **"Memo ✨"** badge.
- **Opt out** of one component or file temporarily with the directive `"use no memo"` at the top of its body; fix the underlying issue and remove it later.
- Adopt **incrementally** on large codebases (react.dev has a guide: compile one directory first, or gate it behind a flag), and pin the exact version if your test coverage is thin, because a new compiler version can memoize differently.

What happens to `useMemo`, `useCallback` and `memo`?

- They still work and the compiler respects them. You don't need to delete them.
- For **new** code in a compiled project, rely on the compiler and use `useMemo` / `useCallback` only as **escape hatches** — for example, to guarantee that a value used as an Effect dependency keeps its identity.
- The compiler can memoize things the Hooks cannot, such as values computed **after an early return**.
- The compiler does not fix algorithmic problems (an O(n²) loop is still slow), huge DOM trees (use virtualization), or large bundles (use code splitting).

Real-world examples

The React Compiler 1.0 announcement (react.dev blog, October 2025) cites the Meta Quest Store: initial loads and cross-page navigations up to 12% faster, some interactions more than 2.5× faster, memory unchanged. Treat such numbers as examples, not a promise for your app — measure.

In the interview

> **30-second answer:** "React Compiler is a build-time Babel plugin, stable since v1.0 in October 2025, that automatically memoizes components and Hooks — values, callbacks and JSX — based on a static analysis of their dependencies. It relies on components following the Rules of React and skips ones that don't. With it, manual `memo` / `useMemo` / `useCallback` become mostly unnecessary escape hatches. It doesn't solve slow algorithms, big DOM or bundle size."

- *How do you check it's working?* — The "Memo ✨" badge in React DevTools; the playground shows the transformed code.
- *What if a component misbehaves after compiling?* — Add `"use no memo"` to it, find the Rules-of-React violation (the lint rules help), fix, remove the directive.

TL;DR

React Compiler adds memoization at build time using a per-component cache. Enable it for new projects, follow the Rules of React, and keep manual memo Hooks as escape hatches.

10.8 Code splitting with `lazy` and `<Suspense>`

What it is

By default a bundler puts all your code into one JavaScript file. The browser must download, parse and run **all** of it before the first screen works — including the chart library for a page the user may never open. **Code splitting** breaks the bundle into chunks that load **on demand**.

The building block is the JavaScript **dynamic import**, `import('./chart.jsx')`, which returns a Promise of the module. Bundlers (Vite, webpack, esbuild, Rollup) turn each dynamic import into a separate file. React's `lazy` turns that Promise into a component:

```
import { lazy, Suspense } from 'react';
const HeavyChart = lazy(() => import('./heavy_chart.jsx'));   // at MODULE level

<Suspense fallback={<p>Loading chart...</p>}>
  <HeavyChart points={data} />
</Suspense>
```

How it works

1. `lazy(load)` returns a placeholder component. Nothing is downloaded yet.

2. The first time React tries to render it, React calls `load()`. The Promise is pending, so the component **suspends**.
3. React shows the `fallback` of the **nearest** `<Suspense>` **boundary** above it.
4. When the Promise resolves to a module, React uses its `default` **export** as the component and renders it in place of the fallback. React caches the result — `load` runs only once.
5. If the Promise rejects (the network failed), the error is thrown during rendering and caught by the nearest **error boundary**.

Seeing it — real chunk files

Bundling a two-route app with esbuild's code splitting enabled (`--splitting --format=esm --minify`; a large data array lives in `chart.jsx`):

```
// main.jsx
import { lazy } from 'react';
export const HeavyChart = lazy(() => import('./chart.jsx'));
export const Settings = lazy(() => import('./settings.jsx'));
```

Output (sizes and hashes depend on the input files):

```
  dist/chart-GX2X5FGR.js     22.9kb
  dist/main.js                153b
  dist/settings-7FL34NKO.js   114b
```

and `dist/main.js` now contains `import("./chart-GX2X5FGR.js")` — the 22.9 kB chunk is fetched only when `HeavyChart` first renders. (The hash in the name changes when the file's content changes, so browsers can cache chunks forever.)

Seeing it — loading states and failures

`code/06_lazy_suspense.jsx` (the chunk load is delayed by 100 ms to imitate the network):

```
const delay = ms => new Promise(r => setTimeout(r, ms));

// 1. Declare lazy components at MODULE level (never inside a component).
const HeavyChart = lazy(() => delay(100).then(() => import('./heavy_chart.jsx')));
// lazy() needs a default export; for a named export, map it to { default }.
const Legend = lazy(() => import('./heavy_chart.jsx').then(m => ({ default: m.Legend })));
// A chunk that fails to load (e.g. a deploy replaced the file, or the user went offline).
const Broken = lazy(() => delay(10).then(() => { throw new Error('Failed to fetch dynamically imported module'); }));

class ChunkErrorBoundary extends Component {                  // error boundaries are still classes
  state = { error: null };
  static getDerivedStateFromError(error) { return { error }; }
  render() { return this.state.error ? <p role="alert">Could not load: {this.state.error.message}</p> :
this.props.children; }
}

function Dashboard() {
  const [tab, setTab] = useState('summary');
  return (
    <main>
      <button onClick={() => setTab('chart')}>chart tab</button>
      <button onClick={() => setTab('broken')}>broken tab</button>
      {tab === 'summary' && <p>summary text</p>}
      <Suspense fallback={<p>Loading chart...</p>}>
        {tab === 'chart' && <><HeavyChart points={[1, 2, 3]} /><Legend /></>}
      </Suspense>
      <ChunkErrorBoundary>
        <Suspense fallback={<p>Loading...</p>}>{tab === 'broken' && <Broken />}</Suspense>
      </ChunkErrorBoundary>
    </main>
  );
}
```

`heavy_chart.jsx` prints a line when its code is evaluated. Output:

```
first render             -> summary text
clicked chart tab        -> Loading chart...
    [module] heavy_chart.jsx evaluated (its code was just loaded)
after the chunk loaded   -> chart with 3 points | legend
broken tab               -> Could not load: Failed to fetch dynamically imported module
```

The module did not run until the chart tab was opened. One `<Suspense>` boundary covered both lazy components, so the user saw one loading message, not two.

Where to split

Split point	Why
Routes / pages	The natural boundary: users view one page at a time. React Router supports a `lazy` property on routes, and frameworks such as Next.js split per route automatically (Chapter 9, Chapter 13)
Heavy, rarely used widgets	Rich-text editors, charts, maps, PDF viewers, admin panels
Below-the-fold or behind-a-click UI	Modals, settings drawers, "advanced" tabs
Not worth it	Small components; things on the first screen (splitting them just adds a request and a spinner)

Making it feel fast

- **Preload on intent**: start the download when the user hovers or focuses the link, before the click. Calling the same `import()` early is enough — the module system caches it:

  ```
  const loadChart = () => import('./heavy_chart.jsx');
  const HeavyChart = lazy(loadChart);
  <button onMouseEnter={loadChart} onFocus={loadChart} onClick={() => setTab('chart')}>Chart</button>
  ```

- **Place `<Suspense>` boundaries deliberately**: one around the whole page replaces everything with a spinner; one around each widget keeps the rest usable. Skeletons (grey shapes of the final layout) avoid layout jumps.
- **Don't hide already-visible content**: if switching tabs would replace visible content with a fallback, wrap the state update in `startTransition` — React then keeps showing the old tab until the new one is ready (Chapter 7).
- **Handle chunk failures**: after a new deployment, old chunk files may be gone. An error boundary with a "Reload" button is the common fix.

Common mistakes

- **Declaring `lazy` inside a component.** Each render creates a new lazy component type, so React remounts it and state is lost. Always declare at module level.
- **Named export without mapping** — `lazy(() => import('./x'))` needs a `default` export; otherwise map as shown above.
- **No `<Suspense>` above a lazy component** — React needs a boundary to show something while it loads.
- **Splitting everything** — dozens of tiny chunks mean many requests and waterfalls.

How to find what to split

Use a bundle analyser (`rollup-plugin-visualizer` for Vite/Rollup, `webpack-bundle-analyzer`, or `source-map-explorer`) to see which libraries take space, and Chrome DevTools' **Coverage** tab to see how much downloaded JavaScript never ran.

In the interview

30-second answer: "Code splitting moves code that isn't needed immediately into separate chunks loaded on demand. In React you write `const Page = lazy(() => import('./Page'))` at module level and render it inside `<Suspense fallback>`; the component suspends until the chunk loads. Split at routes and heavy widgets, preload on hover, use an error boundary for failed loads, and use `startTransition` so navigation doesn't flash a spinner over existing content."

- *Why must the module have a default export?* — `lazy` reads the resolved module's `default` property; map named exports to `{ default }`.

TL;DR

`lazy(() => import(...))` at module level + `<Suspense fallback>` + an error boundary = on-demand chunks. Split by route and heavy widget; preload on intent.

10.9 List virtualization

What it is

Virtualization (also called **windowing**) renders only the list rows that are currently visible in a scrollable area (plus a few extra), instead of all of them. As the user scrolls, rows leaving the window are removed and rows entering it are added. A tall empty spacer keeps the scrollbar the correct size, so the list *looks* complete.

Analogy: a train window. The landscape is kilometres long, but you only ever see the part framed by the window — and no one builds the whole landscape in front of you.

Why it matters

Every row costs DOM nodes, memory, style and layout work; 10,000 rows means tens of thousands of nodes, and rendering, scrolling and even typing slow down. Memoization cannot help — the problem is the number of nodes, not re-renders.

How it works (fixed-height rows)

1. Know the row height (`rowHeight`), the viewport height (`height`), and the current `scrollTop` (how many pixels the user has scrolled).
2. First visible row = `floor(scrollTop / rowHeight)`; number of visible rows = `ceil(height / rowHeight)`.
3. Add an **overscan** of a few rows above and below, so fast scrolling doesn't show blank gaps.
4. Render a container of `items.length × rowHeight` pixels (the spacer), and inside it only the rows in range, each absolutely positioned at `top = index × rowHeight`.
5. On every `scroll` event, store the new `scrollTop` in state → re-render → a different slice.

`code/virtual_list.jsx`:

```jsx
export function VirtualList({ items, rowHeight, height, overscan = 3, renderRow }) {
  const [scrollTop, setScrollTop] = useState(0);

  // 1. Which rows are visible? Pure arithmetic - every row has the same height.
  const first = Math.max(0, Math.floor(scrollTop / rowHeight) - overscan);
  const visibleCount = Math.ceil(height / rowHeight) + 2 * overscan;
  const last = Math.min(items.length - 1, first + visibleCount - 1);

  // 2. Render only those rows, absolutely positioned inside a spacer as tall as the WHOLE list,
  //    so the scrollbar looks and behaves as if all rows were there.
  const rows = [];
  for (let i = first; i <= last; i++) {
    rows.push(
      <div key={i} role="listitem" className="row"
           style={{ position: 'absolute', top: i * rowHeight, height: rowHeight, left: 0, right: 0 }}>
        {renderRow(items[i], i)}
      </div>
    );
  }
  return (
    <div role="list" aria-label="virtual list" className="viewport"
         style={{ height, overflowY: 'auto', position: 'relative' }}
         onScroll={e => setScrollTop(e.currentTarget.scrollTop)}>
      <div style={{ height: items.length * rowHeight, position: 'relative' }}>{rows}</div>
    </div>
  );
}
```

`code/07_virtualization.jsx` renders 10,000 customers both ways, then simulates scrolling (jsdom has no layout, so the test sets `scrollTop` itself and fires a `scroll` event):

```
render(<NaiveList items={ITEMS} />);                    // every row is a DOM node
// ...
render(<VirtualList items={ITEMS} rowHeight={30} height={300} overscan={3} renderRow={it => it.name} />);
for (const y of [3000, 150_000, 299_700]) {
  viewport.scrollTop = y;
  fireEvent.scroll(viewport);
  // print which rows are in the DOM
}
```

Output:

```
naive list    : 10000 row nodes, 10006 elements in document
virtual list  : 16 row nodes, 23 elements in document
  at top          -> rows Customer #0 .. Customer #15
  scrollTop=3000   -> rows Customer #97 .. Customer #112  (16 in DOM)
  scrollTop=150000 -> rows Customer #4997 .. Customer #5012  (16 in DOM)
  scrollTop=299700 -> rows Customer #9987 .. Customer #9999  (13 in DOM)

mount time (varies per run): naive 130 ms, virtual 6 ms
```

At `scrollTop = 3000` the first visible row is 3000 / 30 = 100; minus the overscan of 3 gives 97. The window is always 300 / 30 = 10 rows + 2 × 3 overscan = 16 rows, however long the list is. At the very end it shrinks to 13 because there are no rows after #9999.

In Chrome (`code/virtual.html`, scrolled half-way; rows 32 px, viewport 320 px):

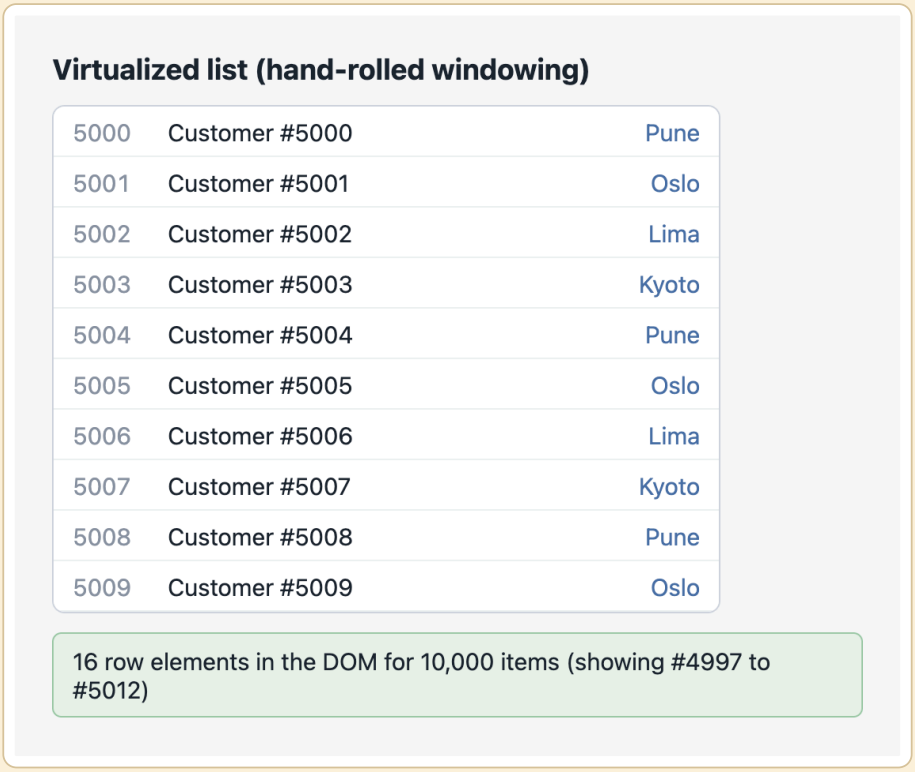


Figure 10.1 — The hand-rolled `VirtualList` in Chrome. Ten rows are visible; only sixteen exist in the DOM.

Use a library in real projects

Production lists need more than the hand-rolled version: **variable or unknown row heights** (measured after render), grids, sticky headers, stable scroll position when items are prepended (chat), infinite loading. Well-known libraries:

Library	Notes
TanStack Virtual	Headless (you write the markup); lists, grids, variable sizes
react-window	Small and simple; fixed and variable-size lists and grids
react-virtuoso	Automatic variable heights, grouped lists, chat-style reverse scrolling

A CSS-only alternative for moderately long pages is `content-visibility: auto`, which lets the browser skip layout and painting for off-screen sections while keeping them in the DOM (so browser find-in-page still works). It reduces rendering cost but not the number of React components.

Trade-offs

	Plain list	Virtualized list
DOM size	grows with data	constant (~visible rows)
Browser find (Ctrl+F)	finds any row	finds only rendered rows
Accessibility	screen readers see all rows	need `aria-setsize` / `aria-posinset` on rows to announce totals
Complexity	none	scroll math, measuring, focus management
Good for	< a few hundred rows	thousands of rows, feeds, logs, tables

When not to use it

- Short lists (under a few hundred simple rows). Pagination or "load more" is often a better user experience anyway.
- Content that must be printable or fully searchable on the page.

Common mistakes

- Using the array index as the row `key` when items can be inserted or removed — rows then show the wrong state. Use a stable ID (in the demo the list never changes, so index keys are safe).
- Forgetting the overscan, which leaves blank flashes during fast scrolling.
- Giving the viewport no fixed height — then it never scrolls, and everything "fits".

In the interview

30-second answer: "Virtualization renders only the visible rows plus some overscan inside a spacer the height of the full list, and recomputes the slice on scroll, so the DOM stays small for thousands of rows. Fixed heights need only `floor(scrollTop / rowHeight)`; variable heights need measuring, so in production I'd use TanStack Virtual or react-window. Downsides: browser find and screen-reader counts need extra care."

- *Why doesn't `memo` fix a 10,000-row list?* — The cost is the number of DOM nodes and the initial render, not repeated re-renders.
- *How would you do infinite scroll?* — Virtualize, and when the last rendered index nears the end, fetch the next page (an `IntersectionObserver` on a sentinel row, or the library's range callback).

TL;DR

Render only what's visible: `first = floor(scrollTop / rowHeight) − overscan`, a full-height spacer, absolutely positioned rows. 10,000 items → ~16 DOM rows. Use a library for real apps.

10.10 Common performance bugs

What it is

Four mistakes cause a large share of slow React screens. `code/08_perf_bugs.jsx` measures each one before and after the fix, counting **renders** (the function ran) and **mounts** (a brand-new instance: new DOM, state reset, effects re-run — much more expensive).

```
function Tracked({ label }) {                // counts renders AND mounts
  renders++;
  useEffect(() => { mounts++; }, []);
  return <span>{label}</span>;
}

// BUG 1: a component defined INSIDE another component
function Bad1() {
  const [n, setN] = useState(0);
  function Inner() { return <Tracked label="inner" />; }        // new type every render!
  return <><button onClick={() => setN(n + 1)}>bump</button><Inner /></>;
}

// BUG 2: unstable keys
<ul>{ITEMS.map(it => <li key={Math.random()}><Tracked label={it} /></li>)}</ul>

// BUG 3: a new context object every render
const Consumer = memo(function Consumer() { renders++; const { user } = useContext(Ctx); return <b>{user}</b>; });
function Provider({ stable }) {
  const [user] = useState('ada');
  const [, setTick] = useState(0);                                  // unrelated state
  const fresh = { user };                                           // new object each render
  const cached = useMemo(() => ({ user }), [user]);                 // same object until user changes
  return <Ctx value={stable ? cached : fresh}>
    <button onClick={() => setTick(t => t + 1)}>bump</button><Consumer /></Ctx>;
}

// BUG 4: state kept too high
function AppHigh() {
  const [q, setQ] = useState('');
  return <><input aria-label="q" value={q} onChange={e => setQ(e.target.value)} /><Expensive /><Expensive /></>;
}
function SearchBox() {                                              // state moved DOWN
  const [q, setQ] = useState('');
  return <input aria-label="q" value={q} onChange={e => setQ(e.target.value)} />;
}
const AppLow = () => <><SearchBox /><Expensive /><Expensive /></>;
```

Output:

```
1. component defined inside a component (3 parent re-renders)
  bad : Inner declared in render           renders=3, mounts=3
  good: Inner declared at top level        renders=3, mounts=0
2. unstable keys on a 3-item list (3 parent re-renders)
  bad : key={Math.random()}                renders=9, mounts=9
  good: key={item.id}                      renders=9, mounts=0
3. new context object each render (3 unrelated provider re-renders)
  bad : value={{ user }}                   renders=3, mounts=0
  good: value={useMemo(...)}               renders=0, mounts=0
4. input state kept too high (typing 5 characters)
  bad : query state in <App>               renders=10, mounts=0
  good: query state in <SearchBox>         renders=0, mounts=0
```

The four bugs explained

#	Bug	Why it's slow	Fix
1	Component declared inside another component	Each render creates a **new component type**; React unmounts the old subtree and mounts a new one — DOM rebuilt, state (e.g. text in an input) lost, effects re-run	Declare every component at module top level; pass data as props
2	`key={Math.random()}` or keys that change	React thinks every item is new → unmount + mount every render	Stable IDs from the data (Chapter 2)
3	`value={{ user, setUser }}` on a provider	New object each render → **every** consumer re-renders, `memo` can't stop it	`useMemo` the value, split contexts (state vs setters), or use a store with selectors
4	State owned higher than needed	Every keystroke re-renders the whole page under that owner	**Colocate** state: move it into the smallest component that uses it; or pass expensive parts as `children`

More perf traps worth knowing

- **Effects that set state on every render** ("render → effect → setState → render") — often a derived value that should be computed during render (Chapter 5, "You might not need an Effect").
- **Fetch waterfalls**: a parent fetches, then renders a child that fetches, and so on. Fetch in parallel, in route loaders, or with TanStack Query (Chapter 9).
- **Huge images and heavy libraries** (a whole date library for one format call) — use modern image formats and `Intl.DateTimeFormat`/tree-shakable imports.
- **Expensive work in event handlers** blocks the next paint — split it, use `startTransition`, or move it to a Web Worker.
- **Development build in production** — always deploy the production build (`vite build`, `next build`).

In the interview

> **30-second answer:** "Components defined inside components (remount every render), unstable keys, context values recreated each render, state kept too high, and effect-driven render loops. Fix the structure first — colocate state, pass JSX as children, stable keys, memoized or split context — then `memo` / `useMemo`."

Self-check

1. Why does declaring `Inner` inside `Bad1` reset its state? → Each render produces a different function, i.e. a different component type, so React mounts a fresh instance.
2. What is "colocating state"? → Keeping state in the lowest component that needs it, so updates re-render as little as possible.
3. A memoized consumer re-renders when an unrelated provider state changes. Why? → The provider's `value` is a new object each render; context changes bypass `memo`.

TL;DR

Top-level components, stable keys, memoized (or split) context values, colocated state. These structural fixes beat any amount of `memo`.

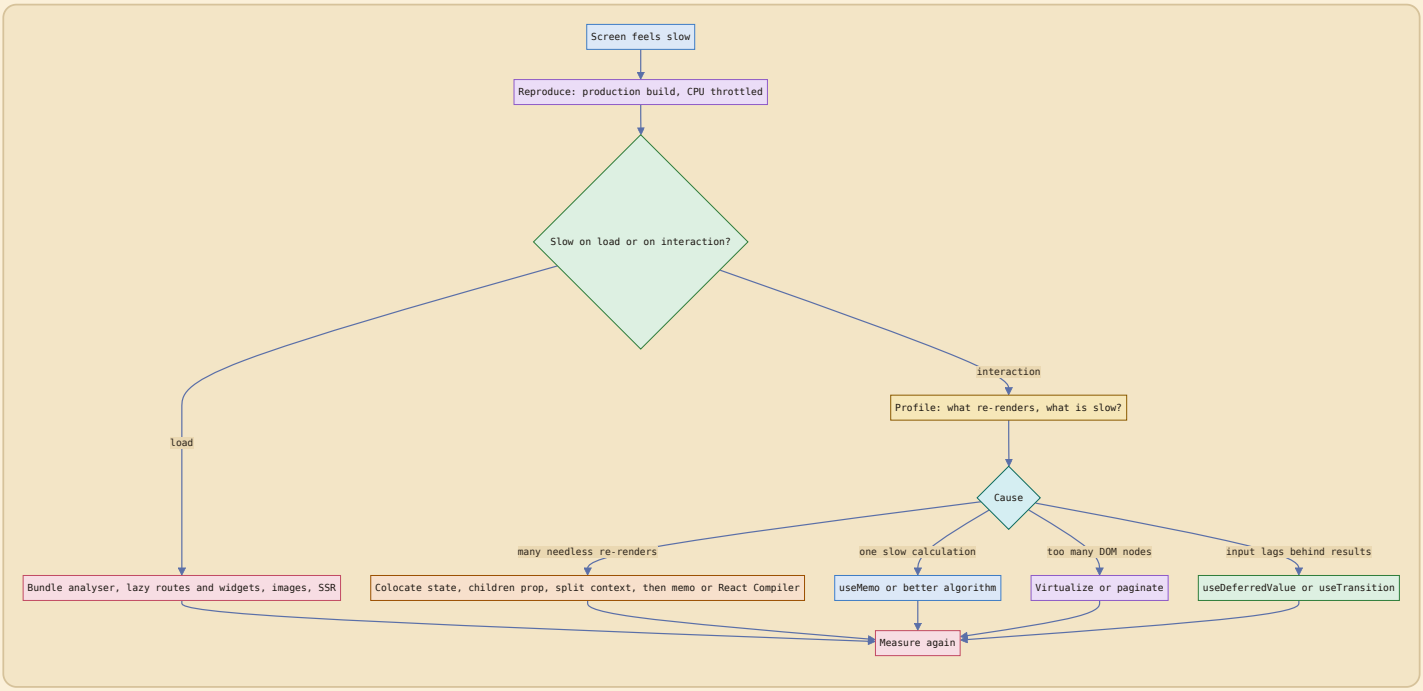
10.11 A performance checklist

When a screen feels slow, work through this in order and stop when it's fast enough:

1. **Reproduce with a production build**, CPU throttled, on realistic data.
2. **Find the slow interaction** (Chrome Performance panel with React tracks, or the DevTools Profiler). Is it the initial load, or a specific interaction?
3. **Slow initial load?** Check bundle size (analyser), split routes and heavy widgets with `lazy` (§10.8), optimise images, consider server rendering (Chapter 13).
4. **Slow interaction?** Turn on "highlight updates" and look at what re-renders.

- Too much re-renders → colocate state, pass `children`, split context (§10.10); then `memo` + stable props (§10.4–10.5), or enable React Compiler (§10.7).
- One component is slow on its own → `useMemo` the heavy calculation (§10.6), or improve the algorithm.
- Too many DOM nodes → virtualize (§10.9) or paginate.
- Typing feels laggy while results render → `useDeferredValue` / `useTransition` (Chapter 7).

5. **Measure again** and keep the change only if it made a measurable difference.



In the interview — putting it together

"How would you speed up a slow React page?" — "Measure first: production build, CPU throttling, the React Profiler to find the slowest commit and why things rendered. For load time, check bundle size and split routes and heavy widgets with `lazy` + `Suspense`. For interactions, reduce re-renders structurally (colocate state, pass children, stable context values), then `memo` with stable props or React Compiler. Virtualize long lists, memoize expensive calculations, mark non-urgent updates as transitions — and measure again."

Self-check (whole chapter)

1. What is the difference between render and commit? → Render calls components to compute the UI; commit applies the differences to the DOM.
2. Why did V2 (memo with inline props) render as much as V1? → New function and object props each render fail the shallow comparison.
3. What does `baseDuration` tell you? → An estimate of how long a full re-render of the subtree would take — compare with `actualDuration`.
4. How many DOM rows does a 300 px viewport with 30 px rows and overscan 3 render? → 10 + 6 = 16.

Chapter 10 TL;DR

Idea	Remember
Render cycle	Trigger → render (call components) → commit (DOM diff) → effects; a re-render is not a DOM update
Re-render causes	Own state, parent render, context, external store — props aren't compared without `memo`
Measure	DevTools highlight + Profiler ("why did this render?"), Performance tracks (19.2+), `<Profiler onRender>`; production profiling needs the profiling build
`memo`	Skip re-render on shallow-equal props; useless with new objects/functions as props
`useCallback` / `useMemo`	Stable references for `memo` children and effect deps; cache calculations ≥ ~1 ms
React Compiler	v1.0 (Oct 2025); build-time auto-memoization; follow the Rules of React; `"use no memo"` escape hatch
Code splitting	`lazy(() => import())` at module level + `<Suspense>` + error boundary; split by route/heavy widget; preload on intent
Virtualization	Render only visible rows + overscan in a full-height spacer; use TanStack Virtual / react-window in production
Perf bugs	Nested component definitions, unstable keys, new context objects, state too high, effect loops
Order of fixes	Structure first (colocate, children, keys) → memoization → specialised tools

Next: Chapter 11 — TypeScript with React.

Chapter 11 — TypeScript with React

TypeScript is JavaScript plus a type system that is checked **before** your code runs. In a React project it turns a whole class of bugs — a missing prop, a misspelled variant, reading `user.name` while `user` can still be `null`, forgetting to handle one action in a reducer — into red squiggles in your editor instead of broken screens in production. It also makes components self-documenting: the props type *is* the component's API.

This chapter assumes the TypeScript basics from the JavaScript Handbook (Chapter 16 — types, narrowing, generics) and shows how to apply them to React: props, children, events, state, reducers, refs, Hooks, context, generic components and discriminated-union props. For the "wrong" examples you will see the **real compiler errors**.

What you'll learn

#	Topic	You will be able to…
11.1	Setup and how type-checking fits in	create a React + TS project and run `tsc --noEmit`
11.2	Typing props	write props types with required, optional, union and callback props
11.3	Typing children and extending native elements	choose between `ReactNode`, `ReactElement`, `PropsWithChildren`, `ComponentProps<'button'>`
11.4	Typing events	type handlers with `ChangeEvent`, `SubmitEvent`, `MouseEvent`, `KeyboardEvent`
11.5	Typing state and reducers	type `useState` (including `null` and `[]`) and `useReducer` with action unions
11.6	Typing refs	type DOM refs, value refs, `ref` as a prop (React 19) and `useImperativeHandle`
11.7	Typing custom Hooks and Context	return tuples `as const`, write generic Hooks, make context non-nullable
11.8	Generic components	build `List<T>`, `Select<V>` and `Table<Row>` that infer their types
11.9	Discriminated unions for props	make impossible prop combinations impossible to write
11.10	Reading type errors, and common mistakes	decode the 12 errors beginners hit most

Code: `code/`. Each `.tsx` file **runs** (bundled with esbuild, React 19.3 in jsdom) **and** is type-checked with the TypeScript compiler (`tsc` 7.0.2, `@types/react` 19.3, `strict` mode). The deliberately broken examples live in `code/bad_types.tsx`, which is only type-checked.

11.1 Setup and how type-checking fits in

What it is

A React + TypeScript project has three pieces:

Piece	Role
`.tsx` files	TypeScript files that may contain JSX (`.ts` for files without JSX)
`@types/react`, `@types/react-dom`	the type definitions for React (React itself is written in JavaScript; these packages describe its API)
`tsconfig.json`	compiler options: strictness, JSX mode, which files to check

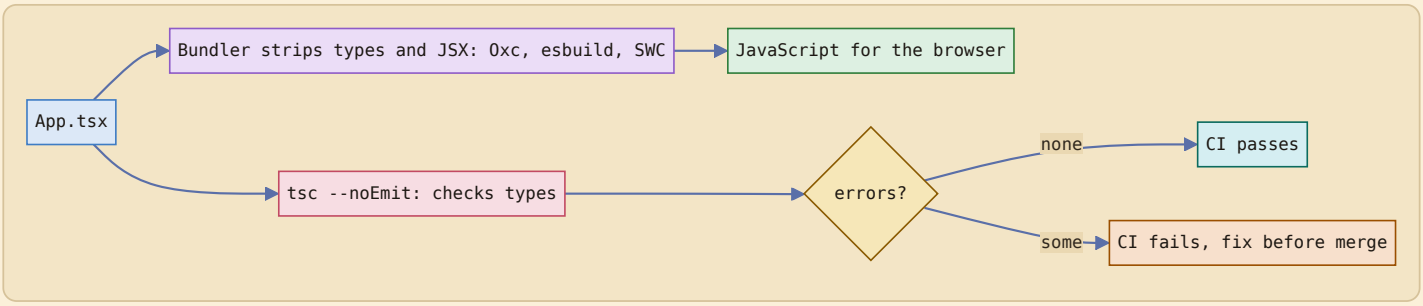
To start a new project: `npm create vite@latest my-app -- --template react-ts` (Vite), or a framework such as Next.js, which asks whether you want TypeScript and configures it (Chapter 1).

How it works — two separate jobs

1. **Removing types.** Browsers can't run TypeScript, so the bundler (Vite 8 uses Oxc, earlier Vite versions esbuild; Next.js uses SWC) strips the types and turns JSX into function calls. This is fast because it **does not check** anything.

2. **Checking types.** The TypeScript compiler, `tsc`, reads all files and reports errors. With `--noEmit` it only checks and writes no files. Your editor runs the same checker continuously.

Because the bundler does not check types, **a project with type errors can still build and run**. Add `tsc --noEmit` to your CI (and often to `npm run build`, e.g. `"build": "tsc --noEmit && vite build"`) so errors block merges.



The tsconfig options that matter for React

`code/tsconfig.check.json` (this book keeps npm packages in a shared folder, hence the `paths` line; a normal project doesn't need it and names the file `tsconfig.json`):

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",     // resolve imports the way Vite/esbuild do
    "jsx": "react-jsx",                // the modern JSX transform: no `import React` needed
    "strict": true,                    // turns on strictNullChecks, noImplicitAny, ... - always use it
    "noEmit": true,                    // the bundler produces JS; tsc only checks
    "skipLibCheck": true,              // don't re-check .d.ts files in node_modules (faster)
    "allowImportingTsExtensions": true,
    "lib": ["ES2023", "DOM", "DOM.Iterable"],   // DOM types such as HTMLInputElement
    "types": [],
    "paths": { "*": ["../../../../.web-books/node_modules/@types/*", "../../../../.web-books/node_modules/*"] }
  },
  "include": ["*.tsx", "*.ts"],
  "exclude": ["bad_*.tsx"]
}
```

Checking every example in this chapter:

```
$ tsc --noEmit -p tsconfig.check.json && echo "0 type errors"
0 type errors
```

Common mistakes

- Turning `strict` off to "get it compiling" — you lose most of the value, especially null checks.
- Using `any` to silence an error. `any` switches checking off for that value *and everything it flows into*. Prefer `unknown` and narrow it, or fix the type.
- Forgetting `@types/react` / `@types/react-dom` (or having versions that don't match your React major version).

TL;DR

Bundlers strip types; `tsc --noEmit` checks them. Use `strict: true`, `jsx: "react-jsx"`, and run the checker in CI.

11.2 Typing props

What it is

A component's props are one object, so you describe that object with a `type` (or `interface`) and annotate the destructured parameter.

`code/01_props_and_children.tsx` (part 1):

```tsx
type BadgeProps = {
  label: string;                        // required
  count?: number;                       // optional -> number | undefined inside the component
  tone?: 'info' | 'warning' | 'error';  // only these three strings are allowed
};
function Badge({ label, count = 0, tone = 'info' }: BadgeProps) {
  return <span className={`badge ${tone}`}>{label}: {count}</span>;
}

type AvatarProps = {
  user: { name: string; imageUrl: string };  // an object shape
  size?: number;
  style?: CSSProperties;                     // a style object, checked key by key
  onSelect?: (name: string) => void;         // a callback prop
};
function Avatar({ user, size = 32, style, onSelect }: AvatarProps) {
  return <img src={user.imageUrl} alt={user.name} width={size} style={style} onClick={() => onSelect?.(user.name)} />;
}
```

How it works

1. At each `<Badge ...>` usage, TypeScript checks the JSX attributes against `BadgeProps`, exactly like checking an object literal.
2. A `?` makes the prop optional; inside the component its type includes `undefined` until you give a **default value** in the destructuring (`count = 0` makes `count` a plain `number`).
3. A **union of string literals** (`'info' | 'warning' | 'error'`) is the TypeScript way to say "one of these values" — editors autocomplete them, and typos are errors.
4. A callback prop is a **function type**: `(name: string) => void` says "called with a string, return value ignored". Call optional callbacks with `onSelect?.(...)`.

`type` or `interface`?

Both work for props. `interface` can be extended with `extends` and merged by re-declaring; `type` can express unions (`A | B`), which props often need (§11.9). Many teams simply use `type` everywhere for consistency; pick one convention per codebase.

What about `React.FC`?

`const Badge: React.FC<BadgeProps> = (props) => ...` also works. Since React 18's types, `FC` no longer adds an implicit `children` prop, so the old main objection is gone; still, annotating the props parameter directly (as above) is simpler, works with generics (§11.8) and is what most modern code does.

Real-world examples

Design-system components (buttons, badges, alerts) use literal unions for `variant`/`size`/`tone` props, so the design tokens are enforced by the compiler.

TL;DR

`function C({ a, b = 1 }: Props)`. `?` = optional, defaults remove `undefined`, literal unions restrict values, function types describe callbacks.

11.3 Typing children and extending native elements

Which type for `children`?

Type	Accepts	Use for
`ReactNode`	anything React can render: JSX, strings, numbers, `null`, `undefined`, booleans, arrays of these	**`children` and "slot" props** — the default choice
`ReactElement`	only a JSX element (`<Foo />`), not strings or `null`	a prop that must be one element (rare)
`JSX.Element`	the type of a JSX expression (basically `ReactElement<any, any>`)	the return type of components, if you annotate it
`PropsWithChildren<P>`	adds `children?: ReactNode` to `P`	a short way to accept optional children

`code/01_props_and_children.tsx` (part 2):

```
type CardProps = { title: string; children: ReactNode; footer?: ReactNode };
function Card({ title, children, footer }: CardProps) {
  return <section><h2>{title}</h2>{children}{footer && <footer>{footer}</footer>}</section>;
}
function Note({ children }: PropsWithChildren) { return <aside>{children}</aside>; }
```

Extending a native element's props

A reusable `<Button>` should accept **everything** a `<button>` accepts — `type`, `disabled`, `aria-*`, `onClick` — without you listing them. `ComponentProps<'button'>` is the full props type of the built-in element:

```
type ButtonProps = ComponentProps<'button'> & { variant?: 'primary' | 'ghost' };
function Button({ variant = 'primary', className, ...rest }: ButtonProps) {
  return <button className={[variant, className].filter(Boolean).join(' ')} {...rest} />;
}
```

Usage and result:

```
<Card title="Inbox" footer={<small>updated now</small>}>
  <Badge label="unread" count={3} tone="warning" />
  <Note>plain text child</Note>
  <Button type="submit" disabled aria-label="send">Send</Button>
  <Avatar user={{ name: 'Ada', imageUrl: '/ada.png' }} style={{ borderRadius: '50%' }} />
</Card>
```

Output:

```
<section>
<h2>Inbox</h2>
<span class="badge warning">unread: 3</span>
<aside>plain text child</aside>
<button class="primary" type="submit" disabled="" aria-label="send">Send</button>
<img alt="Ada" width="32" style="border-radius: 50%;" src="/ada.png">
<footer>
<small>updated now</small>
</footer>
</section>
```

Related helpers: `ComponentProps<typeof MyComponent>` extracts the props type of **your** component (handy when wrapping a library component whose props type isn't exported); `ComponentPropsWithoutRef<'input'>` omits `ref`; `Omit<ComponentProps<'a'>, 'href'>` removes one prop so you can redefine it.

Common mistakes

- Typing `children` as `JSX.Element` — then `<Card>hello</Card>` (a string) or two children fail. Use `ReactNode`.

- Spreading `{...rest}` onto the element *before* your own props, so the caller's `className` overwrites yours. Merge deliberately, as above.

> **TL;DR**

`children: ReactNode`; extend native props with `ComponentProps<'button'> & {...}` and spread the rest.

11.4 Typing events

What it is

React passes your handlers its own event objects (synthetic events, Chapter 3). Their types are generic over the **element** the handler is attached to, which is what makes `e.target.value` or `e.currentTarget.checked` type-safe.

Event	Type	Typical element	Common props
`onChange` on input / select / textarea	`ChangeEvent<HTMLInputElement>` (etc.)	`HTMLInputElement`, `HTMLSelectElement`, `HTMLTextAreaElement`	`e.target.value`, `e.target.checked`
`onSubmit`	`SubmitEvent<HTMLFormElement>` (older code: `FormEvent`)	`HTMLFormElement`	`e.preventDefault()`, `e.submitter`
`onClick`	`MouseEvent<HTMLButtonElement>`	any element	`e.currentTarget`, `e.shiftKey`, `e.clientX`
`onKeyDown`	`KeyboardEvent<HTMLInputElement>`	any focusable element	`e.key`, `e.ctrlKey`
`onFocus` / `onBlur`	`FocusEvent<HTMLInputElement>`		`e.relatedTarget`

Import them from `react`, **not** from the DOM (`MouseEvent` or `SubmitEvent` without an import refers to the browser's native event type, which is different).

Version note: recent `@types/react` 19 releases (used here: 19.3) type `onSubmit` with `SubmitEvent` and mark `FormEvent` as **deprecated** (React never had a real "form event"). `FormEvent<HTMLFormElement>` still compiles, so you'll see it in most existing code and tutorials; prefer `SubmitEvent` in new code if your `@types/react` has it.

How it works

1. **Inline handlers need no annotation**: in `onChange={e => setEmail(e.target.value)}`, TypeScript already knows the `onChange` prop's type, so `e` is inferred (this is called *contextual typing*).
2. **Separately declared handlers need a type**, because TypeScript can't see where they'll be used. Either annotate the parameter (`e: ChangeEvent<HTMLInputElement>`) or the whole function (`const h: ChangeEventHandler<HTMLSelectElement> = e => ...`).
3. Tip: not sure of the type? Write the handler inline first and hover `e` in your editor; it shows the exact type to copy.

`code/02_events.tsx`:

```tsx
function SignupForm({ onSubmit }: { onSubmit: (data: { email: string; plan: string }) => void }) {
  const [email, setEmail] = useState('');
  const [plan, setPlan] = useState('free');

  function handleEmail(e: ChangeEvent<HTMLInputElement>) {
    setEmail(e.target.value);                       // e.target is an HTMLInputElement -> .value is a string
  }
  const handlePlan: ChangeEventHandler<HTMLSelectElement> = e => setPlan(e.target.value); // type the whole function
  function handleSubmit(e: SubmitEvent<HTMLFormElement>) {   // older code: FormEvent (now deprecated)
    e.preventDefault();
    console.log('submitted by the button:', e.submitter?.textContent);
    onSubmit({ email, plan });
  }
  function handleKey(e: KeyboardEvent<HTMLInputElement>) {
    if (e.key === 'Escape') setEmail('');
  }
  function handleHelp(e: MouseEvent<HTMLButtonElement>) {
    console.log(`help clicked at button "${e.currentTarget.textContent}" (shift: ${e.shiftKey})`);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input aria-label="email" value={email} onChange={handleEmail} onKeyDown={handleKey} />
      <select aria-label="plan" value={plan} onChange={handlePlan}>
        <option value="free">Free</option><option value="pro">Pro</option>
      </select>
      <button type="button" onClick={handleHelp}>Help</button>
      <button type="submit" onClick={e => console.log('inline handler, e is', e.type)}>Sign up</button>
    </form>
  );
}
```

Output (after typing an email, choosing Pro, shift-clicking Help, submitting, then pressing Escape):

```
help clicked at button "Help" (shift: true)
inline handler, e is click
submitted by the button: Sign up
submitted {"email":"ada@example.com","plan":"pro"}
after Escape, email = ""
```

`target` vs `currentTarget`

`e.currentTarget` is the element the handler is attached to, and its type is the element type you gave (`HTMLButtonElement`). `e.target` is whatever element was actually clicked (maybe an `<svg>` inside the button), so for most events it is typed only as `EventTarget`. For **`change`** events React types `e.target` as the input itself, which is why `e.target.value` works there.

Form actions (React 19)

With form actions (Chapter 4) you don't handle an event at all: `<form action={fn}>` calls `fn(formData: FormData)`. `formData.get('email')` returns `FormDataEntryValue | null` (a `string` or a `File`), so narrow it: `const email = String(formData.get('email') ?? '')`, or validate the whole object with a schema library such as Zod.

> **TL;DR**
>
> Inline handlers are inferred; standalone ones use `XxxEvent<HTMLElementType>` or `XxxEventHandler<...>` from `react`. `currentTarget` has your element's type.

11.5 Typing state and reducers

`useState`

TypeScript **infers** the state type from the initial value, so most calls need nothing extra. Add a type argument when the initial value doesn't show the full picture:

Code	Inferred / declared type	Why
`useState(0)`	`number`	inferred
`useState<User | null>(null)`	`User | null`	`null` alone would infer type `null` forever
`useState<string[]>([])`	`string[]`	`[]` alone infers `never[]` — you could never add anything
`useState<'idle' | 'saving' | 'saved'>('idle')`	the union	`'idle'` alone infers `string`
`useState(() => loadDraft())`	return type of `loadDraft`	lazy initializer is inferred too

`code/03_state_and_reducer.tsx` (part 1):

```tsx
type User = { id: number; name: string };

function Profile() {
  const [count, setCount] = useState(0);                    // inferred: number
  const [user, setUser] = useState<User | null>(null);      // explicit: starts null, later a User
  const [tags, setTags] = useState<string[]>([]);           // [] alone would be never[]
  const [status, setStatus] = useState<'idle' | 'saving' | 'saved'>('idle');  // a union of literals

  return (
    <div>
      <p>{user ? `${user.name} (#${user.id})` : 'signed out'} | {count} | {tags.join(',')} | {status}</p>
      {/* user.name without the check would not compile: 'user' is possibly 'null' */}
      <button onClick={() => { setUser({ id: 7, name: 'Ada' }); setCount(c => c + 1); setTags(t => [...t, 'admin']);
setStatus('saved'); }}>
        sign in
      </button>
    </div>
  );
}
```

The setter's type is `Dispatch<SetStateAction<T>>`: it accepts either a `T` or an updater `(prev: T) => T`. You need that name when passing a setter as a prop: `setUser: Dispatch<SetStateAction<User | null>>` (or simply `(u: User | null) => void` if the child never uses the updater form).

`useReducer` with a discriminated union of actions

A **discriminated union** is a union of object types that share one literal-typed field (the *discriminant*, here `type`). Checking that field **narrows** the object to one member, so TypeScript knows which other fields exist.

`code/03_state_and_reducer.tsx` (part 2):

```ts
type CartItem = { sku: string; qty: number };
type CartState = { items: CartItem[]; coupon: string | null };
type CartAction =
  | { type: 'add'; sku: string }
  | { type: 'remove'; sku: string }
  | { type: 'applyCoupon'; code: string }
  | { type: 'clear' };

function cartReducer(state: CartState, action: CartAction): CartState {
  switch (action.type) {
    case 'add': {
      const found = state.items.find(i => i.sku === action.sku);   // action.sku is known to exist here
      return {
        ...state,
        items: found
          ? state.items.map(i => (i.sku === action.sku ? { ...i, qty: i.qty + 1 } : i))
          : [...state.items, { sku: action.sku, qty: 1 }],
      };
    }
    case 'remove':      return { ...state, items: state.items.filter(i => i.sku !== action.sku) };
    case 'applyCoupon': return { ...state, coupon: action.code };  // action.code exists only here
    case 'clear':       return { items: [], coupon: null };
    default: {
      const unreachable: never = action;                           // compile error if a case is missing
      return unreachable;
    }
  }
}

function Cart() {
  const [cart, dispatch] = useReducer(cartReducer, { items: [], coupon: null });
  // dispatch accepts only CartAction objects: dispatch({ type: 'add', sku: 'MUG' })
  // ...
}
```

Output (sign in, add mug twice, apply coupon, then clear):

```
initial        signed out | 0 |  | idle  ||  empty coupon=none
after clicks   Ada (#7) | 1 | admin | saved  ||  MUGx2 coupon=SAVE10
after clear    Ada (#7) | 1 | admin | saved  ||  empty coupon=none
```

How the `never` check works

In the `default` branch, TypeScript has removed every handled member from `action`'s type. If all cases are handled, nothing is left — the type is `never`, and assigning it to `never` is fine. Add a new action type to `CartAction` and forget its `case`, and `action` in `default` still has that type, so the assignment **fails to compile** (you'll see this error in §11.10). This is called an **exhaustiveness check**.

`useReducer` infers the state type from the reducer, and `dispatch` accepts only valid actions. (React 19's types also let you write `useReducer<CartState, [CartAction]>(...)`, but annotating the reducer function is simpler and usually enough.)

> **TL;DR**

Let `useState` infer; add `<T>` for `null`, `[]` and literal unions. Type reducer actions as a discriminated union and add a `never` check so missing cases are compile errors.

11.6 Typing refs

DOM refs and value refs

Purpose	Declaration	`.current` type
Point at a DOM element	`useRef<HTMLInputElement>(null)`	`HTMLInputElement | null`
Hold a mutable value (timer ID, previous value)	`useRef<number | null>(null)`, `useRef(0)`	what you declared

In React 19's types, `useRef` **always requires an argument** and always returns a `RefObject<T>` whose `current` you can assign (older versions distinguished a read-only `RefObject` from a `MutableRefObject`; `MutableRefObject` is now deprecated). For DOM refs pass `null`, because the element doesn't exist until React attaches it after rendering — which is why you use `ref.current?.focus()`.

`ref` as a prop (React 19)

Since React 19, function components receive `ref` as an **ordinary prop**; `forwardRef` is no longer needed for new code (Chapter 5). `ComponentProps<'input'>` already includes `ref?: Ref<HTMLInputElement>`, so a wrapper that spreads its props forwards the ref automatically. For a custom handle, type the prop as `Ref<YourHandle>` and fill it with `useImperativeHandle`.

`code/04_refs.tsx`:

```tsx
// 1. A DOM ref
function SearchBox() {
  const inputRef = useRef<HTMLInputElement>(null);
  useEffect(() => {
    inputRef.current?.focus();                  // ?. because current is null until React attaches it
  }, []);
  return <input ref={inputRef} aria-label="search" />;
}

// 2. A value ref (not attached to JSX)
function Stopwatch() {
  const intervalId = useRef<ReturnType<typeof setInterval> | null>(null);  // read/written only in handlers
  const [running, setRunning] = useState(false);
  function toggle() {
    if (intervalId.current !== null) { clearInterval(intervalId.current); intervalId.current = null; setRunning(false);
}
    else { intervalId.current = setInterval(() => {}, 1000); setRunning(true); }
  }
  return <button onClick={toggle}>{running ? 'stop' : 'start'} timer</button>;
}

// 3. ref as an ordinary prop - ComponentProps<'input'> includes `ref`
function TextField({ label, ...inputProps }: { label: string } & ComponentProps<'input'>) {
  return <label>{label} <input {...inputProps} /></label>;   // ref travels inside inputProps
}

// 4. Exposing a custom handle instead of the DOM node
export type VideoHandle = { play: () => void; pause: () => void };
function Video({ ref, src }: { ref?: Ref<VideoHandle>; src: string }) {
  const [state, setState] = useState('paused');
  useImperativeHandle(ref, () => ({ play: () => setState('playing'), pause: () => setState('paused') }), []);
  return <p>video {src}: {state}</p>;
}

function App() {
  const nameRef = useRef<HTMLInputElement>(null);
  const videoRef = useRef<VideoHandle>(null);
  // ... <TextField label="Name" ref={nameRef} placeholder="Ada" />
  //     <Video ref={videoRef} src="intro.mp4" />
  //     <button onClick={() => { ...; videoRef.current?.play(); }}>go</button>
}
```

Output:

```
focused after mount: search
stopwatch: stop timer
nameRef.current is INPUT with placeholder Ada
video intro.mp4: playing
```

`ReturnType<typeof setInterval>` is a trick worth knowing: the timer ID is a `number` in browsers but an object in Node, and this expression gives whatever the current environment's `setInterval` returns.

Common mistakes

- `useRef<HTMLInputElement>()` with no argument — an error with React 19 types (see §11.10).
- Declaring the wrong element type: `useRef<HTMLDivElement>(null)` on an `<input>` is a type error at `ref={...}`.
- Reading or writing `ref.current` **during render**. Refs are for event handlers and Effects; React Compiler's lint rules flag render-time ref access (Chapter 10).

TL;DR

`useRef<HTMLXxxElement>(null)` for DOM, `useRef<T>(initial)` for values. In React 19 `ref` is a normal prop typed `Ref<T>`; use `useImperativeHandle` to expose a custom handle.

11.7 Typing custom Hooks and Context

Returning a tuple: `as const`

A Hook that returns `[on, toggle]` would be inferred as `(boolean | (() => void))[]` — an array where each element could be either, so `on` isn't known to be a boolean. `as const` makes it a **readonly tuple** `readonly [boolean, () => void]`:

```tsx
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn(v => !v), []);
  return [on, toggle] as const;
}
```

(Returning an **object** — `{ on, toggle }` — avoids the problem entirely and is easier to extend; prefer objects when a Hook returns more than two things.)

A generic Hook with a discriminated-union result

`code/05_hooks_and_context.tsx`:

```tsx
type FetchState<T> =
  | { status: 'loading' }
  | { status: 'error'; error: string }
  | { status: 'success'; data: T };

function useJson<T>(url: string): FetchState<T> {
  const [state, setState] = useState<FetchState<T>>({ status: 'loading' });
  useEffect(() => {
    let ignore = false;
    setState({ status: 'loading' });
    fetch(url)
      .then(r => (r.ok ? (r.json() as Promise<T>) : Promise.reject(new Error(`HTTP ${r.status}`))))
      .then(data => { if (!ignore) setState({ status: 'success', data }); })
      .catch((e: unknown) => { if (!ignore) setState({ status: 'error', error: e instanceof Error ? e.message :
String(e) }); });
    return () => { ignore = true; };
  }, [url]);
  return state;
}
```

The caller picks `T` (`useJson<Todo>('/api/todos/1')`). Because the result is a union, the component *must* check `status` before touching `data`:

```tsx
function TodoTitle() {
  const result = useJson<Todo>('/api/todos/1');
  const [details, toggle] = useToggle();
  const { user, logout } = useAuth();
  if (result.status === 'loading') return <p>loading...</p>;
  if (result.status === 'error') return <p>error: {result.error}</p>;
  // Here TypeScript knows result.data is a Todo.
  return (/* ... {user}: {result.data.title}{details && ` (#${result.data.id})`} ... */);
}
```

This is better than three separate fields (`data?`, `error?`, `loading`), which allow nonsense states such as "loading **and** has an error". Note `catch (e: unknown)`: in strict mode caught values are `unknown`, and you narrow with `instanceof Error`.

`as Promise<T>` is not validation. It tells the compiler "trust me" — if the API returns a different shape, TypeScript cannot know. At trust boundaries (network, `localStorage`, URL params), validate with a schema library such as Zod, which both checks the data at runtime and produces the TypeScript type. Libraries like TanStack Query are generic in the same way (`useQuery<Todo>`), see Chapter 9.

Context that is never `null` for consumers

`createContext` needs a default value. When there's no sensible default, use `null` and a custom Hook that throws if the provider is missing — callers then get the non-nullable type:

```tsx
type Auth = { user: string; logout: () => void };
const AuthContext = createContext<Auth | null>(null);

function useAuth(): Auth {                        // callers get Auth, never null
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}
function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState('ada');
  // (in a real app, memoize this object - see Chapter 10, section 10.10)
  return <AuthContext value={{ user, logout: () => setUser('guest') }}>{children}</AuthContext>;
}
```

Output (fetch is replaced by a local fake so the example runs offline; the second part renders `TodoTitle` without a provider):

```
loading...
guest: loaded from /api/todos/1 (#1)
without provider -> useAuth must be used inside <AuthProvider>
```

The alternative, `createContext<Auth>({} as Auth)`, compiles but lies: a component outside the provider would crash later with a confusing `undefined is not a function`.

TL;DR

Tuples need `as const` (or return an object). Generic Hooks take `<T>`; return discriminated unions for async state. Context: `createContext<T | null>(null)` + a `useX()` that throws.

11.8 Generic components

What it is

A **generic component** has a type parameter, like a generic function: `function List<T>(props: ListProps<T>)`. TypeScript **infers** `T` from the props you pass, then checks the other props against it. One `List` works for books, users or strings, and `renderItem` always receives the right item type.

`code/06_generic_components.tsx`:

```tsx
type ListProps<T> = {
  items: T[];
  getKey: (item: T) => Key;
  renderItem: (item: T) => ReactNode;
  emptyText?: string;
};
function List<T>({ items, getKey, renderItem, emptyText = 'Nothing here' }: ListProps<T>) {
  if (items.length === 0) return <p>{emptyText}</p>;
  return <ul>{items.map(item => <li key={getKey(item)}>{renderItem(item)}</li>)}</ul>;
}
// In an arrow function inside a .tsx file write <T,> so the parser doesn't read <T> as JSX:
const Count = <T,>({ items }: { items: T[] }) => <small>{items.length} items</small>;

// A select whose value type is a union of string literals, inferred from `options`
type SelectProps<V extends string> = {
  label: string;
  value: V;
  options: readonly { value: V; label: string }[];
  onChange: (value: V) => void;
};
function Select<V extends string>({ label, value, options, onChange }: SelectProps<V>) {
  return (
    <label>{label}{' '}
      <select value={value} onChange={e => onChange(e.target.value as V)}>
        {options.map(o => <option key={o.value} value={o.value}>{o.label}</option>)}
      </select>
    </label>
  );
}

// A table whose columns may only name real keys of the row type
type Column<Row> = { key: keyof Row & string; header: string };
function Table<Row extends { id: number }>({ rows, columns }: { rows: Row[]; columns: Column<Row>[] }) {
  // ... <th>{c.header}</th> ... <td>{String(r[c.key])}</td> ...
}

type Book = { id: number; title: string; year: number };
const SORTS = [{ value: 'title', label: 'Title' }, { value: 'year', label: 'Year' }] as const;

function App() {
  const [sort, setSort] = useState<'title' | 'year'>('title');
  const sorted = [...books].sort((a, b) => (sort === 'year' ? a.year - b.year : a.title.localeCompare(b.title)));
  return (<>
      <Select label="Sort by" value={sort} options={SORTS} onChange={setSort} />
      <List items={sorted} getKey={b => b.id} renderItem={b => `${b.title} (${b.year})`} />
      <List items={[] as string[]} getKey={s => s} renderItem={s => s} emptyText="No tags" />
      <Count items={books} />
      <Table rows={books} columns={[{ key: 'title', header: 'Title' }, { key: 'year', header: 'Year' }]} />
    </>);
}
```

Output (then changing the select to "year"):

```
list : Beloved (1987) | Dune (1965) | Neuromancer (1984)
year : Dune (1965) | Neuromancer (1984) | Beloved (1987)
other: No tags / 3 items
table: TitleYear | Dune1965 | Neuromancer1984 | Beloved1987
```

How it works

1. In `<List items={sorted} ...>`, TypeScript sees `items: Book[]`, so `T = Book`.
2. `renderItem={b => ...}` is then checked as `(item: Book) => ReactNode` — `b.title` autocompletes, `b.author` would be an error.
3. **Constraints** (`V extends string`, `Row extends { id: number }`) restrict what `T` may be, and let the component use those guarantees (`r.id` as a key).
4. `keyof Row & string` is the union of `Row`'s property names (`'id' | 'title' | 'year'`), so a column for a missing field is a compile error (§11.10, error 8).

5. `as const` on `SORTS` keeps the literal values `'title'` and `'year'`, so `Select` infers `V = 'title' | 'year'` and `onChange={setSort}` type-checks.

When not to use generics

If a component only ever handles one type, a plain props type is clearer. Generics pay off in reusable building blocks: lists, tables, selects, comboboxes, form fields.

TL;DR

`function List<T>(props: ListProps<T>)` — T is inferred from props and flows into callbacks. Constrain with `extends`, use `keyof` for field names, `<T,>` for arrow functions in `.tsx`.

11.9 Discriminated unions for props

The problem it solves

Props like `{ status: string; error?: string; count?: number; onRetry?: () => void }` allow nonsense: a "success" message with an error, an "error" without a retry handler. Every consumer must guess which combinations are valid, and the component needs defensive checks. A **union of props types**, one per case, makes the invalid combinations **impossible to write**.

`code/07_union_props.tsx`:

```tsx
// 1. Each `status` value comes with exactly the props it needs
type StatusProps =
  | { status: 'loading' }
  | { status: 'error'; error: string; onRetry: () => void }
  | { status: 'success'; count: number };

function Status(props: StatusProps) {
  switch (props.status) {                    // narrow on the discriminant `status`
    case 'loading': return <p>Loading…</p>;
    case 'error':   return <p role="alert">{props.error} <button onClick={props.onRetry}>Retry</button></p>;
    case 'success': return <p>{props.count} results</p>;
  }
}

// 2. Either a link or a button
type LinkLike = { as: 'a'; href: string } & Omit<ComponentProps<'a'>, 'href'>;
type ButtonLike = { as?: 'button' } & ComponentProps<'button'>;
type ActionProps = (LinkLike | ButtonLike) & { children: ReactNode };

function Action(props: ActionProps) {
  if (props.as === 'a') {
    const { as: _as, ...rest } = props;       // here props is LinkLike
    return <a className="action" {...rest} />;
  }
  const { as: _as, ...rest } = props;         // here props is ButtonLike
  return <button className="action" type="button" {...rest} />;
}

// 3. "One or the other, not both": the `never` trick forbids the other prop
type ImageSource =
  | { src: string; icon?: never }
  | { icon: 'user' | 'star'; src?: never };
function Thumb(props: ImageSource & { alt: string }) {
  return props.src ? <img src={props.src} alt={props.alt} /> : <span aria-label={props.alt}>[{props.icon}]</span>;
}
```

Usage and result:

```
<Status status="loading" />
<Status status="error" error="Network down" onRetry={() => {}} />
<Status status="success" count={42} />
<Action as="a" href="/docs">Docs</Action>
<Action onClick={() => {}}>Save</Action>
<Thumb src="/a.png" alt="avatar" />
<Thumb icon="star" alt="favourite" />
```

Output:

```
<p>Loading…</p>
<p role="alert">Network down <button>Retry</button>
</p>
<p>42 results</p>
<a class="action" href="/docs">Docs</a>
<button class="action" type="button">Save</button>
<img alt="avatar" src="/a.png">
<span aria-label="favourite">[star]</span>
```

How it works

1. The shared literal field (`status`, `as`) is the **discriminant**.
2. At the call site, TypeScript picks the union member whose discriminant matches and checks the remaining props against it: `<Status status="success" />` without `count` fails (§11.10, error 6).
3. Inside the component, checking the discriminant (`switch`, `if`) narrows `props`, so `props.error` is only accessible in the `'error'` branch.
4. Don't destructure the props in the parameter list (`function Status({ status, error })`) — destructuring before narrowing loses the link between the fields. Narrow `props` first, then read fields. (Recent TypeScript versions can narrow destructured `const` variables in simple cases, but keeping `props` whole is the reliable pattern.)
5. `prop?: never` means "this prop must not be passed" — the standard way to express "exactly one of these".

Real-world examples

- Component libraries use `as` / `asChild` props or link-vs-button unions so a clickable thing renders correct, accessible HTML.
- Alerts/toasts with `variant` unions, form fields where `type: 'select'` requires `options`, modals where `dismissible: true` requires `onClose`.

In the interview

30-second answer: "I type props with a type alias on the destructured parameter, `children` as `ReactNode`, and extend native elements with `ComponentProps<'button'>`. Handlers use React's event types such as `ChangeEvent<HTMLInputElement>`, though inline handlers are inferred. For state I let inference work and add generics for `null` or empty arrays. For variant props I use discriminated unions so invalid combinations don't compile, and reducers get action unions with a `never` exhaustiveness check. Reusable lists and tables are generic components."

- *`ReactNode` vs `ReactElement` vs `JSX.Element`?* — `ReactNode` is anything renderable (use for children); `ReactElement` / `JSX.Element` is only a JSX element.
- *How do you type `ref` in React 19?* — As a normal prop: `ref?: Ref<HTMLInputElement>`, or get it for free from `ComponentProps<'input'>`; `forwardRef` isn't needed.
- *Why not `any`?* — It disables checking and spreads; use `unknown` and narrow, or a proper type.

TL;DR

One union member per valid prop combination, sharing a literal discriminant; narrow `props` before reading fields; `?: never` forbids a prop.

11.10 Reading type errors, and common mistakes

TypeScript error messages look intimidating, but they follow one pattern: *"Type **A** (what you gave) is not assignable to type **B** (what was expected)"*, often followed by indented lines that zoom in on the exact property that differs. Read the **last** indented line first — it's usually the real reason.

`code/bad_types.tsx` collects twelve common mistakes (it is excluded from the normal check and from the runner):

```tsx
const e1 = <Badge />;                                  // 1. required prop missing
const e2 = <Badge label="x" tone="danger" />;           // 2. value outside the union

function Profile() {
  const [user] = useState<{ name: string } | null>(null);
  const [tags, setTags] = useState([]);
  setTags(['a']);                                       // 4. useState([]) without a type -> never[]
  return <p>{user.name} {tags.length}</p>;              // 3. user may be null
}

function Plan() {
  function handle(e: ChangeEvent<HTMLInputElement>) { console.log(e.target.value); }
  return <select onChange={handle} />;                  // 5. input handler on a <select>
}

const e6 = <Status status="success" />;                 // 6. 'success' requires count
const e7 = <Thumb src="/a.png" icon="star" />;          // 7. both src and icon
const e8 = <Table rows={[{ id: 1, title: 'Dune' }]} columns={[{ key: 'author', header: 'Author' }]} />;  // 8. no such key

type Action = { type: 'add'; sku: string } | { type: 'clear' };
function reducer(state: string[], action: Action): string[] {
  switch (action.type) {
    case 'add': return [...state, action.sku];
    default: { const unreachable: never = action; return unreachable; }   // 9. 'clear' not handled
  }
}
function Cart() { const [items, dispatch] = useReducer(reducer, []); dispatch({ type: 'remove' }); return <p>{items}</p>; }  // 10. unknown action

function Focus() {
  const ref = useRef<HTMLInputElement>();               // 11. React 19 types: useRef needs an argument
  return <input ref={ref} />;
}

const e12 = <Box>{{ title: 'not renderable' }}</Box>;   // 12. a plain object is not a ReactNode
```

Output of `tsc --noEmit -p tsconfig.bad.json` (real, unedited):

```
bad_types.tsx(7,13): error TS2741: Property 'label' is missing in type '{}' but required in type 'BadgeProps'.
bad_types.tsx(8,29): error TS2322: Type '"danger"' is not assignable to type '"error" | "info" | "warning" |
undefined'.
bad_types.tsx(13,12): error TS2322: Type 'string' is not assignable to type 'never'.
bad_types.tsx(14,14): error TS18047: 'user' is possibly 'null'.
bad_types.tsx(19,18): error TS2322: Type '(e: ChangeEvent<HTMLInputElement, Element>) => void' is not assignable to
type 'ChangeEventHandler<HTMLSelectElement, HTMLSelectElement>'.
  Types of parameters 'e' and 'event' are incompatible.
    Type 'ChangeEvent<HTMLSelectElement, HTMLSelectElement>' is not assignable to type 'ChangeEvent<HTMLInputElement,
Element>'.
      Type 'HTMLSelectElement' is missing the following properties from type 'HTMLInputElement': accept, align, alt,
capture, and 38 more.
bad_types.tsx(24,13): error TS2322: Type '{ status: "success"; }' is not assignable to type 'IntrinsicAttributes &
StatusProps'.
  Property 'count' is missing in type '{ status: "success"; }' but required in type '{ status: "success"; count:
number; }'.
bad_types.tsx(28,32): error TS2322: Type 'string' is not assignable to type '"star" | "user" | undefined'.
bad_types.tsx(32,65): error TS2322: Type '"author"' is not assignable to type '"id" | "title"'.
bad_types.tsx(38,22): error TS2322: Type '{ type: "clear"; }' is not assignable to type 'never'.
bad_types.tsx(41,81): error TS2322: Type '"remove"' is not assignable to type '"add" | "clear"'.
bad_types.tsx(44,15): error TS2554: Expected 1 arguments, but got 0.
bad_types.tsx(49,21): error TS2353: Object literal may only specify known properties, and 'title' does not exist in
type 'Iterable<ReactNode> | Promise<AwaitedReactNode> | ReactElement<unknown, string | JSXElementConstructor<any>> |
ReactPortal'.
```

Decoding each error

#	Error says	What it means	Fix
1	`Property 'label' is missing`	required prop not passed	pass it, or make it optional with a default
2	`"danger"` not assignable to the union	typo or unsupported variant	use an allowed value, or extend the union
3	`'user' is possibly 'null'`	state can still be `null`	narrow first: `user ? ... : ...` or `user?.name`
4	`string` not assignable to `never`	`useState([])` inferred `never[]`	`useState<string[]>([])`
5	handler for `HTMLInputElement` on a `<select>`	wrong element type in the event	`ChangeEvent<HTMLSelectElement>` (note: in `@types/react` 19 the event types have a second type parameter for `target`, which you normally leave out)
6	`Property 'count' is missing ... required in type '{ status: "success"; count: number; }'`	the chosen union member needs `count`	pass `count`
7	`string` not assignable to `"star" | "user" | undefined` on `icon`	with `src` given, TypeScript tried the members and none fits both props — the message points at one of them	pass only one of `src` / `icon`
8	`"author"` not assignable to `"id" | "title"`	column names a field the row type doesn't have	fix the key, or add the field to the data
9	`{ type: "clear" }` not assignable to `never`	exhaustiveness check caught an unhandled action	add `case 'clear'`
10	`"remove"` not assignable to `"add" | "clear"`	dispatching an action the reducer doesn't know	dispatch a valid action or add it to the union
11	`Expected 1 arguments, but got 0`	React 19 types require an initial value for `useRef`	`useRef<HTMLInputElement>(null)`
12	`'title' does not exist in type ... ReactNode ...`	rendering a plain object as a child (would also crash at runtime)	render a field: `{post.title}`

Other common mistakes

- **`as` everywhere.** `as Something` silences the checker without checking. Use it only when you truly know more than the compiler (as in `e.target.value as V` above), never to hide a real mismatch.
- **Non-null assertions** `user!.name` — same risk as `as`; narrow instead.
- **Duplicating types that already exist.** Use `ComponentProps<typeof X>`, `ReturnType<typeof fn>`, `Parameters<...>`, or export the props type from the component file.
- **Typing API responses by hand and trusting them** — validate at the boundary (Zod or similar) and derive the type from the schema.
- **Over-engineering generics** — a component used for one type doesn't need them.

Self-check

1. Why does `useState([])` cause trouble? → The type is inferred as `never[]`, so nothing can be added.
2. Which type do you use for `children`? → `ReactNode`.
3. How do you get all the props of a native `<input>`? → `ComponentProps<'input'>`.
4. How do you make a reducer fail to compile when an action isn't handled? → Assign `action` to a `never` variable in `default`.
5. In React 19, how does a function component receive a ref? → As a regular `ref` prop typed `Ref<T>`.
6. Does `tsc` run when Vite builds your app? → No; the bundler only strips types. Run `tsc --noEmit` separately (CI or build script).

Chapter 11 TL;DR

Idea	Remember
Setup	`.tsx`, `@types/react(-dom)`, `strict: true`, `jsx: "react-jsx"`; bundler strips types, `tsc --noEmit` checks them
Props	`function C({ a, b = 1 }: Props)`; `?` optional; literal unions for variants; function types for callbacks
Children	`ReactNode`; `PropsWithChildren<P>`; native props via `ComponentProps<'button'>`
Events	Inline = inferred; otherwise `ChangeEvent<HTMLInputElement>`, `SubmitEvent<HTMLFormElement>` (older: `FormEvent`), `MouseEvent<...>`, `KeyboardEvent<...>` from `react`
State	Infer; `useState<T | null>(null)`, `useState<string[]>([])`, literal unions
Reducers	Action = discriminated union; `never` exhaustiveness check
Refs	`useRef<HTMLXxxElement>(null)`; argument required in React 19 types; `ref` is a prop typed `Ref<T>`; `useImperativeHandle` for custom handles
Hooks & Context	`as const` tuples or objects; generic Hooks with union results; `createContext<T | null>(null)` + throwing `useX()`
Generic components	`List<T>` infers `T` from props; constraints with `extends`; `keyof`; `<T,>` in arrow functions
Union props	One member per valid combination; narrow before destructuring; `?: never` to forbid
Errors	Read "A is not assignable to B" bottom-up; avoid `any`, `as`, `!` as escape hatches

Next: Chapter 12 — Testing React.

Chapter 12 — Testing React

A test is a small program that uses your code and checks the result, so that a computer — not you, clicking around — notices when something breaks. For React components the most useful tests **render the component, interact with it the way a user would, and check what the user would see**. That is the idea behind **React Testing Library** (RTL), the standard tool for component tests, and it's what this chapter teaches.

General testing ideas — what a test is, `node:test`, assertions, test doubles and mocking — are covered in the JavaScript Handbook (Chapter 16). Here we apply them to React.

What you'll learn

#	Topic	You will be able to…
12.1	What to test, and the Testing Library philosophy	decide what a good React test checks, and what it ignores
12.2	Your first component test	render, find, interact, assert — and clean up
12.3	Queries and their priority	choose `getByRole` first, and know `get` / `query` / `find` and `All` variants
12.4	`user-event` — interacting like a user	type, click, select, tab and press keys realistically
12.5	Async tests	wait for UI with `findBy`, `waitFor`, `waitForElementToBeRemoved`
12.6	Mocking `fetch` (and when to use MSW)	test loading, success, HTTP errors, network errors and races
12.7	Testing custom Hooks	use `renderHook`, `act`, fake timers — or a test component
12.8	Providers, routers and custom render	test components that need context or a router
12.9	Setting up Vitest or Jest in a real project	configure jsdom, jest-dom matchers and scripts
12.10	Good tests, bad tests, and the bigger picture	avoid brittle tests; where E2E and other tests fit

Code: `code/`. Every file is a **runnable test file**: React 19.3, React Testing Library 16.3, `@testing-library/user-event` 14.6 and jsdom, executed by Node's built-in test runner (`node:test`) with `node:assert/strict` for assertions. The output shown is Node's `spec` reporter (`node --test-reporter=spec file.mjs`); times in parentheses vary per run, and the summary lines for suites, cancelled, skipped and todo (all 0) are left out. Real projects usually run the same tests with Vitest or Jest — §12.9 shows that setup (not run here).

12.1 What to test, and the Testing Library philosophy

What it is

React Testing Library is a small set of helpers built on **DOM Testing Library**: `render` a component into a real (simulated) DOM, **query** elements the way a person finds them (by role, label, text), fire **events**, and assert on the resulting DOM. Its guiding principle, in short: the closer a test is to how people actually use the software, the more you can trust it when it passes.

In practice that means:

Test this (what users experience)	Not this (implementation details)
The heading shows "Ada Lovelace" after loading	The component's `user` state variable
Clicking **Add to cart** updates the badge to 2	That `dispatch` was called with `{ type: 'add' }`
The **Submit** button is disabled until terms are accepted	Which Hook or CSS class name is used
An error message appears when the server returns 404	The number of re-renders

Analogy: a restaurant inspector who orders food and tastes it, rather than reading the chef's recipe cards. If the chef reorganises the kitchen (a refactor) but the food is the same, the inspection still passes — and it fails exactly when the food gets worse.

Why it matters

- **Refactor-proof**: tests that don't touch internals keep passing when you rename state, split components, switch from `useState` to `useReducer`, or enable React Compiler (Chapter 10).
- **Accessibility for free**: finding elements by role and accessible name only works if the UI is accessible (you'll see a test catch an unlabelled icon button in §12.3).
- **Confidence**: a test that clicks the real button through the real event system catches wiring bugs that a unit test of a handler function would miss.

Where component tests sit

Kind	Checks	Tools	Speed
Unit	one function (a reducer, a formatter)	`node:test`, Vitest, Jest	fastest
Component / integration	one component or a feature of several, in a simulated DOM	RTL + jsdom (this chapter)	fast
End-to-end (E2E)	the whole app in a real browser against a real or staged backend	Playwright, Cypress	slowest

A common strategy (sometimes called the "testing trophy") puts most effort into **integration-style component tests**, a few E2E tests for critical flows (sign-up, checkout), and unit tests for tricky pure logic — plus static checks (TypeScript, Chapter 11; ESLint) that catch whole classes of mistakes without running anything.

> **TL;DR**
>
> Render, find like a user, act like a user, assert on what a user sees. Don't test state, props, or which functions were called internally.

12.2 Your first component test

How it works

1. **Arrange** — `render(<Counter />)` mounts the component into `document.body` (jsdom provides the DOM in Node).
2. **Act** — find an element with a query (`screen.getByRole('button', { name: 'Increment' })`) and interact (`fireEvent.click(...)` or `user-event`, §12.4).
3. **Assert** — check the DOM: text, disabled state, presence or absence of elements.
4. **Clean up** — unmount after each test so tests don't see each other's DOM. With Jest or Vitest (with `globals: true`) RTL does this automatically; with `node:test` you register `afterEach(cleanup)` yourself.

`screen` is an object with every query bound to `document.body`, so you don't need to keep the return value of `render`.

`code/01_first_test.jsx`:

```
import '../../../../tools_dom.mjs';                    // jsdom globals (window, document) for Node
import { test, afterEach } from 'node:test';
import assert from 'node:assert/strict';
import { useState } from 'react';
import { render, screen, fireEvent, cleanup } from '@testing-library/react';

function Counter({ initial = 0, step = 1 }) {
  const [count, setCount] = useState(initial);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + step)}>Increment</button>
      <button onClick={() => setCount(initial)} disabled={count === initial}>Reset</button>
    </div>
  );
}

afterEach(cleanup);

test('shows the initial count', () => {
  render(<Counter initial={5} />);
  assert.ok(screen.getByText('Count: 5'));                              // getBy* throws if not found
});

test('increments by step when the button is clicked', () => {
  render(<Counter step={10} />);
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  assert.equal(screen.getByText(/Count:/).textContent, 'Count: 20');
});

test('Reset is disabled until the count changes', () => {
  render(<Counter />);
  const reset = screen.getByRole('button', { name: 'Reset' });
  assert.equal(reset.disabled, true);
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  assert.equal(reset.disabled, false);
  fireEvent.click(reset);
  assert.equal(screen.getByText(/Count:/).textContent, 'Count: 0');
});
```

Output:

```
✔ shows the initial count (13.84825ms)
✔ increments by step when the button is clicked (50.187958ms)
✔ Reset is disabled until the count changes (3.544833ms)
ℹ tests 3
ℹ pass 3
ℹ fail 0
ℹ duration_ms 70.571583
```

What about `act()`?

React batches updates and runs effects asynchronously. `act(() => { ... })` tells React "apply everything this callback caused before I look at the DOM". RTL's `render`, `fireEvent`, `user-event` actions and async utilities already wrap themselves in `act`, so in component tests you rarely write it. You do need it when you trigger state updates **directly**, e.g. calling a Hook's function in `renderHook` (§12.7) or advancing fake timers. The warning *"An update to X inside a test was not wrapped in act(...)"* usually means something finished **after** your assertions — await it with `findBy`/`waitFor` (§12.5).

With jest-dom (in Vitest/Jest)

Projects usually add `@testing-library/jest-dom`, which gives readable matchers with good failure messages: `expect(reset).toBeDisabled()`, `expect(el).toHaveTextContent('Count: 20')`, `expect(el).toBeInTheDocument()`. This book's runner uses plain `assert`, which checks the same DOM properties (`.disabled`, `.textContent`).

> **TL;DR**

`render` → `screen.getByRole(...)` → interact → assert on the DOM; `cleanup` after each test (automatic in Jest/Vitest).

12.3 Queries and their priority

Query variants

Every query comes in six forms. The prefix decides **what happens when zero or several elements match**, and whether it waits:

Variant	0 matches	1 match	2+ matches	Waits?	Use it to…
`getBy…`	throws	returns element	throws	no	find something that must be there **now**
`queryBy…`	returns `null`	returns element	throws	no	assert something is **absent**
`findBy…`	rejects	resolves element	rejects	**yes** (retries, default 1000 ms)	wait for something to **appear**
`getAllBy…`	throws	array	array	no	find several
`queryAllBy…`	`[]`	array	array	no	count, possibly zero
`findAllBy…`	rejects	array	array	**yes**	wait for several

Which query to use (priority order)

Testing Library recommends choosing queries in this order — the higher, the closer to how people (including screen-reader users) find things:

1. **Accessible to everyone**
    1. `ByRole` — elements by their ARIA role (button, link, heading, textbox, checkbox, list, …) and **accessible name** (`{ name: 'Save' }`). Use this for almost everything.
    2. `ByLabelText` — form fields by their `<label>`.
    3. `ByPlaceholderText` — only if a field truly has no label (a placeholder is not a label).
    4. `ByText` — non-interactive text (paragraphs, spans, divs).
    5. `ByDisplayValue` — a form field by its current value.
2. **Semantic attributes**
    1. `ByAltText` — images and other elements with `alt`.
    2. `ByTitle` — the `title` attribute (not reliably announced by screen readers).
3. **Test IDs**: `ByTestId` — `data-testid="…"`, only when nothing user-visible identifies the element.

The **accessible name** is what assistive technology announces: a button's text, an input's label, an `aria-label` or `aria-labelledby`, an image's `alt`. Text matchers can be exact strings (`'Save'`), case-insensitive regexes (`/save/i`), or functions.

`code/02_queries.jsx` (component and key tests):

```
function ProductCard({ product, inCart }) {
  return (
    <article aria-labelledby="p-title">
      <img src={product.image} alt={`Photo of ${product.name}`} />
      <h2 id="p-title">{product.name}</h2>
      <p>{product.price.toFixed(2)} EUR</p>
      <label htmlFor="qty">Quantity</label>
      <input id="qty" type="number" defaultValue={1} />
      <input placeholder="Gift message (optional)" />
      <ul aria-label="features">{product.features.map(f => <li key={f}>{f}</li>)}</ul>
      <button>Add to cart</button>
      <button title="Save for later">♡</button>
      {inCart && <p role="status">Already in your cart</p>}
      <span data-testid="sku">{product.sku}</span>
    </article>
  );
}

test('priority 1: queries everyone can perceive (role, label, placeholder, text, display value)', () => {
  render(<ProductCard product={lamp} />);
  assert.equal(screen.getByRole('heading', { name: 'Desk lamp', level: 2 }).tagName, 'H2');
  assert.equal(screen.getByRole('button', { name: 'Add to cart' }).tagName, 'BUTTON');
  assert.equal(screen.getByRole('article', { name: 'Desk lamp' }).tagName, 'ARTICLE');  // name from aria-labelledby
  assert.equal(screen.getByLabelText('Quantity').value, '1');
  assert.ok(screen.getByPlaceholderText(/gift message/i));
  assert.ok(screen.getByText('39.50 EUR'));
  assert.equal(screen.getByDisplayValue('1').id, 'qty');
});

test('priority 2: semantic attributes (alt text, title)', () => {
  render(<ProductCard product={lamp} />);
  assert.equal(screen.getByAltText(/photo of desk lamp/i).getAttribute('src'), '/lamp.jpg');
  assert.ok(screen.getByTitle('Save for later'));
  // A button's accessible name comes from its CONTENT first; title is only a fallback. So a screen
  // reader announces this icon button as "♡" - the test just revealed an accessibility bug.
  assert.ok(screen.getByRole('button', { name: '♡' }));
  assert.equal(screen.queryByRole('button', { name: 'Save for later' }), null);
  // Fix in the component: <button aria-label="Save for later">♡</button>
});

test('getAllBy for many matches, within() to search inside one element', () => {
  render(<ProductCard product={lamp} />);
  const list = screen.getByRole('list', { name: 'features' });
  const items = within(list).getAllByRole('listitem');
  assert.deepEqual(items.map(li => li.textContent), ['LED', 'Dimmable', 'USB-C']);
});

test('queryBy to assert that something is NOT there', () => {
  const { rerender } = render(<ProductCard product={lamp} inCart={false} />);
  assert.equal(screen.queryByRole('status'), null);         // queryBy returns null instead of throwing
  rerender(<ProductCard product={lamp} inCart />);
  assert.equal(screen.getByRole('status').textContent, 'Already in your cart');
});
// ... plus: a failing getBy's message, logRoles, and getByTestId('sku')
```

Output:

```
    error message (first 3 lines):
    | Unable to find an accessible element with the role "button" and name "Buy now"
    |
    | Here are the accessible roles:
    logRoles output:
    | list:
    | Name "tiny":
    | <ul
    |   aria-label="tiny"
    | />
    | --------------------------------------------------
    | listitem:
    | Name "":
    | <li />
    | --------------------------------------------------
✔ priority 1: queries everyone can perceive (role, label, placeholder, text, display value) (61.198666ms)
✔ priority 2: semantic attributes (alt text, title) (6.331375ms)
✔ priority 3: test IDs, only when nothing user-visible identifies the element (2.198959ms)
✔ getAllBy for many matches, within() to search inside one element (8.569083ms)
✔ queryBy to assert that something is NOT there (4.234708ms)
✔ a failing getBy explains itself (13.676958ms)
✔ logRoles prints every role and accessible name - handy when writing queries (2.698125ms)
ℹ tests 7
ℹ pass 7
ℹ fail 0
```

Notes on the output

- A failing `getBy` lists **every accessible role and name** on the page (cut to 3 lines here), followed by the DOM. Reading it usually tells you immediately whether you used the wrong name or the element really is missing.
- `logRoles(container)` prints the same role list on demand; `screen.debug()` prints the current DOM. Both are for writing tests, not for leaving in them.
- Roles you'll use most: `button`, `link`, `heading` (with `level`), `textbox`, `checkbox`, `radio`, `combobox` (a `<select>`), `spinbutton` (a number input), `list` / `listitem`, `table` / `row` / `cell`, `dialog`, `alert`, `status`, `img`, `navigation`, `main`.
- The icon-button test shows RTL doing accessibility work: the button had a `title` but its announced name was "♡". Queries by role make such bugs visible.

Common mistakes

Mistake	Better
`container.querySelector('.btn-primary')`	`screen.getByRole('button', { name: 'Save' })`
`getByTestId` everywhere	role/label/text first; test IDs only as a last resort
`expect(queryByText('x')).toBeTruthy()` to check presence	`getByText` (throws with a helpful message)
`getByText` to assert absence (it throws)	`queryByText(...) === null`
`getBy` for things that appear later	`findBy` (§12.5)
Exact long text strings that break on copy edits	a regex for the stable part: `/order confirmed/i`

TL;DR

Prefer `getByRole(role, { name })`; then label, placeholder, text, display value; then alt/title; test IDs last. `get` = must exist now, `query` = may be absent, `find` = wait for it.

12.4 `user-event` — interacting like a user

What it is

`fireEvent` dispatches **one** DOM event. A real user typing "hi" produces focus, click, and for each key `keydown`, `keypress`, `input`, `keyup` — and a real click on a disabled button does nothing, a real Tab moves focus in document

order. `@testing-library/user-event` simulates those full interactions, so your tests exercise the same paths as real users (including bugs that only appear with the full sequence, like a handler on `keydown`).

```
import userEvent from '@testing-library/user-event';
const user = userEvent.setup();             // v14 API: create a session, then await each action
await user.type(input, 'Ada');
await user.click(button);
await user.selectOptions(select, 'pro');
await user.keyboard('{Enter}');             // special keys in braces: {Enter} {Escape} {Backspace} {Shift>}...
await user.tab();
await user.clear(input);
await user.upload(fileInput, file);
await user.hover(element);
```

All methods are **async** — always `await` them.

`code/03_user_event.jsx`:

```
function SignupForm({ onSubmit }) {
  const [form, setForm] = useState({ name: '', plan: 'free', terms: false });
  const set = (k, v) => setForm(f => ({ ...f, [k]: v }));
  return (
    <form onSubmit={e => { e.preventDefault(); onSubmit(form); }}>
      <label>Name <input value={form.name} onChange={e => set('name', e.target.value)} /></label>
      <label>Plan
        <select value={form.plan} onChange={e => set('plan', e.target.value)}>
          <option value="free">Free</option><option value="pro">Pro</option>
        </select>
      </label>
      <label><input type="checkbox" checked={form.terms} onChange={e => set('terms', e.target.checked)} /> I accept the
terms</label>
      <button type="submit" disabled={!form.terms || !form.name}>Create account</button>
    </form>
  );
}

test('fireEvent vs userEvent: which DOM events does "typing" produce?', async () => {
  render(<input aria-label="box" />);
  const box = screen.getByLabelText('box');
  const seen = [];
  for (const type of ['focus', 'keydown', 'keypress', 'input', 'keyup', 'click'])
    box.addEventListener(type, () => seen.push(type));
  fireEvent.change(box, { target: { value: 'hi' } });
  console.log('    fireEvent.change   ->', seen.join(', ') || '(no listened-to events; only a synthetic change)');
  seen.length = 0; box.value = '';
  const user = userEvent.setup();
  await user.type(box, 'hi');
  console.log('    user.type(box, hi) ->', seen.join(', '));
});

test('fills in and submits the whole form like a user', async () => {
  const user = userEvent.setup();
  const submitted = [];
  render(<SignupForm onSubmit={data => submitted.push(data)} />);
  const button = screen.getByRole('button', { name: 'Create account' });
  assert.equal(button.disabled, true);
  await user.type(screen.getByRole('textbox', { name: 'Name' }), 'Ada');
  await user.selectOptions(screen.getByRole('combobox', { name: 'Plan' }), 'pro');
  await user.click(screen.getByRole('checkbox', { name: 'I accept the terms' }));
  assert.equal(button.disabled, false);
  await user.click(button);
  assert.deepEqual(submitted, [{ name: 'Ada', plan: 'pro', terms: true }]);
});

test('keyboard only: Tab moves focus in DOM order, Enter submits', async () => {
  const user = userEvent.setup();
  const submitted = [];
  render(<SignupForm onSubmit={data => submitted.push(data)} />);
  await user.tab();
  assert.equal(document.activeElement, screen.getByRole('textbox', { name: 'Name' }));
  await user.keyboard('Grace');
  await user.tab(); await user.tab();                    // skip the select, reach the checkbox
  assert.equal(document.activeElement, screen.getByRole('checkbox'));
  await user.keyboard(' ');                              // Space toggles a checkbox
  await user.click(screen.getByRole('textbox', { name: 'Name' }));
  await user.keyboard('{Enter}');                        // Enter in a text field submits the form
  assert.deepEqual(submitted, [{ name: 'Grace', plan: 'free', terms: true }]);
});

test('clicking a disabled button does nothing', async () => {
  const user = userEvent.setup();
  let clicks = 0;
  render(<button disabled onClick={() => clicks++}>Pay</button>);
  await user.click(screen.getByRole('button', { name: 'Pay' }));
  assert.equal(clicks, 0);
});
```

Output:

```
    fireEvent.change   -> (no listened-to events; only a synthetic change)
    user.type(box, hi) -> focus, click, keydown, keypress, input, keyup, keydown, keypress, input, keyup
✔ fireEvent vs userEvent: which DOM events does "typing" produce? (72.232958ms)
✔ fills in and submits the whole form like a user (55.993083ms)
✔ keyboard only: Tab moves focus in DOM order, Enter submits (49.1405ms)
✔ clicking a disabled button does nothing (7.998ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

`fireEvent` **vs** `user-event`

	`fireEvent`	`user-event`
What it does	dispatches exactly one event	simulates the full interaction (focus, pointer, keyboard, input)
Sync/async	sync	async (`await`)
Respects disabled, `pointer-events: none`, focus order	no	yes
When to use	events user-event doesn't model (e.g. `scroll`, custom events), or very low-level tests	**`default`** for anything a user does

The keyboard test is also an accessibility check: it proves the form is usable without a mouse.

TL;DR

`const user = userEvent.setup()` then `await user.type/click/selectOptions/keyboard/tab(...)`. Prefer it over `fireEvent` for user actions.

12.5 Async tests

The problem it solves

Real components load data, wait for timers and animate. Right after `render` the data isn't there yet, so a `getBy` fails. You must **wait for the UI**, not for a fixed amount of time (`await sleep(500)` is slow and flaky — it passes on your machine and fails on a busy CI server).

Tools

Tool	Waits for	Typical use
`await screen.findByRole(...)`	an element to appear (retries a `getBy` query)	data loaded, dialog opened
`await waitFor(() => { ...assertions... })`	a callback to stop throwing	anything that isn't "an element appears": a text change, a mock being called
`await waitForElementToBeRemoved(() => screen.queryByText('Loading…'))`	an element to disappear	spinners, closing dialogs

All of them retry every 50 ms (default) until success or a timeout (default 1000 ms), and wrap updates in `act` for you.

`code/04_async.jsx`:

```
const api = {
  loadOrders: () => new Promise(r => setTimeout(() => r([{ id: 1, total: 20 }, { id: 2, total: 35 }]), 50)),
  save: () => new Promise(r => setTimeout(r, 30)),
};

function Orders() {
  const [orders, setOrders] = useState(null);
  const [saved, setSaved] = useState(false);
  useEffect(() => { api.loadOrders().then(setOrders); }, []);
  if (!orders) return <p>Loading orders…</p>;
  return (
    <>
      <ul>{orders.map(o => <li key={o.id}>Order {o.id}: {o.total} EUR</li>)}</ul>
      <button onClick={async () => { await api.save(); setSaved(true); }}>Save</button>
      {saved && <p role="status">Saved!</p>}
    </>
  );
}

test('right after render only the loading state exists', () => {
  render(<Orders />);
  assert.ok(screen.getByText('Loading orders…'));
  assert.equal(screen.queryByText(/Order 1/), null);
});

test('findBy waits (retries up to 1000 ms by default) until the element appears', async () => {
  render(<Orders />);
  const items = await screen.findAllByRole('listitem');
  assert.deepEqual(items.map(li => li.textContent), ['Order 1: 20 EUR', 'Order 2: 35 EUR']);
  assert.equal(screen.queryByText('Loading orders…'), null);
});

test('waitForElementToBeRemoved waits for the spinner to go away', async () => {
  render(<Orders />);
  await waitForElementToBeRemoved(() => screen.queryByText('Loading orders…'));
  assert.equal(screen.getAllByRole('listitem').length, 2);
});

test('waitFor retries any assertion until it passes', async () => {
  const user = userEvent.setup();
  render(<Orders />);
  await user.click(await screen.findByRole('button', { name: 'Save' }));
  await waitFor(() => assert.equal(screen.getByRole('status').textContent, 'Saved!'));
});

test('findBy gives up after its timeout with a clear error', async () => {
  render(<Orders />);
  const t0 = Date.now();
  await assert.rejects(screen.findByText('Order 99', {}, { timeout: 200 }), /Unable to find an element with the text:
Order 99/);
  console.log(`    rejected after ~${Math.round((Date.now() - t0) / 100) * 100} ms`);
});
```

Output:

```
✔ right after render only the loading state exists (14.355875ms)
✔ findBy waits (retries up to 1000 ms by default) until the element appears (98.487708ms)
✔ waitForElementToBeRemoved waits for the spinner to go away (55.889ms)
✔ waitFor retries any assertion until it passes (101.515625ms)
    rejected after ~200 ms
✔ findBy gives up after its timeout with a clear error (202.66725ms)
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

Rules for `waitFor`

- Put **only assertions** inside the callback — it may run many times. Do actions (clicks, typing) outside it.

- One thing to wait for per `waitFor`; then assert the rest normally.
- Prefer `findBy` when you're waiting for an element; it reads better and gives better errors.
- Don't wrap `findBy` in `waitFor` — it already waits.

Common mistakes

- Forgetting `await` before `findBy`/`waitFor`/`user.*` — the test finishes before the UI updates, passes by accident, and prints an `act(...)` warning.
- Arbitrary `sleep`s. Wait for a **condition** instead. (For code that genuinely depends on time — debounce, polling — use fake timers, §12.7.)

TL;DR

`await findBy…` for appearing elements, `waitForElementToBeRemoved` for disappearing ones, `waitFor(() => assertion)` for anything else. Never sleep.

12.6 Mocking `fetch` (and when to use MSW)

What it is

A **mock** replaces a real dependency with a controllable fake. Components that call `fetch` shouldn't hit a real server in tests: it's slow, needs a network, and returns different data over time. Mocking `fetch` lets each test choose the response — success, an HTTP error, a network failure, a slow reply — and check the UI for each.

With `node:test`, `mock.method(globalThis, 'fetch', impl)` replaces the global `fetch` and records every call; `mock.restoreAll()` puts the original back. (Vitest: `vi.spyOn(globalThis, 'fetch').mockResolvedValue(...)`; Jest: `jest.spyOn(global, 'fetch')`.) Node 18+ has `Response` built in, so the fake can return a real `Response` object.

`code/05_mocking_fetch.jsx`:

```
function UserProfile({ userId }) {
  const [state, setState] = useState({ status: 'loading' });
  useEffect(() => {
    const controller = new AbortController();
    setState({ status: 'loading' });
    fetch(`/api/users/${userId}`, { signal: controller.signal })
      .then(res => { if (!res.ok) throw new Error(`Server said ${res.status}`); return res.json(); })
      .then(user => setState({ status: 'done', user }))
      .catch(err => { if (err.name !== 'AbortError') setState({ status: 'error', message: err.message }); });
    return () => controller.abort();                  // cancel if userId changes or we unmount
  }, [userId]);

  if (state.status === 'loading') return <p>Loading…</p>;
  if (state.status === 'error') return <p role="alert">Could not load user: {state.message}</p>;
  return <h1>{state.user.name}</h1>;
}

const json = (body, status = 200) => new Response(JSON.stringify(body), { status, headers: { 'Content-Type':
'application/json' } });
afterEach(() => { cleanup(); mock.restoreAll(); });    // put the real fetch back after every test

test('shows the user returned by the server, and calls the right URL', async () => {
  const fetchMock = mock.method(globalThis, 'fetch', async () => json({ id: 7, name: 'Ada Lovelace' }));
  render(<UserProfile userId={7} />);
  assert.equal((await screen.findByRole('heading')).textContent, 'Ada Lovelace');
  assert.equal(fetchMock.mock.callCount(), 1);
  assert.equal(fetchMock.mock.calls[0].arguments[0], '/api/users/7');
});

test('shows an error for an HTTP error status (fetch does not reject on 404)', async () => {
  mock.method(globalThis, 'fetch', async () => json({ error: 'not found' }, 404));
  render(<UserProfile userId={999} />);
  assert.equal((await screen.findByRole('alert')).textContent, 'Could not load user: Server said 404');
});

test('shows an error when the network fails', async () => {
  mock.method(globalThis, 'fetch', async () => { throw new TypeError('Failed to fetch'); });
  render(<UserProfile userId={1} />);
  assert.match((await screen.findByRole('alert')).textContent, /Failed to fetch/);
});

test('a slow old response cannot overwrite a newer one (race condition)', async () => {
  // user 1 answers after 100 ms, user 2 after 10 ms; honour the abort signal like real fetch does
  const delays = { '/api/users/1': 100, '/api/users/2': 10 };
  mock.method(globalThis, 'fetch', (url, { signal }) => new Promise((resolve, reject) => {
    const t = setTimeout(() => resolve(json({ name: `User from ${url}` })), delays[url]);
    signal.addEventListener('abort', () => { clearTimeout(t); reject(new DOMException('aborted', 'AbortError')); });
  }));
  const { rerender } = render(<UserProfile userId={1} />);
  rerender(<UserProfile userId={2} />);                  // user switches before the first answer
  assert.equal((await screen.findByRole('heading')).textContent, 'User from /api/users/2');
  await new Promise(r => setTimeout(r, 150));             // wait past the slow response
  assert.equal(screen.getByRole('heading').textContent, 'User from /api/users/2');
});
// ... plus a test that clicks "Next user" and waits for the second request's result
```

Output:

```
✔ shows the user returned by the server, and calls the right URL (67.915ms)
✔ shows an error for an HTTP error status (fetch does not reject on 404) (6.110375ms)
✔ shows an error when the network fails (4.473667ms)
✔ a slow old response cannot overwrite a newer one (race condition) (168.044584ms)
✔ mock only the network call, then drive the UI as a user (20.320625ms)
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

Does the race test really test something? A test that can't fail is worthless. Removing the `controller.abort()` cleanup from `UserProfile` (a temporary copy of the file) makes it fail:

```
✖ a slow old response cannot overwrite a newer one (race condition) (168.159708ms)
  + actual - expected
    actual: 'User from /api/users/1',
    expected: 'User from /api/users/2',
```

Checking that a test fails when the code is broken is a good habit for any test you're not sure about. (The race itself and the `AbortController` fix are explained in Chapter 5.)

The last test shows the preferred style: mock **only the network boundary**, then drive the real UI with `user-event`.

Mock Service Worker (MSW) — the usual choice in real projects

MSW intercepts requests at the network level instead of replacing `fetch`, so it works for `fetch`, `axios`, TanStack Query, and anything else, and the same handlers can power the dev server in the browser. Sketch (MSW 2 — not run here):

```
// test/server.js
import { setupServer } from 'msw/node';
import { http, HttpResponse } from 'msw';
export const server = setupServer(
  http.get('/api/users/:id', ({ params }) => HttpResponse.json({ id: params.id, name: 'Ada Lovelace' })),
);
// setupTests.js
beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());       // undo per-test overrides
afterAll(() => server.close());
// in one test: server.use(http.get('/api/users/:id', () => new HttpResponse(null, { status: 404 })));
```

Approach	Good for	Drawback
Replace `fetch` (`mock.method`, `vi.spyOn`)	small projects, a few tests, no extra dependency	tied to `fetch`; each test repeats response setup
MSW	real apps; any HTTP client; shared handlers for tests, Storybook and dev	one more library and setup file
Mock your own API module (`vi.mock('./api')`)	quick isolation	skips your fetch/parse/error code — lower confidence

Components using **TanStack Query** also need a `QueryClientProvider` in the test wrapper — create a **new** `QueryClient` per test with `retry: false`, so tests don't share cache and failures appear immediately (Chapter 9; providers in §12.8).

> **TL;DR**

Mock at the network boundary, return real `Response` objects, test success, HTTP error (`res.ok` false), network error and races; restore mocks after each test. In real projects use MSW.

12.7 Testing custom Hooks

Two approaches

1. **Through a component** that uses the Hook the way the app does — the most realistic, and often all you need.
2. **`renderHook`** from RTL for reusable Hooks with a clear API (a library-style `useCounter`, `useDebounce`): it renders a hidden test component that calls your Hook and exposes the latest return value as `result.current`.

Hooks can't be called outside a component (Chapter 7), which is why both approaches render something.

`renderHook`, `act`, and `rerender`

- `const { result, rerender, unmount } = renderHook(() => useX(args), { initialProps, wrapper })`.
- Read `result.current` **after** each update — it's replaced on every render, so don't keep an old copy of it.
- Wrap calls that update state in **`act(() => ...)`**, because no RTL event helper is doing it for you.

- `rerender(newProps)` re-renders the same Hook instance with new arguments (use `initialProps` + a callback taking props).
- For Hooks that need context, pass `wrapper` exactly as in §12.8.

Fake timers

Time-based Hooks (debounce, throttle, polling, timeouts) should not make tests actually wait. **Fake timers** replace `setTimeout` & co. with versions you advance manually. In `node:test`: `mock.timers.enable({ apis: ['setTimeout'] })` and `mock.timers.tick(ms)`; in Vitest `vi.useFakeTimers()` / `vi.advanceTimersByTime(ms)`; in Jest `jest.useFakeTimers()` / `jest.advanceTimersByTime(ms)`. Advance timers inside `act` so React processes the resulting updates.

`code/06_testing_hooks.jsx`:

```
function useCounter(initial = 0, { min = -Infinity, max = Infinity } = {}) {
  const [count, setCount] = useState(initial);
  const increment = useCallback(() => setCount(c => Math.min(max, c + 1)), [max]);
  const decrement = useCallback(() => setCount(c => Math.max(min, c - 1)), [min]);
  const reset = useCallback(() => setCount(initial), [initial]);
  return { count, increment, decrement, reset };
}
function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}

afterEach(() => { cleanup(); mock.timers.reset(); });

test('useCounter: increments, respects max, resets', () => {
  const { result } = renderHook(() => useCounter(8, { max: 10 }));
  assert.equal(result.current.count, 8);
  act(() => result.current.increment());              // state updates must be wrapped in act()
  act(() => result.current.increment());
  act(() => result.current.increment());              // would be 11, capped at 10
  assert.equal(result.current.count, 10);
  act(() => result.current.reset());
  assert.equal(result.current.count, 8);
});

test('useCounter: rerender with new arguments', () => {
  const { result, rerender } = renderHook(({ max }) => useCounter(0, { max }), { initialProps: { max: 1 } });
  act(() => { result.current.increment(); result.current.increment(); });
  assert.equal(result.current.count, 1);
  rerender({ max: 5 });                               // same Hook instance, new props
  act(() => result.current.increment());
  assert.equal(result.current.count, 2);
});

test('useDebounce: only the last value within the delay gets through', () => {
  mock.timers.enable({ apis: ['setTimeout'] });
  const { result, rerender } = renderHook(({ v }) => useDebounce(v, 300), { initialProps: { v: 'r' } });
  rerender({ v: 're' });
  act(() => mock.timers.tick(200));
  rerender({ v: 'rea' });                             // restarts the 300 ms timer
  act(() => mock.timers.tick(299));
  assert.equal(result.current, 'r');                  // nothing has settled yet
  act(() => mock.timers.tick(1));
  assert.equal(result.current, 'rea');                // 're' was skipped entirely
});

function Stepper() {
  const { count, increment, decrement } = useCounter(0, { min: 0 });
  return <><button onClick={decrement}>-</button><output aria-label="count">{count}</output><button onClick=
{increment}>+</button></>;
}
test('useCounter through a component: cannot go below min', async () => {
  const user = userEvent.setup();
  render(<Stepper />);
  await user.click(screen.getByRole('button', { name: '-' }));
  await user.click(screen.getByRole('button', { name: '+' }));
  await user.click(screen.getByRole('button', { name: '+' }));
  assert.equal(screen.getByLabelText('count').textContent, '2');
});
```

Output:

```
✔ useCounter: increments, respects max, resets (7.892375ms)
✔ useCounter: rerender with new arguments (1.192583ms)
✔ useDebounce: only the last value within the delay gets through (1.022ms)
✔ useCounter through a component: cannot go below min (79.11725ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

The debounce test runs in about 1 ms even though it covers 800 ms of "time".

Common mistakes

- Destructuring once — `const { count } = result.current` — and expecting `count` to update. Read `result.current.count` again after each `act`.
- Calling the Hook directly (`useCounter()` outside a component) → "Invalid hook call".
- Leaving fake timers on after a test; reset them in `afterEach`. Also note that `user-event` waits between actions using timers — with fake timers in Vitest/Jest, create it with `userEvent.setup({ advanceTimers: vi.advanceTimersByTime })` (or `jest.advanceTimersByTime`).
- Importing `renderHook` from `@testing-library/react-hooks` — that separate package is for React 17 and older; since RTL 13.1, `renderHook` is part of `@testing-library/react`.

TL;DR

Test Hooks through a component when you can; otherwise `renderHook` + `act` + `result.current`, `rerender` for new args, fake timers for time.

12.8 Providers, routers and custom render

The problem it solves

Real components depend on providers: context (theme, auth, cart), a router, a TanStack Query client, a Redux store. Rendering such a component alone crashes. Repeating the provider tree in every test is noisy, so projects write one **custom** `render` that wraps the UI in everything the app needs, using RTL's `wrapper` option.

`code/07_providers_and_router.jsx`:

```
export function cartReducer(state, action) {            // pure function: unit-test it directly
  switch (action.type) {
    case 'add': return { ...state, [action.sku]: (state[action.sku] ?? 0) + 1 };
    case 'clear': return {};
    default: throw new Error(`unknown action ${action.type}`);
  }
}
const CartContext = createContext(null);
function CartProvider({ children, initial = {} }) {
  const [cart, dispatch] = useReducer(cartReducer, initial);
  return <CartContext value={{ cart, dispatch }}>{children}</CartContext>;
}
// CartBadge reads the cart; ProductPage reads :sku from the URL and dispatches 'add';
// App = <nav> with <Link>s and <CartBadge/> + <Routes> for "/" and "/p/:sku".

function renderWithProviders(ui, { route = '/', cart = {} } = {}) {
  const Wrapper = ({ children }) => (
    <MemoryRouter initialEntries={[route]}>             {/* a router that keeps history in memory */}
      <CartProvider initial={cart}>{children}</CartProvider>
    </MemoryRouter>
  );
  return { user: userEvent.setup(), ...render(ui, { wrapper: Wrapper }) };
}

test('reducer: plain unit test, no React needed', () => {
  assert.deepEqual(cartReducer({ MUG: 1 }, { type: 'add', sku: 'MUG' }), { MUG: 2 });
  assert.deepEqual(cartReducer({ MUG: 1 }, { type: 'clear' }), {});
  assert.throws(() => cartReducer({}, { type: 'oops' }), /unknown action oops/);
});

test('starts on a given route with a pre-filled cart', () => {
  renderWithProviders(<App />, { route: '/p/LAMP', cart: { MUG: 2 } });
  assert.equal(screen.getByRole('heading').textContent, 'Product LAMP');
  assert.equal(screen.getByLabelText('items in cart').textContent, '2');
});

test('navigates with a link, then adds to cart', async () => {
  const { user } = renderWithProviders(<App />);
  assert.equal(screen.getByRole('heading').textContent, 'Home');
  await user.click(screen.getByRole('link', { name: 'Mug' }));
  assert.equal(screen.getByRole('heading').textContent, 'Product MUG');
  await user.click(screen.getByRole('button', { name: 'Add to cart' }));
  await user.click(screen.getByRole('button', { name: 'Add to cart' }));
  assert.equal(screen.getByLabelText('items in cart').textContent, '2');
});

test('without the provider the component crashes - which is why the wrapper exists', () => {
  const quiet = console.error; console.error = () => {};
  assert.throws(() => render(<CartBadge />), /Cannot destructure property 'cart'/);
  console.error = quiet;
});
```

Output:

```
✔ reducer: plain unit test, no React needed (0.762041ms)
✔ starts on a given route with a pre-filled cart (58.361708ms)
✔ navigates with a link, then adds to cart (45.55225ms)
✔ without the provider the component crashes - which is why the wrapper exists (2.687916ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

How it works

1. `render(ui, { wrapper: Wrapper })` renders `<Wrapper>{ui}</Wrapper>`; `rerender` keeps the wrapper.
2. `MemoryRouter` (from `react-router`) keeps navigation history in memory instead of the address bar; `initialEntries` sets the starting URL. For apps built with data routers (`createBrowserRouter`, loaders/actions), use `createMemoryRouter(routes, { initialEntries })` + `<RouterProvider>` in tests.

3. Options such as `route` and `cart` let each test start from a specific state **through the public API** (props of the providers), not by poking internal state.
4. Put `renderWithProviders` in a shared `test-utils` file and import it instead of RTL's `render`. With Redux Toolkit, create a **fresh store per test** in the wrapper; with TanStack Query, a **fresh** `QueryClient` — shared instances leak state between tests.

Unit tests still have a place

The reducer is a pure function, so it gets a plain, very fast unit test — no rendering. Test pure logic (reducers, formatters, validation schemas) directly, and test **components** through the UI.

> **TL;DR**

One `renderWithProviders` with a `wrapper` (router with `initialEntries`, context, store/query client created fresh per test). Unit-test pure functions directly.

12.9 Setting up Vitest or Jest in a real project

The tests in this chapter run with `node:test` so they need no extra tooling. Real React projects usually use **Vitest** (natural for Vite projects; Jest-compatible API) or **Jest**. The component tests themselves look the same — only imports, `expect` and mock helpers differ. The configurations below are standard setups, **not run in this book**.

Vitest (Vite projects)

```
npm install -D vitest jsdom @testing-library/react @testing-library/dom @testing-library/user-event @testing-library/jest-dom
```

```
// vite.config.js (or vitest.config.js)
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',          // simulated browser DOM (alternative: 'happy-dom')
    globals: true,                 // describe/test/expect/afterEach as globals -> RTL auto-cleanup works
    setupFiles: './src/setupTests.js',
  },
});
```

```
// src/setupTests.js
import '@testing-library/jest-dom/vitest';   // adds toBeInTheDocument(), toBeDisabled(), toHaveTextContent()...
```

```
// package.json
{ "scripts": { "test": "vitest", "test:run": "vitest run", "coverage": "vitest run --coverage" } }
```

A test file (`Counter.test.jsx`):

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Counter } from './Counter';

test('increments', async () => {
  const user = userEvent.setup();
  render(<Counter />);
  await user.click(screen.getByRole('button', { name: /increment/i }));
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

With TypeScript, add `"types": ["vitest/globals", "@testing-library/jest-dom"]` to `tsconfig.json` so `test`/`expect` and the jest-dom matchers are typed. Without `globals: true`, import `test`, `expect`, `afterEach` from `vitest` and call `afterEach(cleanup)` yourself.

Jest

```
npm install -D jest jest-environment-jsdom babel-jest @babel/preset-env @babel/preset-react \
  @testing-library/react @testing-library/dom @testing-library/user-event @testing-library/jest-dom
```

```
// jest.config.js
module.exports = {
  testEnvironment: 'jsdom',                       // needs jest-environment-jsdom (separate since Jest 28)
  setupFilesAfterEnv: ['<rootDir>/jest.setup.js'], // runs after the test framework is installed
  moduleNameMapper: { '\\.(css|less|svg)$': '<rootDir>/test/fileMock.js' },  // stub non-JS imports
};
// jest.setup.js
import '@testing-library/jest-dom';
// babel.config.js
module.exports = { presets: ['@babel/preset-env', ['@babel/preset-react', { runtime: 'automatic' }]] };
```

TypeScript with Jest uses `ts-jest` or Babel's TypeScript preset. Vitest needs none of this because it reuses Vite's transform pipeline — one reason new Vite projects usually choose it.

Useful extras

Tool	For
`@testing-library/jest-dom`	readable DOM matchers
MSW	network mocking (§12.6)
`vitest --coverage` / `jest --coverage`	which lines ran — a guide to untested code, not a goal in itself
`eslint-plugin-testing-library`, `eslint-plugin-jest-dom`	lint rules that catch misuse (e.g. `getBy` inside `waitFor`, missing `await`)
Vitest Browser Mode, Playwright component testing	running component tests in a real browser when jsdom isn't enough (layout, real CSS, canvas)
Playwright / Cypress	end-to-end tests

What jsdom can't do

jsdom implements the DOM and events but **no layout or painting**: sizes and positions are 0, `scrollTop` changes only when you set it (as in Chapter 10's virtualization demo), CSS media queries don't apply, `IntersectionObserver`, `ResizeObserver` and `matchMedia` aren't there by default and must be mocked. Anything that depends on real rendering belongs in browser-based or E2E tests.

> **TL;DR**
>
> Vitest: `environment: 'jsdom'`, `globals: true`, a setup file importing jest-dom. Jest: `jest-environment-jsdom`, Babel/TS transform, `setupFilesAfterEnv`. Same RTL tests either way.

12.10 Good tests, bad tests, and the bigger picture

What makes a good component test

1. **Named after behaviour**: "shows an error when the server returns 404", not "test 3" or "sets error state".
2. **Arrange–act–assert**, one behaviour per test (several `assert`s are fine if they describe one behaviour).
3. **Uses the public interface**: props in, DOM out, user events in between, network mocked at the boundary.
4. **Independent**: no shared DOM, mocks, stores or caches between tests; any order works.
5. **Deterministic**: no real network, no real time, no randomness (or seeded); waits for conditions, never sleeps.
6. **Fails for the right reason**: break the feature and the test goes red with a clear message (§12.6's race test).

Brittle tests to avoid

Brittle	Why	Instead
Asserting on state, props, or Hook internals	breaks on refactor, passes when UI is broken	assert on rendered output
`container.querySelector('.css-1x2y3z')`	class names are implementation details	role/label/text queries
Large **snapshot** tests of whole components	any markup change fails; reviewers approve new snapshots without reading them	explicit assertions on what matters; small inline snapshots at most
Mocking child components	the test no longer checks real integration	render the real tree; mock only the network or truly heavy leaf modules
Asserting that a handler was called with internal arguments	couples to wiring	assert the visible effect of the handler
Testing third-party libraries	not your code	test your usage of them via the UI

Real-world examples

- A checkout form: tests for "button disabled until valid", "shows field errors", "submits the right payload" (checked at the mocked network), "shows server error", plus one E2E test of the full purchase in a real browser.
- A design-system `Dialog`: role `dialog`, labelled by its title, focus moves inside on open, Escape closes, focus returns to the trigger — all testable with RTL + user-event, and each one is also an accessibility requirement (Chapter 14).

In the interview

30-second answer: "I use React Testing Library with Vitest or Jest. Tests render the component, find elements the way users do — by role and accessible name first — interact with user-event, and assert on the DOM, not on state or internals. Async UI is awaited with `findBy` or `waitFor`, never sleeps. The network is mocked at the boundary, ideally with MSW, covering success, errors and races. Hooks are tested through components or `renderHook` with `act`, and components that need providers use a custom render with a wrapper. E2E with Playwright covers the few critical flows."

- *`getBy` vs `queryBy` vs `findBy`?* — `getBy` throws if missing (must exist now), `queryBy` returns `null` (to assert absence), `findBy` returns a promise that retries (for async appearance).
- *Why prefer `getByRole`?* — It matches how users and assistive technology find elements, doubles as an accessibility check, and survives markup refactors.
- *What does `act` do and when do you need it?* — It flushes React updates and effects before assertions; RTL wraps its helpers in it, so you need it only for direct updates such as `renderHook` calls or advancing fake timers.
- *How do you test a component that fetches?* — Mock the network (MSW or a `fetch` spy), `await findBy…` the loaded UI, and test error and loading states too.

Self-check

1. Which query would you use to find a "Save" button? → `screen.getByRole('button', { name: 'Save' })`.
2. How do you assert an error message is **not** shown? → `assert.equal(screen.queryByRole('alert'), null)` (or `expect(...).not.toBeInTheDocument()`).
3. Why is `await new Promise(r => setTimeout(r, 500))` a bad way to wait for data? → It's slow and flaky; wait for the condition with `findBy` / `waitFor`.
4. How do you test a `useDebounce` Hook without waiting? → Fake timers, advanced inside `act`.
5. Why does the custom render create a fresh store/query client per test? → Shared instances leak state between tests and make them order-dependent.
6. Why must `node:test` files call `afterEach(cleanup)`? → RTL auto-cleans only when a global `afterEach` exists (Jest, Vitest with globals).

Chapter 12 TL;DR

Idea	Remember
Philosophy	Test what users see and do; ignore implementation details
Basic test	`render` → `screen` query → interact → assert → `cleanup`
Query priority	Role (+ name) → label → placeholder → text → display value → alt → title → test ID
Variants	`get` now / `query` maybe-absent / `find` wait; `All` for many; `within` to scope
Interactions	`const user = userEvent.setup(); await user.click/type/selectOptions/keyboard/tab(...)`
Async	`findBy`, `waitFor(() => assertion)`, `waitForElementToBeRemoved`; never sleep
Network	Mock at the boundary (MSW, or a `fetch` spy returning `Response`s); cover success, HTTP error, network error, races
Hooks	Through a component, or `renderHook` + `act` + `result.current`; fake timers for time
Providers	Custom render with `wrapper`: `MemoryRouter`, context, fresh store/query client per test
Setup	Vitest (`jsdom`, `globals`, jest-dom setup) or Jest (`jest-environment-jsdom`, Babel/TS)
Quality	Behaviour-named, independent, deterministic, fails for the right reason; few snapshots; E2E for critical paths

Next: Chapter 13 — Server Rendering and Frameworks.

Chapter 13 — Server Rendering and Frameworks

So far React has run only in the browser. This chapter moves it onto the **server**: what the server sends (captured byte by byte), how the browser takes that HTML over (**hydration**), what breaks when the two disagree, how slow parts stream in later, and how **React Server Components** run some components *only* on the server. Then we tour the two frameworks that package all this — **Next.js** 16 (App Router) and **React Router** 8 (framework mode) — and finish with how to choose.

Chapter 1 §1.4 already introduced the vocabulary (MPA, SPA/CSR, SSR, SSG) with a tiny `renderToString` + `hydrateRoot` example. This chapter builds on it rather than repeating it.

What you'll learn

#	Topic	You will be able to…
13.1	Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR	pick when and where each page's HTML is produced
13.2	The `react-dom/server` and `react-dom/static` APIs	choose the right server rendering function
13.3	Streaming SSR with `<Suspense>`	read a streamed response chunk by chunk and explain every marker
13.4	Hydration end to end	serve, stream and hydrate a real page in Chrome
13.5	Hydration mismatches	recognise the error, name the causes and fix them
13.6	Static generation and partial prerendering	prerender at build time and resume the dynamic part later
13.7	React Server Components	explain what runs where, what crosses the boundary and why
13.8	Directives: `'use client'` and `'use server'`	place boundaries correctly and secure Server Functions
13.9	Next.js App Router overview	read and write a basic App Router project
13.10	React Router framework mode	read a route module with loaders and actions
13.11	Choosing a framework	pick Next.js, React Router, Expo, TanStack Start or a plain Vite SPA

Code: `code/` — React 19.3 on Node 24. Files `01`–`05` are runnable. `04_ssr_server_hydrate.jsx` starts a real `node:http` server, loads the page in headless Chrome and takes the screenshots in this chapter. Next.js and React Router snippets are marked **(not run)**: they need the framework's bundler. Background: Suspense and `use()` (Chapter 9), Effects (Chapter 5), `useSyncExternalStore` and `useId` (Chapter 7), Form Actions (Chapter 4). HTTP basics: the HTML & CSS book's How the web works.

13.1 Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR

What it is

A **rendering strategy** answers two questions for each page: *where* is the HTML produced (browser or server) and *when* (at build time, at request time, or some mix)? Chapter 1 covered the first four rows below; this chapter adds the last three.

Strategy	Where / when HTML is made	First HTML contains	Data freshness
CSR (client-side rendering, SPA)	Browser, after JS loads	Empty `<div id="root">`	Live (fetched by the browser)
SSR (server-side rendering)	Server, every request	Full content	Live per request
SSG (static site generation)	Build machine, once per deploy	Full content	As old as the last build
ISR (incremental static regeneration)	Build, then re-built in the background on a timer or on demand	Full content	"Stale for at most N seconds", or until you invalidate
Streaming SSR	Server, per request, sent in pieces	Page frame + loading placeholders first, slow parts later	Live
PPR (partial prerendering)	Static shell at build time + dynamic "holes" filled per request	Static shell with placeholders, holes streamed in	Mixed per section
RSC (React Server Components)	Some components run only on the server (build or request time)	Works *with* any of the above	Per component

Analogy — a restaurant. CSR hands the customer a kit and a recipe. SSR cooks each plate to order. SSG pre-plates sandwiches every morning; ISR re-makes the tray when it's over an hour old. Streaming serves the starter now and the main when ready. PPR keeps pre-plated sides and cooks only the main per order.

Why it matters

It decides how soon something appears (**FCP**, First Contentful Paint), how soon the main content appears (**LCP**, Largest Contentful Paint), when clicks work (hydration done) and how fresh data is. It also decides cost: SSG pages are files on a **CDN** (Content Delivery Network — caching servers near the user); SSR runs your code on every request.

How ISR works

1. The page is generated at build time and cached, like SSG; requests are served from the cache.
2. After the revalidation window (say 60 s), the next request **still gets the stale page** but triggers a background re-render.
3. The new page replaces the cached copy; later visitors get fresh HTML.
4. You can also invalidate on demand (e.g. when an editor publishes).

This is the **stale-while-revalidate** idea (also used by HTTP caches and TanStack Query, Chapter 6 §6.9): serve what you have now, refresh behind the scenes. ISR is a framework feature (Next.js named it). It is not a React API.

Trade-offs

	CSR	SSR	SSG	ISR	Streaming SSR / PPR
Time to first content	Slow	Medium (waits for server render)	Fastest	Fastest	Fast (shell first)
Needs a server	No	Yes	No	Yes (for regeneration)	Yes
Per-user content in HTML	No	Yes	No	No	Yes (in the holes)
Good for	Dashboards behind login	Personalised pages	Docs, blogs	Catalogues, news	Fast frame + slow widgets

Common mistakes

- Thinking SSR makes pages *interactive* sooner. It makes them *visible* sooner; interactivity still waits for JavaScript and hydration (§13.4).
- Using SSG for data that must be correct per user (prices in a cart, "logged in as…"). Put that in a dynamic hole or fetch it on the client.
- Treating strategies as whole-app choices. Modern frameworks choose **per route**, and with PPR even **per component**.

TL;DR

CSR = browser builds it; SSR = server per request; SSG = at build; ISR = SSG plus background refresh; streaming = send the frame now and slow parts later; PPR = static shell plus streamed dynamic holes.

13.2 The server rendering APIs

What it is

React ships two server entry points. `react-dom/server` renders **per request**. `react-dom/static` renders **ahead of time** and waits for all data.

Function	Package	Output	Waits for `<Suspense>` data?	Use it for
`renderToString(el)`	`react-dom/server`	A string	No — emits the fallback	Legacy code; simple SSR of pages without async data
`renderToStaticMarkup(el)`	`react-dom/server`	A string, no hydration markers	No	Non-interactive HTML: emails, static snippets
`renderToPipeableStream(el, opts)`	`react-dom/server` (Node)	Node stream: `{ pipe, abort }`	Streams it later	**SSR on Node** (Express, `node:http`)
`renderToReadableStream(el, opts)`	`react-dom/server` (Web Streams)	`Promise<ReadableStream>`	Streams it later	SSR on edge runtimes, Deno, Bun, Cloudflare Workers
`prerenderToNodeStream(el, opts)`	`react-dom/static`	`{ prelude }` Node stream	**Yes — waits for everything**	SSG on Node
`prerender(el, opts)`	`react-dom/static`	`{ prelude, postponed }` Web stream	Yes (unless aborted)	SSG with Web Streams; partial prerendering
`resume(el, postponed)` / `resumeToPipeableStream`	`react-dom/server`	Web / Node stream	Streams it	Filling the holes left by `prerender` at request time (§13.6)
`resumeAndPrerender` / `…ToNodeStream`	`react-dom/static`	`{ prelude, postponed }`	Yes	Continuing a postponed prerender at build time

On the browser side there is one partner: **`hydrateRoot(domNode, element, options)`** from `react-dom/client`, which attaches React to existing server HTML (instead of `createRoot`, which builds DOM from nothing).

Seeing the string APIs

`code/01_render_to_string.jsx`:

```jsx
function Greeting({ name }) {
  const [likes] = useState(3);                    // state works: initial value is rendered
  return <p className="greet">Hello, {name}! Likes: {likes}</p>;
}
console.log('renderToString      :', renderToString(<Greeting name="Ada" />));
console.log('renderToStaticMarkup:', renderToStaticMarkup(<Greeting name="Ada" />));

const never = new Promise(() => {});              // data that never arrives
function Slow() { return <p>{use(never)}</p>; }
const page = (
  <main>
    <h1>Shop</h1>
    <Suspense fallback={<p>Loading…</p>}><Slow /></Suspense>
  </main>
);
console.log(renderToString(page));                // (output shortened in the file)
```

Output:

```
renderToString      : <p class="greet">Hello, <!-- -->Ada<!-- -->! Likes: <!-- -->3</p>
renderToStaticMarkup: <p class="greet">Hello, Ada! Likes: 3</p>
renderToString + pending Suspense:
  <main><h1>Shop</h1><!--$!--><template data-msg="Switched to client rendering because the server rendering aborted due to: …"></template><p>Loading…</p><!--/$--></main>
```

What each part teaches:

1. **Hooks run once on the server.** `useState` returns its initial value; Effects **never** run there and no event handlers are attached — the server only produces markup.
2. `<!-- -->` **comments** separate adjacent text pieces so hydration can match each text node. `renderToStaticMarkup` drops them, so its HTML can't be hydrated.
3. `renderToString` **cannot wait.** A suspended component makes it emit the fallback marked `<!--$!-->` ("render this boundary on the client"), plus a development-only hint to use a streaming API.

Common mistakes

- `renderToString` in new code — data components only ever show the fallback on the server. Use a streaming API.
- `hydrateRoot` on `renderToStaticMarkup` output — it lacks the markers hydration needs.
- Reading `window`, `document` or `localStorage` during render — `ReferenceError` on the server. Use Effects or handlers (§13.5).

TL;DR

`renderToPipeableStream` (Node) or `renderToReadableStream` (Web Streams) for per-request SSR; `prerender*` for build time; `hydrateRoot` in the browser. The string APIs are legacy and don't support Suspense on the server.

13.3 Streaming SSR with `<Suspense>`

What it is

Streaming sends one HTML response in several pieces. React cuts it at your `<Suspense>` boundaries: everything **outside** pending boundaries is the **shell**, sent first with fallbacks; each boundary's content follows when its data is ready, with a tiny inline script that swaps it in.

The problem it solves

Plain SSR is all or nothing: one slow query (reviews taking 1.5 s) delays the **whole** page. Streaming lets fast parts leave at once, so the browser loads CSS and JS, and even hydrates the shell, while slow parts still render.

How it works

1. `renderToPipeableStream(<App />, options)` starts rendering at once.
2. When everything outside pending boundaries is rendered, React calls `onShellReady`: set status and headers, then `pipe(res)` — the shell goes out with fallbacks.
3. As each boundary's data resolves, React writes a chunk: the content in a **hidden** `<div>` plus a `<script>` that moves it into place and removes the fallback.
4. When all is done, `onAllReady` fires and the response ends.
5. `onShellError` — the shell itself failed (send an error page). `onError` — any error, including inside boundaries (log it). `abort()` — stop waiting; pending boundaries fall back to client rendering.

Seeing every chunk

`code/02_stream_chunks.jsx` pipes into a fake response that prints each chunk and when it arrived. Price data takes 100 ms, reviews take 300 ms.

```
const pricePromise = later('$18', 100);                      // fast data
const reviewsPromise = later(['Great mug', 'Too small'], 300); // slow data
function Price() { return <b>{use(pricePromise)}</b>; }
function Reviews() { return <ul>{use(reviewsPromise).map(r => <li key={r}>{r}</li>)}</ul>; }

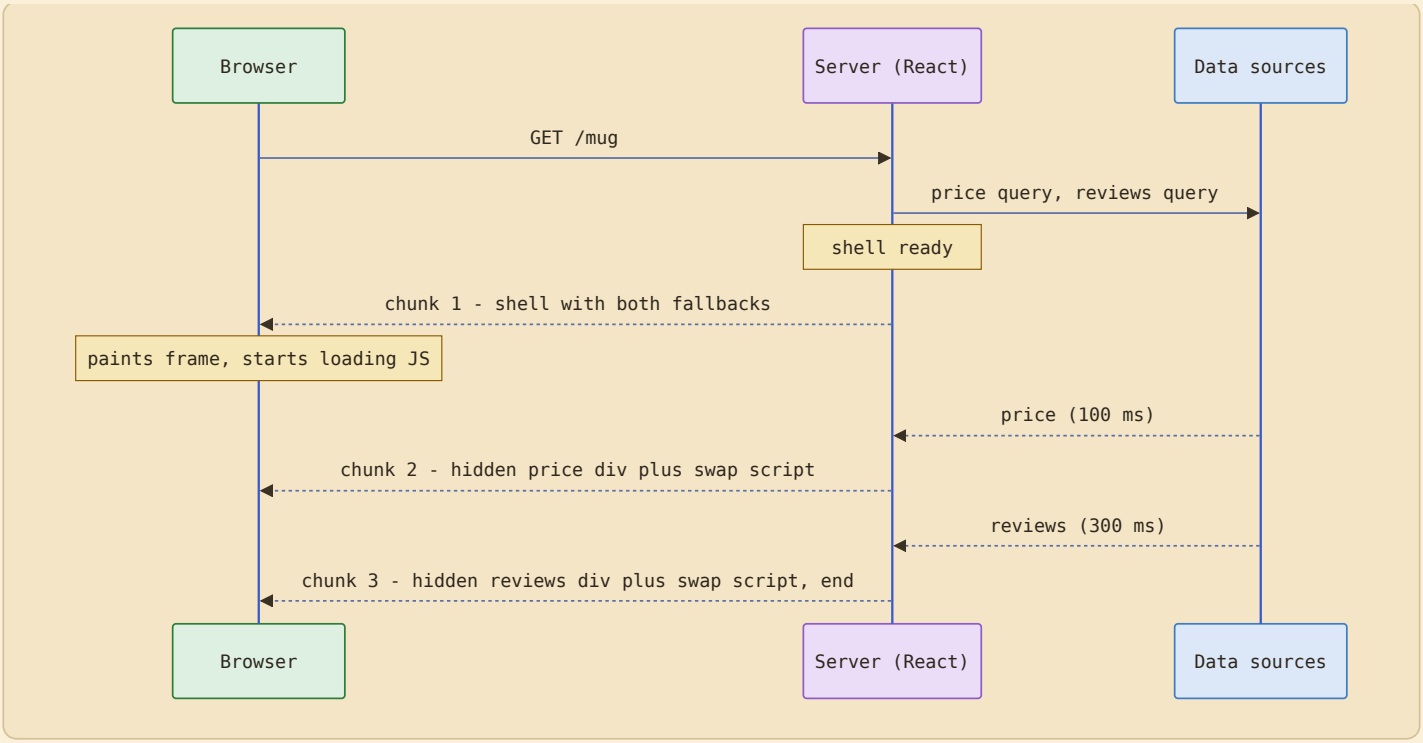
function Page() {
  return (
    <main>
      <h1>Trail Mug</h1>
      <Suspense fallback={<i>price…</i>}><Price /></Suspense>
      <Suspense fallback={<i>reviews…</i>}><Reviews /></Suspense>
    </main>
  );
}
const res = new Writable({ write(chunk, _e, done) { console.log(`chunk ${++n} @ ${ms()}: ${chunk}`); done(); } /* ...
*/ });
const { pipe } = renderToPipeableStream(<Page />, {
  onShellReady() { console.log(`onShellReady @ ${ms()} -> pipe(res)`); pipe(res); },
  onAllReady()   { console.log(`onAllReady   @ ${ms()}`); },
  onError(e)     { console.error(e); },
});
```

Output (long scripts shortened by the program; timings vary by a few ms):

```
onShellReady @    3 ms -> pipe(res)
chunk 1 @    5 ms: <main><h1>Trail Mug</h1><!--$?--><template id="B:0"></template><i>price…</i><!--/$--><!--$?-->
<template id="B:1"></template><i>reviews…</i><!--/$--></main><script>/* 74 chars of React's swap code */</script>
chunk 2 @  101 ms: <div hidden id="S:0"><b>$18</b></div><script>/* 862 chars of React's swap code */</script>
onAllReady   @  302 ms
chunk 3 @  302 ms: <div hidden id="S:1"><ul><li>Great mug</li><li>Too small</li></ul></div><script>$RC("B:1","S:1")
</script>
end      @  302 ms
```

Marker	Meaning
`<!--$?-->` … `<!--/$-->`	A Suspense boundary that is still **pending**; what's between is the fallback
`<template id="B:0">`	Placeholder that marks *where* boundary 0's content will go
`<div hidden id="S:0">`	The finished content ("segment") for boundary 0, sent later and hidden
`$RC("B:1","S:1")`	"React: complete boundary" — moves segment S:1 into B:1's place, removes the fallback
`<!--$-->`	A boundary that was **complete** when sent (no swap needed)
`<!--$!-->`	A boundary that **failed** on the server; the browser renders it instead

Chunk 2 carries the helper that defines `$RC`; chunk 3 only calls it. The shell left at 5 ms instead of waiting 300 ms.



Status codes, errors and crawlers

- The HTTP **status code** can only be set before the first byte — decide it in `onShellReady`. A later error inside a boundary can't change the 200; the boundary is client-rendered instead, and your error boundary shows if it fails again.
- For **crawlers** and static generation, wait for `onAllReady` so the whole document arrives at once.
- Always `abort()` **after a timeout**, or a hung query holds the connection open.
- Since React 19.2 the server briefly **batches reveals** of boundaries that finish close together, so they appear in one go instead of flickering in one by one.

Where to put boundaries

Wrap parts whose data is slow **and** not needed for the first view (reviews, recommendations, charts). Keep the main content (title, price, article text) in the shell, or LCP suffers. Many tiny boundaries make the page "pop" piece by piece; group related slow parts under one boundary.

In the interview

> **30-second answer:** "The shell — everything outside pending Suspense boundaries — is sent as soon as it's ready, with fallbacks. Each boundary's HTML follows in a hidden div plus a tiny swap script, so one slow query no longer blocks the page and the browser can load JS and hydrate the shell meanwhile. Status goes in `onShellReady`, logging in `onError`, crawlers wait for `onAllReady`, and a timeout calls `abort()`."

- *What if a component inside a boundary throws on the server?* — The boundary is marked for client rendering (`<!--$!-->`), the fallback stays, and the browser tries to render it; if it fails there too, the nearest error boundary shows.

Self-check

1. What is the "shell"? → Everything outside pending Suspense boundaries — sent first, with fallbacks.
2. Why can't you return a 404 after the shell is sent? → The status line is the first thing in the response; once bytes are out it's fixed.
3. What does `<div hidden id="S:1">` contain? → The finished HTML of a boundary, hidden until `$RC` moves it into place.

TL;DR

Streaming SSR = shell first, each Suspense boundary later with a swap script. Set status in `onShellReady`, wait for `onAllReady` for bots, always have an abort timeout.

13.4 Hydration end to end

What it is

Hydration is React in the browser "adopting" the server-rendered DOM: it renders your components in memory, checks the result matches the existing HTML, **reuses** those DOM nodes (no re-creation), attaches event handlers and runs Effects. Analogy: the server builds and paints the house; hydration is the electrician. The house is visible all along, but the switches work only after wiring.

How it works

1. The server HTML includes a `<script>` for your client bundle (React adds it for you via the `bootstrapScripts` option).
2. The bundle calls **`hydrateRoot(document, <App {...sameProps} />)`** — the *same* component tree with the *same* data as the server.
3. React hydrates boundary by boundary (**selective hydration**): the shell hydrates while a boundary still streams, a click inside an unhydrated boundary moves it to the front, and each streamed boundary is hydrated when it arrives.

The full demo — server, stream, Chrome

`code/04_ssr_server_hydrate.jsx` bundles the browser entry `ssr_client.jsx` with esbuild, starts a `node:http` server that streams `shop_app.jsx` (reviews take 1.5 s), then drives headless Chrome: screenshot the shell, click, wait for reviews, click again. The shared component (`code/shop_app.jsx`, key parts) renders the **whole document**:

```
function AddToCart() {
  const [count, setCount] = useState(0);
  useEffect(() => { window.__hydrated = true; }, []);   // Effects never run on the server: proves hydration
  return <button onClick={() => setCount(c => c + 1)}>Add to cart ({count})</button>;
}
function Reviews({ reviewsPromise }) {
  const reviews = use(reviewsPromise);                  // suspends until the data is ready
  return <ul>{reviews.map(r => <Review key={r.id} r={r} />)}</ul>;
}
export default function App({ reviewsPromise }) {
  return (
    <html lang="en">
      <head><meta charSet="utf-8" /><title>Trail Mug — Shop</title><style>{css}</style></head>
      <body>
        <header>Hill &amp;amp; Trail Outfitters</header>
        <main>
          <section className="card"><h1>Trail Mug</h1><p>Enamel camping mug, 350 ml. <b>$18</b></p><AddToCart />
</section>
          <section className="card">
            <h2>Reviews</h2>
            <Suspense fallback={<ReviewsSkeleton />}><Reviews reviewsPromise={reviewsPromise} /></Suspense>
          </section>
        </main>
      </body>
    </html>
  );
}
```

The browser entry (`code/ssr_client.jsx`):

```
import { hydrateRoot } from 'react-dom/client';
import App from './shop_app.jsx';
const reviewsPromise = fetch('/api/reviews').then(r => r.json());   // the browser needs the data too
hydrateRoot(document, <App reviewsPromise={reviewsPromise} />, {
  onRecoverableError(err) { console.error('recoverable:', err.message); },
});
```

The server (key part):

```
const server = http.createServer((req, res) => {
  if (req.url === '/client.js')   { res.setHeader('content-type', 'text/javascript'); return res.end(clientJs); }
  if (req.url === '/api/reviews') { res.setHeader('content-type', 'application/json'); return
res.end(JSON.stringify(REVIEWS)); }
  let didError = false;
  const { pipe, abort } = renderToPipeableStream(<App reviewsPromise={slowReviews()} />, {
    bootstrapScripts: ['/client.js'],          // React emits <script src="/client.js" async>
    onShellReady() {
      res.statusCode = didError ? 500 : 200;
      res.setHeader('content-type', 'text/html; charset=utf-8');
      pipe(res);
    },
    onShellError() { res.statusCode = 500; res.end('<h1>Something went wrong</h1>'); },
    onError(err) { didError = true; console.error(err); },
  });
  setTimeout(abort, 10_000);                   // never hang forever on a slow data source
});
// ... then puppeteer: goto (without waiting), wait for #reviews-fallback, click, screenshot, wait for .review, click,
screenshot
```

Output (timings vary run to run):

```
[ 349 ms] server: GET / -> start rendering
[ 354 ms] server: onShellReady -> send shell (with fallback) now
[ 387 ms] browser: shell painted, reviews fallback visible
[ 402 ms] browser: shell hydrated while reviews still loading -> Add to cart (1), fallback still shown: true
[1852 ms] server: onAllReady -> reviews streamed, response ends
[1857 ms] browser: streamed reviews swapped in place of the fallback
[1883 ms] browser: after clicking Helpful -> ["Add to cart (1)","Helpful (5)","Helpful (1)"]
```

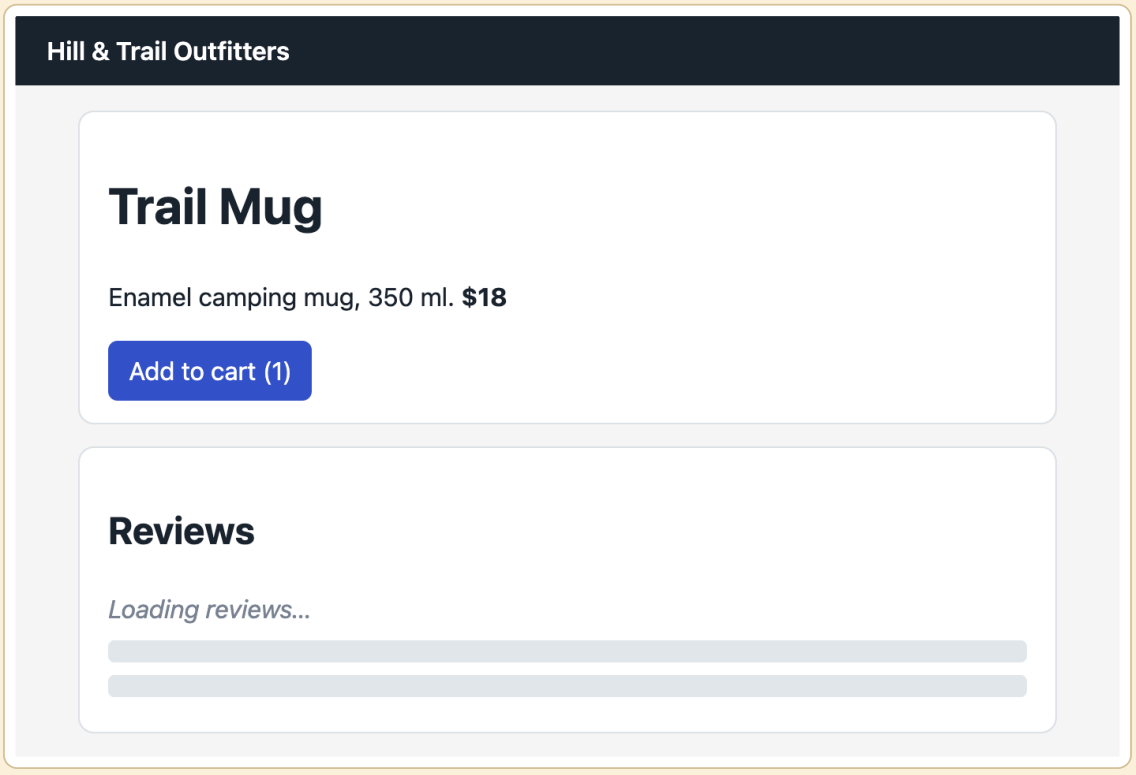


Figure 13.1 — About 40 ms after the request: the shell is painted and hydrated ("Add to cart (1)" responded to a click) while the reviews boundary still shows its fallback.

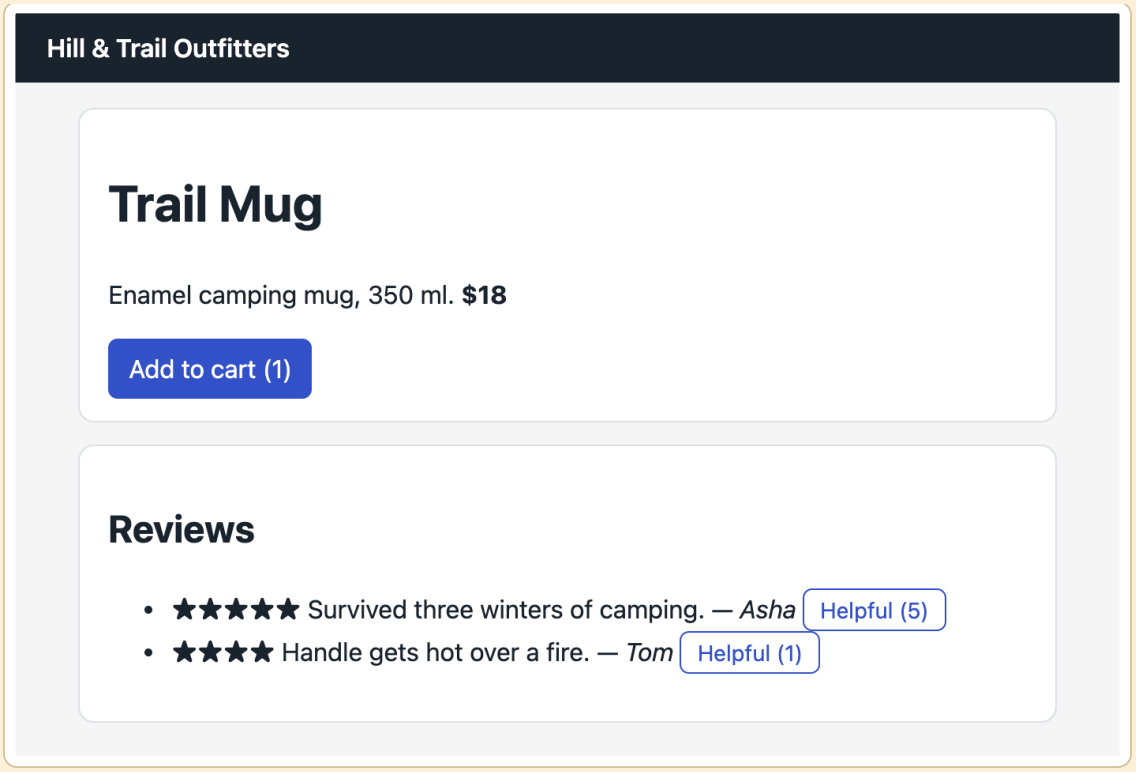


Figure 13.2 — 1.5 s later: the reviews chunk replaced the skeleton; clicking "Helpful" on the first review changed 4 → 5, so the streamed part is hydrated too.

The run proves three things: the shell was **usable 1.4 s before the slow data** (selective hydration), the **streamed part hydrated too** (its button responds), and hydration **reused the server DOM** (no blank page or flicker).

The double-fetch problem (and how frameworks fix it)

In `ssr_client.jsx` the browser **fetches the reviews a second time** to render the same tree during hydration, although the server already had them. Frameworks instead **serialize the server's data into the page** (Next.js embeds the RSC payload, §13.7; React Router embeds loader data). That, plus routing, per-route code splitting and CSS handling, is most of what a framework does — which is why react.dev recommends one for server rendering.

Common mistakes

- `createRoot` instead of `hydrateRoot` on server HTML — React discards the server DOM and rebuilds it (a flash).
- A different tree or props on the client (§13.5).
- One giant `<Suspense>` around everything, so the shell is empty.
- Forgetting the page is **not interactive** until the bundle loads — keep client JS small and use real `<a>` and `<form>` so basics work before hydration.

In the interview

30-second answer: "`hydrateRoot` renders the same tree in the browser, matches it against the server HTML, reuses those nodes and attaches handlers and Effects. With streaming it's selective: the shell becomes interactive while boundaries still load, and a click in an unhydrated boundary gets priority. The client must render exactly what the server did — hence frameworks serialize server data into the page."

- *What happens to clicks between "visible" and "interactive"?* — Before the bundle runs they do nothing, unless the element works without JS (links, forms). Once React is running, a click on a not-yet-hydrated part makes React hydrate it first and then replay the event.
- *Why hydrate `document` instead of a `<div>`?* — Rendering the whole `<html>` lets React manage `<head>` tags too.

Self-check

1. Which Hook proved hydration in the demo, and why does it work? → `useEffect` — Effects never run on the server, only after hydration.
2. Why did "Add to cart" work before the reviews arrived? → Selective hydration: the shell hydrates independently of pending boundaries.

3. What is the double-fetch problem? → The client refetches data the server already used; frameworks embed it in the page instead.

> **TL;DR**

Server streams HTML → browser paints → bundle calls `hydrateRoot` with the same tree and data → React adopts the DOM boundary by boundary. Visible ≠ interactive.

13.5 Hydration mismatches

> **What it is**

A **hydration mismatch** happens when the HTML React renders in the browser during hydration differs from the HTML the server sent. React can't safely "adopt" DOM that doesn't match, so it throws away that part and **re-renders it on the client** — costing performance, causing a visible flash, and logging an error.

Seeing one

`code/05_hydration_mismatch.jsx` renders on a pretend "server" (no `window`), then hydrates in jsdom:

```
function BadBanner() {                      // BUG: output depends on the environment
  return <p>{isBrowser() ? 'Welcome back!' : 'Hello, guest'}</p>;
}
function GoodBanner() {                     // FIX 1: same first render, switch in an Effect
  const [mounted, setMounted] = useState(false);
  useEffect(() => setMounted(true), []);
  return <p>{mounted && isBrowser() ? 'Welcome back!' : 'Hello, guest'}</p>;
}
const subscribe = () => () => {};
function GoodBanner2() {                    // FIX 2: useSyncExternalStore with a server snapshot
  const inBrowser = useSyncExternalStore(subscribe, () => isBrowser(), () => false);
  return <p>{inBrowser ? 'Welcome back!' : 'Hello, guest'}</p>;
}
function Greeting() { use(browser()); return <p>Welcome back!</p>; }   // FIX 3 (React 19.3)
function GoodBanner3() { return <Suspense fallback={<p>Hello, guest</p>}><Greeting /></Suspense>; }
// each banner: renderToString on the "server", put the HTML in the DOM, flip to "browser",
// hydrateRoot(..., { onRecoverableError }) and print the result
function Stamp({ now }) { return <time suppressHydrationWarning>{now}</time>; }
```

Output:

```
BadBanner    server: <p>Hello, guest</p>          after hydrate: <p>Welcome back!</p>
   recoverable error: Hydration failed because the server rendered text didn't match the client. As a result this tree
will be regenerated on the client. This can happen if a SSR-ed Client Component used:
GoodBanner   server: <p>Hello, guest</p>          after hydrate: <p>Welcome back!</p>
GoodBanner2  server: <p>Hello, guest</p>          after hydrate: <p>Welcome back!</p>
GoodBanner3  server: <!--$!--><template…><p>Hello, guest</p><!--/$-->
             after hydrate: <p>Welcome back!</p>  errors: 0
Stamp        server: <time>10:00:00</time>        after hydrate: <time>10:00:00</time>  errors: 0
```

Reading it:

1. `BadBanner` ends right only because React **gave up and re-rendered** the tree on the client, reported via `onRecoverableError` (development also logs a server/client diff).
2. Fixes 1 and 2 end in the same place **with no error**: hydration reproduces the server output, then a normal update changes it. `useSyncExternalStore` uses its third argument (`getServerSnapshot`) on the server *and during hydration*.
3. `use(browser())` (React 19.3, from `react-dom`) suspends **only on the server**: the server sends the nearest fallback marked "render on the client" (`<!--$!-->`) and the browser renders the real content — no mismatch, no error.
4. `suppressHydrationWarning` silences the warning for **one element's own text/attributes**, and React **keeps the server text** (`10:00:00`) — it does not patch it. Use it only for values that are expected to differ and are fine to be slightly stale, like a timestamp.

Common causes and fixes

Cause	Example	Fix
Environment checks in render	`typeof window !== 'undefined' ? A : B`	Render the server version first; switch in `useEffect` or with `useSyncExternalStore`
Browser-only data	`localStorage.getItem('theme')` during render	Same; or store the value in a cookie so the server knows it
Time, randomness, locale	`Date.now()`, `Math.random()`, `toLocaleDateString()` → `9/28/2026` vs `28/09/2026`	Compute once on the server and pass as a prop; give `Intl` an explicit locale and `timeZone`; `useId` for IDs (Chapter 7 §7.8)
Invalid HTML nesting	`<p><div/></p>`, `<a>` inside `<a>`, `<table>` without `<tbody>`	Fix the markup — the browser's parser "repairs" it, so the DOM no longer matches
Different data	Client refetched and got newer data	Serialize server data into the page (frameworks do this)
Browser extensions / proxies	An extension injects attributes; a CDN minifier strips comments	React 19 tolerates unexpected tags inserted in `<head>`/`<body>`; don't let proxies rewrite HTML

When the difference is intentional

Sometimes a component genuinely must show something different in the browser (a "you are offline" badge, a chart that needs the window size). Use **two-pass rendering** (render a neutral placeholder on the server and during hydration, then the real thing after mount — Fix 1 or 2), or, for a whole browser-only component, `use(browser())` inside `<Suspense>` (Fix 3). Next.js also offers `dynamic(() => import('./Chart'), { ssr: false })` inside a Client Component.

In the interview

30-second answer: "The client's first render didn't produce the server's HTML, so React can't adopt that DOM: it client-renders the tree and reports a recoverable error. Causes: environment checks, `localStorage`, dates, randomness, locale, invalid nesting, different data. Fix: make the first client render identical and switch after mount (an Effect, `useSyncExternalStore`'s server snapshot, or `use(browser())` in 19.3); `suppressHydrationWarning` only for single expected differences like timestamps."

- *Does `suppressHydrationWarning` fix the text?* — No, it keeps the server's text and silences the warning, one level deep only.
- *Why is `useId` safe for SSR?* — It derives IDs from the component's position in the tree, which is the same on server and client.

Self-check

1. Why does `GoodBanner` not mismatch? → Its first client render uses `mounted = false`, same as the server.
2. What does `getServerSnapshot` do during hydration? → Supplies the value used on the server so the first client render matches.
3. Why can `<p><div>x</div></p>` cause a mismatch? → The HTML parser closes the `<p>` before `<div>`, so the real DOM differs from React's tree.

TL;DR

First client render must equal the server render. Move environment-dependent output behind an Effect, `useSyncExternalStore` or `use(browser())`; `suppressHydrationWarning` only for single expected differences.

13.6 Static generation and partial prerendering

What it is

Prerendering means rendering ahead of any request (at build time). `react-dom/static`'s `prerenderToNodeStream` and `prerender` **wait for all Suspense data** and return the finished HTML as `prelude`.

Partial prerendering (PPR) prerenders what it can, **stops** at per-request data (cart, cookies), saves a small "where I stopped" object (`postponed`), and at request time `resume()` renders only the holes. The static part sits on a CDN; the server does only the dynamic work. `resume` arrived in React 19.2.

Seeing both

`code/03_prerender_ssg.jsx`:

```
// 1. SSG
const { prelude } = await prerenderToNodeStream(<Blog />);   // resolves only when everything is ready
console.log(await text(prelude));

// 2. PPR
let cart = new Promise(() => {});                // per-user data: not available at build time
function Cart() { return <p>Cart: {use(cart)}</p>; }
function Store() {
  return (
    <main>
      <h1>Store</h1><p>Free shipping over $50</p>
      <Suspense fallback={<p>Cart: …</p>}><Cart /></Suspense>
    </main>
  );
}
const controller = new AbortController();
setTimeout(() => controller.abort(), 50);        // "build" stops waiting after 50 ms
const pre = await prerender(<Store />, { signal: controller.signal, onError() {} });
const saved = JSON.stringify(pre.postponed);     // store next to the prelude

cart = Promise.resolve('3 items');               // later, a real request: data is available
const rest = await resume(<Store />, JSON.parse(saved));
```

Output:

```
SSG (prerenderToNodeStream):
  <main><h1>Blog</h1><!--$--><ul><li>Packing list for a weekend hike</li><li>Choosing a camping stove</li></ul><!--/$--
></main>

PPR prelude (cache on a CDN):
  <main><h1>Store</h1><p>Free shipping over $50</p><!--$?--><template id="B:0"></template><p>Cart: …</p><!--/$-->
</main><script>…</script>
postponed state: 610 bytes of JSON

resume() stream appended after the prelude:
  <div hidden id="S:0"><p>Cart: <!-- -->3 items</p></div><script>/* swap code */</script>
```

What happened:

1. **SSG**: the boundary is `<!--$-->` (complete) — no fallback, no swap script; ready for a CDN.
2. **PPR**: the abort left the cart boundary pending, so the prelude is a shell with a fallback and `postponed` records what's missing.
3. `resume` rendered **only** that boundary. At request time the server sends the cached prelude at once, then appends the resumed stream (the §13.3 hidden-div-plus-swap mechanism).

Where you meet this in practice

You rarely call these APIs yourself. Next.js 16 with **Cache Components** uses exactly this model by default: static and `'use cache'` parts form the prerendered shell, and anything reading cookies, headers or uncached data must sit inside `<Suspense>` and streams at request time (§13.9). Static-site setups (React Router `prerender`, Astro, custom scripts) call the `prerender*` functions at build time.

Common mistakes

- Prerendering per-user data into a static page — every visitor sees the first user's cart. Keep it in a dynamic hole.
- Expecting `prerender` to be fast for pages with slow data: it waits. Use an abort signal and PPR, or move slow data behind a boundary rendered at request time.

TL;DR

`prerender*` waits for everything → static HTML. Abort it early → `prelude` + `postponed`; `resume()` fills the holes per request. That's PPR, and Next.js Cache Components is built on it.

13.7 React Server Components

What it is

A **React Server Component (RSC)** runs **only** on the server — at build time or per request — and its code never ships to the browser. It can be `async`, `await` a database query, read files and use secrets; only its *output* (the elements it returns) is sent. A **Client Component** is the kind you've written all book: its code ships, it can use state, Effects and handlers, and SSR still pre-renders it to HTML. In an RSC app (like the Next.js App Router) components are **Server Components by default**; `'use client'` opts into client code (§13.8).

Analogy: Server Components are the cooks in the kitchen — they use the fridge (database) and knives (secrets), and only the finished plate goes out. Client Components are the salt shaker on the table, which the diner handles directly.

RSC is not SSR

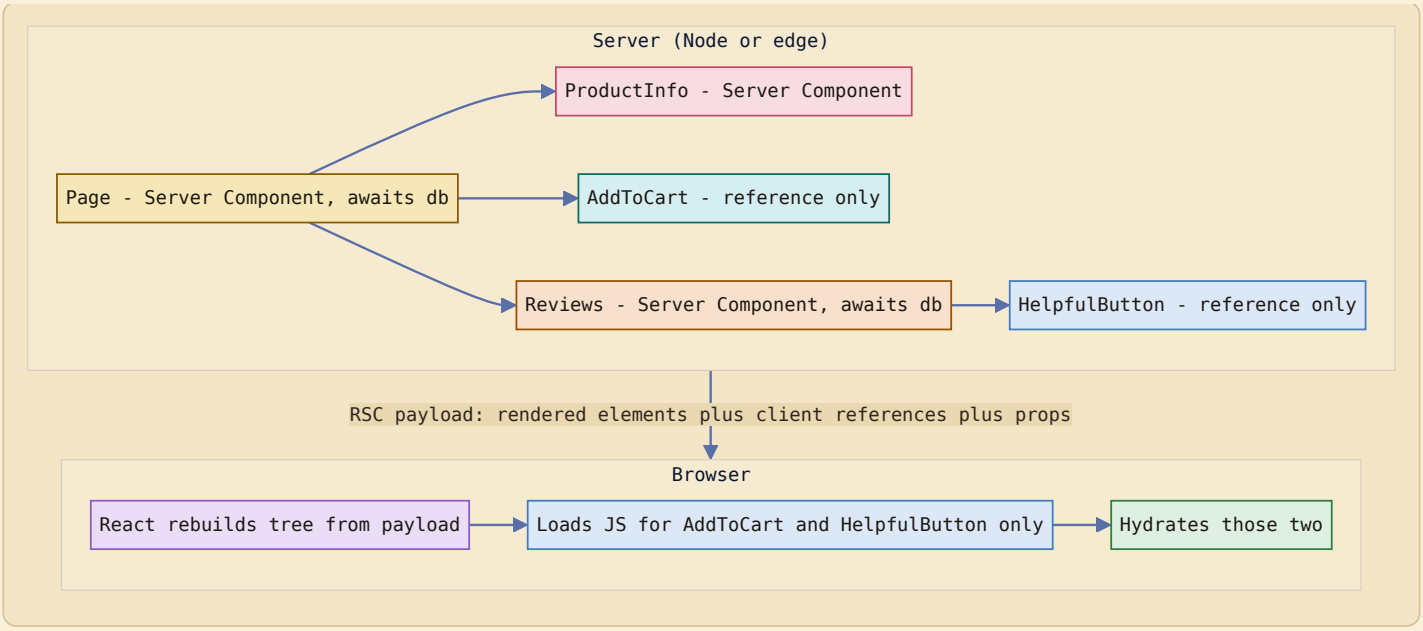
This is the most common confusion.

	SSR	Server Components
What it produces	**HTML** for the first page load	A serialized **component tree** (the RSC payload)
Which components	All of them (client ones included)	Only Server Components
Code shipped to browser	All component code (for hydration)	None for Server Components
Runs again on navigation?	No — after load, navigation is client-side	Yes — the server sends a new payload for the new route
Can use state/Effects?	Yes (initial render only on the server)	No
Can `await` data in render?	Via Suspense + `use()`	Yes, directly (`async function`)

They are **complementary**: Next.js runs Server Components to get the payload, then SSR turns payload + Client Components into first-load HTML.

How it works

1. The **server** renders the Server Component tree (awaiting their data). At a Client Component React does **not** run it; it records a **reference** ("module `LikeButton.js`, export `default`") plus its props.
2. The result is serialized into the **RSC payload**: rendered elements, client references and their props.
3. On the **first load**, SSR turns payload + Client Components into HTML and embeds the payload; the browser hydrates only the Client Components.
4. On **client navigation**, the browser fetches the new route's payload (no HTML) and React merges it into the tree — **client state is preserved** where the tree is unchanged.



Conceptually the payload is already-rendered elements (`<main><h1>Trail Mug</h1>…<ul>…reviews…</ul>`) plus holes like *client ref* `AddToCart.js#default`, *props* `{ productId: "mug-1" }`; the real wire format is internal and changes between versions.

Why it matters

- **Smaller bundles** — a Server Component using a 200 KB Markdown parser ships 0 KB of it, only the resulting elements.
- **No client-server waterfalls** — instead of "load JS → Effect → fetch → render", the component awaits data where it lives.
- **Safe data access** — database queries and API keys stay on the server.
- **Streaming built in** — async Server Components suspend, so `<Suspense>` streams as in §13.3.

Code (Next.js 16 — not run)

```
// app/products/[id]/page.jsx — a Server Component (no directive needed)
import { db } from '@/lib/db';
import AddToCart from './AddToCart';        // a Client Component
import { Suspense } from 'react';

export default async function ProductPage({ params }) {
  const { id } = await params;                          // Next.js 15+: params is a Promise
  const product = await db.product.findUnique({ where: { id } });   // direct DB access
  return (
    <main>
      <h1>{product.name}</h1>
      <p>{product.description}</p>
      <AddToCart productId={product.id} price={product.price} />   {/* serializable props only */}
      <Suspense fallback={<p>Loading reviews…</p>}>
        <Reviews productId={id} />                    {/* async Server Component, streams */}
      </Suspense>
    </main>
  );
}

async function Reviews({ productId }) {
  const reviews = await db.review.findMany({ where: { productId } });
  return <ul>{reviews.map(r => <li key={r.id}>{r.text}</li>)}</ul>;
}
```

`AddToCart.jsx` starts with `'use client'` and is an ordinary component with `useState` and `onClick`.

What can cross the server → client boundary

Props passed from a Server Component to a Client Component must be **serializable** by React:

Allowed	Not allowed
string, number, bigint, boolean, `null`, `undefined`	Ordinary functions (event handlers, callbacks)
Plain objects and arrays of allowed values	Class instances (e.g. an ORM model object, a `class User`)
`Date`, `Map`, `Set`, typed arrays, `ArrayBuffer`	Objects with a `null` prototype
JSX / React elements (including Server Components as `children`)	Symbols not created with `Symbol.for`
Promises (read with `use()` on the client)	
Server Functions (`'use server'`, §13.8)	

Composition patterns

1. **Push `'use client'` to the leaves.** Keep pages and layouts as Server Components; make only the interactive pieces (buttons, forms, menus) Client Components. The directive marks a module *and everything it imports* as client code, so putting it high up ships the whole subtree.
2. **Pass Server Components *through* Client Components as `children`.** A Client Component cannot *import* a Server Component, but it can *render* one it receives as a prop — the server renders it first and passes the result.

```jsx
// Modal.jsx — a Client Component (needs state)
'use client';
import { useState } from 'react';
export default function Modal({ children }) {
  const [open, setOpen] = useState(false);
  return (
    <>
      <button onClick={() => setOpen(!open)}>Cart</button>
      {open && children}
    </>
  );
}
```

```jsx
// page.jsx — a Server Component
import Modal from './Modal';
import Cart from './Cart';                  // a Server Component that awaits the database
export default function Page() {
  return <Modal><Cart /></Modal>;           // Cart runs on the server; Modal receives its output
}
```

3. **Context is created and read in Client Components.** Server Components can't call `createContext`, `useContext` or `use(Context)`. Define the context in a `'use client'` file; since React 19.3 a Server Component may *render* it directly (`<UserContext value={user}>{children}</UserContext>`) — before 19.3 you needed a `'use client'` provider component wrapping `children`.
4. **Stream data to the client with a promise.** A Server Component can start a fetch *without awaiting* and pass the promise as a prop; the Client Component calls `use(promise)` inside `<Suspense>` (Chapter 9).
5. **Keep server code out of client bundles.** `import 'server-only'` at the top of a data module makes the build fail if a Client Component imports it.

When not to use RSC

- Highly interactive screens (editors, canvases, live-filter dashboards) where nearly everything is client state anyway.
- Pure static hosting — RSC can run at build time, but there is no per-request rendering.
- Without a framework: RSC needs deep bundler integration, and React's low-level RSC bundler APIs are not covered by semver. Next.js App Router is the complete implementation; React Router and TanStack Start support is still experimental.

Common mistakes

- Thinking `'use server'` marks a Server Component. It doesn't — there is **no directive** for Server Components; `'use server'` marks Server Functions.
- Passing an `onClick` from a Server Component to a Client Component — functions aren't serializable (unless they're Server Functions).

- Calling `useState`/`useEffect` in a component that runs as a Server Component (no `'use client'` above it) — the build errors.
- Assuming Server Components re-run when client state changes. They re-run only on a new server request (navigation, `router.refresh()`, or revalidation after a Server Function).

In the interview

> **30-second answer:** "Server Components run only on the server, can be async, read data directly and ship zero JS — the browser gets their output as an RSC payload. Client Components (`'use client'`) ship JS and handle state and events. SSR is different: it turns the tree into first-load HTML; RSC decides which components exist in the browser at all. Together: RSC payload → SSR HTML → hydrate only Client Components. Props crossing must be serializable; functions cross only as Server Functions."

- *Can a Client Component render a Server Component?* — Not by importing it, but yes if it receives it as `children` or another JSX prop from a Server Component.
- *How do several Server Components share one query without repeating it?* — Wrap the fetcher in React's `cache()` (per-request memoization, Server Components only).

Self-check

1. Can a Server Component use `useState`? → No — no state, Effects or event handlers; only Client Components ship code.
2. Why can't you pass `onClick={() => …}` from server to client? → Functions aren't serializable; only Server Functions can cross.
3. How does `<Modal><Cart /></Modal>` keep `Cart` on the server? → The Server Component page renders `Cart` and passes the result as `children`.

TL;DR

Server Components = server-only, async, zero JS, output sent as a payload. Client Components = `'use client'`, interactive. Default to server, push client to the leaves, pass server output as `children`, only serializable props cross.

13.8 Directives: `'use client'` and `'use server'`

What it is

A **directive** is a string literal at the very top of a file (or function body), like `'use strict'`. Bundlers that support RSC read them to decide what goes where. React defines exactly two:

Directive	Where	Marks	Meaning
`'use client'`	Top of a **file** (before imports; comments allowed)	That module **and its imports**	"This is where client code starts" — the boundary from server to client
`'use server'`	Top of a **file**, or first line of an **async function** body	Exported async functions (file), or that one function	"This is a **Server Function**: callable from the client, runs on the server"

`'use cache'` (and its variants) you'll see in Next.js is a **Next.js directive**, not a React one (§13.9).

`'use client'` rules

1. It must be the first statement in the file, in single or double quotes (not backticks).
2. It marks a **module-graph boundary**: the file and everything it imports becomes client code. You don't need it in every client file — only at the entry points that Server Components import.
3. Exports that Server Components use must be components or serializable values.
4. A directive-less `Button.jsx` is client code when imported from a `'use client'` file and server code when imported from a Server Component.
5. Libraries put `'use client'` on Hook-using entry points so Server Components can import them.

`'use server'` — Server Functions

A **Server Function** is an async function that runs on the server but is **called from client code** as if it were local: the bundler swaps it for a reference, and a call POSTs the serialized arguments, runs the function and returns the serialized result. Used as a `<form action>` or with `useActionState` it is often called a **Server Action** (React's name for all of them until September 2024; Next.js still uses it). Chapter 4 §4.8 covered the form side.

```
// app/actions.js — (Next.js 16 — not run)
'use server';
import { revalidatePath } from 'next/cache';
import { auth } from '@/lib/auth';
import { db } from '@/lib/db';

export async function addReview(prevState, formData) {
  const user = await auth();                                  // 1. authenticate
  if (!user) return { error: 'Please sign in' };
  const text = String(formData.get('text') ?? '').trim();     // 2. validate — never trust input
  if (text.length < 5) return { error: 'Review is too short' };
  await db.review.create({ data: { text, userId: user.id, productId: formData.get('productId') } });
  revalidatePath(`/products/${formData.get('productId')}`);   // 3. refresh server-rendered UI
  return { ok: true };
}
```

```
// Inline Server Function inside a Server Component (not run)
export default async function Page() {
  async function subscribe(formData) {
    'use server';
    await db.newsletter.add(formData.get('email'));
  }
  return <form action={subscribe}><input name="email" type="email" /><button>Subscribe</button></form>;
}
```

Calling one outside a form (from a Client Component): wrap it in a transition so you get a pending state — `startTransition(async () => { await like(postId); })` (Chapter 7 §7.10).

Security — treat every Server Function as a public API endpoint

- Anyone can call it with any arguments (the reference is in the page). **Authenticate and authorise inside**; a hidden button is not protection.
- **Validate** every argument (e.g. Zod, Chapter 4 §4.9).
- Values an inline Server Function captures from its component travel to the client and back (Next.js encrypts them) — never close over secrets, and re-check anything important.
- Arguments and return values must be serializable (the §13.7 table, plus `FormData`).

Common mistakes

- Putting `'use client'` on every file "to be safe" — you lose all RSC benefits.
- `'use server'` on a file of private helpers — every exported async function becomes a public endpoint. Use `import 'server-only'` instead.
- Server Functions for **reading** data — they're designed for mutations; read in Server Components or loaders.

In the interview

30-second answer: "`'use client'` marks where the client module graph starts — that file and its imports ship to the browser. `'use server'` marks Server Functions: async functions that run on the server but are callable from the client, typically as form actions (with progressive enhancement). Server Components have no directive — they're the default. Each Server Function is a public POST endpoint: authenticate, authorise and validate inside."

- *Does `'use client'` mean the component doesn't render on the server?* — No. Client Components are still pre-rendered to HTML by SSR; the directive means their code ships and they hydrate.
- *Why wrap a Server Function call in `startTransition`?* — To get `isPending` and keep the UI responsive; Actions are transitions.

TL;DR

`'use client'` = start of client code (file + imports). `'use server'` = Server Function (callable from the client, runs on the server). No directive for Server Components. Server Functions are public endpoints: auth + validate.

13.9 Next.js App Router overview

What it is

Next.js (by Vercel) is the framework react.dev lists first and the most complete Server Components implementation. Its **App Router** (`app/`, since Next.js 13) is built on RSC, streaming and Server Functions; the older **Pages Router** (`pages/`, `getServerSideProps`) is supported but gets no new features. This section describes **Next.js 16** (current major since October 2025; checked at 16.3; Node.js 20.9+). `npx create-next-app@latest` sets up TypeScript, Tailwind, ESLint, the App Router and Turbopack.

File-system routing

Folders define URL segments; special file names define what each segment renders.

```
app/
├── layout.jsx            ← root layout: <html><body>; layouts persist (keep state) across navigations
├── page.jsx              ← "/" — a page file makes the path public
├── loading.jsx           ← auto-wraps the page in <Suspense fallback={<Loading/>}>
├── error.jsx             ← error boundary for the segment ('use client'; gets error + retry)
├── not-found.jsx         ← UI for notFound()
├── products/
│   ├── page.jsx          ← "/products"
│   └── [id]/             ← dynamic segment
│       ├── page.jsx      ← "/products/42"   (params.id = "42")
│       └── AddToCart.jsx ← not a route: only page/route files are public
├── (marketing)/          ← route group: organises files, not part of the URL
│   └── about/page.jsx    ← "/about"
├── api/health/route.js   ← Route Handler: GET/POST functions returning Response
└── _components/          ← private folder (underscore): never routed
proxy.js                  ← runs before routes (was middleware.js before v16)
next.config.js
```

Also: `template.jsx` is a layout that re-mounts on every navigation; `[...slug]` and `[[...slug]]` are catch-all and optional catch-all segments.

Pages, layouts and data (not run)

```jsx
// app/layout.jsx
export const metadata = { title: 'Hill & Trail', description: 'Outdoor gear' };  // <head> tags
export default function RootLayout({ children }) {
  return <html lang="en"><body><nav><a href="/">Home</a></nav>{children}</body></html>;
}

// app/products/page.jsx — Server Component fetching data
import Link from 'next/link';
export default async function Products({ searchParams }) {
  const { q = '' } = await searchParams;                    // Promise in Next.js 15+
  const products = await getProducts(q);                    // DB or fetch — on the server
  return (
    <ul>
      {products.map(p => <li key={p.id}><Link href={`/products/${p.id}`}>{p.name}</Link></li>)}
    </ul>
  );
}
```

Per-page `<head>` data comes from `export async function generateMetadata({ params })`, and `notFound()` (from `next/navigation`) renders `not-found.jsx`.

- `<Link>` from `next/link` does client-side navigation and **prefetches** routes that come into view.

- `useRouter`, `usePathname`, `useSearchParams` come from `next/navigation` (Client Components only). `redirect()` and `notFound()` work on the server.
- **Mutations** use Server Functions (§13.8) and then `revalidatePath('/products')` or tag-based `revalidateTag('products', 'max')` so server-rendered pages show fresh data. Next.js 16 added `updateTag(tag)` (read-your-own-writes inside a Server Action) and `refresh()`.

Rendering and caching in Next.js 16

Next.js 16 has two caching models:

1. **Previous model (default unless you opt in).** A route is static (prerendered at build) unless it uses request data (`cookies()`, `headers()`, `searchParams`) or uncached fetches. ISR is configured per route:

```
// app/blog/[id]/page.jsx (not run)
export const revalidate = 60;                      // ISR: regenerate at most every 60 s
export async function generateStaticParams() {     // which [id]s to build ahead of time
  const posts = await getPosts();
  return posts.map(p => ({ id: String(p.id) }));
}
```

2. **Cache Components** (`cacheComponents: true` in `next.config.js`, new in 16). Everything runs at request time **unless** you cache it with the `'use cache'` directive. The build prerenders a **static shell** — static parts, cached parts and `<Suspense>` fallbacks — and dynamic parts stream per request. That's partial prerendering (§13.6), and it's the default behaviour once Cache Components is on.

```
// app/blog/page.jsx — with cacheComponents: true (not run)
import { Suspense } from 'react';
import { cookies } from 'next/headers';
import { cacheLife, cacheTag } from 'next/cache';

export default function BlogPage() {
  return (
    <>
      <h1>Our Blog</h1>                                {/* static: in the shell */}
      <Posts />                                        {/* cached: in the shell */}
      <Suspense fallback={<p>Loading your preferences…</p>}>
        <Preferences />                                {/* per request: streamed */}
      </Suspense>
    </>
  );
}
async function Posts() {
  'use cache';
  cacheLife('hours');                                  // how long the cached result lives
  cacheTag('posts');                                   // invalidate later with revalidateTag / updateTag
  const posts = await db.post.findMany();
  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
async function Preferences() {
  const theme = (await cookies()).get('theme')?.value ?? 'light';   // request data → must be inside Suspense
  return <p>Theme: {theme}</p>;
}
```

With Cache Components on, reading request or uncached data **outside** `<Suspense>` is reported in development and at build, because the route could no longer produce a static shell.

Other pieces worth knowing

Feature	What it does
`proxy.js` (was `middleware.js`)	Redirects, rewrites and auth checks before a route runs
`next/image`, `next/font`	Image resizing/lazy-loading; self-hosted fonts without layout shift
Parallel & intercepting routes (`@slot`, `(.)photo`)	Several pages in one layout; modals that have their own URL
Static export (`output: 'export'`)	Build a plain static site (no server features)

Common mistakes

- Adding `'use client'` to `layout.jsx` or `page.jsx` to use one Hook — the whole subtree becomes client code. Extract the interactive bit instead.
- Forgetting `await params` / `await searchParams` / `await cookies()` — they're Promises since Next.js 15 (sync access removed in 16).
- Expecting data to update after a mutation without revalidating (`revalidatePath` / `revalidateTag` / `updateTag`).
- Following Next.js 13–14 tutorials on implicit fetch caching — 15 changed the defaults and 16's Cache Components makes caching opt-in with `'use cache'`.

TL;DR

App Router = folders are routes, `page` / `layout` / `loading` / `error` files, Server Components by default, Server Functions for mutations, revalidate to refresh. Next.js 16: `proxy.js`, Turbopack default, and opt-in Cache Components with `'use cache'` + PPR.

13.10 React Router framework mode

What it is

React Router (maintained by Shopify; Remix's successor) has three modes: **declarative**, **data** (loaders/actions, Chapter 9) and **framework**. Framework mode adds a **Vite plugin** for SSR, prerendering, per-route code splitting, typed route modules and deployment adapters. **React Router 8** (June 2026) needs React 19.2.7+, Node 22.22+ and Vite 7+, is ESM-only, and drops the `react-router-dom` package (import from `react-router` and `react-router/dom`). Start with `npx create-react-router@latest`; Chapter 9 §9.7 gave the introduction, so here we focus on server rendering.

Structure (not run)

```ts
// app/routes.ts — the route config
import { type RouteConfig, index, route, layout } from '@react-router/dev/routes';
export default [
  index('routes/home.tsx'),
  layout('routes/shop-layout.tsx', [
    route('products/:id', 'routes/product.tsx'),
  ]),
] satisfies RouteConfig;
```

```tsx
// app/routes/product.tsx — a route module
import { Form, useNavigation } from 'react-router';
import type { Route } from './+types/product';        // generated types for this route

export async function loader({ params }: Route.LoaderArgs) {      // runs on the server (SSR / prerender)
  const product = await db.product.find(params.id);
  if (!product) throw new Response('Not found', { status: 404 });
  return { product };
}

export async function action({ request, params }: Route.ActionArgs) { // handles <Form method="post">
  const form = await request.formData();
  await db.review.create({ productId: params.id, text: String(form.get('text')) });
  return { ok: true };                                // loaders re-run automatically afterwards
}

export const meta = ({ loaderData }: Route.MetaArgs) => [{ title: loaderData.product.name }];

export default function Product({ loaderData }: Route.ComponentProps) {
  const nav = useNavigation();
  return (
    <main>
      <h1>{loaderData.product.name}</h1>
      <Form method="post">
        <textarea name="text" />
        <button disabled={nav.state === 'submitting'}>Post review</button>
      </Form>
    </main>
  );
}

export function ErrorBoundary() { return <p role="alert">Something went wrong.</p>; }
```

```tsx
// react-router.config.ts — choose the rendering strategy
import type { Config } from '@react-router/dev/config';
export default {
  ssr: true,                                          // false = SPA (CSR only)
  async prerender() { return ['/', '/about']; },      // these URLs become static HTML at build
} satisfies Config;
```

How it maps to this chapter

Concept	React Router framework mode
SSR	`ssr: true`; `loader` runs on the server per request; HTML streams
SSG	`prerender` (boolean, array of paths, or async function)
CSR per route	`clientLoader` instead of `loader` (data fetched in the browser)
Hydration data	Loader data is serialized into the page — no double fetch (§13.4)
Mutations	`action` + `<Form>`; works without JS (progressive enhancement), then revalidates loaders
Streaming	Return a promise from the loader and read it with `<Await>` or `use()` inside `<Suspense>`
RSC	Experimental (`unstable_` APIs) in v8 — not yet for production

Next.js vs React Router at a glance

	Next.js App Router	React Router framework mode
Data loading	`async` Server Components	`loader` / `clientLoader` per route
Mutations	Server Functions (`'use server'`)	`action` + `<Form>`
Server Components	Stable, central	Experimental
Bundler	Turbopack (webpack optional)	Vite
Mental model	React-specific (RSC, directives, caching)	Web standards (Request, Response, FormData, HTTP caching)
Can start as a SPA	Static export (limited)	Yes — `ssr: false`, add SSR later

> **TL;DR**

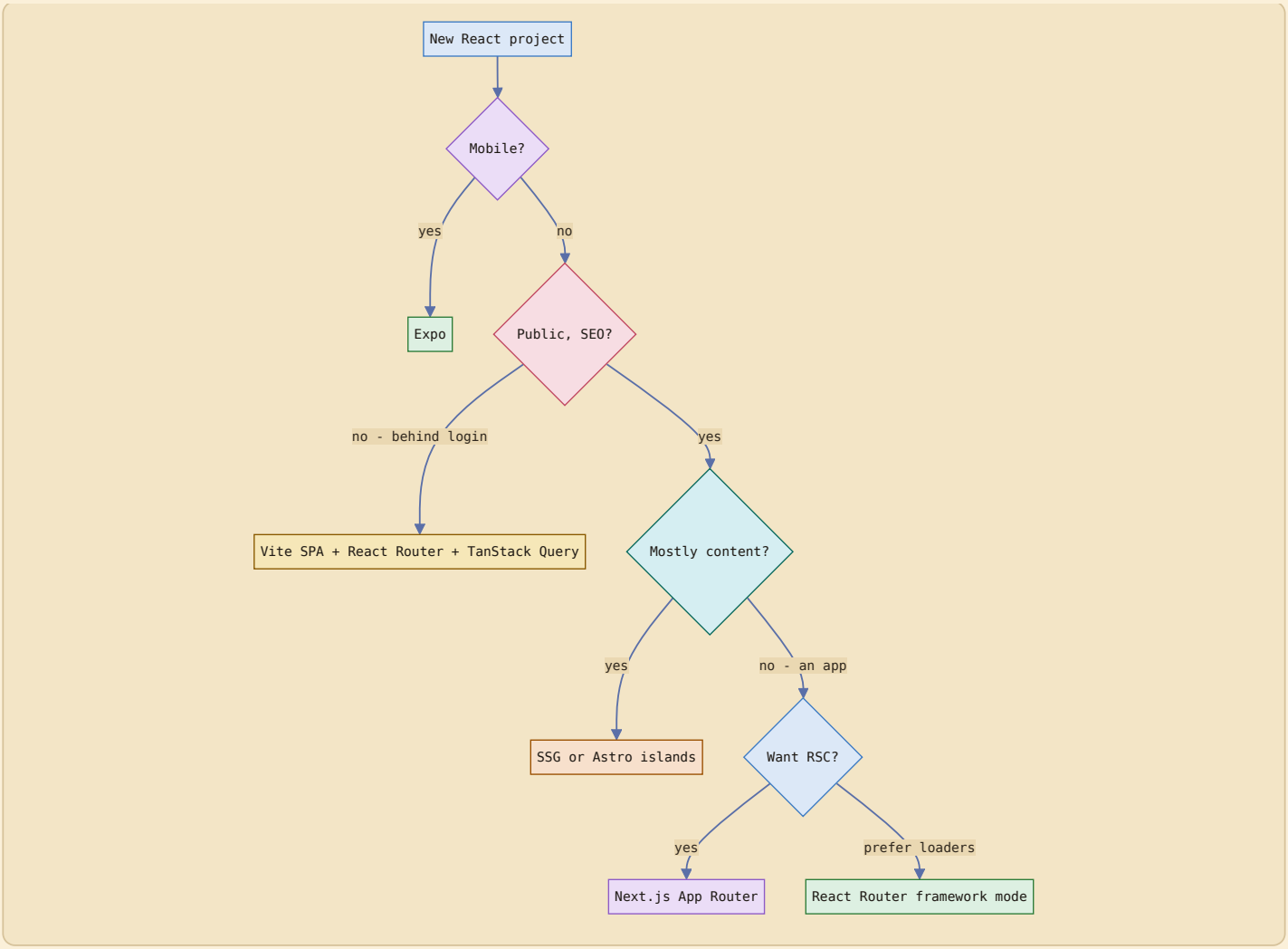
React Router framework mode = `routes.ts` + route modules (`loader`, `action`, `meta`, `ErrorBoundary`, default component) + a config that picks SSR, SPA or prerendering. Web-standard, Vite-based, and an easy upgrade from data mode.

13.11 Choosing a framework

The options (September 2026)

Option	What it is	Pick it when
Next.js (App Router)	Full-stack, RSC-first, by Vercel	You want Server Components, SEO-heavy pages with mixed static/dynamic content, one framework for front + back end
React Router (framework mode)	Full-stack, web-standard, Vite-based, by Shopify	You like loaders/actions and HTTP semantics, are coming from a React Router SPA, or deploy to many platforms
Expo	React Native framework for iOS, Android and web	You're building a mobile app
TanStack Start	Newer full-stack framework on TanStack Router (type-safe routing, server functions, streaming); v1 release candidate, RSC experimental	You want end-to-end type safety and already use TanStack tools
Astro (not React-specific)	Content site framework with React "islands"	Mostly static content (docs, marketing) with a few interactive widgets
Vite SPA (no framework)	`npm create vite@latest` + React Router data mode + TanStack Query	Internal tools and dashboards behind a login, static hosting only, no SEO needs

react.dev recommends a framework (Next.js, React Router or Expo) for new apps, and Vite, Parcel or Rsbuild when none fits.



Questions to ask

1. **Who must see content before JS runs?** Crawlers, link previews, slow phones → server rendering or SSG.
2. **Where will it run?** Static hosting → SPA or SSG; Node / serverless / edge → any framework (check the adapter).
3. **How fresh must data be?** Static → SSG; hourly → ISR / `'use cache'`; per user → SSR or client fetching.
4. **What does the team know?** A team fluent in React Router data mode moves to framework mode easily.
5. **Hosting and lock-in.** All run on Node/Docker, but check self-hosting docs for ISR and caching if you won't use the vendor's platform.

Common mistakes

- SSR for an internal dashboard "for performance" — a server to run, little gain.
- A SPA for an e-commerce catalogue — slow product pages, empty link previews.
- Choosing by popularity instead of by where it will be deployed.

In the interview

30-second answer: "Requirements first. Public pages needing SEO and fast first paint → a framework: Next.js for Server Components and mixed static/cached/streamed content, React Router framework mode for loaders, actions and web standards or an existing React Router app. An internal tool behind a login → Vite SPA + React Router + TanStack Query. Mobile → Expo. Mostly static content → SSG or Astro."

- *What does a framework add over Vite + `react-dom/server`?* — Routing tied to data loading and code splitting, data serialization for hydration (no double fetch), caching/revalidation, server + client bundling and deployment adapters — what §13.4 would otherwise make you build.

TL;DR

Public, SEO-sensitive app → Next.js or React Router framework mode. Behind a login → Vite SPA. Mobile → Expo. Mostly content → SSG/Astro. Decide from users, hosting and data freshness.

Chapter 13 TL;DR

Idea	Remember
Strategies	CSR, SSR, SSG, ISR, streaming, PPR — chosen per route
Server APIs	`renderToPipeableStream` / `renderToReadableStream` (SSR), `prerender*` (build), `resume*` (PPR); string APIs are legacy
Streaming	Shell first; boundaries arrive as hidden divs + `$RC`; status in `onShellReady`, crawlers on `onAllReady`, `abort` on timeout
Hydration	Same tree and data in `hydrateRoot`; reuses DOM; selective; visible ≠ interactive
Mismatches	First client render = server HTML; Effect, `useSyncExternalStore` or `use(browser())`; fix nesting
Server Components	Server-only, async, zero JS; RSC payload; not SSR; only serializable props cross; `'use client'` at the leaves
Server Functions	`'use server'`; public endpoints → auth + validate
Frameworks	Next.js 16 (`app/`, Cache Components), React Router 8 (loaders/actions, RSC unstable); SPA for tools behind a login

Next: Chapter 14 — Patterns and Architecture.

Chapter 14 — Patterns and Architecture

You now know every React building block. This chapter is about **arranging** them: the reusable patterns you'll meet in every codebase and component library (compound components, render props, higher-order components, custom Hooks, portals, error boundaries), how to make components **accessible**, how to organise **folders** so a project stays navigable as it grows, and how a **design system** ties it all together.

Each pattern answers one question — "how do I share *this* kind of thing?" — so for each one you'll see the problem, a runnable implementation, and when a simpler tool is better. The short version, which the chapter justifies: **composition and custom Hooks cover most needs**; the other patterns are for specific jobs.

What you'll learn

#	Topic	You will be able to…
14.1	Composition first	build flexible components with `children` and slots instead of many boolean props
14.2	Container / presentational	separate "where data comes from" from "how it looks"
14.3	Extracting custom Hooks	refactor a tangled component into a Hook + a view, and test each
14.4	Compound components	build `<Tabs>` / `<Tabs.Tab>` / `<Tabs.Panel>` that share state implicitly
14.5	Render props	let callers decide the markup while you own the behaviour
14.6	Higher-order components	read and write HOCs, and avoid their traps
14.7	Choosing a reuse pattern	pick between Hooks, render props, HOCs and compound components
14.8	Portals	render modals and tooltips outside the parent's DOM
14.9	Error boundaries	contain crashes, reset them, and know what they can't catch
14.10	Accessibility in React	write accessible JSX and check it with Testing Library role queries
14.11	Folder structure	organise by feature, define public APIs, enforce boundaries
14.12	Design systems	tokens, primitives, headless libraries and a `Button` with variants
14.13	Client architecture patterns	compare MVC, MVP, MVVM, VIPER and unidirectional flow, and explain where React fits

Code: `code/` — React 19.3 in jsdom with Testing Library; every numbered file prints what a user (or a screen reader) would get. Reusable modules: `Tabs.jsx`, `Dialog.jsx`, `ErrorBoundary.jsx`, `Button.jsx`. The screenshots come from `ds_demo.html` in headless Chrome. Background: composition and `children` (Chapter 2 §2.4), Context (Chapter 6), custom Hooks and `useId` (Chapter 7), refs (Chapter 5), Testing Library queries (Chapter 12 §12.3). Accessibility fundamentals (landmarks, ARIA, WCAG) are in the HTML & CSS book's Chapter 3; general design patterns (decorator, observer, strategy) in the LLD book's structural and behavioural pattern chapters.

14.1 Composition first

What it is

Composition means building a component out of other components you pass in (`children`, or JSX in named props called **slots**) instead of adding a configuration prop for every variation. Chapter 2 §2.4 introduced it; here it's the baseline every other pattern is compared against.

The problem it solves — "prop explosion"

A component that tries to handle every case through props grows like this:

```
<Card title="Trail Mug" subtitle="Enamel" showImage imageSrc="mug.png" imagePosition="left"
      showFooter footerButtonLabel="Buy" onFooterButtonClick={buy} compact highlighted />
```

Every new need adds a prop, a branch inside `Card`, and a test case. Callers can't do anything the author didn't foresee (two buttons in the footer? a badge next to the title?).

The composed version exposes **structure** and lets callers fill it:

```
<Card>
  <Card.Media src="mug.png" alt="Trail Mug" />
  <Card.Title>Trail Mug <Badge>New</Badge></Card.Title>
  <p>Enamel, 350 ml.</p>
  <Card.Footer><Button onClick={buy}>Buy</Button><Button variant="secondary">Save</Button></Card.Footer>
</Card>
```

How to apply it

1. Use `children` for the main content and **slot props** (`header={…}`, `actions={…}`) for secondary areas.
2. Make **specialised components by wrapping**, not by flags: `function DangerDialog(props) { return <Dialog {...props} tone="danger" icon={<WarningIcon/>} />; }`.
3. When parts need **shared state** (which tab is selected), move to compound components (§14.4).
4. Pass elements down instead of data to avoid prop drilling (Chapter 2 §2.4) — the middle layers just render `{children}`.

When not to use it

For tiny, fixed components (a `Price` that formats a number) a couple of plain props are simpler. Composition pays off when callers genuinely need different layouts.

TL;DR

Prefer `children` and slots over growing lists of boolean props. Specialise by wrapping. Composition is the default; the patterns below solve the cases it doesn't.

14.2 Container / presentational

What it is

A split of one feature into two components:

- **Presentational** ("view") component — receives everything as props and returns markup. No fetching, no global state, no Effects. Easy to reuse, test and show in Storybook.
- **Container** component — knows *where data comes from*: fetches, reads context or a store, handles events, and renders the presentational component with props.

Analogy: the presentational component is an actor who reads whatever script they're handed; the container is the director who decides which script.

History and today

The pattern became popular in 2015 (class components, Redux `connect`). Once Hooks arrived, its best-known advocate (Dan Abramov) added a note saying he no longer recommends splitting components this way *by rule*, because a custom Hook now does the container's job without an extra component. The idea is still valuable in three places:

1. **Design-system and shared UI** — visual components should stay data-agnostic.
2. **Testing and Storybook** — you can render every visual state (loading, empty, error) by passing props.
3. **Server Components** (Chapter 13 §13.7) — a Server Component "container" fetches data and renders a Client Component "view". The split is now an architectural boundary.

The runnable version is part of §14.3: `ProductSearch` (container) = `useProductSearch` (logic) + `ProductSearchView` (presentational).

Common mistakes

- Splitting everything mechanically into `FooContainer` + `Foo` — twice the files, little benefit. Split when the view is reused or has many states worth testing alone.
- A "presentational" component that secretly calls `useContext(AuthContext)` — then it isn't reusable anymore.

TL;DR

Keep visual components props-in/markup-out; put data access in a Hook or a container (or a Server Component). Do it where reuse or testing benefits, not everywhere.

14.3 Extracting custom Hooks

What it is

Moving stateful logic (state + Effects + derived values) out of a component into a **custom Hook** (a function whose name starts with `use` and which calls other Hooks — Chapter 7). The component keeps only what it renders.

Why it matters

- The component becomes readable — markup only.
- The logic becomes **reusable** (another screen can call `useProductSearch`) and **testable on its own** with `renderHook`.
- The view becomes testable **without** mocking the network.

How to do it (recipe)

1. Find a group of state + Effects that work together (here: query → loading → results).
2. Move them into `function useX(inputs)` and **return only what the UI needs** (`{ status, results }`), not setters for internal details.
3. Pass dependencies in as parameters (the `search` function) so tests can substitute them.
4. Make the component call the Hook and render. Optionally split the markup into a presentational component (§14.2).
5. Check the behaviour didn't change (same test before and after).

`code/04_custom_hook_extraction.jsx` — before:

```
function ProductSearchBefore() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [status, setStatus] = useState('idle');
  useEffect(() => {
    if (!query) { setResults([]); setStatus('idle'); return; }
    let ignore = false;
    setStatus('loading');
    api.search(query).then(r => { if (!ignore) { setResults(r); setStatus('done'); } });
    return () => { ignore = true; };
  }, [query]);
  return (
    <section>
      <label>Search <input value={query} onChange={e => setQuery(e.target.value)} /></label>
      {status === 'loading' && <p role="status">Searching…</p>}
      {status === 'done' && <ul>{results.map(r => <li key={r}>{r}</li>)}</ul>}
    </section>
  );
}
```

After — Hook + view + thin container:

```
export function useProductSearch(query, search = api.search) {        // logic, injectable dependency
  const [state, setState] = useState({ status: 'idle', results: [] });
  useEffect(() => {
    if (!query) { setState({ status: 'idle', results: [] }); return; }
    let ignore = false;                                               // ignore stale responses
    setState(s => ({ ...s, status: 'loading' }));
    search(query).then(results => { if (!ignore) setState({ status: 'done', results }); });
    return () => { ignore = true; };
  }, [query, search]);
  return state;
}

export function ProductSearchView({ query, onQueryChange, status, results }) {   // markup only
  return (
    <section>
      <label>Search <input value={query} onChange={e => onQueryChange(e.target.value)} /></label>
      {status === 'loading' && <p role="status">Searching…</p>}
      {status === 'done' && <ul>{results.map(r => <li key={r}>{r}</li>)}</ul>}
    </section>
  );
}

function ProductSearch() {                                            // wiring
  const [query, setQuery] = useState('');
  const { status, results } = useProductSearch(query);
  return <ProductSearchView query={query} onQueryChange={setQuery} status={status} results={results} />;
}

// tests: same user flow for both versions; then the Hook alone and the view alone
const { result, rerender } = renderHook(({ q }) => useProductSearch(q, fakeSearch), { initialProps: { q: '' } });
render(<ProductSearchView query="x" onQueryChange={() => {}} status="loading" results={[]} />);
```

Output:

```
before: Trail Mug, Trail Stove
after : Trail Mug, Trail Stove
hook, empty query : {"status":"idle","results":[]}
hook, "mug"       : {"status":"done","results":["mug result"]}
view in loading   : Searching… (no API needed)
```

The behaviour is identical, and now the Hook can be tested with a fake search function and the view's loading state rendered directly. In a real app the Hook would typically wrap TanStack Query (Chapter 9) and add debouncing (`useDebounce`, Chapter 7 §7.5).

Common mistakes

- A Hook that returns 12 values including internal setters — the caller now depends on internals. Return a small, intention-revealing API.
- Extracting a "Hook" that uses no Hooks — that's a plain function; don't name it `use…`.
- Expecting two components calling the same Hook to **share** state. Each call has its own state; share via context or a store (Chapter 6).

In the interview

30-second answer: "When a component mixes data logic and markup, I extract the logic into a custom Hook that takes inputs (and injectable dependencies) and returns a small API like `{ status, results }`. The component just renders, often via a presentational component. That makes the logic reusable and testable with `renderHook`, and the view testable with plain props. Hooks share *logic*, not state."

TL;DR

Logic → `useSomething(inputs)` returning a small API; markup → a props-only view; a thin component wires them. Same behaviour, easier to test and reuse.

14.4 Compound components

What it is

A set of components designed to be used **together**, sharing state **implicitly** through a private context: `<Tabs>`, `<Tabs.List>`, `<Tabs.Tab>`, `<Tabs.Panel>`. The parent owns the state; the children read it. Analogy: HTML's `<select>` and `<option>` — you arrange the options, the select knows which one is chosen.

The problem it solves

A single `<Tabs tabs={[{ label, content }]}>` component with an array prop can't handle custom layouts (tabs below the panel, an icon in one tab, a badge, a panel shared by two tabs) without more and more props. Compound components give callers the **structure** (composition) while the component keeps the **behaviour** (selection, keyboard, ARIA).

How it works

1. The root (`Tabs`) holds the state (`selected`) and puts it plus a `select` function in a **context**.
2. Each part (`Tab`, `Panel`) reads the context with a small Hook that **throws a clear error** when used outside the root.
3. Parts are attached as properties (`Tabs.Tab = Tab`) so one import gives the whole family.
4. The root supports **uncontrolled** (`defaultValue`) and **controlled** (`value` + `onValueChange`) use — the same convention as `<input>` (Chapter 3 §3.9).
5. The component owns accessibility: roles, `aria-selected`, `aria-controls`, keyboard navigation.

`code/Tabs.jsx` (key parts):

```
const TabsContext = createContext(null);
function useTabs(part) {
  const ctx = useContext(TabsContext);
  if (!ctx) throw new Error(`<Tabs.${part}> must be used inside <Tabs>`);
  return ctx;
}

export function Tabs({ defaultValue, value, onValueChange, children }) {
  const [inner, setInner] = useState(defaultValue);
  const selected = value ?? inner;                                  // controlled wins
  const select = (v) => { if (value === undefined) setInner(v); onValueChange?.(v); };
  const baseId = useId();                                           // stable ids for aria links
  return <TabsContext value={{ selected, select, baseId }}>{children}</TabsContext>;
}

function List({ label, children }) {
  const listRef = useRef(null);
  function onKeyDown(e) {                                           // arrows, Home, End
    const tabs = [...listRef.current.querySelectorAll('[role="tab"]')];
    const i = tabs.indexOf(document.activeElement);
    const next = { ArrowRight: i + 1, ArrowLeft: i - 1, Home: 0, End: tabs.length - 1 }[e.key];
    if (next === undefined) return;
    e.preventDefault();
    const tab = tabs[(next + tabs.length) % tabs.length];
    tab.focus(); tab.click();
  }
  return <div role="tablist" aria-label={label} ref={listRef} onKeyDown={onKeyDown}>{children}</div>;
}

function Tab({ value, children }) {
  const { selected, select, baseId } = useTabs('Tab');
  const isSelected = selected === value;
  return (
    <button role="tab" type="button" id={`${baseId}-tab-${value}`} aria-controls={`${baseId}-panel-${value}`}
      aria-selected={isSelected} tabIndex={isSelected ? 0 : -1} onClick={() => select(value)}>
      {children}
    </button>
  );
}

function Panel({ value, children }) {
  const { selected, baseId } = useTabs('Panel');
  if (selected !== value) return null;
  return <div role="tabpanel" id={`${baseId}-panel-${value}`} aria-labelledby={`${baseId}-tab-${value}`} tabIndex={0}>
{children}</div>;
}
Tabs.List = List; Tabs.Tab = Tab; Tabs.Panel = Panel;
```

Usage and checks (`code/01_compound_tabs.jsx`) — every check uses **role queries**, i.e. what assistive technology sees:

```
<Tabs defaultValue="details">
  <Tabs.List label="Product information">
    <Tabs.Tab value="details">Details</Tabs.Tab>
    <Tabs.Tab value="reviews">Reviews</Tabs.Tab>
    <Tabs.Tab value="shipping">Shipping</Tabs.Tab>
  </Tabs.List>
  <Tabs.Panel value="details">Enamel mug, 350 ml.</Tabs.Panel>
  <Tabs.Panel value="reviews">4.5 stars from 12 reviews.</Tabs.Panel>
  <Tabs.Panel value="shipping">Ships in 2 days.</Tabs.Panel>
</Tabs>
// user.click(getByRole('tab', { name: 'Reviews' })), user.keyboard('{ArrowRight}'), '{End}', ...
```

Output (`*` = `aria-selected="true"`):

```
tablist name : Product information
start        : Details* | Reviews | Shipping  panel="Enamel mug, 350 ml."
click Reviews: Details | Reviews* | Shipping  panel="4.5 stars from 12 reviews."
ArrowRight   : Details | Reviews | Shipping*  panel="Ships in 2 days."  focus: Shipping
ArrowRight   : Details* | Reviews | Shipping  panel="Enamel mug, 350 ml."  (wrapped around)
End          : Details | Reviews | Shipping*  panel="Ships in 2 days."
panel labelled by its tab: Ships in 2 days.
tab stops    : -1,-1,0
controlled   : parent state: a | Panel A
misuse       : <Tabs.Tab> must be used inside <Tabs>
```

What it shows:

1. Selection, arrow keys (with wrap-around), `End` — the keyboard behaviour the WAI-ARIA Authoring Practices "Tabs" pattern describes.
2. **Roving tabindex** (`tab stops: -1,-1,0`): only the selected tab is in the Tab-key order; arrows move within the list. The whole widget is one Tab stop.
3. The panel is findable by its **accessible name**, which comes from its tab via `aria-labelledby`.
4. The same component works **controlled** by a parent.
5. Misuse fails loudly with a helpful message.

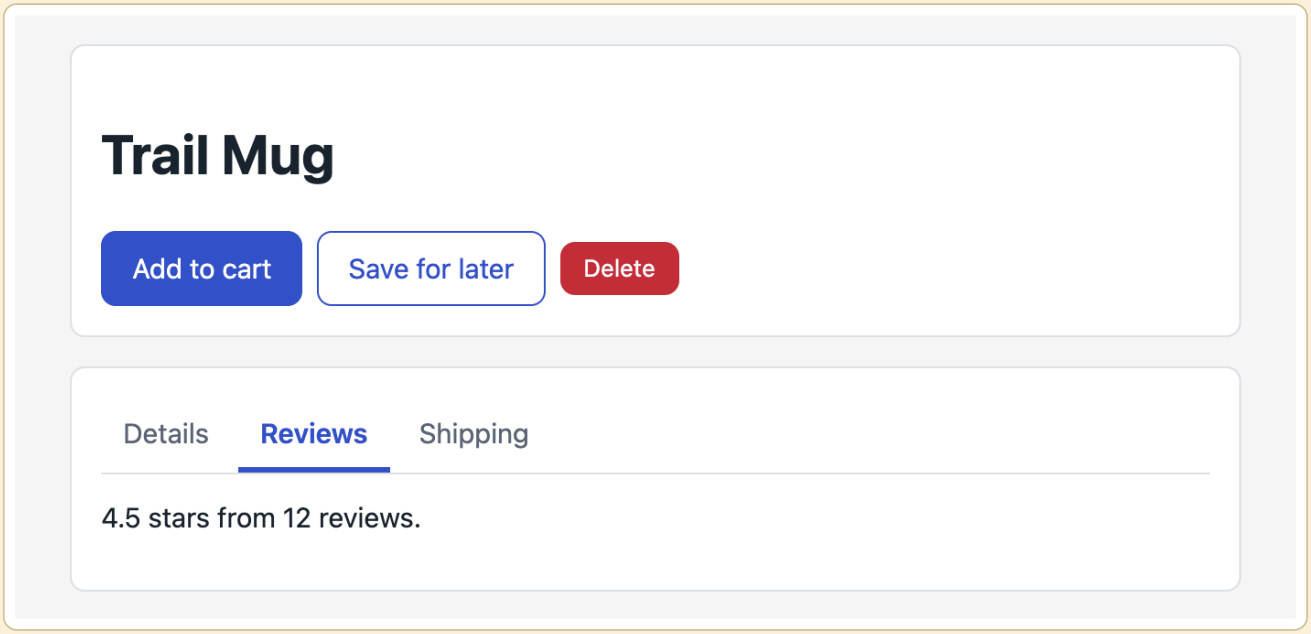


Figure 14.1 — The compound `Tabs` (and the `Button` from §14.12) rendered in Chrome with design tokens (`code/ds_demo.html`).

Trade-offs

	Array/config prop (`tabs={[…]}`)	Compound components
Simple cases	Shortest to use	Slightly more JSX
Custom layout / content	Needs new props	Free — callers arrange parts
Type safety of structure	Easy (typed array)	Harder to enforce order/nesting
Implementation	Simple	Context + part Hooks

Real-world examples

Radix UI (`<Tabs.Root>`, `<Tabs.List>`, `<Tabs.Trigger>`, `<Tabs.Content>`), React Aria Components, Headless UI, Reach UI — and native HTML's `<select>`/`<option>`, `<table>`/`<tr>`, `<details>`/`<summary>`.

Common mistakes

- Relying on `React.Children.map` + `cloneElement` to inject props — breaks as soon as a caller wraps a part in a `<div>` or another component. Context works at any depth.
- Forgetting the error Hook — parts used outside the root then fail silently with `undefined` state.

- Rebuilding the context value object every render in a large tree (a perf consideration — Chapter 6 §6.4; React Compiler handles it).

In the interview

30-second answer: "Compound components are a family like `Tabs`, `Tabs.List`, `Tabs.Tab`, `Tabs.Panel` that share state through a private context. The root owns the selected value — controlled or uncontrolled — and the parts read it, so callers control layout while the component controls behaviour and accessibility. I use context rather than `cloneElement` so parts can be nested at any depth, and I throw a clear error if a part is used outside its root."

- *Why not `cloneElement`?* — It only reaches direct children and silently breaks with wrappers; it's also listed as a legacy API on react.dev.
- *How do you support both controlled and uncontrolled?* — Keep internal state for `defaultValue`, but prefer the `value` prop when it's defined, and always call `onValueChange`.

Self-check

1. How do `Tab` and `Panel` know which tab is selected? → They read the root's context.
2. What is a roving tabindex? → Only the active item has `tabIndex=0`; others `-1`; arrow keys move focus within the group.
3. What links a panel to its tab for screen readers? → `aria-labelledby` (panel → tab) and `aria-controls` (tab → panel), with ids from `useId`.

TL;DR

Root owns state in context; parts read it; callers arrange the parts. Support controlled + uncontrolled, own the ARIA and keyboard behaviour, and throw on misuse.

14.5 Render props

What it is

A **render prop** is a prop whose value is a **function that returns JSX**. The component owns some behaviour or data and **calls your function** to decide what to render. When the function is passed as `children`, it's called "function as children".

The problem it solves

Sharing behaviour while letting each caller choose the markup — before Hooks this was the main way to reuse stateful logic (e.g. React Router v4's `<Route render>`, Formik's `<Formik>{(form) => …}</Formik>`, Downshift). Today it's still the right tool when a component must **control *where* and *how often*** your markup is rendered: lists (`renderItem`), virtualized rows, tables (`cell` renderers), and empty/loading slots.

Seeing it

`code/02_render_props.jsx`:

```jsx
function Disclosure({ defaultOpen = false, children }) {          // function as children
  const [open, setOpen] = useState(defaultOpen);
  return children({ open, toggle: () => setOpen(o => !o) });
}

function List({ items, renderItem, renderEmpty = () => <p>Nothing here.</p> }) {   // named render props
  if (items.length === 0) return renderEmpty();
  return <ul>{items.map((item, i) => <li key={item.id}>{renderItem(item, i)}</li>)}</ul>;
}

function useDisclosure(defaultOpen = false) {                        // the Hook equivalent
  const [open, setOpen] = useState(defaultOpen);
  return { open, toggle: () => setOpen(o => !o) };
}

<Disclosure>
  {({ open, toggle }) => (
    <>
      <button aria-expanded={open} onClick={toggle}>Shipping details</button>
      {open && <p>Ships in 2 days.</p>}
    </>
  )}
</Disclosure>
<List items={products} renderItem={(p, i) => <>{i + 1}. {p.name} — ${p.price}</>} />
```

Output:

```
render prop, closed : <button aria-expanded="false">Shipping details</button>
render prop, open   : <button aria-expanded="true">Shipping details</button><p>Ships in 2 days.</p>
renderItem          : <ul><li>1. Trail Mug — $18</li><li>2. Camp Stove — $45</li></ul>
renderEmpty         : <p>No products match.</p>
same thing via Hook : <button aria-expanded="true">Shipping details</button><p>Ships in 2 days.</p>
```

`Disclosure` never decides what the button looks like; `List` decides *where* each item goes (inside `<li>`, with a key) but not what it shows.

Trade-offs

- ✔ Explicit: you see exactly which values the caller receives.
- ✔ Works in class components and anywhere JSX is allowed.
- ✘ Nesting several render-prop components creates "callback pyramids".
- ✘ A new function every render can defeat `memo` on the component (React Compiler usually handles it).
- For **sharing stateful logic** only, a custom Hook is simpler (last line of the output).

Common mistakes

- Using a render prop just to pass data one level down — a normal prop or `children` is enough.
- Calling Hooks inside the render function — it's a plain callback, not a component, so the Rules of Hooks forbid it (the linter flags it) and it breaks as soon as the call becomes conditional. Put stateful parts in a component.

TL;DR

A render prop is "give me a function and I'll call it to render". Reach for it when the component decides placement and repetition of caller markup (lists, tables, slots); share plain logic with a Hook.

14.6 Higher-order components (HOCs)

What it is

A **higher-order component** is a function that **takes a component and returns a new component** that renders the original with extra props or behaviour: `const Enhanced = withSomething(Component)`. The name mirrors higher-order functions (JavaScript book, Chapter 4); the idea is the **decorator pattern** from OOP (LLD Chapter 9).

Where you still meet them

Older code and some libraries: Redux's `connect(mapState)(Component)`, React Router v5's `withRouter`, `React.memo(Component)` (technically a HOC-like wrapper), analytics/feature-flag wrappers, and auth guards. New code usually uses Hooks instead, but you must be able to read and maintain HOCs.

Seeing it

`code/03_hoc.jsx`:

```
function withAuth(Component) {
  function WithAuth(props) {
    const user = useContext(UserContext);
    if (!user) return <p>Please sign in.</p>;
    return <Component {...props} user={user} />;          // pass everything through + inject one prop
  }
  WithAuth.displayName = `withAuth(${Component.displayName || Component.name})`;   // readable in DevTools
  return WithAuth;
}

function withRenderLog(Component) { /* logs each render with its props, same shape */ }

function OrderHistory({ user, limit }) { return <p>{user.name}'s last {limit} orders</p>; }
const ProtectedHistory = withRenderLog(withAuth(OrderHistory));   // apply once, at module level

// PITFALL demo: applying a HOC inside render
function Page({ bad }) {
  const [, force] = useState(0);
  const Wrapped = bad ? withNothing(Counter) : StableCounter;    // bad: a new component type every render
  return <><Wrapped /><button onClick={() => force(x => x + 1)}>re-render page</button></>;
}
```

Output:

```
displayName: withRenderLog(withAuth(OrderHistory))
  [log] withAuth(OrderHistory) rendered with {"limit":3}
signed out : Please sign in.
  [log] withAuth(OrderHistory) rendered with {"limit":3}
signed in  : Asha's last 3 orders
HOC at module    : count 2 -> after parent re-render: count 2
HOC inside render : count 2 -> after parent re-render: count 0
```

What it shows:

1. HOCs **compose** from the inside out, and `displayName` makes the wrapper chain readable in React DevTools.
2. Props flow through (`limit`), and the HOC injects its own (`user`).
3. **Never create a HOC during render.** Each call returns a *new component type*, so React unmounts the old one and mounts a fresh one — state (`count 2`) is lost (Chapter 3 §3.10).

Common mistakes (the classic HOC problems)

Problem	What happens	Fix
HOC applied inside render	Remount every render, state lost	Apply at module level
Prop name collisions	Two HOCs both inject `data`; one silently wins	Namespacing, or use Hooks
"Wrapper hell"	Deep trees of anonymous wrappers in DevTools	`displayName`; Hooks
Static methods lost	`Enhanced.someStatic` is undefined	Copy them (`hoist-non-react-statics`)
Refs	Before React 19, `ref` didn't pass through without `forwardRef`	React 19: `ref` is a normal prop for function components; spread it through
Hidden dependencies	Reader can't see where `user` came from	A Hook call inside the component is explicit

In the interview

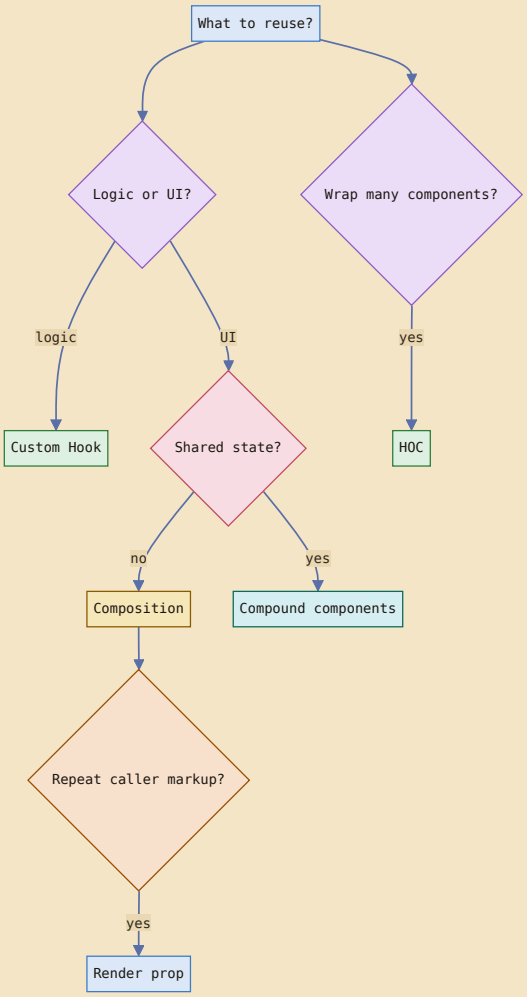
> **30-second answer:** "A HOC is a function that takes a component and returns a wrapped one, used for cross-cutting concerns like auth guards, logging or injecting store data — Redux's `connect` is the famous example. They compose but have downsides: prop collisions, implicit props, wrapper nesting, and they must be created at module level or the component remounts every render. For new code I'd use a custom Hook; I'd still write a HOC when I need to wrap *rendering* itself, like an auth guard around many route components."

TL;DR

`withX(Component)` → new component with extra props/behaviour. Apply once at module level, pass props through, set `displayName`. Prefer Hooks for new logic reuse.

14.7 Choosing a reuse pattern

Need	Best fit	Why
Share **stateful logic** (fetching, form state, subscriptions)	**Custom Hook**	No extra components, explicit, composable
Share **UI structure** with flexible content	**Composition** (`children`, slots)	Simplest; no new concepts
A family of parts sharing **implicit state** (tabs, accordion, menu, select)	**Compound components**	Callers arrange parts; component owns behaviour + a11y
Component controls **where/how often** caller markup renders (lists, tables, virtualized rows)	**Render prop** (`renderItem`)	The component decides placement, the caller decides content
Wrap **many components** with the same guard/behaviour, or legacy code	**HOC**	Works on components you don't own; decorator style
Render **outside** the parent's DOM box (modal, tooltip, toast)	**Portal**	Escapes `overflow`/`z-index` while staying in the React tree
Contain crashes of a subtree	**Error boundary**	The only way to catch render errors



14.8 Portals

What it is

`createPortal(children, domNode)` (from `react-dom`) renders `children` into a **different DOM node** — usually `document.body` — while they stay in the **same place in the React tree**. So the DOM location changes, but props, context, state and event bubbling behave as if the content were still inside its parent.

The problem it solves

Modals, dropdowns, tooltips and toasts must visually sit **on top of everything**. If rendered inside a card with `overflow: hidden` or inside a stacking context (`transform`, `z-index`), they get clipped or covered — CSS can't escape its ancestors. Rendering them at the end of `<body>` fixes that.

How it works

1. React renders the portal's children as usual (they're part of the parent component's tree).
2. At commit, it inserts their DOM nodes into `domNode` instead of the parent's DOM node.
3. **Context** flows through (the React tree is unchanged).
4. **React events bubble through the React tree**, not the DOM tree: a click inside the portal triggers `onClick` handlers on React ancestors, even though in the DOM they aren't ancestors.

Seeing it

`code/Dialog.jsx` — a modal with the accessibility essentials:

```jsx
export function Dialog({ open, onClose, title, children }) {
  const titleId = useId();
  const panelRef = useRef(null);
  const onEscape = useEffectEvent(() => onClose());     // latest onClose, without re-running the Effect
  useEffect(() => {
    if (!open) return;
    const previouslyFocused = document.activeElement;   // remember the trigger
    panelRef.current.focus();                           // move focus into the dialog
    const onKey = (e) => { if (e.key === 'Escape') onEscape(); };
    document.addEventListener('keydown', onKey);
    return () => {
      document.removeEventListener('keydown', onKey);
      previouslyFocused?.focus?.();                     // give focus back on close
    };
  }, [open]);                                          // re-runs only when opened/closed

  if (!open) return null;
  return createPortal(
    <div className="backdrop" onClick={(e) => { if (e.target === e.currentTarget) onClose(); }}>
      <div role="dialog" aria-modal="true" aria-labelledby={titleId} tabIndex={-1} ref={panelRef} className="dialog">
        <h2 id={titleId}>{title}</h2>
        {children}
        <button type="button" className="btn btn-secondary btn-sm" onClick={onClose}>Close</button>
      </div>
    </div>,
    document.body,
  );
}
```

`code/05_portal.jsx` opens it from inside a card with `overflow: hidden` and an `onClick` counter:

```jsx
function ProductCard() {
  const [open, setOpen] = useState(false);
  const [bubbled, setBubbled] = useState(0);
  return (
    <div className="card" style={{ overflow: 'hidden' }} onClick={() => setBubbled(n => n + 1)}>
      <button onClick={() => setOpen(true)}>Delete product</button>
      <span data-testid="bubbled">{bubbled}</span>
      <Dialog open={open} onClose={() => setOpen(false)} title="Delete Trail Mug?">
        <p>This cannot be undone.</p>
        <ThemedText />
      </Dialog>
    </div>
  );
}
```

Output:

```
dialog DOM parent        : BODY (not inside .card)
inside .card?            : false
aria-modal               : true
focus moved into dialog  : true
context crosses portal   : theme from context: dark
React events bubble to the React parent: card onClick count 1 -> 2
Escape closes            : true
focus back on trigger    : true
```

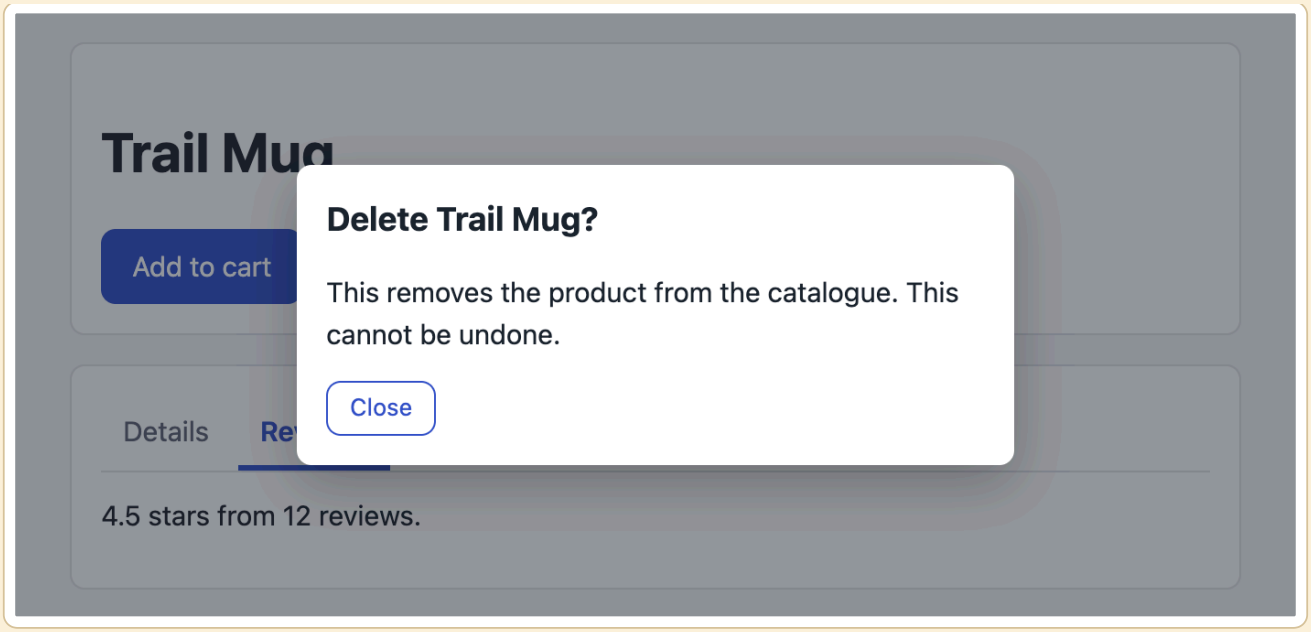


Figure 14.2 — The same `Dialog` in Chrome: rendered at the end of `<body>`, so no card's `overflow` or stacking can clip it.

Reading the output:

1. The dialog's DOM lives under `<body>`, outside `.card` — so `overflow: hidden` can't clip it.
2. Context from above the card still reaches `ThemedText` inside the portal.
3. The click **inside the dialog** incremented the card's counter: React bubbling follows the component tree. This surprises people — e.g. a "click outside to close" handler on a parent fires for clicks inside the portal. Check `e.target`, or stop propagation deliberately.
4. Focus moves in on open and returns to the trigger on close; Escape closes it — the minimum keyboard behaviour for a modal. (This small version doesn't trap Tab inside; the modal build in Chapter 15 §15.13 adds that.) `useEffectEvent` (stable since React 19.2, Chapter 5) lets the Escape listener call the latest `onClose` without re-running the Effect — otherwise a parent passing a new arrow function each render would bounce focus back and forth.

The native `<dialog>` alternative

HTML's `<dialog>` element with `dialog.showModal()` (call it from an Effect via a ref) renders in the browser's **top layer** — above everything without a portal — and gives you Escape-to-close, an inert background, focus moved into the dialog on open and **returned to the previously focused element on close** (specified in the HTML standard and implemented in all major browsers) for free. It's supported in all modern browsers and is often the simplest correct modal (HTML & CSS book, Chapter 3). Portals are still the tool for tooltips, popovers in older setups, toasts, and rendering into other containers (e.g. an iframe or a widget root). The Popover API (`popover` attribute) is another native option for non-modal pop-ups.

Common mistakes

- A home-made modal without focus management — keyboard and screen-reader users end up "behind" the dialog. Move focus in, trap it (or make the background `inert`), return it on close. Use a library (Radix Dialog, React Aria) or `<dialog>` for production.
- Forgetting portal content still counts as a child for events (point 3 above).
- Rendering a portal during SSR: `document` doesn't exist on the server. Render portals only after mount (or use `<dialog>`).

In the interview

30-second answer: "`createPortal` renders children into another DOM node, like `document.body`, while keeping them in the same React tree. That lets modals and tooltips escape `overflow: hidden` and stacking contexts, while context, state and even event bubbling still follow the React tree — a click inside a portal bubbles to React ancestors. For modals I'd also handle focus, Escape and `aria-modal`, or use native `<dialog>` with `showModal()`, which uses the top layer."

TL;DR

Portal = DOM elsewhere, React tree unchanged: context and events follow the component tree. Use for overlays; add focus management; consider native `<dialog>`.

14.9 Error boundaries

What it is

An **error boundary** is a component that catches errors thrown **while rendering** its descendants (including in lifecycle methods and Effects' setup during commit) and shows a **fallback UI** instead of letting the whole app disappear. Without any boundary, React 19 **unmounts the entire root** on an uncaught render error — a blank page.

Error boundaries must still be **class components** — there's no Hook equivalent of the two methods involved. You write one once (or use the `react-error-boundary` package) and use it everywhere. (Route-level boundaries in React Router are covered in Chapter 9; lazy-chunk failures in Chapter 10 §10.8.)

How it works

1. A descendant throws during render.
2. React finds the nearest boundary above it and calls **`static getDerivedStateFromError(error)`** — return new state so the next render shows the fallback.
3. After commit, React calls **`componentDidCatch(error, info)`** — log it; `info.componentStack` tells you where.
4. React 19 also calls the root's **`onCaughtError`** (for errors caught by a boundary) or **`onUncaughtError`** (no boundary) — options of `createRoot`/`hydrateRoot`, useful for one central error reporter. (`onRecoverableError` reports errors React recovered from itself, like hydration mismatches — Chapter 13 §13.5.)
5. To try again, reset the boundary's state — manually (a "Try again" button) or automatically when some input changes (**reset keys**), or by changing the boundary's `key`.

`code/ErrorBoundary.jsx`:

```
export class ErrorBoundary extends Component {
  state = { error: null };
  static getDerivedStateFromError(error) { return { error }; }        // render the fallback next
  componentDidCatch(error, info) { this.props.onError?.(error, info); } // log (Sentry, console…)
  componentDidUpdate(prevProps) {                                     // reset when resetKeys change
    const { resetKeys = [] } = this.props;
    if (this.state.error && resetKeys.some((k, i) => !Object.is(k, prevProps.resetKeys?.[i]))) {
      this.setState({ error: null });
    }
  }
  reset = () => this.setState({ error: null });
  render() {
    if (this.state.error) return this.props.fallback({ error: this.state.error, reset: this.reset });
    return this.props.children;
  }
}
```

What boundaries catch — and what they don't

`code/06_error_boundary.jsx` puts a price and a reviews widget in separate boundaries, and has an event handler that can fail:

```
const root = createRoot(container, {
  onCaughtError(error) { console.log(`   onCaughtError   : "${error.message}"`); },
  onUncaughtError(error) { console.log(`   onUncaughtError : "${error.message}" -> whole root unmounted`); },
});

function ProductPrice({ product }) { return <p>Price: ${product.price.toFixed(2)}</p>; }   // throws on null

function Reviews() {
  const [error, setError] = useState(null);
  if (error) throw error;                                   // re-throw during render → boundary catches
  return <button onClick={() => {
    Promise.reject(new Error('network down')).catch(e => setError(e));   // async error moved into render
  }}>Load reviews</button>;
}

// in Shop:
<ErrorBoundary resetKeys={[id]} fallback={(p) => <Fallback {...p} />} onError={…}>
  <ProductPrice product={product} />
</ErrorBoundary>
<ErrorBoundary fallback={(p) => <Fallback {...p} />}><Reviews /></ErrorBoundary>
<button onClick={() => {
  try { JSON.parse('{bad json'); }                          // event-handler errors: NOT caught by boundaries
  catch { setImportMsg('Could not read the file'); }        // handle them yourself
}}>Import</button>
```

Output:

```
1. start              : Price: $18.00
   onCaughtError   : "Cannot read properties of null (reading 'price')"
   componentDidCatch: stack starts at ProductPrice
2. render error       : Could not show this part (Cannot read properties of null (reading 'price')). Try again
   onCaughtError   : "Cannot read properties of null (reading 'price')"
   componentDidCatch: stack starts at ProductPrice
3. reset, still broken: Could not show this part (Cannot read properties of null (reading 'price')). Try again
4. resetKeys changed  : Price: $21.00
5. handler error      : Could not read the file | price still fine: Price: $21.00
   onCaughtError   : "network down"
6. async error        : Could not show this part (network down). Try again | price still fine: Price: $21.00
7. no boundary        : container HTML = ""
```

Reading it:

1. **Step 2** — the render error is contained: only the price area shows the fallback; the rest of the page works.
2. **Step 3** — "Try again" re-renders the same broken data, so it fails again. Resetting only helps if something changed.
3. **Step 4** — changing `id` (a reset key) cleared the error, and the new product renders.
4. **Step 5** — errors in **event handlers** aren't render errors; boundaries never see them. Catch them and turn them into state.
5. **Step 6** — **async** errors (promises, timers) aren't caught either, unless you move them into render by storing and re-throwing them (TanStack Query's `throwOnError` and `use()` with a rejected promise do this for you).
6. **Step 7** — with no boundary at all, React unmounts the whole root: the container is empty. (In a browser, `onUncaughtError` would be called; in tests, `act()` re-throws the error instead.)

Caught by error boundaries	Not caught
Errors during render of descendants	Event handlers (use `try/catch`)
Errors in lifecycle methods / Effects of descendants	Async code: `setTimeout`, promise callbacks (move into state and re-throw)
Rejected promises read with `use()` inside Suspense	Errors in the boundary itself (a parent boundary catches those)
Errors from `lazy()` chunk loading	Server-side rendering errors (they fall back to client rendering, Chapter 13 §13.3)

Where to put boundaries

- **One at the top** (app shell): "Something went wrong — reload" instead of a blank page.
- **One per route** (frameworks do this: `error.js` in Next.js, `ErrorBoundary` exports in React Router).
- **One around independent widgets** (a chart, a comments panel, a third-party embed) so one failure doesn't take down the page.
- Pair them with `<Suspense>`: Suspense handles "not ready yet", the error boundary handles "failed".

Common mistakes

- Expecting a boundary to catch `onClick` errors.
- A fallback that can itself throw (e.g. reads the same broken data).
- A "Try again" that doesn't change anything — reset together with a refetch or new input (reset keys).
- Swallowing errors without logging: always report in `componentDidCatch` or `onCaughtError`.

In the interview

> **30-second answer:** "Error boundaries are class components with `getDerivedStateFromError` to show a fallback and `componentDidCatch` to log. They catch errors thrown while rendering their subtree — including `use()` rejections and lazy-loading failures — but not event handlers, async callbacks, SSR, or their own errors. Without one, React 19 unmounts the whole root. I place one at the top, per route and around risky widgets, reset them with reset keys, and in React 19 I report centrally via `createRoot`'s `onCaughtError` / `onUncaughtError`."

- *Why still a class?* — There's no Hook for `getDerivedStateFromError` / `componentDidCatch`; most teams use `react-error-boundary`, which wraps a class and adds `resetKeys` and a `useErrorBoundary` Hook to push async errors in.
- *How do you send an async error to a boundary?* — Store it in state and throw it during render (or `showBoundary(error)` from `react-error-boundary`).

Self-check

1. Why did "Try again" in step 3 fail? → The data was still `null`; reset re-rendered the same failing input.
2. Which error in the demo did no boundary catch? → The `JSON.parse` error in the event handler.
3. What happens without any boundary in React 19? → The whole root unmounts; `onUncaughtError` is called.
4. What's the difference between `getDerivedStateFromError` and `componentDidCatch`? → The first runs during render to choose the fallback (pure); the second runs after commit for side effects like logging.

TL;DR

Class component with `getDerivedStateFromError` + `componentDidCatch`; catches render-time errors below it only. Handle event/async errors yourself (or re-throw via state). Place at root, per route, per risky widget; reset with keys; report via `onCaughtError`.

14.10 Accessibility in React

What it is

Accessibility (a11y) means people can use your UI regardless of how they perceive or operate it: screen readers, keyboard only, voice control, zoom, reduced motion. React doesn't make a page accessible or inaccessible by itself — it renders the HTML you write. What React adds is (a) a few JSX spellings, (b) state-driven ARIA attributes, and (c) **focus management** in single-page apps, where there are no full page loads. The fundamentals (landmarks, headings, ARIA rules, WCAG 2.2) are in the HTML & CSS book's Chapter 3; this section is the React-specific layer.

React-specific rules

Topic	In React
Labels	`<label htmlFor={id}>` (`for` is a JS keyword); generate ids with `useId()`, never hard-code them in reusable components
ARIA attributes	Stay hyphenated and lower-case: `aria-label`, `aria-expanded={open}`; booleans render as `"true"` / `"false"`
Semantic elements first	`<button>`, `<a href>`, `<nav>`, `<main>`, `<h1>`–`<h6>`, `<ul>`: they come with roles, keyboard support and focusability for free
Fragments	Use `<>…</>` instead of wrapper `<div>`s that break list/table semantics (`<ul>` must contain `<li>`)
Focus management	Use refs to move focus when UI changes: into dialogs, to a new page heading after client-side navigation, back to the trigger after closing
Announcements	Status messages in a live region: `role="status"` (polite) or `role="alert"` (urgent), present in the DOM before the text changes
Icon-only buttons	Give an accessible name: `aria-label="Remove item"`, and hide the icon with `aria-hidden="true"`
Linting	`eslint-plugin-jsx-a11y` flags missing alt text, click handlers on non-interactive elements, invalid ARIA
Automated audits	axe-core (browser extension, `vitest-axe` / `jest-axe`, Playwright's axe integration), Lighthouse

Checking it with role queries

Testing Library's `getByRole` finds elements the way assistive technology does — by **role** and **accessible name**. If a test can't find your button by role, a screen-reader user can't either (Chapter 12 §12.3). `code/07_a11y_checks.jsx` audits the same checkout written badly and well:

```
function BadCheckout({ onBuy }) {
  return (
    <div>
      <div className="title">Checkout</div>                {/* looks like a heading, isn't one */}
      <img src="mug.png" />                                {/* no alt */}
      <input placeholder="Email" />                        {/* placeholder is not a label */}
      <div className="btn" onClick={onBuy}>Buy now</div>   {/* not focusable, no role */}
      <button><svg aria-hidden="true" /></button>          {/* icon-only: no name */}
    </div>
  );
}

function GoodCheckout({ onBuy }) {
  const emailId = useId(), errId = useId();
  // ... error + done state, submit() validates and calls onBuy ...
  return (
    <form onSubmit={submit} noValidate>
      <h1>Checkout</h1>
      <img src="mug.png" alt="Trail Mug, blue enamel" />
      <label htmlFor={emailId}>Email</label>
      <input id={emailId} name="email" type="email" aria-invalid={!!error} aria-describedby={error ? errId : undefined}
/>
      {error && <p id={errId}>{error}</p>}
      <button type="submit">Buy now</button>
      <button type="button" aria-label="Remove item"><svg aria-hidden="true" /></button>
      <p role="status">{done ? 'Order placed' : ''}</p>
    </form>
  );
}
// audit(): getByRole('heading', { name: 'Checkout' }), getByRole('img'), getByRole('textbox', { name: 'Email' }),
//          getByRole('button', { name: 'Buy now' }), computeAccessibleName(each button), user.tab() ×3
```

Output:

```
--- bad
heading "Checkout"      : false
image with alt text     : missing
textbox named "Email"   : false
button named "Buy now"  : false
button names            : [""]
Tab key visits          : input -> button -> body (the "Buy now" div is never reached)
--- good
heading "Checkout"      : true
image with alt text     : Trail Mug, blue enamel
textbox named "Email"   : true
button named "Buy now"  : true
button names            : ["Buy now","Remove item"]
invalid email -> aria-invalid: true | description: "Enter a valid email"
valid email   -> bought: 1 | aria-invalid: false | status: Order placed
```

What the audit shows:

1. The bad version **looks** the same with CSS, but a screen reader gets no heading, an unnamed image, an unlabeled field, a nameless button — and the "Buy now" `div` is **not reachable with the Tab key** at all.
2. In the good version every control has a role and a name; the error message is linked as the field's **accessible description** (`aria-describedby`), and `aria-invalid` reflects state.
3. A real `<form>` + `<button type="submit">` makes **Enter** submit — keyboard support without extra code.
4. The `role="status"` live region announces "Order placed" to screen readers without moving focus.

Focus after client-side navigation

In an SPA, clicking a link swaps content without a page load, so screen readers may not announce the new page and focus stays on the old link. Common fixes: move focus to the new page's `<h1>` (with `tabIndex={-1}`) in an Effect keyed on the location, and update `document.title` (in React 19 you can render `<title>` in any component). Frameworks help: Next.js announces route changes; React Router offers scroll restoration and you add focus handling.

Common mistakes

- `<div onClick>` instead of `<button>` — no keyboard, no role, no focus. If you truly must, you need `role="button"`, `tabIndex={0}` and Enter/Space handlers — just use `<button>`.
- Removing focus outlines (`outline: none`) without a visible replacement — use `:focus-visible` styles.
- ARIA overuse: `role="button"` on a `<button>`, or `aria-label` that contradicts visible text. The first rule of ARIA: use native elements when they exist.
- Conditional rendering that removes the live region itself — screen readers only announce changes to a region that already exists.
- Testing only with a mouse. Tab through every new screen at least once.

In the interview

30-second answer: "React renders whatever HTML I write, so accessibility is mostly semantic HTML: real buttons, links, headings and labels. The React-specific parts are `htmlFor`, `useId` for label/description ids, ARIA attributes driven by state like `aria-expanded`, live regions for status messages, and focus management with refs — into dialogs, back to triggers, and to the heading after client-side navigation. I test with role queries in Testing Library, which fail when elements lack roles or names, plus `eslint-plugin-jsx-a11y`, axe, and manual keyboard and screen-reader checks."

Self-check

1. Why can't `getByRole('button', { name: 'Buy now' })` find the `<div onClick>`? → A `div` has no button role (and isn't focusable).
2. How does the error message become part of what a screen reader reads for the input? → `aria-describedby` points at its id, making it the accessible description.
3. Why use `useId` instead of `id="email"`? → Reusable components may render many times per page; ids must be unique and match between server and client.
4. What must exist before a status message changes for it to be announced? → The live region element (`role="status"`).

TL;DR

Semantic elements first; `htmlFor` + `useId`; ARIA from state; live regions for updates; manage focus with refs; test with role queries, jsx-a11y and axe, and the keyboard.

14.11 Folder structure

What it is

How you arrange files decides how easily people find code, how far a change ripples, and whether features can be deleted cleanly. React itself has **no opinion** (the React docs have long said so); frameworks impose some (Next.js `app/`, React Router `app/routes/`). Start simple (Chapter 1 §1.6); grow deliberately.

Three common layouts

1. By type — fine for small apps (< ~15 components):

```
src/
├── components/   Button.jsx, ProductCard.jsx, CartPanel.jsx, ...
├── hooks/        useCart.js, useProductSearch.js
├── pages/        Home.jsx, Product.jsx
├── services/     api.js
└── utils/        formatPrice.js
```

Problem at scale: one feature is spread over five folders; `components/` grows to 200 files.

2. By feature — the common choice for medium and large apps:

```
src/
├── app/                ← app shell: providers, router, global styles
├── features/
│   ├── cart/
│   │   ├── components/   CartPanel.jsx, CartItem.jsx
│   │   ├── hooks/        useCart.js
│   │   ├── api.js        cart endpoints
│   │   ├── cart.test.jsx
│   │   └── index.js      ← public API: the only file other features import
│   └── catalog/ ...
└── shared/             ← feature-agnostic: ui/ (design system), lib/ (helpers), config
```

Everything about "cart" lives together (**colocation**: keep files that change together, close together — including tests and styles). Deleting a feature = deleting a folder.

3. Framework routes + features — in Next.js / React Router, route files stay thin and import from features:

```
app/
├── layout.jsx
├── products/[id]/page.jsx      ← fetch + compose: imports from features/catalog
└── cart/page.jsx
src/features/..., src/shared/...  (or inside app/ in private _folders)
```

Rules that keep it healthy

1. **Dependency direction:** `app → features → shared`. Shared code never imports features; features never import the app shell.
2. **Public APIs:** each feature exposes an `index.js`; others import `features/cart`, never `features/cart/store.js`. Internals can then change freely.
3. **Enforce, don't hope:** ESLint `no-restricted-imports` or `eslint-plugin-boundaries`, or path rules in your monorepo tool.
4. **Path aliases** (`@/features/cart`) instead of `../../../`: configure in `tsconfig`/`jsconfig` `paths` and your bundler.
5. **Be careful with barrel files** (`index.js` re-exporting everything): one per feature is fine; giant barrels that re-export a whole library slow dev servers and tests and can hurt tree-shaking and create import cycles.

Seeing a boundary check

`code/08_import_boundaries.mjs` scans the sample project in `code/feature-app/` and applies rules 1–2 — a miniature version of what a lint rule does:

```
if (fl === 'shared' && tl !== 'shared') problems.push([from, spec, 'shared/ must not import from ' + tl + '/']);
if (fl === 'features' && tl === 'app') problems.push([from, spec, 'features must not import app/']);
if (fl === 'features' && tl === 'features' && featureOf(from) !== featureOf(target)
    && target.split(path.sep).length > 2) {                    // deeper than features/<name>
  problems.push([from, spec, `deep import into feature "${featureOf(target)}" (use its index)`]);
}
```

Output:

```
src/
  app/
    App.jsx
  features/
    cart/
      components/
        CartPanel.jsx
      hooks/
        useCart.js
      index.js
      store.js
    catalog/
      components/
        ProductCard.jsx
        ProductList.jsx
      index.js
  shared/
    lib/
      formatPrice.js
    ui/
      Button.jsx

2 boundary violation(s):
  features/catalog/components/ProductList.jsx
    imports '../../cart/store.js' -> deep import into feature "cart" (use its index)
  shared/lib/formatPrice.js
    imports '../../features/cart' -> shared/ must not import from features/
```

`ProductCard.jsx` imports `useCart` from `'../../cart'` (the public index) and passes; `ProductList.jsx` reaches into `cart/store.js` and fails; `formatPrice.js` in `shared/` depends on a feature and fails. The equivalent ESLint rule (not run):

```
// eslint.config.js (excerpt)
rules: {
  'no-restricted-imports': ['error', { patterns: [
    { group: ['@/features/*/*'], message: 'Import features through their index (public API).' },
  ] }],
}
```

Common mistakes

- Designing a deep folder hierarchy on day one. Start flat; introduce `features/` when two or three features exist.
- A `shared/` or `utils/` folder that becomes a dumping ground — if only one feature uses it, it belongs in that feature.
- Circular imports between features — usually a sign that the shared piece belongs in `shared/` or that one feature should own it.

TL;DR

Small: by type. Growing: by feature with colocated tests/styles, an `index.js` public API per feature, `app → features → shared` direction, enforced by lint. Frameworks: thin route files that compose features.

14.12 Design systems

What it is

A **design system** is the shared set of decisions, components and documentation a product uses to look and behave consistently. In a React codebase it usually has three layers:

Layer	What it is	Example
Design tokens	Named values for design decisions: colours, spacing, radii, typography, shadows, motion	`--color-brand: #3553c9`, `--space-2: 8px`
Primitives / components	Accessible building blocks using the tokens	`Button`, `Tabs`, `Dialog`, `TextField`, `Menu`
Patterns and docs	How to combine them, when to use which, usage rules	"Destructive actions use `variant="danger"` and a confirm dialog"

Why it matters

Consistency (one Button, not 14), speed (teams assemble instead of rebuilding), accessibility solved once (focus, ARIA, keyboard in every `Menu`), and themeability (change tokens → rebrand or dark mode — Chapter 8 §8.8).

Tokens in practice

`code/ds.css` defines tokens as CSS custom properties and components use only tokens:

```
:root {
  --color-brand: #3553c9; --color-danger: #c62f3b; --color-border: #dde1e7;
  --radius-md: 8px; --space-1: 4px; --space-2: 8px; --space-3: 12px; --space-4: 16px;
  --focus-ring: 0 0 0 3px #9db0ff;
}
.btn-primary { background: var(--color-brand); color: var(--color-brand-text); }
.btn:focus-visible, .tab:focus-visible { outline: none; box-shadow: var(--focus-ring); }
.tab[aria-selected="true"] { color: var(--color-brand); border-bottom-color: var(--color-brand); }
```

Notice the Tabs style keys off **`aria-selected`** — styling from the accessibility state keeps visuals and semantics in sync.

A primitive: `Button`

`code/Button.jsx`:

```
const variants = { primary: 'btn-primary', secondary: 'btn-secondary', danger: 'btn-danger' };
const sizes = { sm: 'btn-sm', md: 'btn-md' };

export function Button({ variant = 'primary', size = 'md', className = '', type = 'button', ref, ...rest }) {
  return <button ref={ref} type={type} className={`btn ${variants[variant]} ${sizes[size]} ${className}`.trim()}
{...rest} />;
}
```

`code/09_design_system_button.jsx`:

```
<Button>Add to cart</Button>
<Button variant="secondary" size="sm" disabled>Save for later</Button>
<Button variant="danger" aria-label="Delete Trail Mug" ref={ref}>×</Button>
<Button type="submit" className="wide">Pay</Button>
```

Output:

```
<button type="button" class="btn btn-primary btn-md">Add to cart</button>
<button type="button" class="btn btn-secondary btn-sm" disabled="">Save for later</button>
<button type="button" class="btn btn-danger btn-md" aria-label="Delete Trail Mug">×</button>
<button type="submit" class="btn btn-primary btn-md wide">Pay</button>
ref points at the DOM node: true BUTTON
found by accessible name  : true
```

Design choices worth copying:

1. **A small, closed set of variants** (`primary | secondary | danger`) instead of free colours — the system decides what's allowed (typed as a union in TypeScript, Chapter 11 §11.9).
2. **Safe defaults:** `type="button"` avoids accidental form submits (a native `<button>` inside a form defaults to `submit`).
3. **Pass native props through** (`disabled`, `aria-*`, `onClick`) with `...rest`, and allow an extra `className` — callers never lose HTML features.
4. **`ref` is a normal prop in React 19** — no `forwardRef` needed, so focus management and popover positioning work (Chapter 5 §5.10).

Build or adopt?

Few teams should build complex widgets (menus, comboboxes, date pickers) from scratch — the keyboard and ARIA details take a long time to get right. Common approaches:

Approach	Examples	You get	You still do
Headless / unstyled primitives	Radix UI Primitives, React Aria (Adobe), Headless UI, Base UI	Behaviour, keyboard, ARIA, focus — no styles	All the visual design (with your tokens)
Copy-in components	shadcn/ui (source copied into your repo, built on headless primitives + Tailwind)	Styled, editable source you own	Maintenance of the copied code
Full component libraries	MUI, Chakra UI, Mantine, Ant Design	Styled, themed, many components	Theming to your brand; accept their look/API
Build your own	Large companies' internal systems	Full control	Everything, including accessibility and docs

A common modern stack: tokens (CSS variables or Tailwind theme) + headless primitives + your own styled wrappers (`Button`, `Dialog`) exported from `shared/ui`, documented in **Storybook** (a workshop app that renders each component in isolation with its states), and tested with Testing Library plus visual-regression tests.

API conventions for design-system components

- **Controlled + uncontrolled** (`value` / `defaultValue` / `onValueChange`) like the Tabs in §14.4.
- **Compound parts** for complex widgets (`Dialog.Title`, `Menu.Item`).
- **Composition over configuration** (§14.1): slots and children rather than dozens of props.
- **Polymorphism** when a component must render as another element: an `as` prop (`<Button as="a" href="/cart">`) or Radix's `asChild` (merge the component's behaviour onto its single child). Keep it limited — a `<Button as="div">` loses button semantics.
- **Accessible by default**: correct roles, names required for icon-only components, visible focus.

Common mistakes

- Letting every team add "just one more" variant — the system fragments. Changes go through the system's owners.
- Components that hard-code colours instead of tokens — dark mode and rebrands become rewrites.
- Wrapping a headless library and accidentally dropping its accessibility props (not spreading `...rest`, losing `ref`).

In the interview

30-second answer: "A design system is tokens — named colours, spacing, type — plus accessible primitives built on them, plus usage docs. In React I'd expose components like `Button` with a closed set of variants, safe defaults, native props and `ref` passed through, and compound APIs for complex widgets. Rather than hand-rolling menus and dialogs, I'd build on headless primitives like Radix or React Aria and style them with our tokens, document them in Storybook and test with Testing Library and visual regression."

TL;DR

Tokens → accessible primitives → documented patterns. Closed variants, safe defaults, pass-through props and `ref`. Build on headless primitives; style with tokens; document in Storybook.

14.13 Client architecture patterns: MVC, MVP, MVVM, VIPER — and where React fits

What it is

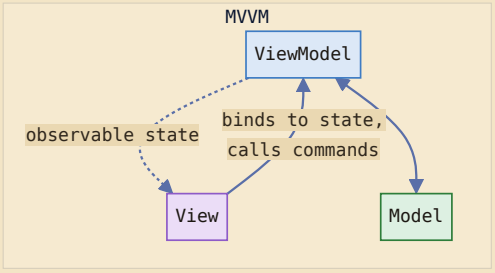
Sections 14.1–14.12 organised *components*. One level up, every UI codebase — web, iOS, Android, desktop — must answer the same question: **where does the logic live** between the data and the pixels? The classic answers have three-letter names. They all split a screen into a **Model** (data and business rules, no UI), a **View** (what the user sees) and a third part that connects them; they differ in **who talks to whom** and **how much the view knows**. You will hear these names in mobile interviews, in Angular/Vue/SwiftUI/Android codebases, and in "how does React compare?" questions.

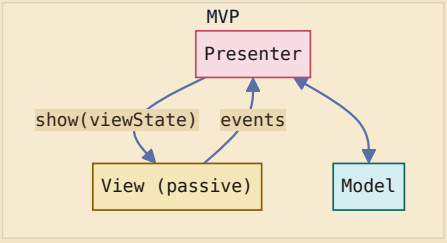
Analogy: a restaurant. The **model** is the kitchen and its recipes, the **view** is the table the guest sees. In **MVC** a head waiter takes orders and the table watches the kitchen's pass directly; in **MVP** a waiter carries every plate and arranges it (the table is passive); in **MVVM** the table has a display board that updates itself whenever the kitchen's state changes.

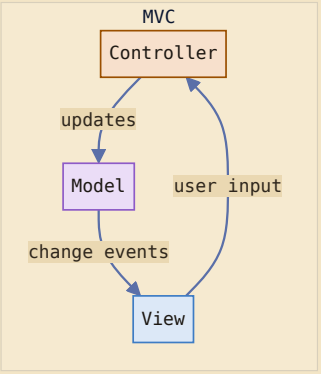
The problem it solves

Without a pattern, screens become "massive view controllers": network calls, formatting, validation, navigation and drawing in one class that only a UI test can exercise. Each pattern moves logic **out of the view** into something you can test with plain function calls, and makes each piece replaceable.

How it works — the four classics







	MVC (Smalltalk, 1979)	**MVP (1990s; Android before 2017)**	**MVVM (Microsoft WPF, 2005; Android Jetpack, SwiftUI, Vue, Angular)**	**VIPER (iOS, 2014)**
Parts	Model, View, Controller	Model, **passive** View, Presenter	Model, View, ViewModel	**V**iew, **I**nteractor, **P**resenter, **E**ntity, **R**outer
Who updates the view	the view observes the model (or, in web/iOS variants, the controller pushes)	the presenter calls methods on a view interface	the view **binds** to observable ViewModel state; the ViewModel never references the view	presenter → view
Business logic	model / controller	presenter + model	ViewModel + model	**Interactor** (use cases); presenter only formats; **Router** owns navigation
Testing without UI	controller and model	presenter, with a fake view	ViewModel, with no view at all	every part separately
Weak spot	controllers grow huge ("Massive View Controller" on iOS)	lots of view-interface boilerplate	binding "magic"; ViewModels can grow huge too	many files per screen; heavy for small apps

MVC was invented for Smalltalk-80 (Trygve Reenskaug); server frameworks (Rails, Spring MVC, ASP.NET MVC) reused the name for request → controller → model → rendered template, which is related but not the same thing. **MVP** makes the view as dumb as possible so the presenter can be unit-tested against a fake view. **MVVM** replaces the presenter's explicit `view.show(...)` calls with **data binding**: the view declares "this label shows `vm.total`", and the framework re-renders when it changes. **VIPER** is Clean Architecture applied to one iOS screen: use-case logic in the interactor, plain data entities, and a router for screen-to-screen navigation.

Two newer patterns matter even more for React:

- **Unidirectional data flow** (Facebook's **Flux**, 2014; **Redux**; the **Elm Architecture**): the view dispatches an **action**, a pure **reducer** computes the next state, and the view re-renders from state. Data moves in one direction only, so every change is traceable and replayable (Chapter 6 — `useReducer`, Redux Toolkit, Zustand).
- **MVI** (Model–View–Intent, Android/Kotlin): MVVM with a single immutable state object and reducer-style updates — essentially Redux on mobile.

How React fits

React began as "the V in MVC", but the way it is used today maps onto these patterns without their names:

Classic role	**In a React codebase**
View	function components returning JSX — ideally presentational, props in (14.2)
ViewModel / Presenter	a **custom Hook** that owns state and exposes display-ready values plus commands (14.3): `const { lines, totalText, add } = useCartViewModel()`
Binding	React re-renders from state — it is one-way binding with the view as a function of state, `UI = f(state)`
Model / store	server state in TanStack Query or a loader (Chapter 9); client state in `useReducer`, Context or an external store read with `useSyncExternalStore`
Controller / Interactor	event handlers, reducers and service modules; with Server Components and Server Actions (Chapter 13) some of it moves to the server
Router	React Router / Next.js routing

So when an interviewer asks "is React MVC or MVVM?", a good answer is: "React is a view library with one-way data flow. A custom Hook plays the ViewModel role, reducers or a store give Flux-style unidirectional updates, and the component is a pure view of that state."

Seeing it: `code/10_client_architectures.jsx`

One tiny cart ("Add Mug", "Add Tent", a total) built four ways — MVC (a model object the view subscribes to with `useSyncExternalStore`), MVP (a plain-JS presenter pushing view state into a passive view), MVVM (a `useCartViewModel` Hook) and a reducer — all rendering the same `CartView`. The same clicks give the same screen; then each architecture's logic is tested without rendering a component:

```
MVC     : Mug $8.50, Tent $120.00 | Total: $128.50
MVP     : Mug $8.50, Tent $120.00 | Total: $128.50
MVVM    : Mug $8.50, Tent $120.00 | Total: $128.50
Reducer : Mug $8.50, Tent $120.00 | Total: $128.50
MVP presenter test, fake view got   : {"lines":["Tent $120.00"],"totalText":"$120.00"}
MVVM view-model test, state         : {"lines":["Mug $8.50"],"totalText":"$8.50"}
Reducer test, replaying 2 actions   : 2 items
all assertions passed
```

The presenter is tested with a fake view object, the ViewModel Hook with `renderHook`, and the reducer as a pure function — the shared payoff of all these patterns.

When not to use it

- Don't bolt VIPER-style layering onto a five-screen app; a component + custom Hook per feature is enough.
- Don't copy MVP into React with imperative `view.show()` calls everywhere; React's state-driven rendering already gives you the binding MVP was simulating (the demo does it only for comparison).
- Don't put everything in one global store "because Redux"; keep server state in a query cache and local state local (Chapter 6).

Common mistakes

- **Saying React "is MVC"** → it is a view library; the patterns describe how you arrange state and logic around it.
- **ViewModels that know about the view** (DOM nodes, component refs) → they must expose state and commands only.
- **Fat views** → formatting (`$8.50`), validation and derived values belong in the Hook/presenter/selector, not inline JSX.
- **Two-way binding everywhere** → hard to trace who changed what; prefer one-way flow with explicit events.

In the interview

> **30-second answer:** "All of them separate Model and View and differ in the middle piece. In MVC a controller handles input and updates the model, and the view observes the model. In MVP a presenter does all the work and pushes formatted data into a passive view through an interface, so it is easy to unit-test. In MVVM the view binds to observable state on a ViewModel, which never references the view. VIPER splits an iOS screen into View, Interactor, Presenter, Entity and Router. React is a view library with one-way data flow: components are views, custom Hooks act as ViewModels, and reducers or stores give Flux-style unidirectional updates."

- *MVP vs MVVM?* → MVP's presenter holds a view reference and calls it; MVVM's ViewModel exposes state and the view binds to it, with no reference back.
- *Why did Flux/Redux appear?* → Two-way bindings and many models updating many views made changes hard to trace; a single direction (action → reducer → state → view) makes them predictable and replayable.

Self-check

1. Which pattern makes the view completely passive? → MVP.
2. In React, what usually plays the ViewModel role? → A custom Hook returning display-ready state and commands.
3. What does the R in VIPER own? → Navigation between screens (routing).
4. Why is a reducer easy to test? → It is a pure function: same state and action → same next state.

TL;DR

MVC: controller → model → view observes. MVP: presenter drives a passive view (testable with a fake view). MVVM: view binds to a ViewModel's observable state (no view reference). VIPER: View-Interactor-Presenter-Entity-Router for large iOS screens. Flux/Redux/MVI: one-way action → reducer → state → view. React = views as functions of state + custom Hooks as ViewModels + reducers/stores for unidirectional updates.

Sources: Trygve Reenskaug, "MVC — Xerox PARC 1978-79" notes; Martin Fowler, "GUI Architectures" (2006) and "Presentation Model" (2004); John Gossman, "Introduction to Model/View/ViewModel pattern" (Microsoft, 2005); objc.io issue 13, "Architecting iOS Apps with VIPER" (2014); Flux documentation (Meta); Redux documentation ("Three Principles"); Elm guide ("The Elm Architecture"); Android Developers "Guide to app architecture"; React docs (`useSyncExternalStore`, reusing logic with custom Hooks).

Chapter 14 TL;DR

Idea	Remember
Composition	`children` and slots beat boolean-prop explosions; specialise by wrapping
Container / presentational	Props-in views; data access in a Hook, container or Server Component — where it helps, not by rule
Custom Hook extraction	Logic → `useX(inputs)` with a small return API; injectable deps; test with `renderHook`
Compound components	Root owns state in a private context; parts read it; controlled + uncontrolled; throw on misuse
Render props	A function prop the component calls to render; best for lists/tables/slots; Hooks for logic
HOCs	`withX(Component)`; module level only; pass props through; `displayName`; mostly legacy
Portals	DOM elsewhere, React tree unchanged — context and event bubbling follow React; manage focus; consider `<dialog>`
Error boundaries	Class with `getDerivedStateFromError` + `componentDidCatch`; render errors only; reset keys; root `onCaughtError` / `onUncaughtError`
Accessibility	Semantic elements, `htmlFor` + `useId`, state-driven ARIA, live regions, focus with refs; test by role
Folder structure	By feature with public `index.js`; `app → features → shared`; enforce with lint
Design systems	Tokens + accessible primitives + docs; closed variants, safe defaults, pass-through props, headless libraries
Client architectures	MVC (view observes model), MVP (presenter drives passive view), MVVM (view binds to ViewModel), VIPER (use-case layers + router), Flux (one-way); React: custom Hook ≈ ViewModel, reducers = unidirectional

Next: Chapter 15 — React Interview and Machine Coding.

Chapter 15 — React Interview and Machine Coding

A front-end React interview usually has two technical parts. The **concepts round** asks "why" questions — why keys matter, why an Effect ran twice, why a memoized child still re-rendered — and listens for a precise mental model. The **machine-coding round** has you build a small widget (autocomplete, modal, tabs…) in 45–90 minutes while explaining your choices: how you split the problem, where state lives, whether the keyboard works, how you handle awkward cases.

Part one is **53 questions with short, correct answers** for React 19, each linked to the section that teaches it; the most-disputed answers are *proved* by a runnable program. Part two is **eight machine-coding builds**, done as in the room: requirements → component tree → state design → code → a React Testing Library + `user-event` test → a Chrome screenshot → follow-ups and edge cases.

What you'll learn

#	Topic	You will be able to…
15.1	How the rounds work	follow a repeatable method and time plan for a machine-coding round
15.2–15.9	53 interview questions	answer rendering, state, effects, refs, forms, context, performance, Suspense, transitions, Server Components, hydration, error boundaries, portals, testing and accessibility questions in 30 seconds
15.10	Todo list with filters	use a reducer, derived state, inline editing and `localStorage` persistence
15.11	Autocomplete	debounce input, abort stale requests, build the ARIA combobox keyboard pattern
15.12	Infinite scroll	load pages with `IntersectionObserver` and never fire duplicate requests
15.13	Modal dialog	render through a portal, trap focus, close on Escape, restore focus
15.14	Tabs	implement the ARIA tabs pattern with a roving `tabIndex` and arrow keys
15.15	Star rating	build on native radio buttons so keyboard and screen readers work for free
15.16	Pagination	write a pure page-range function with ellipses and a controlled pager
15.17	Multi-step form wizard	validate per step, wire errors to fields, move focus correctly
15.18	Other common prompts	map ten more prompts to the techniques in this chapter

Code: `code/` — React 19.3 in jsdom (a browser-like DOM that runs in Node). Each build is one component file (`todo.jsx`, `autocomplete.jsx`, …) plus a test file (`02_todo.test.jsx`, …) that uses Node's built-in test runner (`node:test`), React Testing Library and `@testing-library/user-event`, exactly like Chapter 12. The screenshots come from `code/machine-coding.html`: open it directly to see all eight builds, or add `#todo`, `#autocomplete`, `#infinite`, `#modal`, `#tabs`, `#stars`, `#pagination` or `#wizard` to see one. Run everything with `bash tools_run_js.sh react` from the repository root.

15.1 How React interviews are run

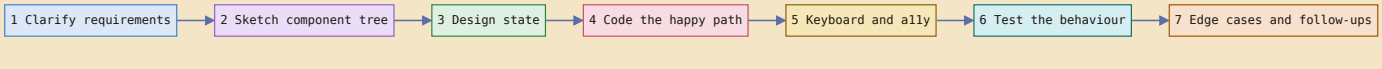
What it is

Most companies run some mix of these rounds for a React role:

Round	What happens	What is really being tested
Concepts / "React internals"	30–60 minutes of questions, often with a snippet: "what does this log?"	A correct mental model of rendering, state and effects
Machine coding (UI build)	Build a widget in 45–90 minutes, in a sandbox or your own editor	Decomposition, state design, working code, accessibility, testing, talking while coding
JavaScript	Closures, the event loop, `this`, and implement-from-scratch utilities such as `debounce`	The language under React — see JavaScript Handbook Chapter 17
Front-end system design	Design a news feed, a chat app, a design system	Data flow, caching, rendering strategy (CSR/SSR/RSC), performance, accessibility at scale

The machine-coding method

The same seven steps work for every build in this chapter. Say each step out loud; interviewers score the process, not only the final code.



1. **Clarify requirements** (3–5 min): what must work, then the quiet requirements — keyboard, screen readers, mobile, loading/error states, data size, browsers. Write a checklist; agree what's out of scope.
2. **Sketch the component tree** (2–3 min): who renders whom, props down, callbacks up.
3. **Design state** (3–5 min). List every piece of data, then ask of each: *can it be computed from something else?* (then it is derived, not state — §3.8); *who needs it?* (the closest common parent owns it — §3.7); *does it belong outside React?* (server data, URL, `localStorage`).
4. **Code the happy path** (20–30 min): something on screen early, then grow it; prefer native elements (`<button>`, `<input type="radio">`, `<form>`), which bring keyboard and screen-reader behaviour.
5. **Keyboard and accessibility** (5–10 min): Tab order, visible focus, Escape, arrows where the pattern expects them, labels, roles, `aria-*` state.
6. **Test** (5–10 min): one or two user-like tests — find by role and label, act with `user-event`, assert what the user sees (§12.4).
7. **Edge cases and follow-ups** (the rest): empty, loading, error, huge lists, fast typing, double clicks, unmount mid-request, "how would this scale?".

60-minute budget	Minutes
Requirements + tree + state	10
Happy path	25
Keyboard + a11y	8
Test	7
Edge cases, discussion	10

Common mistakes

- Coding before agreeing requirements, then building the wrong thing.
- Storing derived values in state (a `filteredTodos` array next to `todos`) and then fighting to keep them in sync.
- `<div onClick>` instead of `<button>`: not focusable, no Enter/Space, no role.
- Silence. The interviewer can't give credit for reasoning they didn't hear.
- Leaving no time to talk about edge cases — the part that separates "works" from "senior".

15.2 Interview questions: rendering, reconciliation and keys

How to use sections 15.2–15.9

Each answer takes about 30 seconds to say and links to the section that teaches it. Where an answer is often disputed, `code/01_interview_facts.jsx` proves it and its real output is shown. Answer aloud first, then check.

Q1. What happens, step by step, when you call a state setter? Nothing changes immediately. (1) **Trigger** — the update is queued and a re-render scheduled; (2) **render** — React calls the component (and its children) and compares the new element tree with the previous one (reconciliation); (3) **commit** — React applies only the differences to the DOM, runs layout effects, the browser paints, then `useEffect` Effects run. → §10.1, §1.3

Q2. What is the difference between the render phase and the commit phase? Which one must be pure? The **render phase** calls components to compute the UI. It must be **pure** (same inputs → same JSX, no side effects) because React may call it twice, pause it or discard it (concurrent rendering, StrictMode). The **commit phase** applies DOM changes and runs effects, once per committed update; side effects belong there or in event handlers. → §2.5, §10.1

Q3. What is reconciliation? Is "the virtual DOM" still the right way to describe React? Reconciliation compares the previous element tree with the next to find the DOM changes. A general tree diff is $O(n^3)$; React makes it $O(n)$ with two rules: **a different element type means a different subtree** (`<div>` → `<section>`, or `ComponentA` → `ComponentB`, destroys the old subtree and its state), and **keys identify list children** across renders. "Virtual DOM" is an older name for the in-memory element tree; today's docs speak of *rendering* and *committing*, and the idea that matters is identity by *type* + *position* + *key*. → §1.3, §3.10

Q4. Why do lists need keys, and why is the array index a bad key? A key tells React which item is which between renders, so the right DOM nodes and **state** move with it. With index keys, inserting at the top or re-sorting gives key `0` to a different item, which then inherits the first row's DOM and state (typed text, a checked box, focus). Use a stable ID from the data; index keys only for static lists. Never `Math.random()` — new keys every render remount every item. → §2.8

Q5. What does changing a component's `key` do? React treats it as a **different instance**: the old one unmounts (state gone, Effects cleaned up) and a new one mounts fresh — the idiomatic way to reset a form when the selected user changes: `<Profile key={userId} />`.

```
# a different key = a new component instance = fresh state
  Draft state initialised
  (same key: no new initialisation)
  Draft state initialised
```

→ §3.10

Q6. When does a component re-render? When its own **state** changes, its **parent re-renders** (children re-render by default, props changed or not), or a **context** it reads changes. "Props changed" is not a separate trigger — props change only because the parent re-rendered. A re-render with identical output commits nothing to the DOM. → §10.2

Q7. Why does `{items.length && <List />}` sometimes render a `0`? `&&` returns its left side when it is falsy, and React renders the number `0`. Use a boolean (`items.length > 0 && …`) or a ternary; `false`, `null`, `undefined` and `true` render nothing. → §2.7

15.3 Interview questions: state, events and forms

Q8. I call `setCount(count + 1)` three times in one click. Why does the count go up by one? State is a **snapshot**: within one render `count` is a constant, so all three calls queue "replace with 1". An **updater function** builds on the queued value: `setCount(c => c + 1)` three times gives +3. The handler still sees the old value after calling the setter.

```
# snapshot + batching: three setN(n + 1) vs three setN(m => m + 1)
  render n=0
  handler still sees n=0
  render n=1
  render n=4
```

One render per click — that is **batching** (Q9). → §3.3, §3.5

Q9. What is batching? Did React 18 change it? React applies all updates queued during an event in **one** re-render. Before React 18 only React event handlers were batched; since React 18 (`createRoot`) batching is **automatic everywhere** — timeouts, promises, native listeners. To force a synchronous DOM update (rare: scrolling to a just-added item, focusing a heading that must already show new text) use `flushSync` from `react-dom`. → §3.4

Q10. Why must state be updated immutably? React detects change with `Object.is(old, new)`; a mutated object is the same reference, so React may skip the render, and `memo` and dependency checks miss it too. Mutation also breaks

snapshots. Create new objects/arrays (spread, `map`, `filter`, `toSorted`, `with`) or use Immer for deep updates. → §3.6

Q11. What is "derived state", and what's wrong with copying props into state? Anything **computable** from props or state during render — a filtered list, a count, "is the form valid". Store the minimum and compute the rest, or two copies drift apart. `useState(prop)` uses only the prop's **first** value; use the prop directly, name it `initialX` to make that explicit, or reset with a `key`. → §3.8

Q12. Where should a piece of state live? As **low** as possible but **high enough** that all its users are below it — the closest common parent ("lifting state up"), passing down the value and a callback. Colocation keeps re-renders small; lifting avoids out-of-sync copies. → §3.7, §6.1

Q13. Controlled vs uncontrolled inputs — which should I use? Controlled: `value` from state plus `onChange`; React is the source of truth, so you can validate or format per keystroke. **Uncontrolled**: the DOM keeps the value (`defaultValue`) and you read it with a ref or `FormData` on submit — less code, fewer re-renders. Controlled when the UI reacts to each keystroke; uncontrolled for submit-and-read forms, especially with React 19 form Actions (`<form action={fn}>` receives `FormData` and resets uncontrolled fields after success). Don't switch one input between the two — starting with `value={undefined}` and later passing a string triggers a warning. → §4.2, §4.4

Q14. What do `useActionState`, `useFormStatus` and `useOptimistic` do?

- `useActionState(action, initialState)` → `[state, formAction, isPending]`: the action's return value (e.g. validation errors) becomes state.
- `useFormStatus()`, called in a component *inside* a `<form>`, → `{ pending, data, method, action }` — ideal for a reusable submit button.
- `useOptimistic(state, updateFn)` shows a temporary value while an async action runs, then falls back to the real state when it finishes or fails. → §4.5, §4.6, §4.7

Q15. How are React events different from DOM events? Handlers get a **synthetic event**, a cross-browser wrapper with the usual interface (`target`, `preventDefault()`, `stopPropagation()`). Since React 17 listeners sit on the **root container**, not `document`, and events propagate through the **React** tree (portals — Q49). `onChange` fires on every keystroke (like the DOM `input` event). Returning `false` doesn't prevent default; call `e.preventDefault()`. → §3.1

15.4 Interview questions: effects and refs

Q16. What is `useEffect` for — and when is it the wrong tool? Effects **synchronise a component with an external system**: a request tied to what's on screen, a subscription, a timer, a non-React widget, `document.title`. Wrong tool for computing from props/state (do it in render), reacting to a user action (event handler), resetting state on a prop change (`key`) or passing data up (lift state). Each unnecessary Effect costs an extra render and invites bugs. → §5.1, §5.6

Q17. What do no array, `[]` and `[a, b]` mean as the second argument? No array: after **every** commit. `[]`: after mount only (cleanup on unmount). `[a, b]`: after mount and whenever `a` or `b` changed (`Object.is`). Dependencies aren't a choice — every reactive value the Effect reads must be listed (`react-hooks/exhaustive-deps` checks). Objects or functions created during render are new each time and re-run the Effect every render; create them inside the Effect or memoize them. → §5.5

Q18. When does cleanup run, and in what order do Effects run? Cleanup runs **before the Effect re-runs** (seeing the *old* values of its own render) and on **unmount**. Children's Effects run before their parent's, and all layout effects before any passive `useEffect`.

```
# effect order: child before parent, layout before passive, cleanup sees OLD values
  child  layout setup   v=1
  child  effect setup   v=1
  parent effect setup   v=1
  --- re-render with v=2
  child  layout cleanup v=1
  child  layout setup   v=2
  child  effect cleanup v=1
  parent effect cleanup v=1
  child  effect setup   v=2
  parent effect setup   v=2
```

→ §5.3

Q19. Why does my Effect run twice, and my component render twice, in development? `<StrictMode>` (in most project templates) deliberately runs, **in development only**, component bodies, initialisers and updaters twice and each Effect's **setup → cleanup → setup** on mount (since React 19, ref callbacks too). It surfaces impure renders and missing cleanups: whatever breaks would also break on a real remount (navigation, `<Activity>`, fast refresh). Fix the cleanup — don't disable StrictMode or add "has run" flags. Production runs everything once.

```
# StrictMode (development only): render twice, effects setup -> cleanup -> setup
  effect setup
  effect cleanup
  effect setup
  component function called 2 times for one mount
```

→ §5.4

Q20. What race condition can happen when you fetch in an Effect, and how do you prevent it? If `userId` goes 1 → 2 quickly, the response for 1 can arrive **after** the one for 2 and overwrite it. In the cleanup, set an `ignore` flag the `.then` checks, or abort with an `AbortController` (`fetch(url, { signal })`). Better: TanStack Query or router loaders, which also cache and de-duplicate. Build 2 (§15.11) tests this race. → §5.7, §7.6

Q21. `useEffect` vs `useLayoutEffect`? `useLayoutEffect` runs after the DOM update but **before the browser paints**, so you can measure (a tooltip's height) and re-position without flicker. It blocks painting — use it only for measure-then-position; everything else goes in `useEffect`, which normally runs after paint. → §5.11

Q22. `useRef` vs `useState`? Both persist across renders. Setting state **re-renders**; changing `ref.current` does **not** and is visible immediately (no snapshot). Refs hold values that don't affect output — timer IDs, an `AbortController`, "request in flight" flags, DOM nodes. Don't read or write `ref.current` during render (except lazy initialisation): render must stay a pure function of props and state.

```
# changing ref.current does not re-render; state does
  render (ref=0)
  (clicked ref++ twice: no renders logged)
  render (ref=2)
```

→ §5.8, §5.9

Q23. How do you pass a ref to your own component in React 19? As an ordinary prop: `function Input({ ref, ...props }) { return <input ref={ref} {...props} /> }`. `forwardRef` is no longer needed (it still works; deprecation is planned). Ref callbacks may return a **cleanup function**. To expose a limited API (only `focus()`), use `useImperativeHandle`. → §5.10

Q24. What problem does `useEffectEvent` solve? An Effect sometimes needs the *latest* value without re-running when it changes — a chat connection that logs with the current theme, an interval calling the latest `onTick`. As a dependency it reconnects on every change; left out it goes stale. `useEffectEvent(fn)` (stable since React 19.2) returns a non-reactive function that always sees the latest props/state, callable only from Effects. → §7.7

15.5 Interview questions: hooks, context and state management

Q25. What are the Rules of Hooks, and why do they exist? Call Hooks only at the **top level** of components and custom Hooks — not in conditions, loops, nested functions or after an early `return`. React stores Hook state in a list matched **by call order**; if the order changes, state attaches to the wrong Hook. `use()` is the exception: allowed in `if` and loops (still only in components/Hooks). → §7.1, §7.2

Q26. What is a custom Hook? Do two components using the same Hook share state? A function named `use…` that calls other Hooks, to reuse **stateful logic** (`useDebouncedValue`, `useLocalStorage`). It shares logic, **not state**: each call has its own state, as if written inline. To share state, lift it or use context or a store. → §7.3

Q27. `useState` or `useReducer`? `useState` for simple independent values. `useReducer` when values change together, the next state depends on the previous in several ways, or you want the logic in one **pure, testable function** with named actions (`{ type: 'toggled', id }`) — easy to move into Redux or Zustand later. Build 1 uses one. → §6.2

Q28. Is Context a state-management tool? No — Context is a **transport** that passes a value to any depth; the state still lives in `useState`/`useReducer` in a provider. Good for theme, current user, locale, a reducer's `dispatch`. React 19 renders `<ThemeContext value={...}>` directly (`.Provider` still works). → §6.3, §6.5

Q29. What is the performance problem with Context, and how do you fix it? When a provider's `value` changes (`Object.is`), **every** consumer re-renders, `memo` or not. An inline `value={{ user, setUser }}` is a new object each render, so consumers re-render even when nothing changed:

```
# memo does not stop context updates; a new object value re-renders every consumer
  Label rendered (dark)
  Plain rendered
  --- parent re-renders, nothing really changed
  Label rendered (dark)
```

(`Plain` and `Label` are both memoized; only `Label` reads the context, and only it re-rendered.) Fixes: memoize the value (`useMemo`, or let the React Compiler); **split** contexts (state vs dispatch, fast vs slow); move state down; or use an external store with selectors (Zustand, Redux, `useSyncExternalStore`) for high-frequency shared state. → §6.4

Q30. How do you choose a state-management approach? First classify the state:

Kind	Put it in
UI state used by one component	`useState` / `useReducer`
Shared by a few nearby components	Lift to the common parent
Needed deep in the tree, changes rarely	Context (+ reducer)
Client state shared widely, changes often	Zustand or Redux Toolkit (selectors limit re-renders)
Server data (cache of the backend)	TanStack Query, router loaders, or Server Components
Shareable/bookmarkable (filters, page, tab)	The URL (search params)
Form fields	Form Actions, or a form library (React Hook Form) for big forms

→ §6.6, §6.10

Q31. Why treat server data differently from client state? Server data is a **cache** of something you don't own: it goes stale, is shared, and needs loading/error states, de-duplication, refetching, pagination and invalidation. Hand-writing that with `useEffect` + `useState` is subtle; TanStack Query (or loaders/RSC) solves it once. Keep `useState`/stores for true client state. → §6.9, §9.8

Q32. What is `useSyncExternalStore` for? Subscribing to a store **outside** React (`navigator.onLine`, a hand-written store, a library) safely under concurrent rendering — no "tearing", where parts of the screen show different versions. You pass `subscribe`, `getSnapshot` and optionally `getServerSnapshot`; Zustand and React Redux use it internally. → §7.9

15.6 Interview questions: performance and the React Compiler

Q33. What does `memo` do, and when doesn't it help? `memo(Component)` skips a re-render caused by the parent when every prop is `Object.is`-equal to last time. It doesn't help with new objects/functions each render (`style={{…}}`, `onClick={() => …}`), a changed context, the component's own state changes, or cheap components (the comparison costs too). → §10.4

Q34. `useMemo` vs `useCallback`? Both cache between renders until a dependency changes. `useMemo(() => compute(a), [a])` caches a **value**; `useCallback(fn, [a])` caches the **function** itself (= `useMemo(() => fn, [a])`). They are hints — React may drop the cache. Use them for measured slow calculations or when identity matters (a `memo` child, an Effect dependency). → §10.5, §10.6

Q35. What is the React Compiler, and does it make `memo`/`useMemo`/`useCallback` obsolete? A build-time tool (v1.0 stable since October 2025) that inserts memoization of values, callbacks and JSX automatically, more finely than people do by hand. With it, manual memoization is rarely needed; the Hooks remain escape hatches (e.g. to control an Effect's dependencies). It assumes the Rules of React (pure render, no mutation), skips code it can't prove safe, and doesn't fix slow algorithms, huge DOMs or big bundles. → §10.7

Q36. A page feels slow. How do you investigate? Measure first: React DevTools **Profiler** (what rendered, why, how long), the browser Performance panel, and a **production** build (dev is slower and StrictMode doubles renders). Usual causes: state too high (a keystroke re-renders the page), new objects breaking `memo`, over-eager context values, expensive render calculations, long lists, big bundles. → §10.3, §10.10

Q37. How does code splitting work in React? `const Chart = lazy(() => import('./Chart.jsx'))` puts `Chart` in a separate file downloaded on first render; `<Suspense fallback={…}>` covers the wait. Split by route first, then heavy rarely-used widgets (editors, charts). → §10.8

Q38. How do you render 10,000 rows? Virtualize: render only rows in or near the viewport, with spacers for the rest (TanStack Virtual, react-window). Or paginate (Build 7) or scroll infinitely (Build 3). Keep keys stable and rows cheap. → §10.9

15.7 Interview questions: Suspense, `use()` and transitions

Q39. What is Suspense, and what can "suspend"? `<Suspense fallback={<Spinner />}>` shows the fallback until everything inside is ready. What suspends: `lazy()` components, `use(promise)` on a pending promise, and Suspense-enabled sources (`useSuspenseQuery`, framework loaders, async Server Components) — **not** data fetched in `useEffect`. Nested boundaries let parts load independently. → §9.11

Q40. What does `use()` do, and how is it different from other Hooks? `use(resource)` reads a **promise** (suspends until it resolves; a rejection goes to the nearest error boundary) or a **context**, and may be called in `if` and loops. In a Client Component the promise must be **cached or created outside render** (from a Server Component, loader or cache) — one created during render is new each time, so the component suspends forever. → §9.11

Q41. `useTransition` vs `useDeferredValue`? Both mark work **non-urgent** so typing stays responsive. `useTransition` → `[isPending, startTransition]`: wrap a **state update you control** (tab switch, filter, navigation) and React renders it interruptibly in the background; in React 19 the function may be async (an "Action"). `useDeferredValue(value)` suits a value handed to you (a prop): the slow part renders with an older copy that catches up when React has time. Neither is for a controlled input's own value, which must update urgently. → §7.10, §7.11

Q42. When content is already on screen and a new update suspends, does the user see the fallback again? Not in a **transition**: React keeps the old content (dim it with `isPending`) until the new content is ready. Without a transition the nearest fallback reappears — which is why routers wrap navigations in transitions. → §9.11, §9.12

15.8 Interview questions: server rendering, hydration and Server Components

Q43. CSR, SSR, SSG, streaming — what's the difference? CSR: an almost empty page plus JS; the browser builds the UI. **SSR**: HTML rendered per request, visible before JS loads, then hydrated. **SSG**: HTML rendered once at build time, served as files. **Streaming SSR**: HTML sent in chunks as each `<Suspense>` boundary is ready, so a slow part doesn't hold back the page. → §1.4, §13.1, §13.3

Q44. What is hydration, and what causes a hydration mismatch? Hydration (`hydrateRoot`) makes server HTML interactive: React renders the same tree in the browser, **adopts** the existing DOM and attaches handlers, expecting the first client render to match the server exactly. Mismatches come from `Date.now()`, `Math.random()`, `window`/`localStorage` checks in render, locale formatting, invalid nesting (`<div>` in `<p>`) or extensions editing the DOM; React reports a recoverable error and client-renders that part. Fixes: render the same on both sides and switch after hydration (an Effect, or React 19.3's `use(browser())` inside Suspense); `useId` for IDs; `suppressHydrationWarning` only for a single unavoidable difference like a timestamp. → §13.4, §13.5

Q45. What are React Server Components, and how do they differ from Client Components? Server Components run only on the server (per request or at build), can be `async` and read data directly, and **ship no code** — only their output. No state, Effects, handlers or browser APIs. **Client Components** are the React you know: pre-rendered to HTML, then hydrated and run in the browser. `'use client'` marks the **boundary** — that module and its imports become client code. Server → Client props must be **serializable** (data, JSX, promises, Server Functions — not ordinary functions or class instances). A Client Component can't import a Server Component but can receive one as `children`. → §13.7, §13.8

Q46. What does `'use server'` mean? Is it how you mark a Server Component? No — Server Components are the default and need no directive. `'use server'` marks **Server Functions**: async functions that run on the server but are callable from the client (`<form action>`, or in a transition). Each becomes a network endpoint, so validate input and check authorisation inside, like any public API. → §4.8, §13.8

Q47. What goes wrong if you put `'use client'` at the top of the root layout? Everything it imports becomes client code: no zero-JS static parts, no direct data access, a bigger bundle. Push `'use client'` down to interactive **leaves** (a like button, a search box); keep pages and layouts as Server Components. → §13.8, §13.9

15.9 Interview questions: error boundaries, portals, patterns, testing and accessibility

Q48. What do error boundaries catch — and what don't they? A **class** component with `static getDerivedStateFromError` (show a fallback) and `componentDidCatch` (log); there's no Hook version, so many apps use `react-error-boundary`. It catches errors thrown while **rendering**, in lifecycle methods and in Effects **below** it — **not** in event handlers, async code you started (`setTimeout`, a `.then` outside render), server rendering, or itself. Handle handler errors with `try/catch` + state; to route an async error to a boundary, throw it during render (a rejected `use(promise)`) or inside `startTransition`. React 19 adds root `onCaughtError` / `onUncaughtError` for reporting.

```
# error boundaries catch render errors, not event-handler errors
  render error -> Fallback: render failed
  click error -> reached window 'error' event: click failed
  boundary fallback shown after the click? false
```

→ §14.9, §9.12

Q49. What is a portal, and how do events behave inside one? `createPortal(children, domNode)` renders into a **different DOM node** (usually `document.body`) while keeping the same place in the **React** tree — for modals, tooltips and toasts that must escape `overflow: hidden` or a stacking context. Context still flows in, and events **bubble through React parents**, not DOM parents:

```
# events from a portal bubble through the REACT tree, not the DOM tree
  DOM parent of the button: <body>, inside the React parent's div? false
  React parent onClick fired
```

→ §14.8, Build 4 (§15.13)

Q50. Compound components, render props, HOCs, custom Hooks — when would you use each? Custom Hooks — the default for sharing *logic*. **Compound components** (`<Tabs><Tabs.List/><Tabs.Panel/></Tabs>`, state via context) — flexible *markup*, common in design systems. **Render props** (a function prop) — the caller decides what to render with the component's data; still handy for lists and virtualization. **HOCs** (`withAuth(Page)`) — wrap a component to inject props; mostly replaced by Hooks but common in older code. → §2.4, §14.4–§14.7

Q51. What is the philosophy of React Testing Library? The closer a test is to how people use the UI, the more confidence it gives. Find elements as a user or screen reader would — **role and accessible name first** (`getByRole('button', { name: 'Save' })`), then label and text, `data-testid` last — act with `user-event` (real key presses, focus, clicks) and assert on what's visible. Skip implementation details (state, Hook calls, class names) so tests survive refactors. Use `findBy…`/`waitFor` for async UI; fake the network at the boundary (a function prop, or MSW), not your own modules. → §12.1, §12.3, §12.4, §12.6

Q52. How do you make React components accessible? Semantic HTML first (`<button>`, `<a href>`, `<label>`, `<fieldset>`, headings, `<dialog>`) — roles, focus and keyboard for free. Label every input (`htmlFor` + `useId`). Manage **focus** on big changes (into a dialog and back, to a new step's heading, to the first invalid field). Support expected keys (Escape closes, arrows inside tabs/menus/listboxes). ARIA only for what HTML can't say (`aria-expanded`, `aria-current`, `aria-invalid`, `aria-describedby`, live regions) — no ARIA beats wrong ARIA. Check with `eslint-plugin-jsx-a11y`, role-based tests, a keyboard and a screen reader. → §7.8, §14.10, HTML & CSS Chapter 3

Q53. How would you type a component's props, children and events in TypeScript? Props with a `type`/`interface` and destructuring (`function Button({ variant = 'primary', ...rest }: ButtonProps)`); children as `React.ReactNode`; native props via `React.ComponentProps<'button'>`; events as `React.ChangeEvent<HTMLInputElement>` or inferred from inline handlers; refs as `useRef<HTMLInputElement>(null)`; mutually exclusive props as **discriminated unions**. → §11.2, §11.4, §11.9

Self-check (15.2–15.9)

1. A child component is wrapped in `memo`, receives no props, and reads a context whose provider passes `value={{ theme }}`. Its parent re-renders. Does the child re-render? → Yes: the provider creates a new object, so the context value changed by `Object.is`, and context updates go around `memo` (Q29).
2. Why is `key={Math.random()}` worse than `key={index}`? → Index keys only misbehave when the list reorders; random keys remount every item on every render, losing state and focus and wasting work (Q4).
3. What single change stops a stale fetch response from overwriting a newer one? → Cleanup in the Effect that aborts the request or sets an `ignore` flag (Q20).

4. Can a Server Component pass `onClick={() => …}` to a Client Component? → No — ordinary functions aren't serializable. Pass data, or a Server Function marked `'use server'` (Q45–Q46).
5. An error thrown in an `onClick` handler — which error boundary shows it? → None; boundaries only catch errors during rendering, lifecycles and Effects. Catch it in the handler and put the error in state (Q48).

15.10 Build 1 — Todo list with filters

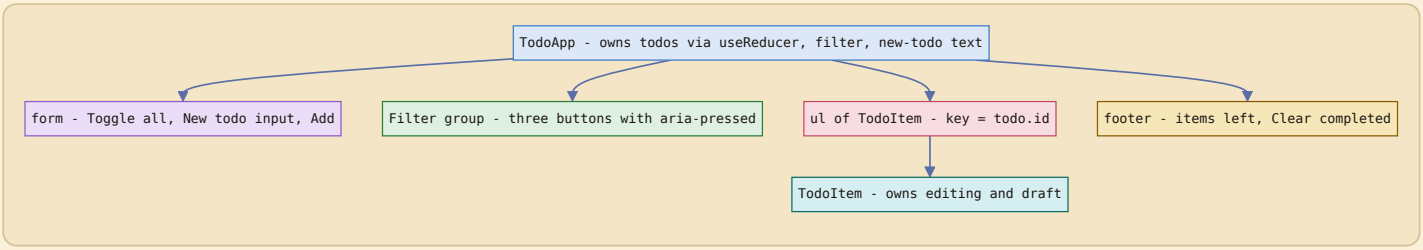
Requirements

The prompt: *"Build a todo app: add, complete, delete, filter by All / Active / Completed."* Clarifying questions turn that into:

Functional	Quiet requirements (ask about them)
Add a todo (Enter or button); ignore blank input	Keyboard-only use works; every control has an accessible name
Toggle done; delete	Survives a page reload (`localStorage`)
Filter: all / active / completed	Counter text is announced ("2 items left")
Edit a title (double-click or ✎); Enter saves, Escape cancels, empty deletes	Toggle-all and "Clear completed"

A simpler version of this app (no reducer, no editing) is built step by step in §3.11; this is the interview-grade version.

Component tree



State design

Data	Kind	Owner	Why
`todos`	state (`useReducer`)	`TodoApp`	many kinds of change → one pure reducer; the list and the counter both need it
`filter`	state	`TodoApp`	UI choice (a follow-up could move it to the URL)
`text`	state	`TodoApp`	controlled "New todo" input
`editing`, `draft`	state	each `TodoItem`	only that row cares — colocated, so typing re-renders one row
`visible`, `left`	**derived**	computed in render	never stored, so never out of sync
next ID	ref	`TodoApp`	needed by handlers only, never displayed

Code

`code/todo.jsx`:

```jsx
// Machine coding 1 — todo list with filters, inline editing and localStorage persistence.
import { useEffect, useReducer, useRef, useState } from 'react';

export const FILTERS = {
  all: () => true,
  active: (t) => !t.done,
  completed: (t) => t.done,
};

// All todo changes live in one pure function: easy to read, easy to unit-test.
export function todosReducer(todos, action) {
  switch (action.type) {
    case 'added':
      return [...todos, { id: action.id, title: action.title, done: false }];
    case 'toggled':
      return todos.map((t) => (t.id === action.id ? { ...t, done: !t.done } : t));
    case 'edited':
      return todos.map((t) => (t.id === action.id ? { ...t, title: action.title } : t));
    case 'deleted':
      return todos.filter((t) => t.id !== action.id);
    case 'toggledAll': {
      const allDone = todos.every((t) => t.done);
      return todos.map((t) => ({ ...t, done: !allDone }));
    }
    case 'clearedCompleted':
      return todos.filter((t) => !t.done);
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

function loadTodos(storageKey) {
  try {
    return JSON.parse(localStorage.getItem(storageKey)) ?? [];
  } catch {
    return [];                                  // corrupt JSON or storage blocked
  }
}

function TodoItem({ todo, dispatch }) {
  const [editing, setEditing] = useState(false);
  const [draft, setDraft] = useState(todo.title);
  const inputRef = useRef(null);
  const cancelled = useRef(false);              // Escape must win over the blur that follows

  useEffect(() => {
    if (editing) { inputRef.current.focus(); inputRef.current.select(); }   // ready to type over
  }, [editing]);

  function startEdit() {
    cancelled.current = false;
    setDraft(todo.title);
    setEditing(true);
  }

  function finish() {
    if (cancelled.current) { cancelled.current = false; return; }
    const title = draft.trim();
    if (!title) dispatch({ type: 'deleted', id: todo.id });   // emptying a todo deletes it
    else dispatch({ type: 'edited', id: todo.id, title });
    setEditing(false);
  }

  if (editing) {
    return (
      <li className="editing">
        <input
          ref={inputRef}
          aria-label={`Edit ${todo.title}`}
          value={draft}
```

```
          onChange={(e) => setDraft(e.target.value)}
          onBlur={finish}
          onKeyDown={(e) => {
            if (e.key === 'Enter') finish();
            if (e.key === 'Escape') { cancelled.current = true; setEditing(false); }
          }}
        />
      </li>
    );
  }
  return (
    <li className={todo.done ? 'done' : undefined}>
      <input
        type="checkbox"
        aria-label={todo.title}
        checked={todo.done}
        onChange={() => dispatch({ type: 'toggled', id: todo.id })}
      />
      <span className="title" onDoubleClick={startEdit}>{todo.title}</span>
      <button className="icon" aria-label={`Edit ${todo.title}`} onClick={startEdit}>✎</button>
      <button className="icon" aria-label={`Delete ${todo.title}`} onClick={() => dispatch({ type: 'deleted', id:
todo.id })}>×</button>
    </li>
  );
}

export default function TodoApp({ storageKey = 'todos' }) {
  const [todos, dispatch] = useReducer(todosReducer, storageKey, loadTodos);   // lazy init: read storage once
  const [filter, setFilter] = useState('all');
  const [text, setText] = useState('');
  const nextId = useRef(Math.max(0, ...todos.map((t) => t.id)) + 1);

  useEffect(() => {
    localStorage.setItem(storageKey, JSON.stringify(todos));                  // sync to an external system
  }, [storageKey, todos]);

  const visible = todos.filter(FILTERS[filter]);          // derived — never stored
  const left = todos.filter((t) => !t.done).length;

  function handleSubmit(e) {
    e.preventDefault();
    const title = text.trim();
    if (!title) return;
    dispatch({ type: 'added', id: nextId.current++, title });
    setText('');
  }

  return (
    <section className="card todo" aria-labelledby="todo-h">
      <h2 id="todo-h">Todos</h2>
      <form onSubmit={handleSubmit} className="row">
        {todos.length > 0 && (
          <button type="button" className="icon" aria-label="Toggle all" onClick={() => dispatch({ type: 'toggledAll'
})}>⌄</button>
        )}
        <input aria-label="New todo" placeholder="What needs to be done?" value={text} onChange={(e) =>
setText(e.target.value)} />
        <button>Add</button>
      </form>

      <div className="row" role="group" aria-label="Filter">
        {Object.keys(FILTERS).map((name) => (
          <button key={name} className="chip" aria-pressed={filter === name} onClick={() => setFilter(name)}>
            {name}
          </button>
        ))}
      </div>

      {visible.length === 0
        ? <p className="muted">{todos.length === 0 ? 'Nothing to do yet.' : `No ${filter} todos.`}</p>
        : <ul className="list">{visible.map((t) => <TodoItem key={t.id} todo={t} dispatch={dispatch} />)}</ul>}
```

```
      <footer className="row between">
        <span role="status">{left} {left === 1 ? 'item' : 'items'} left</span>
        {left < todos.length && <button onClick={() => dispatch({ type: 'clearedCompleted' })}>Clear
completed</button>}
      </footer>
    </section>
  );
}
```

What to point out while coding:

- **Reducer** actions name *what happened* (`added`, `toggled`); the reducer is pure, so it's unit-tested without rendering (first test below).
- **Lazy initialiser**: `useReducer(reducer, storageKey, loadTodos)` reads `localStorage` once. The **Effect** writes back after each change — syncing with an external system, exactly what Effects are for (§7.4 makes it `useLocalStorage`).
- **Keys** are IDs, so filtering never moves the edit box to another item.
- **Escape vs blur**: removing the focused edit input can fire `blur`, which would call `finish()` and save; the `cancelled` ref makes Escape win.

Test

`code/02_todo.test.jsx` (key parts):

```
test('edit: Enter saves, Escape cancels, empty text deletes', async () => {
  const user = userEvent.setup();
  render(<TodoApp />);
  await user.type(screen.getByRole('textbox', { name: 'New todo' }), 'Wash car{Enter}');

  await user.dblClick(screen.getByText('Wash car'));
  const edit = screen.getByRole('textbox', { name: 'Edit Wash car' });
  assert.equal(document.activeElement, edit);               // focused automatically
  await user.clear(edit);
  await user.type(edit, 'Wash bike{Enter}');
  assert.deepEqual(titles(), ['Wash bike']);

  await user.click(screen.getByRole('button', { name: 'Edit Wash bike' }));
  await user.type(screen.getByRole('textbox', { name: 'Edit Wash bike' }), ' NOPE{Escape}');
  assert.deepEqual(titles(), ['Wash bike']);               // unchanged
  // ... clearing the text and pressing Enter deletes the todo
});

test('persists to localStorage and restores on the next mount', async () => {
  // ... add "Survive reload", tick it, unmount
  render(<TodoApp storageKey="t1" />);                     // a "reload"
  const item = screen.getByRole('listitem');
  assert.equal(within(item).getByRole('checkbox').checked, true);
  console.log('    stored JSON:', localStorage.getItem('t1'));
});
```

Output:

```
✔ reducer: toggledAll ticks everything, or unticks if all are done
✔ adds with Enter, ignores blank input, clears the box
✔ toggle + filters + clear completed
✔ edit: Enter saves, Escape cancels, empty text deletes
    stored JSON: [{"id":1,"title":"Survive reload","done":true}]
✔ persists to localStorage and restores on the next mount
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

In the browser

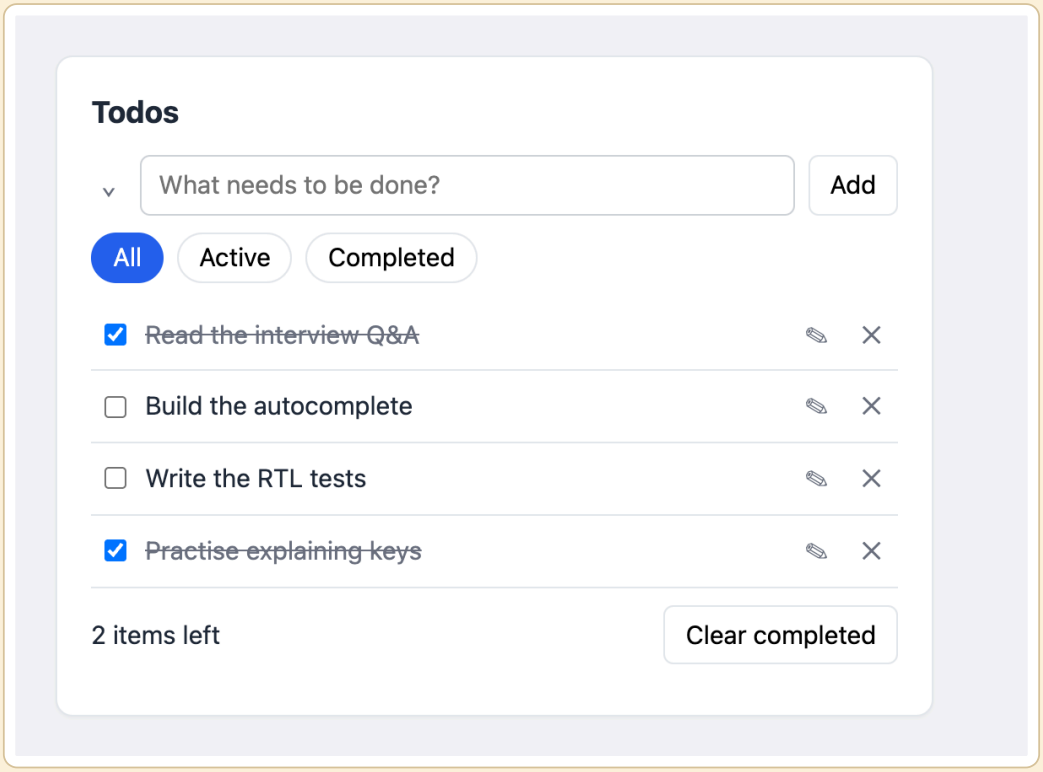


Figure 15.1 — `TodoApp` in Chrome, restored from `localStorage`. The strike-through, the count and the "Clear completed" button are all derived from the one `todos` array.

Follow-ups and edge cases

- **Persist to a server** → send each change to the API, showing it at once with `useOptimistic` and rolling back on failure (§4.7), or a TanStack Query mutation with invalidation (§9.10).
- **Shareable filter** → keep it in the URL (`?filter=active`) via the router's search params (Chapter 9).
- **Two tabs open** → listen for the `storage` event (fired in *other* tabs) and dispatch a `replaced` action, or subscribe with `useSyncExternalStore`.
- **10,000 todos** → virtualize; `memo(TodoItem)` with stable callbacks (or the React Compiler) stops every row re-rendering as you type.
- **Undo** → a `past` stack in the reducer state; `undo` pops it.
- **IDs** → `crypto.randomUUID()` in real apps; a counter ref is fine in an interview if you say why.

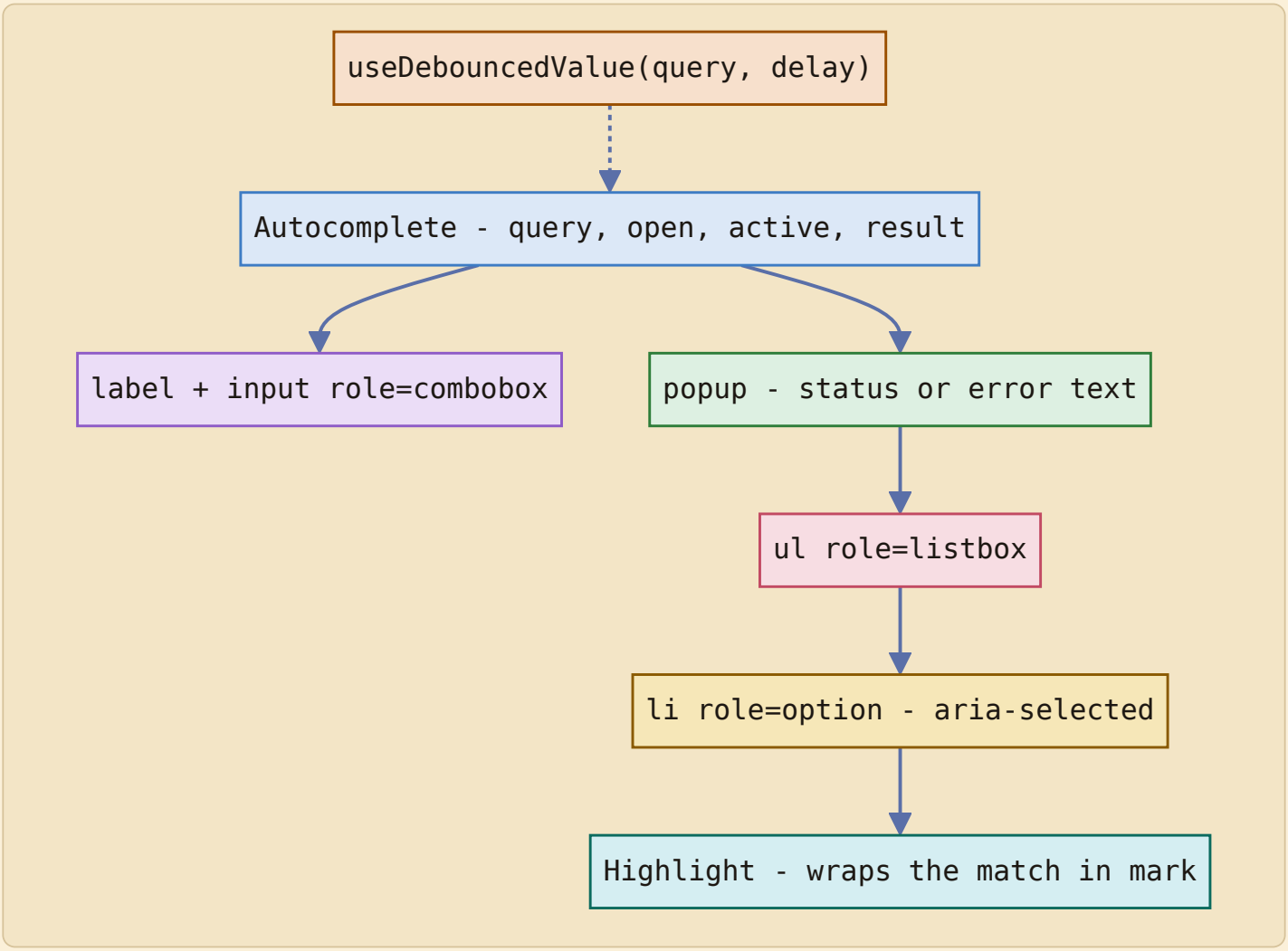
15.11 Build 2 — Autocomplete (debounce, abort, keyboard)

Requirements

Functional	Quiet requirements
Suggestions appear as the user types	Don't call the API on every keystroke — **debounce** (300 ms)
Click or Enter selects a suggestion	Ignore **stale responses** (a slow answer for "ap" must not replace "apr")
ArrowUp/ArrowDown move the highlight, wrapping at the ends; Escape closes	Screen readers announce the list and the highlighted option (ARIA **combobox** pattern)
Loading, empty and error states	The matching part of each suggestion is highlighted

The data source is a prop, `search(query, { signal })` → promise of strings, so the component is API-independent and easy to test.

Component tree



State design

Data	Kind	Why
`query`	state	controlled input — must update on every keystroke (urgent)
`debounced`	state inside `useDebouncedValue`	the value that actually triggers a search, 300 ms after typing stops
`result` = `{ status, items, for }`	state	one object so status and items can't disagree; `for` remembers which query the items belong to
`open`	state	the user can close the list (Escape, blur, selection) without clearing the query
`active`	state	index of the highlighted option, `-1` for none
`showList`, `items` shown	derived	e.g. no list when the query is shorter than `minChars`
option IDs	`useId`	unique on the page, stable between server and client

Debounce — why a Hook, not a `debounce()` wrapper? A debounced *function* (JavaScript Chapter 17 — Debounce, Throttle) must survive re-renders (`useRef` / `useMemo`) and be cancelled on unmount. Debouncing the *value* in an Effect is shorter: each keystroke's cleanup clears the previous timeout (§7.5).

Code

`code/autocomplete.jsx`:

```
import { useEffect, useId, useState } from 'react';

export function useDebouncedValue(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);                  // a new keystroke cancels the pending update
  }, [value, delay]);
  return debounced;
}

function Highlight({ text, query }) {
  const i = text.toLowerCase().indexOf(query.toLowerCase());
  if (i === -1 || !query) return text;
  return <>{text.slice(0, i)}<mark>{text.slice(i, i + query.length)}</mark>{text.slice(i + query.length)}</>;
}

export default function Autocomplete({ search, onSelect, label = 'Search', delay = 300, minChars = 1 }) {
  const [query, setQuery] = useState('');
  const [open, setOpen] = useState(false);
  const [active, setActive] = useState(-1);                // highlighted option index, -1 = none
  const [result, setResult] = useState({ status: 'idle', items: [], for: '' });
  const debounced = useDebouncedValue(query.trim(), delay);
  const id = useId();
  const listId = `${id}-list`;

  useEffect(() => {
    if (debounced.length < minChars) return;
    const controller = new AbortController();
    setResult((r) => ({ ...r, status: 'loading' }));
    search(debounced, { signal: controller.signal })
      .then((items) => { setResult({ status: 'success', items, for: debounced }); setActive(-1); })
      .catch((err) => { if (err.name !== 'AbortError') setResult({ status: 'error', items: [], for: debounced }); });
    return () => controller.abort();                       // newer query (or unmount) cancels this one
  }, [debounced, minChars, search]);

  const tooShort = query.trim().length < minChars;
  const items = tooShort ? [] : result.items;
  const showList = open && !tooShort;

  function choose(item) {
    setQuery(item);
    setOpen(false);
    setActive(-1);
    onSelect?.(item);
  }

  function handleKeyDown(e) {
    if (e.key === 'ArrowDown' || e.key === 'ArrowUp') {
      e.preventDefault();                                  // keep the caret where it is
      setOpen(true);
      if (items.length === 0) return;
      const step = e.key === 'ArrowDown' ? 1 : -1;
      setActive((i) => (i + step + items.length) % items.length);   // wraps around both ends
    } else if (e.key === 'Enter' && showList && active >= 0) {
      e.preventDefault();                                  // don't submit a surrounding form
      choose(items[active]);
    } else if (e.key === 'Escape') {
      setOpen(false);
      setActive(-1);
    }
  }

  return (
    <div className="card ac">
      <label htmlFor={`${id}-input`}>{label}</label>
      <input
        id={`${id}-input`}
        role="combobox"
        aria-expanded={showList}
```

```
        aria-controls={listId}
        aria-autocomplete="list"
        aria-activedescendant={showList && active >= 0 ? `${id}-opt-${active}` : undefined}
        autoComplete="off"
        value={query}
        onChange={(e) => { setQuery(e.target.value); setOpen(true); setActive(-1); }}
        onKeyDown={handleKeyDown}
        onBlur={() => setOpen(false)}
      />
      {showList && (
        <div className="popup">
          {result.status === 'loading' && <p className="muted" role="status">Searching…</p>}
          {result.status === 'error' && <p className="error" role="alert">Something went wrong. Keep typing to retry.
</p>}
          {result.status === 'success' && items.length === 0 && <p className="muted" role="status">No matches for
“{result.for}”</p>}
          <ul id={listId} role="listbox" aria-label={`${label} suggestions`}>
            {items.map((item, i) => (
              <li
                key={item}
                id={`${id}-opt-${i}`}
                role="option"
                aria-selected={i === active}
                onMouseDown={(e) => e.preventDefault()}     // keep focus in the input so blur doesn't close first
                onMouseEnter={() => setActive(i)}
                onClick={() => choose(item)}
              >
                <Highlight text={item} query={result.for} />
              </li>
            ))}
          </ul>
        </div>
      )}
    </div>
  );
}
```

How the tricky parts work:

1. **Debounce → search**: `query` changes every keystroke; `debounced` only after `delay` ms of quiet, and only then does the search Effect run.
2. **Abort**: each search Effect owns an `AbortController`. When `debounced` changes, React runs the old Effect's **cleanup** first, aborting the old request (its `AbortError` is ignored), so an old response can never overwrite a newer one (§5.7). Note the ordering: the abort happens when the *debounced* value changes, not on the keystroke — a response for "ap" that lands during the pause after typing "r" is still shown (labelled by `result.for`) until the "apr" results replace it.
3. **Focus stays in the input.** Options are never focused; `aria-activedescendant` names the active one (the WAI-ARIA combobox pattern), so the user keeps typing.
4. **Mouse vs blur.** Clicking an option would blur the input and close the list *before* the click lands; `onMouseDown={e => e.preventDefault()}` prevents the blur.
5. **Enter** selects only when an option is highlighted, with `preventDefault()` so a surrounding `<form>` doesn't submit.

Test

`code/03_autocomplete.test.jsx` uses a fake `search` whose requests the test resolves **by hand**, so it can make answers arrive out of order:

```
function fakeSearch() {
  const calls = [];
  const search = (q, { signal }) => new Promise((resolve, reject) => {
    const call = { q, signal, resolve: () => resolve(FRUITS.filter((f) => f.toLowerCase().includes(q.toLowerCase())))
};
    signal.addEventListener('abort', () => reject(new DOMException('aborted', 'AbortError')));
    calls.push(call);
  });
  return { search, calls };
}

test('abort: a newer query cancels the older request, so stale results never show', async () => {
  const user = userEvent.setup();
  const { search, calls } = fakeSearch();
  render(<Autocomplete search={search} delay={50} label="Fruit" />);
  const box = screen.getByRole('combobox');
  await user.type(box, 'ap');
  await waitFor(() => assert.equal(calls.length, 1));      // request #1 ("ap") is in flight
  await user.type(box, 'r');
  await waitFor(() => assert.equal(calls.length, 2));      // request #2 ("apr") started
  assert.equal(calls[0].signal.aborted, true);             // ...and #1 was aborted by the effect cleanup
  calls[1].resolve();                                      // the answers arrive OUT of order:
  await waitFor(() => assert.deepEqual(options(), ['Apricot']));
  calls[0].resolve();                                      // the late answer for "ap" must be ignored
  await new Promise((r) => setTimeout(r, 20));
  assert.deepEqual(options(), ['Apricot']);
});

test('keyboard: arrows move (and wrap), Enter selects, Escape closes', async () => {
  // ... type "ap", resolve, 5 options
  await user.keyboard('{ArrowDown}{ArrowDown}');
  const second = screen.getAllByRole('option')[1];
  assert.equal(box.getAttribute('aria-activedescendant'), second.id);   // focus stays in the input
  await user.keyboard('{ArrowUp}{ArrowUp}');                // 1 -> 0 -> wraps to the last
  await user.keyboard('{ArrowDown}{Enter}');                // wraps to the first, selects it
  assert.deepEqual(picked, ['Apple']);
  // ... Escape closes and sets aria-expanded="false"
});
```

Output:

```
✔ debounce: typing quickly sends ONE request, for the final text
✔ abort: a newer query cancels the older request, so stale results never show
✔ keyboard: arrows move (and wrap), Enter selects, Escape closes
✔ mouse: clicking an option selects it (mousedown does not blur the input first)
✔ empty and error states
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

The tests pass short real delays (10–100 ms) as a prop, so no fake timers are needed.

In the browser

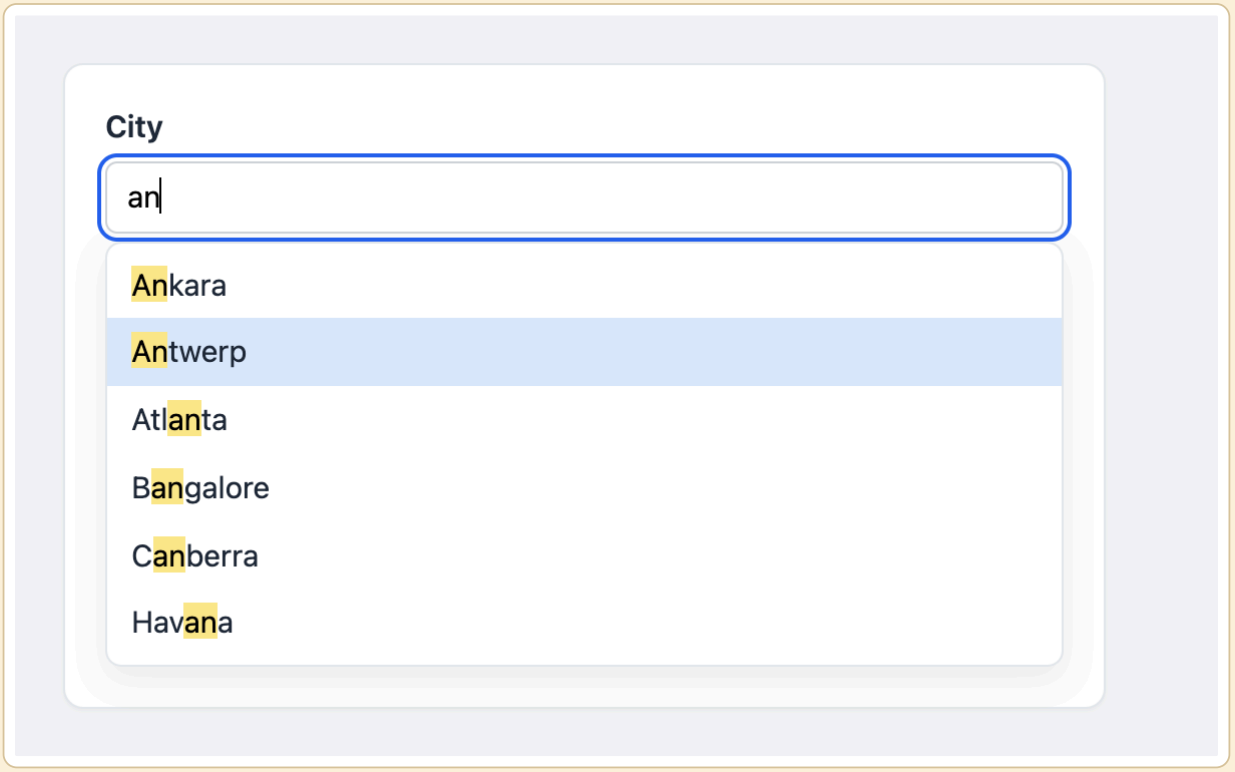


Figure 15.2 — After typing "an" and pressing ArrowDown twice. The matching text is wrapped in `<mark>`; the blue row is the option named by `aria-activedescendant`, while keyboard focus stays in the input.

Follow-ups and edge cases

- **Cache results** → a `Map` of query → results in a ref (or TanStack Query keyed by the query), so Backspace shows earlier results instantly.
- **Why not throttle?** → throttle fires at most once per interval *while* typing (scroll/resize); debounce waits for a pause (search). Both are in JavaScript §17.3.
- **Huge result lists** → cap results (top 10) or render them from `useDeferredValue(query)` (§7.11).
- **Active option out of view** → in an Effect on `active`, `scrollIntoView({ block: 'nearest' })` the option.
- **Selecting searches the chosen text** → harmless here (the list is closed), or skip when `query` equals the last selection.
- **Highlighting user input** → never `dangerouslySetInnerHTML`; splitting the string and rendering `<mark>` in JSX, as `Highlight` does, is XSS-safe.
- **Rate limits** → debounce plus `minChars`, caching and server-side limits.

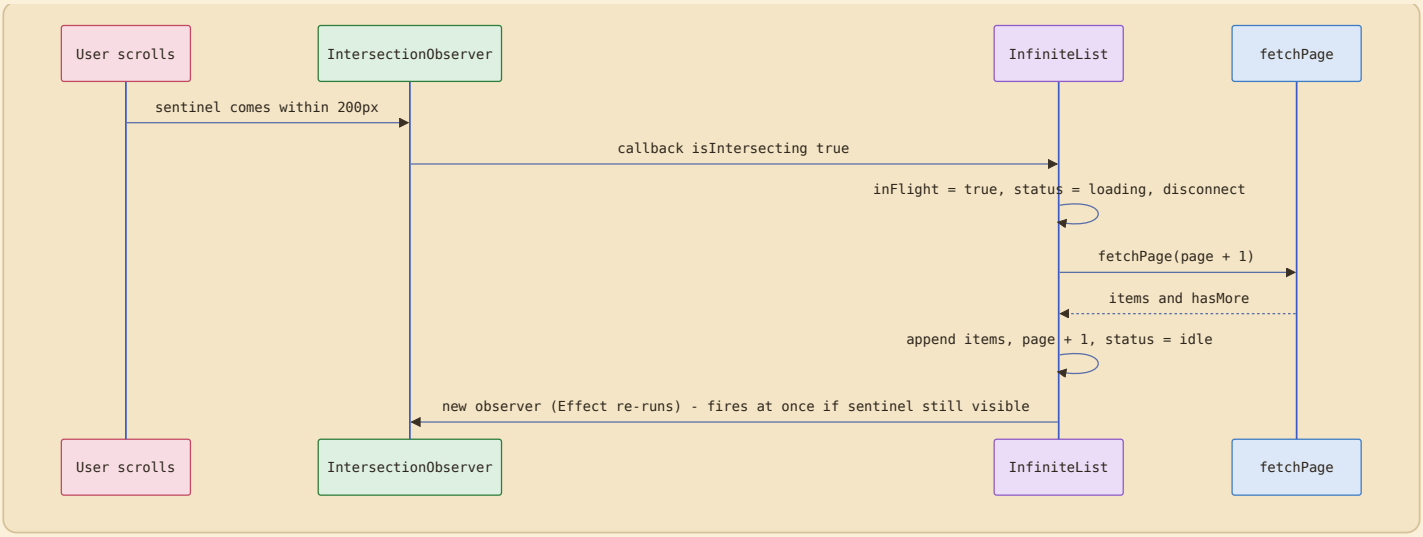
15.12 Build 3 — Infinite scroll

Requirements

Functional	Quiet requirements
Show the first page, load the next when the user nears the bottom	**Never** send two requests for the same page (fast scrolling, observer firing twice)
Show "Loading more…"; stop at the end with a message	If one page doesn't fill the box, keep loading until it does
On failure, show an error with Retry	Don't update state after unmount; work inside a scrollable box, not only the window

How it works

An `IntersectionObserver` tells you when an element (here an invisible 1-pixel **sentinel** after the last item) enters a **root** (the scroll box) — no `scroll` listeners, no layout reads on every scroll event. `rootMargin: '200px'` grows the root so loading starts 200 px *before* the user reaches the end.



State design

Data	Kind	Why
`items`	state	the growing list
`page`	state	last page loaded
`status`	state `'idle' | 'loading' | 'error'`	one variable, so "loading and error at once" is impossible
`hasMore`	state	stop observing at the end
`inFlight`	ref	a **synchronous** guard: `status` only changes on the next render, but a ref changes immediately
`mounted`	ref	ignore a response that arrives after unmount
scroller, sentinel	refs to DOM nodes	the observer needs real elements

Code

`code/infinite-list.jsx`:

```jsx
import { useEffect, useRef, useState } from 'react';

export default function InfiniteList({ fetchPage, renderItem = (item) => item.name, rootMargin = '200px' }) {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(0);                      // last page loaded (0 = none yet)
  const [status, setStatus] = useState('idle');             // 'idle' | 'loading' | 'error'
  const [hasMore, setHasMore] = useState(true);
  const scrollerRef = useRef(null);
  const sentinelRef = useRef(null);
  const mounted = useRef(false);
  const inFlight = useRef(false);                           // sync guard: state updates aren't instant

  useEffect(() => {
    mounted.current = true;
    return () => { mounted.current = false; };              // late responses after unmount are dropped
  }, []);

  // Re-created whenever status/page/hasMore change. A new observer reports the sentinel's current
  // state immediately, so if page N didn't fill the box, page N+1 loads without any scrolling.
  useEffect(() => {
    if (status !== 'idle' || !hasMore) return;              // never two requests at once, stop at the end
    const observer = new IntersectionObserver(
      ([entry]) => {
        if (!entry.isIntersecting || inFlight.current) return;
        inFlight.current = true;
        observer.disconnect();
        setStatus('loading');
        fetchPage(page + 1).then(
          (res) => {
            inFlight.current = false;
            if (!mounted.current) return;
            setItems((prev) => [...prev, ...res.items]);
            setPage(page + 1);
            setHasMore(res.hasMore);
            setStatus('idle');
          },
          () => { inFlight.current = false; if (mounted.current) setStatus('error'); },
        );
      },
      { root: scrollerRef.current, rootMargin },            // start loading 200px before the end
    );
    observer.observe(sentinelRef.current);
    return () => observer.disconnect();
  }, [status, page, hasMore, fetchPage, rootMargin]);

  return (
    <section className="card feed" aria-labelledby="feed-h">
      <h2 id="feed-h">Feed <small className="muted">({items.length} loaded)</small></h2>
      <div className="scroller" ref={scrollerRef} tabIndex={0} aria-label="Feed items">
        <ul className="list" aria-busy={status === 'loading'}>
          {items.map((item) => <li key={item.id}>{renderItem(item)}</li>)}
        </ul>
        <div ref={sentinelRef} aria-hidden="true" className="sentinel" />
        {status === 'loading' && <p role="status" className="muted">Loading more…</p>}
        {status === 'error' && (
          <p role="alert" className="error">
            Couldn’t load more. <button onClick={() => setStatus('idle')}>Retry</button>
          </p>
        )}
        {!hasMore && <p className="muted">You’ve reached the end.</p>}
      </div>
    </section>
  );
}
```

Design notes:

- **The Effect is the loop.** It observes only while `status === 'idle'` and `hasMore`. After each page the dependencies change, cleanup disconnects the old observer and a new one immediately reports whether the sentinel is visible — so short pages chain until the box is full.

- **Why `inFlight` too?** An observer callback can run several times before React re-renders with `status: 'loading'`; state is a snapshot (Q8), a ref isn't.
- **Retry** sets `status` back to `'idle'`, so the Effect observes again.
- **`aria-busy`** and a `role="status"` message tell screen-reader users more is coming.

Test

jsdom does no layout, so it has no `IntersectionObserver`. The test installs a tiny fake and an `intersect()` helper that plays the part of the user scrolling:

```
const observers = new Set();
class FakeIntersectionObserver {
  constructor(callback, options) { this.callback = callback; this.options = options; this.targets = []; }
  observe(el) { this.targets.push(el); observers.add(this); }
  unobserve(el) { this.targets = this.targets.filter((t) => t !== el); }
  disconnect() { this.targets = []; observers.delete(this); }
}
globalThis.IntersectionObserver = window.IntersectionObserver = FakeIntersectionObserver;

async function intersect() {                  // "scroll" until the sentinel is visible
  await act(async () => {
    for (const o of [...observers]) o.callback(o.targets.map((target) => ({ target, isIntersecting: true })), o);
    await new Promise((r) => setTimeout(r, 0));
  });
}

test('no duplicate requests while one is in flight', async () => {
  // ... fetchPage that never resolves until release() is called
  render(<InfiniteList fetchPage={fetchPage} />);
  const [o] = observers;
  await act(async () => {                     // the observer fires three times quickly
    for (let i = 0; i < 3; i++) o.callback([{ isIntersecting: true }], o);
  });
  assert.equal(screen.getByRole('status').textContent, 'Loading more…');
  assert.deepEqual(calls, [1]);
  // ...
});
```

Output:

```
✔ loads the next page each time the sentinel becomes visible, then stops
✔ no duplicate requests while one is in flight
✔ error shows Retry, and Retry resumes loading
ℹ tests 3
ℹ pass 3
ℹ fail 0
```

Without the `inFlight` ref this test fails with `calls` = `[1, 1, 1]` — three requests for page 1, the classic infinite-scroll bug.

In the browser

In Chrome the real `IntersectionObserver` drives it. The demo page scrolls the box to the bottom twice:

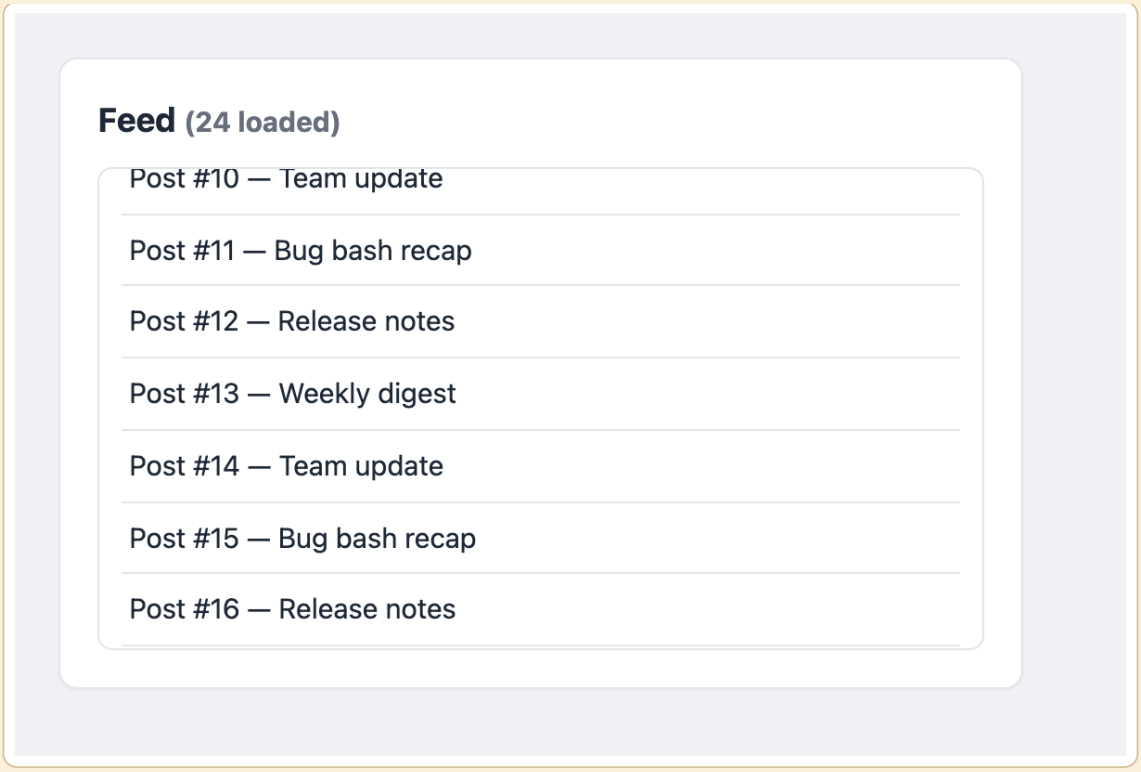


Figure 15.3 — The real `IntersectionObserver` in Chrome: two scrolls to the bottom loaded pages 2 and 3 (24 posts). The third page arrived while the box still showed posts 10–16, so the user never waits at the bottom.

Follow-ups and edge cases

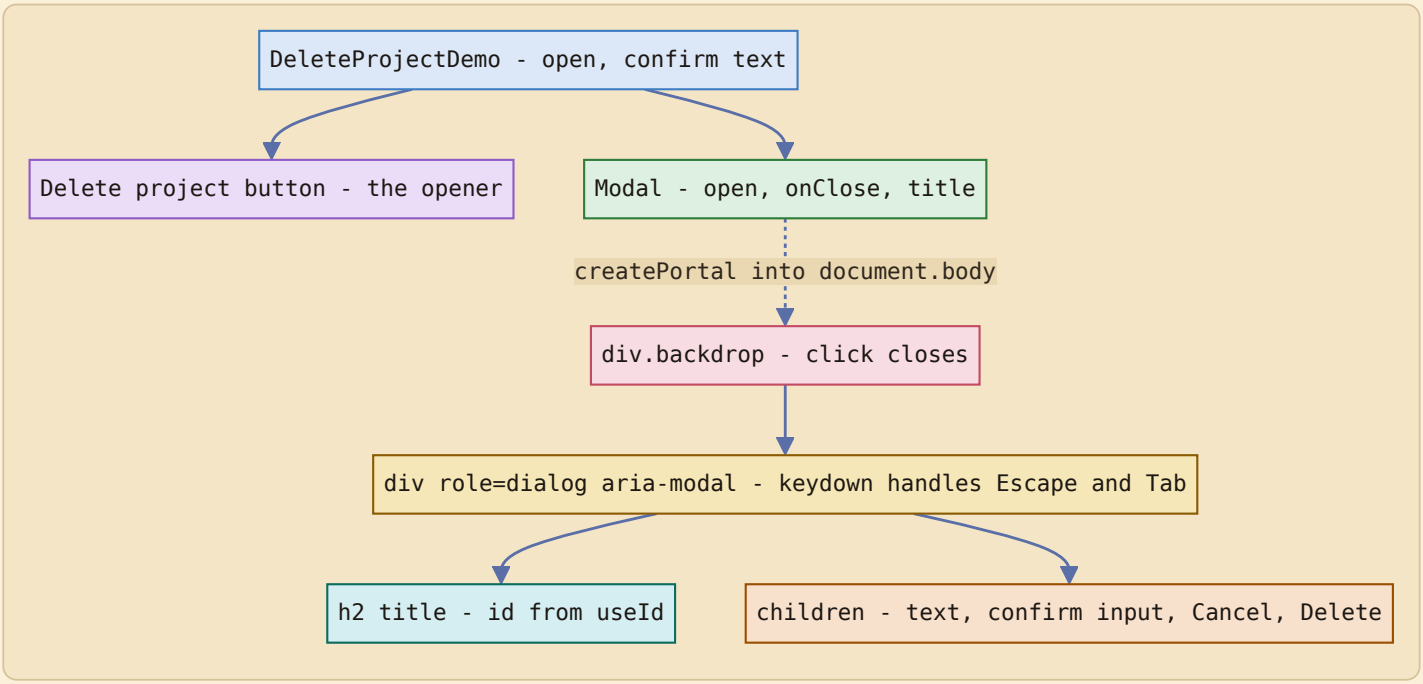
- **Library** → TanStack Query's `useInfiniteQuery` handles pages, caching and `hasNextPage`; an observer still calls `fetchNextPage()` (§9.9).
- **Thousands of items** → add virtualization (§10.9).
- **Cursor vs page numbers** → inserts while scrolling shift page numbers (duplicates, gaps); use a cursor (`?after=<lastId>`) and de-duplicate by `id`.
- **Back-navigation** → cache the data (TanStack Query) and restore `scrollTop`, or open details in a modal that keeps the list mounted.
- **Accessibility** → the footer becomes unreachable; offer a "Load more" button too (it can be the sentinel) and announce new items via `role="status"`.
- **Abort on unmount** → pass an `AbortSignal` to `fetchPage` and abort in a mount-only Effect's cleanup.

15.13 Build 4 — Modal dialog (portal, focus trap, Escape)

Requirements

Functional	Quiet requirements (the real test)
Open from a button, show a title, content and actions	Rendered at the top level so no parent's `overflow` or `z-index` clips it
Close with Cancel, Escape, or a click on the backdrop	Focus moves **into** the dialog on open and **back** to the opener on close
A destructive confirm: "type the name to enable Delete"	**Tab** and **Shift+Tab** cycle inside the dialog; the page behind doesn't scroll
	Screen readers announce a dialog with its title: `role="dialog"`, `aria-modal`, `aria-labelledby`

Component tree



State design

The `Modal` is **controlled**: the parent owns `open` and passes `onClose`. That lets the parent decide what closing means (reset the confirm text, save a draft) and makes the modal reusable. The modal itself holds no state — only refs (the dialog node, and the element that had focus before opening, captured inside the Effect).

Code

`code/modal.jsx`:

```jsx
// Machine coding 4 — modal dialog: portal, focus trap, Escape, backdrop click, focus restore, scroll lock.
import { useEffect, useId, useRef, useState } from 'react';
import { createPortal } from 'react-dom';

const FOCUSABLE = 'a[href], button:not([disabled]), input:not([disabled]), select:not([disabled]),
textarea:not([disabled]), [tabindex]:not([tabindex="-1"])';

export function Modal({ open, onClose, title, children }) {
  const dialogRef = useRef(null);
  const titleId = useId();

  useEffect(() => {
    if (!open) return;
    const previouslyFocused = document.activeElement;          // usually the button that opened us
    const first = dialogRef.current.querySelector(FOCUSABLE);
    (first ?? dialogRef.current).focus();                       // move focus INTO the dialog
    const { overflow } = document.body.style;
    document.body.style.overflow = 'hidden';                    // stop the page behind from scrolling
    return () => {
      document.body.style.overflow = overflow;
      previouslyFocused?.focus?.();                             // give focus back on close
    };
  }, [open]);

  if (!open) return null;

  function handleKeyDown(e) {
    if (e.key === 'Escape') {
      e.stopPropagation();                                      // a parent modal shouldn't close too
      onClose();
      return;
    }
    if (e.key !== 'Tab') return;
    const nodes = [...dialogRef.current.querySelectorAll(FOCUSABLE)];
    if (nodes.length === 0) { e.preventDefault(); return; }
    const first = nodes[0];
    const last = nodes[nodes.length - 1];
    const outside = !nodes.includes(document.activeElement);    // e.g. the dialog box itself, after a click on its
text
    if (e.shiftKey && (outside || document.activeElement === first)) { e.preventDefault(); last.focus(); }
    else if (!e.shiftKey && (outside || document.activeElement === last)) { e.preventDefault(); first.focus(); }
  }

  return createPortal(
    <div className="backdrop" onMouseDown={(e) => { if (e.target === e.currentTarget) onClose(); }}>
      <div
        ref={dialogRef}
        role="dialog"
        aria-modal="true"
        aria-labelledby={titleId}
        tabIndex={-1}
        className="dialog"
        onKeyDown={handleKeyDown}
      >
        <h2 id={titleId}>{title}</h2>
        {children}
      </div>
    </div>,
    document.body,                                              // escape parents' overflow/z-index
  );
}
```

`DeleteProjectDemo` in the same file (the page used by the tests and the screenshot) owns `open` and the confirm text, and passes `onClose={close}` to `Modal`; Delete is `disabled` until the input says "Apollo".

Walk-through:

1. **Portal**: `createPortal(…, document.body)` puts the DOM at the end of `<body>`, but React-wise the modal is still a child of `DeleteProjectDemo`, so state, context and event bubbling work normally (Q49).

2. **Focus in, focus back**: the Effect runs when `open` becomes `true`: it remembers `document.activeElement` (the opener), focuses the first focusable element, and its **cleanup** — which runs when `open` becomes `false` or the modal unmounts — restores focus and the body's `overflow`.
3. **Focus trap**: Tab on the last focusable element wraps to the first; Shift+Tab on the first wraps to the last. If focus is on none of them — clicking the dialog's text focuses the `tabIndex={-1}` box itself — Shift+Tab would otherwise leave the dialog, so it also wraps. The list is re-queried on every key press, so it stays right when Delete becomes enabled.
4. **Escape** closes; `stopPropagation()` stops an outer modal (if any) from closing too.
5. **Backdrop**: close only when the mousedown target *is* the backdrop, not a child — otherwise selecting text inside the dialog and releasing outside would close it.

Test

`code/05_modal.test.jsx` (key parts):

```
test('renders into document.body through a portal, labelled by its title', async () => {
  const { container, dialog } = await openModal();
  assert.equal(container.contains(dialog), false);        // not inside the component's own DOM
  assert.equal(document.body.contains(dialog), true);
  assert.equal(dialog.getAttribute('aria-modal'), 'true');
  assert.equal(document.body.style.overflow, 'hidden');    // background scroll locked
});

test('focus moves in, and Tab / Shift+Tab stay trapped inside', async () => {
  const { user } = await openModal();
  assert.equal(document.activeElement, screen.getByRole('textbox'));   // first focusable element
  await user.tab();
  assert.equal(focused(), 'Cancel');
  await user.tab();                                        // Delete is disabled, so Cancel is last:
  assert.equal(document.activeElement, screen.getByRole('textbox'));   // wraps to the first
  await user.tab({ shift: true });                         // Shift+Tab on the first wraps to the last
  assert.equal(focused(), 'Cancel');
  // ... type "Apollo": Delete becomes the last element in the cycle
});

test('Escape closes and focus returns to the button that opened it', async () => {
  const { user, trigger } = await openModal();
  await user.keyboard('{Escape}');
  assert.equal(screen.queryByRole('dialog'), null);
  assert.equal(document.activeElement, trigger);
  assert.equal(document.body.style.overflow, '');          // scroll lock removed
});
```

Output:

```
✔ renders into document.body through a portal, labelled by its title
✔ focus moves in, and Tab / Shift+Tab stay trapped inside
✔ Escape closes and focus returns to the button that opened it
✔ clicking the backdrop closes; clicking inside does not
✔ the confirm flow: Delete stays disabled until the name is typed
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

`user.tab()` respects `preventDefault()` on the `keydown`, like a browser, so the trap is genuinely tested.

In the browser

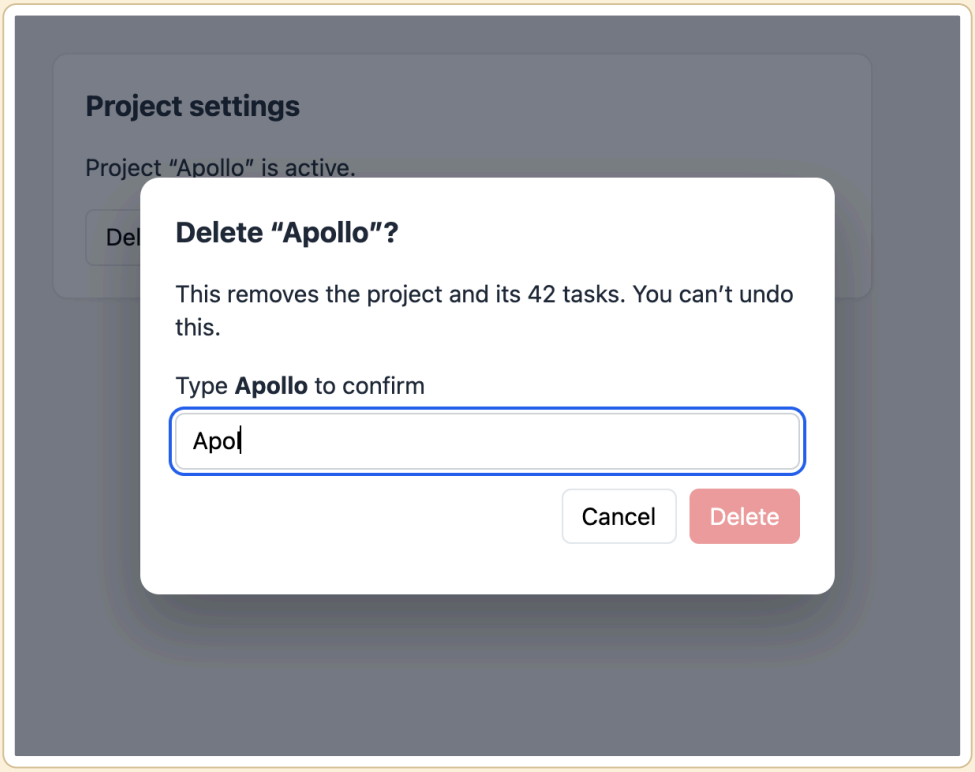


Figure 15.4 — The modal in Chrome, rendered into `<body>` over a dimmed page. "Delete" stays disabled (and outside the focus cycle) until the full name is typed.

Follow-ups and edge cases

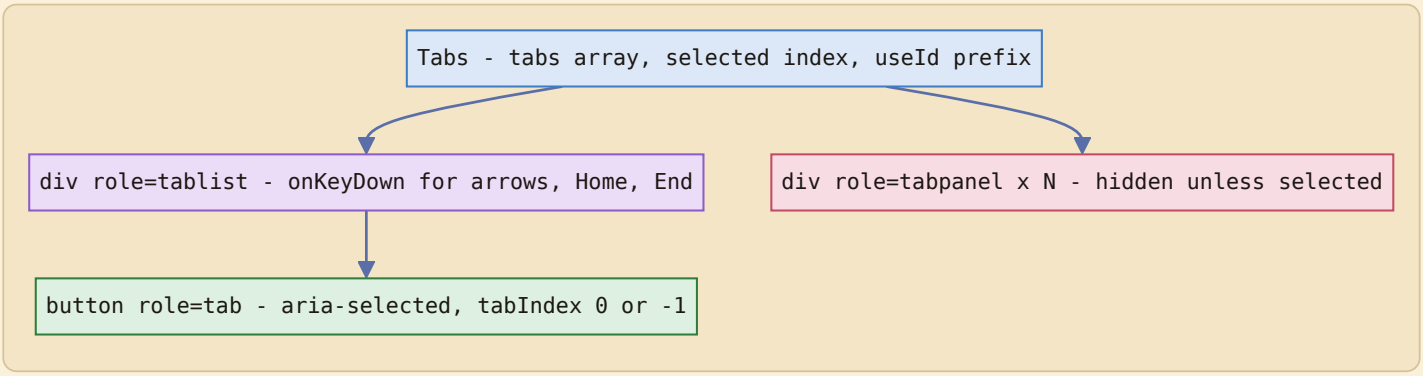
- **Why not native `<dialog>`?** → In real apps, prefer it: `showModal()` uses the browser's **top layer** (no portal or z-index), makes the page behind **inert**, closes on Escape (a `cancel` event) and returns focus to the opener on close (HTML standard behaviour, in all major browsers). Call `showModal()`/`close()` in an Effect when `open` changes and listen for `close` to sync React state. Interviewers still ask for the hand-built version to see you know what the browser does.
- **Screen readers can reach the page behind** a hand-built modal; `aria-modal="true"` helps, but `inert` on the app root is the robust fix (`<dialog>` does it for you).
- **Nested modals** → keep a stack; only the top one handles Escape (the `stopPropagation` above) and traps focus.
- **Exit animations** → keep it mounted in a `closing` state until `transitionend`.
- **SSR** → no `document` on the server; create the portal only after mount.
- **Focusable detection** → the selector misses `contenteditable`, `<details>` and similar; `focus-trap` or Radix/React Aria handle the long tail.

15.14 Build 5 — Accessible tabs

Requirements

Functional	Quiet requirements (WAI-ARIA tabs pattern)
A row of tabs; clicking one shows its panel	`role="tablist"`, `role="tab"` with `aria-selected` and `aria-controls`, `role="tabpanel"` with `aria-labelledby`
Optional initial tab and `onChange` callback	**One** Tab-key stop for the whole tab row (roving `tabIndex`), then Tab moves into the panel
Disabled tabs are skipped	ArrowLeft/ArrowRight move between tabs and wrap; Home/End jump to the ends
Switching tabs keeps what the user typed in other panels	Selection follows focus ("automatic activation")

Component tree



State design

Only one piece of state: `selected` (an index). The IDs that connect each tab to its panel come from `useId`, so two `Tabs` on one page never clash. `tabRefs` (an array of DOM nodes filled by **ref callbacks**) lets the keyboard handler move focus. The `tabs` array is a prop, so what is shown is derived from it.

Roving tabindex — what it is: in a composite widget (tabs, toolbar, radio group, menu), only the *current* item has `tabIndex={0}`; the others have `-1`. Tab enters the widget at the current item and the next Tab leaves it; arrow keys move *within* it. Keyboard users don't have to press Tab through every tab to reach the content.

Code

`code/tabs.jsx`:

```jsx
// Machine coding 5 — accessible tabs (WAI-ARIA tabs pattern, automatic activation, roving tabindex).
import { useId, useRef, useState } from 'react';

export default function Tabs({ tabs, defaultIndex = 0, onChange, label }) {
  const [selected, setSelected] = useState(defaultIndex);
  const tabRefs = useRef([]);                              // one DOM node per tab, for focus()
  const id = useId();

  function select(i) {
    setSelected(i);
    tabRefs.current[i].focus();
    onChange?.(i);
  }

  // Find the next enabled tab in `step` direction, wrapping around the ends.
  function nextEnabled(from, step) {
    for (let k = 1; k <= tabs.length; k++) {
      const i = (from + step * k + tabs.length) % tabs.length;
      if (!tabs[i].disabled) return i;
    }
    return from;
  }

  function handleKeyDown(e) {
    const moves = {
      ArrowRight: () => nextEnabled(selected, 1),
      ArrowLeft: () => nextEnabled(selected, -1),
      Home: () => nextEnabled(-1, 1),                      // first enabled
      End: () => nextEnabled(tabs.length, -1),             // last enabled
    };
    if (!moves[e.key]) return;
    e.preventDefault();                                     // Home/End would scroll the page
    select(moves[e.key]());
  }

  return (
    <div className="card tabs">
      <div role="tablist" aria-label={label} onKeyDown={handleKeyDown}>
        {tabs.map((tab, i) => (
          <button
            key={tab.label}
            ref={(el) => { tabRefs.current[i] = el; }}
            role="tab"
            id={`${id}-tab-${i}`}
            aria-selected={i === selected}
            aria-controls={`${id}-panel-${i}`}
            tabIndex={i === selected ? 0 : -1}               // roving tabindex: only one tab stop
            disabled={tab.disabled}
            onClick={() => select(i)}
          >
            {tab.label}
          </button>
        ))}
      </div>
      {tabs.map((tab, i) => (
        <div
          key={tab.label}
          role="tabpanel"
          id={`${id}-panel-${i}`}
          aria-labelledby={`${id}-tab-${i}`}
          tabIndex={0}
          hidden={i !== selected}                           // keep panels mounted: their state survives
        >
          {tab.content}
        </div>
      ))}
    </div>
  );
}
```

Notes:

- `select(i)` sets state **and** focuses the tab, so focus and selection stay together.
- `nextEnabled` walks one direction with wrap-around, skipping disabled tabs; `Home`/`End` reuse it by starting just outside the ends; `preventDefault()` stops them scrolling the page.
- Panels use `hidden` instead of conditional rendering, so **inactive panels stay mounted** and keep their state (the last test proves it). For heavy panels render only the active one and accept losing its state.

Test

`code/06_tabs.test.jsx` (key parts):

```
test('arrow keys move selection AND focus, wrap, and skip disabled tabs', async () => {
  const user = userEvent.setup();
  const changes = [];
  render(<Tabs tabs={TABS} label="Account" onChange={(i) => changes.push(i)} />);
  await user.click(screen.getByRole('tab', { name: 'Overview' }));
  await user.keyboard('{ArrowRight}');
  assert.equal(selectedTab(), 'Billing');
  assert.equal(document.activeElement.textContent, 'Billing');
  await user.keyboard('{ArrowRight}');                      // Audit log is disabled -> Members
  assert.equal(selectedTab(), 'Members');
  await user.keyboard('{ArrowRight}');                      // wraps to the start
  assert.equal(selectedTab(), 'Overview');
  // ... ArrowLeft wraps to the end, Home, End
});

test('Tab key: one stop for the whole tablist, then into the panel', async () => {
  const user = userEvent.setup();
  render(<><button>before</button><Tabs tabs={TABS} label="Account" defaultIndex={1} /></>);
  await user.tab();                                         // "before"
  await user.tab();
  assert.equal(document.activeElement.textContent, 'Billing');   // the SELECTED tab, not the first
  await user.tab();
  assert.equal(document.activeElement.getAttribute('role'), 'tabpanel');
});
```

Output:

```
✔ ARIA wiring: tablist > tab[aria-selected], tabpanel labelled by its tab
✔ arrow keys move selection AND focus, wrap, and skip disabled tabs
✔ Tab key: one stop for the whole tablist, then into the panel
✔ panels stay mounted, so typed text survives switching tabs
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

Note `getByRole('tabpanel')` finds exactly one panel: elements with `hidden` are removed from the accessibility tree, and Testing Library's role queries ignore them by default — just like a screen reader.

In the browser

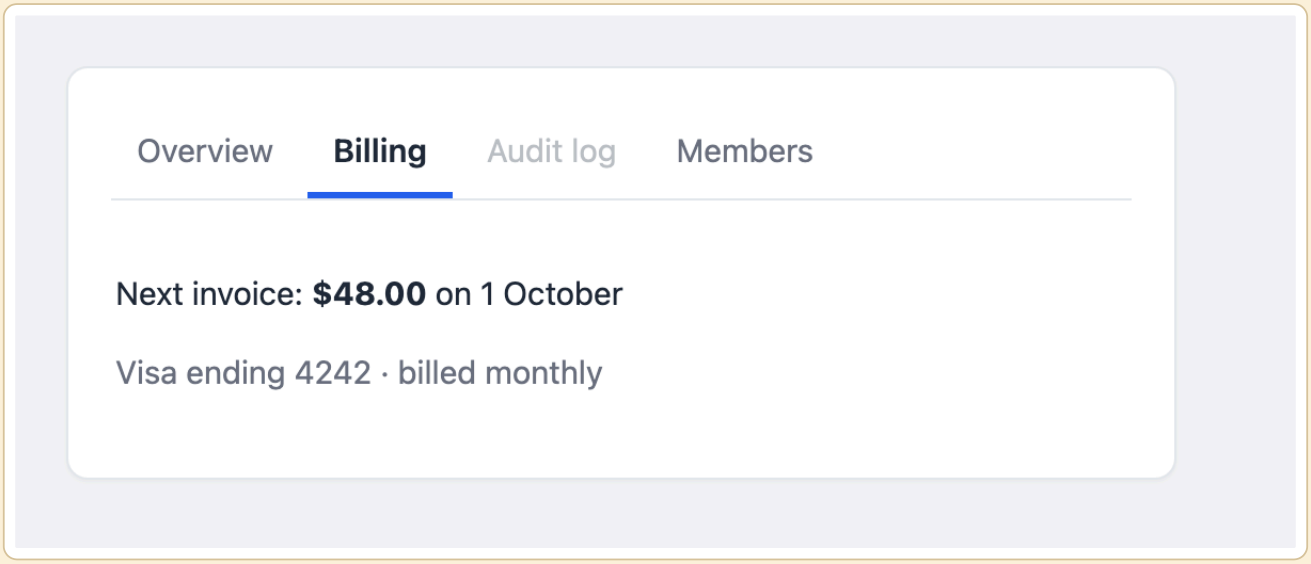


Figure 15.5 — `Tabs` with the second tab selected and a disabled third tab that arrow keys skip.

Follow-ups and edge cases

- **Manual activation** (slow panels) → arrows only *move focus*, Enter/Space selects; keep a separate `focused` index.
- **Controlled version** (e.g. synced with `?tab=billing`) → use the `value` prop when given, else internal state.
- **Compound API** (`<Tabs><TabList><Tab/></TabList><TabPanel/></Tabs>`) → share `selected` and the ID prefix through context (§14.4).
- **Disabled tabs** → ARIA guidance allows keeping them *focusable* with `aria-disabled="true"` so screen-reader users discover them; native `disabled`, used here, removes them from focus. Say which you chose.
- **Heavy panels that keep state** → React 19.2's `<Activity mode="hidden">` hides content and unmounts its Effects while keeping state, and can pre-render at low priority.
- **Vertical tabs** → `aria-orientation="vertical"` and ArrowUp/ArrowDown.

15.15 Build 6 — Star rating

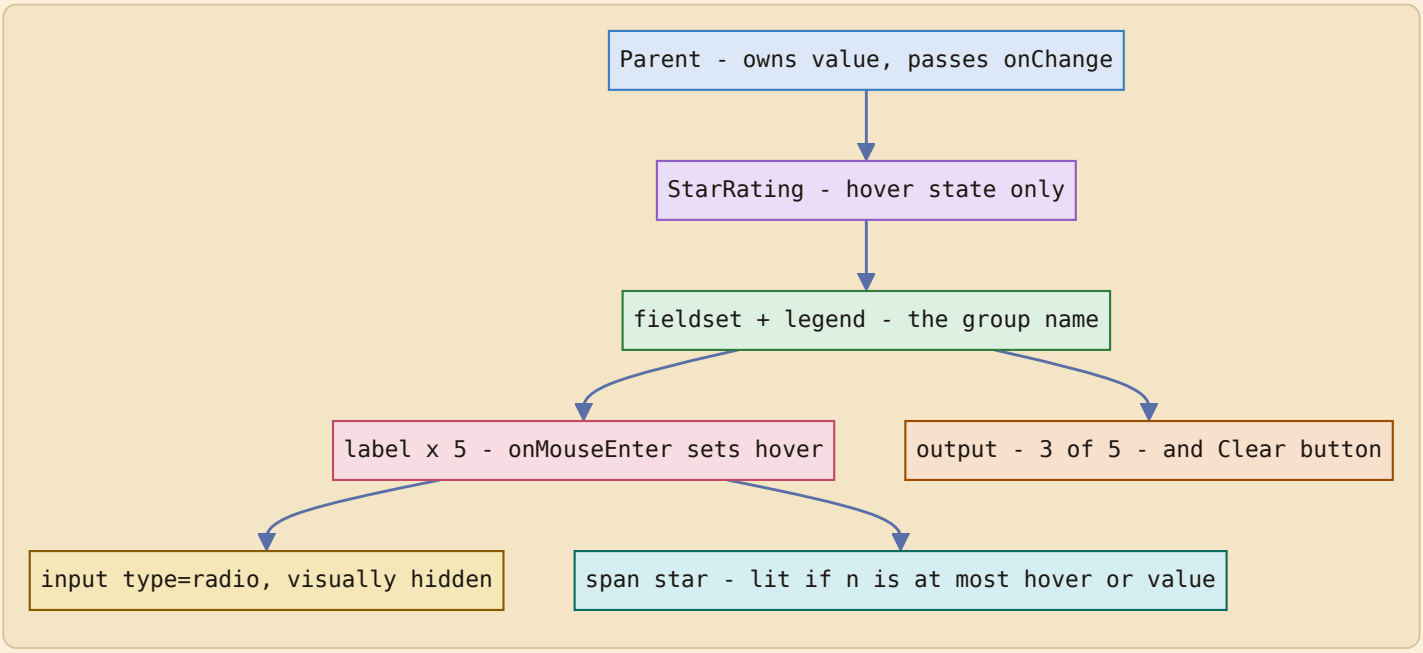
Requirements

Functional	Quiet requirements
5 stars; click to rate; show "3 of 5"	Works with the keyboard: Tab to the group, arrows to change
Hover previews a rating without committing it	Screen readers announce the group name, each star's label ("3 stars") and which one is checked
Clear the rating	Read-only mode for showing an average (one image with text alternative)
Controlled: parent owns the number	`max` configurable

The key design decision

A rating is **one choice out of N** — a **radio group**. Real `<input type="radio">` elements, hidden with the standard `visually-hidden` CSS (still focusable and announced), give you one Tab stop, arrow keys, the checked state and correct screen-reader output for free. `<span onClick>` stars give none of that.

Component tree and state



Data	Kind	Owner
`value`	prop (controlled)	parent
`hover`	state	`StarRating` — purely visual preview
`shown = hover ?? value`	derived	computed in render
group `name`	`useId`	radios in one group share a name

Code

`code/star-rating.jsx`:

```jsx
// Machine coding 6 — star rating built on native radio buttons (keyboard + screen readers for free).
import { useId, useState } from 'react';

export default function StarRating({ value, onChange, max = 5, label = 'Rating', readOnly = false }) {
  const [hover, setHover] = useState(null);                // preview while the pointer is over a star
  const name = useId();
  const shown = hover ?? value;                            // what the stars display right now

  if (readOnly) {
    return (
      <span className="stars" role="img" aria-label={`${label}: ${value} out of ${max}`}>
        {Array.from({ length: max }, (_, i) => <span key={i} className={i < value ? 'star on' : 'star'} aria-
hidden="true">★</span>)}
      </span>
    );
  }

  return (
    <fieldset className="stars" onMouseLeave={() => setHover(null)}>
      <legend>{label}</legend>
      {Array.from({ length: max }, (_, i) => {
        const n = i + 1;
        return (
          <label key={n} onMouseEnter={() => setHover(n)}>
            <input
              type="radio"
              className="visually-hidden"
              name={name}
              value={n}
              checked={value === n}
              onChange={() => onChange(n)}
              aria-label={`${n} ${n === 1 ? 'star' : 'stars'}`}
            />
            <span className={n <= shown ? 'star on' : 'star'} aria-hidden="true">★</span>
          </label>
        );
      })}
      <output className="muted">{value ? `${value} of ${max}` : 'Not rated'}</output>
      {value > 0 && <button type="button" className="link" onClick={() => onChange(0)}>Clear</button>}
    </fieldset>
  );
}
```

Test

`code/07_star_rating.test.jsx` (key parts). One jsdom gap: `user-event` calls `CSS.escape()` to find the other radios of a group, and jsdom has no `CSS` object, so the test adds a minimal stand-in.

```
window.CSS ??= { escape: (s) => s.replace(/[^\w-]/g, (c) => `\\${c}`) };

test('hover previews without changing the value; leaving restores it', async () => {
  const user = userEvent.setup();
  render(<Harness initial={4} />);
  const labels = document.querySelectorAll('fieldset label');
  await user.hover(labels[1]);                              // pointer over star 2
  assert.equal(lit(), 2);
  assert.equal(screen.getByText('4 of 5').tagName, 'OUTPUT');   // value unchanged
  await user.unhover(screen.getByRole('group'));
  assert.equal(lit(), 4);
});

test('keyboard: arrow keys move between stars like any radio group', async () => {
  const user = userEvent.setup();
  render(<Harness initial={2} />);
  await user.tab();                                         // focus lands on the CHECKED radio
  assert.equal(document.activeElement, screen.getByRole('radio', { name: '2 stars' }));
  await user.keyboard('{ArrowRight}');
  assert.equal(screen.getByRole('radio', { name: '3 stars' }).checked, true);
  await user.keyboard('{ArrowLeft}{ArrowLeft}');
  assert.equal(lit(), 1);
});
```

Output:

```
✔ clicking a star selects it; the group is a labelled set of radios
✔ hover previews without changing the value; leaving restores it
✔ keyboard: arrow keys move between stars like any radio group
✔ Clear resets to 0; read-only mode is a single image with a text alternative
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

The component has **no keyboard code at all**; the keyboard test passes because native radios already behave that way.

In the browser

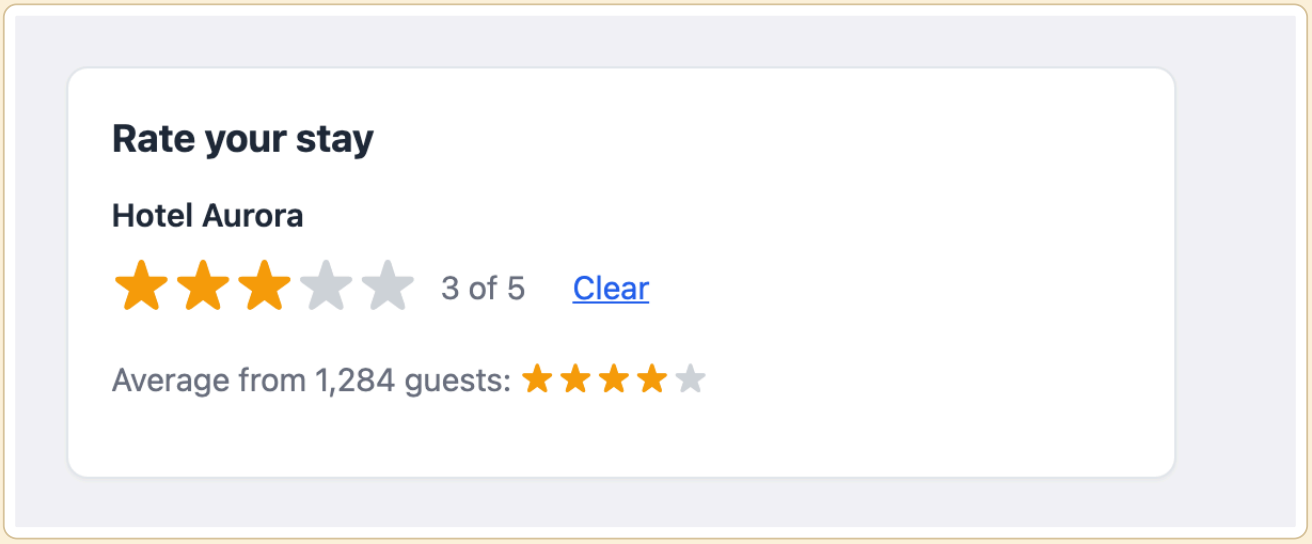


Figure 15.6 — An interactive rating (3 of 5) and a read-only average rendered as one `role="img"` with the label "Average: 4 out of 5".

Follow-ups and edge cases

- **Half stars** → 10 radios (0.5…5), each half drawn by clipping; a read-only 4.3 can be an 86%-wide overlay.
- **Visible focus** → the radio is hidden, so style the star after it (`input:focus-visible + .star`, as the demo CSS does).
- **Uncontrolled version** → `defaultValue` + internal state, and a `name` so a `<form>` submits it (Form Actions read it from `FormData`).
- **Touch devices** have no hover; the preview simply doesn't happen.

- **Why `<fieldset>` / `<legend>`?** → they give the group its accessible name ("Food" in the test, "Hotel Aurora" in the demo); `role="radiogroup"` + `aria-labelledby` is the ARIA alternative.

15.16 Build 7 — Pagination

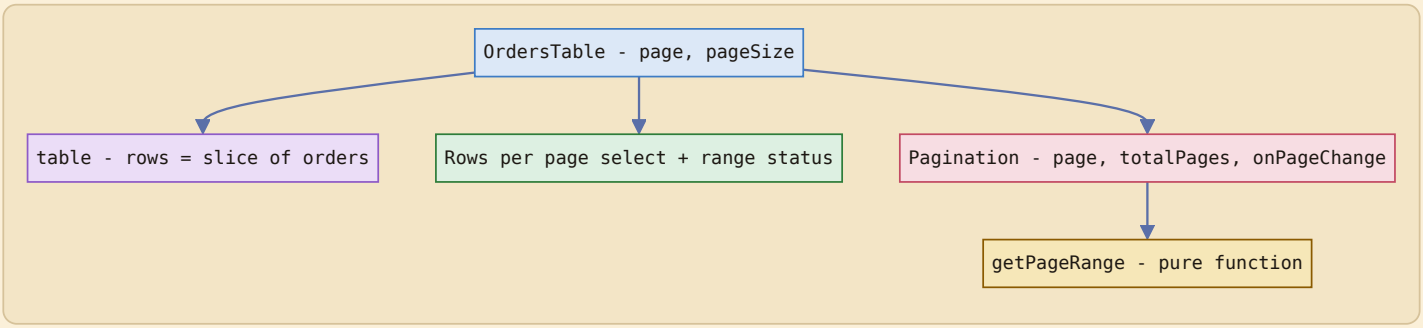
Requirements

Functional	Quiet requirements
Show 10 rows per page; Previous / Next; numbered page buttons	For many pages show `1 … 4 5 6 … 20`, with first and last always visible
Choose rows per page (5/10/25)	The number of buttons stays constant so they don't jump under the mouse
"21–25 of 47" range text	`<nav aria-label="Pagination">`, `aria-current="page"`, disabled ends
	Never show an empty page when the data shrinks (e.g. after filtering)

Split the problem in two

1. A **pure function** `getPageRange(current, total, siblings)` returning what to draw, e.g. `[1, '…', 9, 10, 11, '…', 20]` — easy to test with a table of cases (the heart of every "usePagination" Hook).
2. A **controlled** `<Pagination page totalPages onPageChange>` that draws it; the table owns `page` and `pageSize`.

Component tree and state



Data	Kind	Why
`page`, `pageSize`	state in `OrdersTable`	user choices
`totalPages`, `rows`, range text	derived	from `orders.length`, `page`, `pageSize`
`current = min(page, totalPages)`	derived (clamped)	if the list shrinks, show the last real page — no Effect needed to "fix" state

Code

`code/pagination.jsx`:

```jsx
// Machine coding 7 — pagination: a pure page-range function, a controlled <Pagination>, and a table using it.
import { useState } from 'react';

export const DOTS = '…';

// Pages to show: always first and last, `siblings` on each side of the current page, DOTS for gaps.
// The result always has the same length once there are enough pages, so buttons don't jump around.
export function getPageRange(current, total, siblings = 1) {
  const range = (a, b) => Array.from({ length: b - a + 1 }, (_, i) => a + i);
  const slots = siblings * 2 + 5;                          // first, last, current, 2 dots, siblings
  if (total <= slots) return range(1, total);

  const left = Math.max(current - siblings, 1);
  const right = Math.min(current + siblings, total);
  const showLeftDots = left > 3;                           // a gap of at least 2 pages before `left`
  const showRightDots = right < total - 2;

  if (!showLeftDots) return [...range(1, 3 + 2 * siblings), DOTS, total];
  if (!showRightDots) return [1, DOTS, ...range(total - (2 + 2 * siblings), total)];
  return [1, DOTS, ...range(left, right), DOTS, total];
}

export function Pagination({ page, totalPages, onPageChange, siblings = 1 }) {
  if (totalPages <= 1) return null;
  return (
    <nav aria-label="Pagination">
      <ul className="pager">
        <li><button onClick={() => onPageChange(page - 1)} disabled={page === 1} aria-label="Previous page">‹</button>
</li>
        {getPageRange(page, totalPages, siblings).map((p, i) =>
          p === DOTS
            ? <li key={`dots-${i}`} aria-hidden="true" className="dots">{DOTS}</li>
            : (
              <li key={p}>
                <button onClick={() => onPageChange(p)} aria-current={p === page ? 'page' : undefined} aria-label=
{`Page ${p}`}>
                  {p}
                </button>
              </li>
            ))}
        <li><button onClick={() => onPageChange(page + 1)} disabled={page === totalPages} aria-label="Next
page">›</button></li>
      </ul>
    </nav>
  );
}

// Client-side pagination of an in-memory list.
export default function OrdersTable({ orders, initialPageSize = 10 }) {
  const [page, setPage] = useState(1);
  const [pageSize, setPageSize] = useState(initialPageSize);
  const totalPages = Math.max(1, Math.ceil(orders.length / pageSize));
  const current = Math.min(page, totalPages);              // clamp, e.g. after the list shrinks
  const rows = orders.slice((current - 1) * pageSize, current * pageSize);

  return (
    <section className="card">
      <h2>Orders</h2>
      <table>
        <thead><tr><th>#</th><th>Customer</th><th>Total</th></tr></thead>
        <tbody>{rows.map((o) => <tr key={o.id}><td>{o.id}</td><td>{o.customer}</td><td>${o.total.toFixed(2)}</td>
</tr>)}</tbody>
      </table>
      <div className="row between">
        <label>
          Rows per page{' '}
          <select value={pageSize} onChange={(e) => { setPageSize(Number(e.target.value)); setPage(1); }}>
            {[5, 10, 25].map((n) => <option key={n} value={n}>{n}</option>)}
          </select>
        </label>
```

```
        <span className="muted" role="status">
          {orders.length === 0 ? 'No orders' : `${(current - 1) * pageSize + 1}–${Math.min(current * pageSize,
orders.length)} of ${orders.length}`}
        </span>
      </div>
      <Pagination page={current} totalPages={totalPages} onPageChange={setPage} />
    </section>
  );
}
```

How `getPageRange` thinks: with `siblings = 1` there are 7 slots — first, last, current, one neighbour each side, and two places that are either a page or `…`. A `…` only replaces **two or more** hidden pages (showing `1 … 3` would hide just page 2, so the function shows `1 2 3` instead). Near the start it shows the first five pages, near the end the last five, otherwise the current page with its neighbours.

Test

`code/08_pagination.test.jsx` (key parts):

```
test('getPageRange: first/last always shown, dots for gaps, constant length', () => {
  const cases = [[1, 5], [3, 7], [1, 20], [4, 20], [5, 20], [10, 20], [16, 20], [17, 20], [20, 20]];
  for (const [page, total] of cases) console.log(`    page ${String(page).padStart(2)} of ${total}:`,
getPageRange(page, total).join(' '));
  assert.deepEqual(getPageRange(10, 20), [1, '…', 9, 10, 11, '…', 20]);
  assert.deepEqual(getPageRange(10, 20, 2), [1, '…', 8, 9, 10, 11, 12, '…', 20]);
  for (let p = 1; p <= 20; p++) assert.equal(getPageRange(p, 20).length, 7);   // nothing jumps
});

test('clamps the page when the data shrinks under it', async () => {
  const user = userEvent.setup();
  const { rerender } = render(<OrdersTable orders={orders} />);
  await user.click(screen.getByRole('button', { name: 'Page 5' }));
  rerender(<OrdersTable orders={orders.slice(0, 12)} />);                // e.g. a filter was applied
  assert.equal(screen.getByRole('status').textContent, '11–12 of 12');   // page 2, not an empty page 5
  // ... an empty list shows "No orders" and no pager
});
```

Output:

```
    page  1 of 5: 1 2 3 4 5
    page  3 of 7: 1 2 3 4 5 6 7
    page  1 of 20: 1 2 3 4 5 … 20
    page  4 of 20: 1 2 3 4 5 … 20
    page  5 of 20: 1 … 4 5 6 … 20
    page 10 of 20: 1 … 9 10 11 … 20
    page 16 of 20: 1 … 15 16 17 … 20
    page 17 of 20: 1 … 16 17 18 19 20
    page 20 of 20: 1 … 16 17 18 19 20
✔ getPageRange: first/last always shown, dots for gaps, constant length
    pager on page 5 of 5: ‹ 1 2 3 4 5 ›
✔ Next / Previous / page buttons, aria-current and disabled ends
    pager with 5 per page: ‹ 1 2 3 4 5 … 10 ›
✔ changing rows-per-page goes back to page 1
✔ clamps the page when the data shrinks under it
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

In the browser

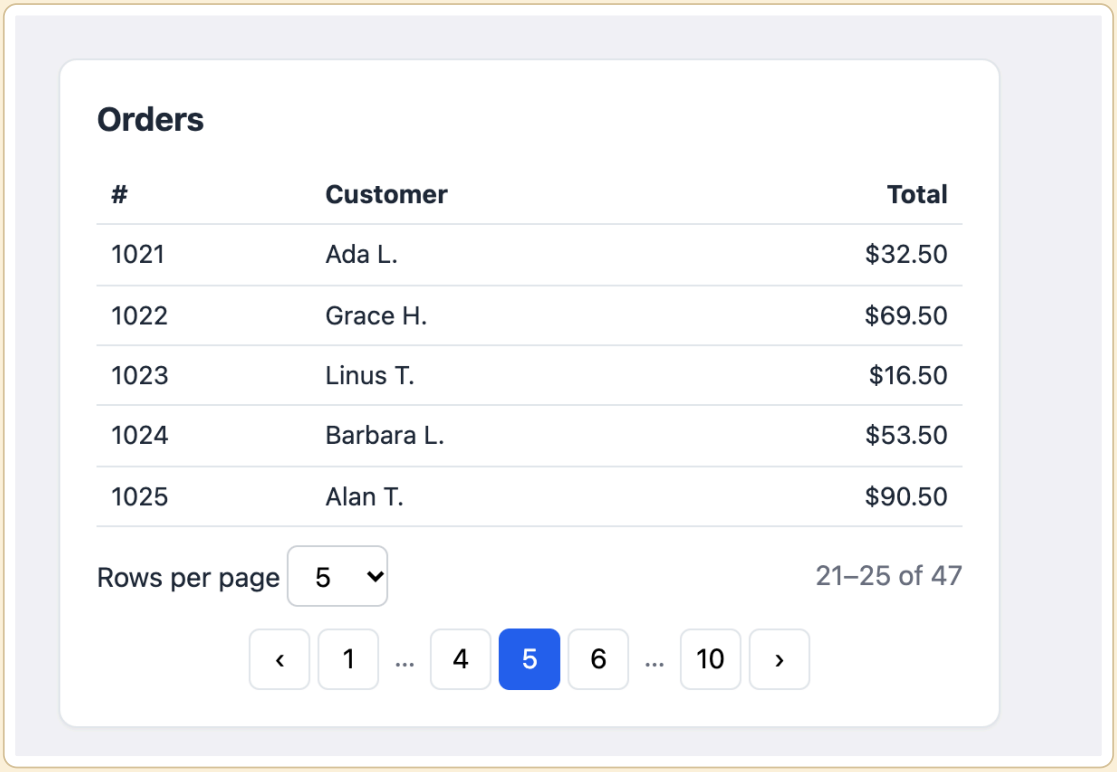


Figure 15.7 — Page 5 of 10 with five rows per page: first and last pages, the current page with one neighbour each side, and ellipses for the gaps.

Follow-ups and edge cases

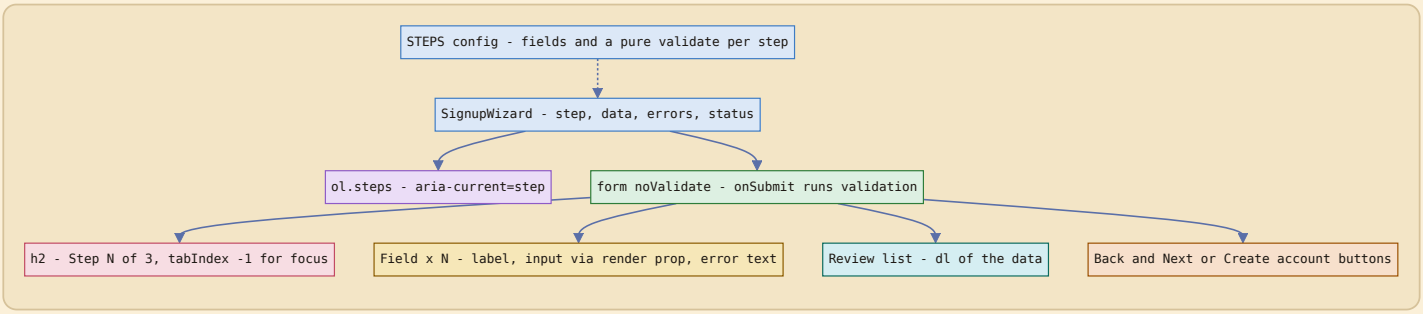
- **Server-side pagination** → the API returns one page plus a total (`?page=5&limit=5` or a cursor); TanStack Query keyed by `['orders', page, pageSize]` with `placeholderData: keepPreviousData` keeps the old page visible while the next loads (§9.9).
- **Page in the URL** (`?page=5`) → shareable links and a working Back button.
- **Offset vs cursor** → offsets allow "jump to page 17" but slow down on deep pages and shift on inserts; cursors are stable and fast but only next/previous.
- **Clamping keeps the stored page** → `page` stays 5 while page 2 is shown, so if the data grows back before the user pages again, the view jumps to 5. Resetting to page 1 whenever the filter changes (as rows-per-page does) avoids it.
- **Focus after a page change** → move focus to the table heading and scroll to it, so screen-reader users hear the new rows.
- **Huge counts** → counting is expensive; show "Next" without a total, or an estimate.

15.17 Build 8 — Multi-step form wizard with validation

Requirements

Functional	Quiet requirements
Three steps: Account (email, password) → Profile (name, plan) → Review → Create account	Validate **per step** when Next is pressed; don't advance while errors exist
Back keeps everything entered	Errors tied to fields: `aria-invalid`, `aria-describedby`; focus moves to the **first** invalid field
A step indicator	The indicator marks the current step with `aria-current="step"`; on step change, focus moves to the new step's heading
Submit shows a pending state and a success or error message	No double submit; a failed submit can be retried

Component tree



State design

Data	Kind	Why
`data`	one state object for **all** steps	Back/Next never lose input; the Review step and the submit read one object
`step`	state	which step is showing
`errors`	state `{ field: message }`	set on Next; a field's error clears as the user edits it
`status`	state `'editing' | 'submitting' | 'done' | 'failed'`	one variable instead of several booleans
step definitions (`STEPS`)	constant outside the component	fields + a **pure** `validate(data)` per step — testable without rendering

The wizard is one `<form>`, so Enter in any field "clicks" the submit button — Next on steps 1–2, Create account on step 3. `noValidate` turns off the browser's validation bubbles so messages are consistent and accessible; `type="email"` and `autoComplete` still help mobile keyboards and password managers.

Code

`code/wizard.jsx`:

```jsx
// Machine coding 8 — multi-step form wizard: one data object, per-step validation, focus management.
import { useRef, useState } from 'react';
import { flushSync } from 'react-dom';

// Each step lists its fields and a pure validate(data) -> { field: message }.
export const STEPS = [
  {
    title: 'Account',
    fields: ['email', 'password'],
    validate: (d) => ({
      ...(!d.email ? { email: 'Email is required' } : !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(d.email) ? { email: 'Enter an
email like name@example.com' } : {}),
      ...(d.password.length < 8 ? { password: 'Use at least 8 characters' } : {}),
    }),
  },
  {
    title: 'Profile',
    fields: ['name', 'plan'],
    validate: (d) => ({
      ...(!d.name.trim() ? { name: 'Name is required' } : {}),
      ...(!d.plan ? { plan: 'Choose a plan' } : {}),
    }),
  },
  { title: 'Review', fields: [], validate: () => ({}) },
];

const INITIAL = { email: '', password: '', name: '', plan: '' };

function Field({ name, label, error, children }) {
  const errorId = `${name}-error`;
  return (
    <div className="field">
      <label htmlFor={name}>{label}</label>
      {children({ id: name, name, 'aria-invalid': error ? true : undefined, 'aria-describedby': error ? errorId :
undefined })}
      {error && <p id={errorId} className="error">{error}</p>}
    </div>
  );
}

export default function SignupWizard({ onSubmit, initialStep = 0, initialData = INITIAL }) {
  const [step, setStep] = useState(initialStep);
  const [data, setData] = useState(initialData);
  const [errors, setErrors] = useState({});
  const [status, setStatus] = useState('editing');          // 'editing' | 'submitting' | 'done' | 'failed'
  const formRef = useRef(null);
  const headingRef = useRef(null);

  function goTo(next) {
    flushSync(() => {                                       // update the DOM now...
      setErrors({});
      setStep(next);
    });
    headingRef.current.focus();                             // ...so the focused heading already has the new
  }                                                         // text when a screen reader announces it

  const update = (field) => (e) => {
    setData((d) => ({ ...d, [field]: e.target.value }));
    if (errors[field]) setErrors(({ [field]: _gone, ...rest }) => rest);   // clear that error as they fix it
  };

  async function handleSubmit(e) {
    e.preventDefault();
    const found = STEPS[step].validate(data);
    if (Object.keys(found).length > 0) {
      setErrors(found);
      const firstBad = STEPS[step].fields.find((f) => found[f]);
      formRef.current.elements[firstBad].focus();           // take the user to the first problem
      return;
    }
```

```
    if (step < STEPS.length - 1) { goTo(step + 1); return; }
    setStatus('submitting');
    try {
      await onSubmit(data);
      setStatus('done');
    } catch {
      setStatus('failed');
    }
  }

  if (status === 'done') return <section className="card"><h2>Welcome, {data.name}!</h2><p>Your {data.plan} account is
ready.</p></section>;

  return (
    <section className="card wizard">
      <ol className="steps">
        {STEPS.map((s, i) => (
          <li key={s.title} aria-current={i === step ? 'step' : undefined} className={i < step ? 'complete' :
undefined}>{s.title}</li>
        ))}
      </ol>
      <form ref={formRef} onSubmit={handleSubmit} noValidate>
        <h2 ref={headingRef} tabIndex={-1}>Step {step + 1} of {STEPS.length}: {STEPS[step].title}</h2>

        {step === 0 && (
          <>
            <Field name="email" label="Email" error={errors.email}>
              {(p) => <input {...p} type="email" autoComplete="email" value={data.email} onChange={update('email')} />}
            </Field>
            <Field name="password" label="Password" error={errors.password}>
              {(p) => <input {...p} type="password" autoComplete="new-password" value={data.password} onChange=
{update('password')} />}
            </Field>
          </>
        )}
        {step === 1 && (
          <>
            <Field name="name" label="Full name" error={errors.name}>
              {(p) => <input {...p} autoComplete="name" value={data.name} onChange={update('name')} />}
            </Field>
            <Field name="plan" label="Plan" error={errors.plan}>
              {(p) => (
                <select {...p} value={data.plan} onChange={update('plan')}>
                  <option value="">Choose…</option><option value="free">Free</option><option value="pro">Pro</option>
                </select>
              )}
            </Field>
          </>
        )}
        {step === 2 && (
          <dl className="summary">
            <dt>Email</dt><dd>{data.email}</dd>
            <dt>Name</dt><dd>{data.name}</dd>
            <dt>Plan</dt><dd>{data.plan}</dd>
          </dl>
        )}

        {status === 'failed' && <p role="alert" className="error">Sign-up failed. Please try again.</p>}
        <div className="row between">
          <button type="button" onClick={() => goTo(step - 1)} disabled={step === 0 || status ===
'submitting'}>Back</button>
          <button type="submit" disabled={status === 'submitting'}>
            {step < STEPS.length - 1 ? 'Next' : status === 'submitting' ? 'Creating account…' : 'Create account'}
          </button>
        </div>
      </form>
    </section>
  );
}
```

Focus management, the part most candidates skip:

- **On a validation error**, focus jumps to the first invalid field, whose error is read out via `aria-describedby`.
- **On a step change**, `goTo` updates state inside `flushSync` so the DOM already shows the new step, then focuses the `<h2>` (`tabIndex={-1}`), announced as "Step 2 of 3: Profile". Doing it in the handler, not an Effect, keeps it off the first mount (and StrictMode's double run).
- **While submitting**, both buttons are disabled (implicit Enter submission is blocked too) and the label reads "Creating account…", so nothing submits twice.

Test

`code/09_wizard.test.jsx` (key parts):

```
test('Next on an empty step shows errors, marks fields invalid and focuses the first one', async () => {
  const user = userEvent.setup();
  render(<SignupWizard onSubmit={async () => {}} />);
  await user.click(next());
  assert.equal(heading(), 'Step 1 of 3: Account');                  // did not advance
  const email = screen.getByLabelText('Email');
  assert.equal(email.getAttribute('aria-invalid'), 'true');
  assert.equal(screen.getByText('Email is required').id, email.getAttribute('aria-describedby'));
  assert.equal(document.activeElement, email);
  // ... typing clears the error; "ada" then shows the format message
});

test('full journey: steps, Back keeps data, review, submitting state, success', async () => {
  // ... fill Account, Next
  assert.equal(heading(), 'Step 2 of 3: Profile');
  assert.equal(document.activeElement, screen.getByRole('heading', { level: 2 }));   // focus moved to the step
  await user.click(screen.getByRole('button', { name: 'Back' }));
  assert.equal(screen.getByLabelText('Email').value, 'ada@example.com');   // nothing lost
  // ... Profile errors, fix them, Review, Create account
  assert.equal(screen.getByRole('button', { name: 'Creating account…' }).disabled, true);   // no double submit
  finish();
  assert.equal((await screen.findByRole('heading', { name: 'Welcome, Ada Lovelace!' })).tagName, 'H2');
});
```

Output:

```
✔ validators are pure functions: easy to unit-test on their own
✔ Next on an empty step shows errors, marks fields invalid and focuses the first one
    review: Email=ada@example.com, Name=Ada Lovelace, Plan=pro
✔ full journey: steps, Back keeps data, review, submitting state, success
✔ a failed submit shows an alert and lets the user retry
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

In the browser

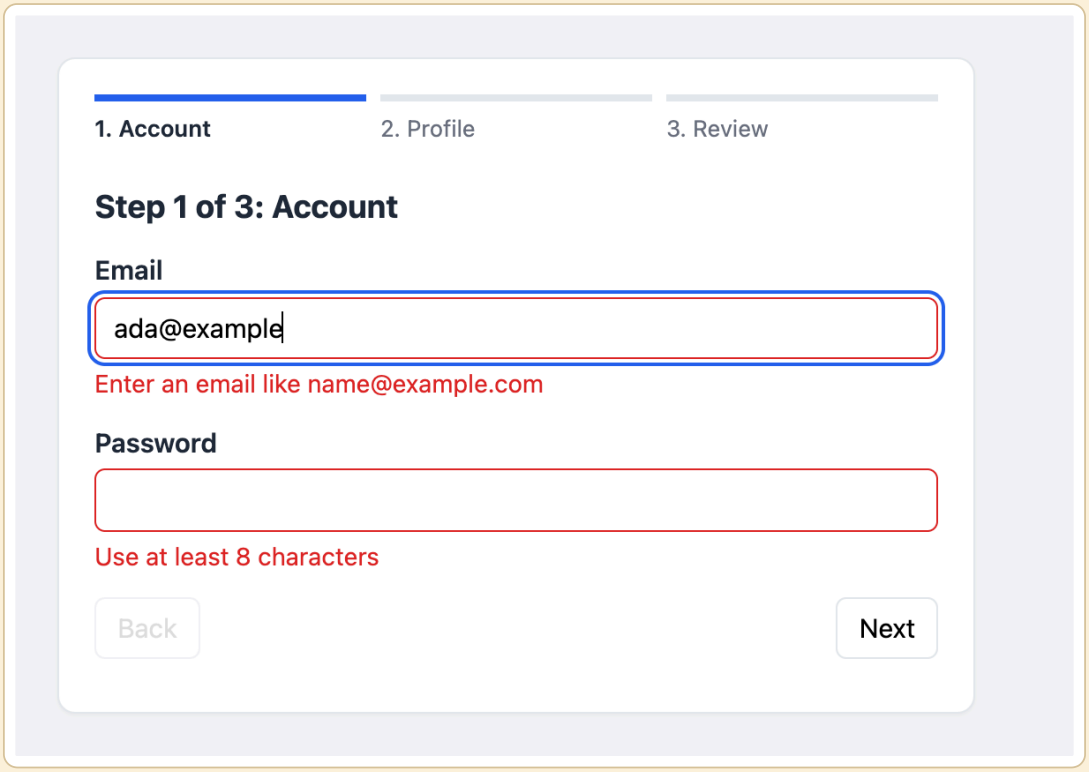


Figure 15.8 — Pressing Next with an incomplete email and no password: both fields are marked invalid, each message is linked to its field, and focus (blue ring) is on the first problem.

Follow-ups and edge cases

- **React 19 form Actions** → `<form action={createAccount}>` with `useActionState` for server errors and `useFormStatus` for the pending button (§4.5); Actions reset only *uncontrolled* fields, so these controlled ones keep their values.
- **Big forms** → React Hook Form with a Zod schema per step; `trigger(['email', 'password'])` validates one step (§4.9).
- **Validate again on the server** — client validation is convenience, not security.
- **Save progress** → `data` (without the password!) in `sessionStorage`, or `step` in the URL so refresh and Back work.
- **Conditional steps** (e.g. "Company details" for Pro) → compute the visible steps from `data`.
- **IDs** → field names as `id`s clash with two wizards on a page; prefix with `useId()`.
- **Async validation** ("email already taken") → a debounced check on blur or Next, with a pending state on the field.
- **A failed submit, then Back** → the "Sign-up failed" alert stays visible on earlier steps; reset `status` to `'editing'` in `goTo` if that bothers you.

15.18 Other common prompts — which techniques they need

Prompt	Core state	Techniques from this chapter
Accordion	`openIds` (a `Set`, or one ID)	`<button aria-expanded aria-controls>`; or native `<details>`/`<summary>`; like Tabs
Dropdown / menu button	`open`, `active`	Autocomplete's keyboard handling + Modal's Escape/click-outside + focus return
Toast notifications	queue of `{ id, text }`	portal, `role="status"` live region, timers cleared in Effect cleanup
Tooltip / popover	`open`	portal, `useLayoutEffect` to measure and position, `aria-describedby`; or the native Popover API
Countdown / stopwatch	`startedAt`, `elapsed`	`setInterval` in an Effect with cleanup; compute from timestamps, not by counting ticks
Image carousel	`index`	wrap-around like Tabs; pause autoplay on hover/focus; `aria-roledescription="carousel"`
File explorer / nested tree	`expanded` set; tree data as a prop	recursive components with keys; ARIA `tree` roles and arrow keys
Sortable, filterable data table	`sort`, `filters`, `page`	all derived with `useMemo` if large; Pagination; URL state
Kanban board (drag and drop)	columns → card IDs	normalised state in a reducer; `@dnd-kit` or native drag events; keyboard alternative
OTP / PIN input	array of digits	refs array to move focus; paste handling; `autoComplete="one-time-code"`
Progress bar / file upload	`progress`, `status`	`<progress>` element; abort with `AbortController`
Chat window	messages; `status` per message	`useOptimistic` for sending; auto-scroll with a ref; virtualise long histories

Chapter 15 TL;DR

Idea	Remember
Machine-coding method	Requirements → tree → state → happy path → keyboard/a11y → test → edge cases; talk throughout
State design	Minimal state, derive the rest; colocate, lift only when shared; one `status` value, not several booleans
Rendering & keys	Trigger → render (pure) → commit; identity = type + position + key; stable ID keys; new key = fresh instance
State updates	Snapshot per render; batched everywhere since React 18; updater functions build on queued values
Effects	Only to sync with outside systems; cleanup runs before re-run and on unmount; StrictMode runs setup → cleanup → setup in dev
Refs	Mutable, don't re-render, not for render output; `ref` is a normal prop in React 19
Context	A transport, not a store; every consumer re-renders on a new value; split or memoize
Performance	Measure first; `memo` / `useMemo` / `useCallback` or the React Compiler; virtualize long lists; split code with `lazy`
Concurrency	Suspense shows fallbacks; `use()` reads cached promises; transitions keep old UI while new UI loads
Server	SSR → HTML then hydrate; mismatches from non-deterministic render; Server Components ship no JS; `'use client'` marks the boundary, `'use server'` marks Server Functions
Errors & portals	Boundaries catch render/Effect errors, not event handlers; portal events bubble through the React tree
Testing	Roles and labels, `user-event`, assert what users see; fake the network at the boundary
Autocomplete	Debounced value + Effect with `AbortController`; `aria-activedescendant`; `mousedown` preventDefault
Infinite scroll	Sentinel + `IntersectionObserver` inside the scroll box; ref guard against duplicate requests
Modal	Portal, focus in/trap/restore, Escape, backdrop mousedown; prefer native `<dialog>` in production
Tabs, stars, pagination, wizard	Roving tabindex; native radios; pure range function + clamping; per-step pure validators and focus management

Next: Chapter 16 — Cheat Sheets and Glossary.

Chapter 16 — Cheat Sheets and Glossary

This last chapter squeezes Chapters 1–15 into pages you can scan in a minute. There is one **cheat sheet** per area (JSX, components, every Hook, state, the render/commit lifecycle, Effects, refs, forms, state management, routing and data, performance, styling, TypeScript, testing, server rendering, patterns, accessibility), a **"common errors and fixes"** table built from the real messages React printed in earlier chapters, an **index of the 53 interview questions** in Chapter 15, a **"where to go next"** plan, and an alphabetical **glossary** of 222 terms, each linked to the section that explains it properly.

Nothing here is new. If a line doesn't make sense, follow its link back to the full explanation. References look like **§9.5** — Chapter 9, section 5.

What you'll learn: how to use this chapter as a revision tool. Read a sheet, cover the right-hand column, and say the answer before you look. The glossary is a good last check before an interview: if you can't define a term in one sentence, follow its link.

How the facts were checked. Every concrete behaviour quoted on these sheets — batching, updater queues, key resets, StrictMode double calls, Effect order, default values, error messages — is asserted by `code/01_cheatsheet_checks.jsx`. It runs React 19.3 in jsdom (plus React Router 8.4 and TanStack Query 5.104) and fails loudly if React behaves differently from what a sheet says. Run it with `bash tools_run_js.sh react` from the repository root. Real output:

```
OK   16.1 JSX rules
OK   16.2 components and props
OK   16.3 hooks table (identity, lazy init, memo caches, useId, deferred value)
OK   16.4 state: snapshot, updaters, batching, bail-out, immutability, position & key
OK   16.5 render/commit lifecycle order
OK   16.6 effects: dependency arrays, cleanup, StrictMode double run
OK   16.7 refs: no re-render, null during first render, callback cleanup, ref as a prop
OK   16.8 forms: FormData rules, form actions, useActionState, useFormStatus, useOptimistic
OK   16.9 context: default, nearest provider, memo does not block it, use() may be conditional
OK   16.10 React Router params and ranking; TanStack Query defaults and de-duplication
OK   16.11 performance: memo, unstable props, children from above, useMemo, lazy
OK   16.14 testing: get/query/find semantics, role + name, user-event vs fireEvent
OK   16.15 server rendering: markers, no effects on the server, suspended boundaries, hydration reuse, use(browser()),
resume APIs
OK   16.16 patterns: portals bubble through React, boundaries catch render errors only, HOC in render remounts
OK   16.18 common errors: the exact messages React 19.3 produces
all 15 cheat-sheet checks passed (React 19.3.0)

messages seen in 16.18:
  Too many re-renders.
  Objects are not valid as a React child (found: object with keys {name}).
  Rendered more hooks than during the previous render.
  Rendered fewer hooks than expected. This may be caused by an accidental early return statement.
  Invalid hook call. Hooks can only be called inside of the body of a function component.
  The result of getSnapshot should be cached to avoid an infinite loop
  Maximum update depth exceeded.
  Element type is invalid. Received a promise that resolves to: undefined. Lazy element type must resolve to a class or
function.
  Each child in a list should have a unique "key" prop.
  Encountered two children with the same key, `1`.
  A component is changing an uncontrolled input to be controlled.
  You provided a `value` prop to a form field without an `onChange` handler.
  Invalid DOM property `class`.
  The tag <profile> is unrecognized in this browser.
  Unsupported style property @media (max-width: 600px).
  A component was suspended by an uncached promise.
  React has blocked a javascript: URL as a security precaution.
  An update to T inside a test was not wrapped in act(...).
  Hydration failed because the server rendered text didn't match the client.
```

(§16.12 Styling, §16.13 TypeScript and §16.17 Accessibility summarise results that Chapters 8, 11 and 14 already verified with a real browser, the TypeScript compiler and role queries, so they have no separate check here.)

16.1 JSX rules

Rule	Write	Not	Why
One root element	`<>…</>` or `<div>…</div>`	two siblings at the top	A function returns one value; JSX becomes one object
Close every tag	`<img />`, ` `, `<input />`	`<img>`	JSX is stricter than HTML
camelCase attributes	`className`, `htmlFor`, `onClick`, `tabIndex`	`class`, `for`, `onclick`	Attributes become JS object keys; `class` / `for` are reserved. `aria-*` and `data-*` keep dashes
`{}` holds an **expression**	`{user.name}`, `{a ? b : c}`, `{list.map(…)}`	`{if (…) …}`	Statements (`if`, `for`, `const`) go above the `return`
Strings in quotes, everything else in braces	`alt="text"`, `size={50}`, `admin` (= `true`)	`size="50"` for a number	Quotes always pass a string
`style` is an object	`style={{ fontSize: 14, lineHeight: 1.5 }}` → `font-size:14px;line-height:1.5`	`style="font-size:14px"`	Numbers get `px`, except unitless properties (`lineHeight`, `zIndex`, `opacity`, `flexGrow`)
Comments	`{/* note */}`	`<!-- note -->`	JSX is JavaScript
Keyed fragment	`<Fragment key={id}>…</Fragment>`	`<key={id}>`	The short `<>` takes no props

What renders what

Value in `{}`	Screen shows
`null`, `undefined`, `true`, `false`	nothing
`0`, `NaN`	"0", "NaN" — the **`{count && <Badge/>}` trap**; write `count > 0 &&`
`''`	an empty text node (invisible)
strings, numbers	the text, **HTML-escaped** (`<script>` shows as text — built-in XSS protection)
arrays of elements	each element (give each a `key`)
a plain object	throws *"Objects are not valid as a React child"*

Remember

- JSX compiles to `jsx(type, props)` / `createElement(type, props, ...children)` and returns an **immutable element object** `{ type, props, key }`. Since React 17 (automatic runtime) you don't `import React`. §1.8
- Capitalised tag = your component (`<Profile />`); lowercase = HTML tag (`<profile>` is just an unknown element, and React warns). §1.7, §2.1
- Raw HTML only through `dangerouslySetInnerHTML={{ __html: sanitised }}`. React 19 replaces a `javascript:` URL in `href` with one that throws. §1.8
- Conditional rendering: `if` + early `return` for big branches, `? :` for either/or, `&&` with a **boolean** on the left, `null` for nothing. §2.7

16.2 Components and props

```
// A component: capitalised function, one props object in, JSX out, declared at module top level.
export function Avatar({ person, size = 100, className, ...rest }) {   // destructure, default, collect the rest
  return <img className={className} src={person.url} alt={person.name} width={size} {...rest} />;
}
<Avatar person={ada} size={50} />         // strings in quotes, other values in braces
```

Topic	Rule
Name and place	Capitalised; defined at **module top level**, never inside another component (a new type every render → state and DOM reset) §1.7
Use	`<Profile />`, never `Profile()` — calling it directly gives it no place in the tree and its Hooks belong to the caller §2.1
Props	One **read-only** object from the parent; frozen in development (mutating throws `TypeError: Cannot assign to read only property`) §2.2
Defaults	Destructuring default `({ size = 100 })` applies when the prop is **missing or** `undefined` — not for `null` or `0`. `defaultProps`/`propTypes` are gone for function components in React 19 §2.2
Special props	`key` is consumed by React (`props.key` is `undefined`); `children` = whatever is nested; `ref` is an ordinary prop in React 19 §2.2, §5.10
Forwarding	`({ variant, ...rest }) => <button {...rest} />`; later props override earlier ones §2.2
`children`	Anything renderable; render it as `{children}` to build wrappers (`Card`, `Layout`) §2.3
Composition	Pass JSX as `children` or **slot** props (`header={…}`) instead of inheritance or boolean-prop explosions; first cure for prop drilling §2.4, §14.1
Purity	Same props/state/context → same JSX; no changes to outside variables during render (local mutation of values created in this render is fine) §2.5
Size	One job per component; extract when JSX has a name, repeats, has its own logic or needs its own state §2.6
Lists	`items.map(i => <Row key={i.id} … />)`; key on the outermost element returned by `map` §2.8

Keys in one table §2.8, §3.10

Key choice	Result
Stable ID from the data	Correct: items keep DOM, state and focus when moved
Array index	OK only for static lists; on insert/sort/filter, state stays at the **position** (the "note moves to Eggs" bug)
`Math.random()` / `crypto.randomUUID()` in JSX	Every item remounts every render — slow, loses input focus
Missing	Warning, index used implicitly
Duplicate	Warning, items duplicated or dropped
Changing the key of one component on purpose	Fresh instance: the idiomatic way to **reset** state (`<Profile key={userId} />`)

16.3 Every Hook in one table

"Since" = the React version that made it stable. All Hooks follow the Rules of Hooks (top level of a component or custom Hook, same order every render) — except `use`, which may be called in `if`/loops (not in `try/catch`). §7.2

Hook	Signature	What it's for	Gotchas	Since
`useState`	`const [v, setV] = useState(init | () => init)`	Memory that changes with interaction; setter schedules a re-render	Value is a snapshot; initial value used on first render only; pass a function for lazy init; setter identity is stable; `Object.is`-equal value → bail-out §3.2	16.8
`useReducer`	`const [s, dispatch] = useReducer(reducer, initialArg, init?)`	Many related updates in one pure `(state, action) => next`	Reducer must be pure (StrictMode calls it twice); `dispatch` is stable; throw on unknown actions §6.2	16.8
`useContext`	`const v = useContext(Ctx)`	Read the nearest provider's value	Default only when **no** provider; re-renders on every value change, even through `memo` §6.3, §6.4	16.8
`use`	`const v = use(promise | Ctx)`	Read a promise (suspends; rejection → error boundary) or a context	Promise must be cached/created outside render — a new promise each render triggers "suspended by an uncached promise"; allowed in `if`/loops §9.11	19.0
`useRef`	`const r = useRef(init)` → `{ current }`	Values that don't affect rendering (timer IDs, DOM nodes, instances)	Changing `r.current` never re-renders; don't read/write it during render (except lazy init); DOM refs are `null` until commit §5.8, §5.9	16.8
`useImperativeHandle`	`useImperativeHandle(ref, () => api, deps?)`	Expose a limited API (`focus`, `clear`) instead of the DOM node	Pair with `ref` as a prop (React 19) §5.10	16.8
`useEffect`	`useEffect(() => { setup; return cleanup }, deps?)`	Synchronise with an external system after commit	No array = every commit; `[]` = mount only; `[a]` = when `a` changes (`Object.is`); setup can't be `async`; doesn't run on the server §5.2	16.8
`useLayoutEffect`	same as `useEffect`	Measure layout and fix it **before paint** (no flicker)	Blocks painting; does nothing during SSR; default to `useEffect` §5.11	16.8
`useInsertionEffect`	same shape	CSS-in-JS libraries inserting `<style>` before layout effects	Not for app code §5.11	18.0
`useEffectEvent`	`const onX = useEffectEvent(fn)`	Non-reactive logic inside an Effect that must see the **latest** props/state	Call only from Effects; never list it in dependencies; don't pass it to other components §5.5, §7.7	19.2
`useMemo`	`const v = useMemo(() => calc(a), [a])`	Cache an expensive result (~1 ms+) or keep an object's identity stable	Must be pure; a cache, not a guarantee; unstable deps make it useless §10.6, §10.5	16.8
`useCallback`	`const fn = useCallback(fn, [a])`	Same function identity for `memo` children / Effect deps	= `useMemo(() => fn, deps)`; missing deps → stale closure §10.5	16.8
`useTransition`	`const [isPending, startTransition] = useTransition()`	Mark updates non-urgent; keep old UI during Suspense	Can't wrap a controlled input's value; since 19 the callback may be `async` (an Action) — updates **after an**	18.0

Hook	Signature	What it's for	Gotchas	Since
			`await` need their own `startTransition` §7.10	
`useDeferredValue`	`const d = useDeferredValue(v, initial?)`	A lagging copy of a value for a slow child	Needs `memo` on the slow child; not a debounce (no timer, no fewer requests); `initialValue` added in 19 §7.11	18.0
`useId`	`const id = useId()`	IDs for `htmlFor` / `aria-*` that match between server and client	One call + suffixes (`${id}-hint`); never for list keys; format is not stable API §7.8	18.0
`useSyncExternalStore`	`useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?)`	Subscribe to data outside React without tearing	`getSnapshot` must return the **same** value if nothing changed (else "should be cached" + infinite loop); keep `subscribe` stable §7.9	18.0
`useDebugValue`	`useDebugValue(label)`	Label a custom Hook in React DevTools	Library Hooks only §7.3	16.8
`useActionState`	`const [state, action, isPending] = useActionState(fn, initial, permalink?)`	Result of the last Action (errors, success) + pending flag	`fn(prevState, payload)` — previous state comes **first**; calls are queued; manual dispatch must be inside `startTransition` §4.5	19.0
`useFormStatus` (`react-dom`)	`const { pending, data, method, action } = useFormStatus()`	Status of the **parent** `<form>` from any child	Always `pending: false` in the component that renders the `<form>`; import from `react-dom` §4.6	19.0
`useOptimistic`	`const [opt, addOpt] = useOptimistic(value, reducer?)`	Show the expected result while an Action runs	Call `addOpt` inside an Action/transition; rolls back automatically when the Action ends; reducer must be pure §4.7	19.0

Related non-Hook APIs: `memo` §10.4, `lazy` + `<Suspense>` §10.8, `startTransition` (standalone) §7.10, `createContext` / `<Ctx value>` §6.3, `createPortal` §14.8, `flushSync` §3.4, `<Profiler>` §10.3, `<StrictMode>` §5.4, `<Activity>` (19.2) §5.4, `cache()` for Server Components §13.7. Library Hooks (React Router, TanStack Query, Redux, Zustand) are in §16.9 and §16.10.

16.4 State update rules

Rule	Example	Result
State is a **snapshot** per render	`setN(n + 1); console.log(n)`	Logs the old value; a timeout created in that render also sees the old value forever §3.3
Plain value = **replace**	three × `setN(n + 1)` from 0	**1**
Updater = **compute from the queued value**	three × `setN(m => m + 1)` from 0	**3**
Mixed queue, processed in order	`setN(n + 5); setN(m => m + 1)` from 0	**6**
Last replace wins	`setN(m => m + 1); setN(n + 5)` from 0	**5** §3.5
Batching	several setters in one event — also inside `setTimeout`, promises, native listeners (React 18+)	**one** re-render §3.4
Opt out of batching	`flushSync(() => setItems(next))`	DOM updated before the next line (e.g. to scroll to a new item)
Same value	`setN(n)`	Bail-out: no re-render of that subtree (React may call the component once more and discard it) §3.2, §10.1
Never mutate	`p.name = 'x'; setP(p)`	Same reference → **no re-render**, stale screen. Use `setP({ ...p, name: 'x' })` §3.6
State lives at a **position**	same component type at the same place	State kept, even with different props
…	different type at that place, or a **different** `key`	State destroyed, fresh instance §3.10

Immutable update recipes §3.6

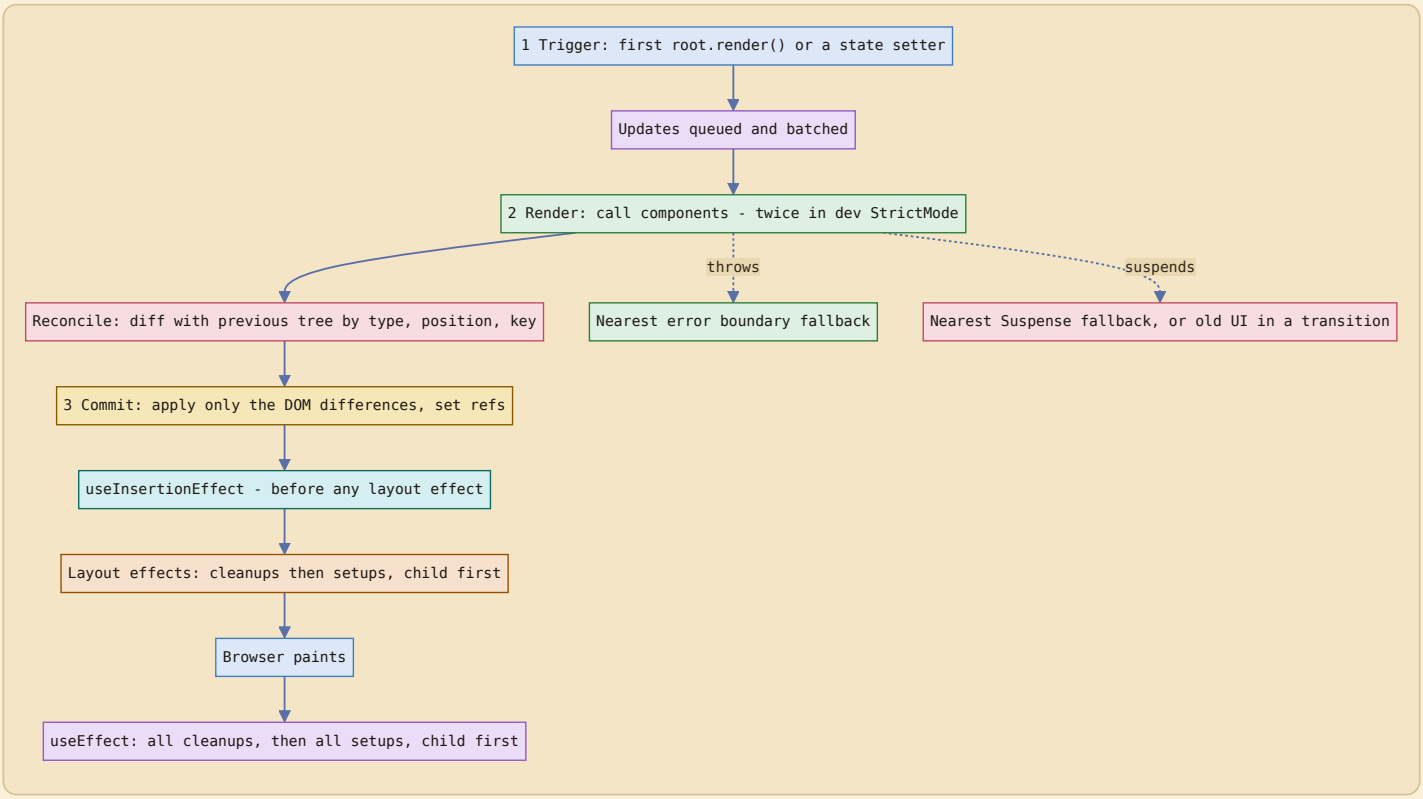
Goal	Use (returns new)	Avoid (mutates)
Change a field	`{ ...obj, field: v }`; nested: copy every level `{ ...u, address: { ...u.address, zip } }`	`obj.field = v`
Add	`[...arr, x]`, `[x, ...arr]`	`push`, `unshift`
Insert at `i`	`arr.toSpliced(i, 0, x)` or `[...arr.slice(0, i), x, ...arr.slice(i)]`	`splice`
Remove	`arr.filter(it => it.id !== id)`	`splice`, `pop`, `shift`
Update one	`arr.map(it => it.id === id ? { ...it, done: true } : it)`, `arr.with(i, x)`	`arr[i] = x`
Sort / reverse	`toSorted()`, `toReversed()`, or `[...arr].sort()`	`sort()`, `reverse()` in place
Deep changes	Immer: `produce(draft => { draft.a.b = 1 })`, `useImmer`	hand-copying 4 levels

Choosing what to store §3.8, §6.1

- Keep the **minimal** state; compute the rest in render (full names, totals, filtered lists, "is valid").
- Store **IDs**, not copies of objects; don't mirror props into state (`useState(prop)` only reads the first value — name it `initialX` if that's the intent).
- One `status: 'idle' | 'sending' | 'sent'` instead of contradictory booleans.
- State as **low as possible, as high as necessary**: lift shared state to the closest common parent §3.7.
- `useState` for independent values; `useReducer` when many handlers change the same state in related ways §6.2.

16.5 The render and commit lifecycle

Every update goes **trigger → render → commit**. "Re-render" means *React called your function again* — the DOM changes only where the output differs. §10.1, §1.3



What the check program observed for a `Parent` → `Child` pair when a prop changes from 1 to 2 (§16.5 check):

Step	Order
Mount	render parent → render child → child **layout** setup → child **effect** setup → parent **effect** setup
Update	render parent → render child → child layout cleanup (v=1) → child layout setup (v=2) → child effect cleanup (v=1) → parent effect cleanup (v=1) → child effect setup (v=2) → parent effect setup (v=2)
Unmount	every remaining cleanup runs, with the values of the render that created it

Phase	Must be	What happens there
Render	**Pure** — may run twice, be paused (concurrent rendering) or thrown away	Call components, compute JSX and derived values
Commit	Runs once per committed update	DOM mutations, refs set, layout effects
After paint	—	`useEffect` setups (subscriptions, fetching, logging)

What triggers a re-render — own state change; **parent re-rendered** (all children by default, props are *not* compared unless `memo`); a **context** it reads changed; a subscribed **external store** changed. Not: mutating props, changing `ref.current`, changing a plain variable. §10.2

16.6 Effects: decision guide

Ask "why does this code run?" §5.1, §5.6

Go down the list and stop at the first "yes":

#	Question	If yes
1	Can it be computed from props or state?	Compute it during render (`useMemo` only if expensive)
2	Is it caused by a specific user action (click, submit)?	Event handler or form Action
3	Must **all** state reset when a prop changes?	Give the component a `key`
4	Is it reading data that lives outside React and changes on its own?	`useSyncExternalStore`
5	Is it fetching server data?	TanStack Query or a router loader — or an Effect **with cleanup**
6	Must it stay in sync with an external system while the component is on screen?	`useEffect` with a cleanup
7	None of the above	Probably no Effect at all

You might not need an Effect §5.6

You want to…	Instead of an Effect
Compute a value from props/state	A variable in render (an Effect costs an extra render and a stale frame)
Cache an expensive calculation	`useMemo`
Reset all state when `userId` changes	`<Profile key={userId} />`
Adjust some state when a prop changes	Derive it (store an ID, look the object up)
Run logic because the user clicked/submitted	The event handler / form Action
Chain updates (A → B → C)	Compute all next state in one handler
Notify the parent of a change	Call its callback in the same handler, or lift the state
Initialise the app once	Module-level code or a `didInit` flag
Subscribe to an external store	`useSyncExternalStore`

What legitimately needs an Effect: sockets and connections, browser events (`resize`, `online`, key shortcuts), timers while visible, non-React widgets (maps, video, `dialog.showModal()`), `document.title`, logging an impression because something was shown, fetching (with cleanup).

Dependencies §5.5

Situation	Fix
The linter wants a value you don't want to react to	Don't silence it — split the logic: reactive part in the Effect, non-reactive part in `useEffectEvent`
Object/function created in render is a dependency	New every render → Effect runs every render. Create it **inside** the Effect or outside the component; depend on primitives
Interval reads state	`setCount(c => c + 1)` so `count` isn't a dependency
Two unrelated jobs in one Effect	Two Effects
Can be left out	setters, `dispatch`, refs, module constants

Cleanup pairs §5.3: `connect`/`disconnect` · `setInterval`/`clearInterval` · `addEventListener`/`removeEventListener` (same function reference) · `subscribe`/`unsubscribe` · `new AbortController` + `fetch(url, { signal })`/`controller.abort()` · `observer.observe`/`observer.disconnect` · `showModal`/`close`.

StrictMode (development only) — you opt in by wrapping the app in `<StrictMode>`, which Vite and most templates do — calls component bodies, state initialisers, updaters and reducers **twice**, runs every Effect **setup → cleanup → setup** on mount, and re-runs ref callbacks. Production runs once. The fix for a visible double is a correct cleanup, never a "has run" flag. §5.4, §2.5

Fetching in an Effect §5.7, §7.6: loading/error/success in one `status` value; `ignore` flag or `AbortController` in cleanup (else the slower old response wins); check `res.ok` (`fetch` doesn't reject on 404); ignore `AbortError`. For real apps prefer TanStack Query or loaders (cache, dedupe, retries, no waterfalls).

16.7 Refs

	useState	useRef
Returns	`[value, setValue]`	`{ current }` — the same object every render
Changing it	setter → **re-render**	`ref.current = x` → **no re-render**, visible immediately
Read during render?	Yes (a snapshot)	No — except lazy init `if (r.current === null) r.current = new Engine()`
Use for	anything shown on screen	timer IDs, DOM nodes, `AbortController`, third-party instances, "in flight" flags

§5.8

DOM refs §5.9, §5.10

- `const inputRef = useRef(null)` + `<input ref={inputRef} />` → `inputRef.current` is the node **after commit** (`null` during the first render), `null` again after removal.
- Good uses: focus, scroll into view, measure (`getBoundingClientRect` in `useLayoutEffect`), media `play()`, `dialog.showModal()`, non-React libraries. Not for changing text/classes React renders.
- Lists: one ref holding a `Map` + a **ref callback** per item. In React 19 a ref callback may **return a cleanup**; use a block body (`ref={n => { x = n; }}`), because an implicit-return arrow now looks like a cleanup to TypeScript.
- To scroll to an item you just added: `flushSync(() => setItems(next))`, then scroll.

Refs to your own components

React 19	Legacy
`function Field({ ref, ...p }) { return <input ref={ref} {...p} /> }`	`forwardRef((props, ref) => …)` — still works, planned for deprecation
`useImperativeHandle(ref, () => ({ focus, clear }))` to expose a limited API	same
`<Fragment ref={r}>` (19.3) → `FragmentInstance` with `focus()`, `addEventListener`, `observeUsing`	wrapper `<div>` just to hold a ref

---

16.8 Forms and Actions (React 19)

Two ways to own a field §4.2, §3.9

	Controlled	Uncontrolled
Value lives in	React state: `value={v}` + `onChange` (`checked` for boxes/radios)	The DOM: `defaultValue` / `defaultChecked`
Read it	From state	`FormData` on submit, or a ref
Re-renders	Every keystroke	None while typing
Good for	Live formatting, limits, dependent fields, counters	Most "fill in and submit" forms; pairs with form Actions
Traps	`value` without `onChange` → read-only + warning; `useState()` (undefined) → "uncontrolled to controlled" warning — start with `''`	Changing `defaultValue` later does nothing (remount with `key`); fields without `name` are skipped

`<input type="file">` is always uncontrolled. React's `onChange` fires on every keystroke (like the DOM `input` event).

`FormData` rules §4.1: only fields with a `name` are sent · every value is a **string** (`Number(fd.get('age'))`) · an **unchecked** checkbox is **absent** · a checked box without `value` sends `"on"` · `getAll('topic')` for repeated names · `Object.fromEntries(fd)` keeps only the last of repeated names · disabled fields are skipped.

The React 19 form toolkit

API	Shape	Remember
Form Action	`<form action={fn}>` → `fn(formData)`	React prevents navigation, runs `fn` in a Transition (may be async), **resets uncontrolled fields after success**; throwing goes to the error boundary — return expected errors instead §4.4
`formAction`	`<button formAction={saveDraft}>`	Per-button override of the form's action §4.4
`useActionState`	`const [state, action, isPending] = useActionState(async (prev, formData) => next, initial)`	Return errors **and** submitted values (use them as `defaultValue`s so a failed submit doesn't wipe the form); calls queue sequentially §4.5
`useFormStatus`	`const { pending, data } = useFormStatus()` (from `react-dom`)	Only in a **child** of the `<form>` — perfect for a reusable Submit button §4.6
`useOptimistic`	`const [opt, add] = useOptimistic(real, (cur, x) => …)`	Call `add` inside the Action; update the real state on success; rollback is automatic §4.7
`requestFormReset(form)`	from `react-dom`, inside a Transition	Reset by hand §4.4
Server Function	`'use server'` async function used as `action`	Works before hydration (progressive enhancement); arguments must be serializable; authenticate and validate inside §4.8, §13.8

Validation §4.3: three layers — HTML constraints (`required`, `type="email"`, `minLength`, `pattern`), a **pure** `validate(values)` computed during render, and **server validation (mandatory)**. Show errors on blur or after the first submit, then live while fixing; `aria-invalid="true"` + `aria-describedby` to the message; focus the first invalid field; `noValidate` hands the UI to you while `validity` keeps working. Big forms: React Hook Form + Zod (`zodResolver`), with the same schema on the server §4.9.

16.9 Context and choosing state management

Context in three steps §6.3: `const Ctx = createContext(default)` → `<Ctx value={v}>…</Ctx>` (React 19; `<Ctx.Provider>` still works) → `useContext(Ctx)` or `use(Ctx)` (conditional allowed). Nearest provider wins; the default applies **only when there is no provider** (a provider with `value={undefined}` gives `undefined`).

Context performance §6.4, §10.10

Problem	Fix
`value={{ user, setUser }}` is a new object every render → every consumer re-renders, `memo` can't stop it	`useMemo` the value (`useCallback` for functions) — or let the React Compiler do it
Consumers that need only actions re-render when data changes	Split into a state context and an actions/dispatch context
One big context with unrelated values	One context per concern
High-frequency values (typing, mouse, timers)	Keep local, or use a store with selectors

Reducer + context store §6.5: `useReducer` in a provider component, two contexts (state, dispatch), and `useTasks()` / `useTasksDispatch()` Hooks that **throw** outside the provider (use `null` as the default so the check is possible).

Where does this state live? §6.1, §6.10

Kind of state	Examples	Home
Local UI	open/closed, current tab, input text	`useState` in the component
Complex local	wizard, editor	`useReducer`
Shared by nearby components	selected item	Lift to the closest common parent
App-wide, changes rarely	theme, locale, current user	Context
App-wide, changes often, read by many	cart, editor document, dashboard filters	**Zustand** (small, no provider) or **Redux Toolkit** (structure, DevTools, middleware)
Server data	lists, profiles, search results	**TanStack Query** / RTK Query / router loaders / Server Components
Shareable / bookmarkable	filters, sort, page, selected ID	The **URL** (`useSearchParams`, route params)
Form fields	values, errors, pending	Form Actions + `useActionState`, or React Hook Form
Derived	totals, filtered lists	**Not state** — compute in render

Library essentials §6.7, §6.8

Redux Toolkit	Zustand
`createSlice({ name, initialState, reducers })` → reducer + action creators; reducers may "mutate" (Immer) — mutate the draft **or** return new state, not both	`const useStore = create((set, get) => ({ n: 0, inc: () => set(s => ({ n: s.n + 1 })) }))` — `set` merges one level
`configureStore` adds thunks, DevTools, mutation/serializability checks	No provider; `useStore.getState()/setState()/subscribe()` work outside React
`useSelector(sel)` re-renders only when the selected value changes; `useDispatch()`	`useStore(s => s.n)`; several values → wrap in `useShallow`
Selector returning a new array/object → re-render on every dispatch; memoise with `createSelector`	Selector returning a new object without `useShallow` → "Maximum update depth exceeded" loop
Async: `createAsyncThunk` (pending/fulfilled/rejected) or **RTK Query**	Middleware: `persist`, `devtools`, `immer`

16.10 Routing (React Router v8) and TanStack Query

React Router v8 — one package (`npm i react-router`; `react-router-dom` is gone), requires React 19.2.7+ and Node 22.22+, ESM-only, middleware on by default. `RouterProvider` comes from `react-router/dom`. §9.2

Mode	You write	You get
Declarative	`<BrowserRouter><Routes><Route path element/></Routes></BrowserRouter>`	Matching, links, params
Data	`createBrowserRouter([{ path, Component, loader, action, ErrorBoundary, children }])` + `<RouterProvider>`	+ loaders, actions, pending UI, route error boundaries, fetchers, middleware
Framework	`app/routes.ts` + route modules + Vite plugin	+ SSR / SPA / prerender, per-route code splitting, generated types §13.10

Task	API §9.3–§9.7
Dynamic segment	`path="/products/:id"` → `useParams()` → `{ id: '42' }` (**always strings**)
Ranking	Static segments beat `:params`, which beat `*` — order in the file doesn't matter; `path="*"` = 404
Links	`<Link to>`; `<NavLink>` gets `isActive` (`end` for exact); relative links resolve against the **route**, `..` = parent route
Query string	`const [params, setParams] = useSearchParams()` — shareable UI state
Navigate in code	`const navigate = useNavigate(); navigate('/x')`, `navigate(-1)`, `{ replace: true }`; or render `<Navigate to>`
Layouts	Parent renders `<Outlet />`; `index` route for the default child; pathless route for shared UI; `<Outlet context>` + `useOutletContext()`
Load before render	`loader({ params, request, context })` → `useLoaderData()`; loaders of all matched routes run **in parallel**; `useNavigation().state` = `'idle' | 'loading' | 'submitting'`
Errors	`throw data('Not found', { status: 404 })` → route `ErrorBoundary`, check `isRouteErrorResponse(error)`
Writes	`<Form method="post">` → route `action` → `return data({ errors }, { status: 400 })` (`useActionData`) or `redirect('/list')`; loaders **revalidate** automatically
Without navigating	`useFetcher()` — `fetcher.Form`, `fetcher.state`, `fetcher.data`
Guards, logging	`middleware: [fn]` with `await next()`; `createContext()` + `context.get/set` (UX only — the server still authorises)
Code splitting	`{ path, lazy: () => import('./route.jsx') }`
Tests	`MemoryRouter initialEntries` / `createMemoryRouter`

TanStack Query essentials §6.9, §9.8–§9.10

Concept	Remember
Setup	One `QueryClient` created **once** (module scope or `useState(() => new QueryClient())`), `<QueryClientProvider client>`
Read	`useQuery({ queryKey: ['product', id], queryFn })` — every input of `queryFn` goes in the key; `queryFn` must **throw** on failure (check `res.ok`)
Defaults	`staleTime` **0** (stale at once → background refetch on mount, window focus, reconnect), `gcTime` **5 min**, **3 retries** in the browser with backoff
Flags	`isPending` = no data yet · `isFetching` = request in flight (incl. background) · `isLoading` = both · `isError`, `error`, `refetch()`
Dedupe & cache	Same key at the same time → one request; remount within `staleTime` → no request
Keys	Arrays, compared by value, hierarchical: invalidating `['products']` also hits `['products', 7]`
Reuse	`queryOptions({ queryKey, queryFn })` shared by `useQuery`, `useSuspenseQuery`, `ensureQueryData`, `prefetchQuery`
Pagination / dependent	`placeholderData: keepPreviousData` (`isPlaceholderData`) · `enabled: !!id` · `useInfiniteQuery` for "load more"
Writes	`useMutation({ mutationFn, onSuccess: () => qc.invalidateQueries({ queryKey }) })`
Optimistic	`onMutate`: `cancelQueries` → snapshot → `setQueryData` → return rollback · `onError`: restore · `onSettled`: invalidate
With a router	Loader calls `queryClient.ensureQueryData(opts)` (critical) or `prefetchQuery` (non-critical); component reads the same `opts` §9.13
Tests	New `QueryClient` per test with `retry: false` §12.6

Suspense and `use()` §9.11: `<Suspense fallback>` shows a placeholder while `lazy` code, `use(promise)` or Suspense-enabled data (`useSuspenseQuery`, loaders, async Server Components) isn't ready — data fetched in `useEffect` does **not**

suspend. A **transition** keeps already-visible content instead of flashing the fallback. Pair each important boundary with an error boundary. Loading UX: skeletons for first loads, old content + progress bar for navigation, nothing for fast refreshes, inline errors for validation, Retry buttons for failures §9.12.

16.11 Performance: checklist, memo and the React Compiler

Checklist — in this order, stop when fast enough §10.11

1. Reproduce with a **production build**, CPU throttled, realistic data.
2. Find the slow interaction: React DevTools **Profiler** (enable "why did this render"), "Highlight updates", Chrome Performance panel with React tracks (19.2+).
3. Slow **load**: bundle analyser → `lazy` routes and heavy widgets, images, SSR/SSG.
4. Slow **interaction**, too much re-renders → **structure first**: colocate state, pass JSX as `children`, stable keys, split/memoise context; then `memo` + stable props, or the React Compiler.
5. One slow component → `useMemo` the calculation (≥ ~1 ms) or fix the algorithm.
6. Too many DOM nodes → virtualize (TanStack Virtual, react-window) or paginate.
7. Typing lags behind results → `useDeferredValue` / `useTransition` with a `memo`'d slow child.
8. Measure again; keep only changes that made a measurable difference.

`memo` **rules** §10.4, §10.5

- `memo(C)` skips a re-render **caused by the parent** when every prop is `Object.is`-equal. It does not block the component's own state changes or context changes.
- Inline objects, arrays, functions and JSX (`style={{…}}`, `onAdd={() => …}`, `children`) are new every render → `memo` does nothing (V2 in §10.4). Stabilise with `useCallback` / `useMemo`, module constants, or primitives.
- Don't bother for cheap components or props that change anyway; a custom `arePropsEqual` that ignores a callback causes stale closures.
- Free wins without `memo`: state moved down, and **elements passed as `children` from above** keep their identity (the check in §16.11 shows `Shell` re-rendering with 0 child renders).

The four common performance bugs §10.10: component declared inside a component (remount every render) · unstable keys · a new context object each render · state owned too high (every keystroke re-renders the page). Also: Effects that set state every render, fetch waterfalls, huge images/libraries, development builds in production.

React Compiler §10.7

Fact	Detail
What	Build-time Babel plugin (`babel-plugin-react-compiler`); v1.0 stable 7 October 2025; best on React 19 (17/18 need `react-compiler-runtime`)
Does	Memoizes values, callbacks and JSX per component with a cache (`useMemoCache` from `react/compiler-runtime`) — `memo` / `useMemo` / `useCallback` become mostly unnecessary
Needs	The Rules of React: pure render, no mutation of props/state, no `ref.current` during render — it **skips** components that break them
Verify	"Memo ✨" badge in React DevTools; lint rules ship in `eslint-plugin-react-hooks`
Escape hatches	`"use no memo"` directive; keep `useMemo` / `useCallback` when an Effect dependency's identity must be guaranteed
Doesn't fix	Slow algorithms, huge DOM trees, big bundles

Code splitting §10.8: `const Chart = lazy(() => import('./Chart.jsx'))` at **module level**, rendered inside `<Suspense fallback>` with an error boundary for failed chunks; `lazy` needs a **default export** (map named ones: `.then(m => ({ default: m.Chart }))`); preload on hover/focus by calling the same `import()`.

Virtualization math (fixed rows) §10.9: `first = floor(scrollTop / rowHeight) − overscan`, `count = ceil(height / rowHeight) + 2 × overscan`, absolutely positioned rows inside a spacer `items.length × rowHeight` tall — 300 px / 30 px rows / overscan 3 → **16 DOM rows** for 10,000 items.

16.12 Styling

Approach	Write	Scoping	Runtime	Pick when §8.10
Plain CSS	`import './Card.css'`, `className="card"`	**None** — imports are global; BEM or `@layer`	none	Content sites, small apps
Inline `style`	`style={{ width: `${p}%` }}`	Per element	tiny	Truly dynamic values, ideally setting a CSS variable (`style={{ '--progress': … }}`); no `:hover`/media queries
CSS Modules	`Card.module.css`, `className={styles.card}`	Automatic per file (`composes`, `:global`)	none	Teams that like writing CSS
Tailwind v4	`className="p-4 rounded-xl hover:bg-blue-700"`	Nothing to scope	none	Fast iteration; configure tokens with `@theme`
Runtime CSS-in-JS	`styled.button`…`` (styled-components, Emotion)	Automatic	**yes**	Existing codebases; styled-components is in maintenance mode (March 2025); awkward with Server Components
Zero-runtime CSS-in-JS	vanilla-extract, StyleX, Panda	Automatic	none	Typed, colocated styles without the runtime

Rules worth memorising §8.3–§8.8

- `style` numbers get `px` except unitless properties; vendor prefixes are capitalised (`WebkitLineClamp`, but `msTransform`); inline styles beat stylesheets (only `!important` overrides them).
- Build class strings with `clsx`; always accept a `className` prop; use variant maps, not ternary chains. Tailwind: write **complete** class names (`bg-${color}-500` is never generated); resolve conflicts with `tailwind-merge`.
- Theming: tokens as CSS custom properties on `:root`, overridden under `[data-theme="dark"]`; a provider writes the attribute; a blocking `<head>` script prevents the flash of the wrong theme; `color-scheme` for native controls; CSS recolours without re-rendering components.
- React 19 hoists `<link rel="stylesheet" precedence>` and `<style href precedence>` into `<head>` and de-duplicates them.
- Component libraries: styled kits (MUI, Mantine) for speed; headless (Radix, React Aria, Headless UI) for custom design with accessibility; copy-in (shadcn/ui) to own the code §8.9.

16.13 TypeScript + React types

Bundlers (Oxc, esbuild, SWC) only **strip** types; `tsc --noEmit` **checks** them — run it in CI. Use `strict: true` and `"jsx": "react-jsx"`. §11.1

Need	Type / pattern
Props	`type Props = { label: string; count?: number; tone?: 'info' | 'error' }; function Badge({ label, count = 0 }: Props)` §11.2
Callback prop	`onSelect?: (id: string) => void`, called as `onSelect?.(id)`
Children, slots	`children: ReactNode` (anything renderable) · `PropsWithChildren<P>` · `ReactElement` only when it must be one element §11.3
Native element props	`ComponentProps<'button'> & { variant?: 'primary' | 'ghost' }` (includes `ref`) · `ComponentPropsWithoutRef<'input'>` · `ComponentProps<typeof MyComp>`
Change event	`ChangeEvent<HTMLInputElement>` / `HTMLSelectElement` / `HTMLTextAreaElement` — or `ChangeEventHandler<…>` §11.4
Submit	`SubmitEvent<HTMLFormElement>` in recent `@types/react` 19 (`FormEvent` still compiles, now deprecated)
Click, keys, focus	`MouseEvent<HTMLButtonElement>`, `KeyboardEvent<HTMLInputElement>`, `FocusEvent<…>` — imported from `react`, not the DOM globals
Inline handler	No annotation — contextually typed
State	Inferred from the initial value; add `<T>` for `useState<User | null>(null)`, `useState<string[]>([])` (else `never[]`), literal unions §11.5
Setter as a prop	`Dispatch<SetStateAction<T>>`
Reducer actions	Discriminated union `{ type: 'add'; sku: string } | { type: 'clear' }` + `const _: never = action` in `default` §11.5
DOM ref	`useRef<HTMLInputElement>(null)` → `current: HTMLInputElement | null`; React 19 types **require** an argument §11.6
`ref` prop / handle	`ref?: Ref<HTMLInputElement>`; custom handle `Ref<Handle>` + `useImperativeHandle`
Tuple from a Hook	`return [on, toggle] as const` — or return an object §11.7
Context with no default	`createContext<Auth | null>(null)` + `useAuth(): Auth` that throws outside the provider
Generic component	`function List<T>({ items, renderItem }: { items: T[]; renderItem: (t: T) => ReactNode })`; `<T,>` in arrow functions; constraints with `extends`; field names with `keyof T & string` §11.8
Mutually exclusive props	Union per case; `prop?: never` forbids a prop; narrow `props` before destructuring §11.9
Untrusted data	`unknown` + narrowing, or a Zod schema — `as T` is not validation §11.7

Read type errors bottom-up: *"Type A is not assignable to type B"*, then the last indented line names the property that differs §11.10.

16.14 Testing: query priority and user-event

Philosophy §12.1: render → find like a user → act like a user → assert on what the user sees. Don't assert on state, props, Hook calls or class names.

Query priority (Testing Library) §12.3

1. `getByRole(role, { name })` — almost everything (`button`, `link`, `heading` + `level`, `textbox`, `checkbox`, `combobox`, `dialog`, `alert`, `status`, `list`/`listitem`…)
2. `getByLabelText` — form fields
3. `getByPlaceholderText` — only if there's truly no label
4. `getByText` — non-interactive text
5. `getByDisplayValue` — a field by its current value
6. `getByAltText`, `getByTitle`

7. `getByTestId` — last resort

Variant	0 matches	1	2+	Waits?	Use to…
`getBy…`	throws	element	throws	no	find what must be there now
`queryBy…`	`null`	element	throws	no	assert absence
`findBy…`	rejects	element	rejects	yes (default 1000 ms)	wait for something to appear
`getAllBy…` / `queryAllBy…` / `findAllBy…`	throws / `[]` / rejects	array	array	as above	several

user-event §12.4: `const user = userEvent.setup();` then **`await`** `user.click`, `user.type(el, 'hi{Enter}')`, `user.keyboard('{Escape}')`, `user.tab()`, `user.selectOptions`, `user.clear`, `user.upload`, `user.hover`. It produces the full sequence (focus, keydown, input, keyup…), respects `disabled` and focus order; `fireEvent` dispatches exactly one event (keep it for `scroll` and custom events).

Async §12.5: `await screen.findByRole(…)` (appear) · `await waitForElementToBeRemoved(() => screen.queryByText('Loading…'))` (disappear) · `await waitFor(() => assertion)` (anything else — assertions only inside) · never `sleep`. Forgetting `await` → passes by accident + `act(...)` warning.

Everything else §12.6–§12.9: mock the **network boundary** (MSW, or `mock.method(globalThis, 'fetch')` / `vi.spyOn` returning real `Response`s) and cover success, HTTP error, network error and races · Hooks through a component or `renderHook` + `act` + `result.current` · fake timers for debounce (`advanceTimers` option for user-event) · a `renderWithProviders` with `wrapper` creating a **fresh** store / `QueryClient` per test · Vitest: `environment: 'jsdom'`, `globals: true`, setup file importing `@testing-library/jest-dom/vitest` · jsdom has no layout (sizes are 0; `IntersectionObserver`, `matchMedia` need mocks).

16.15 Rendering strategies and directives

Strategy	HTML made where / when	First HTML has	Needs a server	Good for
CSR (SPA)	Browser, after JS loads	empty `<div id="root">`	No (static hosting)	Dashboards and tools behind a login
SSR	Server, every request	full content	Yes	Personalised or fast-changing public pages
SSG	Build machine, once per deploy	full content	No (CDN)	Docs, blogs, marketing
ISR	Build, then re-built in the background after a window or on demand (framework feature)	full content, stale for at most N s	Yes	Catalogues, news
Streaming SSR	Server, in chunks per `<Suspense>` boundary	shell + fallbacks, slow parts later	Yes	Fast frame, slow widgets
PPR	Static shell at build + dynamic holes per request (`prerender` → `postponed` → `resume`)	static shell with placeholders	Yes	Mixed static/per-user pages (Next.js Cache Components)
RSC	Some components run **only** on the server (build or request)	works with any row above	Yes (or build time)	Data-heavy pages with little interactivity; zero JS for server parts

§1.4, §13.1, §13.6

Server APIs §13.2

Function	Package	Waits for Suspense data?	Use
`renderToString`	`react-dom/server`	No — emits the fallback marked `<!--$!-->` (client renders it)	Legacy / simple pages
`renderToStaticMarkup`	`react-dom/server`	No; no hydration markers (`<!-- -->`)	Emails, static snippets (can't be hydrated)
`renderToPipeableStream`	`react-dom/server` (Node)	Streams later	SSR on Node: status in `onShellReady`, `onAllReady` for crawlers, always `abort()` on a timeout
`renderToReadableStream`	`react-dom/server` (Web Streams)	Streams later	Edge, Deno, Bun, Workers
`prerenderToNodeStream` / `prerender`	`react-dom/static`	**Yes**	SSG; `prerender` + abort → PPR
`resume` / `resumeToPipeableStream`	`react-dom/server` (Web / Node)	Streams	Fill PPR holes per request
`resumeAndPrerender` / `resumeAndPrerenderToNodeStream`	`react-dom/static`	**Yes**	Continue a postponed prerender at build time
`hydrateRoot(domNode, <App/>)`	`react-dom/client`	—	Adopt server HTML in the browser (never `createRoot` on server HTML)

Streaming markers §13.3: `<!--$-->` complete boundary · `<!--$?-->` pending (fallback inside) · `<!--$!-->` failed on the server → client renders it · `<template id="B:0">` where content will go · `<div hidden id="S:0">` finished content · `$RC("B:0","S:0")` swap script.

Hydration §13.4, §13.5: same tree + same data on the client; React **reuses** the DOM and attaches handlers; streaming allows **selective hydration** (the shell is interactive before slow parts arrive; clicked boundaries hydrate first); visible ≠ interactive. Mismatch sources: `window`/`localStorage` checks, `Date.now()`, `Math.random()`, locale/time zone, invalid nesting (`<div>` in `<p>`), refetched data, extensions. Fix: render the server version first and switch after mount (`useEffect` flag, or `useSyncExternalStore` with `getServerSnapshot`); for a whole browser-only component, `use(browser())` (React 19.3, `react-dom`) inside `<Suspense>` — it suspends only on the server, so the client renders it; `useId` for IDs; `suppressHydrationWarning` only for one expected difference (it keeps the **server** text). Hooks run on the server once; **Effects never run on the server**.

Server vs Client Components §13.7

	Server Component	Client Component
Marked by	Nothing — the default in RSC frameworks	A `'use client'` entry file (and everything it imports)
Runs	Server only (build or request)	Server for SSR HTML **and** the browser
Code shipped	None	Yes
Can	be `async`, `await` a database, use secrets and files	use state, Effects, event handlers, browser APIs, context
Can't	use state, Effects, handlers, `createContext`/`useContext`/`use(Ctx)` — but since 19.3 it may **render** a context provider imported from a `'use client'` module	import a Server Component (but can render one passed as `children`)

Crossing the boundary — props from server to client must be serializable: primitives, plain objects/arrays, `Date`, `Map`, `Set`, typed arrays, JSX, **promises** (read with `use`), **Server Functions**. Not: ordinary functions, class instances, null-prototype objects, local symbols.

Directives §13.8

Directive	Where	Means
`'use client'`	First statement of a **file**	Start of client code: the module and its imports ship to the browser. Push it to the leaves
`'use server'`	Top of a file, or first line of an `async` function	**Server Function**: runs on the server, callable from the client (a public POST endpoint — authenticate, authorise, validate)
(none)	—	Server Component (there is **no** "use server component" directive)
`'use cache'`	Next.js only	Cache a function/component under Cache Components §13.9
`'use no memo'`	Component/file	Opt out of the React Compiler §10.7
`import 'server-only'`	Module	Build fails if a Client Component imports it

Frameworks §13.9–§13.11: **Next.js 16** App Router — folders are routes; `page`, `layout`, `loading`, `error` (client), `not-found`, `route` files; `await params`; `revalidatePath` / `revalidateTag` / `updateTag`; `proxy.js` (was `middleware.js`); opt-in Cache Components. **React Router 8 framework mode** — `routes.ts`, route modules with `loader` / `clientLoader` / `action` / `ErrorBoundary`, `ssr` / `prerender` config, RSC experimental. **TanStack Start** — full-stack on TanStack Router, v1 release candidate, RSC experimental. **Choose**: public + SEO → Next.js or React Router framework; behind a login → Vite SPA + React Router + TanStack Query; mobile → Expo; mostly content → SSG/Astro.

16.16 Patterns

Need	Pattern	Shape	Watch out
Flexible UI structure	**Composition**	`children` + slot props	Prefer to growing lists of boolean props §14.1
Share **stateful logic**	**Custom Hook**	`useProductSearch(query)` → small API	Shares logic, not state; name `use…` only if it calls Hooks §7.3, §14.3
Separate data from looks	**Container / presentational**	Hook or Server Component + props-only view	Not by rule — where reuse/testing benefits §14.2
Parts sharing implicit state	**Compound components**	`<Tabs><Tabs.List><Tabs.Tab/></Tabs.List><Tabs.Panel/></Tabs>` via private context	Throw a clear error outside the root; support controlled + uncontrolled §14.4
Component decides where/how often caller markup renders	**Render prop**	`renderItem={(item) => …}`, `children` as a function	Callback pyramids; a Hook is simpler for logic only §14.5
Wrap many components with the same behaviour	**HOC**	`withAuth(Page)`	Create at module level (in render → remount every time); `displayName`; prop collisions §14.6
Escape `overflow`/stacking	**Portal**	`createPortal(node, document.body)`	Events bubble through the **React** tree; manage focus; read the latest `onClose` in the Escape-key Effect with `useEffectEvent`; consider native `<dialog>` §14.8
Contain crashes	**Error boundary**	Class with `getDerivedStateFromError` + `componentDidCatch` (or `react-error-boundary`)	Catches render/lifecycle/Effect errors below it, `use()` rejections and `lazy` failures — **not** event handlers, async callbacks, SSR or itself §14.9
Wait for code/data	**Suspense boundary**	`<Suspense fallback>`	Pair with an error boundary; transitions keep old UI §9.11
Organise files	**By feature**	`features/cart/index.js` public API; `app → features → shared`	Enforce with lint; avoid giant barrels §14.11
Consistency	**Design system**	Tokens → accessible primitives → docs; closed variants, `type="button"` default, pass-through props and `ref`	Build on headless primitives §14.12

React 19 root error options: `createRoot(el, { onCaughtError, onUncaughtError, onRecoverableError })`. With **no** boundary, an uncaught render error unmounts the **whole root** §14.9.

16.17 Accessibility checklist

§14.10, §7.8, §4.3

- [ ] **Semantic elements first**: `<button>` (not `<div onClick>`), `<a href>`, `<nav>`, `<main>`, headings in order, lists, `<form>` + `type="submit"` (Enter works for free).
- [ ] Every input has a **label**: `<label htmlFor={id}>` with `id` from `useId()` (never hard-coded in reusable components); placeholders aren't labels.
- [ ] Images have `alt` (empty `alt=""` for decoration); icon-only buttons have `aria-label` and `aria-hidden="true"` on the icon.
- [ ] ARIA driven by state and spelled with dashes: `aria-expanded={open}`, `aria-selected`, `aria-current`, `aria-controls`, `aria-invalid`, `aria-describedby`. First rule of ARIA: use a native element when one exists.
- [ ] **Live regions** (`role="status"` polite, `role="alert"` urgent) exist in the DOM *before* their text changes.

- [ ] **Focus management** with refs: into a dialog on open, trapped (or background `inert`), back to the trigger on close; to the first invalid field on a failed submit; to the new page's `<h1>` (`tabIndex={-1}`) after client-side navigation.
- [ ] Keyboard patterns: Escape closes, arrow keys move inside tabs/menus/listboxes (roving `tabIndex`), everything reachable with Tab, visible `:focus-visible` styles (never remove outlines without a replacement).
- [ ] Contrast ≥ 4.5:1 for normal text (WCAG 2.2), in both themes; don't signal errors by colour alone.
- [ ] Update `document.title` (React 19 lets any component render `<title>`).
- [ ] Test with `getByRole(role, { name })`, `eslint-plugin-jsx-a11y`, axe (`vitest-axe`, Playwright), and a real keyboard + screen reader.
- [ ] Complex widgets (menus, comboboxes, date pickers): use headless libraries (Radix, React Aria) rather than hand-rolling ARIA.

16.18 Common errors and fixes

Every message below appeared in the output of an earlier chapter's program. The React, Testing Library and props messages are reproduced verbatim by the check program at the top of this chapter (first sentence shown); the Hook-order warning comes from Chapter 7's run and the TypeScript errors from Chapter 11's `tsc` run.

Message (React 19.3, development)	Usual cause	Fix	Read
Too many re-renders. React limits the number of renders to prevent an infinite loop.	A setter called **during render** — often `onClick={setOpen(true)}` or `onClick={handle()}`	Pass a function: `onClick={() => setOpen(true)}`; move logic to a handler	§3.1
Objects are not valid as a React child (found: object with keys {…}).	Rendering an object (`{user}`, a `Date`, a Promise)	Render a field `{user.name}`, format it, or `map` an array	§1.8
Each child in a list should have a unique "key" prop.	`map` without `key`, or key on the wrong element	`key={item.id}` on the outermost element returned by `map`	§2.8
Encountered two children with the same key, `1`.	Duplicate IDs, or keys built from non-unique fields	Use a truly unique, stable ID	§2.8
The tag <profile> is unrecognized in this browser.	Component name starts with a lowercase letter	Capitalise: `Profile`	§2.1
Invalid DOM property `class`. Did you mean `className`?	HTML attribute names in JSX	`className`, `htmlFor`, camelCase events	§1.8
Unsupported style property @media (max-width: 600px).	Selectors/media queries in the `style` object	Use CSS (a class, CSS Module, Tailwind variant)	§8.3
You provided a `value` prop to a form field without an `onChange` handler.	Controlled `value` with no handler → read-only field	Add `onChange`, or use `defaultValue`, or `readOnly`	§3.9
A component is changing an uncontrolled input to be controlled.	`value` was `undefined`/`null`, later a string	Initialise with `''`	§4.2
Rendered more hooks than during the previous render. / Rendered fewer hooks than expected. This may be caused by an accidental early return statement.	A Hook inside an `if`, loop, or after an early `return`	Call every Hook unconditionally at the top; put the condition inside the Hook	§7.2
React has detected a change in the order of Hooks called by …	Same count, different Hook kinds between renders (conditional Hooks)	Same fix — stable call order	§7.2
Invalid hook call. Hooks can only be called inside of the body of a function component.	Hook called in a plain function, handler or module scope; calling a Hook outside `renderHook` in tests; **two copies of React** in the bundle	Call Hooks only in components/custom Hooks; dedupe React (`npm ls react`)	§7.2, §12.7
The result of getSnapshot should be cached to avoid an infinite loop → Maximum update depth exceeded.	`useSyncExternalStore` `getSnapshot` returns a new object each call (same with a Zustand selector returning a new object)	Return the same reference until the store changes; select primitives; `useShallow`	§7.9, §6.8
A component was suspended by an uncached promise.	`use(fetch(...))` — a new promise on every render	Create the promise once (cache, loader, Server Component prop) or use `useSuspenseQuery`	§9.11
Element type is invalid. Received a promise that resolves to: undefined. Lazy element type must resolve to a class or function.	`lazy(() => import('./X'))` where `X` has no **default** export	`.then(m => ({ default: m.X }))`	§10.8
Hydration failed because the server rendered text didn't match the client.	Render output differs between server and client (`window`, dates, random, locale, invalid nesting)	Same first render; switch after mount; `useId`; fix the HTML nesting	§13.5
An update to T inside a test was not wrapped in act(...).	Something updated state after the test stopped waiting (missing `await`, un-awaited timer or fetch)	`await findBy…` / `waitFor`; `act` around direct Hook calls and fake-timer advances	§12.2, §12.5
React has blocked a javascript: URL as a security precaution. (in the rewritten `href`)	A `javascript:` URL in `href`	Validate URLs; use a button with a handler	§1.8

Message (React 19.3, development)	Usual cause	Fix	Read
`TypeError: Cannot assign to read only property 'title' of object`	Mutating props (frozen in development)	Ask the parent via a callback, or use local state	§2.2
`Cannot read properties of null (reading 'price')` inside a boundary	Rendering before data exists / missing provider	Handle loading states; error boundary + reset keys; context Hook that throws a clear message	§14.9, §6.5
`useTasks must be used inside <TasksProvider>` *(your own message)*	Component rendered outside its provider	Wrap the tree (or the test) in the provider	§6.5, §12.8
RTL: *Unable to find an accessible element with the role "button" and name "…"*	Wrong name — or the element isn't accessible (a `<div onClick>`, an unlabelled icon button)	Read the printed role list; fix the query **or** the markup	§12.3, §14.10
RTL: *Found multiple elements with the role "button"*	Query not specific enough; DOM left over from a previous test	Add `{ name }`, use `within()`, run `cleanup` after each test	§12.3, §12.2
TS2322 *Type 'string' is not assignable to type 'never'.*	`useState([])` inferred `never[]`	`useState<string[]>([])`	§11.10
TS18047 *'user' is possibly 'null'.*	Using nullable state without narrowing	`user ? … : …`, `user?.name`	§11.10
TS2554 *Expected 1 arguments, but got 0.*	`useRef<HTMLInputElement>()` with React 19 types	`useRef<HTMLInputElement>(null)`	§11.10
TS2741 *Property 'label' is missing in type '{}' but required in type 'BadgeProps'.*	Required prop not passed	Pass it, or make it optional with a default	§11.10

Silent bugs (no message at all) — the ones to recognise by symptom:

Symptom	Cause	Fix
Counter goes up by 1 instead of 3	Snapshot + plain values	Updater functions §3.5
Screen doesn't update after "changing" state	Mutation, same reference	New object/array §3.6
Typed text jumps to another row	Index keys + insert/sort	Stable IDs §2.8
Input loses focus on every keystroke	Component defined inside a component, or random keys	Top-level components, stable keys §10.10
Effect runs twice in development	StrictMode double run	Correct cleanup §5.4
Old search results overwrite new ones	Fetch race	`AbortController` / ignore flag §5.7
`memo` child still re-renders	Inline object/function props, or context	Stable props, split/memoise context §10.4
Interval always sees the first value	Stale closure (`[]` deps)	Updater, or `useEffectEvent` §7.7
Changing a prop doesn't reset the child's state	Same type at same position keeps state	`key` §3.10
Clicks inside a modal trigger the parent's "click outside"	Portal events bubble through React	Check `e.target`, or stop propagation deliberately §14.8
Error in `onClick` doesn't show the fallback	Boundaries don't catch handler errors	`try/catch` → state §14.9
Page reloads on submit, state lost	No `preventDefault` / no function `action`	`e.preventDefault()` or `<form action={fn}>` §4.1
404 when refreshing `/products/3` in an SPA	Server doesn't fall back to `index.html`	Configure the fallback §9.3

16.19 Interview question index

The 53 questions of Chapter 15, grouped as there. Each link opens the section that holds the question; the question numbers are in bold there.

Area	Questions
Rendering, reconciliation, keys	Q1 what happens after a setter · Q2 render vs commit, which must be pure · Q3 reconciliation and "virtual DOM" · Q4 why keys, why not the index · Q5 changing a component's `key` · Q6 when a component re-renders · Q7 why `{items.length && …}` renders 0
State, events, forms	Q8 three `setCount(count + 1)` · Q9 batching and React 18 · Q10 immutability · Q11 derived state, props copied into state · Q12 where state should live · Q13 controlled vs uncontrolled · Q14 `useActionState`, `useFormStatus`, `useOptimistic` · Q15 React vs DOM events
Effects and refs	Q16 what `useEffect` is for · Q17 no array vs `[]` vs `[a, b]` · Q18 cleanup timing and Effect order · Q19 why things run twice in development · Q20 fetch race conditions · Q21 `useEffect` vs `useLayoutEffect` · Q22 `useRef` vs `useState` · Q23 refs to your own components in React 19 · Q24 `useEffectEvent`
Hooks, context, state management	Q25 Rules of Hooks · Q26 custom Hooks share state? · Q27 `useState` or `useReducer` · Q28 is Context state management? · Q29 Context performance · Q30 choosing an approach · Q31 server vs client state · Q32 `useSyncExternalStore`
Performance and the Compiler	Q33 `memo` · Q34 `useMemo` vs `useCallback` · Q35 React Compiler · Q36 investigating a slow page · Q37 code splitting · Q38 10,000 rows
Suspense, `use()`, transitions	Q39 what can suspend · Q40 `use()` · Q41 `useTransition` vs `useDeferredValue` · Q42 fallback on a suspending update
Server rendering and RSC	Q43 CSR/SSR/SSG/streaming · Q44 hydration and mismatches · Q45 Server vs Client Components · Q46 `'use server'` · Q47 `'use client'` on the root layout
Errors, portals, patterns, testing, a11y, TS	Q48 what error boundaries catch · Q49 portals and event bubbling · Q50 compound vs render props vs HOCs vs Hooks · Q51 Testing Library philosophy · Q52 accessible components · Q53 typing props, children, events

Machine-coding builds (each with requirements, state design, code, an RTL test and a screenshot): §15.10 todo list with filters · §15.11 autocomplete (debounce, abort, combobox keyboard) · §15.12 infinite scroll (`IntersectionObserver`) · §15.13 modal (portal, focus trap, Escape) · §15.14 accessible tabs (roving tabindex) · §15.15 star rating (native radios) · §15.16 pagination (pure range function) · §15.17 multi-step form wizard · §15.18 ten more prompts mapped to techniques. The method: requirements → component tree → state design → happy path → keyboard/a11y → test → edge cases §15.1.

16.20 Where to go next

Step 1 — Lock in the core (1–2 weeks). Rebuild three of the Chapter 15 builds from a blank file without looking, with tests. If you get stuck, the sheets in this chapter tell you which section to reread. Make sure you can explain snapshot + updaters, keys, Effects vs handlers and Context re-renders out loud — they are the most common interview topics.

Step 2 — Build one real app (2–4 weeks). Something with routing, server data, a form and auth — for example a reading list or a small shop:

- Vite + React Router data mode + TanStack Query + one form with `useActionState`, deployed as an SPA;
- then move it to a framework (React Router framework mode or Next.js App Router) and compare SSR, loaders vs Server Components, and the bundle size;
- add TypeScript (`strict`), Vitest + Testing Library tests, and turn on the React Compiler.

Step 3 — Go deeper where your job needs it.

Direction	Read / do next
Framework work	Next.js docs (App Router, Cache Components), React Router framework docs; Chapter 13 again
Performance	React DevTools Profiler on your own app; web.dev on Core Web Vitals (LCP, INP, CLS) and the last chapter of the HTML & CSS book (critical rendering path, Core Web Vitals); responsive images
Design systems / accessibility	WAI-ARIA Authoring Practices patterns; Radix / React Aria source; the HTML & CSS book's accessibility chapter
JavaScript depth	The JavaScript book: closures, async and the event loop, its cheat sheets
Front-end system design	Component architecture and machine coding in the LLD book: UI components case, machine coding
Mobile	Expo (React Native) — the same components-and-Hooks model on iOS and Android

Step 4 — Keep up to date. Primary sources, in this order: **react.dev** (Learn + Reference + the blog, where every release and deprecation is announced), the React GitHub releases, then the docs of the libraries you use (React Router, TanStack Query, Redux Toolkit, Zustand, Testing Library, Next.js). Check the date on any tutorial: anything using class components, Create React App, `ReactDOM.render`, `forwardRef` for new code, or `useFormState` predates the React 19 era this book teaches.

16.21 Glossary (A–Z)

Each term links to the section that explains it (the first link is the main one).

A

- **Accessible name** — What assistive technology announces for an element — a button's text, an input's label, `aria-label` / `aria-labelledby`, an image's `alt`; what `getByRole(role, { name })` matches. §12.3, §14.10
- **`act()`** — Test helper that makes React finish every pending render and Effect before the next line runs; RTL wraps its helpers in it, so you call it only for direct updates (e.g. `renderHook` calls, fake timers). §12.2, §12.7
- **Action** — In React 19, a function run inside a Transition — passed to `<form action>`, `formAction`, or `startTransition(async …)`; React tracks its pending state. §4.4, §7.10
- **Action (React Router)** — A route's function for writes; `<Form method="post">` calls it, then the router re-runs loaders (revalidation). §9.6
- **Action (reducer)** — A plain object describing what happened, e.g. `{ type: 'added', text }`, sent to a reducer with `dispatch`. §6.2
- **Action creator** — A function returning an action object; Redux Toolkit's `createSlice` generates them (`added('p1')` → `{ type: 'cart/added', payload: 'p1' }`). §6.7
- **`<Activity>`** — React 19.2 component that hides a subtree while keeping its state; one of the real remounts StrictMode's double run prepares you for. §5.4
- **`aria-describedby`** — Links an element to text that describes it (e.g. a field's error message), making that text its accessible description. §4.3, §14.10
- **`aria-invalid`** — Attribute set to `true` on a field whose value failed validation, so screen readers announce it. §4.3, §14.10
- **`aria-live` / live region** — An element (`role="status"` or `role="alert"`) whose text changes are announced; it must already exist before the text changes. §14.10
- **Automatic batching** — Since React 18, all state updates from one event — including in timeouts, promises and native listeners — are combined into one re-render. §3.4
- **Automatic JSX runtime** — The React 17+ compile mode that imports `jsx` from `react/jsx-runtime` for you, so files don't need `import React`. §1.8

B

- **Bail-out** — When a setter receives a value `Object.is`-equal to the current state, React skips re-rendering that component and its children. §3.2, §10.1
- **Barrel file** — An `index.js` that re-exports a module's contents; one per feature is fine, giant ones slow tools and hurt tree-shaking. §14.11
- **`baseDuration`** — `<Profiler>` value: estimated time for a full re-render of the subtree — compare with `actualDuration` to see what memoization saves. §10.3

- **BEM** — Block__Element--Modifier CSS naming convention that avoids clashes in global stylesheets. §8.2
- **`browser()`** — React 19.3 `react-dom` function for `use(browser())`: suspends only on the server, so the nearest `<Suspense>` fallback is sent and the client renders the real content — no hydration mismatch. §13.5
- **Bundler** — Tool (Vite, which bundles with Rolldown since Vite 8; esbuild; webpack; Turbopack) that combines modules into files for the browser and splits chunks for `import()`. §10.8, §1.5

C

- **Cache Components** — Next.js 16 opt-in model: everything is dynamic unless cached with `'use cache'`; the build prerenders a static shell (PPR). §13.9
- **Children (`children` prop)** — Whatever is nested between a component's tags; render it with `{children}` to build wrappers. §2.3
- **Class component** — Legacy component style (`class X extends Component` with `render()`); still required for error boundaries. §2.1, §14.9
- **Cleanup function** — The function an Effect's setup returns; it undoes the setup and runs before the next setup and on unmount. §5.3
- **Client Component** — A component whose code ships to the browser (marked by a `'use client'` entry point); may use state, Effects and events. §13.7
- **Client-side routing** — Changing screens without a page load: the router intercepts links, updates the URL with the History API, and renders the matching route. §9.1
- **Closure** — A function that remembers the variables of the render that created it — why handlers see a state snapshot. §3.3
- **`clsx`** — Tiny helper that builds a `className` string from strings, conditions and objects, dropping falsy values. §8.5
- **Code splitting** — Splitting the bundle into chunks loaded on demand, e.g. `lazy(() => import('./Chart'))`. §10.8
- **Colocation** — Keeping state, files and tests next to where they are used, so changes stay local and re-renders small. §14.11, §10.10
- **Commit phase** — The step where React applies DOM changes, sets refs and runs layout effects; runs once per committed update. §10.1, §1.3
- **Component** — A capitalised JavaScript function that takes props and returns JSX (or `null`). §1.7, §2.1
- **Component tree** — The hierarchy of components React renders; visible in React DevTools, not in the DOM. §1.7
- **Composition** — Building UIs by nesting components and passing JSX (`children`, slots) instead of inheritance or many boolean props. §2.4, §14.1
- **Compound components** — A family like `Tabs` / `Tabs.Tab` / `Tabs.Panel` that shares state through a private context. §14.4
- **Concurrent rendering** — React 18+ ability to prepare UI in the background, pause it for urgent work, and discard stale work. §7.10, §1.3
- **Container / presentational** — Splitting "where data comes from" (container, Hook or Server Component) from "how it looks" (props-only view). §14.2
- **Context** — `createContext` + `<Ctx value>` + `useContext` / `use`: passes a value to any depth without prop drilling; a transport, not a store. §6.3
- **Controlled component** — A component whose value comes from props/state (`value` + `onChange`, or `value` / `onValueChange` in design systems). §3.9, §4.2, §14.4
- **Controlled input** — A form field whose displayed value is React state: `value={state}` plus `onChange`. §3.9, §4.2
- **`createPortal`** — `react-dom` API that renders children into another DOM node while keeping them in the same React tree. §14.8
- **`createRoot`** — `react-dom/client` API that creates a React root for a DOM element; `root.render(<App />)` starts rendering. §1.6
- **`createSlice`** — Redux Toolkit function that generates a reducer and action creators; its reducers use Immer. §6.7
- **CSR** — Client-side rendering: the server sends a near-empty page and JavaScript builds the UI in the browser (an SPA). §1.4, §13.1
- **CSS Modules** — `*.module.css` files whose class names are renamed per file, so `styles.card` can't clash. §8.4
- **CSS-in-JS** — Writing styles in JavaScript; runtime libraries (styled-components, Emotion) inject styles while rendering, zero-runtime ones extract CSS at build. §8.7
- **Custom Hook** — A function named `useSomething` that calls other Hooks; shares logic, never state. §7.3, §14.3

D

- **Debounce** — Wait until input has been quiet for N ms before acting (e.g. searching). §7.5
- **`defaultValue`** — Starting value of an uncontrolled input; the DOM owns the value afterwards. §4.2

- **Dependency array** — The second argument of `useEffect`/`useMemo`/`useCallback`: every reactive value the callback reads; compared with `Object.is`. §5.5, §10.5
- **Derived state** — A value computed from props/state during render instead of being stored; never kept in `useState`. §3.8
- **Design system** — Tokens + accessible primitives + usage docs shared across a product. §14.12
- **Design tokens** — Named design values (colours, spacing, radii) usually stored as CSS custom properties. §8.8, §14.12
- **Directive** — A string literal at the top of a file or function (`'use client'`, `'use server'`) that bundlers read. §13.8
- **Discriminated union** — A TypeScript union of object types sharing a literal field (e.g. `status`) that narrows to one member. §11.9, §11.5
- **Dispatch** — Sending an action to a reducer: `dispatch({ type: 'added' })`; stable identity. §6.2
- **Double fetch** — When the client refetches data the server already used for SSR; frameworks serialize data into the page instead. §13.4

E

- **Effect** — Code in `useEffect` that runs after commit to synchronise a component with an external system. §5.2
- **Effect Event** — A function created by `useEffectEvent`: called from Effects, always sees the latest props/state, and is not a dependency. §5.5, §7.7
- **Error boundary** — A class component with `getDerivedStateFromError`/`componentDidCatch` that shows a fallback when a descendant throws while rendering. §14.9
- **Event delegation** — React attaches one listener per event type at the root container and dispatches to your handlers. §3.1
- **Event handler** — A function passed as an `on…` prop; the right place for side effects caused by a user action. §3.1
- `exhaustive-deps` — `eslint-plugin-react-hooks` rule that checks dependency arrays of Effects, `useMemo` and `useCallback`. §5.5, §7.2
- **Exhaustiveness check** — Assigning a value to `never` in a `default` branch so an unhandled union case becomes a compile error. §11.5
- **External store** — Data that lives outside React and changes on its own (browser APIs, Zustand, Redux); read it with `useSyncExternalStore`. §7.9

F

- **Fetcher** — React Router `useFetcher()`: submit or load without navigating (like buttons, inline edits). §9.6
- **Fiber** — React's internal record for each component instance, holding its Hook slots; also the name of the reconciler since React 16. §7.1, §1.3
- `flushSync` — `react-dom` function that commits a state update synchronously, opting out of batching; use sparingly. §3.4, §5.9
- `<Form>` **(React Router)** — Router form that serialises fields, calls the route `action`, then revalidates loaders. §9.6
- **Form Action** — React 19 `<form action={fn}>`: React prevents navigation, calls `fn(formData)` in a Transition, and resets uncontrolled fields after success. §4.4
- `formAction` — Prop on a submit button that overrides the form's action for that button. §4.4
- `FormData` — Browser object holding a form's name/value pairs; values are strings, unchecked boxes are missing, `getAll` for repeated names. §4.1
- `forwardRef` — Legacy API to pass a ref to a function component; React 19 passes `ref` as a normal prop instead. §5.10
- **Fragment** — `<>…</>` or `<Fragment key>`: groups children without adding a DOM node. §1.8
- **Fragment ref** — React 19.3: a `ref` on `<Fragment>` gives an instance with `focus()`, `addEventListener`, `observeUsing` for its children. §5.10
- **Framework mode** — React Router's full-stack mode: route modules, SSR/prerendering, per-route code splitting, type generation. §9.7, §13.10

G

- `gcTime` — TanStack Query: how long unused cached data stays in memory (default 5 minutes). §6.9, §9.8
- **Generic component** — A component with a type parameter, e.g. `List<T>`, whose `T` is inferred from props. §11.8
- `getDerivedStateFromError` — Static error-boundary method that returns state so the next render shows the fallback. §14.9

- `getServerSnapshot` — Third argument of `useSyncExternalStore`: the value used on the server and during hydration. §7.9, §13.5

H

- **Headless UI library** — Components that provide behaviour and accessibility but no styles (Radix, React Aria, Headless UI, Base UI). §8.9, §14.12
- **Higher-order component (HOC)** — A function that takes a component and returns a wrapped one (`withAuth(Page)`); must be created at module level. §14.6
- **HMR** — Hot Module Replacement: the dev server swaps an edited module in the running page, keeping state. §1.5
- **Hook** — A function starting with `use` that lets a component use a React feature; called at the top level only. §7.1
- `hydrateRoot` — `react-dom/client` API that hydrates existing server HTML instead of creating new DOM. §13.2, §13.4
- **Hydration** — React in the browser adopting server-rendered HTML: it renders the same tree, reuses the DOM and attaches handlers. §13.4, §1.4
- **Hydration mismatch** — The client's first render differs from the server HTML; React client-renders that tree and reports a recoverable error. §13.5

I

- **Immer** — Library that lets you "mutate" a draft and produces a new immutable state with structural sharing; used by Redux Toolkit. §3.6, §6.7
- **Immutability** — Never changing existing state objects/arrays; create new ones so `Object.is` sees the change. §3.6
- **Index route** — A child route with `index` that renders at the parent's exact URL. §9.4
- **Invalidation** — Marking cached queries stale so they refetch: `queryClient.invalidateQueries({ queryKey })`. §9.10, §6.9
- `isFetching` — TanStack Query flag: a request is in flight, including background refetches. §9.8
- `isPending` — TanStack Query: no data yet. `useTransition`/`useActionState`: the transition or Action is still running. §9.8, §7.10, §4.5
- **ISR** — Incremental static regeneration: static pages re-built in the background after a revalidation window. §13.1

J

- **jsdom** — A JavaScript implementation of the DOM used to run component tests in Node; no layout or painting. §12.9, §1.5
- **JSX** — HTML-like syntax in JavaScript that compiles to `jsx()`/`createElement` calls returning element objects. §1.8

K

- **Key** — A string/number that identifies an element among its siblings; also resets a component when it changes. §2.8, §3.10

L

- **Layout (route)** — A parent route that renders shared UI and an `<Outlet />` for the matching child. §9.4
- `lazy` — `lazy(() => import('./X'))`: a component whose code loads on first render; needs a default export and a `<Suspense>` above it. §10.8
- **Lifting state up** — Moving shared state to the closest common parent and passing it down with a callback. §3.7
- **Loader** — React Router data-mode function that loads a route's data before it renders; read with `useLoaderData()`. §9.5
- **Local mutation** — Mutating values created during the same render (a local array) — allowed inside a pure component. §2.5

M

- `memo` — Wraps a component so it skips re-rendering when its parent re-renders with shallow-equal props. §10.4
- **Middleware (React Router)** — Functions that run before a route's loaders/actions and can wrap them with `await next()`; on by default in v8. §9.7
- **Middleware (Redux)** — Code between `dispatch` and the reducer (thunks, logging). §6.7
- **Mount / unmount** — A component instance being created and inserted into the tree / removed from it (running every cleanup). §5.2
- **MPA** — Multi-page application: every navigation loads a new HTML page from the server. §1.4
- **MSW** — Mock Service Worker: intercepts HTTP requests at the network level for tests and development. §12.6

- **Mutation (TanStack Query)** — A server write run with `useMutation`; not cached by key; followed by invalidation or a cache update. §9.10

N

- `noValidate` — Form attribute that turns off the browser's validation bubbles while keeping the `validity` API. §4.3

O

- `Object.is` — The comparison React uses for state, dependencies and `memo` props: same value for primitives, same reference for objects. §3.6, §10.4
- **Optimistic update** — Showing the expected result before the server confirms it, rolling back on failure (`useOptimistic`, `onMutate`). §4.7, §9.10
- **Outlet** — `<Outlet />`: where a layout route renders its matched child route. §9.4
- **Overscan** — Extra rows rendered above and below the visible window of a virtualized list to avoid blank gaps. §10.9

P

- **Partial prerendering (PPR)** — Prerender a static shell at build time, save a `postponed` state, and `resume()` only the dynamic holes per request. §13.6
- **Pending UI** — What the user sees while something loads: old content + indicator, skeletons, or disabled buttons. §9.12
- **Portal** — Content rendered into another DOM node with `createPortal`; context and events still follow the React tree. §14.8
- **Position (in the tree)** — Where a component sits among its parent's children; React keys state to type + position + key. §3.10
- **Preflight** — Tailwind's CSS reset, which removes default styles such as heading sizes. §8.6
- **Prerender** — Rendering HTML ahead of time; `react-dom/static` `prerender*` waits for all Suspense data. §13.6, §13.2
- **Profiler** — React DevTools tab (and `<Profiler onRender>` component) that records which components rendered, why, and how long. §10.3
- **Progressive enhancement** — The basic feature works with plain HTML; JavaScript makes it nicer (forms with Server Functions work before hydration). §4.8
- **Prop drilling** — Passing a prop through components that don't use it, only to reach a deep child; fixed by composition or context. §2.4, §6.3
- **Props** — The read-only object of inputs a parent passes to a component. §2.2
- **Pure component** — Given the same props, state and context, returns the same JSX and changes nothing outside itself during render. §2.5

Q

- **Query function** — TanStack Query `queryFn`: async function that returns data or throws (check `res.ok`). §6.9, §9.8
- **Query key** — Array that identifies cached data, e.g. `['product', id]`; every input of the query function belongs in it. §9.9, §6.9
- `queryOptions` — TanStack Query helper that bundles a key and function for reuse in `useQuery`, loaders and prefetching. §9.9

R

- **Race condition (fetch)** — A slower, older response arriving last and overwriting newer data; fixed with an ignore flag or `AbortController` in cleanup. §5.7, §7.6
- **React Compiler** — Build-time Babel plugin (1.0 since October 2025) that inserts memoization automatically. §10.7
- **React DevTools** — Browser extension to inspect the component tree, props, state and profile renders. §10.3, §1.5
- **React element** — A plain immutable object (`{ type, props, key }`) describing a piece of UI; what JSX creates. §1.3, §1.8
- **React Hook Form** — Form library that manages big forms with uncontrolled inputs and refs. §4.9
- **React Router** — The most used React router; v8 has declarative, data and framework modes and ships as one `react-router` package. §9.2
- **Reactive value** — Anything declared in the component body that can change between renders: props, state, and values computed from them. §5.2, §5.5
- **Reconciliation** — Comparing the new element tree with the previous one to find the minimal DOM changes. §1.3
- **Reducer** — A pure function `(state, action) => nextState`. §6.2

- **Redux Toolkit (RTK)** — The official way to write Redux: `createSlice`, `configureStore`, `createAsyncThunk`, RTK Query. §6.7
- **Ref** — A `{ current }` object from `useRef` that persists across renders; changing it doesn't re-render. §5.8
- **Ref callback** — A function passed as `ref`; called with the node on attach; in React 19 may return a cleanup. §5.9
- **Render (phase)** — React calling component functions to compute the next element tree; must be pure; may repeat or be discarded. §10.1, §1.3
- **Render prop** — A prop whose value is a function returning JSX, called by the component to decide what to render. §14.5
- `renderHook` — RTL helper that runs a Hook inside a hidden test component; read `result.current`. §12.7
- `renderToPipeableStream` — `react-dom/server` API for streaming SSR on Node; `onShellReady`, `onAllReady`, `abort()`. §13.2, §13.3
- `renderToString` — Legacy SSR API returning an HTML string; can't wait for Suspense data. §13.2
- **Reset keys** — Values that, when changed, clear an error boundary's error state. §14.9
- **Revalidation** — Re-running loaders (React Router) or refetching/regenerating cached data so the UI shows fresh data. §9.5, §13.9
- **Roving tabindex** — Only the active item in a group has `tabIndex=0`; arrow keys move focus within the group. §14.4
- **RSC payload** — The serialized output of Server Components (rendered elements + client references + props) sent to the browser. §13.7
- **Rules of Hooks** — Call Hooks only at the top level of components/custom Hooks, and only from React functions. §7.2
- **Rules of React** — Components and Hooks are pure, props/state are immutable, Hooks follow their rules — assumptions the React Compiler relies on. §10.7, §2.5

S

- **Search params** — The `?key=value` part of a URL; read/write with `useSearchParams()` for shareable UI state. §9.3
- **Selective hydration** — With streaming, React hydrates boundaries independently and prioritises one the user interacts with. §13.4
- **Selector** — A function that reads part of a store (`state => state.cart.total`); components re-render only when its result changes. §6.7, §6.8
- **Server Component** — A component that runs only on the server, can be `async`, and ships no JavaScript; can't create or read context, but since 19.3 may render a provider imported from a `'use client'` module. §13.7
- **Server Function** — An async function marked `'use server'` that runs on the server but can be called from the client; a public endpoint. §13.8, §4.8
- **Server state** — A client-side cache of data owned by a server; needs fetching, caching, invalidation — a TanStack Query job. §6.1, §6.9
- **Shallow comparison** — Comparing each top-level property with `Object.is`, as `memo` does with props. §10.4
- **Shell (streaming)** — Everything outside pending Suspense boundaries, sent first with fallbacks in place. §13.3
- **Side effect** — Anything code does besides computing its return value: DOM changes, requests, timers, logging. §5.1, §2.5
- **Skeleton** — Grey placeholder shapes in the final layout shown during a first load; avoids layout shift. §9.12
- **Slot** — A named prop that receives JSX (`header={…}`), next to `children`. §2.4, §14.1
- **Snapshot (state)** — The fixed state values of one render; setters request a new render but never change the variable you hold. §3.3
- **SPA** — Single-page application: one HTML page; JavaScript renders screens and handles navigation. §1.4, §9.1
- **SSG** — Static site generation: HTML produced once at build time and served from a CDN. §1.4, §13.1
- **SSR** — Server-side rendering: HTML rendered per request on the server, then hydrated. §1.4, §13.1
- **Stale closure** — A function that keeps reading values from an old render (missing dependency, or a timer set up once). §3.3, §7.7
- **Stale-while-revalidate** — Serve cached data now and refresh it in the background (TanStack Query, ISR, HTTP caches). §6.9, §13.1
- `staleTime` — TanStack Query: how long fetched data counts as fresh (default 0). §6.9, §9.8
- **State** — Data a component remembers between renders that changes over time; setting it schedules a re-render. §3.2
- **State updater** — A function passed to a setter, `setN(n => n + 1)`, computed from the queued value. §3.5
- **Streaming SSR** — Sending HTML in chunks as each Suspense boundary resolves, with inline swap scripts. §13.3
- `StrictMode` — Development-only checks you opt into by wrapping the app (Vite and most templates do): renders twice, runs Effects setup→cleanup→setup on mount, re-runs ref callbacks. §5.4, §2.5, §1.6

- **Structural sharing** — Reusing unchanged parts of an old object in a new version (Immer, TanStack Query). §3.6
- `suppressHydrationWarning` — Silences a hydration mismatch warning for one element's own text/attributes; keeps the server value. §13.5
- **Suspense** — `<Suspense fallback>` shows a placeholder while something inside (lazy code, `use(promise)`, suspense-enabled data) isn't ready. §9.11, §10.8
- **Synthetic event** — React's cross-browser wrapper around a native event, with the same interface; `e.nativeEvent` gives the original. §3.1

T

- **Tailwind CSS** — Utility-first CSS framework; generates only the classes it finds in your source; v4 configures with `@theme`. §8.6
- **TanStack Query** — Server-state library: caching, de-duplication, background refetch, retries, mutations and invalidation. §6.9, §9.8
- **TanStack Start** — Full-stack React framework built on TanStack Router (type-safe routes, server functions, streaming); v1 release candidate, RSC experimental. §13.11
- **Tearing** — Different parts of the UI showing different versions of the same external data in one render; prevented by `useSyncExternalStore`. §7.9
- **Thunk** — A function dispatched to a Redux store for async logic; `createAsyncThunk` dispatches pending/fulfilled/rejected. §6.7
- **Transition** — A non-urgent state update (`startTransition`) React can render in the background and interrupt; keeps old UI during suspense. §7.10, §9.11
- **Tree-shaking** — Removing unused exports from a bundle at build time. §14.11, §10.10

U

- **Uncontrolled input** — A form field whose value lives in the DOM (`defaultValue`), read with `FormData` or a ref. §4.2
- `use` — React 19 API that reads a promise (suspending) or a context; may be called conditionally, not in `try/catch`. §9.11, §6.3
- `'use cache'` — Next.js directive (not React's) that caches a function or component's output under Cache Components. §13.9
- `'use client'` — Directive marking the entry of client code: that module and its imports ship to the browser. §13.8
- `'use no memo'` — Directive that opts one component or file out of the React Compiler. §10.7
- `'use server'` — Directive marking Server Functions (a file's exports, or one async function); not a marker for Server Components. §13.8
- `useActionState` — Hook returning `[state, formAction, isPending]`; the action gets `(prevState, payload)` and its return value becomes state. §4.5
- `useCallback` — Caches a function between renders until a dependency changes; `useMemo(() => fn, deps)`. §10.5
- `useContext` — Reads the nearest provider's value of a context (or the default); a Hook, top level only. §6.3
- `useDeferredValue` — Returns a lagging copy of a value so a slow `memo` child renders in the background; React 19 adds `initialValue`. §7.11
- `useEffect` — Runs setup after commit (normally after paint) and cleanup before the next run/unmount; for external systems. §5.2
- `useEffectEvent` — React 19.2 Hook creating an Effect Event: latest values inside Effects without re-running them. §7.7, §5.5
- `useFormStatus` — `react-dom` Hook for a component inside a `<form>`: `{ pending, data, method, action }` of that form. §4.6
- `useId` — Unique, SSR-stable ID per component instance for `htmlFor`/`aria-*`; never for keys. §7.8
- `useImperativeHandle` — Exposes a custom, limited object through a ref instead of the raw DOM node. §5.10
- `useInsertionEffect` — Runs before layout effects; for CSS-in-JS libraries to insert `<style>` tags. §5.11, §8.7
- `useLayoutEffect` — Runs after DOM updates but before paint; for measuring and positioning without flicker; blocks paint. §5.11
- `useLoaderData` — React Router Hook that returns the current route's loader data. §9.5
- `useMemo` — Caches a computed value until a dependency changes; for expensive calculations (~1 ms+) and stable identities. §10.6, §10.5
- `useMutation` — TanStack Query Hook for writes: `mutate`, `isPending`, `onMutate`/`onError`/`onSettled`. §9.10
- `useNavigate` — React Router Hook returning a function to navigate in code (`navigate('/x')`, `navigate(-1)`). §9.3
- `useNavigation` — React Router data-mode Hook: `state` is `'idle'`, `'loading'` or `'submitting'`. §9.5

- **`useOptimistic`** — Hook showing an optimistic value while an Action runs; falls back to the real value when it ends. §4.7
- **`useParams`** — React Router Hook returning the URL's dynamic segments as strings. §9.3
- **`useQuery`** — TanStack Query Hook: `{ data, status, isPending, isFetching, error }` for a key + function. §9.8, §6.9
- **`user-event`** — `@testing-library/user-event`: simulates full user interactions (focus, keys, pointer) — async, `await` each call. §12.4
- **`useReducer`** — Hook returning `[state, dispatch]` for a pure reducer; for complex or related state updates. §6.2
- **`useRef`** — Hook returning a persistent `{ current }` object; for DOM nodes and values that don't affect rendering. §5.8
- **`useSearchParams`** — React Router Hook that reads and writes `?query` params like `useState`. §9.3
- **`useState`** — Hook returning `[value, setValue]`; the initial value is used on the first render only. §3.2
- **`useSuspenseQuery`** — TanStack Query Hook that suspends instead of returning a pending state; `data` is always defined. §9.11
- **`useSyncExternalStore`** — Subscribes to an external store: `(subscribe, getSnapshot, getServerSnapshot?)`; snapshot must be cached. §7.9
- **`useTransition`** — Returns `[isPending, startTransition]`; marks updates as non-urgent; async callbacks are Actions in React 19. §7.10
- **Utility-first CSS** — Styling by composing single-purpose classes in markup (Tailwind). §8.6

V

- **Virtual DOM** — Informal name for React's in-memory element tree that is diffed against the previous one. §1.3
- **Virtualization (windowing)** — Rendering only the visible rows of a long list plus overscan inside a full-height spacer. §10.9
- **Vite** — Build tool and dev server with native-ES-module dev serving and HMR; common for React SPAs. Vite 8 strips types/JSX with Oxc (earlier versions: esbuild). §1.5, §11.1

W

- **Waterfall** — Requests that start one after another because each waits for the previous render (fetch-in-Effect chains). §9.5, §5.7

Z

- **Zod** — Schema library: describe data once, `parse`/`safeParse` it at runtime and infer TypeScript types. §4.9
- **Zustand** — Small store library: `create((set, get) => …)` returns a Hook read with selectors; no provider. §6.8

Chapter 16 TL;DR

Sheet	The one line to remember
JSX	One root, closed tags, camelCase attributes, `{expressions}`, `style` objects; `0 &&` renders 0; objects can't be children
Components	Capitalised, top-level, pure; props read-only; composition over configuration; stable keys
Hooks	Top level, same order; `use` is the exception; 19 added `use`, `useActionState`, `useFormStatus`, `useOptimistic`; 19.2 added `useEffectEvent`
State	Snapshot per render; updaters for queued changes; batched everywhere; never mutate; position + key decide identity
Lifecycle	Trigger → render (pure) → commit → layout effects → paint → Effects; children's Effects before parents'
Effects	Only for external systems; cleanup mirrors setup; StrictMode double run in dev; derive, key or handle instead where possible
Refs	Mutable box, no re-render, not during render; `ref` is a prop in 19
Forms	Uncontrolled + form Action + `useActionState` by default; controlled when every keystroke matters; validate on the server
State homes	Local → lifted → context (rare changes) → Zustand/RTK (busy); server data → TanStack Query; shareable → URL
Data & routing	Loaders start fetching early; the query cache avoids refetching; transitions keep old UI
Performance	Measure, fix structure, then memoize (or the Compiler), split code, virtualize
Server	SSR/SSG/ISR/streaming/PPR per route; RSC ships no JS; `'use client'` marks the boundary, `'use server'` marks Server Functions
Testing	Roles and names, `user-event`, `findBy`, mock the network, fresh providers per test
Errors	Read the message, find it in §16.18, follow the link

This is the end of the React Handbook. Next: build something (the plan in §16.20) — and when you forget a detail, come back to these sheets. Then continue with the LLD Handbook's front-end machine-coding case, the HTML & CSS Handbook, or go back to the book's front page or Chapter 1 for a second pass.